Frankenstein Transformer: Biblioteca Unificada de Codificador-Decodificador, CLI y Notas de Diseño Basadas en Investigación

Erick F. Merino M.
Este trabajo no está afiliado
erickfmm@gmail.com

Octubre 2026

Resumen

Las arquitecturas transformer dominan el modelado de secuencias en procesamiento del lenguaje natural, visión por computadora y biología computacional. El trabajo reciente ha diversificado la atención más allá del softmax denso—bloques recurrentes y retentivos, patrones dispersos, mecanismos con compuerta, proyecciones latentes, capas con memoria y reglas de peso rápido—mientras que los optimizadores se han ampliado más allá de AdamW con métodos de reducción de varianza, eficientes en memoria, sin programación, de segundo orden y de bajo rango. Comparar estas ideas bajo condiciones consistentes sigue siendo costoso, pues cada variante debe implementarse, integrarse y mantenerse, lo que dificulta la reproducibilidad. Proponemos Frankenstein Transformer, un conjunto de herramientas CLI basado en configuración (`frankenstein-transformer`) que unifica la definición, el entrenamiento, el despliegue y la inferencia de modelos tras un único esquema YAML estricto. El conjunto de herramientas cubre cuarenta y cinco mezcladores de secuencia en nueve familias y veinticuatro familias de optimizadores con control por grupo de parámetros, y soporta clases de modelo encoder, decoder, Vision Transformer y audio para masked language modeling, modelado causal de lenguaje, embeddings de oraciones, tareas de imagen, reconocimiento de voz y clasificación de audio. El aporte de este software es un contrato de experimentación validado: las configuraciones inválidas fallan antes de entrenar, cada arquitectura y optimizador comparte una interfaz operativa, y el mismo documento gestiona entrenamiento, despliegue cuantizado, exportación y un constructor web generado desde el esquema. Describimos la taxonomía, el enrutamiento de optimizadores, las rutas de despliegue y la integración, y remitimos el detalle matemático y de implementación a los apéndices.

Índice

1. Introducción **11**
1.1. Motivación y Planteamiento del Problema 11
1.2. Propuesta y Contribuciones 11
1.3. Guía de Lectura 12

2. Trabajo Relacionado **12**
2.1. Frameworks de Aprendizaje Profundo Base 12
2.2. Frameworks de Entrenamiento Listos para Usar 13
2.3. Herramientas de Bajo Código y Eficiencia 13
2.4. Experimentación Basada en Configuración 14

3. Diseño del Sistema y Arquitectura **14**
3.1. Arquitectura Centrada en la Configuración . . . 14
3.2. Alcance del Esquema y Reglas de Validación . . . 15
3.3. Esquema de Configuración . . . 15
3.4. Tareas de Entrenamiento . . . 15
3.5. Seguridad de Entrenamiento y Semántica de Runtime . . . 16
3.6. Variantes de Normalización . . . 17
3.7. Arquitecturas de Flujo Residual . . . 17

4. Taxonomía de Arquitecturas e Implementación **17**
4.1. Familias de Atención y Mezcladores de Secuencia . . . 17
4.2. Familias Densa y Recurrente . . . 18
4.3. Familia de Atención Latente y de Bajo Rango . . . 19
4.4. Extensiones de Atención Dispersa . . . 19
4.5. Atención con Compuerta y de Peso Rápido . . . 19

5. Familias de Optimizadores y Dinámicas de Entrenamiento **20**

6. Cuantización y Despliegue **21**

7. Tareas Posteriores de SBERT **21**

8. Integración Sin Código: El Plugin `dashai-frankenstein` **21**
8.1. Componentes Registrados . . . 22
8.2. Esquema de Configuración de Passthrough . . . 22
8.3. Estado . . . 22

9. Evaluación Empírica **22**
9.1. Qué mide una ronda . . . 23
9.2. Hallazgos . . . 24
9.3. Amenazas a la validez . . . 25

10. Discusión **25**
10.1. Compromisos de Diseño Basado en Esquemas . . . 25
10.2. Cobertura Arquitectónica y Vacíos . . . 26
10.3. Fragmentación del Panorama de Optimizadores . . . 26
10.4. Consideraciones de Despliegue y Producción . . . 26
10.5. Desafíos de Integración y Extensibilidad . . . 26

11. Conclusión **27**
11.1. Limitaciones y Direcciones Futuras . . . 28

Bibliografía **29**

Anexos

1. Introducción Conceptual—Transformers y Atención para Principiantes **38**
1.1. ¿Qué es un Transformer? . . . 39
1.2. El Mecanismo de Atención: Una Analogía . . . 39
1.3. Ejemplo Práctico Paso a Paso . . . 39
1.3.1. Contexto 1: "El gato come pescado" . . . 39
1.3.2. Contexto 2: "El restaurante come los márgenes de beneficio" . . . 39
1.4. Cómo Funciona la Atención Matemáticamente (Versión Simple) . . . 40

1.4.1. Paso 1: Consultas, Claves y Valores . . . 40
1.4.2. Paso 2: Compatibilidad . . . 40
1.4.3. Paso 3: Enfoque . . . 40
1.4.4. Paso 4: Combinar . . . 40
1.5. Múltiples Cabezas de Atención: Múltiples Perspectivas . . . 40
1.6. Apilar Capas . . . 41
1.7. ¿Por Qué es Importante? . . . 41
1.8. Desafíos Prácticos . . . 41
1.8.1. Complejidad Computacional . . . 41
1.8.2. Requisitos de Memoria . . . 41
1.8.3. Eficiencia del Dispositivo . . . 41
1.9. Soluciones Modernas . . . 42
1.10. Conclusiones Clave . . . 42

2. Estructura del Esquema de Configuración 42
2.1. Estructura de Nivel Superior . . . 42
2.2. Configuración del Modelo . . . 43
2.2.1. Enumeración del Patrón de Capas . . . 48
2.3. Configuración de Entrenamiento . . . 51
2.4. Configuración del Tokenizador . . . 53
2.5. Configuración del Optimizador . . . 54
2.5.1. Sufijos Compartidos por Grupo . . . 54
2.5.2. Sufijos Globales Específicos del Optimizador . . . 55
2.6. Configuración SBERT . . . 55
2.7. Reglas de Validación . . . 56

3. Tablas de Resumen Completas 57
3.1. Resumen de Mezcladores de Secuencia . . . 57
3.2. Resumen de Optimizadores . . . 60
3.3. Variantes de Normalización . . . 62
3.4. Conexiones Residuales . . . 62
3.5. Funciones de Activación . . . 63
3.6. Variantes de Embedding . . . 64

4. Familias de Optimizadores 65
4.1. La Evolución de la Optimización en Redes Neuronales . . . 65
4.2. Línea Base Estándar y Optimizadores Adaptativos . . . 66
4.3. Momentum Avanzado y Reducción de Varianza (2024–2025) . . . 72
4.4. Optimizadores de Lote Grande, Eficientes en Memoria y Sin Tasa de Aprendizaje 81
4.5. Optimizadores de Segundo Orden, Geométricos y de Ortogonalidad . . . 96

5. Transformadores Densos, Recurrentes y Aumentados con Memoria 112
5.1. Atención Densa de Referencia: Estándar y Sigmoide . . . 113
5.1.1. Atención Softmax Estándar . . . 113
5.1.2. Atención Sigmoide . . . 115
5.1.3. Atención por Consultas Agrupadas (GQA) . . . 117
5.2. Arquitecturas Recurrentes y Retentivas . . . 119
5.2.1. Redes Retentivas (RetNet) . . . 119
5.2.2. Mamba: Modelos de Espacio de Estado Selectivos . . . 121
5.3. Transformers de Profundidad Continua: Integración EDO . . . 123
5.3.1. Transformer EDO . . . 123
5.4. Memoria en Tiempo de Prueba: Titans . . . 125

5.5. Despacho de Mezclador Guiado por Patrón . . . 126

6. Familias de Atención Latente y de Rango Bajo **126**
6.1. Atención Latente Multi-Cabeza (MLA) . . . 128
6.1.1. Formulación Matemática . . . 129
6.1.2. Pseudocódigo Algorítmico . . . 129
6.1.3. Características Clave . . . 129
6.2. Atención Latente por Consultas Agrupadas (GQLA) . . . 130
6.2.1. Formulación Matemática . . . 131
6.2.2. Pseudocódigo Algorítmico . . . 131
6.2.3. Características Clave . . . 131
6.3. Atención Multi-Cabeza de Rango Bajo (MLRA) . . . 132
6.3.1. Formulación Matemática . . . 133
6.3.2. Pseudocódigo Algorítmico . . . 133
6.3.3. Características Clave . . . 133
6.4. Atención Tucker . . . 134
6.4.1. Formulación Matemática . . . 135
6.4.2. Pseudocódigo Algorítmico . . . 135
6.4.3. Características Clave . . . 135
6.5. Atención de Cabezas Entrelazadas (IHA) . . . 136
6.5.1. Formulación Matemática . . . 137
6.5.2. Pseudocódigo Algorítmico . . . 137
6.5.3. Características Clave . . . 137
6.6. Atención laTenT por Cabezas Agrupadas (GTA) . . . 138
6.6.1. Formulación Matemática . . . 139
6.6.2. Pseudocódigo Algorítmico . . . 139
6.6.3. Características Clave . . . 139
6.7. Atención Latente Temporal Multi-Cabeza (MTLA) . . . 140
6.7.1. Formulación Matemática . . . 141
6.7.2. Pseudocódigo Algorítmico . . . 141
6.7.3. Características Clave . . . 141
6.8. Comparación y Síntesis Arquitectónica . . . 142
6.9. Atención Convolucional Comprimida (CCA) . . . 142
6.9.1. Formulación Matemática . . . 143
6.9.2. Características Clave . . . 143
6.10. Atención Convolucional Comprimida por Consultas Agrupadas (CCGQA) . . . 144
6.10.1. Características Clave . . . 145
6.11. Atención de Mezcla Gaussiana (GMA) . . . 146
6.11.1. Formulación Matemática . . . 147
6.11.2. Pseudocódigo Algorítmico . . . 147
6.11.3. Características Clave . . . 147

7. Mecanismos de Atención Dispersa Exhaustivos **148**
7.1. Resumen Ejecutivo . . . 148
7.2. Sparse Transformer: Patrones Estrideados y Fijos Factorizados . . . 149
7.2.1. Formulación Matemática . . . 150
7.2.2. Pseudocódigo Algorítmico . . . 150
7.2.3. Características Clave . . . 151
7.3. Longformer: Ventana Deslizante, Dilatación y Tokens Globales . . . 152
7.3.1. Formulación Matemática . . . 153
7.3.2. Pseudocódigo Algorítmico . . . 153
7.3.3. Características Clave . . . 153

7.4. SWA con Sumideros de Atención: Ventana Deslizante Más un Prefijo Sumidero . 154
7.4.1. Formulación Matemática . . . 155
7.4.2. Pseudocódigo Algorítmico . . . 155
7.4.3. Características Clave . . . 155
7.5. BigBird: Grafo Disperso Aleatorio, Local y Global . . . 157
7.5.1. Formulación Matemática . . . 158
7.5.2. Garantía Teórica . . . 158
7.5.3. Pseudocódigo Algorítmico . . . 158
7.5.4. Características Clave . . . 159
7.6. Atención SparseK: Selección Diferencial Top-k . . . 160
7.6.1. Formulación Matemática . . . 161
7.6.2. Pseudocódigo Algorítmico . . . 161
7.6.3. Características Clave . . . 161
7.7. NSA (Native Sparse Attention): Ramas Jerárquicas Alineadas con el Hardware . 162
7.7.1. Formulación Matemática . . . 163
7.7.2. Pseudocódigo Algorítmico . . . 164
7.7.3. Características Clave . . . 164
7.8. FASA: Atención Dispersa Consciente de la Frecuencia . . . 165
7.8.1. Formulación Matemática . . . 166
7.8.2. Pseudocódigo Algorítmico . . . 167
7.8.3. Características Clave . . . 167
7.9. SpargeAttn: Filtrado de Dos Etapas a Nivel de Bloques . . . 168
7.9.1. Formulación Matemática . . . 169
7.9.2. Pseudocódigo Algorítmico . . . 169
7.9.3. Características Clave . . . 170
7.10. MSA (MiniMax Sparse Attention): Atención Dispersa por Bloques con Rama de Índice Ligera . . . 171
7.10.1. Formulación Matemática . . . 172
7.10.2. Pseudocódigo Algorítmico . . . 173
7.10.3. Características Clave . . . 173
7.11. SparDA (Atención Dispersa Desacoplada): Selección de Bloques Anticipada Guiada por Pronóstico . . . 174
7.11.1. Formulación Matemática . . . 175
7.11.2. Pseudocódigo Algorítmico . . . 175
7.11.3. Características Clave . . . 175
7.12. Espacio de Diseño y Criterios de Selección . . . 176

8. Familias de Atención con Compuerta—Análisis Exhaustivo de Literatura 176
8.1. Resumen Ejecutivo: Compuertas para el Control de Memoria . . . 176
8.2. 1. Atención Lineal con Compuerta (GLA) . . . 178
8.2.1. Fundamento Matemático . . . 178
8.2.2. Entrenamiento Eficiente en Hardware . . . 179
8.2.3. Fortalezas y Limitaciones . . . 180
8.3. 2. DeltaNet: Atención Lineal con Corrección de Errores . . . 181
8.3.1. Fundamento Matemático . . . 181
8.3.2. Entrenamiento Paralelo Eficiente . . . 182
8.3.3. Fortalezas y Limitaciones . . . 183
8.4. 3. Gated DeltaNet: Síntesis de Compuerta y Corrección de Errores . . . 184
8.4.1. Fundamento Matemático . . . 184
8.4.2. Rendimiento Empírico y Compensaciones . . . 185
8.5. 4. HGRN2: Compuerta Jerárquica con Expansión de Producto Externo . . . 186

8.5.1. Fundamento Matemático . . . 186
8.5.2. Escalado y Resultados Empíricos . . . 187
8.6. 5. Forgetting Transformer (FoX): Compuerta en el Espacio de Logits Softmax . . 188
8.6.1. Fundamento Matemático . . . 189
8.6.2. Integración con FlashAttention . . . 190
8.6.3. Fortalezas y Limitaciones . . . 190
8.7. 6. Atención con Compuerta (Post-SDPA Sigmoide) . . . 191
8.7.1. Fundamento Matemático . . . 192
8.7.2. Hallazgos Clave . . . 192
8.8. 7. Gated DeltaNet-2: Borrado y Escritura Canalizados Decouplados . . . 193
8.8.1. Fundamento Matemático . . . 193
8.8.2. Rendimiento Empírico y Compromisos . . . 194
8.9. 8. Kimi Delta Attention (KDA): Compuerta de Decaimiento Canalizada para la Regla Delta . . . 196
8.9.1. Fundamento Matemático . . . 196
8.9.2. Rendimiento Empírico y Compromisos . . . 197
8.10. Principio Unificado de Compuerta . . . 198
8.11. Implementación y Orientación Práctica . . . 198
8.11.1. Cuándo Usar Cada Arquitectura . . . 198
8.11.2. Consideraciones de Hardware . . . 199

9. Normalización, Cuantización, Enrutamiento de Profundidad y Algoritmos de Ejecución 199
9.1. Variantes de Normalización: LayerNorm, RMSNorm, pRMSNorm, Tanh Dinámico, Erf Dinámico y FlashNorm . . . 199
9.1.1. LayerNorm (Línea Base) . . . 200
9.1.2. RMSNorm . . . 200
9.1.3. RMSNorm Parcial (pRMSNorm) . . . 200
9.1.4. Tanh Dinámico (DyT) . . . 200
9.1.5. Erf Dinámico (Derf) . . . 200
9.1.6. FlashNorm (RMSNorm sin pesos) . . . 201
9.1.7. Comparación . . . 202
9.2. Normalización Query–Key y Control de Logits de Atención . . . 203
9.3. Ruta de Cuantización Ternaria BitNet . . . 205
9.4. Enrutamiento de Tokens Mixture-of-Depths (MoD) . . . 205
9.4.1. Configuración . . . 205
9.5. Memoria Condicional Engram . . . 206
9.5.1. Configuración . . . 207
9.6. Embeddings Factorizados y Convolución de Embedding . . . 207
9.6.1. Embeddings Factorizados . . . 207
9.6.2. Convolución de Embedding . . . 207
9.7. Controles de Estabilidad del Paso de Entrenamiento . . . 207
9.8. Partición de Test Reservada y Evaluación al Final del Entrenamiento . . . 208
9.9. Enrutador de Modo de Inferencia SBERT . . . 209

10. Funciones de Activación 209
10.1. Familia Clásica / Sigmoid–Tanh (12 activaciones) . . . 209
10.2. Familia Rectificada (12 activaciones) . . . 210
10.3. Familia Exponencial / ELU (12 activaciones) . . . 211
10.4. Swish y Activaciones Paramétricas . . . 212
10.5. Funciones de Activación Racionales (RAF) . . . 212
10.6. Variantes de FFN con Compuerta: SwiGLU, GEGLU, ReGLU . . . 213

10.7. Esquema de Configuración . . . 213
10.8. Guía de Selección . . . 214

11. Hyper-Connexiones Restringidas por Variedad (mHC) 214
11.1. Motivación: la Propiedad de Mapeo de Identidad . . . 214
11.2. Formulación . . . 215
11.3. Propiedades de la Restricción de Variedad . . . 215
11.4. Implementación en Frankenstein Transformer . . . 216
11.5. Resultados Reportados . . . 217
11.6. Hyperloop: Hiperconexiones a Nivel de Bucle en Transformers en Bucle . . . 217

12. Residuales de Atención (AttnRes) 219
12.1. Motivación: De una Suma Fija a Atención sobre la Profundidad . . . 219
12.2. Cuatro Estrategias . . . 219
12.2.1. Residual Estándar (`"standard"`) . . . 219
12.2.2. Sin Residual (`"none"`, experimental) . . . 220
12.2.3. Residuales de Atención Completa (`"full_attn"`, paper § 3.1) . . . 220
12.2.4. Residuales de Atención por Bloques (`"block_attn"`, paper § 3.2) . . . 220
12.3. Implementación en Frankenstein Transformer . . . 220
12.4. Integración con mHC y Mixture-of-Depths . . . 221
12.5. Resultados Reportados . . . 222

13. Vision Transformer: Predicción de Parches, Clasificación y Segmentación 222
13.1. Embedding de Parches y Codificación Posicional . . . 223
13.2. Predicción de Parches Enmascarados (Autosupervisado) . . . 223
13.3. Clasificación de Imágenes . . . 223
13.4. Segmentación de Imágenes . . . 223
13.4.1. Cabeza Por-Pixel (`pixel`) . . . 223
13.4.2. Encoder-only Mask Transformer (`eomt`) . . . 224
13.5. Reutilización de la Infraestructura Existente . . . 224
13.6. Pseudocódigo en PyTorch . . . 224

14. Codificaciones Posicionales de Todo el Modelo 224
14.1. Arquitectura: Un Módulo Compartido, Exclusión Por Mezclador . . . 225
14.2. Estilos de Inyección . . . 225
14.3. Formulaciones Matemáticas . . . 225
14.3.1. RoPE — Codificación Posicional Rotatoria . . . 225
14.3.2. HoPE — Codificación Posicional Rotatoria Hiperbólica . . . 226
14.3.3. NoPE — Sin Codificación Posicional . . . 226
14.3.4. ALiBi — Atención con Sesgos Lineales . . . 226
14.3.5. BAM — Mecanismo de Atención Bayesiana . . . 226
14.3.6. PaPE — Codificación Posicional Parabólica . . . 227
14.3.7. Sinusoidal Absoluta . . . 227
14.3.8. Sinusoidal Rotatoria . . . 227
14.3.9. Absoluta Aprendida . . . 227
14.4. Banderas `use_pe` Por Mezclador . . . 228
14.5. Unificación del Transformer de Visión . . . 228
14.6. Compatibilidad con Legado . . . 228
14.7. Perfil Arquitectónico: Codificaciones Posicionales . . . 228

15. SSOG: Atención Separable Sum of Gaussians **229**
15.1. Formulación Matemática . . . 232
15.2. Pseudocódigo Algorítmico . . . 232
15.3. Características Clave . . . 232

16. Atención de Peso Rápido para Aprendizaje Continuo **233**
16.1. Preliminares: Alineación, Normalización, Renormalización . . . 233
16.2. La Familia de Regresión (Falcon-1/2/3) . . . 234
16.2.1. Falcon-1: Regresión NLMS Escalar . . . 235
16.2.2. Falcon-2: Regresión NLMS por Columnas . . . 237
16.2.3. Falcon-3: Regresión de Minilote con Ventana Deslizante . . . 239
16.3. La Familia de Producto Interno (Falcon-1A/2A/3A) . . . 240
16.3.1. Falcon-1A y Falcon-2A . . . 241
16.3.2. Falcon-3A: Producto Interno con Ventana Deslizante . . . 244
16.4. Interfaz de Configuración . . . 245
16.5. Resultados Publicados . . . 246
16.6. Relación con Trabajos Previos . . . 246
16.6.1. Cuándo Usar Cada Variante . . . 246

17. Caché KV y Decodificación Incremental **247**
17.1. Motivación y Alcance . . . 247
17.2. Arquitectura de la Caché . . . 247
17.2.1. Gestor Central e Identidad de Capa . . . 247
17.2.2. Slots Tipados . . . 248
17.2.3. Registro de Capacidades . . . 249
17.2.4. Propagación en el Forward y Semántica de Respaldo . . . 249
17.3. Contratos de Corrección . . . 249
17.3.1. Almacenamiento Post-RoPE y el Calendario Absoluto . . . 250
17.3.2. Sesgos Aditivos en las Puntuaciones . . . 250
17.3.3. Reescalado Dependiente de la Longitud . . . 250
17.3.4. Enmascaramiento Causal desde Posiciones . . . 250
17.3.5. Codificaciones Posicionales Absolutas . . . 250
17.3.6. Interbloqueos con Mezcla de Profundidades, mHC, Hyperloop y AttnRes . . . 251
17.4. Recetas Incrementales por Familia . . . 251
17.4.1. Familia Softmax Cuadrática . . . 251
17.4.2. Familia Latente . . . 252
17.4.3. Familias Dispersas y de Ventana . . . 252
17.4.4. Familia Recurrente y de Estado Lineal . . . 253
17.4.5. Estado Auxiliar y Respaldos Documentados . . . 253
17.5. Cuantización de la Caché KV . . . 254
17.5.1. int8 Simétrico con Ejes Asimétricos . . . 254
17.5.2. int4 por Grupos con Ventana Protegida . . . 254
17.5.3. Interfaz y Códecs Futuros . . . 255
17.6. Compresión y Expulsión . . . 255
17.7. Interfaz de Configuración y Despliegue . . . 255
17.8. Validación . . . 256
17.9. Relación con Trabajo Previo . . . 257

18. Softmax y Cross-Entropy Numéricamente Corregidos **258**
18.1. Los Dos Defectos de Punto Flotante . . . 258
18.2. Formulaciones Sin Cancelación . . . 259
18.3. Implementación en el Toolkit . . . 260
18.3.1. Paso Backward . . . 260
18.3.2. Superficie de Integración . . . 261
18.4. Costo, Alcance y Validación . . . 262

19. Conformer: Atención Aumentada con Convolución **262**
19.1. Resumen Ejecutivo . . . 262
19.2. Conformer: Atención Aumentada con Convolución . . . 263
19.2.1. Formulación Matemática . . . 264
19.2.2. Pseudocódigo Algorítmico . . . 265
19.2.3. Mapeo de Implementación y Desviaciones . . . 265
19.3. Ablaciones Publicadas . . . 266
19.4. Reutilización de la Infraestructura Existente . . . 267
19.5. Cuándo Usarlo . . . 267

20. Audio: Reconocimiento de Voz y Clasificación de Audio **268**
20.1. Resumen Ejecutivo . . . 268
20.2. La Clase de Modelo `frankenstein_audio` . . . 269
20.2.1. Frontend de Audio . . . 270
20.2.2. Submuestreo Convolucional . . . 270
20.2.3. Modo Encoder Forzado y la Pila HybridLayer . . . 270
20.3. Reconocimiento Automático de Voz . . . 271
20.3.1. CTC . . . 271
20.3.2. RNN-Transducer . . . 271
20.3.3. Decodificación y Tasa de Error . . . 271
20.4. Clasificación de Audio . . . 272
20.5. SpecAugment . . . 272
20.6. Tamaños de Referencia . . . 272
20.7. Configuraciones de Ejemplo . . . 272
20.8. Señales 1-D Genéricas: EEG y Más Allá . . . 273
20.9. Pseudocódigo en PyTorch . . . 274
20.10Desviaciones y Notas de Implementación . . . 274

21. Embeddings de Tokens Dinámicos (EELBERT) **275**
21.1. Resumen Ejecutivo . . . 275
21.2. Algoritmo . . . 275
21.3. Contrato de Configuración . . . 276
21.4. Inyección del Vocabulario y Ciclo de Vida . . . 277
21.5. Contabilidad de Tamaño y Resultados del Paper . . . 277
21.6. Mapa de Implementación . . . 278
21.7. Alcance y Limitaciones . . . 278

22. El arnés de experimentación **278**
22.1. Etapas y registros . . . 278
22.2. El libro de etapas de la ronda 2 . . . 279
22.3. Datasets consolidados . . . 279
22.4. Reconstruir una ronda . . . 280
22.5. Ronda 1: el piloto . . . 280

23. Ronda 2: el atlas de los mixers **281**
23.1. Configuración . . . 281
23.2. El paisaje de parámetros . . . 281
23.3. Resultados de entrenamiento . . . 281
23.4. Dinámica de entrenamiento . . . 282
23.5. El atlas A×B . . . 282
23.6. Ablaciones . . . 284
23.7. Despliegue e inferencia . . . 286
23.8. Fiabilidad del pipeline . . . 286
23.9. Qué predice la calidad . . . 286
23.10 Lectura práctica y próximos pasos . . . 287

1. Introducción

1.1. Motivación y Planteamiento del Problema

Las arquitecturas transformer son el paradigma dominante para el modelado de secuencias en procesamiento del lenguaje natural [113], visión por computadora y biología computacional, con modelos como BERT [27] y GPT estableciendo el estándar para el aprendizaje de representaciones. Desde entonces, el campo se ha diversificado rápidamente: bloques recurrentes y retentivos, atención dispersa, mecanismos con compuerta, proyecciones latentes, capas con memoria, reglas de peso rápido y formulaciones de profundidad continua abordan cada uno una limitación distinta de la atención densa, mientras que el panorama de optimizadores se ha ampliado más allá de AdamW con métodos de reducción de varianza, eficientes en memoria, sin programación, de segundo orden y de bajo rango [20, 109, 43, 11, 97, 119, 87, 126, 102, 133, 25, 46, 114].

La consecuencia práctica es un ecosistema fragmentado. Comparar mezcladores de secuencia u optimizadores bajo condiciones de entrenamiento idénticas exige reimplementar cada variante, integrarla en un pipeline de entrenamiento y mantener el resultado ante cambios del framework. Este esfuerzo dificulta la reproducibilidad, ralentiza el progreso científico y eleva la barrera de entrada para nuevos investigadores.

1.2. Propuesta y Contribuciones

Proponemos **Frankenstein Transformer** (`https://github.com/erickfmm/frankenstein-transformer`), un conjunto de herramientas basado en configuración que trata las decisiones de arquitectura, optimizador y entrenamiento como un único documento validado. El proyecto ofrece:

1. **Diseño basado en esquema**: un contrato YAML estricto que soporta cuarenta y cinco mezcladores de secuencia en nueve familias y veinticuatro familias de optimizadores, validado antes de iniciar el entrenamiento.
2. **Entrenamiento agnóstico a la arquitectura**: un único pipeline para líneas base densas, bloques recurrentes y retentivos, patrones dispersos, mecanismos con compuerta, compresión latente, memoria condicional, atención de peso rápido y atención de campo geométrico, con modos encoder y decoder [136, 19, 120, 132, 91].
3. **Enrutamiento de optimizadores**: grupos de hiperparámetros con prefijo que controlan de forma independiente embeddings, capas de normalización, bloques de atención y el resto de parámetros para cada familia de optimizadores [134].
4. **Flujos de trabajo de extremo a extremo**: entrenamiento (incluidos objetivos de embeddings de oraciones), despliegue y exportación (formatos cuantizado, estándar, HuggingFace Transformers y BitNet GGUF), inferencia y un constructor web generado desde el esquema, inspirado en flujos de trabajo estilo SBERT [98].
5. **Soporte de visión**: una clase de modelo Vision Transformer que cubre predicción de parches enmascarados, clasificación y segmentación bajo la misma infraestructura de mezcladores y optimizadores.
6. **Herramientas de validación**: pruebas unitarias, validación de presets YAML e integración continua en las versiones de Python soportadas.

La superficie de comandos es `frankenstein-transformer`, con cuatro subcomandos: `train`, `deploy`, `infer` y `web-server`. El último lanza un constructor de configuración en Streamlit que renderiza los metadatos del esquema como formularios, valida la entrada en vivo, previsualiza el YAML resultante y genera el comando CLI equivalente, sirviendo tanto de ayuda a la configuración como de documentación del espacio de diseño disponible.

1.3. Guía de Lectura

Este documento se organiza como una referencia técnica:

1. **Requisitos previos**: el Apéndice 1 introduce transformers y atención para lectores nuevos en el área.

2. **Contrato de configuración**: la Sección 3.1 describe el esquema y la Sección 3.2 sus reglas de validación.

3. **Selección de arquitectura**: la Sección 4.1 compara las familias de mezcladores de secuencia; los Apéndices 5, 6, 7 y 8 recopilan la literatura de soporte, con los detalles de normalización, BitNet y memoria condicional en el Apéndice 9 y los residuales con restricción de variedad en el Apéndice 11.

4. **Optimización**: la Sección 5 resume el enrutamiento de optimizadores; el Apéndice 4 ofrece el análisis completo de las familias.

5. **Despliegue e inferencia**: las Secciones 6 y 7 cubren el despliegue cuantizado y los embeddings de oraciones.

2. Trabajo Relacionado

Este trabajo se sitúa en la intersección de frameworks de aprendizaje profundo, herramientas de entrenamiento y experimentación basada en configuración. Mientras que las Secciones 4.1 y 5 examinan en profundidad la literatura arquitectónica y de optimización, esta sección se centra en el ecosistema de software para construir, entrenar y configurar modelos transformer.

2.1. Frameworks de Aprendizaje Profundo Base

El ecosistema moderno de investigación en transformers se apoya en varios frameworks fundamentales que proporcionan diferenciación automática, aceleración GPU y APIs de alto nivel para construcción de modelos.

PyTorch [89] se ha convertido en el framework dominante para la investigación en transformers. Su modelo de programación imperativa y grafos de cómputo dinámicos permiten depuración flexible y prototipado rápido de arquitecturas novedosas. La abstracción `torch.nn.Module`, el entrenamiento automático con precisión mixta mediante `torch.cuda.amp` y el compilador JIT `torch.compile` forman la columna vertebral de la mayoría de las implementaciones contemporáneas de transformers. Frankenstein Transformer está construido completamente sobre PyTorch, aprovechando su sistema de módulos para definiciones de arquitectura componibles.

TensorFlow [1] fue pionero en aprendizaje profundo orientado a producción con su paradigma de grafo de cómputo estático y la API de alto nivel Keras. Aunque su influencia en la investigación de transformers ha disminuido en relación con PyTorch, la infraestructura de serving de TensorFlow (`TF Serving`, `TF Lite`) y la biblioteca `tensorflow-text` siguen siendo relevantes para escenarios de despliegue.

JAX [15] ofrece un modelo de programación funcional construido alrededor de transformaciones componibles (`jit`, `vmap`, `grad`) y compilación XLA. Su diseño permite implementaciones limpias de algoritmos complejos y ha sido adoptado por varias bibliotecas recientes de transformers (por ejemplo, Flax, Haiku). El diseño sin estado de JAX y su vectorización automática son particularmente adecuados para la investigación en mecanismos de atención novedosos y algoritmos de optimización.

scikit-learn [90] proporciona algoritmos clásicos de aprendizaje automático y utilidades (preprocesamiento, métricas, selección de modelos) que sirven como líneas base y herramientas complementarias en flujos de trabajo con transformers. Aunque no está diseñado para aprendizaje

profundo, su API consistente y documentación extensa lo convierten en un punto de referencia estándar para comparar enfoques basados en transformers con métodos tradicionales.

2.2. Frameworks de Entrenamiento Listos para Usar

Varias bibliotecas proporcionan pipelines de entrenamiento completos que abstraen la complejidad del entrenamiento distribuido, la carga de datos y la gestión de hiperparámetros.

Hugging Face Transformers [117] es el estándar de facto para acceder, ajustar y compartir modelos transformer preentrenados. Su API `Trainer`, el hub de modelos con más de 500,000 modelos y su estrecha integración con las bibliotecas `datasets` y `tokenizers` lo han convertido en el punto de entrada principal para NLP basado en transformers. Sin embargo, su arquitectura está optimizada para el ajuste fino de modelos preentrenados existentes en lugar de experimentar con diseños novedosos de mezcladores de secuencia o entrenar desde cero con estrategias de optimización personalizadas.

Oumi (https://github.com/oumi-ai/oumi, 2025) es una plataforma de código abierto de extremo a extremo para entrenar, ajustar, evaluar y desplegar modelos fundacionales. Proporciona una interfaz unificada a través de múltiples proveedores de nube y soporta tanto ajuste fino supervisado como optimización de preferencias (DPO, RLHF). Oumi enfatiza la preparación para producción con monitoreo integrado, checkpointing y herramientas de despliegue.

LLaMA Factory (https://github.com/hiyouga/LLaMA-Factory, 2024) ofrece un framework unificado para el ajuste fino eficiente de más de 100 modelos de lenguaje grandes. Proporciona tanto una interfaz web como una CLI, soportando LoRA, QLoRA, ajuste fino de parámetros completos y diversas técnicas de alineación. Su enfoque está en adaptar LLMs preentrenados existentes a tareas posteriores en lugar de experimentación arquitectónica.

Axolotl (https://github.com/axolotl-ai-cloud/axolotl, 2024) es una herramienta simplificada de ajuste fino que soporta LoRA, QLoRA, DeepSpeed y FSDP. Enfatiza flujos de trabajo basados en configuración mediante archivos YAML y se ha vuelto popular en la comunidad de ajuste fino de LLMs de código abierto. Al igual que LLaMA Factory, se enfoca en el ajuste fino de modelos preentrenados en lugar de entrenar arquitecturas novedosas desde cero.

2.3. Herramientas de Bajo Código y Eficiencia

Una categoría creciente de herramientas prioriza la accesibilidad y la eficiencia computacional, reduciendo la barrera para el entrenamiento de transformers.

Unsloth (https://github.com/unslothai/unsloth, 2024) proporciona ajuste fino de bajo código con aceleración de 2–5× y requisitos de VRAM significativamente reducidos mediante kernels CUDA escritos a mano y operaciones Triton optimizadas. Soporta exportación GGUF para despliegue cuantizado y se integra con el ecosistema Hugging Face. El enfoque de Unsloth está en hacer el ajuste fino más rápido y eficiente en memoria, no en exploración arquitectónica.

dashAI (https://github.com/DashAISoftware/dashAI, v0.9.7.post2, 2026) es una aplicación de escritorio open-source y local-first para aprendizaje automático desarrollada por un consorcio académico chileno (Universidad de Chile/FCFM, CENIA, IMFD y financiamiento ANID). Su interfaz es completamente basada en esquemas: los formularios de configuración se generan directamente desde esquemas Pydantic de los componentes sin código frontend escrito a mano, y su catálogo abarca unos 76 modelos multitask—clasificadores tabulares, modelos de NLP, traducción, LLMs de pesos abiertos y generación de texto a imagen—unificados tras una docena de abstracciones base tipadas. La extensibilidad es central en su diseño: los plugins son paquetes PyPI ordinarios descubiertos mediante el grupo de entry-points `dashai.plugins` e instalables desde la UI, un mecanismo lo bastante central que incluso las capacidades generativas propias de dashAI (p. ej., Stable Diffusion, Flux) se distribuyen como plugins. Mientras que releases anteriores se enfocaban en ML clásico, la plataforma actual abarca modelado predictivo y generativo en múltiples dominios, lo que la convierte en un anfitrión natural para componentes

transformer vía su sistema de plugins; la Sección 8 presenta el adaptador `dashai-frankenstein` construido sobre este mecanismo.

2.4. Experimentación Basada en Configuración

Un hilo común entre las herramientas examinadas anteriormente es su enfoque en el ajuste fino de modelos preentrenados existentes. Hugging Face Transformers, LLaMA Factory, Axolotl y Unsloth asumen un checkpoint preentrenado como punto de partida. Oumi soporta entrenamiento desde cero pero se enfoca en arquitecturas estándar. Ninguna de estas herramientas proporciona un marco sistemático para experimentar con arquitecturas novedosas de mezcladores de secuencia, comparar diversos algoritmos de optimización bajo condiciones controladas o explorar la interacción entre elecciones arquitectónicas y dinámicas de entrenamiento.

Frankenstein Transformer llena este vacío proporcionando un sistema de configuración basado en esquemas diseñado específicamente para la experimentación arquitectónica. Sus diferenciadores clave incluyen:

- **43 mezcladores de secuencia** que abarcan atención densa, bloques recurrentes/retentivos, patrones de atención dispersa, mecanismos con compuerta, enrutamiento adaptativo por profundidad y capas de memoria condicional, todos seleccionables mediante un único campo YAML.
- **23 familias de optimizadores** con un sistema de enrutamiento de hiperparámetros con prefijos que permite control por grupo de parámetros a través de embeddings, capas de normalización, bloques de atención y redes feed-forward.
- **Modos de entrenamiento duales** que soportan tanto modelado de lenguaje enmascarado (MLM) estilo encoder como predicción del siguiente token autorregresiva (AR) estilo decoder, con flujos de trabajo de ajuste fino especializados para cada uno.
- **Validación estricta de esquemas** mediante restricciones `additionalProperties: false` que previenen errores de configuración antes de que comience el entrenamiento, asegurando reproducibilidad entre experimentos.
- **Flujos de trabajo de extremo a extremo** que abarcan despliegue cuantizado (empaquetado de pesos ternarios, activaciones INT8) y entrenamiento de sentence embeddings inspirado en SBERT [98].
- **Interfaz de configuración web** que proporciona renderizado de formularios basado en esquemas con documentación en línea, validación en tiempo real y generación de comandos CLI.

Al desacoplar la definición de arquitectura de la infraestructura de entrenamiento mediante un contrato de configuración validado, Frankenstein Transformer permite la comparación sistemática de familias de mezcladores de secuencia y estrategias de optimización bajo condiciones de entrenamiento idénticas—una capacidad no proporcionada por ninguna herramienta existente en el ecosistema.

3. Diseño del Sistema y Arquitectura

3.1. Arquitectura Centrada en la Configuración

La experimentación en este proyecto es *primero el esquema*. En lugar de exponer banderas con validación laxa, la topología del modelo, la familia del optimizador, los límites de entrenamiento y la telemetría pasan por un único documento de configuración validado, lo que reduce la

ambigüedad al reproducir resultados y permite que muchas arquitecturas compartan una misma interfaz operativa.

El contrato autoritativo es `src/schema.yaml`, un esquema JSON modular cuyos módulos bajo `src/schema/` cubren los bloques de modelo, clase de modelo, tokenizer, optimizador, SBERT, entrenamiento y datasets. Los objetos de primer nivel son `base_model`, `tokenizer`, `model_class`, `model`, `training`, `image`, `vision_dataset` y `text_dataset`.

Clase de modelo. El campo `model_class` selecciona entre tres variantes:

- `frankenstein_text_encoder`: encoder de arquitectura mixta con toda la gama de mezcladores de secuencia, MoE y normalización avanzada, entrenado con masked language modeling.
- `frankenstein_text_decoder`: decoder causal autorregresivo para generación de siguiente token; el runtime fuerza el enmascaramiento de decoder.
- `frankenstein_vit`: Vision Transformer sobre parches de imagen para predicción de parches enmascarados, clasificación y segmentación; el runtime fuerza el enmascaramiento de encoder y la configuración requiere los bloques `image` y `vision_dataset`.

Modo de entrenamiento. `model.dims.mode` controla el enmascaramiento: `encoder` usa atención bidireccional para masked language modeling y `decoder` usa enmascaramiento causal para predicción autorregresiva.

Selección de mezcladores. `model.dims.layer_pattern` es una lista ordenada de identificadores de mezclador tomados de las ocho familias descritas en la Sección 4; el esquema fija los identificadores aceptados y la enumeración completa aparece en la Figura 2 y el Apéndice 3.

Selección de optimizador. `training.optimizer.optimizer_class` selecciona una de veinticuatro familias de optimizadores, resumidas en la Sección 5 y detalladas en el Apéndice 4.

3.2. Alcance del Esquema y Reglas de Validación

Los objetos de primer nivel y anidados fijan `additionalProperties: false`, de modo que las claves desconocidas fallan de inmediato en lugar de ignorarse silenciosamente. Los parámetros del optimizador se restringen además mediante reglas de prefijo por familia. La validación entre componentes rechaza combinaciones incompatibles antes del entrenamiento: el tamaño oculto debe ser divisible por el número de cabezas, el número de cabezas clave–valor debe dividir el número de cabezas, el tamaño del vocabulario debe coincidir con el tokenizer, un modelo base requiere una tarea y (para MLM) un tokenizer, cada tarea requiere su bloque de optimizador correspondiente, las banderas de routers de BitNet requieren BitNet habilitado, y las clases decoder y de visión fuerzan sus respectivos modos de enmascaramiento.

3.3. Esquema de Configuración

El esquema completo—campos, tipos, rangos y reglas de validación—se documenta en el Apéndice 2. Los bloques en bucle se configuran con `num_layers` y `num_loops`, lo que da `num_layers`×`num_loops` capas lógicas con un conteo de parámetros fijo.

3.4. Tareas de Entrenamiento

`training.task` selecciona el objetivo: `mlm` (masked language modeling), `causal_lm` (predicción autorregresiva de siguiente token, solo clase decoder), `sbert` (embeddings de oraciones), `text_classification` (solo clase encoder) y las tareas de visión `patch_prediction`, `classification`

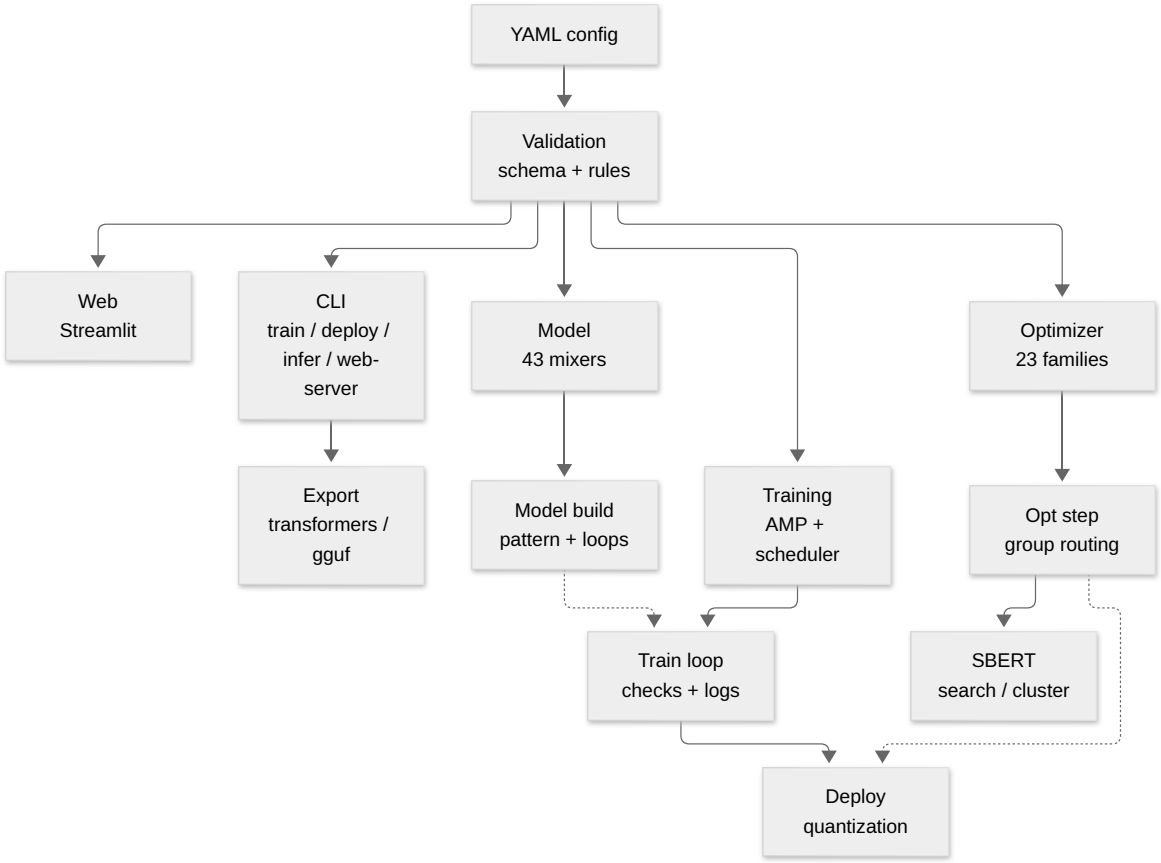


Figura 1: Arquitectura del sistema: la configuración pasa por la validación, se particiona en componentes de modelo, entrenamiento y optimizador, se ejecuta en runtime y produce artefactos de despliegue y SBERT.

y `segmentation` (solo Vision Transformer). Las tareas de NLP consumen `text_dataset`; las de visión consumen `vision_dataset`. Los flujos de embeddings de oraciones se describen en la Sección 7.

3.5. Seguridad de Entrenamiento y Semántica de Runtime

El paso de entrenamiento acumula gradientes sobre micro-lotes, recorta la norma global, verifica desbordamientos posteriores al recorte, reintenta un número acotado de veces ante valores no finitos, despacha el scheduler configurado y rota checkpoints rodantes y del mejor modelo. Cuando `training.test_percentage` es positivo, una fracción determinista del dataset se reserva antes del entrenamiento y los pesos finales se evalúan sobre ella; la pérdida y la precisión de la tarea (tasa de error de caracteres para ASR) se escriben en `test_metrics.json`. Una única `training.seed` (por defecto 42) controla la aleatoriedad de la ejecución: siembra los generadores de Python, NumPy, PyTorch y CUDA, la inicialización de pesos, el generador y los workers de mezcla de los DataLoader y la permutación reservada, y activa cuDNN determinista, de modo que ejecuciones repetidas con el mismo hardware y software reproducen inicialización, orden de lotes y partición. El pseudocódigo completo y la formulación de la partición se remiten al Apéndice 9.

3.6. Variantes de Normalización

La normalización es a la vez una decisión de modelado y una cuestión de compatibilidad del esquema. Los valores aceptados de `norm.type` son `layer_norm`, `dynamic_tanh`, `derf`, `rms_norm`, `prms_norm` (RMSNorm parcial) y `flash_norm` (RMSNorm sin pesos). Sus formulaciones y comparación se detallan en el Apéndice 9.

3.7. Arquitecturas de Flujo Residual

Las conexiones residuales se exponen como una decisión de macro-diseño mediante el bloque `model.residuals`:

- **Estándar**: el residual de identidad clásico [50].
- **Ninguno**: sin conexión de salto, conservado solo como sonda de ablación.
- **Attention Residuals** (`full_attn` o `block_attn`): atención aprendida sobre la profundidad que reemplaza la suma residual fija, ya sea sobre todas las capas anteriores o sobre resúmenes por bloques [110].
- **Hiper-conexiones con restricción de variedad** (`mhc.enabled`): un residual de n flujos cuya matriz de mezcla se restringe al politopo de Birkhoff [120].

Attention Residuals se compone con mHC y Mixture-of-Depths; las formulaciones, el cableado y los resultados reportados se recopilan en los Apéndices 12 y 11.

4. Taxonomía de Arquitecturas e Implementación

4.1. Familias de Atención y Mezcladores de Secuencia

El sistema implementa cuarenta y cinco mezcladores de secuencia organizados en nueve categorías funcionales que reflejan cómo cada método equilibra expresividad, costo computacional y memoria.

1. **Líneas base densas** (3): atención softmax estándar, atención sigmoide y Grouped-Query Attention (GQA) ofrecen enrutamiento global completo a costo cuadrático.
2. **Recurrentes y retentivas** (5): RetNet, Mamba, bloques tipo ODE y Titans mantienen estado para inferencia en tiempo constante mediante recurrencia, espacios de estados selectivos, profundidad continua o memoria en tiempo de prueba.
3. **Patrones dispersos** (10): Sparse Transformer, Longformer, SWA con sumideros de atención, BigBird, SparseK, NSA, SpargeAttn, FASA, MSA y SparDA reducen el costo cuadrático mediante dispersión estructurada, selección de tokens o poda sin entrenamiento.
4. **Mecanismos con compuerta** (8): GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, FoX, Gated Softmax y KDA introducen control dependiente de los datos sobre retención, olvido y fuerza de actualización.
5. **Compresión latente** (10): MLA, GQLA, MLRA, Tucker, IHA, GTA, MTLA, CCA, CCGQA y GMA comprimen el estado clave–valor en cuellos de botella de bajo rango o enrutan mediante una memoria latente de mezcla gaussiana.
6. **Memoria condicional** (1): Engram proporciona memoria de lectura/escritura en tiempo de prueba mediante búsquedas aprendidas.

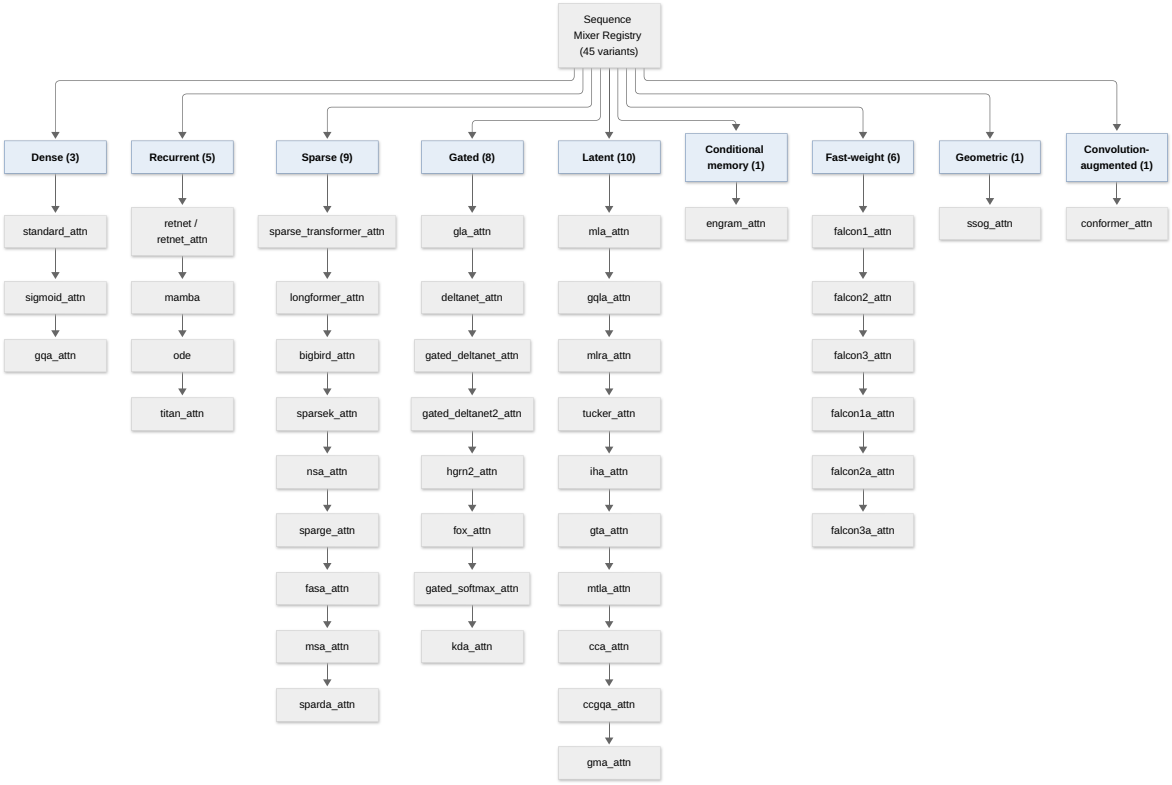


Figura 2: Taxonomía de los cuarenta y cinco mezcladores de secuencia en nueve categorías funcionales. Las líneas base densas ofrecen enrutamiento global completo a costo cuadrático; los métodos recurrentes comprimen el estado para inferencia en tiempo constante; las variantes dispersas restringen el patrón de atención; los mecanismos con compuerta modulan retención y olvido; los métodos latentes comprimen las representaciones clave–valor; la memoria condicional añade búsquedas de lectura/escritura en tiempo de prueba; los métodos de peso rápido formulan la transición de estado como una regla de aprendizaje en línea; los métodos geométricos codifican priors estructurados.

7. **Atención de peso rápido** (6): Falcon-1/2/3 y Falcon-1A/2A/3A tratan la transición de estado recurrente como una regla de aprendizaje continuo en línea [132].

8. **Campo geométrico** (1): SSOG aplica un prior geométrico estructurado sobre la matriz de atención [91].

9. **Aumentados con convolución** (1): el bloque Conformer [45] fusiona atención multi-cabeza con un módulo de convolución depthwise entre dos capas feed-forward Macaron, aportando a cada capa una rama local junto al campo receptivo global (Anexo 19).

4.2. Familias Densa y Recurrente

Las líneas base densas—softmax estándar [113] y atención sigmoide [97]—y la familia recurrente—RetNet [109], Mamba [43], bloques tipo ODE [130] y Titans [11]—definen el compromiso entre expresividad y eficiencia. Las formulaciones, las formas paralela y recurrente, el pseudocódigo y una tabla comparativa se recopilan en el Apéndice 5.

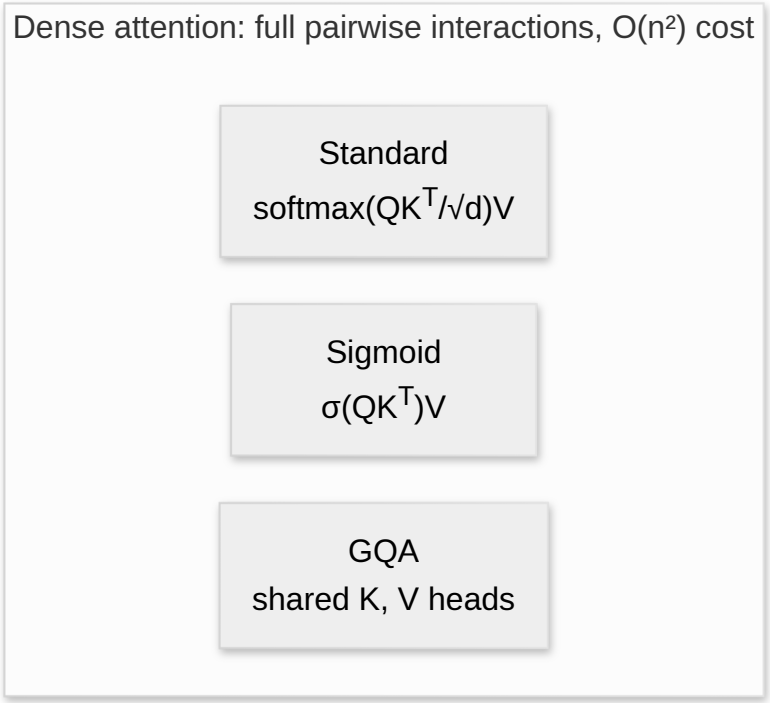


Figura 3: Líneas base de atención densa. La atención estándar y la sigmoide calculan interacciones por pares completas; Grouped-Query Attention comparte cabezas clave–valor entre grupos de consultas, reduciendo el tráfico de memoria sin perder la expresividad de la atención completa.

4.3. Familia de Atención Latente y de Bajo Rango

Esta familia comprime el estado clave–valor por token en un latente de bajo rango: MLA, GQLA, MLRA, Tucker, IHA, GTA y MTLA unifican diseños agrupados, latentes, factorizados y temporales. CCA y CCGQA formulan la atención en un latente comprimido con agrupamiento desacoplado, mientras que GMA reemplaza la interacción por pares con enrutamiento probabilístico mediante componentes de mezcla gaussiana aprendidos. Las formulaciones, el pseudocódigo y una tabla comparativa se recopilan en el Apéndice 6.

4.4. Extensiones de Atención Dispersa

El registro incluye diez bloques dispersos [20, 12, 58, 127, 77, 125, 131, 116, 67, 36]. Dos métodos en tiempo de inferencia, `fasa_attn` y `sparge_attn`, aplican su heurística dispersa solo cuando el modelo está en modo evaluación: durante el entrenamiento la capa híbrida contenedora sustituye atención estándar densa sobre las mismas proyecciones **q**/**k**/**v**/**o**, de modo que los pesos entrenados son exactamente los que lee la ruta dispersa en inferencia. `swa_attn` también es sin entrenamiento pero es un mixer entrenable normal, por lo que ejecuta su patrón de ventana tanto en entrenamiento como en inferencia. Las formulaciones, el pseudocódigo, las características por bloque y una tabla comparativa se recopilan en el Apéndice 7.

4.5. Atención con Compuerta y de Peso Rápido

La familia con compuerta contiene ocho bloques [121, 122, 123, 48, 94, 71, 95, 63]. La compuerta controla qué información sobrevive, ya sea en las actualizaciones de estado recurrente (GLA, variantes de DeltaNet, HGRN2, KDA) o en la ruta de atención completa (FoX, Gated Softmax). Las formulaciones, el pseudocódigo y una tabla comparativa se recopilan en el Apéndice 8.

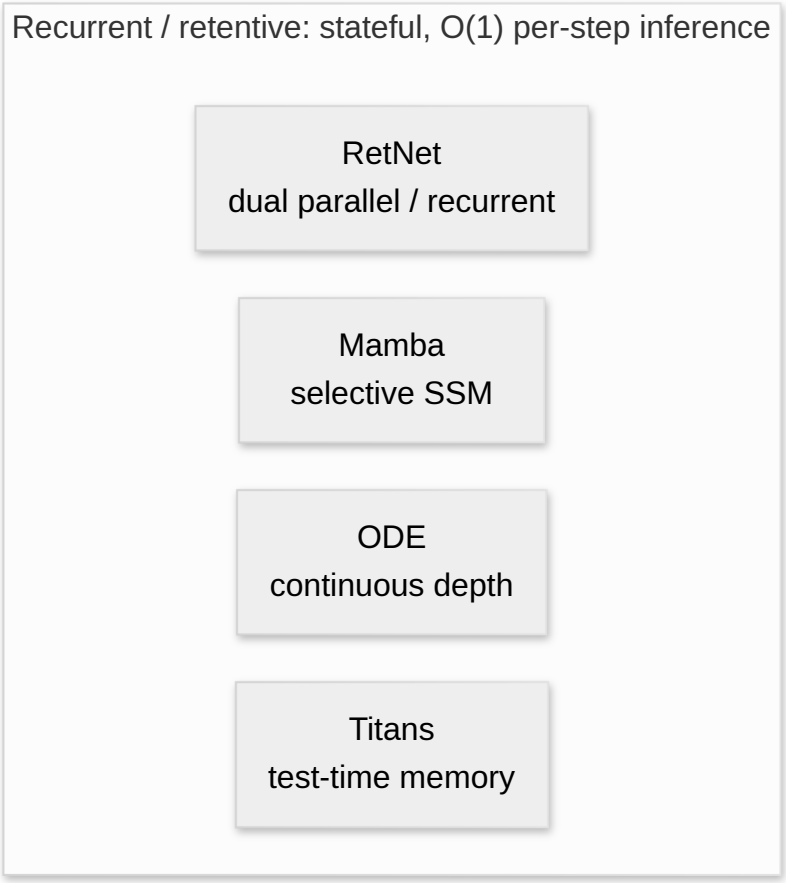


Figura 4: Familia recurrente y retentiva. RetNet ofrece formas paralela y recurrente, Mamba usa parámetros selectivos de espacio de estados, los bloques ODE modelan profundidad continua y Titans añaden memoria en tiempo de prueba; todos logran inferencia en tiempo constante por paso.

La atención de peso rápido reformula la actualización recurrente como una regla de aprendizaje continuo en línea; las seis variantes Falcon [132] unifican la regresión estilo DeltaNet y las escrituras de atención lineal bajo semántica de lectura-tras-escritura con tamaños de paso normalizados y olvido explícito. Cada variante incluye un escaneo recurrente de referencia y un kernel paralelo por fragmentos. Las formulaciones, los parámetros de configuración y los resultados publicados se recopilan en el Apéndice 16.

5. Familias de Optimizadores y Dinámicas de Entrenamiento

Optimizar arquitecturas transformer grandes se complica por paisajes no convexos, puntos de silla y grupos de parámetros heterogéneos. El sistema soporta veintiséis familias de optimizadores en siete categorías: líneas base clásicas (SGD con momentum, AdamW, RAdam); momentum y reducción de varianza (Adan, ADOPT, AdEMAMix, MARS, Cautious AdamW); métodos eficientes en memoria y de bajo rango (Adafactor, GaLore, Lion, APOLLO, APOLLO-Mini, Q-APOLLO); métodos sin programación y sin parámetros (Schedule-Free AdamW, Prodigy) con escalado para lotes grandes (LAMB); métodos conscientes de la curvatura y de segundo orden (Sophia, Shampoo, SOAP); métodos orientados a geometría (Muon, Turbo-Muon, AF-Muon, AdaMuon, Dion3); y optimizadores de adaptividad ajustable (Anon). Los algoritmos, el pseudocódigo y las comparaciones de memoria y complejidad se recopilan en el Apéndice 4.

6. Cuantización y Despliegue

La etapa de despliegue convierte un checkpoint entrenado en un artefacto compacto mediante empaquetado ternario de pesos [115] y escalado de activaciones INT8, con estimaciones de tamaño alineadas con objetivos de despliegue ligero [100, 16]. El mismo subcomando emite artefactos estándar o cuantizados, o exporta formatos HuggingFace Transformers y BitNet GGUF para runtimes externos. Las formulaciones y las estimaciones de tamaño se remiten al Apéndice 9.

7. Tareas Posteriores de SBERT

El embedding de oraciones se construye sobre entrenamiento estilo Siamese [98]. Para el par de oraciones (s_1, s_2) con embeddings (e_1, e_2):

$$\cos(e_1, e_2) = \frac{e_1^\top e_2}{\|e_1\|\|e_2\|}$$

y pérdida coseno estilo regresión:

$$\mathcal{L}_{\cos} = (\cos(e_1, e_2) - y)^2$$

con $y \in [-1, 1]$ en este pipeline.

Modos posteriores soportados:

- **Similitud**: puntuación por pares entre dos oraciones.
- **Búsqueda**: k vecinos más cercanos sobre un corpus.
- **Clustering**: agrupación de embeddings (ej., k-means).
- **Codificación**: exportación persistente de embeddings para recuperación posterior.

El pseudocódigo del enrutador de modo de inferencia SBERT—despachando entre los modos de similitud, búsqueda, cluster y codificación sobre un codificador compartido—se remite al Apéndice 9.

8. Integración Sin Código: El Plugin `dashai-frankenstein`

El CLI y el generador web descritos en la Sección 3 sirven a investigadores cómodos con archivos de configuración YAML. Una audiencia complementaria—especialistas de dominio, educadores y analistas que necesitan un modelo entrenado pero no un script de entrenamiento—está mejor servida por un entorno gráfico. Por ello, Frankenstein Transformer distribuye `dashai-frankenstein`, un adaptador que registra sus clases de modelo como componentes de **dashAI** (https://github.com/DashAISoftware/dashAI), una aplicación de escritorio open-source y local-first para aprendizaje automático desarrollada por un consorcio académico chileno (Universidad de Chile/FCFM, CENIA, IMFD y financiamiento ANID). Ambos sistemas están basados en esquemas—dashAI genera formularios desde esquemas Pydantic y Frankenstein valida YAML contra un JSON Schema estricto—por lo que la integración deja intacto el backend de dashAI; el plugin se descubre mediante el grupo de entry-points estándar `dashai.plugins` documentado en la guía de plugins de dashAI (https://docs.dash-ai.com/learn/guides/plugins/).

8.1. Componentes Registrados

Un único grupo de entry-points registra cinco componentes:

- `FrankensteinMLMModel` (`BaseModel`, `TextClassificationTask`): encoder con una cabeza de clasificación sobre la salida agrupada; `predict` devuelve probabilidades por clase.
- `FrankensteinDecoderModel` (`BaseGenerativeModel`, `TextToTextGenerationTask`): fuerza la clase decoder y expone muestreo autorregresivo top-k.
- `FrankensteinViTClassifier` (`BaseModel`, `ImageClassificationTask`): usa la cabeza de clasificación del ViT (Apéndice 13).
- `FrankensteinViTSegmenter` (`BaseModel`, `SegmentationTask` del plugin): usa la cabeza de segmentación del ViT y devuelve mapas de clases por píxel.
- `SegmentationTask` (`BaseTask`): declara el contrato imagen-entrada/máscara-salida que dashAI no incluye.

Todos comparten una fachada delgada sobre la API del motor, de modo que el plugin tiene una única dependencia pesada.

8.2. Esquema de Configuración de Passthrough

El JSON Schema de Frankenstein sigue siendo la única fuente de verdad en lugar de traducir sus campos a formularios nativos de dashAI. Cada componente expone un campo `frankenstein_json` que contiene una configuración completa de entrenamiento como JSON de una sola línea. Antes de ejecutar cualquier entrenamiento, el adaptador parsea el JSON, lo valida contra el esquema resuelto y aplica las verificaciones cruzadas del cargador de configuración; los fallos se presentan como un error legible etiquetado con la etapa fallida. Los adaptadores de datasets mapean los datasets de dashAI a las expectativas de Frankenstein, inyectan el tamaño de vocabulario desde el tokenizador resuelto y transmiten las métricas por época a dashAI.

8.3. Estado

Las tres fases de implementación—MVP de clasificación, decoder generativo y clasificador ViT, y segmentador ViT—están completas, pero el paquete aún no se ha ejecutado dentro de una instancia viva de dashAI y la publicación en PyPI está pendiente. Las limitaciones conocidas incluyen un bucle de clasificación directo en lugar de preentrenamiento MLM, campos nativos diferidos para ajustes de alto impacto y renderizado no confirmado de máscaras de segmentación. El adaptador deja deliberadamente intacto el núcleo de dashAI, de modo que futuros releases no puedan romperlo por acoplamiento.

9. Evaluación Empírica

Las secciones anteriores describen lo que el conjunto de herramientas puede expresar. Esta sección informa qué ocurrió cuando el contrato se ejercitó a escala: cada par de mezcladores, cada optimizador y cada ruta de despliegue se entrenó, exportó, cuantizó y evaluó a través de la misma interfaz YAML. La campaña se organiza en *rondas*. La ronda 1 fue un ensayo a escala de humo del arnés (5,129 registros de etapas, corridas de entrenamiento de alrededor de medio minuto cada una) para depurar el propio pipeline. La ronda 2 es la campaña de medición que se analiza aquí: 868 configuraciones entrenadas de principio a fin, 2,656,372 pasos de optimización registrados y 905 horas de GPU, publicadas como un informe interactivo con datasets descargables.[1]

[1] https://erickfmm.github.io/frankenstein-transformer/charts/index.html

Dos apéndices acompañan a esta sección. El Apéndice 22 documenta el arnés: el pipeline de cinco etapas, los tres registradores independientes, los datasets consolidados y la cadena de herramientas de reporte, de modo que una nueva ronda consista en apuntar los mismos scripts a una carpeta nueva. El Apéndice 23 reporta la ronda 2 en detalle, con el censo de parámetros, las curvas de entrenamiento, el atlas A×B completo, las barridas de ablación y las mediciones de despliegue.

9.1. Qué mide una ronda

Todas las configuraciones pasan por las mismas cinco etapas, que muestra la Figura 11. Un único archivo YAML selecciona el modelo (un mezclador, un par de mezcladores como `layer_pattern: [A, B]`, un transformer de visión o una ablación del esqueleto), la tarea y el presupuesto de entrenamiento. El pipeline entrena 100 épocas, exporta un checkpoint estándar, lo despliega dos veces (estándar y cuantizado post-entrenamiento), evalúa cuatro consignas enmascaradas reservadas por formato y exporta un paquete de Hugging Face. Tres registradores observan el proceso de forma independiente: un log de entrenamiento paso a paso, un censo de parámetros por grupo de módulos y un libro con una fila por ejecución de etapa. El informe une los tres por el slug de configuración.

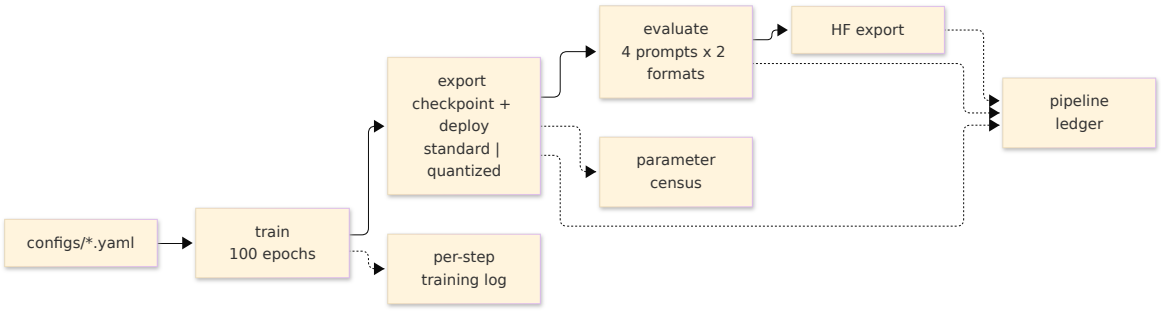


Figura 11: Una configuración de una ronda. Las cinco etapas las gobierna un único archivo YAML y las registran tres registradores independientes; la cadena de reporte offline que convierte los logs en el informe publicado se documenta en el Apéndice 22.

La ronda 2 se generó programáticamente. Los pares híbridos son las combinaciones de la lista de 42 mezcladores tomadas de dos en dos, materializadas como un modelo de dos capas cuya capa 0 ejecuta el mezclador A y la capa 1 el mezclador B; los singles usan una sola capa. Como las combinaciones siguen el orden fijo de la lista, no existe ningún par invertido (B, A), así que la asimetría "primer versus segundo mezclador" que se reporta abajo está confundida con el orden de la lista y debe leerse de forma descriptiva. La Tabla 1 resume la población.

Cuadro 1: Población de la ronda 2. Las medianas se calculan por familia; la métrica de validación es top-1 para los pares MLM y precisión para las familias de visión, audio y datos reales.

Familia	Configs	Parám. med.	Pérd. med.	Métrica med.	tok/s med.	Min med.
Pares híbridos	703	220 928	9,591	4,08 %	356	75,3
Singles	38	157 584	9,762	3,57 %	366	24,8
ViT	48	315 694	1,205	75,97 %	23	9,4
Optimizadores	23	210 752	10,404	7,76 %	391	21,2
Estructurales	20	210 752	10,397	5,71 %	387	21,2
Codif. posicional	12	210 752	10,381	7,76 %	402	21,9
Normalización	6	210 512	10,348	1,61 %	407	20,7
Datos reales	6	247 788	1,060	39,04 %	392	7,6
Tarea	4	210 752	8,603	5,90 %	457	19,4
Tokenizer	4	210 752	5,978	4,37 %	4 202	1,1
Tamaño de lote	2	210 752	10,368	5,40 %	596	13,0
BitNet/GGUF	1	209 856	10,369	—	562	—
Audio	1	147 171	1,099	0,00 %	31 082	1,2

9.2. Hallazgos

El segundo mezclador es la palanca más fuerte. Entre los pares híbridos, el segundo mezclador mueve la pérdida final media desde 9,37 nats (`sparsek`) hasta 10,18 (`mamba`), una brecha de 0,81 nats, mientras que el primero solo abarca 0,55 (9,26–9,81 nats, restringido a las familias con suficientes muestras). Leído columna a columna, un buen B mantiene casi todo A en la banda de 8,9–9,6 nats; leído fila a fila, ningún primer mezclador rescata a un segundo malo. A esta escala, la elección de B domina a la de A por un factor de aproximadamente 1,5×.

Emparejar no es aditivo. Si se define la sinergia de un par como la media de sus dos marginales menos su pérdida medida, $S_{A,B} = (\mu_A + \mu_B)/2 - L_{A,B}$, la mejor interacción es `nsa+engram` con +1,304 nats y la peor `gqa+tucker` con −1,018. La cola positiva empareja un primer mezclador barato y estable con un segundo de memoria pesada; la cola negativa está dominada por segundos mezcladores que parecen promedio en aislamiento (`gta`, `gqa`, `tucker`) pero chocan al apilarse tras un A fuerte. El ruido de interacción (±0,3 nats) es mayor que la diferencia entre la mayoría de los optimizadores de la ablación.

La escala es una palanca débil a este tamaño. Sobre los pares híbridos, el número de parámetros correlaciona con la pérdida final a $r = -0{,}20$ y con la precisión de validación a $r = -0{,}07$, mientras que predice el coste de forma fiable ($r = 0{,}68$ con el tiempo por paso, $r = 0{,}65$ con el tiempo total). El modelo más grande (`engram__conformer_attn`, 415 904 parámetros) no es el mejor. Sobre las 868 filas la correlación parece más fuerte ($r = -0{,}43$ para la pérdida), pero eso es una confusión de familias: los transformers de visión son a la vez más grandes y mucho más fáciles de ajustar, así que la comparación de interés es la de los pares.

La cuantización post-entrenamiento es prácticamente gratis. De las 810 configuraciones desplegadas en ambos formatos, 800 conservan una tasa top-5 idéntica; el top-1 medio pasa de 5,62 % a 5,43 % y el top-5 medio de 13,36 % a 13,24 %, ambos deltas dentro de una muestra de evaluación. El despliegue cuantizado *no* es más rápido de extremo a extremo (32,3 % de las corridas, ratio mediano 0,64×) porque la etapa está dominada por la serialización y la conversión; la evaluación en sí es más rápida en el 51,2 % de las corridas con una mediana de 1,03×.

La tanda es honesta con sus fallos y sus puntos ciegos. El libro contiene 5,271 registros de etapas con 275 fallos (171 de entrenamiento, 104 de despliegue), todos fuera de las 868 configuraciones analizadas. Cuatro mezcladores (`cca`, `ccgqa`, `msa`, `ssog`) nunca entrenan, lo que explica exactamente las 158 celdas ausentes del atlas de 861 celdas. Otros cuatro (`standard`, `bigbird`, `longformer`, `titan`) producen pérdidas bit a bit idénticas porque sus ventanas (al menos 64 tokens) cubren toda la secuencia de 32 tokens; el atlas mide un mismo modelo cuatro veces. Por separado, 37 pares aparecen bajo las grafías `retnet` y `retnet_attn` con métricas

idénticas, una implementación duplicada que el análisis colapsa.

El bloque de visión funciona. La segmentación alcanza 90,7 % de precisión media de validación, la clasificación 75,8 % y la predicción de parches 69,9 %, un orden de magnitud por encima de los pares MLM. Esa brecha refleja la dificultad de la tarea, no la calidad del mezclador, y por eso las familias se mantienen separadas en todo el informe.

9.3. Amenazas a la validez

El modelo es pequeño (127k–416k parámetros, un vocabulario español de 512 tokens, secuencias de 32 tokens, una semilla por configuración y 100 épocas). Todas las curvas de aprendizaje siguen bajando en la época 100, así que el ranking premia a los que aprenden rápido y no a la calidad convergida. La inferencia usa cuatro frases por configuración, de modo que un acierto extra mueve una tasa top-5 en 25 puntos porcentuales; la señal de validación es un detector de fallos catastróficos, no un instrumento de ranking. Varios optimizadores convergen a la misma pérdida porque la corrida es demasiado corta para separarlos, y una corrida de clasificación de texto colapsó a una predicción constante, lo que es un modo de fallo y no un buen modelo. Los rankings del Apéndice 23 son, por tanto, direccionales. La próxima ronda debería repetir el atlas con 256–512 tokens, un conjunto de evaluación de 200 frases, al menos tres semillas y 300 épocas para los mejores pares.

10. Discusión

Las decisiones de diseño en Frankenstein Transformer reflejan varias tensiones de ingeniería e investigación en las herramientas modernas de aprendizaje profundo.

La campaña reportada en la Sección 9 convierte varias de esas tensiones en mediciones en lugar de opiniones. El esquema estricto rindió operativamente: 868 configuraciones se entrenaron, desplegaron y evaluaron a través del mismo contrato, y los fallos que quedaron se concentraron en cuatro mezcladores y en la serialización de checkpoints, no en la deriva de configuración. El sistema de optimizadores prefijados resultó expresivo pero, a 100 épocas, indeciso—una docena de optimizadores convergió a la misma pérdida—lo que es en sí mismo un argumento a favor del esquema: solo una barrida controlada puede mostrar que una dimensión de diseño no importa a una escala dada.

10.1. Compromisos de Diseño Basado en Esquemas

El enfoque basado en esquemas proporciona beneficios significativos de reproducibilidad al imponer contratos explícitos y fallar rápidamente ante configuraciones inválidas. Sin embargo, este enfoque también introduce rigidez: agregar nuevas arquitecturas u optimizadores requiere extensiones del esquema en lugar de argumentos de línea de comandos flexibles. El sistema de hiperparámetros prefijados permite un control detallado pero aumenta la complejidad de configuración para usuarios acostumbrados a interfaces más simples.

La decisión de imponer `additionalProperties: false` en todos los niveles del esquema elimina la absorción silenciosa de parámetros que ha afectado a sistemas de configuración anteriores, pero esta rigurosidad requiere un mantenimiento cuidadoso del esquema al extender las capacidades del sistema. Cada nuevo mecanismo de atención o variante de optimizador debe integrarse adecuadamente en el marco de validación, incluyendo definiciones de campos del esquema con tipos y restricciones apropiados, mapeo de hiperparámetros prefijados para grupos específicos de optimizadores, valores predeterminados alineados con las mejores prácticas de investigación y cadenas de documentación para la representación en la interfaz web.

10.2. Cobertura Arquitectónica y Vacíos

Las cuarenta y cinco arquitecturas de mezclador implementadas abarcan las principales direcciones de investigación en modelado de secuencias, pero ciertos vacíos permanecen. El sistema carece de arquitecturas híbridas recientes como Griffin [105] y Jamba [44], que combinan compuertas con modelos de espacio de estados. El enrutamiento MoE (Mezcla de Expertos) está implementado para capas FFN pero no para el cómputo de atención, donde trabajos recientes han mostrado beneficios [69].

La cobertura de atención dispersa es amplia. Los dos métodos en tiempo de inferencia (FASA, SpargeAttn) conservan su carácter sin entrenamiento en el sentido de que su regla de dispersidad nunca se aprende; en cambio, la implementación entrena la capa como atención estándar densa sobre las mismas proyecciones y activa la ruta de selección/poda dispersa solo en evaluación, de modo que ambos mixers pueden entrenarse desde cero y su comportamiento disperso es exactamente el descrito en los artículos originales.

La cobertura de mecanismos de compuerta y peso rápido abarca las categorías principales, incluidas las seis variantes Falcon [132]. El macro-diseño del flujo residual está cubierto por Attention Residuals [110] y por el flujo residual de n streams restringido por variedad (mHC) [120], cuya proyección Sinkhorn–Knopp sobre el politopo de Birkhoff restaura la propiedad de mapeo de identidad que pierden las hiper-conexiones sin restricción; mHC está validado solo al nivel de los experimentos MoE publicados de 3B–27B y se beneficiaría de reproducciones independientes a escalas menores (Apéndice 11).

10.3. Fragmentación del Panorama de Optimizadores

El soporte para veinticuatro familias de optimizadores en siete categorías demuestra exhaustividad pero también resalta el estado fragmentado de la investigación en optimización. Los usuarios enfrentan una complejidad significativa de decisión al elegir entre métodos de reducción de varianza (Adan, MARS), variantes eficientes en memoria (GaLore, Adafactor, APOLLO) y enfoques conscientes de curvatura (Sophia, Shampoo). El sistema de hiperparámetros prefijados, aunque poderoso, requiere comprender qué parámetros son relevantes para cada clase de optimizador.

La calidad de implementación varía entre optimizadores: los métodos clásicos (AdamW, SGD con momento) están altamente optimizados en PyTorch, mientras que los métodos más nuevos (Muon, Turbo-Muon, AF-Muon, SOAP) pueden requerir implementaciones personalizadas que afectan la estabilidad numérica y las características de rendimiento.

10.4. Consideraciones de Despliegue y Producción

El pipeline de cuantización demuestra preocupaciones prácticas de despliegue pero realiza compromisos de ingeniería específicos. El empaquetado ternario de pesos reduce el almacenamiento a aproximadamente 1,58 bits por parámetro, pero esta compresión agresiva puede degradar el rendimiento, especialmente para modelos más pequeños donde el error de cuantización es más significativo. La implementación actual aplica cuantización uniforme entre todos los tipos de parámetros.

Los flujos de trabajo SBERT proporcionan utilidad práctica para tareas de similitud semántica y recuperación, pero la implementación asume estrategias de pooling estándar (token CLS, pooling promedio). Avances recientes como los embeddings Matryoshka [84] y los refinamientos de aprendizaje contrastivo [61] aún no están incorporados.

10.5. Desafíos de Integración y Extensibilidad

La estructura actual del código base, aunque funcional, presenta desafíos de mantenimiento a medida que se expanden las familias de arquitecturas y optimizadores. El patrón de despachador

para la selección de mezcladores y el enrutamiento de optimizadores maneja la extensibilidad pero corre el riesgo de convertirse en un "cajón de sastre" de lógica condicional. Las versiones futuras se beneficiarían de arquitecturas basadas en plugins donde nuevos mezcladores y optimizadores puedan registrarse de forma declarativa en lugar de modificar la lógica central de despacho.

La interfaz de configuración web proporciona mejoras significativas de usabilidad pero introduce complejidad de despliegue: ejecutar Streamlit junto con trabajos de entrenamiento requiere recursos adicionales y consideraciones de infraestructura que pueden no ser apropiadas para todos los entornos, particularmente clústeres HPC sin acceso web.

11. Conclusión

Frankenstein Transformer presenta una plataforma de experimentación unificada, impulsada por configuración, que aborda tres desafíos de la investigación moderna en aprendizaje profundo: fragmentación arquitectónica entre atención densa, modelos recurrentes, patrones dispersos, mecanismos de compuerta, compresión latente, memoria condicional, memorias de peso rápido y priors geométricos; complejidad del panorama de optimizadores que abarca líneas base clásicas, métodos de reducción de varianza, variantes eficientes en memoria, enfoques sin programación, algoritmos conscientes de curvatura y métodos orientados a geometría; y flujos de trabajo de despliegue de extremo a extremo que abarcan cuantización y embeddings de oraciones.

Las contribuciones principales del sistema son:

1. **Diseño Basado en Esquemas**: Un contrato de configuración estricto basado en YAML con validación y enrutamiento de hiperparámetros prefijados que permite experimentos reproducibles en cuarenta y cinco mezcladores de secuencia y veinticuatro familias de optimizadores.

2. **Soporte Arquitectónico Integral**: Implementación que abarca líneas base densas, bloques recurrentes y retentivos, atención dispersa, mecanismos con compuerta, compresión latente, memoria condicional, atención de peso rápido y atención de campo geométrico, con clases de modelo encoder, decoder y Vision Transformer.

3. **Marco Unificado de Optimizadores**: Grupos de hiperparámetros prefijados que permiten control detallado sobre embeddings, capas de normalización, pesos de atención y el resto de parámetros en líneas base clásicas (SGD+Momentum, AdamW, RAdam), reducción de varianza (Adan, ADOPT, AdEMAMix, MARS, Cautious), métodos eficientes en memoria y de bajo rango (Adafactor, GaLore, Lion, APOLLO, APOLLO-Mini, Q-APOLLO), lotes grandes y simplificación de programación (LAMB, Schedule-Free AdamW, Prodigy), conscientes de curvatura (Sophia), segundo orden (Shampoo, SOAP), orientados a geometría (Muon, Turbo-Muon, AF-Muon) y adaptividad ajustable (Anon).

4. **Flujos de Trabajo de Extremo a Extremo**: Pipeline de despliegue integrado que soporta empaquetado ternario de pesos y cuantización de activaciones INT8; entrenamiento e inferencia inspirados en SBERT para similitud semántica, recuperación y tareas de clustering.

5. **Configuración Interactiva**: Interfaz web basada en Streamlit que proporciona generación de formularios impulsada por esquemas, validación en tiempo real, documentación en línea y síntesis de comandos CLI.

Este sistema permite iteración experimental rápida mientras mantiene reproducibilidad mediante contratos de configuración estrictos. Al consolidar diversas contribuciones de investigación en un conjunto de herramientas unificado, reduce las barreras para explorar arquitecturas y estrategias de optimización novedosas, particularmente para investigadores que pueden carecer de recursos para implementar y validar cada variante de forma independiente.

11.1. Limitaciones y Direcciones Futuras

Varias limitaciones y direcciones prometedoras para trabajo futuro surgen del diseño e implementación de este sistema:

1. **Integración Arquitectónica**: Arquitecturas híbridas recientes (Griffin, Jamba, Mamba-X) demuestran beneficios de combinar múltiples mecanismos en bloques unificados. Versiones futuras deberían integrar estas arquitecturas y explorar patrones de composición sistemática.

2. **Cuantización Avanzada**: La implementación actual utiliza empaquetado ternario uniforme en todos los parámetros. La investigación sobre cuantización por capas, por canales y consciente de importancia sugiere que estrategias más sofisticadas podrían mejorar los compromisos calidad-eficiencia.

3. **Extensibilidad Basada en Plugins**: El patrón de despacho actual se vuelve cada vez más complejo con cada nueva adición. Una arquitectura de plugins que permita el registro declarativo de nuevos mezcladores, optimizadores y métodos de normalización mejoraría el mantenimiento y reduciría el riesgo de errores en la lógica central de despacho.

4. **Optimización Automatizada de Hiperparámetros**: El esquema soporta espacios extensos de hiperparámetros, pero los usuarios deben explorar estos espacios manualmente. La integración con optimización bayesiana, estrategias de bandidos multi-brazo o ajuste de hiperparámetros basado en gradientes podría automatizar el descubrimiento de configuraciones efectivas.

5. **Despliegue en Producción**: La interfaz web mejora la usabilidad pero puede no ser apropiada para todos los entornos de despliegue. Modos de configuración sin interfaz gráfica, gestión de configuración basada en API o ergonomía CLI mejorada podrían servir a flujos de trabajo HPC y de producción.

6. **Evaluación Comparativa**: La ronda 2 (Sección 9) aporta el primer atlas comparativo de la lista de mezcladores, pero es una medición de 32 tokens, una semilla y 100 épocas. Las rondas futuras deberían extenderla a contexto más largo, más frases de validación y múltiples semillas antes de leer los rankings como conclusiones arquitectónicas.

7. **Garantías de Estabilidad de Entrenamiento**: La implementación actual incluye guardas contra NaN/Inf y recorte de gradiente, pero el análisis formal de condiciones de estabilidad para diferentes combinaciones mezclador-optimizador, particularmente con bloques en bucle y cuantización agresiva, sigue abierto.

8. **Extensiones Multimodales y Específicas de Tarea**: El diseño actual cubre texto e imágenes individuales mediante la clase Vision Transformer. Extensiones a modelos de visión-lenguaje, arquitecturas multimodales y flujos de trabajo de ajuste fino específicos de tarea (ej., ajuste por instrucciones, RLHF) ampliarían la aplicabilidad.

La trayectoria de investigación del modelado de secuencias continúa hacia enfoques híbridos que combinan fortalezas de múltiples paradigmas—compresión de la recurrencia, selectividad de la atención, compuertas para gestión de memoria y dispersión para eficiencia. Una plataforma de experimentación unificada como Frankenstein Transformer es cada vez más valiosa a medida que esta convergencia se acelera, permitiendo a los investigadores explorar este creciente espacio de diseño con infraestructura reproducible y bien diseñada.

Bibliografía

[1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 265–283, 2016. URL https://arxiv.org/abs/1605.08695.

[2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyansky, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023. URL https://arxiv.org/abs/2305.13245.

[3] Noah Amsel, David Persson, Christopher Musco, and Robert M. Gower. The polar express: Optimal matrix sign methods and their application to the muon algorithm. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. URL https://openreview.net/forum?id=yRtgZ1K8h0.

[4] Noah Amsel, Jack Zhang, Kwangjun Ahn, Ali Naeimi, Austin Feng, Berlin Chen, Tri Dao, and John Langford. Dion3: Full-stack orthogonal updates, 2026. URL https://arxiv.org/abs/2608.11612.

[5] Ben Anson and Laurence Aitchison. Controlling changes to attention logits, 2025. URL https://arxiv.org/abs/2511.21377. QuacK: parameter-dependent query/key learning rates.

[6] Anonymous authors. Anon: Extrapolating adaptivity beyond sgd and adam, 2026. URL https://arxiv.org/abs/2605.02317.

[7] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. URL https://arxiv.org/abs/1607.06450.

[8] Jonathan T. Barron. Continuously differentiable exponential linear units. *arXiv preprint arXiv:1704.07483*, 2017. URL https://arxiv.org/abs/1704.07483.

[9] Jonathan T. Barron, Dan Gnanapragasam, and Michael Tschannen. A fix for softmax and cross-entropy, 2026. URL https://arxiv.org/abs/2610.31642. arXiv:2610.31642.

[10] Mina Basirat and Peter M. Roth. The quest for a better ReLU: ELiSH and HardELiSH activation functions. *arXiv preprint arXiv:1905.10144*, 2019. URL https://arxiv.org/abs/1905.10144.

[11] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time, 2025. URL https://arxiv.org/abs/2501.00663. Version Number: 1.

[12] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020. URL https://arxiv.org/abs/2004.05150.

[13] Arthur S. Bianchessi, Yasmin C. Aguirre, Rodrigo C. Barros, and Lucas S. Kupssinskü. Bayesian attention mechanism: A probabilistic framework for positional encoding and context length extrapolation, 2025. URL http://arxiv.org/abs/2505.22842.

[14] Thibaut Boissin, Thomas Massena, Franck Mamalet, and Mathieu Serrurier. Turbo-muon: Accelerating orthogonality-based optimization with pre-conditioning, 2025. URL https://arxiv.org/abs/2512.04632. Version Number: 1.

[15] James Bradbury, Roy Frostig, Peter Hawkins, Matthew J. Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL http://github.com/jax-ml/jax.

[16] Riccardo Bravin, Massimo Pavan, Hazem Hesham Yousef Shalby, Fabrizio Pittorino, and Manuel Roveri. EmbBERT: Attention under 2 MB memory, 2026. URL http://arxiv.org/abs/2502.10001.

[17] Mingzhi Chen, Taiming Lu, Jiachen Zhu, Mingjie Sun, and Zhuang Liu. Stronger normalization-free transformers, 2025. URL http://arxiv.org/abs/2512.10938.

[18] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic discovery of optimization algorithms, 2023. URL https://arxiv.org/abs/2302.06675. Version Number: 4.

[19] Xin Cheng, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, Kezhao Huang, Xingkai Yu, Zhewen Hao, Yukun Li, Han Zhang, Huishuai Zhang, Dongyan Zhao, and Wenfeng Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models, 2026. URL https://arxiv.org/abs/2601.07372.

[20] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers, 2019. URL https://arxiv.org/abs/1904.10509.

[21] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units (ELUs). *arXiv preprint arXiv:1511.07289*, 2016. URL https://arxiv.org/abs/1511.07289.

[22] Gabrielle Cohn, Rishika Agarwal, Deepanshu Gupta, and Siddharth Patwardhan. EEL-BERT: Tiny models through dynamic embeddings. URL http://arxiv.org/abs/2310.20144. EMNLP 2023 Industry Track.

[23] Chang Dai, Hongyu Shan, Mingyang Song, and Di Liang. HoPE: Hyperbolic rotary positional encoding for stable long-range dependency modeling in large language models, 2025. URL http://arxiv.org/abs/2509.05218.

[24] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 2978–2988, 2019. doi: 10.18653/v1/P19-1285. URL https://arxiv.org/abs/1906.08237.

[25] Aaron Defazio, Xingyu Alice Yang, Harsh Mehta, Konstantin Mishchenko, Ahmed Khaled, and Ashok Cutkosky. The road less scheduled, 2024. URL https://arxiv.org/abs/2405.15682. Version Number: 4.

[26] Keqi Deng and Philip C. Woodland. Multi-head temporal latent attention, 2025. URL https://arxiv.org/abs/2505.13544.

[27] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding, 2019. URL http://arxiv.org/abs/1810.04805.

[28] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale, 2021. URL http://arxiv.org/abs/2010.11929.

[29] Haojie Duanmu, Zhihang Yuan, Xiuhong Li, Jiangfei Duan, Xingcheng Zhang, and Dahua Lin. SKVQ: Sliding-window KV cache quantization for coding LLMs, 2024. URL https://arxiv.org/abs/2405.06219. arXiv:2405.06219.

[30] Shiv Ram Dubey, Satish Kumar Singh, and Bidyut Baran Chaudhuri. Activation functions in deep learning: A comprehensive survey and benchmark. *Neurocomputing*, 453:1–24, 2021. doi: 10.1016/j.neucom.2021.03.063. URL https://arxiv.org/abs/2109.14545.

[31] Sai Surya Duvvuri, Chanakya Ekbote, Rachit Bansal, Rishabh Tiwari, Devvrit Khatri, David Brandfonbrener, Paul Liang, Inderjit Dhillon, and Manzil Zaheer. Interleaved head attention, 2026. URL https://arxiv.org/abs/2602.21371.

[32] Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 106:300–312, 2018. doi: 10.1016/j.neunet.2018.07.012. URL https://arxiv.org/abs/1702.03118.

[33] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, et al. LayerSkip: Enabling early exit inference and self-speculative decoding, 2024. URL https://arxiv.org/abs/2404.16710. arXiv:2404.16710.

[34] Aiming Fang, Suhyune Lee, Nafise Sadat Moosavi, and Iryna Gurevych. Transformers with learnable activation functions. In *Findings of the Association for Computational Linguistics: EACL 2023*, pages 1736–1746, 2023. doi: 10.48550/arXiv.2208.14111. URL https://arxiv.org/abs/2208.14111.

[35] Tomas Figliola, Nicholas Alonso, Rishi Iyer, Quentin Anthony, and Beren Millidge. Compressed convolutional attention: Efficient attention in a compressed latent space, 2025. URL https://arxiv.org/abs/2510.04476.

[36] Yaosheng Fu, Guangxuan Xiao, Xin Dong, Song Han, and Oreste Villa. Sparda: Sparse decoupled attention for efficient long-context llm inference, 2026. URL https://arxiv.org/abs/2606.04511.

[37] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 315–323, 2011. URL https://proceedings.mlr.press/v15/glorot11a.html.

[38] Luke B. Godfrey and Michael S. Gashler. A continuum among logarithmic, linear, and exponential functions, and its potential to improve generalization in neural networks. In *Proceedings of the International Conference on Knowledge Discovery and Information Retrieval (KDIR)*, 2015. URL https://arxiv.org/abs/1602.01321.

[39] Ian J. Goodfellow, David Warde-Farley, Mehdi Mirza, Aaron Courville, and Yoshua Bengio. Maxout networks. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, pages 1319–1327, 2013. URL https://arxiv.org/abs/1302.4389.

[40] Nils Graef, Filip Makraduli, Andrew Wasielewski, and Matthew Clapp. FlashNorm: Fast normalization for transformers, 2026. URL http://arxiv.org/abs/2407.09577.

[41] Alex Graves. Sequence transduction with recurrent neural networks, 2012. URL https://arxiv.org/abs/1211.3711.

[42] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 369–376, 2006. doi: 10.1145/1143844.1143891. URL https://doi.org/10.1145/1143844.1143891.

[43] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023. URL https://arxiv.org/abs/2312.00752. Version Number: 2.

[44] Albert Gu, AI21 Labs, et al. Jamba: A hybrid transformer-mamba language model, 2024. URL https://arxiv.org/abs/2403.19887.

[45] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang. Conformer: Convolution-augmented transformer for speech recognition. In *Proc. Interspeech*, pages 5036–5040, 2020. doi: 10.21437/Interspeech.2020-3015. URL https://arxiv.org/abs/2005.08100.

[46] Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization, 2018. URL https://arxiv.org/abs/1802.09568. Version Number: 2.

[47] Yizhou Han, Yao Zhao, Jun Zhou, Longfei Li, and Ruoyu Sun. QK-normed MLA: QK normalization without full key caching, 2026. URL https://arxiv.org/abs/2606.16310.

[48] Ali Hatamizadeh, Yejin Choi, and Jan Kautz. Gated deltanet-2: Decoupling erase and write in linear attention, 2026. URL https://arxiv.org/abs/2605.22791.

[49] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015. URL https://arxiv.org/abs/1502.01852.

[50] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778. IEEE, 2016. doi: 10.1109/CVPR.2016.90. URL http://arxiv.org/abs/1512.03385.

[51] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked Autoencoders Are Scalable Vision Learners, 2022. URL http://arxiv.org/abs/2111.06377.

[52] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELU). In *arXiv preprint arXiv:1606.08415*, 2016. URL https://arxiv.org/abs/1606.08415.

[53] Alex Henry, Prudhvi Raj Dachapally, Shubham Pawar, and Yuxuan Chen. Query-key normalization for transformers, 2020. URL https://arxiv.org/abs/2010.04245. Findings of EMNLP 2020.

[54] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization, 2024. URL https://arxiv.org/abs/2401.18079. arXiv:2401.18079.

[55] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. Searching for MobileNetV3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1314–1325, 2019. URL https://arxiv.org/abs/1905.02244.

[56] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. In *arXiv preprint arXiv:1704.04861*, 2017. URL https://arxiv.org/abs/1704.04861.

[57] Yongchao Huang and Hassan Raza. Gaussian mixture attention: Linear-time sequence mixing via probabilistic latent routing, 2026. URL https://arxiv.org/abs/2606.18283.

[58] Alexia Jolicoeur-Martineau, Rhea Sanjay Sukthanker, Pashmina Cameron, and Emy Gervais. Sliding window beats linear attention, 2026. URL https://arxiv.org/abs/2608.28444.

[59] Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. The impact of positional encoding on length generalization in transformers, 2023. URL http://arxiv.org/abs/2305.19466.

[60] Daan Kerssies, Quentin Sartenaer, Fabio Ferraro, Gabriel Y. da Silva, Harkanwarpreet Maan, Matthieu Galon, Arsha Nagrani, Nathan Beeler, Bartlomiej Twardowski, Joost van de Weijer, Robert Dangovski, and Luca Berton. Your ViT is Secretly an Image Segmentation Model, 2025. URL http://arxiv.org/abs/2503.19108.

[61] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning, 2020. URL https://arxiv.org/abs/2004.11362.

[62] Kimi Team. Kimi K2: Open agentic intelligence, 2025. URL https://arxiv.org/abs/2507.20534. MuonClip optimizer and QK-Clip (Section 2.1, Algorithm 1).

[63] Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, Wentao Li, Enzhe Lu, Weizhou Liu, Yanru Chen, Weixin Xu, Longhui Yu, Yejie Wang, Yu Fan, Longguang Zhong, Enming Yuan, Dehao Zhang, Yizhi Zhang, T. Y. Liu, Haiming Wang, Shengjun Fang, Weiran He, Shaowei Liu, Yiwei Li, Jianlin Su, Jiezhong Qiu, Bo Pang, Junjie Yan, Zhejun Jiang, Weixiao Huang, Bohong Yin, Jiacheng You, Chu Wei, Zhengtao Wang, Chao Hong, Yutian Chen, Guanduo Chen, Yucheng Wang, Huabin Zheng, Feng Wang, Yibo Liu, Mengnan Dong, Zheng Zhang, Siyuan Pan, Wenhao Wu, Yuhao Wu, Longyu Guan, Jiawen Tao, Guohong Fu, Xinran Xu, Yuzhi Wang, Guokun Lai, Yuxin Wu, Xinyu Zhou, Zhilin Yang, and Yulun Du. Kimi linear: An expressive, efficient attention architecture, 2025. URL https://arxiv.org/abs/2510.26692.

[64] Günter Klambauer, Thomas Unterthiner, Andreas Mayr, and Sepp Hochreiter. Self-normalizing neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL https://arxiv.org/abs/1706.02515.

[65] Timon Klein, Jonas Kusch, Sebastian Sager, Stefan Schnake, and Steffen Schotthöfer. Tucker attention: A generalization of approximate attention mechanisms, 2026. URL https://arxiv.org/abs/2603.30033.

[66] Arash Lagzian, Paniz Halvachi, Junming Zhang, Zhouhan Lin, and Dianbo Liu. AF-Muon: An AdamW-free muon optimizer for tied-embedding models, 2026. URL https://arxiv.org/abs/2610.01395. Version Number: 1.

[67] Xunhao Lai, Weiqi Xu, Yufeng Yang, Qiaorui Chen, Yang Xu, Lunbin Zeng, Xiaolong Li, Haohai Sun, Haichao Zhu, Vito Zhang, Jinkai Hu, Jiayao Li, Rui Gao, Zekun Li, Songquan Zhu, Jingkai Zhou, and Pengyu Zhao. Minimax sparse attention, 2026. URL https://arxiv.org/abs/2606.13392.

[68] Johannes Lederer. Activation functions in artificial neural networks: A systematic overview. *arXiv preprint arXiv:2101.09957*, 2021. doi: 10.48550/arXiv.2101.09957. URL https://arxiv.org/abs/2101.09957.

[69] Mike Lewis, Shruti Bhosale, Tim Dettmers, Douwe Kiela, and Luke Zettlemoyer. Base layers: Simplifying training of large, sparse models, 2021. URL https://arxiv.org/abs/2103.16716.

[70] Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious optimizers: Improving training with one line of code, 2024. URL https://arxiv.org/abs/2411.16085. Version Number: 4.

[71] Zhixuan Lin, Ke Wang, et al. Forgetting transformer: Softmax attention with a forget gate, 2025. URL https://arxiv.org/abs/2503.02130.

[72] Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A scalable stochastic second-order optimizer for language model pre-training, 2023. URL https://arxiv.org/abs/2305.14342. Version Number: 4.

[73] Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. On the variance of the adaptive learning rate and beyond, 2019. URL https://arxiv.org/abs/1908.03265. Version Number: 4.

[74] Songtao Liu, Hongwu Peng, Zhiwei Zhang, Zhengyu Chen, and Yue Guo. Multi-head low-rank attention, 2026. URL https://arxiv.org/abs/2603.02188.

[75] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache, 2024. URL https://arxiv.org/abs/2402.02750. arXiv:2402.02750.

[76] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2017. URL https://arxiv.org/abs/1711.05101. Version Number: 3.

[77] Tianyu Lou, Zheyu Chen, Tao Yu, et al. Efficient sparse attention for long-range transformers, 2024. URL https://arxiv.org/abs/2406.16747.

[78] Yiping Lu, Zhuohan Li, Di He, Zhiqing Sun, Bin Dong, Tao Qin, Liwei Wang, and Tie-Yan Liu. Understanding and improving transformer from a multi-particle dynamic system point of view, 2019. URL https://arxiv.org/abs/1906.02762.

[79] Andrew L. Maas, Awni Y. Hannun, and Andrew Y. Ng. Rectifier nonlinearities improve neural network acoustic models. In *Proc. ICML Workshop on Deep Learning for Audio, Speech and Language Processing*, 2013. URL https://ai.stanford.edu/~amaas/papers/relu_hybrid_icml2013_final.pdf.

[80] Sushant Mehta, Raj Dandekar, Rajat Dandekar, and Sreedath Panat. Latent multi-head attention for small language models, 2025. URL https://arxiv.org/abs/2506.09342.

[81] Fanxu Meng. Gqla: Group-query latent attention for hardware-adaptive large language model decoding, 2026. URL https://arxiv.org/abs/2605.15250.

[82] Konstantin Mishchenko and Aaron Defazio. Prodigy: An expeditiously adaptive parameter-free learner, 2023. URL https://arxiv.org/abs/2306.06101. Version Number: 4.

[83] Diganta Misra. Mish: A self regularized non-monotonic activation function. *arXiv preprint arXiv:1908.08681*, 2019. URL https://arxiv.org/abs/1908.08681.

[84] Niklas Muennighoff et al. Matryoshka representation learning, 2022. URL https://arxiv.org/abs/2205.13147.

[85] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814, 2010. URL https://www.cs.toronto.edu/~fritz/absps/reluICML.pdf.

[86] Christoffer Koo Øhrstrøm, Rafael I. Cabral Muchacho, Yifei Dong, Filippos Moumtzidellis, Ronja Güldenring, Florian T. Pokorny, and Lazaros Nalpantidis. Parabolic position encoding: Vision-centric, principled, extrapolatable, general, 2026. URL http://arxiv.org/abs/2602.01418.

[87] Matteo Pagliardini, Pierre Ablin, and David Grangier. The AdEMAMix optimizer: Better, faster, older, 2024. URL https://arxiv.org/abs/2409.03137. Version Number: 2.

[88] Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. SpecAugment: A simple data augmentation method for automatic speech recognition. In *Proc. Interspeech*, pages 2613–2617, 2019. doi: 10.21437/Interspeech.2019-2680. URL https://arxiv.org/abs/1904.08779.

[89] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 8024–8035, 2019. URL https://arxiv.org/abs/1912.01703.

[90] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL https://arxiv.org/abs/1201.0490.

[91] Raphael Pisoni. A few gaussians is all you need: Ssog-attention that steers instead of scores, 2026. URL https://www.pisoni.ai/posts/ssog/. Reference implementation: https://github.com/4rtemi5/ssog.

[92] Boris T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964. doi: 10.1016/0041-5553(64)90137-5. URL https://doi.org/10.1016/0041-5553(64)90137-5.

[93] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation, 2022. URL http://arxiv.org/abs/2108.12409.

[94] Zhen Qin, Xu Han, et al. Hgrn2: Gated linear rnns with state expansion, 2024. URL https://arxiv.org/abs/2404.07904.

[95] Yuxiang Qiu, Qwen Team, et al. Gated attention for large language models, 2025. URL https://arxiv.org/abs/2505.06708.

[96] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for activation functions. In *arXiv preprint arXiv:1710.05941*, 2017. URL https://arxiv.org/abs/1710.05941.

[97] Jason Ramapuram, Federico Danieli, Eeshan Dhekane, Floris Weers, Dan Busbridge, Pierre Ablin, Tatiana Likhomanenko, Jagrit Digani, Zijin Gu, Amitis Shidani, and Russ Webb. Theory, analysis, and best practices for sigmoid self-attention, 2024. URL https://arxiv.org/abs/2409.04431. Version Number: 2.

[98] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks, 2019. URL http://arxiv.org/abs/1908.10084.

[99] Susmita Roy, Souvik Manna, Subhash Bagui, et al. LiSHT: Non-parametric linearly scaled hyperbolic tangent activation function for neural networks. *arXiv preprint arXiv:1811.05870*, 2019. URL https://arxiv.org/abs/1811.05870.

[100] Hema Hariharan Samson. Lightweight transformer architectures for edge devices in real-time applications, 2026. URL http://arxiv.org/abs/2601.03290.

[101] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020. URL https://arxiv.org/abs/2002.05202.

[102] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost, 2018. URL https://arxiv.org/abs/1804.04235. Version Number: 1.

[103] Wei Shen, Ruichuan Huang, Minhui Huang, Cong Shen, and Jiawei Zhang. On the convergence analysis of muon, 2025. URL https://arxiv.org/abs/2505.23737. Version Number: 1.

[104] Chongjie Si, Debing Zhang, and Wei Shen. AdaMuon: Adaptive Muon optimizer, 2025. URL https://arxiv.org/abs/2507.11005. Version Number: 3.

[105] Rafael Soares et al. Griffin: Mixing gated linear recurrences with local attention for efficient sequence modeling, 2024. URL https://arxiv.org/abs/2402.19427.

[106] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Training very deep networks. In *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015. URL https://proceedings.neurips.cc/paper_files/paper/2015/hash/a58f4765a99bf77336cf034926677066-Abstract.html.

[107] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding, 2024. URL http://arxiv.org/abs/2104.09864.

[108] Luoyang Sun, Cheng Deng, Jiwen Jiang, Xinjian Wu, Haifeng Zhang, Lei Chen, Lionel Ni, and Jun Wang. Gta: Grouped-head latent attention, 2025. URL https://arxiv.org/abs/2506.17286.

[109] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models, 2023. URL https://arxiv.org/abs/2307.08621. Version Number: 4.

[110] Yutao Sun et al. Attention residuals. URL http://arxiv.org/abs/2603.15031.

[111] Shohei Taniguchi, Keno Harada, Gouki Minegishi, Yuta Oshima, Seong Cheol Jeong, Go Nagahara, Tomoshi Iiyama, Masahiro Suzuki, Yusuke Iwasawa, and Yutaka Matsuo. ADOPT: Modified adam can converge with any β_2 with the optimal rate, 2024. URL https://arxiv.org/abs/2411.02853. Version Number: 3.

[112] Ludovic Trottier, Philippe Giguère, and Brahim Chaib-draa. Parametric exponential linear unit for deep convolutional neural networks. In *Proceedings of the 16th IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 207–214, 2017. doi: 10.1109/ICMLA.2017.0-133. URL https://arxiv.org/abs/1605.09332.

[113] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017. URL https://arxiv.org/abs/1706.03762. Version Number: 7.

[114] Nikhil Vyas, Depen Morwani, Rosie Zhao, Mujin Kwun, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and stabilizing shampoo using adam, 2024. URL https://arxiv.org/abs/2409.11321. Version Number: 2.

[115] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit transformers for large language models, 2023. URL http://arxiv.org/abs/2310.11453.

[116] Zhe Wang, Ming Liu, et al. Fasa: Frequency-aware sparse attention, 2026. URL https://arxiv.org/abs/2602.03152.

[117] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, pages 38–45, 2020. URL https://arxiv.org/abs/1910.03771.

[118] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.17453.

[119] Xingyu Xie, Pan Zhou, Huan Li, Zhouchen Lin, and Shuicheng Yan. Adan: Adaptive nesterov momentum algorithm for faster optimizing deep models, 2022. URL https://arxiv.org/abs/2208.06677. Version Number: 5.

[120] Zhenda Xie, Yixuan Wei, Huanqi Cao, Chenggang Zhao, Chengqi Deng, Jiashi Li, Damai Dai, Huazuo Gao, Jiang Chang, Kuai Yu, Liang Zhao, Shangyan Zhou, Zhean Xu, Zhengyan Zhang, Wangding Zeng, Shengding Hu, Yuqing Wang, Jingyang Yuan, Lean Wang, and Wenfeng Liang. mhc: Manifold-constrained hyper-connections, 2025. URL https://arxiv.org/abs/2512.24880.

[121] Songlin Yang, Bailin Wang, et al. Gated linear attention transformers with hardware-efficient training, 2023. URL https://arxiv.org/abs/2312.06635.

[122] Songlin Yang, Bailin Wang, et al. Parallelizing linear transformers with the delta rule over sequence length, 2024. URL https://arxiv.org/abs/2406.06484.

[123] Songlin Yang, Bailin Wang, et al. Gated delta networks: Improving mamba2 with delta rule, 2024. URL https://arxiv.org/abs/2412.06464.

[124] Yang You, Jing Li, Sashank Reddi, Jonathan Hseu, Sanjiv Kumar, Srinadh Bhojanapalli, Xiaodan Song, James Demmel, and Cho-Jui Hsieh. Large batch optimization for deep learning: Training BERT in 76 minutes, 2020. URL https://arxiv.org/abs/1904.00962. Version Number: 3.

[125] Han Yuan, DeepSeek-AI, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025. URL https://arxiv.org/abs/2502.11089.

[126] Huizhuo Yuan, Yifeng Liu, Shuang Wu, Xun Zhou, and Quanquan Gu. MARS: Unleashing the power of variance reduction for training large models, 2024. URL https://arxiv.org/abs/2411.10438. Version Number: 4.

[127] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Pike Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020. doi: 10.48550/ARXIV.2007.14062. URL https://arxiv.org/abs/2007.14062.

[128] Abbas Zeitoun, Lucas Torroba-Hennigen, and Yoon Kim. Hyperloop transformers, 2026. URL https://arxiv.org/abs/2604.21254.

[129] Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization, 2019. URL http://arxiv.org/abs/1910.07467.

[130] Jing Zhang, Peng Zhang, Baiwen Kong, Junqiu Wei, and Xin Jiang. Continuous self-attention models with neural ODE networks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(16):14393–14401, 2021. ISSN 2374-3468, 2159-5399. doi: 10.1609/aaai.v35i16.17692. URL https://ojs.aaai.org/index.php/AAAI/article/view/17692.

[131] Yichi Zhang, Yizhong Wang, et al. Accurate and training-free sparse attention accelerating any model inference, 2025. URL https://arxiv.org/abs/2502.18137.

[132] Yifan Zhang, Steve Ta, Jasper Zhang, Jichen Feng, Shuzhen Li, Yongxin Zhang, Yifeng Liu, Huizhuo Yuan, Mengdi Wang, Quanquan Gu, and Andrew Chi-Chih Yao. Fast weight attention for continual learning, 2026. URL https://arxiv.org/abs/2608.27763.

[133] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection, 2024. URL https://arxiv.org/abs/2403.03507. Version Number: 2.

[134] Hanqing Zhu, Zhenyu Zhang, Wenyan Cong, Xi Liu, Sem Park, Vikas Chandra, Bo Long, David Z. Pan, Zhangyang Wang, and Jinwon Lee. Apollo: Sgd-like memory, adamw-level performance, 2025. URL https://arxiv.org/abs/2412.05270.

[135] Jiachen Zhu, Xinlei Chen, Kaiming He, Yann LeCun, and Zhuang Liu. Transformers without normalization, 2025. URL http://arxiv.org/abs/2503.10622.

[136] Lianghui Zhu, Yuxin Fang, Bencheng Liao, Shijie Wang, Tianheng Cheng, Zilong Huang, Chen Chen, Lai Wei, Yutao Zeng, Ya Wang, Yi Lin, Yu Li, and Xinggang Wang. Mixture-of-depths attention, 2026. URL https://arxiv.org/abs/2603.15619.

1. Introducción Conceptual—Transformers y Atención para Principiantes

Este apéndice proporciona una introducción accesible a los conceptos fundamentales de los transformers y el mecanismo de atención, dirigido a lectores sin conocimientos profundos previos en aprendizaje profundo. Los transformers son el motor detrás de los sistemas modernos de inteligencia artificial como ChatGPT, pero su funcionamiento puede entenderse mediante analogías del mundo real.

1.1. ¿Qué es un Transformer?

Un transformer es una arquitectura neuronal que procesa texto o secuencias de datos interpretando el significado de cada palabra mientras considera todas las demás palabras en contexto. Imagina que lees una oración:

"El banco estaba lleno de gente esperando. María fue al banco."

¿Qué significa "banco" en cada caso? En la primera oración, se refiere a una institución financiera (o un asiento). En la segunda, no está claro sin contexto. Un transformer resuelve esto automáticamente al mirar todas las palabras circundantes. En la segunda oración, al ver "María fue", el modelo comprende mejor que probablemente se trata de una ribera (donde va a nadar) o una institución financiera (donde va a realizar una transacción).

Esta capacidad de entender el significado completo de una palabra considerando toda la oración se llama *contextualización*, y es el corazón de cómo funcionan los transformers modernos.

1.2. El Mecanismo de Atención: Una Analogía

El mecanismo de atención es el "truco mágico" que permite a los transformers entender el contexto. Piensa en ello como participar en una conversación importante en una habitación ruidosa:

1. **Mucho ruido**: Alguien está hablando y es muy importante para ti.
2. **Ignoras el resto**: Tu cerebro automáticamente enfoca la atención en esa persona, ignorando otros sonidos.
3. **Entiendes mejor**: Al concentrarte, captas cada palabra claramente.

El mecanismo de atención neuronal funciona de la misma manera: cada palabra "pregunta" qué tan importante es cada otra palabra para entender su significado, luego "enfoca" su atención en las más relevantes.

1.3. Ejemplo Práctico Paso a Paso

Veamos cómo un transformer entiende la palabra "come" en dos contextos diferentes:

1.3.1. Contexto 1: "El gato come pescado"

El transformer, al procesar "come", pregunta internamente:

- ¿Qué tan relevante es "El"? Poco (es solo un artículo). **Atención baja**.
- ¿Qué tan relevante es "gato"? Muy relevante (es el sujeto, quien come). **Atención alta**.
- ¿Qué tan relevante es "pescado"? Muy relevante (es lo que se come). **Atención alta**.

Por lo tanto, la representación mental de "come" se enfoca principalmente en "gato" y "pescado".

1.3.2. Contexto 2: "El restaurante come los márgenes de beneficio"

Aquí el transformer pregunta en un contexto diferente:

- ¿Qué tan relevante es "restaurante"? Muy relevante (es el sujeto). **Atención alta**.
- ¿Qué tan relevante es "beneficio"? Muy relevante (se ve afectado). **Atención alta**.
- ¿Qué tan relevante es "márgenes"? Muy relevante (es lo que se consume). **Atención alta**.

La palabra "come" obtiene una representación completamente diferente porque "atiende" a palabras distintas en contextos distintos.

1.4. Cómo Funciona la Atención Matemáticamente (Versión Simple)

Detrás del cambio de atención hay matemáticas. Aquí está la versión simplificada sin demasiado detalle técnico:

1.4.1. Paso 1: Consultas, Claves y Valores

Cada palabra en la oración se transforma en tres versiones:

- **Consulta**: "¿Qué información necesito saber?"
- **Clave**: "¿Qué información tengo?"
- **Valor**: "Aquí está mi información importante."

Imagina en una biblioteca, cada visitante es una "consulta", cada libro es una "clave" y el contenido es el "valor". El bibliotecario (atención) empareja las consultas con las claves más relevantes para acceder al valor correcto.

1.4.2. Paso 2: Compatibilidad

El sistema examina qué tan *compatible* es cada consulta con cada clave. Las consultas similares a las claves obtienen un puntaje de compatibilidad *alto*. Esto se calcula multiplicando la consulta por la clave (matemáticamente, el producto punto).

1.4.3. Paso 3: Enfoque

Los puntajes de compatibilidad se convierten en "pesos de enfoque". Las compatibilidades altas significan "enfocar atención aquí", y las bajas significan "ignorar esto". Matemáticamente, una función llamada `softmax` convierte los puntajes en porcentajes (como: 40 % atención aquí, 35 % aquí, 25 % allá).

1.4.4. Paso 4: Combinar

Finalmente, el sistema combina todos los valores, ponderados por los pesos de enfoque. Si una palabra tiene un 80 % de atención en el sujeto, el 80 % de la información del sujeto se mezcla en la representación de esa palabra.

1.5. Múltiples Cabezas de Atención: Múltiples Perspectivas

Una característica clave es que los transformers no usan una única atención sino *múltiples cabezas de atención* en paralelo. Es como si tuvieras 8 personas analizando la oración *simultáneamente*, cada una prestando atención a diferentes aspectos:

- **Persona 1**: "Me enfoco en las relaciones sujeto-verbo."
- **Persona 2**: "Busco el objeto directo."
- **Persona 3**: "Sigo la información sobre el tiempo verbal."
- **Persona 4**: "Busco modificadores y adjetivos."

Cada "cabeza" aprende a enfocarse en diferentes patrones del lenguaje. Juntas, capturan una comprensión mucho más rica que una sola cabeza.

1.6. Apilar Capas

Los transformers procesan información en *múltiples capas* (generalmente de 12 a 48 para modelos prácticos). Imagina editar un ensayo:

1. **Primera pasada**: Corriges ortografía y gramática básica.
2. **Segunda pasada**: Mejoras la claridad y la estructura de las oraciones.
3. **Tercera pasada**: Aseguras consistencia y flujo narrativo.

Los transformers funcionan de la misma manera: cada capa refina la comprensión del texto. La primera capa captura características básicas (palabras simples, géneros), mientras que las capas posteriores entienden conceptos complejos (relaciones entre palabras, significados abstractos).

1.7. ¿Por Qué es Importante?

El mecanismo de atención fue revolucionario porque:

1. **Paralelismo**: Encuentra contexto para *cualquier palabra con cualquier otra palabra*, sin procesarlas secuencialmente (a diferencia de sistemas anteriores). Esto lo hace muy rápido.
2. **Flexibilidad**: Aprende qué patrones buscar automáticamente a partir de los datos, sin necesidad de programar reglas manualmente.
3. **Escalabilidad**: Funciona desde textos pequeños hasta contextos con millones de palabras.
4. **Capacidades generales**: El mismo mecanismo funciona para traducción, resumen, preguntas-respuestas, generación de texto, visión por computadora y más.

1.8. Desafíos Prácticos

A pesar del extraordinario éxito de los transformers, presentan desafíos:

1.8.1. Complejidad Computacional

Si una oración tiene 1,000 palabras, el mecanismo de atención estándar debe comparar cada palabra con las otras 1,000 palabras. Eso es $1,000 \times 1,000 = 1,000,000$ de comparaciones. Para documentos largos con millones de palabras, esto se vuelve prohibitivamente costoso en tiempo y memoria.

1.8.2. Requisitos de Memoria

Especialmente durante la inferencia (cuando se usan modelos entrenados), almacenar la matriz de atención completa puede consumir enormes cantidades de RAM o memoria de GPU, limitando las longitudes de secuencia que se pueden procesar.

1.8.3. Eficiencia del Dispositivo

Los transformers fueron diseñados para TPUs y GPUs potentes. Ejecutarlos en teléfonos o dispositivos integrados es un desafío.

1.9. Soluciones Modernas

Para resolver estos desafíos, la investigación ha propuesto muchas variantes:

- **Atención Dispersa**: En lugar de comparar cada palabra con TODAS las demás, compara solo con un subconjunto estratégico (vecinos cercanos, patrones periódicos). Reduce la complejidad de $\mathcal{O}(n^2)$ a $\mathcal{O}(n \log n)$ o $\mathcal{O}(n)$.
- **Modelos Recurrentes**: Como Mamba, incorporan aspectos de las antiguas redes neuronales recurrentes pero con mejor eficiencia moderna.
- **Atención con Compuerta**: Mecanismos que aprenden selectivamente qué información pasar hacia adelante, reduciendo las necesidades de almacenamiento.
- **Cuantización**: Usar números más pequeños (enteros en lugar de decimales) para reducir la memoria sin perder demasiada precisión.

Estas innovaciones son lo que este kit de herramientas (Frankenstein) te permite experimentar fácilmente.

1.10. Conclusiones Clave

- Un **transformer** es una red neuronal que entiende el contexto del lenguaje muy bien.
- El **mecanismo de atención** permite que cada palabra "se enfoque" en qué otras palabras son relevantes para entender su significado.
- La atención funciona calculando la **compatibilidad** entre cada palabra (consulta) y todas las demás (claves), luego combinando información basada en esa compatibilidad (valores).
- **Múltiples cabezas** de atención exploran diferentes patrones en paralelo.
- **Múltiples capas** refinan progresivamente la comprensión.
- El principal desafío es que la atención estándar tiene complejidad cuadrática, lo que es costoso para textos largos.
- Muchas **soluciones modernas** (atención dispersa, modelos recurrentes, cuantización) abordan estos desafíos, y este kit de herramientas te permite explorarlas todas.

2. Estructura del Esquema de Configuración

El conjunto de herramientas Frankenstein Transformer impone un esquema de configuración YAML estricto con `additionalProperties: false` en todos los niveles. El esquema se define en `src/schema.yaml`, que referencia sub-esquemas modulares en `src/schema/`. Cada campo configurable debe declararse en el esquema; las claves no reconocidas se rechazan en tiempo de validación. Este anexo documenta la estructura completa del esquema, incluyendo todos los campos, sus tipos, restricciones y reglas de validación entre componentes.

2.1. Estructura de Nivel Superior

El objeto raíz del esquema tiene cinco propiedades de nivel superior. Todas son opcionales excepto `training`, que es obligatorio:

- `base_model` (string) — Identificador de modelo HuggingFace o ruta local para preentrenamiento continuo o ajuste fino SBERT. Cuando se establece, `model_class` y `model` no son obligatorios (la arquitectura se infiere del checkpoint).

- `tokenizer` (object) — Configuración del tokenizador, disponible tanto para la ruta `base_model` como para modelos personalizados desde cero. Dos modos se seleccionan mediante `source`: `hf_repo` (por defecto) carga un tokenizador preentrenado desde `name_or_path`; `train_from_dataset` entrena un tokenizador nuevo desde el texto del dataset (ver §2.4). Obligatorio cuando se usa `base_model` con la tarea `mlm`.
- `model_class` (enum) — Variante de arquitectura del modelo: `frankenstein_text_encoder` (codificador de arquitectura mixta) o `frankenstein_text_decoder` (decodificador causal autorregresivo). Obligatorio cuando `base_model` no está establecido.
- `model` (object) — Parámetros de arquitectura del modelo (ver §2.2). Obligatorio cuando `base_model` no está establecido.
- `training` (object, **obligatorio**) — Configuración de entrenamiento (ver §2.3).

El esquema impone una regla de exclusión mutua: o bien `base_model` debe estar presente, o bien tanto `model_class` como `model` deben estar presentes.

2.2. Configuración del Modelo

El objeto `model` define todos los hiperparámetros arquitectónicos. Los campos obligatorios son `dims.vocab_size`, `dims.hidden_size`, `dims.num_layers`, `dims.num_heads` y `dims.layer_pattern`. La Tabla 2 enumera todos los campos del modelo con sus tipos y descripciones.

Campo	Tipo	Descripción
*model.dims.**		
`dims.mode`	enum (encoder, decoder)	Modo de enmascaramiento de atención. `encoder` = bidireccional; `decoder` = causal.
`dims.vocab_size`	int ≥ 1 (obligatorio)	Número de tokens únicos en el vocabulario. Debe coincidir con el tamaño de vocabulario del tokenizador.
`dims.hidden_size`	int ≥ 1 (obligatorio)	Dimensión oculta. Debe ser divisible por `num_heads`. Típico: 512–2048.
`dims.num_layers`	int ≥ 1 (obligatorio)	Número de capas físicas del transformer. Típico: 6–24.
`dims.num_loops`	int ≥ 1	Número de bucles lógicos. Cada capa en `layer_pattern` se ejecuta `num_loops` veces. Por defecto: 1.
`dims.num_heads`	int ≥ 1 (obligatorio)	Número de cabezas de atención paralelas. Debe dividir `hidden_size` exactamente.
`dims.dropout`	float [0,1]	Tasa de dropout global. Típico: 0.1 (estándar BERT).
`dims.layer_pattern`	array of enum (obligatorio)	Secuencia ordenada de tipos de capa. La longitud debe igualar `num_layers`. 43 códigos de mezclador disponibles (ver Tabla 2.2.1).
*model.norm.**		
`norm.type`	enum	Estrategia de normalización: `layer_norm`, `dynamic_tanh`, `derf`, `rms_norm`, `prms_norm`, `flash_norm`.

continúa en la página siguiente

Campo	Tipo	Descripción
`norm.partial_ratio`	float (0,1]	Fracción de dimensiones ocultas para estimación pRMSNorm. Por defecto: 0.0625.
*model.embedding.factorized.**		
`embedding.factorized.enabled`	bool	Activar factorización de matriz de embedding para reducir parámetros.
`embedding.factorized.dim`	int ≥ 1	Dimensión intermedia para embeddings factorizados. Típico: 64–256.
*model.embedding.conv.**		
`embedding.conv.enabled`	bool	Aplicar Conv1d sobre embeddings de tokens para capturar patrones locales de n-gramas.
`embedding.conv.kernel`	int ≥ 1	Tamaño de kernel para Conv1d de embedding. Típico: 3.
*model.attention.ode.**		
`attention.ode.solver`	enum (rk4, euler)	Método de integración numérica para capas ODE. Por defecto: rk4.
`attention.ode.steps`	int ≥ 1	Pasos de integración entre capas transformer consecutivas. Por defecto: 2.
*model.attention.retnet.**		
`attention.retnet.heads`	int ≥ 1	Cabezas de retención para capas `retnet`/`retnet_attn`. Debe dividir `hidden_size`. Por defecto: 8.
*model.attention.mla.**		
`attention.mla.latent_rank`	int ≥ 1	Rango del latente KV conjunto para MLA. Por defecto: `hidden_size // 2`.
*model.attention.gqla.**		
`attention.gqla.latent_rank`	int ≥ 1	Rango del latente KV conjunto para GQLA. Por defecto: `hidden_size // 2`.
`attention.gqla.num_groups`	int ≥ 1	Número de grupos GQA para la ruta de decodificación GQLA. Debe dividir `num_heads`.
`attention.gqla.decode_path`	enum (mqa_absorb, gqa)	Ruta de decodificación para GQLA.
*model.attention.mlra.**		
`attention.mlra.latent_rank`	int ≥ 1	Rango latente total para MLRA. Por defecto: `hidden_size // 2`.
`attention.mlra.num_latent_heads`	int ≥ 1	Número de sub-espacios latentes disjuntos para MLRA. Por defecto: 4.
*model.attention.tucker.**		
`attention.tucker.query_rank`	int ≥ 1	Rango Tucker para el tensor de pesos de consulta. Por defecto: `hidden_size`.

continúa en la página siguiente

Campo	Tipo	Descripción
attention.tucker.key_rank	int ≥ 1	Rango Tucker para el tensor de pesos de clave. Por defecto: hidden_size // 2.
attention.tucker.value_rank	int ≥ 1	Rango Tucker para el tensor de pesos de valor. Por defecto: hidden_size // 2.
*model.attention.iha.**		
attention.iha.num_pseudo_heads	int ≥ 1	Número de pseudo-cabezas por cabeza para IHA. Por defecto: num_heads.
*model.attention.gta.**		
attention.gta.num_shared_groups	int ≥ 1	Número de grupos de cabezas que comparten un mapa de atención para GTA. Debe dividir num_heads.
attention.gta.value_latent_rank	int ≥ 1	Rango del latente de caché de valores para GTA. Por defecto: hidden_size // 2.
*model.attention.mtla.**		
attention.mtla.latent_rank	int ≥ 1	Rango latente para MTLA. Por defecto: hidden_size // 2.
attention.mtla.merge_factor	int ≥ 1	Número de entradas KV consecutivas fusionadas por slot temporal. Por defecto: 2.
attention.mtla.stride	int ≥ 1	Stride entre slots temporales fusionados. Por defecto: mtla_merge_factor.
*model.attention.cca.**		
attention.cca.latent_rank	int ≥ 1	Ancho latente para CCA. Por defecto: hidden_size // 4. Debe ser divisible por num_heads.
attention.cca.num_conv_layers	enum (0, 1, 2)	Número de capas de convolución en CCA. Por defecto: 2.
attention.cca.conv_kernel_seq	int ≥ 1	Tamaño de kernel de convolución causal depth-wise de secuencia. Por defecto: 4.
attention.cca.conv_kernel_ch	int ≥ 1	Tamaño de kernel de convolución agrupada de canales por cabeza. Por defecto: 3.
attention.cca.qk_mean	bool	Activar sesgo q-k-mean en CCA. Por defecto: true.
attention.cca.value_shift	bool	Activar value-shift en CCA. Por defecto: true.
*model.attention.ccgqa.**		
attention.ccgqa.query_latent_rank	int ≥ 1	Ancho latente de consulta para CCGQA. Por defecto: hidden_size // 2.
attention.ccgqa.kv_latent_rank	int ≥ 1	Ancho latente KV para CCGQA. Por defecto: hidden_size // 8.
attention.ccgqa.num_kv_heads	int ≥ 1	Número de cabezas KV para CCGQA. Debe dividir num_heads.

continúa en la página siguiente

Campo	Tipo	Descripción
`attention.ccgqa.num_conv_layers`	enum (0, 1, 2)	Número de capas de convolución en CCGQA. Por defecto: 2.
`attention.ccgqa.conv_kernel_seq`	int ≥ 1	Tamaño de kernel de convolución causal depth-wise de secuencia. Por defecto: 4.
`attention.ccgqa.conv_kernel_ch`	int ≥ 1	Tamaño de kernel de convolución agrupada de canales por cabeza. Por defecto: 3.
`attention.ccgqa.qk_mean`	bool	Activar sesgo q-k-mean en CCGQA. Por defecto: `true`.
`attention.ccgqa.value_shift`	bool	Activar value-shift en CCGQA. Por defecto: `true`.
*model.attention.msa.**		
`attention.msa.block_size`	int ≥ 1	Tamaño de bloque para MiniMax Sparse Attention. Por defecto: 128.
`attention.msa.topk_blocks`	int ≥ 1	Número de bloques seleccionados por grupo GQA por consulta. Por defecto: 16.
`attention.msa.index_dim`	int ≥ 1	Dimensionalidad de las cabezas de la Rama de Índice. Por defecto: 64.
`attention.msa.kl_loss_weight`	float ≥ 0	Peso de la pérdida de alineación KL para la rama índice MSA. Por defecto: 0.0.
*model.attention.sparda.**		
`attention.sparda.block_size`	int ≥ 1	Tamaño de bloque KV para SparDA. Por defecto: 128.
`attention.sparda.topk_blocks`	int ≥ 1	Número de bloques KV seleccionados por grupo GQA. Por defecto: 16.
`attention.sparda.forecast_dim`	int ≥ 1	Dimensionalidad de la proyección Forecast. Por defecto: 64.
*model.attention.engram.**		
`attention.engram.max_ngram_size`	int ≥ 2	Tamaño máximo de N-grama para el módulo de memoria Engram.
`attention.engram.n_heads_per_ngram`	int ≥ 1	Número de cabezas hash por orden de N-grama.
`attention.engram.embed_dim_per_head`	int ≥ 1	Dimensión de embedding por cabeza hash.
`attention.engram.kernel_size`	int ≥ 1	Ancho de kernel de convolución depthwise causal para Engram.
`attention.engram.seed`	int	Semilla aleatoria para multiplicadores hash de N-gramas.
*model.mhc.**		
`mhc.enabled`	bool	Activar el flujo residual de n streams restringido por variedad (mHC, arXiv:2512.24880). Por defecto: `false`.

continúa en la página siguiente

Campo	Tipo	Descripción
`mhc.expansion_rate`	int ≥ 1	Factor de expansión del flujo residual n. El flujo pasa a ser $n\times$ `hidden_size`. 1 recupera el mapeo de identidad. Por defecto: 4.
`mhc.sinkhorn_iters`	int ≥ 1	Rondas de normalización Sinkhorn-Knopp que restringen H^{res} a ser doblemente estocástica. Por defecto: 20.
`mhc.gating_init`	float > 0	Valor inicial de los escalares de compuerta mHC α. Por defecto: 0.01.
`mhc.checkpoint`	bool	Gradient checkpointing en capas mHC para mitigar el aumento de $\sim n\times$ en memoria de activaciones. Por defecto: `false`.
`mhc.full_prec_under_bitnet`	bool	Mantener la proyección de coeficientes mHC en precisión completa bajo BitNet. Por defecto: `true`.
*model.moe.**		
`moe.enabled`	bool	Activar Mezcla de Expertos en capas FFN. Por defecto: `true`.
`moe.num_experts`	int ≥ 1	Número total de redes expertas. Por defecto: 8.
`moe.top_k`	int ≥ 1	Expertos activados por token. Debe ser $\leq$ `num_experts`. Por defecto: 2.
*model.bitnet.**		
`bitnet.enabled`	bool	Activar cuantización ternaria BitLinear. Por defecto: `true`.
`bitnet.routers`	bool	Cuantizar también proyecciones de enrutamiento/puntuación. Requiere `bitnet.enabled=true`. Por defecto: `false`.
`bitnet.conv`	bool	Cuantizar Conv1d de embedding con BitConv1d. Requiere `bitnet.enabled=true` y `embedding.conv.enabled=true`. Por defecto: `false`.
*model.mixture_of_depths.**		
`mixture_of_depths.enabled`	bool	Activar ruteo de tokens Mixture-of-Depths. Por defecto: `false`.
`mixture_of_depths.capacity_ratio`	float (0,1]	Fracción de tokens actualizados por capa. Por defecto: 0.5.
`mixture_of_depths.router_aux_loss_weight`	float ≥ 0	Peso de la pérdida auxiliar del router. Por defecto: 0.0.
*model.ssmax.**		
`ssmax.enabled`	bool	Activar Scalable Softmax (arXiv:2505.22842). Por defecto: `false`.

continúa en la página siguiente

Campo	Tipo	Descripción
`ssmax.s_init`	float > 0	Escalar SSMax inicial por cabeza. Por defecto: 1.0.
*model.numerics.**		
`numerics.fixed_softmax`	bool	Softmax/log-softmax sin cancelación (arXiv:2610.31642). Por defecto: `false`.
`numerics.fixed_cross_entropy`	bool	CE de objetivo duro con gradiente de suma cero por construcción (arXiv:2610.31642). Por defecto: `false`.
*model.ffn.**		
`ffn.hidden_size`	int ≥ 1	Dimensión intermedia FFN. Por defecto: 2× `hidden_size`.
`ffn.activation`	enum	No linealidad FFN (43 valores). Por defecto: `silu`.
`ffn.activation_config`	object	Parámetros para activaciones aprendibles/paramétricas.
*model.classification_head.**		
`classification_head.enabled`	bool	Activar la cabeza de clasificación NLP. Por defecto: `false`.
`classification_head.num_labels`	int o null	Número de clases.
`classification_head.pooling_mode`	enum (cls, gap)	Pooling de la cabeza de clasificación. Por defecto: `cls`.

Cuadro 2: Campos completos de configuración del modelo. Todos los campos son opcionales a menos que se indique como obligatorio.

Los campos se agrupan bajo sub-objetos jerárquicos (`dims`, `norm`, `embedding`, `attention`, `moe`, `bitnet`, `mixture_of_depths`, `ssmax`, `numerics`, `ffn`, `classification_head`, `mhc`, `residuals`); cada fila de sub-encabezado marca el prefijo vigente para las filas que le siguen. No hay claves de características planas en `model.*`.

2.2.1. Enumeración del Patrón de Capas

El array `model.dims.layer_pattern` acepta los siguientes 43 códigos de mezclador. Cada elemento define una capa física en la pila del modelo. La Tabla 3 enumera todos los tipos disponibles con su soporte de entrenamiento e inferencia.

Tipo	Descripción	Entrenar	Inferir
`standard_attn`	Atención multi-cabeza clásica (estilo BERT)	✓	✓
`sigmoid_attn`	Atención con compuerta sigmoide	✓	✓
`gated_softmax_attn`	Atención softmax con compuerta sigmoide post-SDPA	✓	✓

continúa en la página siguiente

Tipo	Descripción	Entrenar	Inferir
titan_attn	Atención Titan con codificación posicional (HoPE/RoPE)	✓	✓
retnet / retnet_attn	Red de Retención (híbrido RNN+transformer)	✓	✓
mamba	Modelo de espacio de estado selectivo (SSM)	✓	✓
ode	Capa ODE de profundidad continua	✓	✓
gla_attn	Atención Lineal con Compuerta con decaimiento multiplicativo	✓	✓
deltanet_attn / gated_deltanet_attn	DeltaNet con regla delta correctiva	✓	✓
gated_deltanet2_attn	Gated DeltaNet-2 con compuertas de borrado/escritura canalizadas desacopladas	✓	✓
hgrn2_attn	HGRN2 con compuertas de olvido jerárquicas	✓	✓
fox_attn	Forgetting Transformer (FoX) con sesgo de olvido en logits	✓	✓
nsa_attn	Native Sparse Attention (tres ramas)	✓	✓
engram_attn	Búsqueda condicional de memoria de N-gramas	✓	✓
gqa_attn	Atención por Consultas Agrupadas con cabezas KV configurables	✓	✓
longformer_attn	Longformer: ventanas deslizantes + tokens globales	✓	✓
bigbird_attn	BigBird: atención local + aleatoria + global	✓	✓
sparse_transformer_attn	Sparse Transformer: patrones factorizados	✓	✓
sparsek_attn	SparseK: selección KV top-k diferenciable	✓	✓
sparge_attn	SpargeAttn — poda de bloques en dos etapas; entrenamiento denso, inferencia dispersa	✓	✓
fasa_attn	FASA — selección de tokens consciente de frecuencia; entrenamiento denso, inferencia dispersa	✓	✓
msa_attn	MiniMax Sparse Attention — bloques dispersos sobre GQA	✓	✓

continúa en la página siguiente

Tipo	Descripción	Entrenar	Inferir
sparda_attn	SparDA — atención dispersa desacoplada con proyección Forecast	✓	✓
kda_attn	Kimi Delta Attention — decaimiento por canal + compuerta de escritura escalar	✓	✓
falcon1_attn / falcon2_attn / falcon3_attn	Familia de regresión Falcon — regla delta NLMS prefix-alineada (RAW)	✓	✓
falcon1a_attn / falcon2a_attn / falcon3a_attn	Familia de producto interno Falcon — escrituras aditivas desplazadas con decaimiento por ridge	✓	✓
mla_attn	Atención Latente Multi-Cabeza + RoPE	✓	✓
gqla_attn	Atención Latente por Consultas Agrupadas — dos rutas de decodificación	✓	✓
mlra_attn	Atención Multi-Cabeza de Rango Bajo — latente particionable	✓	✓
tucker_attn	Atención Tucker — factorización de rango bajo generalizada	✓	✓
iha_attn	Atención de Cabezas Entrelazadas — pseudo-cabezas entre cabezas	✓	✓
gta_attn	Atención laTenT por Cabezas Agrupadas — mapa compartido + valores latentes	✓	✓
mtla_attn	Atención Latente Temporal Multi-Cabeza — fusión temporal KV	✓	✓
cca_attn	Atención Convolucional Comprimida — atención en espacio latente + convs	✓	✓
ccgqa_attn	Atención Convolucional Comprimida por Consultas Agrupadas — CCA + GQA en latente	✓	✓
gma_attn	Atención de Mezcla Gaussiana — enrutamiento latente probabilístico, $\mathcal{O}(NK)$ tiempo lineal	✓	✓

Cuadro 3: Enumeración completa de `layer_pattern`. ✓ = soportado, × = no soportado. `fasa_attn` y `sparge_attn` entrenan como atención estándar densa sobre sus propias proyecciones y aplican su heurística dispersa solo en inferencia.

2.3. Configuración de Entrenamiento

El objeto `training` controla todo el pipeline de entrenamiento. El único campo obligatorio es `task`. La Tabla 4 enumera todos los campos de entrenamiento.

Campo	Tipo	Descripción
`task`	enum (mlm, sbert, causal_lm) (obligatorio)	Objetivo de entrenamiento. `mlm` = Modelado de Lenguaje Enmascarado; `sbert` = ajuste fino Sentence-BERT; `causal_lm` = predicción autorregresiva del siguiente token (requiere `model_class='frankenstein_text_decoder'`).
`num_epochs`	int ≥ 1	Pasadas completas por el dataset de entrenamiento (solo MLM).
`batch_size`	int ≥ 1	Ejemplos por paso del optimizador (antes de acumulación de gradientes).
`dataloader_workers`	int ≥ 0	Procesos trabajadores paralelos para carga de datos.
`max_length`	int ≥ 1	Longitud máxima de secuencia de tokens. Típico: 128–2048.
`mlm_probability`	float [0,1]	Fracción de tokens enmascarados para predicción MLM. Estándar: 0.15.
`max_samples`	int ≥ 1	Muestras totales antes de detenerse, independiente de épocas.
`test_percentage`	float [0,100]	Fracción del dataset reservada fuera del entrenamiento y evaluada al final; el resultado se escribe en `test_metrics.json`. Por defecto: 0 (desactivado).
`dataset_batch_size`	int ≥ 1	Tamaño de lote interno para dataset de streaming.
`num_workers`	int ≥ 0	Trabajadores paralelos para pipeline de streaming de dataset.
`cache_dir`	string	Directorio para cachear datasets procesados.
`local_parquet_dir`	string	Ruta a archivos parquet locales para streaming offline.
`prefer_local_cache`	bool	Verificar caché local primero antes de descarga remota.
`stream_local_parquet`	bool	Hacer streaming desde parquet local cuando `local_parquet_dir` está configurado.
`join_temp_data_context_window`	int ≥ 0	Si > 0, une fragmentos de tokens en caché en secuencias de esta longitud.
`join_temp_data_min_remainder_tokens`	int ≥ 0	Tokens de contenido mínimos en la última ventana parcial unida.
`use_amp`	bool	Activar precisión mixta FP16.
`gradient_accumulation_steps`	int ≥ 1	Acumular gradientes sobre N mini-lotes antes del paso del optimizador.

continúa en la página siguiente

Campo	Tipo	Descripción
optimizer	object	Configuración del optimizador (ver §2.5). Obligatorio cuando task=mlm o task=causal_lm.
sbert	object	Configuración de entrenamiento SBERT (ver §2.6). Obligatorio cuando task=sbert.
scheduler_total_steps	int ≥ 1	Pasos totales del optimizador para la programación de LR.
scheduler_warmup_ratio	float [0,1]	Fracción de scheduler_total_steps para calentamiento de LR.
scheduler_type	enum (cosine, constant, linear_ warmup_ then_ constant)	Estrategia de decaimiento de LR.
grad_clip_max_norm	float ≥ 0	Norma máxima permitida del gradiente. 0 = desactivado.
inf_post_clip_threshold	float ≥ 0	Umbral para marcar explosiones de gradiente post-recorte.
max_nan_retries	int ≥ 0	Reintentos máximos ante gradientes NaN/Inf antes de detenerse.
checkpoint_every_n_steps	int ≥ 1	Guardar checkpoint rotativo cada N pasos.
max_rolling_checkpoints	int ≥ 1	Máximo de checkpoints rotativos mantenidos. El más antiguo se elimina al exceder.
num_best_checkpoints	int ≥ 1	Número de mejores checkpoints conservados permanentemente (por pérdida de validación).
nan_check_interval	int ≥ 1	Pasos entre verificaciones de NaN/Inf.
log_gradient_stats	bool	Registrar normas de gradiente por capa, min/max.
gradient_log_interval	int ≥ 1	Pasos entre registros de estadísticas de gradiente.
csv_log_path	string	Ruta de archivo para métricas de entrenamiento a nivel de paso en formato CSV.
csv_rotate_on_schema_change	bool	Rotar archivo CSV cuando cambia el esquema.
gpu_metrics_backend	enum (nvml, none)	Backend de telemetría GPU.
nvml_device_index	int ≥ 0	Índice de GPU a monitorear (0 = primera GPU).
enable_block_grad_norms	bool	Registrar normas de gradiente por bloque (embeddings, atención, FFN, etc.).
telemetry_log_interval	int ≥ 1	Pasos entre registros de telemetría pesada.

continúa en la página siguiente

Campo	Tipo	Descripción
gpu_temp_guard_enabled	bool	Monitoreo estricto de temperatura GPU. Requiere gpu_metrics_backend=nvml.
switch_on_thermal	bool	Temperatura crítica activa fallback GPU→CPU, reanudación automática al enfriarse.
gpu_temp_pause_threshold_c	float	Temperatura (°C) para pausar entrenamiento.
gpu_temp_resume_threshold_c	float	Temperatura (°C) para reanudar entrenamiento tras pausa.
gpu_temp_critical_threshold_c	float	Marcador de registro informativo para eventos térmicos severos.
gpu_temp_poll_interval_seconds	float	Segundos entre verificaciones de temperatura durante espera de enfriamiento.
use_galore	bool	Activar Proyección de Gradiente de Bajo Rango para entrenamiento eficiente en memoria.
galore_rank	int ≥ 1	Dimensión de proyección de bajo rango para GaLore.
galore_update_interval	int ≥ 1	Pasos entre recomputaciones de la matriz de proyección.
galore_scale	float ≥ 0	Factor de escala para gradientes proyectados.
galore_max_dim	int ≥ 1	Dimensión máxima del tensor para proyección GaLore.
hf_output_dir	string	Directorio para guardar el modelo final mediante save_pretrained().

Cuadro 4: Campos completos de configuración de entrenamiento.

2.4. Configuración del Tokenizador

El objeto tokenizer de nivel superior está disponible para ambas rutas de entrenamiento: la ruta base_model (preentrenamiento continuo / ajuste fino, donde el tokenizador debe coincidir con el checkpoint preentrenado) y la ruta personalizada desde cero (donde resuelve el vocabulario antes de asignar la matriz de embeddings). El campo source selecciona entre dos modos:

- hf_repo (por defecto) — carga un tokenizador preentrenado desde name_or_path (id de repo de HuggingFace o ruta local) mediante AutoTokenizer.from_pretrained(). Banderas complementarias: use_fast (tokenizador rápido Rust, por defecto true) y trust_remote_code (por defecto false).
- train_from_dataset — entrena un tokenizador *nuevo* desde el texto del corpus con la biblioteca tokenizers de HuggingFace. El corpus es el bloque text_dataset de nivel superior (id de HF hub, opcionalmente en streaming, o un directorio local parquet/json); cuando el config es dirigido por un host de embedding como dashAI, el dataset del run del host puede suministrarlo. El tamaño de vocabulario resultante se inyecta en model.dims.vocab_size antes de construir el modelo, de modo que la matriz de embeddings siempre coincide con el tokenizador. En este modo name_or_path debe omitirse.

La Tabla 5 enumera los campos del sub-objeto tokenizer.training.

Campo	Tipo	Descripción
algorithm	enum (bpe, wordpiece, wordlevel, unigram)	Algoritmo de entrenamiento del tokenizador. bpe (Byte Pair Encoding, estilo GPT-2, por defecto); wordpiece (BERT original); wordlevel (tabla de palabras); unigram (modelo de lenguaje de sub-palabras, con poda automática).
vocab_size	int ≥ 1	Tamaño de vocabulario objetivo. Obligatorio para bpe/wordpiece/wordlevel; opcional para unigram.
min_frequency	int ≥ 1	Frecuencia mínima de token a conservar (por defecto 2).
special_tokens	array de string	Tokens especiales registrados en el tokenizador (por defecto [PAD]/[UNK]/[CLS]/[SEP]/[MASK]).
save_dir	string	Directorio persistido vía save_pretrained().
lowercase	bool	Convertir todo el texto a minúsculas antes de entrenar (por defecto false).
strip_accents	bool	Quitar diacríticos antes de entrenar (por defecto false).
pre_tokenizer	enum (whitespace, bytelevel, bert)	Estrategia de pre-tokenización (por defecto whitespace).
max_token_length	int ≥ 0	Límite de caracteres por token; 0 desactiva el límite.

Cuadro 5: Campos tokenizer.training.* (obligatorio cuando source=train_from_dataset).

2.5. Configuración del Optimizador

El objeto training.optimizer tiene dos campos:

- optimizer_class (enum, obligatorio) — Uno de 23 algoritmos optimizadores: sgd_momentum, adamw, adafactor, galore_adamw, prodigy, lion, sophia, muon, turbo_muon, radam, adan, adopt, ademamix, mars_adamw, cautious_adamw, lamb, schedulefree_adamw, shampoo, soap, anon, apollo, apollo_mini, q_apollo.
- parameters (object) — Hiperparámetros del optimizador con claves prefijadas en el formato <optimizador>-<grupo>_<param>.

2.5.1. Sufijos Compartidos por Grupo

Todos los optimizadores soportan los siguientes sufijos por grupo de parámetros, prefijados por el nombre de la clase del optimizador (ej., adamw-lr_embeddings):

- **Grupos de tasa de aprendizaje:** lr_embeddings, lr_norms, lr_attention, lr_other
- **Grupos de decaimiento de peso:** wd_embeddings, wd_norms, wd_attention, wd_other
- **Grupos beta:** betas_embeddings, betas_norms, betas_attention, betas_other
- **Grupos epsilon:** eps_embeddings, eps_norms, eps_attention, eps_other

Nota: los parámetros de los mezcladores ODE, RetNet y Mamba se enrutan al grupo `attention` en lugar de tener grupos dedicados.

2.5.2. Sufijos Globales Específicos del Optimizador

La Tabla 6 enumera los sufijos globales adicionales soportados por cada optimizador (prefijados por el nombre del optimizador).

Optimizador	Sufijos Globales Específicos
sgd_momentum	momentum, nesterov
adamw	(ninguno — solo sufijos compartidos)
adafactor	beta2_decay, clip_threshold, eps1, eps2
galore_adamw	rank, update_proj_gap
prodigy	d_coef
lion	(ninguno — solo sufijos compartidos)
sophia	rho, update_k
muon	momentum, nesterov, ns_steps, ns_eps
turbo_muon	momentum, nesterov, ns_steps, ns_eps
radam	(ninguno — solo sufijos compartidos)
adan	(ninguno — solo sufijos compartidos)
adopt	(ninguno — solo sufijos compartidos)
ademamix	(ninguno — solo sufijos compartidos)
mars_adamw	(ninguno — solo sufijos compartidos)
cautious_adamw	cautious_clip
lamb	(ninguno — solo sufijos compartidos)
schedulefree_adamw	(ninguno — solo sufijos compartidos)
shampoo	(ninguno — solo sufijos compartidos)
soap	(ninguno — solo sufijos compartidos)
anon	gamma
apollo	rank, update_proj_gap, scale, scale_type, proj_type, scale_front, disable_nl
apollo_mini	update_proj_gap, scale, proj_type, scale_front, disable_nl
q_apollo	rank, update_proj_gap, scale, scale_type, proj_type, scale_front, disable_nl, quant_bits

Cuadro 6: Sufijos de parámetros globales específicos del optimizador. Todas las claves están prefijadas con el nombre de la clase del optimizador (ej., `sgd_momentum-momentum`).

2.6. Configuración SBERT

El objeto `training.sbert` configura el ajuste fino Sentence-BERT. Es obligatorio cuando `training.task=sbert`. Los campos clave incluyen:

- `dataset_name` (string) — Identificador de dataset HuggingFace o ruta local.
- `dataset_type` (enum) — `paired_similarity`, `triplets` o `qa`.
- `columns` (object) — Sobrescrituras opcionales de nombres de columna para campos del dataset.
- `query_prefix` / `document_prefix` (string) — Texto antepuesto a campos de consulta/documento durante la tokenización.
- `output_dir` (string) — Directorio para artefactos SBERT.

- `batch_size` (int) — Ejemplos por lote. Típico: 16–64.
- `gradient_accumulation_steps` (int) — Pasos de acumulación para SBERT.
- `grad_clip_max_norm` (float) — Umbral de recorte de gradiente. Típico: 0.5–2.0.
- `num_epochs` (int) — Pasadas completas por el dataset SBERT. Típico: 1–10.
- `warmup_steps` (int) — Pasos para calentamiento de LR.
- `evaluation_steps` (int) — Evaluar cada N pasos.
- `checkpoint_save_steps` (int) — Guardar checkpoint rotativo cada N pasos.
- `resume_from_checkpoint` (string o null) — `auto` reanuda desde el último checkpoint; `null` empieza desde cero.
- `learning_rate` (float) — LR inicial. Típico: 1×10^{-5} a 1×10^{-4}.
- `max_train_samples` / `max_eval_samples` (int) — Limitar muestras de entrenamiento/evaluación.
- `max_length` (int) — Longitud máxima de tokens para textos de entrada. Típico: 128–512.
- `pooling_mode` (enum) — `mean`, `cls` o `max`.
- `trust_remote_code` (bool) — Permitir código personalizado del repositorio del modelo.
- `use_amp` (bool) — Activar precisión mixta FP16.
- `resample_balanced` (bool) — Remuestrear a distribución balanceada sobre puntajes de similitud.
- `standardize_scores` (bool) — Normalizar puntajes de similitud con puntuación Z.
- `resample_std` (float) — Desviación estándar objetivo cuando `standardize_scores=true`.
- `optimizer` (object) — Optimizador personalizado para SBERT (mismas clases que el entrenamiento MLM).
- `wandb_project` (string) — Nombre del proyecto Weights & Biases para telemetría.

2.7. Reglas de Validación

El esquema impone las siguientes reglas de validación entre componentes, implementadas tanto en el JSON Schema (`_conditional_rules.yaml`) como en tiempo de ejecución en el cargador de configuración:

1. `additionalProperties: false` se aplica en todos los niveles de anidamiento. Cualquier clave no reconocida en cualquier objeto desencadena un error de validación.
2. `model.dims.hidden_size` debe ser divisible por `model.dims.num_heads`. La dimensión por cabeza es `hidden_size/num_heads`.
3. `model.dims.num_kv_heads` debe dividir `model.dims.num_heads` exactamente cuando se usa Atención por Consultas Agrupadas.
4. `vocab_size` debe coincidir con el tamaño de vocabulario del tokenizador. Una discrepancia causa corrupción de IDs de token y fallo de entrenamiento irrecuperable.
5. Cuando `base_model` está configurado, `training.task` es obligatorio. Si `task=mlm`, `tokenizer.name_or_path` también es obligatorio — salvo cuando `tokenizer.source=train_from_dataset`, en cuyo caso el tokenizador se entrena desde el corpus y no se necesita nombre.
6. `tokenizer.source=train_from_dataset` requiere el sub-objeto `tokenizer.training` (con `vocab_size` para `bpe`/`wordpiece`/`wordlevel`) y es mutuamente excluyente con `tokenizer.name_or_path`. El vocabulario entrenado sobrescribe `model.dims.vocab_size` antes de construir el modelo.
7. `task: mlm` requiere `training.optimizer`; `task: sbert` requiere `training.sbert`; `task: causal_lm` requiere `training.optimizer` y `model_class='frankenstein_text_decoder'`.
8. `bitnet.routers=true` requiere `bitnet.enabled=true`. La bandera no tiene efecto en caso contrario.
9. `use_mhc=true` (flujo residual de n streams mHC) y `use_mixture_of_depths=true` son mutuamente excluyentes. Habilitar ambos lanza un `ValueError`: el enrutamiento MoD de

tokens opera sobre un flujo C-dimensional único, en conflicto con el flujo residual de n streams.

10. `fasa_attn` y `sparge_attn` son tipos de capa dispersos en inferencia: aplican su heurística dispersa solo en modo evaluación. Durante el entrenamiento la capa híbrida contenedora sustituye atención estándar densa sobre las mismas proyecciones **q**/**k**/**v**/**o**, por lo que pueden aparecer en `model.dims.layer_pattern` para ejecuciones de entrenamiento; los pesos entrenados son los que lee la ruta dispersa en inferencia.
11. `model_class: frankenstein_text_decoder` fuerza `model.dims.mode: decoder` en tiempo de ejecución. Establecer `model.dims.mode: encoder` con esta clase se sobrescribe.
12. O bien `base_model` debe estar presente, o bien tanto `model_class` como `model` deben estar presentes. El esquema rechaza configuraciones que no proporcionen ninguno o una combinación incompleta.

3. Tablas de Resumen Completas

Este anexo proporciona una referencia completa de todos los componentes implementados en el código base de Frankenstein Transformer: 45 mezcladores de secuencia en nueve familias, 26 optimizadores, 6 variantes de normalización, 43 funciones de activación y 4 métodos de embedding. Cada tabla es autocontenida y referencia la bibliografía.

3.1. Resumen de Mezcladores de Secuencia

La Tabla 7 enumera cada mezclador de secuencia soportado por el campo de esquema `layer_pattern`, organizado por familia arquitectónica. La complejidad de entrenamiento e inferencia se expresa en términos de la longitud de secuencia n, la dimensión oculta d y parámetros específicos de cada familia.

Nombre	Familia	Entrenamiento	Inferencia	Característica Clave	Referencia
Densos y aumentados con convolución (4)					
standard_attn	Denso	$\mathcal{O}(n^2 d)$	$\mathcal{O}(n)$ caché	Contexto global completo; cuello de botella KV en secuencias largas	[113]
sigmoid_attn	Denso	$\mathcal{O}(n^2 d)$	$\mathcal{O}(n)$ caché	Compuerta elemento a elemento; reemplazo eficiente en hardware para softmax	[97]
gqa	Denso	$\mathcal{O}(n^2 d)$	$\mathcal{O}(n)$ caché	Compresión KV ajustable; reducción de caché $G\times$ mediante cabezas compartidas	[2]
conformer_attn	Aumentado conv.	$\mathcal{O}(n^2 d + nKd)$	$\mathcal{O}(n)$ (recómputo completo)	FFN Macaron alrededor de MHSA sinusoidal relativa y un módulo conv depthwise ($K = 32$); contexto local y global en un solo bloque	[45]
Recurrente (5)					
retnet_attn	Recurrente	$\mathcal{O}(n^2 d)$ paralelo	$\mathcal{O}(1)$	Decaimiento multi-escala; triple modo de cómputo (paralelo/recurrente/por fragmentos)	[109]

continúa en la página siguiente

Nombre	Familia	Entrenamiento	Inferencia	Característica Clave	Referencia
mamba	Recurrente	$\mathcal{O}(nd)$	$\mathcal{O}(1)$	Modelo de espacio de estado selectivo; escaneo paralelo consciente del hardware	[43]
ode	Recurrente	$\mathcal{O}(k \cdot n^2 d)$	$\mathcal{O}(kn)$	Integración RK4 de profundidad continua; solucionador de paso adaptativo	[130]
titan_attn	Recurrente	$\mathcal{O}(c^2 + nf)$	$\mathcal{O}(c^2 + 1)$	Adaptación de memoria en tiempo de prueba; dinámica de escritura impulsada por sorpresa	[11]
engram_attn	Memoria	$\mathcal{O}(n \cdot N \cdot H)$	$\mathcal{O}(1)$ lookup	Memoria condicional de N-gramas; recuperación escalable basada en hash	[19]
Disperso (10)					
sparse_transformer_attn	Disperso	$\mathcal{O}(n\sqrt{n} \cdot d)$	$\mathcal{O}(n)$	Máscaras factorizadas estrideadas + fijas; patrón de dispersidad $\sqrt{n}$	[20]
longformer_attn	Disperso	$\mathcal{O}(n \cdot w \cdot d)$	$\mathcal{O}(n)$	Ventana deslizante + dilatación + tokens globales; lineal en n	[12]
swa_attn	Disperso	$\mathcal{O}(n \cdot w \cdot d)$	$\mathcal{O}(w)$	Ventana deslizante + sumideros de atención; sin entrenamiento, caché KV fija	[58]
bigbird_attn	Disperso	$\mathcal{O}(n)$	$\mathcal{O}(n)$	Aleatorio + local + global; atención Turing completa	[127]
sparsek_attn	Disperso	$\mathcal{O}(n \cdot d)$	$\mathcal{O}(k)$	Selección top-k diferenciable; máscara de dispersidad aprendida	[77]
nsa_attn	Disperso	$\mathcal{O}((t/d + n_l + w)d)$	$\mathcal{O}(n)$	Diseño de 3 ramas alineado con hardware; token/vecino/ventana	[125]
sparge_attn	Disperso	$\mathcal{O}(n^2 \cdot s \cdot d)$	$\mathcal{O}(n)$	Filtrado de bloques en dos etapas; entrenamiento denso, inferencia dispersa	[131]
fasa_attn	Disperso	$\mathcal{O}(t \cdot N_{\text{tip}} + N_{\text{fac}} \cdot d)$	$\mathcal{O}(n)$	Dispersidad consciente de frecuencia RoPE; entrenamiento denso, inferencia dispersa	[116]
msa_attn	Disperso	$\mathcal{O}(H_{\text{kv}} \cdot d_{\text{idx}} \cdot N^2 + H_q \cdot d_h \cdot N \cdot k \cdot B_k)$	$\mathcal{O}(n)$	Bloques dispersos con rama índice; desplegado a escala 109B	[67]
sparda_attn	Disperso	$\mathcal{O}(H_{\text{kv}} \cdot d_f \cdot N \cdot N_b + H_q \cdot d_h \cdot N \cdot k \cdot B_k)$	$\mathcal{O}(n)$	Prefetch anticipado guiado por pronóstico; dispersidad predictiva	[36]
Con Compuerta (8)					
gla_attn	Con compuerta	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Decaimiento diagonal dependiente de datos; atención lineal con compuerta	[121]

continúa en la página siguiente

Nombre	Familia	Entrenamiento	Inferencia	Característica Clave	Referencia
deltanet_ attn	Con compuerta	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Regla delta; atención lineal con corrección de errores	[122]
gated_ deltanet_ attn	Con compuerta	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Síntesis de decaimiento + compuerta de escritura; marco delta unificado	[123]
gated_ deltanet2_ attn	Con compuerta	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Compuertas de borrado/escritura canalizadas desacopladas; compuerta mejorada	[48]
hgrn2_attn	Con compuerta	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Compuertas de olvido acotadas jerárquicas; recurrencia multi-escala	[94]
fox_attn	Con compuerta	$\mathcal{O}(L^2d)$	$\mathcal{O}(L)$ KV	Sesgo de recencia en espacio de logits; compatible con FlashAttn	[71]
gated_ softmax_ attn	Con compuerta	$\mathcal{O}(L^2d)$	$\mathcal{O}(L)$ KV	Compuerta sigmoide post-SDPA; mitigación del sumidero de atención	[95]
kda_attn	Con compuerta	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Decaimiento por canal + escritura escalar; diseño Kimi Linear	[63]
Peso Rápido (6)					
falcon1_attn	Peso rápido	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Regla delta NLMS prefix-alineada (RAW); mejor perplejidad LM de la familia	[132]
falcon2_attn	Peso rápido	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Tamaños de paso NLMS por canal de valor	[132]
falcon3_attn	Peso rápido	$\mathcal{O}(Ld^2B)$	$\mathcal{O}(d^2)$ + cola $\mathcal{O}(B)$	Repetición de minilote con ventana deslizante y estadísticos exactos	[132]
falcon1a_ attn	Peso rápido	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Atención lineal aditiva desplazada con decaimiento por ridge	[132]
falcon2a_ attn	Peso rápido	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Decaimientos aditivos por canal	[132]
falcon3a_ attn	Peso rápido	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ + cola $\mathcal{O}(B)$	Aditivo con ventana; mejor extrapolación de longitud	[132]
Latente (9)					
mla_attn	Latente	$\mathcal{O}(n^2d)$	$\mathcal{O}(r_{\text{kv}} \cdot n)$	KV latente de rango $r_{\text{kv}} \approx d/2$; estilo DeepSeek	[80]
gqla_attn	Latente	$\mathcal{O}(n^2d)$	$\mathcal{O}(r_{\text{kv}} \cdot n)$	Rutas de decodificación duales adaptativas al hardware; fusión GQA + latente	[81]
mlra_attn	Latente	$\mathcal{O}(n^2d)$	$\mathcal{O}(r_{\text{kv}} \cdot n)$	L sub-cabezas; compatible con tensor-paralelismo de 4 vías	[74]

continúa en la página siguiente

Nombre	Familia	Entrenamiento	Inferencia	Característica Clave	Referencia
tucker_attn	Latente	$\mathcal{O}(n^2 d)$	$\mathcal{O}(k_{\text{rank}} \cdot n)$	Factorización Tucker; unifica MHA/GQA/MLA	[65]
iha_attn	Latente	$\mathcal{O}(n^2 H^2)$	$\mathcal{O}(H \cdot n \cdot d_h)$	Pseudo-cabezas; escalado de parámetros $\Theta(\sqrt{k})$	[31]
gta_attn	Latente	$\mathcal{O}(n^2 \cdot G \cdot d_h)$	$\mathcal{O}(r_v \cdot n)$	Mapa compartido + decodificador SiLU; reducción de FLOPs del 62.5 %	[108]
mtla_attn	Latente	$\mathcal{O}(n^2 d/m)$	$\mathcal{O}(r_{\text{kv}} \cdot n/m)$	Fusión temporal; aceleración de decodificación de 5,3×	[26]
cca_attn	Latente	$\mathcal{O}(n^2 d/C)$	$\mathcal{O}(\tilde{e} \cdot n)$	Atención completa en latente comprimido; $C\times$ menos FLOPs	[35]
ccgqa_attn	Latente	$\mathcal{O}(n^2 d/C_1)$	$\mathcal{O}(\tilde{e}_{\text{kv}} \cdot n)$	CCA + GQA desacoplada; mejor pérdida a 8× reducción de caché	[35]
gma_attn	Latente	$\mathcal{O}(nKd_r + nKd_v)$	$\mathcal{O}(K)$/paso	Atención de Mezcla Gaussiana; enrutamiento latente probabilístico a través de K componentes GMM aprendidos por cabeza; tiempo lineal para K fijo; causal vía cumsums de prefijo	[57]
ssog_attn	Campo geométrico	$\mathcal{O}(R \cdot n\sqrt{n} \cdot d)$	$\mathcal{O}(R \cdot n\sqrt{n} \cdot d)$/pasada	SSOG: campo Gaussiano separable sobre posición relativa; el contenido dirige (residuos acotados $\mu/\sigma/\lambda$ detrás de compuertas de arranque en frío), nunca puntúa; solo encoder	[91]

Cuadro 7: Catálogo completo de mezcladores de secuencia. n = longitud de secuencia, d = tamaño oculto, w = tamaño de ventana, r = dimensión de rango bajo, k = parámetro de dispersidad, C = razón de compresión. Los parámetros específicos de cada familia se definen en el anexo correspondiente.

3.2. Resumen de Optimizadores

La Tabla 8 cataloga los 26 optimizadores soportados con su huella de memoria, costo computacional por paso e hiperparámetros clave. Los búferes de estado se expresan como el número de tensores propiedad del optimizador por parámetro; el costo por paso se enfoca en el término dominante más allá del cálculo del gradiente.

Optimizador	Familia	Búferes de Estado	Costo por Paso	Hiperparámetros Clave	Referencia
Líneas Base Clásicas y Adaptativas					
SGD+ Momentum	Clásico	1 (m)	$\mathcal{O}(n)$	lr, momentum, wd	[92]

continúa en la página siguiente

Optimizador	Familia	Búferes de Estado	Costo por Paso	Hiperparámetros Clave	Referencia
AdamW	Adaptativo	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd	[76]
RAdam	Adaptativo	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd	[73]
Reducción de Varianza y Momentum Avanzado					
Adan	Reducción de varianza	3 (m, v, s)	$\mathcal{O}(n)$	lr, β_1, β_2, β_3, eps, wd	[119]
ADOPT	Variante Adam	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd	[111]
AdEMAMix	Multi-EMA	3 (m_1, m_2, v)	$\mathcal{O}(n)$	lr, β_1, β_2, β_3, eps, wd	[87]
MARS	Varianza reducida	3 (m, v, z)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd, γ	[126]
Cautious AdamW	Momentum enmascarado	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd	[70]
Lote Grande, Eficientes en Memoria y Sin Parámetros					
LAMB	Adaptativo por capas	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd	[124]
Schedule-Free	Sin programación	3 (z, x, n)	$\mathcal{O}(n)$	lr, β_1, β_2, wd	[25]
Adafactor	Eficiente en memoria	1–2 (fila/col)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd	[102]
GaLore	Proyección de bajo rango	2 (m, v) + SVD	$\mathcal{O}(nr)$	lr, rango r, β_1, β_2, eps, wd	[133]
Prodigy	Sin parámetros	3 (m, v, d)	$\mathcal{O}(n)$	β_1, β_2, eps, wd, d_0	[82]
Lion	Momentum de signo	1 (m)	$\mathcal{O}(n)$	lr, β_1, β_2, wd	[18]
Segundo Orden, Geométricos y Ortogonalidad					
Shampoo	Segundo orden	$2d$ (L_i, R_i)	$\mathcal{O}(n^{1+2/d})$	lr, eps, wd	[46]
SOAP	Segundo orden	$2d+2$ (m, v)	$\mathcal{O}(n^{1+1/d})$	lr, β_1, β_2, eps, wd	[114]
Sophia	Consciente de curvatura	3 (m, v, h)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd, k	[72]
Muon	Ortogonalidad (híbrida)	1 (m); + v en el respaldo AdamW	$\mathcal{O}(n \cdot k)$	lr, momentum, wd, pasos NS k; β, eps para embeddings/pos/heads/vectores	[103]
Turbo-Muon	Ortogonalidad (híbrida)	1 (m); + v en el respaldo AdamW	$\mathcal{O}(n \cdot k)$	lr, momentum, wd, pasos NS k; β, eps para embeddings/pos/heads/vectores	[14]
AF-Muon	Tabla atada sin AdamW	1 (m)	$\mathcal{O}(n \cdot k + V\,d\,I)$	lr, momentum, wd, tope c, escala de tabla s, ρ_{out}/ρ_{hid}, pasos NS k	[66]
AdaMuon	Ortogonalidad + adaptativo	2 (m, v)	$\mathcal{O}(n \cdot k)$	lr, momentum, eps, wd, pasos NS k	[104]
Dion3	Ortogonalidad fraccional	1 (m); + v por filas con NorMuon	$\mathcal{O}(r^2(m+r))$, $r = \lceil fn \rceil$	lr, momentum, wd, fracción f, pasos NS k, β_2 (NorMuon)	[4]
Adaptativos de Bajo Rango y Adaptividad Ajustable					
APOLLO	Adaptativo de bajo rango	2 bajo rango + proy	$\mathcal{O}(nr)$	lr, rango r, intervalo de actualización, escala, betas, eps, wd	[134]
APOLLO-Mini	Adaptativo de bajo rango	2 rango-1 + proy	$\mathcal{O}(n)$	lr, intervalo de actualización, escala, betas, eps, wd	[134]

continúa en la página siguiente

Optimizador	Familia	Búferes de Estado	Costo por Paso	Hiperparámetros Clave	Referencia
Q-APOLLO	Bajo rango cuantizado	2 cuantizados + proy	$\mathcal{O}(nr)$	lr, rango r, intervalo de actualización, escala, bits de cuantización, betas, eps, wd	[134]
Anon	Adaptividad ajustable	3 (m, v, fixed_ lr)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd, γ	[6]

Cuadro 8: Catálogo completo de optimizadores. n = número de parámetros, d = dimensionalidad del tensor, V = tamaño del vocabulario, r = dimensión de rango bajo, k = número de iteraciones de Newton–Schulz, I = número de iteraciones de bisección por fila. Búferes de estado enumera el número de tensores de estado del optimizador mantenidos por grupo de parámetros.

3.3. Variantes de Normalización

La Tabla 9 resume los seis métodos de normalización disponibles a través del campo de esquema `norm_type`, desde enfoques clásicos basados en estadísticas hasta transformaciones acotadas sin normalización y RMSNorm sin pesos con ejecución norm-then-project fusionada.

Método	Fórmula	Estadísticas Necesarias	Referencia
LayerNorm	$\gamma_i(x_i - \mu)/\sqrt{\sigma^2 + \epsilon} + \beta_i$	media + varianza	[7]
RMSNorm	$\gamma_i x_i / \sqrt{\frac{1}{d}\sum_j x_j^2 + \epsilon}$	solo RMS	[129]
pRMSNorm	$\gamma_i x_i / \sqrt{\frac{1}{k}\sum_{j \le k} x_j^2 + \epsilon}$	RMS parcial (p %)	[129]
Dynamic Tanh (DyT)	$\tanh(\alpha x)$	ninguna	[135]
Dynamic Erf (Derf)	$\mathrm{erf}(\alpha x + s)$	ninguna	[17]
FlashNorm	$x_i / \sqrt{\frac{1}{d}\sum_j x_j^2 + \epsilon}$	solo RMS (sin pesos)	[40]

Cuadro 9: Variantes de normalización. d = dimensión oculta, $k = \lceil d \cdot p \rceil$ para razón parcial p, α y s son parámetros aprendibles. FlashNorm es sin pesos (sin parámetros aprendibles); la escala por dimensión se pliega en la capa lineal subsecuente según la Prop. 1 de [40].

3.4. Conexiones Residuales

La Tabla 10 compara los esquemas de conexión residual disponibles en el código base: el residual de identidad estándar, las cuatro estrategias de Residuales de Atención (AttnRes) [110] y el flujo residual de n streams restringido por variedad (mHC) [120]. L es la profundidad lógica, C la dimensión oculta, n el factor de expansión del flujo, $t_{\text{máx}}$ el número de iteraciones Sinkhorn–Knopp, α los escalares de compuerta de mHC y $\mathbf{w}_l$ el pseudo-query por capa de AttnRes (inicializado a cero). HC (Hyper-Connexions sin restricción) se lista como referencia pero *no* está expuesto en este código base.

Esquema	Actualización del Flujo	Restricción	Referencia
Residual estándar	$x_{l+1} = x_l + F(x_l, W_l)$	Mapeo de identidad (por construcción)	[50]
Sin residual (experimental)	$x_{l+1} = F(x_l, W_l)$	Sin ruta skip	—
Full AttnRes	$x_{l+1} = \sum_{i=0}^{l} \mathrm{softmax}_i(\mathbf{w}_l^\top \mathrm{RMSNorm}(v_i))\, v_i$	$\mathbf{w}_l$ aprendido, init a cero; RMSNorm en keys; $L \cdot C$ parámetros	[110]
Block AttnRes	$x_{l+1} = \sum_{m=0}^{n} \mathrm{softmax}_m(\mathbf{w}_l^\top \mathrm{RMSNorm}(v_m))\, v_m$	$\mathbf{w}_l$ aprendido, init a cero; RMSNorm en keys; N reps de bloque + suma parcial; $L \cdot C$ parámetros	[110]
Hyper-Connexions (HC)	$x_{l+1} = H_l^{\mathrm{res}} x_l + (H_l^{\mathrm{post}})^\top F(H_l^{\mathrm{pre}} x_l, W_l)$	Sin restricción	[120]
mHC (este código base)	misma actualización que HC	H_l^{res} doblemente estocástica vía Sinkhorn–Knopp ($t_{\mathrm{máx}} = 20$), $H^{\mathrm{pre}} = \sigma(\cdot)$, $H^{\mathrm{post}} = 2\sigma(\cdot)$, init α en 0,01	[120]
Hyperloop (este código base)	$y_t^{(l+1)} = H_l^{\mathrm{res}} y_t^{(l)} + H_l^{\mathrm{post}} \otimes (F(H_l^{\mathrm{pre}} y_t^{(l)}) + e_l)$	HC a nivel de bucle sobre un bloque intermedio en bucle de ciclo medio; $\{W_l, b_l, \alpha_l, e_l\}$ por bucle; expansión por copia/promedio; H^{res} diagonal (por defecto), Sinkhorn o identidad	[128]

Cuadro 10: Esquemas de conexión residual. $x_l \in \mathbb{R}^C$ (estándar / AttnRes / sin residual) o $\mathbb{R}^{n \times C}$ (mHC) es el flujo en la capa l, F la función de capa y $H^{\mathrm{pre}}, H^{\mathrm{post}}, H^{\mathrm{res}}$ los mapeos aprendidos de mHC. Las estrategias AttnRes se controlan con `model.residuals.type`; mHC con `model.mhc`. Detalles completos en los anexos de mHC y AttnRes.

3.5. Funciones de Activación

La Tabla 11 cataloga las 43 funciones de activación disponibles a través del campo de esquema `ffn_activation` (40 elementales más 3 variantes de FFN con compuerta), agrupadas por familia. "Aprend." marca las activaciones con parámetros entrenables; las fórmulas completas están en el Anexo 8.

Nombre	Familia	Fórmula	Aprend.	Referencia
Clásicas / Sigmoid–Tanh (12)				
silu / swish	Clásica	$x\,\sigma(\beta x)$	no / β	[96]
gelu	Clásica	$x\,\Phi(x)$	no	[52]
gelu_tanh	Clásica	$\frac{x}{2}\left(1 + \tanh\sqrt{\frac{2}{\pi}}(x{+}0{,}044715x^3)\right)$	no	[52]
relu	Clásica	$\max(0, x)$	no	[85]
sigmoid	Clásica	$1/(1 + e^{-x})$	no	[68]
tanh	Clásica	$(e^x - e^{-x})/(e^x + e^{-x})$	no	[68]
arctan	Clásica	$\arctan(x)$	no	[68]
softsign / elliott	Clásica	$x/(1 + \lvert x\rvert)$	no	[68]
identity	Clásica	x	no	(interno)
softplus	Clásica	$\log(1 + e^x)$	no	[37]
mish	Clásica	$x \tanh(\log(1 + e^x))$	no	[83]
Familia rectificada (12)				
leaky_relu	Rectificada	$\max(0, x) + a \min(0, x)$, a=0,01	no	[79]
relu6	Rectificada	$\min(\max(0, x), 6)$	no	[55]
hardswish	Rectificada	$x\,\mathrm{ReLU6}(x{+}3)/6$	no	[55]

continúa en la próxima página

Nombre	Familia	Fórmula	Aprend.	Referencia
prelu	Rectificada	$\max(0,x) + \mathbf{p} \odot \min(0,x)$	**sí**	[49]
abs_relu	Rectificada	$\max(0,x) - \max(0,-x)$	no	[30]
nl_relu	Rectificada	$\beta \log(1 + \max(0,x))$	no	[30]
brelu	Rectificada	$\min(\max(0,x), t)$	no	[30]
vrelu	Rectificada	$\lvert x \rvert$	no	[30]
hexpo	Rectificada	$a \max(0,x) - c \max(0,-x)$	no	[30]
ptanh	Rectificada	$\max(0, \tanh(x))$	no	[30]
dis_relu	Rectificada	$\max(0, x - \delta)$	no	[30]
lisht	Rectificada	$x \tanh(x)$	no	[99]
Familia exponencial / ELU (12)				
elu	Exponencial	x si $x \geq 0$ si no $\alpha(e^x - 1)$	no	[21]
selu	Exponencial	$\lambda\, \mathrm{ELU}_{\alpha'}(x)$	no	[64]
celu	Exponencial	x si $x \geq 0$ si no $\alpha(e^{x/\alpha} - 1)$	no	[8]
pelu	Exponencial	$\frac{\alpha}{\beta} x$ / $\alpha(e^{x/\beta} - 1)$	**sí**	[30]
mpelu	Exponencial	ELU + α, β aprendibles	**sí**	[30]
felu / eelu / pdelu / preu	Exponencial	variantes ELU c/ params aprendibles	**sí**	[30]
softexp	Exponencial	interpolación exp/lineal/log	**sí**	[30]
elish / hardelish	Exponencial	Swish $\cup$ ramas ELU con compuerta	no	[10]
Aprendibles / Adaptativas (4)				
swish_trainable	Aprendible	$x\, \sigma(\beta x)$, β entrenable	**sí**	[96]
maxout	Aprendible	$\max_i(W_i x + b_i)$	**sí**	[39]
raf	Aprendible	$P(x)/Q(x)$ Padé$(5,4)$, versión A	**sí**	[34]
Variantes FFN con compuerta (3)				
swiglu	Compuerta	$\mathrm{SiLU}(xW_g) \odot (xW_u)\, W_d$	**sí**	[101]
geglu	Compuerta	$\mathrm{GELU}(xW_g) \odot (xW_u)\, W_d$	**sí**	[101]
reglu	Compuerta	$\mathrm{ReLU}(xW_g) \odot (xW_u)\, W_d$	**sí**	[101]

Cuadro 11: Catálogo de funciones de activación. "Aprend." indica la presencia de parámetros entrenables. RAF = Rational Activation Function (razón de Padé aprendible). σ es la sigmoide logística.

3.6. Variantes de Embedding

La Tabla 12 cataloga las codificaciones posicionales y optimizaciones estructurales de embedding soportadas por el código base. Las 12 codificaciones posicionales se configuran mediante el campo `positional_encoding` de todo el modelo (ver Anexo 12 para las formulaciones completas); las dos optimizaciones estructurales se controlan con sus propias banderas.

Método	Descripción	Referencia
Codificaciones Posicionales (12)		
RoPE	Embedding Posicional Rotatorio; rotación basada en frecuencia de Q/K, codificando posición relativa mediante geometría de producto interno	[107]
HoPE	Codificación Posicional Rotatoria Hiperbólica; rotación inspirada en Lorentz usando funciones hiperbólicas; decaimiento monótono de atención con la distancia; RoPE es un caso especial	[23]
NoPE	Sin Codificación Posicional; el modelo aprende posición implícitamente de la máscara causal; supera a PE explícita en generalización de longitud	[59]
ALiBi	Atención con Sesgos Lineales; penalización estática proporcional a distancia en puntuaciones de atención; habilita extrapolación de longitud	[93]
BAM	Mecanismo de Atención Bayesiana; sesgo de posición relativa con Gaussiana Generalizada, forma (β) y escala (α) aprendibles por cabeza; recupera ALiBi en $\beta = 1$; reescalado SSMax vía `use_ssmax`	[13]

continúa en la página siguiente

Método	Descripción	Referencia
PaPE	Codificación Posicional Parabólica; funciones base parabólicas concatenadas a Q/K; centrada en visión, n-D, extrapolatable	[86]
PaPE-Eficiente	PaPE en PyTorch puro; sin kernels Triton personalizados, misma matemática	[86]
PaPE-RI	PaPE invariante a rotación (PaPE-RI)	[86]
Sinusoidal Absoluta	Sinusoidal fijo añadido a embeddings (Transformer original)	[113]
Sinusoidal Rotatoria	Frecuencias sinusoidales como matriz de rotación en Q/K	[113]
Absoluta Aprendida	Matriz de parámetros aprendible añadida a embeddings	[113]
None	Módulo nulo; ninguna información posicional inyectada	—
Optimizaciones Estructurales de Embedding (4)		
Embeddings Factorizados	Descompone la matriz de embedding $V \times H$ en factores $V \times R$ y $R \times H$; reduce parámetros de $\mathcal{O}(VH)$ a $\mathcal{O}(VR + RH)$; controlado por `factorized_embedding_dim`	(interno)
Convolución de Embedding	Conv1d depthwise aplicada sobre el flujo de embeddings antes del primer bloque transformer; proporciona filtrado ligero de contexto local; controlado por `embedding_conv_kernel`	(interno)
Embeddings Dinámicos (EELBERT)	Reemplaza la tabla $V \times H$ por un hash de agrupación de n-gramas calculado al vuelo sobre las cadenas de tokens; congelado, sin parámetros de embedding entrenables; controlado por el bloque `embedding.dynamic.*` (Anexo 21)	[22]
Frontend de audio	Tramas log-mel/espectrograma/raw a través de dos etapas Conv2d con stride 2 y una proyección lineal (reducción de tramas 4×); lado de entrada de `frankenstein_audio`; controlado por el bloque `audio.*` (Anexo 20)	[45]

Cuadro 12: Variantes de embedding. V = tamaño de vocabulario, H = tamaño oculto, R = dimensión intermedia factorizada. Las primeras 12 filas son codificaciones posicionales (Anexo 12); las últimas cuatro son optimizaciones estructurales de embedding, incluido el frontend de entrada de audio.

4. Familias de Optimizadores

4.1. La Evolución de la Optimización en Redes Neuronales

El estudio de optimizadores enmarca la optimización de transformadores como una respuesta a tres presiones estructurales: paisajes de pérdida no convexos, heterogeneidad severa de curvatura entre bloques de parámetros y el costo de memoria de almacenar el estado del optimizador para modelos muy grandes. El informe sostiene que el campo se ha diversificado en varias trayectorias: líneas base adaptativas de primer orden, métodos de reducción de varianza, métodos eficientes en memoria, precondicionadores estructurados de segundo orden, métodos sin programación de tasa de aprendizaje y actualizaciones orientadas a ortogonalidad.

4.2. Línea Base Estándar y Optimizadores Adaptativos

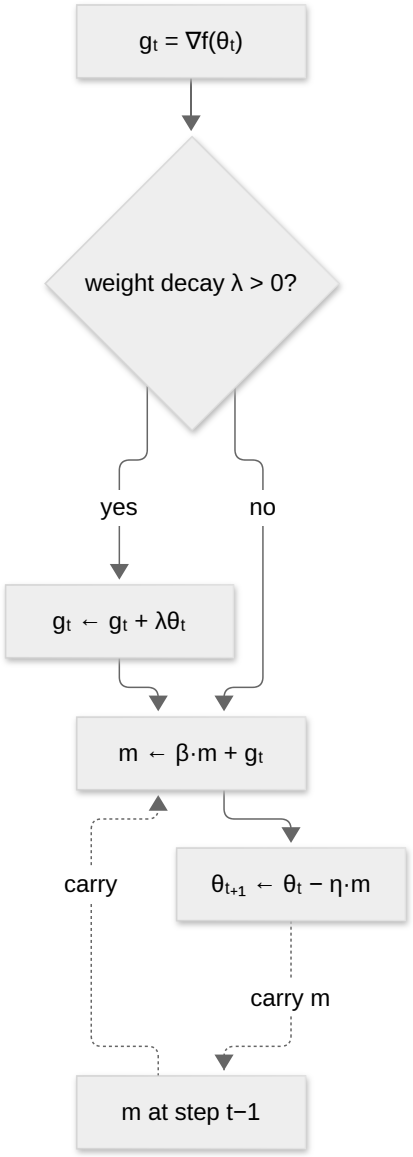


Figura 12: Flujo por paso de SGD con momentum: decaimiento L2 opcional, acumulación exponencial de momentum y actualización de parámetros con paso fijo.

SGD con Momentum. La actualización clásica acumula un búfer de momentum y luego aplica una tasa de aprendizaje fija. En cada iteración t, el algoritmo mantiene un promedio móvil exponencial m_t de los gradientes pasados:

$$m_t = \beta m_{t-1} + g_t \tag{1}$$

$$\theta_{t+1} = \theta_t - \eta m_t \tag{2}$$

donde $g_t = \nabla f(\theta_t)$, $\beta \in [0{,}8, 0{,}99]$ controla la decadencia del momentum y η es la tasa de aprendizaje fija. El término de momentum actúa como velocidad: acelera el movimiento en direcciones consistentes mientras amortigua oscilaciones en direcciones variables.

Algorithm 1 SGD con Momentum

Require: Parámetros iniciales θ_0, tasa de aprendizaje η, coeficiente de momentum β, decadencia de peso λ

Ensure: Parámetros actualizados θ_T

Inicializar: búfer de momentum $m \leftarrow 0$
for $t = 0, 1, 2, \ldots, T-1$ **do**
    Calcular gradiente: $g_t \leftarrow \nabla f(\theta_t)$
    if decadencia_de_peso > 0 **then**
        $g_t \leftarrow g_t + \lambda\theta_t$ ▷ Regularización L2
    end if
    Actualizar momentum: $m \leftarrow \beta \cdot m + g_t$
    Actualizar parámetros: $\theta_{t+1} \leftarrow \theta_t - \eta \cdot m$
end forreturn θ_T

Contras.

- Escasa robustez frente a la heterogeneidad de curvatura: una única tasa global no puede adaptarse a todos los grupos de parámetros.
- Fuerte dependencia de programaciones de tasa de aprendizaje cuidadosamente diseñadas.

Ventajas.

- Baja sobrecarga de memoria: un único búfer de momentum por parámetro.
- Fuerte generalización cuando se ajusta cuidadosamente, y una actualización simple de implementar.

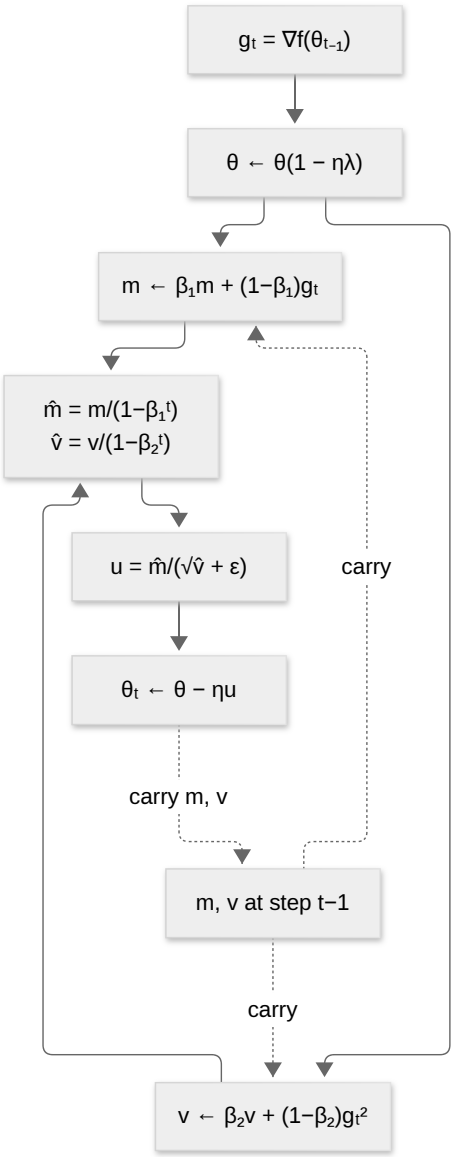


Figura 13: Flujo por paso de AdamW: decaimiento de pesos desacoplado, EMAs de primer y segundo momento, corrección de sesgo y actualización adaptativa elemento a elemento.

Adam y AdamW. Adam rastrea promedios móviles exponenciales tanto del primer momento (media) como del segundo momento (varianza no centrada) para lograr tasas de aprendizaje adaptativas elemento a elemento. La regla de actualización es:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{3}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{4}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad \text{(corrección de sesgo)} \tag{5}$$

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \right) \tag{6}$$

AdamW introduce una modificación crucial: la decadencia de peso se aplica directamente a los parámetros (desacoplada de los gradientes): $\theta_t \leftarrow \theta_t(1 - \eta\lambda)$ en lugar de añadir $\lambda\theta_t$ al gradiente. Este desacoplamiento evita que el escalado adaptativo interfiera con la fuerza de la regularización.

Algorithm 2 AdamW (Adam con Decadencia de Peso Desacoplada)

Require: Parámetros iniciales θ_0, tasa de aprendizaje η, tasas de decadencia exponencial $\beta_1, \beta_2 \in [0, 1)$

Require: Constante de momentum ϵ, decadencia de peso λ

Ensure: Parámetros actualizados θ_T

Inicializar: primer momento $m \leftarrow 0$, segundo momento $v \leftarrow 0$, contador de pasos $t \leftarrow 0$

for $t = 1, 2, \ldots, T$ **do**

  Calcular gradiente: $g_t \leftarrow \nabla f(\theta_{t-1})$

  Decadencia de peso (desacoplada): $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$

  Actualizar momentos: $m \leftarrow \beta_1 m + (1 - \beta_1) g_t$

  $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$

  Corrección de sesgo: $\hat{m} \leftarrow m/(1 - \beta_1^t)$, $\hat{v} \leftarrow v/(1 - \beta_2^t)$

  Actualizar parámetros: $\theta_t \leftarrow \theta_{t-1} - \eta(\hat{m}/(\sqrt{\hat{v}} + \epsilon))$

end forreturn θ_T

Contras.

- Almacena tensores de primer y segundo momento para cada parámetro, duplicando la memoria del estado del optimizador respecto a SGD.
- El escalado adaptativo elemento a elemento puede interferir con la regularización si el decaimiento de pesos no se desacopla.

Ventajas.

- Línea base práctica para el ajuste fino de transformadores: convergencia rápida y tolerancia a la variación de hiperparámetros.
- El decaimiento de pesos desacoplado mantiene la fuerza de la regularización independiente del escalado adaptativo.

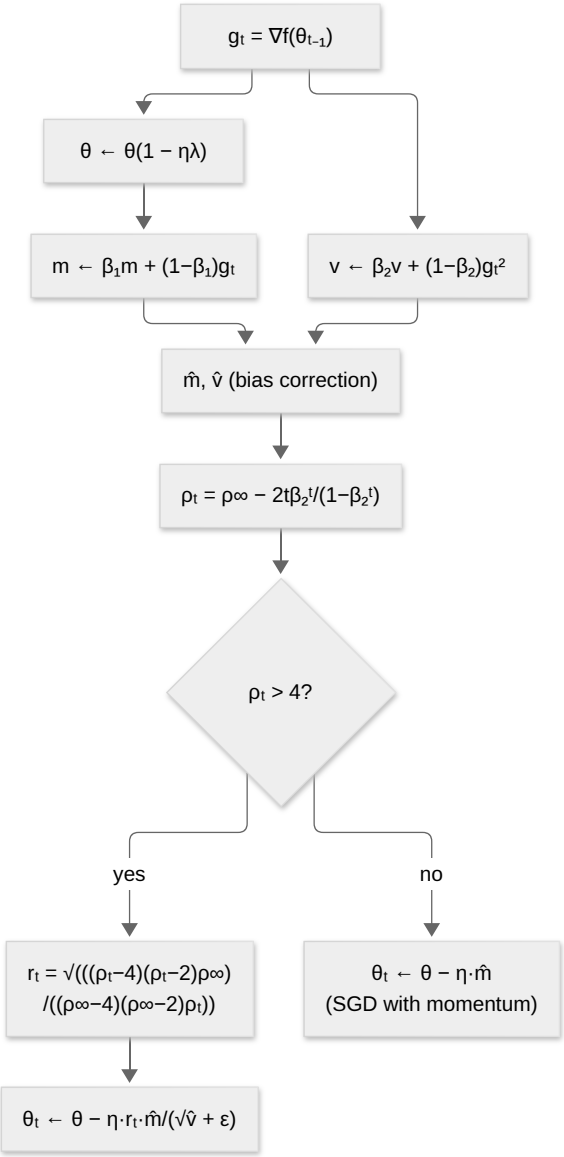


Figura 14: Flujo por paso de RAdam: rectificación por ventana SMA con retroceso a SGD con momentum en los primeros pasos.

RAdam (Adam Rectificado). RAdam aborda la inestabilidad de Adam en el entrenamiento temprano rectificando dinámicamente la tasa de aprendizaje adaptativa. La observación clave es que el segundo momento v_t tiene una varianza muy alta en los primeros pasos, causando un escalado adaptativo poco fiable. RAdam calcula la longitud efectiva de la ventana de promedio móvil simple (SMA):

$$\rho_t = \rho_\infty - \frac{2t\beta_2^t}{1-\beta_2^t}, \quad \text{donde} \quad \rho_\infty = \frac{2}{1-\beta_2} - 1$$

Cuando $\rho_t > 4$ (suficientes muestras para la estimación de varianza), RAdam aplica el escalado adaptativo con un término de rectificación:

$$r_t = \sqrt{\frac{(\rho_t - 4)(\rho_t - 2)\rho_\infty}{(\rho_\infty - 4)(\rho_\infty - 2)\rho_t}}, \quad \theta_{t+1} = \theta_t - \eta r_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Cuando $\rho_t \leq 4$, RAdam retrocede a SGD con momentum: $\theta_{t+1} = \theta_t - \eta \hat{m}_t$.

Algorithm 3 RAdam (Adam Rectificado)

Require: Parámetros iniciales θ_0, tasa de aprendizaje η, β_1, β_2, decadencia de peso λ
Ensure: Parámetros actualizados θ_T

Inicializar: $m \leftarrow 0$, $v \leftarrow 0$, $t \leftarrow 0$
Calcular: $\rho_\infty \leftarrow 2/(1-\beta_2) - 1$
for $t = 1, 2, \ldots, T$ **do**
    $g_t \leftarrow \nabla f(\theta_{t-1})$
    if decadencia_de_peso > 0 **then**
        $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
    end if
    $m \leftarrow \beta_1 m + (1-\beta_1) g_t$
    $v \leftarrow \beta_2 v + (1-\beta_2) g_t^2$
    $\hat{m} \leftarrow m/(1-\beta_1^t)$, $\hat{v} \leftarrow v/(1-\beta_2^t)$
    $\rho_t \leftarrow \rho_\infty - 2t\beta_2^t/(1-\beta_2^t)$
    if $\rho_t > 4$ **then**
        $r_t \leftarrow \sqrt{((\rho_t - 4)(\rho_t - 2)\rho_\infty)/((\rho_\infty - 4)(\rho_\infty - 2)\rho_t)}$
        $\theta_t \leftarrow \theta_{t-1} - \eta r_t \hat{m}/(\sqrt{\hat{v}} + \epsilon)$
    else
        $\theta_t \leftarrow \theta_{t-1} - \eta\hat{m}$ ▷ SGD con momentum
    end if
end forreturn θ_T

Contras.

- Añade contabilidad de rectificación sobre el estado de momentos de Adam.
- El beneficio se reduce cuando ya se usa una programación de calentamiento bien ajustada.

Ventajas.

- Elimina la necesidad de calentamiento manual al rectificar la tasa adaptativa.
- Retrocede con elegancia a SGD con momentum mientras las estimaciones de varianza no son fiables, mejorando la robustez.

4.3. Momentum Avanzado y Reducción de Varianza (2024–2025)

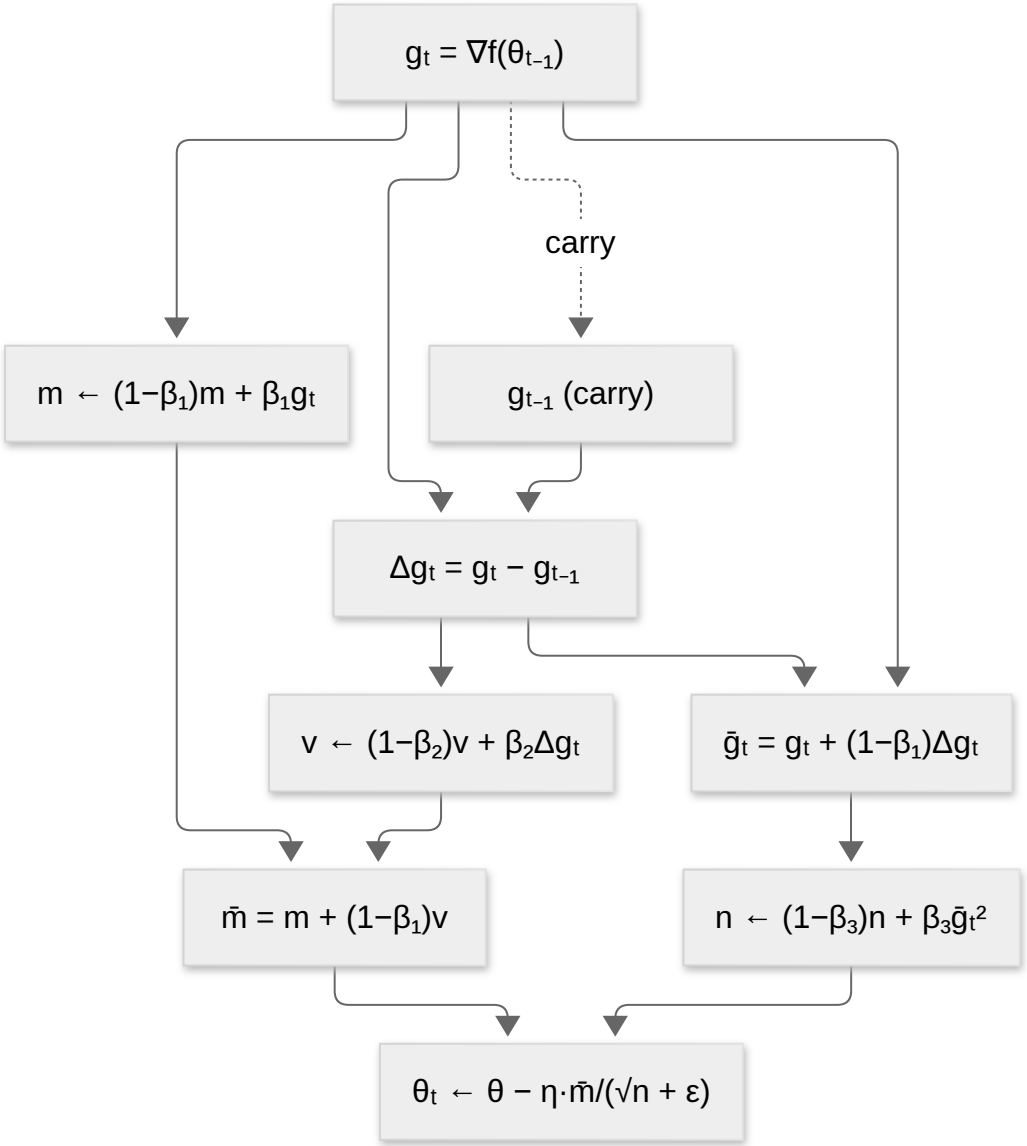


Figura 15: Flujo por paso de Adan: estimación de momentum de Nesterov a partir de la diferencia de gradientes, con tres búferes de estado y un paso de momentum combinado.

Adan (Momentum Nesterov Adaptativo). Adan reformula la aceleración de Nesterov sin el cálculo de gradiente adicional requerido por el SGD Nesterov clásico. El algoritmo mantiene tres búferes de momentum:

$$m_t = (1-\beta_1)m_{t-1} + \beta_1 g_t \quad \text{(primer momento)} \tag{7}$$

$$v_t = (1-\beta_2)v_{t-1} + \beta_2(g_t - g_{t-1}) \quad \text{(velocidad/diferencia de gradientes)} \tag{8}$$

$$n_t = (1-\beta_3)n_{t-1} + \beta_3[g_t + (1-\beta_1)(g_t - g_{t-1})]^2 \quad \text{(segundo momento Nesterov)} \tag{9}$$

El término de **Estimación de Momentum Nesterov** (NME) $\bar{g}_t = g_t + (1-\beta_1)(g_t - g_{t-1})$ estima el gradiente en una posición futura sin evaluarlo. La actualización combina el momentum con la velocidad:

$$\bar{m}_t = m_t + (1-\beta_1)v_t, \quad \theta_{t+1} = \theta_t - \eta \frac{\bar{m}_t}{\sqrt{n_t} + \epsilon}$$

Algorithm 4 Adan (Momentum Nesterov Adaptativo)

Require: Parámetros iniciales θ_0, tasa de aprendizaje η, $\beta_1, \beta_2, \beta_3 \in (0, 1)$
Ensure: Parámetros actualizados θ_T

Inicializar: $m \leftarrow 0$, $v \leftarrow 0$, $n \leftarrow 0$, $g_{-1} \leftarrow 0$ (gradiente anterior)
for $t = 1, 2, \ldots, T$ **do**
    $g_t \leftarrow \nabla f(\theta_{t-1})$
    $m \leftarrow (1 - \beta_1)m + \beta_1 g_t$
    $\Delta g_t \leftarrow g_t - g_{t-1}$
    $v \leftarrow (1 - \beta_2)v + \beta_2 \Delta g_t$
    Estimación Nesterov: $\bar{g}_t \leftarrow g_t + (1 - \beta_1)\Delta g_t$
    $n \leftarrow (1 - \beta_3)n + \beta_3 \bar{g}_t^2$
    Momentum combinado: $\bar{m} \leftarrow m + (1 - \beta_1)v$
    $\theta_t \leftarrow \theta_{t-1} - \eta(\bar{m}/(\sqrt{n} + \epsilon))$
    $g_{t-1} \leftarrow g_t$
end forreturn θ_T

Contras.

- Tres búferes tipo momentum aumentan la sobrecarga de memoria respecto a Adam.
- Contabilidad adicional de la diferencia de gradientes en cada paso.

Ventajas.

- Aceleración estilo Nesterov sin una evaluación adicional del gradiente.
- Convergencia rápida en arquitecturas diversas (CNN, GAN, transformadores).

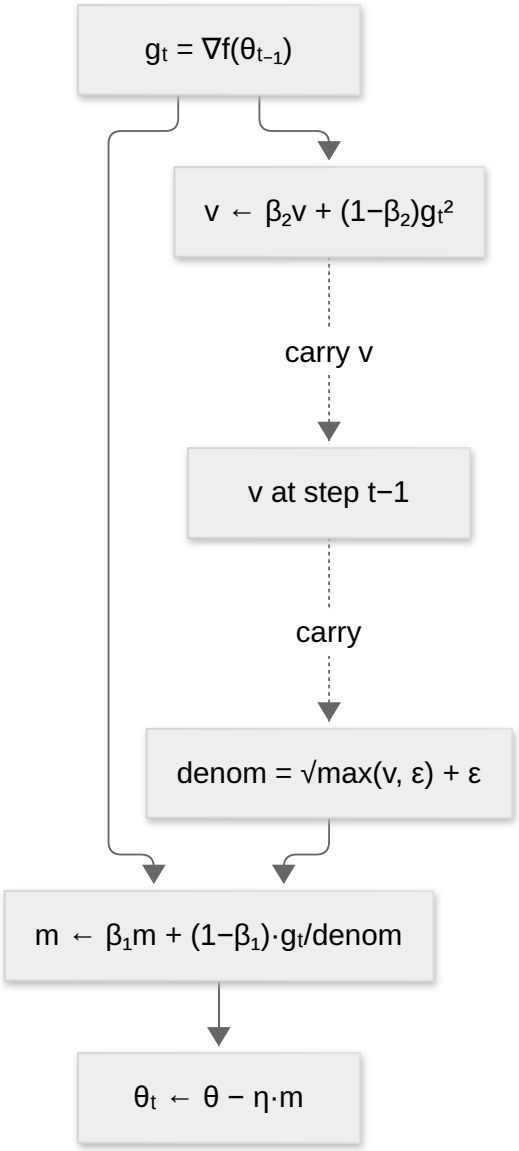


Figura 16: Flujo por paso de ADOPT: el segundo momento del paso anterior normaliza el gradiente actual antes de actualizar el primer momento.

ADOPT (Adam con Ajuste Óptimo Podado). ADOPT corrige un problema teórico fundamental en Adam: el gradiente aparece tanto en la estimación del primer momento como en la del segundo momento, creando circularidad. ADOPT **desacopla** utilizando el segundo momento del paso anterior para el denominador:

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \tag{10}$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) \frac{g_t}{\sqrt{v_{t-1}} + \epsilon} \quad (\text{usa } v_{t-1}, \text{ no } v_t) \tag{11}$$

$$\theta_{t+1} = \theta_t - \eta m_t \tag{12}$$

Algorithm 5 ADOPT (Adam con Ajuste Óptimo Podado)

Require: Parámetros iniciales θ_0, tasa de aprendizaje η, β_1, β_2, tolerancia ϵ
Ensure: Parámetros actualizados θ_T

Inicializar: $m \leftarrow 0$, $v \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
  $g_t \leftarrow \nabla f(\theta_{t-1})$
  if decadencia_de_peso > 0 **then**
    $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
  end if
  Usar segundo momento **anterior**: denom $\leftarrow \sqrt{\text{máx}(v, \epsilon)} + \epsilon$
  Actualizar primer momento usando varianza anterior: $m \leftarrow \beta_1 m + (1 - \beta_1)(g_t/\text{denom})$
  Actualizar segundo momento con gradiente **actual**: $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$
  $\theta_t \leftarrow \theta_{t-1} - \eta m$
end forreturn θ_T

Contras.

- Requiere conservar el segundo momento anterior como denominador explícito.
- Las ganancias empíricas dependen de la arquitectura y del ajuste.

Ventajas.

- Elimina la circularidad del gradiente de Adam y alcanza la tasa óptima $O(1/\sqrt{T})$ con **cualquier** $\beta_2 \in (0, 1)$, sin suposiciones de ruido acotado.
- Reemplazo directo de Adam con rendimiento empírico comparable o superior en visión, PLN, RL y modelado generativo.

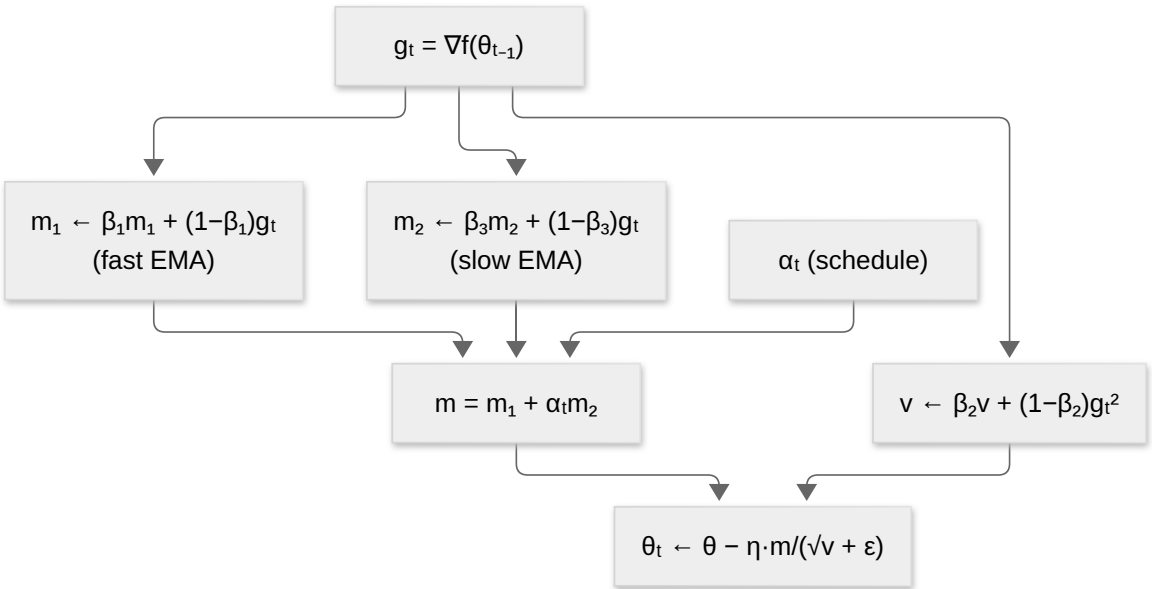


Figura 17: Flujo por paso de AdEMAMix: EMAs rápida y lenta en paralelo, mezcladas con un peso programado α_t antes de la actualización adaptativa.

AdEMAMix (Mezcla de Promedios Móviles Exponenciales). AdEMAMix reemplaza el EMA único de gradientes de Adam por una **mezcla de dos EMA**: uno de decadencia rápida y otro de decadencia lenta. Esto aborda la observación de que los gradientes siguen siendo informativos durante decenas de miles de pasos, no solo cientos:

$$m_{1,t} = \beta_1 m_{1,t-1} + (1-\beta_1) g_t \quad \text{(EMA rápido, } \beta_1 \approx 0{,}9) \tag{13}$$

$$m_{2,t} = \beta_3 m_{2,t-1} + (1-\beta_3) g_t \quad \text{(EMA lento, } \beta_3 \approx 0{,}9999) \tag{14}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \tag{15}$$

$$m_t = m_{1,t} + \alpha_t m_{2,t} \quad \text{(mezcla con peso programado } \alpha_t) \tag{16}$$

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \tag{17}$$

El peso de la mezcla α_t típicamente aumenta durante el entrenamiento, permitiendo que el EMA rápido proporcione adaptación inmediata mientras que el EMA lento acumula correlaciones de gradiente a largo plazo.

Algorithm 6 AdEMAMix (Mezcla de Promedios Móviles Exponenciales)

Require: Parámetros iniciales θ_0, tasa de aprendizaje η, β_1 (rápido), β_3 (lento), β_2 (segundo momento)
Require: Peso de mezcla α_t (típicamente creciente), tolerancia ϵ
Ensure: Parámetros actualizados θ_T
Inicializar: $m_1 \leftarrow 0$ (EMA rápido), $m_2 \leftarrow 0$ (EMA lento), $v \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
  $g_t \leftarrow \nabla f(\theta_{t-1})$
  if decadencia_de_peso > 0 **then**
    $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
  end if
  $m_1 \leftarrow \beta_1 m_1 + (1-\beta_1) g_t$ ▷ EMA rápido
  $m_2 \leftarrow \beta_3 m_2 + (1-\beta_3) g_t$ ▷ EMA lento
  $v \leftarrow \beta_2 v + (1-\beta_2) g_t^2$
  $m_t \leftarrow m_1 + \alpha_t m_2$ ▷ Mezcla
  $\theta_t \leftarrow \theta_{t-1} - \eta(m_t/(\sqrt{v} + \epsilon))$
end forreturn θ_T

Contras.

- Dos EMA de gradiente más el segundo momento casi triplican el estado del optimizador de Adam.
- La programación de mezcla α_t añade un hiperparámetro que ajustar.

Ventajas.

- Explota información de gradiente que sigue siendo informativa durante decenas de miles de pasos.
- Un modelo de 1.3B en 101B tokens iguala a AdamW en 197B tokens (ganancia del 95 % en eficiencia de datos), lo que sugiere menor olvido.

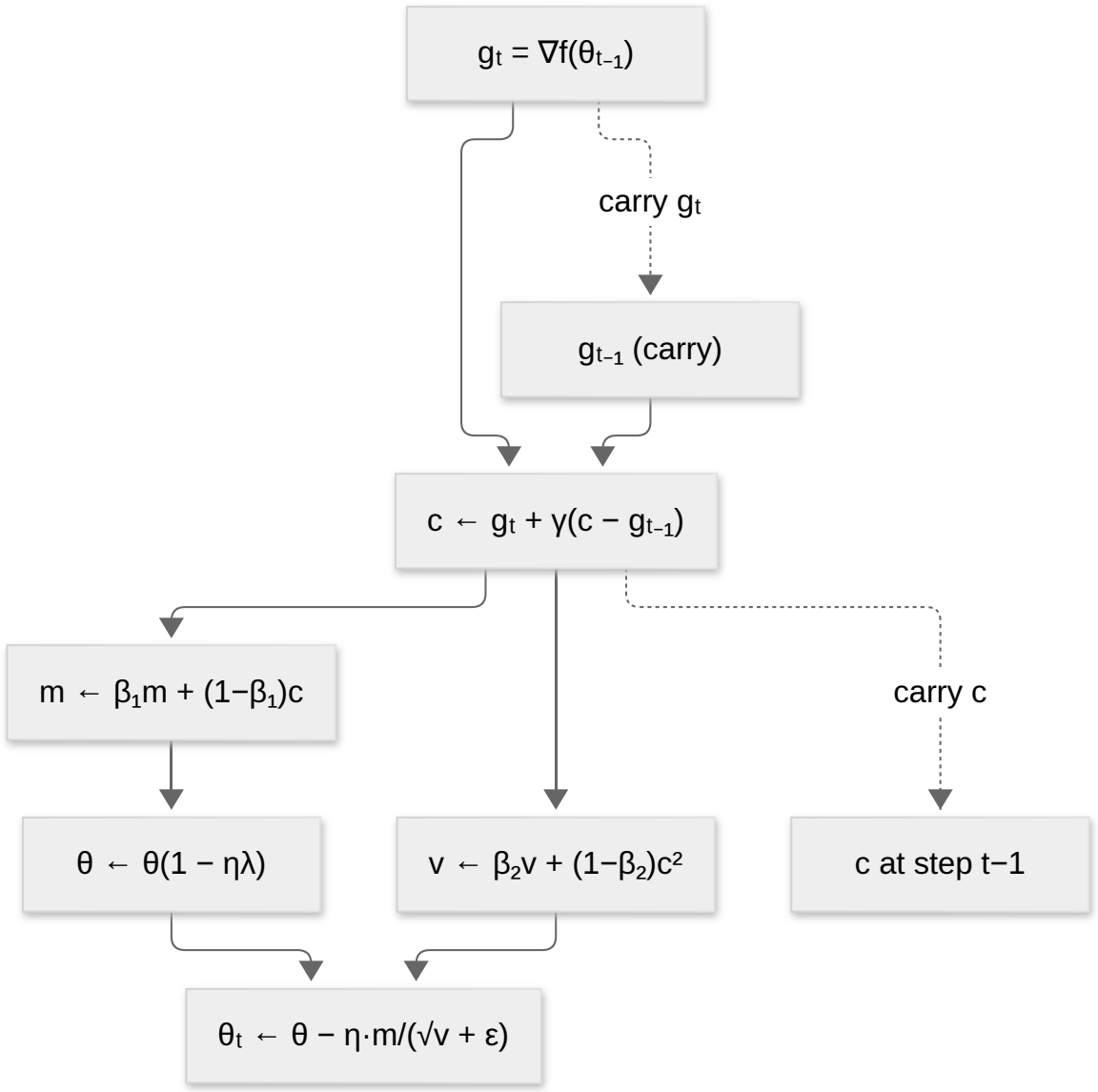


Figura 18: Flujo por paso de MARS: una estimación recursiva de gradiente con varianza reducida que alimenta los momentos de primer y segundo orden estilo Adam.

MARS (Haciendo Brillar las Tasas de Aprendizaje Adaptativas). MARS combina métodos de gradiente precondicionado (ej., AdamW) con **reducción de varianza** mediante momentum recursivo estocástico escalado. La innovación central es una estimación del gradiente con varianza reducida:

$$c_t = g_t + \gamma(c_{t-1} - g_{t-1}) \quad \text{(estimación recursiva tipo SVRG)} \tag{18}$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)c_t \tag{19}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)c_t^2 \tag{20}$$

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \tag{21}$$

donde $\gamma \in [0{,}01, 0{,}1]$ controla cuánta información de gradiente histórico se retiene. El gradiente con varianza reducida c_t actúa como un filtro de ruido implícito, amplificando las direcciones de señal consistentes y amortiguando el ruido contradictorio.

Algorithm 7 MARS (Haciendo Brillar las Tasas de Aprendizaje Adaptativas)

Require: Parámetros iniciales θ_0, tasa de aprendizaje η, β_1, β_2, decadencia de peso λ
Require: Coeficiente de reducción de varianza $\gamma \in [0{,}01, 0{,}1]$
Ensure: Parámetros actualizados θ_T

Inicializar: $m \leftarrow 0$, $v \leftarrow 0$, $c \leftarrow 0$ (gradiente con varianza reducida), $g_{-1} \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
    $g_t \leftarrow \nabla f(\theta_{t-1})$
    Reducción de varianza: $c \leftarrow g_t + \gamma(c - g_{t-1})$
    $m \leftarrow \beta_1 m + (1 - \beta_1)c$
    $v \leftarrow \beta_2 v + (1 - \beta_2)c^2$
    if decadencia_de_peso > 0 **then**
        $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
    end if
    $\theta_t \leftarrow \theta_{t-1} - \eta(m/(\sqrt{v} + \epsilon))$
    $g_{t-1} \leftarrow g_t$
end forreturn θ_T

Contras.

- Almacena el gradiente con varianza reducida y el gradiente anterior además de los momentos de Adam.
- El coeficiente de reducción de varianza $\gamma \in [0{,}01, 0{,}1]$ debe ajustarse.

Ventajas.

- Actúa como un filtro de ruido implícito, amplificando direcciones consistentes y amortiguando el ruido contradictorio.
- MARS-AdamW supera consistentemente a AdamW por márgenes significativos en el preentrenamiento de GPT-2.

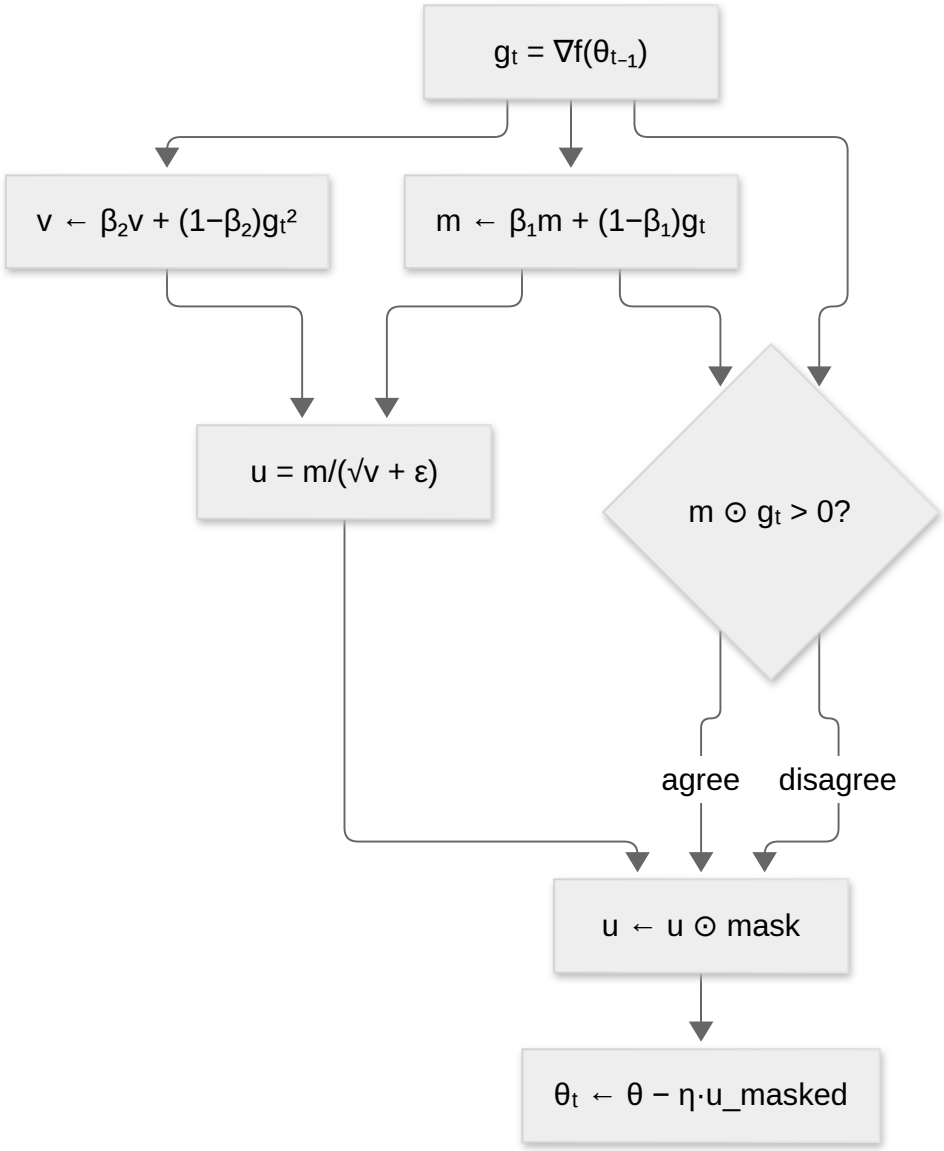


Figura 19: Flujo por paso de Cautious AdamW: una máscara de consenso conserva solo las coordenadas donde momentum y gradiente coinciden.

Optimizadores Cautelosos (Cautious AdamW, Cautious Lion). El marco Cauteloso aplica una **modificación de una línea** a cualquier optimizador basado en momentum: enmascarar la actualización de modo que solo se apliquen las dimensiones donde el momentum y la dirección del gradiente coinciden:

$$\text{mask}_i = \begin{cases} 1 & \text{si } m_t[i] \cdot g_t[i] > 0 \quad \text{(acuerdo)} \\ 0 & \text{en caso contrario} \end{cases} \tag{22}$$

$$u_t = \left(\frac{m_t}{\sqrt{v_t} + \epsilon} \right) \odot \text{mask} \quad \text{(enmascaramiento elemento a elemento)} \tag{23}$$

$$\theta_{t+1} = \theta_t - \eta u_t \tag{24}$$

La intuición: el momentum m_t estima la dirección del gradiente a partir del historial; el gradiente actual g_t es la señal instantánea. Cuando ambos coinciden, el optimizador está seguro y debe actualizar agresivamente. Cuando discrepan, el optimizador está en conflicto—la tendencia histórica apunta hacia una región que pudo haber sido buena antes, pero la evidencia actual la

contradice. Al enmascarar las dimensiones en conflicto, Cauteloso se vuelve más conservador y evita pasos corruptos.

Algorithm 8 Cautious AdamW (Enmascaramiento de Actualización por Consenso)

Require: Parámetros iniciales θ_0, tasa de aprendizaje η, β_1, β_2, decadencia de peso λ
Ensure: Parámetros actualizados θ_T

Inicializar: $m \leftarrow 0$, $v \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
  $g_t \leftarrow \nabla f(\theta_{t-1})$
  $m \leftarrow \beta_1 m + (1 - \beta_1) g_t$
  $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$
  Máscara de consenso: $\text{mask}_i \leftarrow 1$ si $m[i] \cdot g_t[i] > 0$, si no 0 ▷ Acuerdo
  Actualización base Adam: $u \leftarrow m/(\sqrt{v} + \epsilon)$
  Aplicar máscara: $u_{\text{masked}} \leftarrow u \odot \text{mask}$ ▷ Elemento a elemento
  if decadencia_de_peso > 0 **then**
    $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
  end if
  $\theta_t \leftarrow \theta_{t-1} - \eta u_{\text{masked}}$
end forreturn θ_T

Contras.

- El enmascaramiento puede descartar actualizaciones útiles cuando momentum y gradiente discrepan temporalmente.
- Requiere un optimizador base con momentum además de la máscara elemento a elemento.

Ventajas.

- Modificación de una línea aplicable a cualquier optimizador basado en momentum.
- Hasta **1.47× de aceleración** en el preentrenamiento de Llama y MAE preservando las garantías de convergencia.

4.4. Optimizadores de Lote Grande, Eficientes en Memoria y Sin Tasa de Aprendizaje

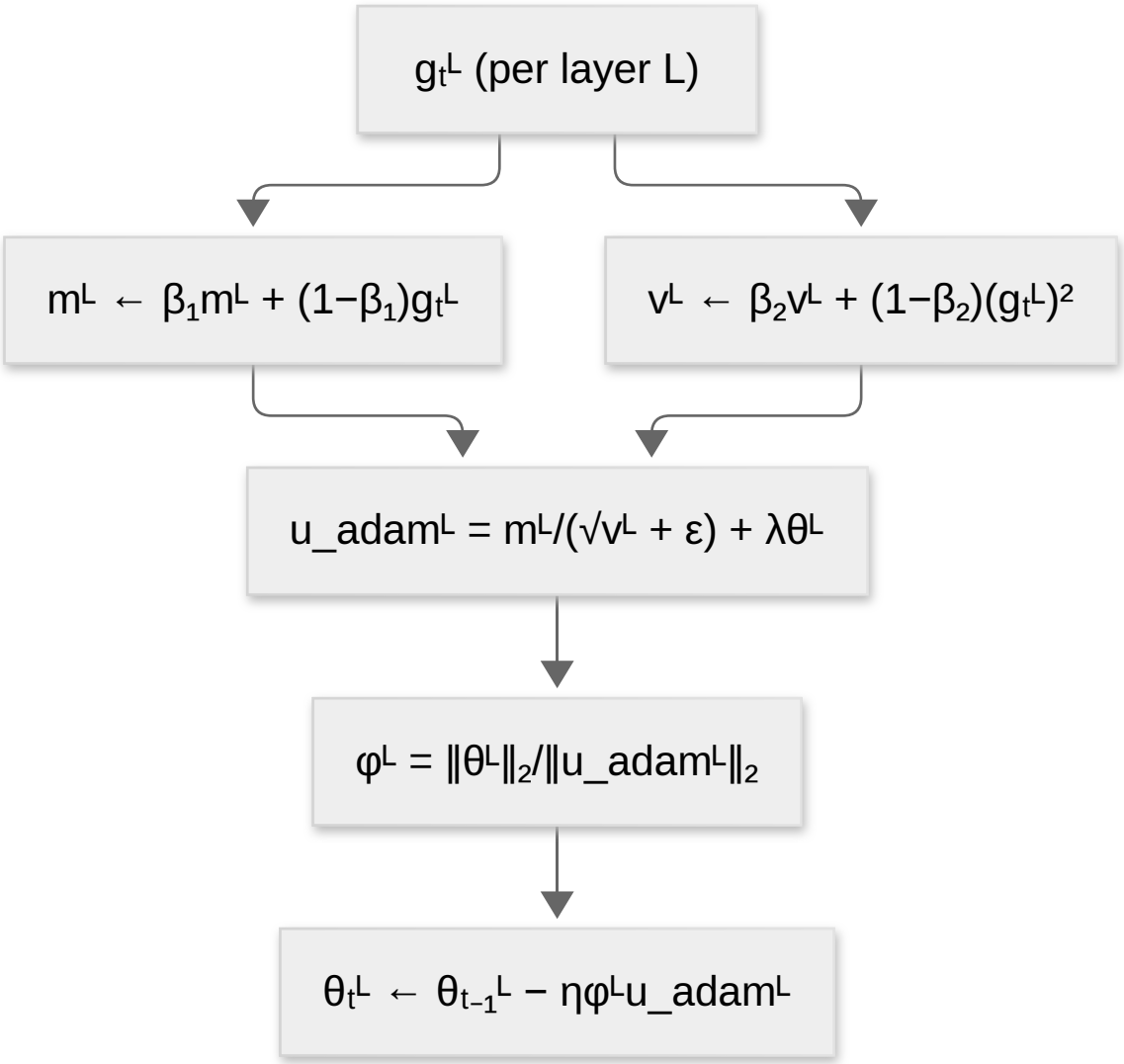


Figura 20: Flujo por paso de LAMB: paso Adam por capas reescalado por la razón de confianza entre las normas de pesos y actualización.

LAMB (Layer-wise Adaptive Moments optimizer for Batch training). LAMB extiende Adam con **escalado de tasa adaptativo por capas** (inspirado en LARS), permitiendo entrenamiento estable con tamaños de lote extremos (e.g., 64K en BERT):

$$m_t^L = \beta_1 m_{t-1}^L + (1-\beta_1) g_t^L \quad \text{(momento de primer orden por capas)} \tag{25}$$

$$v_t^L = \beta_2 v_{t-1}^L + (1-\beta_2)(g_t^L)^2 \tag{26}$$

$$u_{\text{adam}}^L = \frac{m_t^L}{\sqrt{v_t^L} + \epsilon} + \lambda \theta_{t-1}^L \quad \text{(paso adaptativo base con decaimiento)} \tag{27}$$

$$\phi^L = \frac{\|\theta_{t-1}^L\|_2}{\|u_{\text{adam}}^L\|_2} \quad \text{(razón de confianza: normalización por capas)} \tag{28}$$

$$\theta_t^L = \theta_{t-1}^L - \eta \cdot \phi^L \cdot u_{\text{adam}}^L \tag{29}$$

La **razón de confianza** $\phi^L = \|\theta^L\|_2 / \|u^L_{\text{adam}}\|_2$ normaliza el paso de actualización efectivo en relación con la magnitud de los pesos. En lotes muy grandes, el ruido estocástico del gradiente puede superar a la señal, desestabilizando las tasas de aprendizaje por capas. Al escalar las actualizaciones proporcionalmente a las normas de los pesos, LAMB asegura cambios relativos en lugar de absolutos, evitando que actualizaciones pequeñas y seguras en vectores grandes dominen.

Algorithm 9 LAMB (Layer-wise Adaptive Moments optimizer for Batch training)

Require: Parámetros iniciales θ_0, tasa de aprendizaje η, β_1, β_2, decaimiento de pesos λ
Ensure: Parámetros actualizados θ_T

Inicializar: Para cada capa L: $m^L \leftarrow 0$, $v^L \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
  for cada capa L **do**
    $g_t^L \leftarrow \nabla f(\theta_t^L)$
    $m^L \leftarrow \beta_1 m^L + (1 - \beta_1) g_t^L$
    $v^L \leftarrow \beta_2 v^L + (1 - \beta_2)(g_t^L)^2$
    Adam base: $u^L_{\text{adam}} \leftarrow m^L / (\sqrt{v^L} + \epsilon)$
    if weight_decay > 0 **then**
      $u^L_{\text{adam}} \leftarrow u^L_{\text{adam}} + \lambda \theta^L_{t-1}$
    end if
    Razón de confianza: $\phi^L \leftarrow \|\theta^L_{t-1}\|_2 / \|u^L_{\text{adam}}\|_2$ si ambos son no nulos, sino 1
    $\theta_t^L \leftarrow \theta_{t-1}^L - \eta \phi^L u^L_{\text{adam}}$
  end for
end forreturn θ_T

Contras.

- La contabilidad de normas por capas y el cálculo de la razón de confianza añaden sobrecarga por paso.
- La razón de confianza es inestable cuando las normas de capa o de actualización son cercanas a cero.

Ventajas.

- Permite entrenamiento estable con tamaños de lote extremos (e.g., 64K en BERT).
- Preserva los beneficios de generalización de lotes pequeños mientras escala el lote.

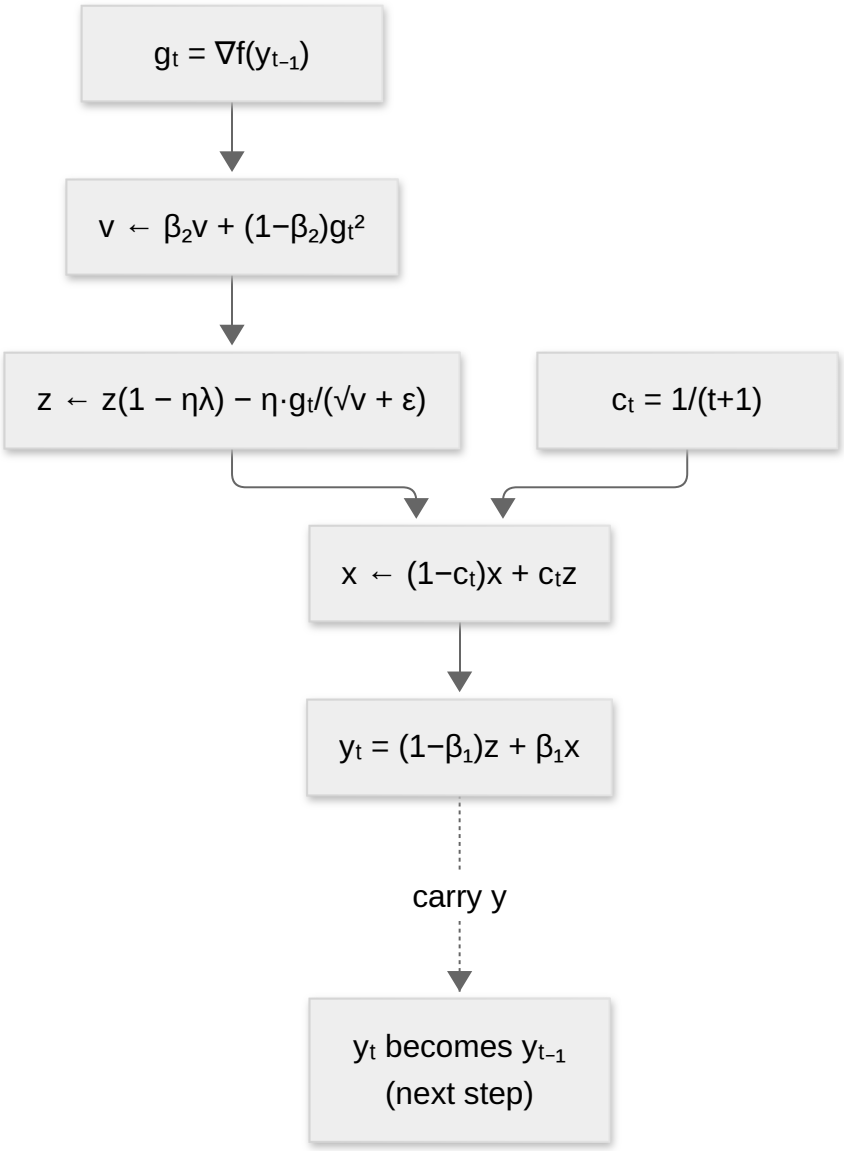


Figura 21: Flujo por paso de Schedule-Free AdamW: flujo de exploración z, promedio de iteraciones x y punto de interpolación y, con promediado implícito $1/(t+1)$.

Schedule-Free AdamW. Los métodos Schedule-Free eliminan el diseño explícito de programación de la receta de optimización. El algoritmo mantiene dos flujos de parámetros: z_t (exploración) y x_t (promedio suavizado):

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \quad \text{(varianza)} \tag{30}$$

$$z_t = z_{t-1}(1-\eta\lambda) - \eta \frac{g_t}{\sqrt{v_t} + \epsilon} \quad \text{(paso adaptativo con decaimiento)} \tag{31}$$

$$c_t = \frac{1}{t+1} \quad \text{(coeficiente de promediado, decrece con el tiempo)} \tag{32}$$

$$x_t = (1-c_t) x_{t-1} + c_t z_t \quad \text{(promediado de iteraciones)} \tag{33}$$

$$y_t = (1-\beta) z_t + \beta x_t \quad \text{(interpolación para el punto de evaluación)} \tag{34}$$

El coeficiente de promediado $c_t = 1/(t+1)$ implementa una programación de tasa de aprendizaje implícita sin especificar explícitamente el número total de pasos T. Este marco unifica la programación y el promediado de iteraciones: el algoritmo explora mediante z_t mientras acumula

dirección estable mediante x_t. El punto de evaluación y_t (usado para el cómputo del gradiente) interpola entre exploración y estabilidad.

Algorithm 10 Schedule-Free AdamW

Require: Parámetros iniciales θ_0, tasa de aprendizaje η (fija), β_1, β_2, decaimiento de pesos λ
Ensure: Parámetros actualizados θ_T o promediado x_T

Inicializar: $z \leftarrow \theta_0$ (exploración), $x \leftarrow \theta_0$ (promedio), $v \leftarrow 0$, $t \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
  $g_t \leftarrow \nabla f(y_{t-1})$ (gradiente en el punto de interpolación)
  $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$ ▷ Varianza
  Decaimiento de pesos sobre z: $z \leftarrow z(1 - \eta\lambda)$
  $z \leftarrow z - \eta g_t / (\sqrt{v} + \epsilon)$ ▷ Paso adaptativo sobre el gradiente crudo
  $c_t \leftarrow 1/(t + 1)$ ▷ Coeficiente de promediado
  $x \leftarrow (1 - c_t)x + c_t z$ ▷ Promediado de iteraciones
  $y_t \leftarrow (1 - \beta_1)z + \beta_1 x$ ▷ Interpolación para la siguiente evaluación
end forreturn x_T o y_T

Contras.

- Mantiene dos flujos de parámetros (z y x) y requiere manejar explícitamente el punto de evaluación.
- Su comportamiento difiere de las líneas base con programación, lo que complica las comparaciones.

Ventajas.

- Elimina la programación explícita de la tasa de aprendizaje y su hiperparámetro de pasos totales.
- Convergencia de última generación en optimización convexa, aprendizaje profundo a gran escala y aprendizaje por refuerzo.

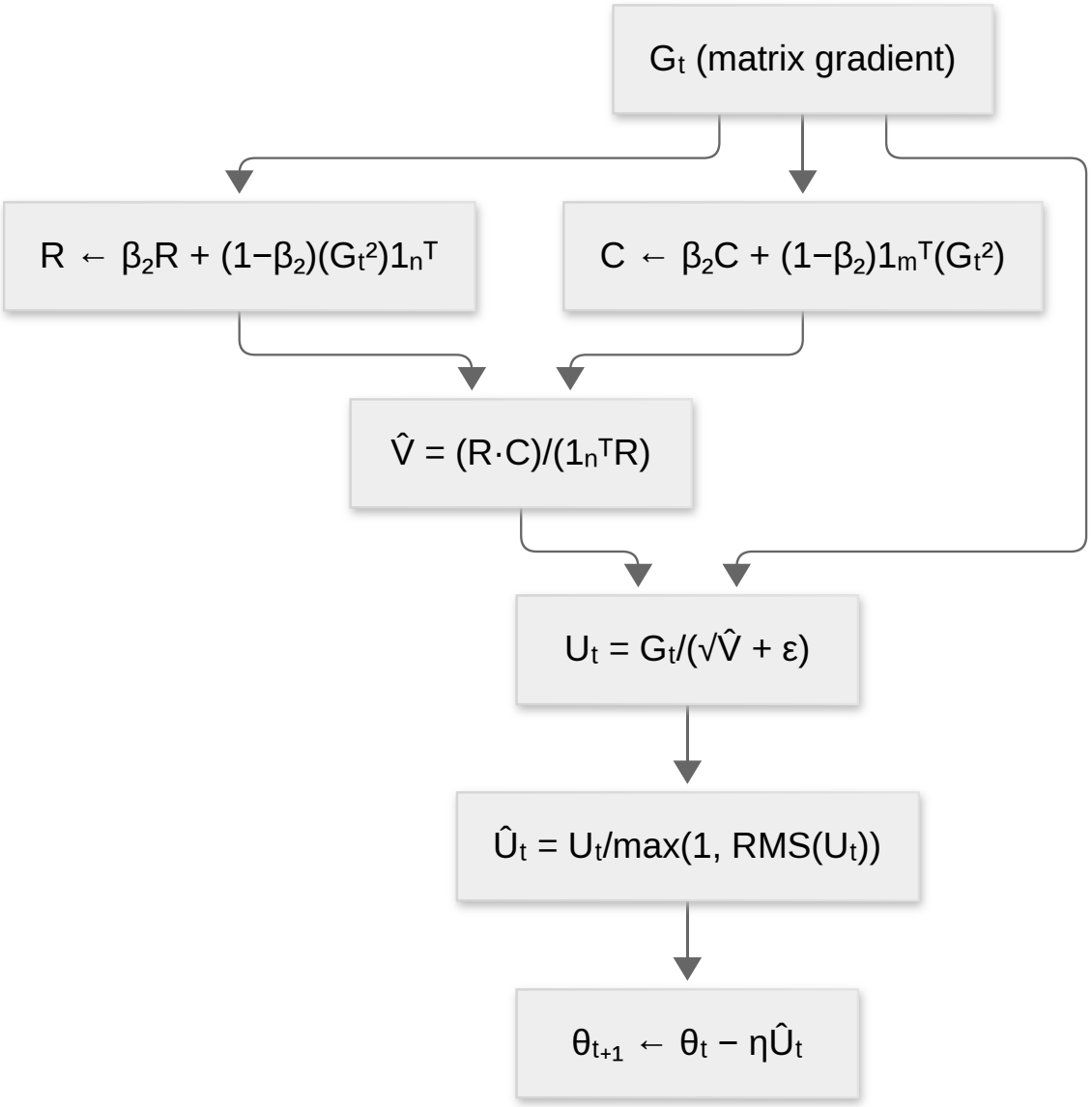


Figura 22: Flujo por paso de Adafactor: acumuladores factorizados de segundo momento por filas y columnas, reconstrucción mediante producto exterior y recorte RMS.

Adafactor. Adafactor reduce la memoria del optimizador **factorizando** estadísticas de segundo momento para parámetros con forma de matriz, almacenando solo acumuladores de filas y columnas en lugar de estados de varianza densos. Para una matriz de gradiente $G_t \in \mathbb{R}^{m \times n}$:

$$R_t = \beta_2 R_{t-1} + (1-\beta_2)(G_t^2)\mathbf{1}_n^T \quad \text{(varianza por filas)} \tag{35}$$

$$C_t = \beta_2 C_{t-1} + (1-\beta_2)\mathbf{1}_m^\top (G_t^2) \quad \text{(varianza por columnas)} \tag{36}$$

$$\hat{V}_t = \frac{R_t C_t}{\mathbf{1}_n^\top R_t} \quad \text{(varianza reconstruida mediante producto exterior)} \tag{37}$$

$$U_t = \frac{G_t}{\sqrt{\hat{V}_t} + \epsilon} \quad \text{(paso adaptativo normalizado)} \tag{38}$$

$$\hat{U}_t = \frac{U_t}{\max(1, \mathrm{RMS}(U_t))} \quad \text{(recorte de estabilidad)} \tag{39}$$

Algorithm 11 Adafactor (Estadísticas de Segundo Momento Factorizadas)

Require: Matriz de gradiente $G_t \in \mathbb{R}^{m \times n}$, tasa de aprendizaje η, $\beta_2 \approx 0{,}999$
Ensure: Parámetros actualizados θ_T

Inicializar: Acumuladores de fila $R \leftarrow \epsilon \mathbf{1}_m^T$, Columna $C \leftarrow \epsilon \mathbf{1}_n$
for $t = 1, 2, \ldots, T$ **do**
  $G_t \leftarrow \nabla f(\theta_t)$ (gradiente matricial)
  $R \leftarrow \beta_2 R + (1 - \beta_2)(G_t^2)\mathbf{1}_n^T$ ▷ Productos internos por filas
  $C \leftarrow \beta_2 C + (1 - \beta_2)\mathbf{1}_m^\top (G_t^2)$ ▷ Productos internos por columnas
  Varianza reconstruida: $\hat{V} \leftarrow (R \cdot C)/(\mathbf{1}_n^\top R)$ ▷ Mediante producto exterior
  Paso adaptativo normalizado: $U_t \leftarrow G_t/(\sqrt{\hat{V}} + \epsilon)$
  Recorte RMS por fila: $\hat{U}_t \leftarrow U_t / \max(1, \mathrm{RMS}(U_t))$
  $\theta_{t+1} \leftarrow \theta_t - \eta \hat{U}_t$
end forreturn θ_T

Contras.

- La varianza factorizada es una aproximación y puede ser inestable en algunas tareas.
- Menor expresividad de optimización en comparación con Adam completo.

Ventajas.

- La memoria baja de $O(n_{\text{params}})$ a $O(\sqrt{n_{\text{params}}})$ para parámetros matriciales.
- Más atractivo cuando la VRAM está dominada por el estado del optimizador en lugar de las activaciones.

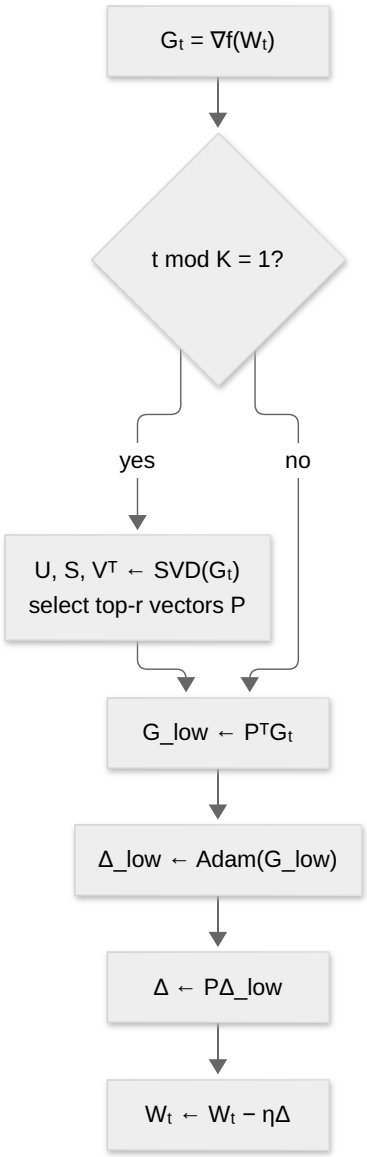


Figura 23: Flujo por paso de GaLore: proyección SVD periódica del gradiente a un subespacio de bajo rango, Adam en ese subespacio y reconstrucción.

GaLore (Gradient Low-Rank Projection). GaLore proyecta gradientes 2D en un subespacio de bajo rango antes de la optimización, reduciendo la memoria del estado del optimizador mientras mantiene el escalado adaptativo del paso. Para una matriz de parámetros 2D $W \in \mathbb{R}^{m\times n}$:

$$U, S, V = \text{SVD}(G_t) \quad \text{(calcular descomposición singular)} \tag{40}$$

$$P \in \mathbb{R}^{n\times r} \quad \text{o} \quad P^\top \in \mathbb{R}^{r\times m} \quad \text{(seleccionar los } r \text{ vectores singulares superiores)} \tag{41}$$

$$G_{\text{low}} = P^\top G_t \quad \text{(proyectar al espacio de bajo rango)} \tag{42}$$

$$\Delta_{\text{low}} = \text{Adam}(G_{\text{low}}) \quad \text{(optimizar en el espacio comprimido)} \tag{43}$$

$$\Delta = P\Delta_{\text{low}} \quad \text{(reconstruir en el espacio original)} \tag{44}$$

Al proyectar en un subespacio de rango r (típicamente $r \ll \text{mín}(m, n)$), el estado del optimizador se reduce de $O(mn)$ a $O(r(m+n))$ para parámetros 2D. Este enfoque complementario de ahorro de memoria es especialmente relevante cuando el modelo es demasiado grande para el estado completo del optimizador.

Algorithm 12 GaLore (Gradient Low-Rank Projection)

Require: Matriz de parámetros 2D $W \in \mathbb{R}^{m \times n}$, tasa de aprendizaje η, rango $r \ll \text{mín}(m, n)$
Ensure: Parámetros actualizados θ_T

Inicializar: Estado de Adam en el espacio de bajo rango (si es variante de rango 2)
for $t = 1, 2, \ldots, T$ **do**
  $G_t \leftarrow \nabla f(W_t)$
  if $t \mod K = 1$ **then** ▷ SVD periódico
    $U_t, S_t, V_t^\top \leftarrow \text{SVD}(G_t)$ (completa o delgada)
    Seleccionar proyección: $P \leftarrow U_t[:, :r]$ o $P \leftarrow V_t[:, :r]$
  end if
  Proyectar gradiente: $G_{\text{low}} \leftarrow P^\top G_t$ (o $G_t P^\top$ para proyección derecha)
  Ejecutar Adam en bajo rango: $\Delta_{\text{low}} \leftarrow \text{Adam}(G_{\text{low}})$
  Reconstruir en espacio original: $\Delta \leftarrow P\Delta_{\text{low}}$ (o $\Delta_{\text{low}} P^\top$)
  $W_t \leftarrow W_t - \eta\Delta$
end forreturn θ_T

Contras.

- El SVD periódico añade cómputo e introduce un intervalo de actualización de la proyección que ajustar.
- La proyección de bajo rango puede descartar información del gradiente.

Ventajas.

- El estado del optimizador se reduce de $O(mn)$ a $O(r(m+n))$ para parámetros 2D.
- Funciona sinérgicamente con otras técnicas y ha mostrado resultados sólidos en modelos de miles de millones de parámetros.

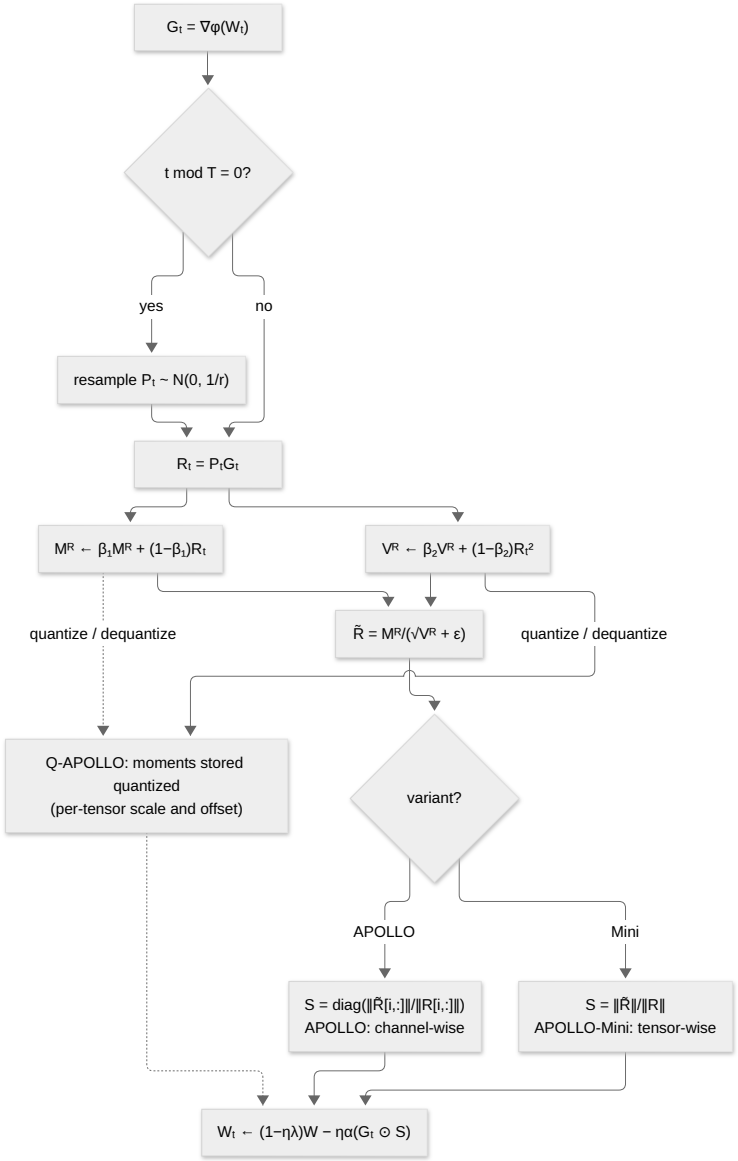


Figura 24: Flujo por paso de la familia APOLLO: momentos proyectados gaussianos, escalado estructurado del gradiente (por canales en APOLLO, escalar en APOLLO-Mini) y estado cuantizado en Q-APOLLO.

APOLLO, APOLLO-Mini, y Q-APOLLO. La familia APOLLO [134] parte de la observación de que el denominador elemento a elemento de AdamW puede **transformarse en una actualización estructurada de la tasa de aprendizaje**. En lugar de almacenar momentos densos para cada entrada de parámetro, APOLLO proyecta un gradiente matricial $G_t \in \mathbb{R}^{m\times n}$ en un subespacio aleatorio compacto y rastrea momentos estilo Adam allí:

$$R_t = P_t G_t \quad \text{o} \quad R_t = G_t P_t^\top \tag{45}$$

$$M_t^R = \beta_1 M_{t-1}^R + (1-\beta_1) R_t \tag{46}$$

$$V_t^R = \beta_2 V_{t-1}^R + (1-\beta_2) R_t^2 \tag{47}$$

$$\widetilde{R}_t = \frac{M_t^R}{\sqrt{V_t^R} + \epsilon} \tag{48}$$

El estado proyectado *no* se expande de vuelta a una actualización densa de bajo rango como en los métodos basados en SVD. En su lugar, APOLLO estima un tensor de escalado estructurado

S_t en el espacio original. En la variante estándar de APOLLO, ese escalado es **por canales**: cada fila o canal recibe su propia razón de norma. En APOLLO-Mini, el escalado se reduce a un único escalar **por tensor**, correspondiente al caso extremo de rango 1 descrito en el artículo. La actualización de parámetros resultante es, por tanto, similar a Adam en adaptación pero mucho más cercana a SGD en costo de estado:

$$W_t \leftarrow (1 - \eta\lambda)W_{t-1} - \eta\,\alpha\,(G_t \odot S_t)$$

donde α es el factor de escala adicional usado para estabilizar variantes altamente comprimidas.

Algorithm 13 APOLLO / APOLLO-Mini Escalado de Gradiente Estructurado

Require: Matriz de pesos $W \in \mathbb{R}^{m\times n}$ con $m \le n$, tasa de aprendizaje η, factor de escala α, tasas de decaimiento (β_1, β_2), decaimiento de pesos λ, rango r, intervalo de actualización de proyección T

Ensure: Parámetros actualizados W_T

Inicializar momentos proyectados: $M^R \leftarrow 0$, $V^R \leftarrow 0$, paso $t \leftarrow 0$
repeat
  Calcular gradiente: $G_t \leftarrow \nabla_W \phi(W_t)$
  if t mód $T = 0$ **then**
    Muestrear proyector gaussiano $P_t \sim \mathcal{N}(0, 1/r)$ con una semilla nueva
  end if
  Proyectar gradiente: $R_t \leftarrow P_t G_t$ ▷ o $G_t P_t^\top$ según la disposición
  Actualizar momentos proyectados AdamW: $M_t^R, V_t^R \leftarrow \text{AdamWState}(R_t; \beta_1, \beta_2)$
  Normalizar estado proyectado: $\widetilde{R}_t \leftarrow M_t^R/(\sqrt{V_t^R} + \epsilon)$
  if APOLLO **then**
    $S_t \leftarrow \text{diag}(s_1^R, \ldots, s_m^R)$, donde $s_i^R = \|\widetilde{R}_t[i,:]\|_2/\|R_t[i,:]\|_2$
  else
    $S_t \leftarrow s_t^R$, donde $s_t^R = \|\widetilde{R}_t\|_2/\|R_t\|_2$
  end if
  Actualizar pesos: $W_t \leftarrow (1 - \eta\lambda)W_{t-1} - \eta\,\alpha\,(G_t \odot S_t)$
  $t \leftarrow t + 1$
until convergencia

Contras.

- El escalado estructurado es una aproximación más gruesa del denominador elemento a elemento de AdamW.
- Las proyecciones deben refrescarse periódicamente y rangos muy pequeños hacen que el escalado sea ruidoso.

Ventajas.

- Reemplaza el SVD repetido con proyección aleatoria gaussiana refrescada periódicamente, de modo que el costo de sistemas es multiplicación de matrices ordinaria en lugar de descomposición espectral.
- Inusualmente tolerante a la compresión extrema: incluso APOLLO-Mini, que solo mantiene estado auxiliar de rango 1, sigue siendo competitivo o mejor que AdamW mientras se acerca al costo de memoria de SGD.

APOLLO-Mini. APOLLO-Mini es el miembro de la familia optimizado para **eficiencia de memoria extrema**. El artículo lo motiva argumentando que, en un espacio compacto de rango 1, el escalado por canales se vuelve demasiado ruidoso, por lo que la actualización se simplifica aún más a un único escalar por tensor. La pérdida de granularidad se compensa parcialmente con el factor de escala explícito α (el artículo discute valores como 128 en este régimen), proporcionando un punto útil en la frontera de Pareto de optimización: estado muy pequeño, sin sobrecarga de SVD, y aún un comportamiento sólido en preentrenamiento.

Contras.

- Un único escalar por tensor pierde granularidad por canales, y el estado de rango 1 puede ser ruidoso.
- Depende del factor de escala adicional α (e.g., 128) para mantenerse estable.

Ventajas.

- El estado más pequeño de la familia APOLLO, sin sobrecarga de SVD.
- Comportamiento sólido en preentrenamiento con un costo de memoria cercano a SGD.

Q-APOLLO. El artículo de APOLLO también enfatiza que la familia se combina naturalmente con **cuantización** para entrenamiento con memoria ultra-baja. Este repositorio convierte esa observación de sistemas en una variante concreta del optimizador, `q_apollo`. En la implementación, los momentos de primer y segundo orden de bajo rango de APOLLO se almacenan en forma cuantizada junto con escalas y desplazamientos por tensor, y se descuantizan solo cuando es necesario para el siguiente paso. Q-APOLLO preserva, por tanto, la lógica de gradiente proyectado de APOLLO mientras reduce la precisión del estado restante del optimizador.

Contras.

- Los momentos cuantizados introducen error de aproximación y sobrecarga de descuantización.
- Requiere gestionar escalas y desplazamientos por tensor.

Ventajas.

- Reduce el estado restante del optimizador a precisión ultra baja.
- Miembro más agresivo en ahorro de memoria de la pila local, preservando la lógica de gradiente proyectado de APOLLO.

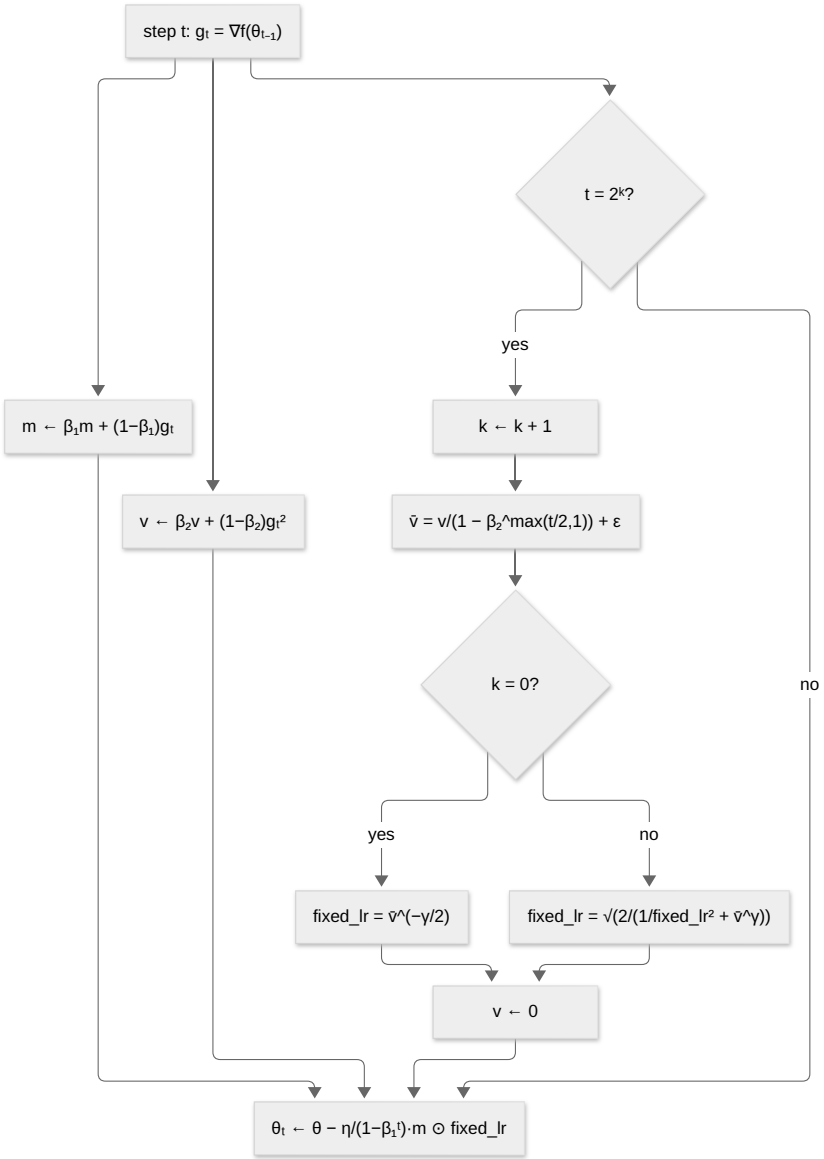


Figura 25: Flujo por paso de Anon: el precondicionador IDU refrescado en potencias de dos, con adaptividad ajustable γ y una actualización de momentum con corrección de sesgo.

Anon (Adaptivity Non-restricted Optimizer with Novel convergence technique). Anon [6] introduce adaptividad ajustable continuamente mediante un único parámetro $\gamma \in \mathbb{R}$, cerrando la brecha entre comportamientos tipo SGD ($\gamma = 0$) y tipo Adam ($\gamma = 1$), y extrapolando más allá de ambos. La innovación central es el mecanismo Incremental Delay Update (IDU), que permite convergencia estable a través de todo el espectro de adaptividad de valores reales:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad \text{(momento de primer orden)} \tag{49}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \quad \text{(momento de segundo orden)} \tag{50}$$

$$\bar{v}_t = \frac{v_t}{1-\beta_2^{\max(t,1)}} + \epsilon \quad \text{(momento de segundo orden con corrección de sesgo)} \tag{51}$$

$$\text{fixed_lr}_k = \begin{cases} \bar{v}_k^{-\gamma/2} & \text{si } k = 0 \\ \sqrt{2/\left(\frac{1}{\text{fixed_lr}_{k-1}^2} + \bar{v}_k^{\gamma}\right)} & \text{si } k > 0 \end{cases} \tag{52}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{1-\beta_1^t} \cdot m_t \odot \text{fixed_lr}_{\lfloor \log_2 t \rfloor} \tag{53}$$

A diferencia de los optimizadores adaptativos estándar, IDU actualiza el precondicionador solo en potencias de dos ($t = 2^k$), acumulando estadísticas de gradiente entre actualizaciones y agregándolas recursivamente. Este acumulador suave y multiescala evita el seguimiento estricto del máximo de AMSGrad mientras estabiliza demostrablemente la convergencia para cualquier $\gamma \in \mathbb{R}$. El parámetro γ controla directamente la adaptividad: $\gamma > 1$ acelera la salida de puntos de silla (favorable para arquitecturas complejas), $\gamma = 1$ recupera el comportamiento tipo Adam, $\gamma = 0$ produce escalado tipo SGD, y $\gamma < 0$ se sesga hacia mínimos más planos (beneficioso para arquitecturas clásicas como CNNs).

Algorithm 14 Anon (Adaptivity Non-restricted Optimizer con IDU)

Require: Parámetros iniciales θ_0, tasa de aprendizaje η, β_1, β_2, tolerancia ϵ, adaptividad $\gamma \in \mathbb{R}$
Ensure: Parámetros actualizados θ_T

Inicializar: $m \leftarrow 0$, $v \leftarrow 0$, $fixed_lr \leftarrow 0$, paso $t \leftarrow 0$, contador IDU $k \leftarrow -1$
for $t = 1, 2, \ldots, T$ **do**
    $g_t \leftarrow \nabla f(\theta_{t-1})$
    $m \leftarrow \beta_1 m + (1-\beta_1) g_t$
    $v \leftarrow \beta_2 v + (1-\beta_2) g_t^2$
    if $t = 2^k$ **then**
        $k \leftarrow k + 1$
        $\bar{v} \leftarrow v / (1 - \beta_2^{\max(t/2,1)}) + \epsilon$
        if $k = 0$ **then**
            fixed_lr $\leftarrow \bar{v}^{-\gamma/2}$
        else
            fixed_lr $\leftarrow \sqrt{2/(1/\text{fixed_lr}^2 + \bar{v}^{\gamma})}$
        end if
        $v \leftarrow 0$
    end if
    $\theta_t \leftarrow \theta_{t-1} - \frac{\eta}{1-\beta_1^t} \cdot m \odot$ fixed_lr
end forreturn θ_T

Contras.

- Mantiene tres búferes de estado (m, v, fixed_lr) y refresca el precondicionador con una programación logarítmica.
- El parámetro de adaptividad γ debe seleccionarse por arquitectura.

Ventajas.

- Misma clase de complejidad $\mathcal{O}(n)$ por paso que AdamW, con adaptividad ajustable de forma continua.
- IDU alcanza demostrablemente regret $\mathcal{O}(\sqrt{T})$ para problemas convexos y $\mathcal{O}(\ln T/\sqrt{T})$ para entornos no convexos.

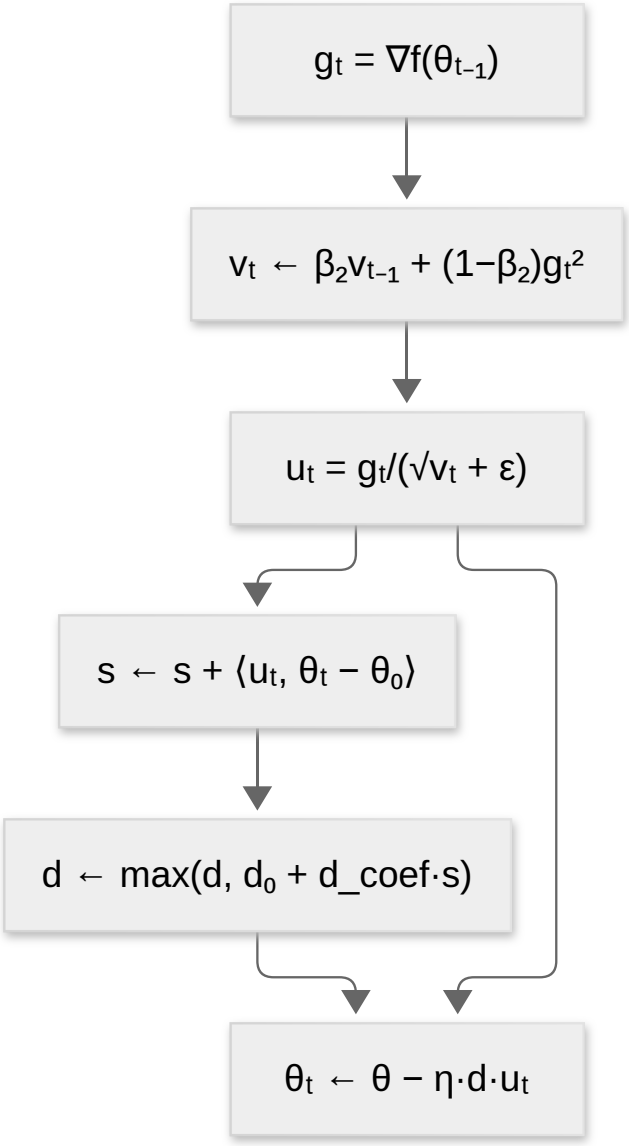


Figura 26: Flujo por paso de Prodigy: una estimación de distancia signada acumulativa que escala el paso adaptativo sin programación de tasa de aprendizaje.

Prodigy (Approximating the Distance Estimate). Prodigy adapta la escala efectiva del paso mediante una estadística tipo distancia acumulativa, eliminando la necesidad de ajuste

explícito de la tasa de aprendizaje. El algoritmo mantiene una estimación de distancia acumulativa:

$$u_t = \frac{g_t}{\sqrt{v_t} + \epsilon} \quad \text{(paso adaptativo no normalizado)} \tag{54}$$

$$s_t = s_{t-1} + \langle u_t, \theta_t - \theta_0 \rangle \quad \text{(distancia signada acumulativa)} \tag{55}$$

$$d_t = \max(d_{t-1}, d_0 + d_{\text{coef}} \cdot s_t) \quad \text{(estimación de distancia con cota inferior)} \tag{56}$$

$$\theta_{t+1} = \theta_t - \eta \cdot d_t \cdot u_t \tag{57}$$

donde d_0 es una cota inicial y d_{coef} controla qué tan agresivamente se adapta la estimación. La estimación de distancia d_t captura la magnitud combinada de gradientes pasados ponderada por el desplazamiento real de los parámetros, implementando una tasa de aprendizaje efectiva implícita.

Algorithm 15 Prodigy (Approximating the Distance Estimate)

Require: Parámetros iniciales θ_0, tasa de aprendizaje η, coeficiente d_{coef}, distancia inicial $d_0 > 0$
Ensure: Parámetros actualizados θ_T

Inicializar: $s \leftarrow 0$ (distancia signada acumulativa), $d \leftarrow d_0$
for $t = 1, 2, \ldots, T$ **do**
  $g_t \leftarrow \nabla f(\theta_{t-1})$
  $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ ▷ Segundo momento
  $u_t \leftarrow g_t / (\sqrt{v_t} + \epsilon)$ ▷ Adaptativo no normalizado
  Actualizar distancia: $s \leftarrow s + \langle u_t, \theta_t - \theta_0 \rangle$ ▷ Distancia signada
  $d \leftarrow \max(d, d_0 + d_{\text{coef}} \cdot s)$ ▷ Distancia con cota inferior
  $\theta_t \leftarrow \theta_{t-1} - \eta \cdot d \cdot u_t$ ▷ Actualización escalada por distancia
end forreturn θ_T

Contras.

- Introduce los hiperparámetros d_0 y d_{coef} para la estimación de distancia.
- Añade un paso de contabilidad de producto interno por iteración.

Ventajas.

- Elimina el ajuste y la programación explícitos de la tasa de aprendizaje.
- Convergencia robusta en distintas escalas de problemas y tamaños de lote.

4.5. Optimizadores de Segundo Orden, Geométricos y de Ortogonalidad

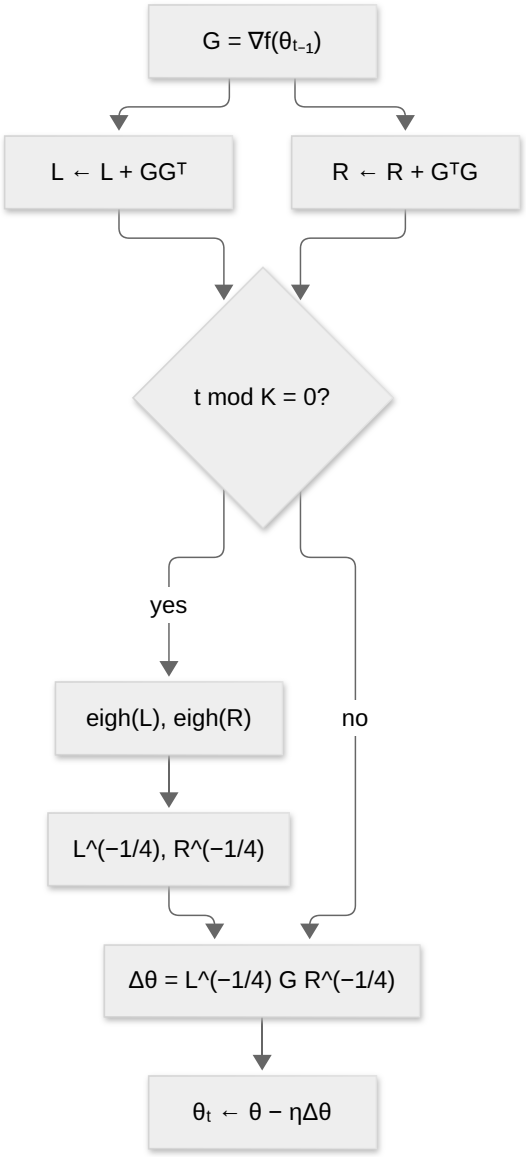


Figura 27: Flujo por paso de Shampoo: matrices de Gram factorizadas por Kronecker, eigendescomposición periódica y precondicionamiento del gradiente con raíz cuarta.

Shampoo (Precondicionamiento Matricial mediante Productos de Kronecker). Shampoo es un **método estructurado de segundo orden** que calcula precondicionadores matriciales a partir de estadísticas de producto exterior con estructura de Kronecker. Para una matriz de parámetros 2D $W \in \mathbb{R}^{m\times n}$:

$$L_t = L_{t-1} + G_t G_t^\top \quad \text{(matriz de Gram izquierda/filas, } m \times m) \tag{58}$$

$$R_t = R_{t-1} + G_t^\top G_t \quad \text{(matriz de Gram derecha/columnas, } n \times n) \tag{59}$$

$$L_t^{-1/4} = Q_L(\Lambda_L)^{-1/4} Q_L^\top \quad \text{(mediante eigendescomposición)} \tag{60}$$

$$R_t^{-1/4} = Q_R(\Lambda_R)^{-1/4} Q_R^\top \tag{61}$$

$$\Delta W_t = L_t^{-1/4} G_t R_t^{-1/4} \quad \text{(gradiente precondicionado)} \tag{62}$$

Shampoo aproxima el precondicionador de matriz completa de Adagrad $H^{-1/2}$ (donde H es el Hessiano) utilizando estructura factorizada de Kronecker. En lugar de almacenar un precondicionador de $(mn) \times (mn)$, Shampoo mantiene dos matrices más pequeñas ($m \times m$ y $n \times n$), capturando correlaciones entre parámetros dentro y entre grupos. Las eigendescomposiciones se realizan periódicamente (e.g., cada $K = 10$ pasos) para amortizar el costo.

Algorithm 16 Shampoo (Precondicionamiento Matricial mediante Productos de Kronecker)

Require: Parámetros iniciales θ_0, tasa de aprendizaje η, intervalo de eigendescomposición K
Ensure: Parámetros actualizados θ_T

Inicializar: $L \leftarrow \epsilon I_m$, $R \leftarrow \epsilon I_n$ (matrices de Gram izquierda y derecha)
for $t = 1, 2, \ldots, T$ **do**
  $G \leftarrow \nabla f(\theta_{t-1})$ ▷ Gradiente
  Actualizar matrices de Gram: $L \leftarrow L + GG^\top$, $R \leftarrow R + G^\top G$
  if $t \mod K == 0$ **then**
    $Q_L, \Lambda_L \leftarrow \text{eigh}(L)$ ▷ Eigendescomposición de L
    $Q_R, \Lambda_R \leftarrow \text{eigh}(R)$ ▷ Eigendescomposición de R
    $L^{-1/4} \leftarrow Q_L(\Lambda_L + \epsilon)^{-1/4} Q_L^\top$
    $R^{-1/4} \leftarrow Q_R(\Lambda_R + \epsilon)^{-1/4} Q_R^\top$
  end if
  $\Delta\theta \leftarrow L^{-1/4} G R^{-1/4}$ ▷ Gradiente precondicionado
  $\theta_t \leftarrow \theta_{t-1} - \eta \cdot \Delta\theta$
end forreturn θ_T

Contras.

- Eigendescomposiciones periódicas $O(m^3 + n^3)$ y mayor cómputo por paso.
- Asume estructura de parámetros 2D; los embeddings y sesgos requieren manejo aparte.

Ventajas.

- Precondicionamiento de segundo orden factorizado por Kronecker sin una matriz densa $(mn) \times (mn)$.
- Demostrado efectivo a escala de producción y adecuado para matrices transformer de alto rango, con mejor condicionamiento y convergencia acelerada.

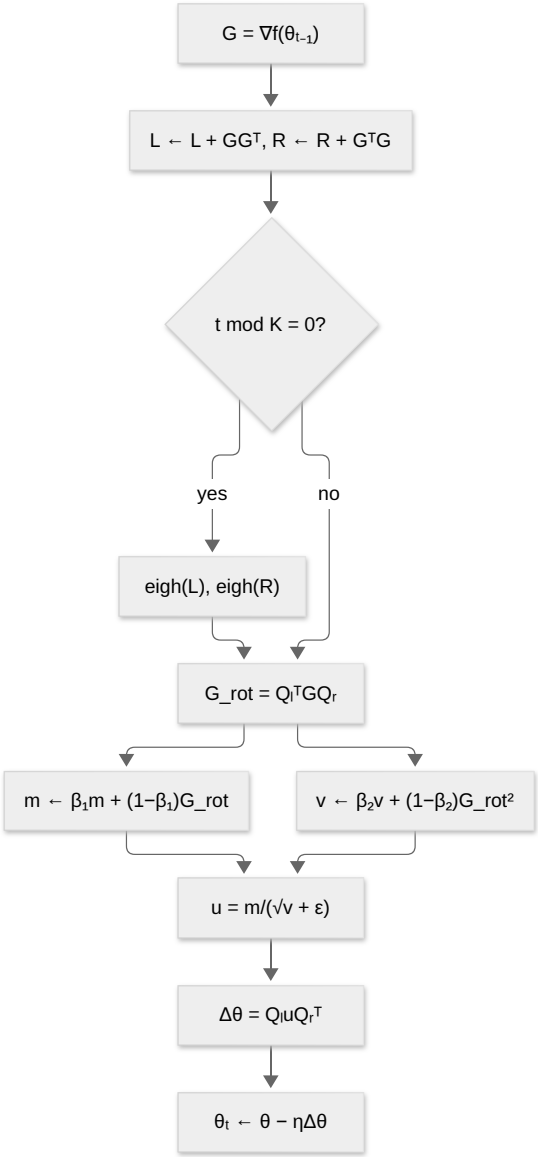


Figura 28: Flujo por paso de SOAP: momentos de Adam en la base de eigenvectores de Shampoo, rotados de vuelta al espacio de parámetros.

SOAP (Shampoo with Adam in eigenbasis). SOAP es una variante simplificada de Shampoo que desacopla el precondicionamiento del seguimiento de momento. En lugar de álgebra matricial compleja sobre gradientes precondicionados, SOAP ejecuta Adam estándar en la base de eigenvectores de los precondicionadores de Shampoo:

$$L_t = L_{t-1} + G_t G_t^\top, \quad R_t = R_{t-1} + G_t^\top G_t \quad \text{(acumulación de Gram)} \tag{63}$$

$$Q_L, \Lambda_L = \text{eigh}(L_t), \quad Q_R, \Lambda_R = \text{eigh}(R_t) \quad \text{(eigendescomposición periódica)} \tag{64}$$

$$G_t^{\text{rot}} = Q_L^\top G_t Q_R \quad \text{(rotar gradiente a la base de eigenvectores)} \tag{65}$$

$$m_t^{\text{rot}} = \beta_1 m_{t-1}^{\text{rot}} + (1-\beta_1) G_t^{\text{rot}} \tag{66}$$

$$v_t^{\text{rot}} = \beta_2 v_{t-1}^{\text{rot}} + (1-\beta_2)(G_t^{\text{rot}})^2 \quad \text{(Adam en el espacio rotado)} \tag{67}$$

$$U_t^{\text{rot}} = \frac{m_t^{\text{rot}}}{\sqrt{v_t^{\text{rot}}} + \epsilon}, \quad U_t = Q_L U_t^{\text{rot}} Q_R^\top \quad \text{(rotar de vuelta)} \tag{68}$$

La idea clave: todo el seguimiento de momento ocurre en la base de eigenvectores bien condicionada, simplificando la estabilidad numérica y el análisis teórico.

Algorithm 17 SOAP (Shampoo with Adam in Eigenbasis)

Require: Parámetros iniciales θ_0, tasa de aprendizaje η, β_1, β_2, intervalo de eigendescomposición K

Ensure: Parámetros actualizados θ_T

Inicializar: $L \leftarrow \epsilon I_m$, $R \leftarrow \epsilon I_n$, $m \leftarrow 0$, $v \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
  $G \leftarrow \nabla f(\theta_{t-1})$ ▷ Gradiente
  Actualizar Gram: $L \leftarrow L + GG^\top$, $R \leftarrow R + G^\top G$
  if $t \mod K == 0$ **then**
    $Q_L, \Lambda_L \leftarrow \text{eigh}(L)$, $Q_R, \Lambda_R \leftarrow \text{eigh}(R)$
  end if
  $G^{\text{rot}} \leftarrow Q_L^\top G Q_R$ ▷ Rotar gradiente a la base de eigenvectores
  $m \leftarrow \beta_1 m + (1 - \beta_1) G^{\text{rot}}$ ▷ Media móvil exponencial (corregir sesgo externamente)
  $v \leftarrow \beta_2 v + (1 - \beta_2)(G^{\text{rot}})^2$ ▷ Segundo momento (corregir sesgo externamente)
  $u \leftarrow m/(\sqrt{v} + \epsilon)$ ▷ Paso adaptativo en la base de eigenvectores
  $\Delta\theta \leftarrow Q_L u Q_R^\top$ ▷ Rotar de vuelta al espacio de parámetros
  $\theta_t \leftarrow \theta_{t-1} - \eta \cdot \Delta\theta$
end forreturn θ_T

Contras.

- Hereda el costo de eigendescomposición periódica de Shampoo.
- La rotación y la rotación inversa añaden multiplicaciones de matrices por paso.

Ventajas.

- Combina la estructura de curvatura explícita de Shampoo con la mecánica de momentum probada de Adam.
- A menudo más limpio y estable que Shampoo completo, con eigendescomposiciones amortizadas cada K pasos.

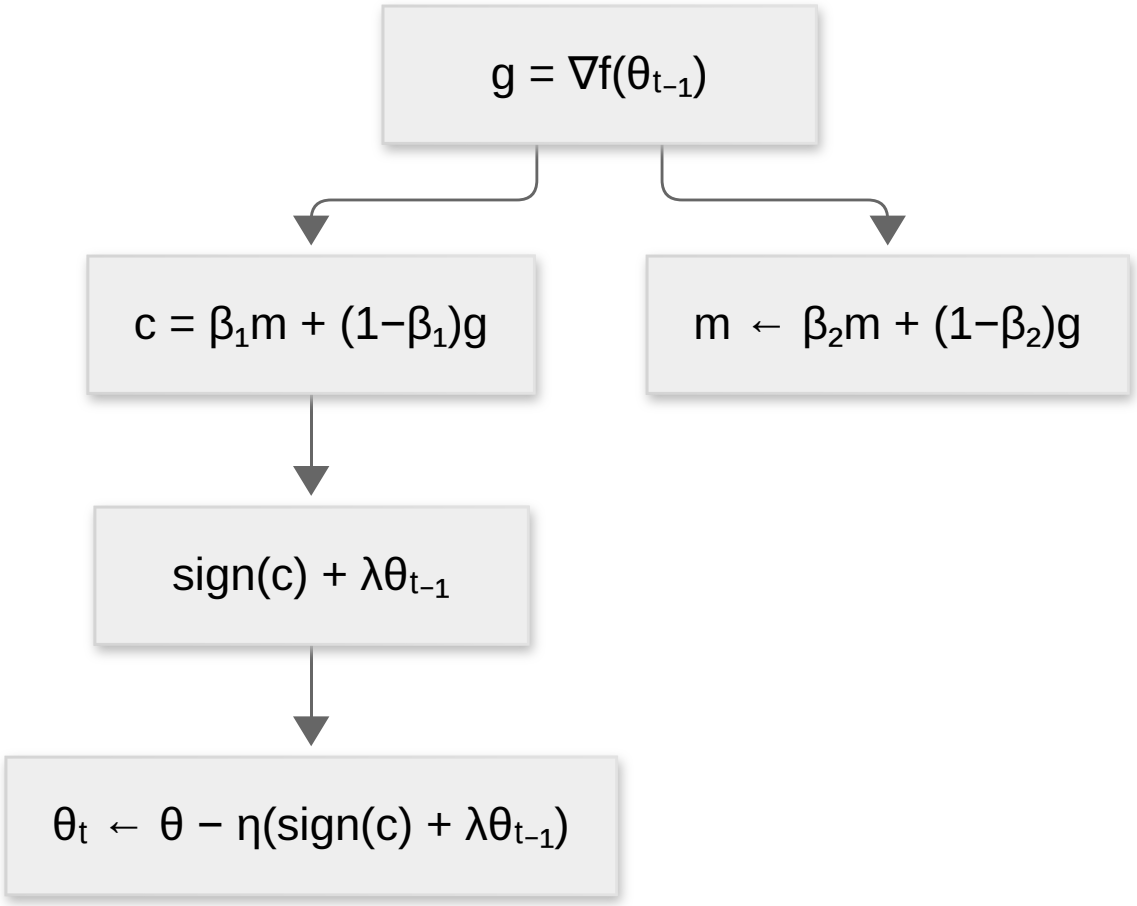


Figura 29: Flujo por paso de Lion: actualización por signo del momentum con acumulación de momentum desacoplada y decaimiento de pesos.

Lion (EvoLved Sign Momentum). Lion logra una **sobrecarga de memoria mínima** al usar actualizaciones basadas en signo en lugar de escalado adaptativo completo:

$$c_t = \beta_1 m_t + (1-\beta_1) g_t \quad \text{(entrada de momento)} \tag{69}$$

$$\theta_{t+1} = \theta_t - \eta\,(\text{sign}(c_t) + \lambda\theta_t) \quad \text{(actualización con operador signo)} \tag{70}$$

$$m_{t+1} = \beta_2 m_t + (1-\beta_2) g_t \quad \text{(acumulación de momento)} \tag{71}$$

Todas las operaciones de escalado elemento a elemento se reemplazan con `sign()`, que produce $\{-1, 0, +1\}$. Esto reduce drásticamente la memoria en comparación con métodos tipo Adam: Lion almacena solo el búfer de momento, sin varianza de segundo momento.

Algorithm 18 Lion (EvoLved Sign Momentum)

Require: Parámetros iniciales θ_0, tasa de aprendizaje η, β_1, β_2, decaimiento de pesos λ
Ensure: Parámetros actualizados θ_T

Inicializar: $m \leftarrow 0$ (búfer de momento)
for $t = 1, 2, \ldots, T$ **do**
  $g \leftarrow \nabla f(\theta_{t-1})$ ▷ Gradiente
  $c \leftarrow \beta_1 m + (1-\beta_1) g$ ▷ Entrada de momento
  $\theta_t \leftarrow \theta_{t-1} - \eta(\text{sign}(c) + \lambda\theta_{t-1})$ ▷ Actualización basada en signo con decaimiento de pesos
  $m \leftarrow \beta_2 m + (1-\beta_2) g$ ▷ Acumulación de momento (desacoplada)
end forreturn θ_T

Contras.

- Las actualizaciones por signo sacrifican la adaptación elemento a elemento de la tasa de aprendizaje que hace efectivo a Adam.
- Puede ser menos robusto que los métodos de la familia Adam en tareas que requieren escalado adaptativo.

Ventajas.

- Almacena solo el búfer de momentum, sin varianza de segundo momento.
- Opción de alto rendimiento y baja memoria para regímenes de lotes grandes donde domina la memoria de activaciones.

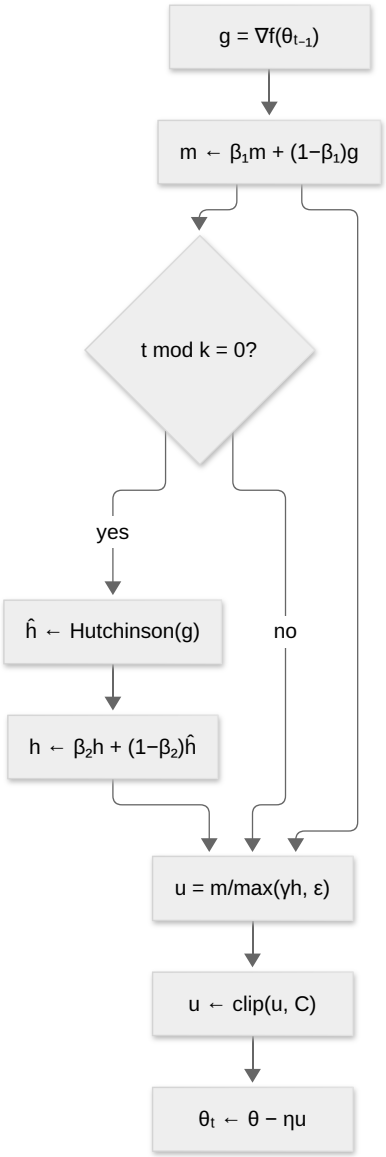


Figura 30: Flujo por paso de Sophia: estimación diagonal del Hessiano mediante Hutchinson con pasos recortados conscientes de la curvatura.

Sophia (Second-Order Hessian Information with Optimized Approximation). Sophia utiliza **estimaciones diagonales del Hessiano** para escalado consciente de la curvatura sin el costo de memoria y cómputo de los métodos densos de segundo orden.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad \text{(momento de primer orden)} \tag{72}$$

$$h_t = \beta_2 h_{t-(k-1)} + (1-\beta_2)\hat{h}_t \quad \text{(Hessiano diagonal, actualizado cada } k \text{ pasos)} \tag{73}$$

$$\text{clip}(x, C) = \text{mín}(\text{máx}(x, -C), C) \quad \text{(recorte elemento a elemento)} \tag{74}$$

$$\theta_{t+1} = \theta_t - \eta \cdot \text{clip}\left(\frac{m_t}{\text{máx}(\gamma h_t, \epsilon)}, 1\right) \tag{75}$$

donde $\hat{h}_t$ es una estimación diagonal (e.g., estimador de traza de Hutchinson) y γ es un factor de escala. El Hessiano diagonal h_t captura la curvatura local, permitiendo que el optimizador tome pasos más pequeños en direcciones pronunciadas y pasos más grandes en direcciones planas. La operación de recorte clip$(\cdot, 1)$ evita que los pasos adaptativos exploten.

Algorithm 19 Sophia (Hessiano Diagonal con Actualizaciones Recortadas)

Require: Parámetros iniciales θ_0, tasa de aprendizaje η, β_1, β_2, frecuencia de actualización del Hessiano k, umbral de recorte C

Ensure: Parámetros actualizados θ_T

Inicializar: $m \leftarrow 0$, $h \leftarrow \epsilon$ (estimación diagonal del Hessiano)
for $t = 1, 2, \ldots, T$ **do**
  $g \leftarrow \nabla f(\theta_{t-1})$
  $m \leftarrow \beta_1 m + (1-\beta_1) g$ ▷ Momento de primer orden (corregir sesgo externamente)
  if $t \mod k == 0$ **then**
    $\hat{h} \leftarrow \text{HutchinsonEstimate}(g)$ ▷ Hessiano diagonal mediante Hutchinson
    $h \leftarrow \beta_2 h + (1-\beta_2)\hat{h}$ ▷ Actualizar estimación del Hessiano
  end if
  $u \leftarrow \frac{m}{\text{máx}(\gamma h, \epsilon)}$ ▷ Paso adaptativo (elemento a elemento)
  $u \leftarrow \text{clip}(u, C)$ ▷ Recorte elemento a elemento a $[-C, C]$
  $\theta_t \leftarrow \theta_{t-1} - \eta \cdot u$
end forreturn θ_T

Contras.

- Las estimaciones de Hutchinson añaden cómputo periódico y ruido a la señal de curvatura.
- La estimación del Hessiano puede quedar desactualizada entre refrescos.

Ventajas.

- Escalado consciente de la curvatura sin el costo $O(n^2)$ u $O(n^3)$ de los métodos completos de segundo orden.
- Los pasos recortados evitan explosiones y funcionan particularmente bien en el ajuste fino de segunda pasada.

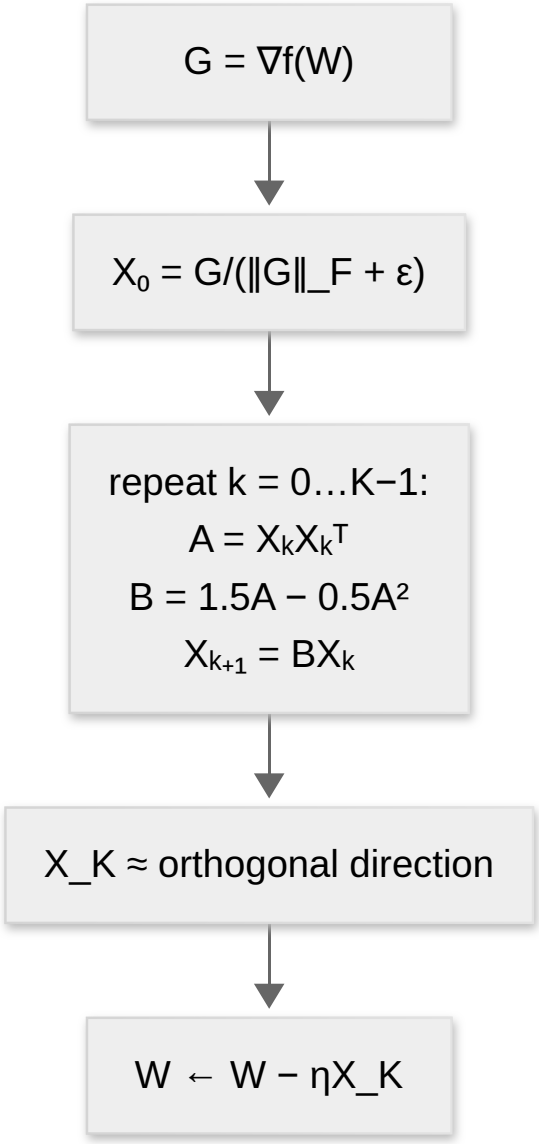


Figura 31: Flujo por paso de Muon: normalización de Frobenius seguida de iteraciones de Newton-Schulz que ortogonalizan la dirección de actualización.

Muon (Ortogonalización de Newton-Schulz). Muon es un **optimizador orientado a la ortogonalidad** que reformula la geometría de actualización utilizando iteraciones polinomiales de Newton-Schulz para ortogonalización. Para una matriz de parámetros 2D W con gradiente G_t:

$$X_0 = \frac{G_t}{\|G_t\|_F + \epsilon} \quad \text{(gradiente normalizado)} \tag{76}$$

$$A_k = X_k X_k^\top \quad \text{(Gramiana)} \tag{77}$$

$$B_k = bA_k + cA_k^2 \quad \text{(paso polinomial, coeficientes } b, c \text{ de Newton-Schulz)} \tag{78}$$

$$X_{k+1} = aX_k + B_k X_k \quad \text{(iteración } k = 0, 1, \ldots, 4) \tag{79}$$

$$W_{t+1} = W_t - \eta X_K \quad \text{(actualización con dirección ortogonalizada)} \tag{80}$$

Muon realiza 5 iteraciones de Newton-Schulz para producir una dirección aproximadamente ortogonal, asegurando que las actualizaciones respeten restricciones geométricas. La justificación: las actualizaciones ortogonales preservan las normas durante el entrenamiento, evitando la deriva de norma observada en métodos basados en momento.

Siguiendo la receta híbrida de Shen et al. [103], la actualización ortogonalizada se reserva para las matrices de pesos de capas ocultas (grupos `attention`, `ffn` y otros bloques ocultos). Las tablas de embeddings, los encodings posicionales aprendidos, los heads de salida y todo parámetro no matricial (sesgos, escalas de normalización, kernels de convolución) se entrenan con AdamW desacoplado en lugar de omitirse, usando los valores por grupo de tasa de aprendizaje, weight decay, β_1, β_2 y ϵ de la configuración.

Algorithm 20 Muon (Ortogonalización de Newton-Schulz)

Require: Parámetros iniciales θ, tasa de aprendizaje η, iteraciones Newton-Schulz K
Ensure: Parámetros actualizados θ_T

for cada parámetro P **do**
  if P es una matriz 2D de capa oculta W **then**
    $G \leftarrow \nabla f(W)$ ▷ Gradiente
    $X_0 \leftarrow \frac{G}{\|G\|_F + \epsilon}$ ▷ Normalizar gradiente por norma de Frobenius
    for $k = 0, 1, \ldots, K-1$ **do**
      $A_k \leftarrow X_k X_k^\top$ ▷ Gramiana (costo: $O(mn^2)$ o $O(m^2 n)$ para matrices altas/anchas)
      $B_k \leftarrow (3/2)A_k - (1/2)A_k^2$ ▷ Coeficientes de Newton-Schulz
      $X_{k+1} \leftarrow B_k X_k$ ▷ Paso polinomial
    end for
    $W \leftarrow W - \eta X_K$ ▷ Actualizar con dirección ortogonalizada
  else
    $P \leftarrow \mathrm{AdamW}(P, \nabla f(P); \eta, \beta_1, \beta_2, \epsilon, \lambda)$ ▷ Respaldo híbrido: embeddings, pos, heads, no matriciales
  end if
end forreturn θ_T

Contras.

- Varias multiplicaciones de matrices por iteración de Newton-Schulz; se requieren kernels personalizados eficientes.
- La construcción ortogonalizada está definida solo para matrices 2D; las clases de parámetros restantes dependen del respaldo AdamW y de su calibración de tasa de aprendizaje por grupo.

Ventajas.

- Las actualizaciones ortogonalizadas preservan las normas y evitan la deriva de norma de los métodos basados en momento.
- Resultados prometedores en entrenamiento a gran escala con una geometría de actualización fundamentalmente mejor condicionada.

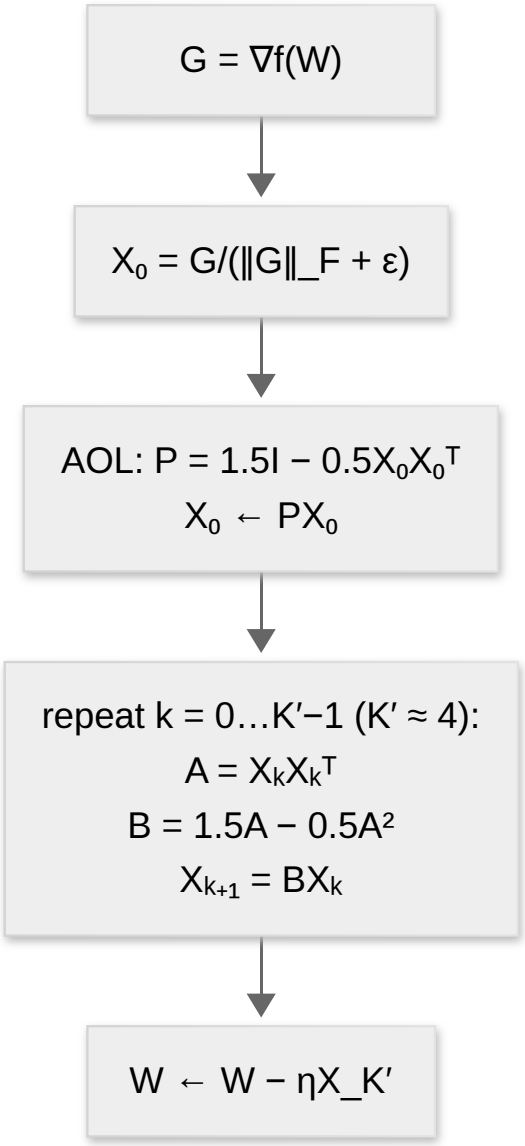


Figura 32: Flujo por paso de Turbo-Muon: precondicionamiento casi ortogonal (AOL) antes de las iteraciones de Newton-Schulz, reduciendo su número.

Turbo-Muon (Ortogonalización Precondicionada con AOL). Turbo-Muon añade un paso de **precondicionamiento casi-ortogonal** (AOL) antes de las iteraciones de Newton-Schulz. El precondicionador $P = 1{,}5I - 0{,}5X_0X_0^\top$ se aplica al gradiente normalizado, reduciendo el número de pasos de ortogonalización requeridos a cuatro mientras preserva la geometría de actualización ortogonalizada. Hereda el enrutamiento híbrido de Muon: solo se ortogonalizan las matrices 2D de capas ocultas, mientras que las tablas de embeddings, los encodings posicionales aprendidos, los heads de salida y los parámetros no matriciales reciben actualizaciones AdamW desacopladas.

Algorithm 21 Turbo-Muon (Ortogonalización Precondicionada con AOL)

Require: Parámetros iniciales θ, tasa de aprendizaje η, iteraciones Newton-Schulz K' (típicamente 4)

Ensure: Parámetros actualizados θ_T

for cada parámetro P **do**
  if P es una matriz 2D de capa oculta W **then**
    $G \leftarrow \nabla f(W)$ ▷ Gradiente
    $X_0 \leftarrow \frac{G}{\|G\|_F+\epsilon}$ ▷ Normalizar gradiente
    AOL (Almost-Orthogonal preconditioner): ▷ Paso de precondicionamiento
    $P \leftarrow (1{,}5I - 0{,}5X_0X_0^\top)$ ▷ Matriz de precondicionamiento
    $X_0 \leftarrow PX_0$ ▷ Inicio precondicionado
    for $k = 0, 1, \ldots, K' - 1$ **do**
      $A_k \leftarrow X_kX_k^\top$
      $B_k \leftarrow (3/2)A_k - (1/2)A_k^2$
      $X_{k+1} \leftarrow B_kX_k$ ▷ Iteraciones reducidas gracias al precondicionamiento AOL
    end for
    $W \leftarrow W - \eta X_{K'}$ ▷ Actualizar con dirección ortogonalizada precondicionada
  else
    $P \leftarrow \text{AdamW}(P, \nabla f(P); \eta, \beta_1, \beta_2, \epsilon, \lambda)$ ▷ Respaldo híbrido: embeddings, pos, heads, no matriciales
  end if
end forreturn θ_T

Contras.

- Sigue siendo intensivo en cómputo; AOL añade una multiplicación de matrices de precondicionamiento.
- Requiere el mismo soporte de kernels personalizados que Muon, y comparte su dependencia del respaldo AdamW para las clases de parámetros no ocultas.

Ventajas.

- AOL reduce el número de iteraciones de Newton-Schulz (típicamente de cinco a cuatro).
- Conserva la geometría de actualización ortogonalizada con menor costo de iteraciones.

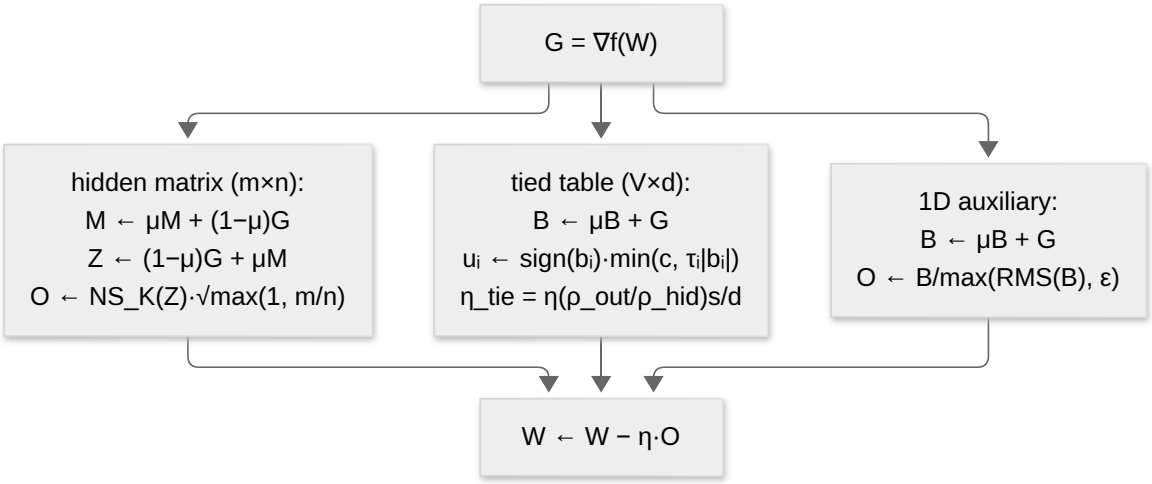


Figura 33: Flujo por paso de AF-Muon: actualización espectral de Muon para matrices ocultas, LMO por fila con tope finito y conciencia de soporte para la tabla de vocabulario atada, y actualizaciones normalizadas por RMS para parámetros auxiliares unidimensionales, todo con un único búfer de primer momento.

AF-Muon (Muon sin AdamW para Vocabulario Atado). AF-Muon es una **extensión de Muon sin AdamW** que elimina el respaldo auxiliar de AdamW de la receta Hybrid Muon asignando a cada clase de parámetro su propia geometría con restricción de norma. Las matrices ocultas conservan la actualización espectral con señal de Nesterov; la tabla de vocabulario, que recibe simultáneamente gradientes dispersos de búsqueda en la entrada y gradientes densos del softmax de salida, se actualiza fila por fila con un *oráculo de minimización lineal con tope finito y conciencia de soporte*; los tensores auxiliares unidimensionales reciben una actualización normalizada por RMS. Ninguna clase de parámetro mantiene estado de segundo momento, de modo que un único búfer de primer momento por parámetro reemplaza el estado de AdamW de la receta híbrida, lo que los autores reportan como aproximadamente un 20 % menos de memoria de estado del optimizador en entrenamiento equiparado.

Para la tabla atada $W \in \mathbb{R}^{V\times d}$ con fila de momento b_i, el oráculo resuelve el problema por fila

$$u_i = \operatorname{sign}(b_i) \odot \min\left(c\,\mathbf{1},\ \tau_i\,|b_i|\right), \tag{81}$$

$$\|u_i\|_2^2 = \min\left(d,\ c^2\left|\operatorname{supp}(b_i)\right|\right), \tag{82}$$

donde la escala por fila τ_i se obtiene por bisección en FP32 sobre fragmentos de filas completas. Las filas cuyo soporte no puede llenar el radio ℓ_2 de fila saturan cada coordenada útil en el tope c, de modo que las filas dispersas de búsqueda se manejan sin inflar las coordenadas ausentes. El tope interpola entre dos extremos conocidos: $c = 1$ recupera la actualización Sign estilo SCION y $c \geq \sqrt{d}$ se reduce a una actualización row-RMS. El tamaño de paso de la tabla es $\eta_{tie} = \eta_M(\rho_{out}/\rho_{hid})\,s/d$; la configuración principal del artículo es $c = 3$ y $s = 0{,}5$ con $\rho_{out}/\rho_{hid} = 60$, y sus diagnósticos reportan el tope finito activo en menos del 0.3 % de las coordenadas aunque afecta a la mayoría de las filas.

Algorithm 22 AF-Muon (Muon sin AdamW para Vocabulario Atado)

Require: Parámetros iniciales θ, LR de matrices η_M, LR de vectores η_{vec}, momento μ, weight decay de matrices λ_M, escala de tabla s, tope c, radios ρ_{out}, ρ_{hid}, iteraciones Newton-Schulz K

Ensure: Parámetros actualizados θ_T

for cada tensor de parámetros W con gradiente G_t **do**
  if W es una matriz oculta en $\mathbb{R}^{m\times n}$ **then**
    $M_t \leftarrow \mu M_{t-1} + (1-\mu)G_t$ ▷ Primer momento
    $Z_t \leftarrow (1-\mu)G_t + \mu M_t$ ▷ Señal de Nesterov
    $O_t \leftarrow \mathrm{NS}_K(Z_t)\sqrt{\max(1, m/n)}$ ▷ Actualización espectral
    $W_t \leftarrow (1-\eta_M\lambda_M)W_{t-1} - \eta_M O_t$
  else if $W \in \mathbb{R}^{V\times d}$ es la tabla de vocabulario atada **then**
    $B_t \leftarrow \mu B_{t-1} + G_t$ ▷ Primer momento sin normalizar
    $\eta_{tie,t} \leftarrow \eta_M(\rho_{out}/\rho_{hid})s/d$
    for cada fila $i = 1, \ldots, V$ **do**
      $u_i \leftarrow L_c\big(\mathrm{fila}_i(B_t)\big)$ ▷ LMO por fila con tope
    end for
    $W_t \leftarrow W_{t-1} - \eta_{tie,t}U$
  else
    $B_t \leftarrow \mu B_{t-1} + G_t$ ▷ Tensor auxiliar 1D
    $O_t \leftarrow B_t / \max(\mathrm{RMS}(B_t), \epsilon)$
    $W_t \leftarrow W_{t-1} - \eta_{vec}O_t$
  end if
end forreturn θ_T

Contras.

- Cada parámetro debe recibir un rol explícito (matriz, tabla atada o vector); un head de salida no atado o una tabla de embeddings adicional no son bloques de tabla atada.
- El oráculo de la tabla requiere bisección por fila con fragmentación de filas completas; el artículo reporta aproximadamente un 1 % de sobrecarga mediana por paso en entrenamiento equiparado.
- La afirmación sobre la tabla atada se establece para modelos con vocabulario atado; no se reclama preentrenamiento a escala frontera ni superioridad universal de los extremos.

Ventajas.

- Sin AdamW: un búfer de primer momento por parámetro y sin estado de segundo momento, reportado como aproximadamente un 20 % menos de memoria de estado del optimizador que Hybrid Muon.
- El tope finito preserva las magnitudes intra-fila mientras acota la concentración de coordenadas, y recupera Sign ($c = 1$) y row-RMS ($c \geq \sqrt{d}$) como casos límite.
- Mejora la pérdida de validación y la perplejidad medias frente a Hybrid Muon y a un extremo Sign estilo SCION en nueve configuraciones de vocabulario atado que abarcan texto, tokens de imagen, proteínas, MoE y modelos encoder-decoder.

AdaMuon (Muon Adaptativo). AdaMuon es una **extensión adaptativa de Muon** que conserva la actualización espectral (polar) y añade normalización de varianza por coordenada. Muon ortogonaliza la matriz de momentum en una actualización puramente direccional, lo que elimina el escalado anisotrópico pero asigna la misma magnitud a todas las coordenadas; AdaMuon aborda el desajuste resultante entre coordenadas ruidosas e informativas siguiendo la varianza por elemento de la dirección *ortogonalizada*, en lugar de la del gradiente crudo o la del momentum. Como el factor polar es globalmente invariante a escala, la transformación de signo aplicada antes de la ortogonalización preserva la dirección de la actualización mientras estabiliza sus magnitudes por elemento; el artículo demuestra que sign es la única aplicación elemental que es libre de escala, consistente con el signo, impar y acotada (Teorema 1). La estimación de segundo momento ve así una señal estable, y su versión normalizada por varianza se reescala a un RMS de 0,2, igualando la escala empírica de actualización de Adam para reutilizar sin ajuste los schedules de tasa de aprendizaje existentes.

Para un peso $W_t \in \mathbb{R}^{n\times m}$, búfer de momentum M_t, momentum β, pasos de Newton–Schulz T y constante pequeña ε:

$$M_t = \beta M_{t-1} + G_t, \tag{83}$$

$$O_t = \mathrm{NewtonSchulz}\big(\mathrm{sign}(M_t), T\big), \tag{84}$$

$$V_t = \beta V_{t-1} + (1-\beta)\, O_t \odot O_t, \tag{85}$$

$$\widehat{O}_t = O_t \oslash \big(\sqrt{V_t} + \varepsilon \mathbf{1}\big), \tag{86}$$

$$\gamma_t = \frac{0{,}2\sqrt{mn}}{\|\widehat{O}_t\|_F}, \tag{87}$$

$$W_{t+1} = W_t - \eta\big(\gamma_t \widehat{O}_t + \lambda W_t\big). \tag{88}$$

El mapa de Newton–Schulz usa $X_0 = \mathrm{sign}(M_t)/\|\,\mathrm{sign}(M_t)\|_F$ y la iteración cúbica $X_k = aX_{k-1} + b(X_{k-1}X_{k-1}^{\mathsf{T}})X_{k-1} + c(X_{k-1}X_{k-1}^{\mathsf{T}})^2 X_{k-1}$ con $(a,b,c) = (3{,}4445, -4{,}7750, 2{,}0315)$ y $T = 5$. No se aplica corrección de sesgo a ninguno de los momentos: un factor constante en V_t se cancela exactamente con $\gamma_t \propto \sqrt{c}$, y un factor constante en M_t se elimina con el mapa polar, $\mathrm{polar}(cM) = \mathrm{polar}(M)$.

Algorithm 23 AdaMuon (Muon Adaptativo)

Require: Pesos iniciales $W_0 \in \mathbb{R}^{n\times m}$, tasa de aprendizaje η, weight decay λ, momentum β, pasos de Newton–Schulz T, constante pequeña ε

Ensure: Pesos actualizados W_T

$M_0 \leftarrow \mathbf{0}$, $V_0 \leftarrow \mathbf{0}$
for cada iteración $t = 1, 2, \ldots$ **do**
  $G_t \leftarrow \nabla_{W_t} \mathcal{L}(W_t)$
  $M_t \leftarrow \beta M_{t-1} + G_t$
  $O_t \leftarrow \mathrm{NewtonSchulz}(\mathrm{sign}(M_t), T)$
  $V_t \leftarrow \beta V_{t-1} + (1-\beta)\, O_t \odot O_t$
  $\widehat{O}_t \leftarrow O_t \oslash (\sqrt{V_t} + \varepsilon \mathbf{1})$
  $\gamma_t \leftarrow 0{,}2\sqrt{mn}/\|\widehat{O}_t\|_F$
  $W_{t+1} \leftarrow W_t - \eta(\gamma_t \widehat{O}_t + \lambda W_t)$
end forreturn W_T

Contras.

- Un búfer de segundo momento adicional por parámetro matricial (dos búferes frente al único búfer de primer momento de Muon), más trabajo por paso de EMA elemental, raíz cuadrada y normalización sobre la actualización.

- La actualización está definida para parámetros bidimensionales; los parámetros unidimensionales (sesgos, escalas de normalización) requieren un optimizador auxiliar en la receta del artículo.
- El objetivo RMS (0,2) es una constante fija y no una cantidad aprendida, por lo que el método asume escalas de actualización tipo Adam al reutilizar schedules existentes.

Ventajas.

- Sin hiperparámetros nuevos: la EMA de segundo momento reutiliza el coeficiente de momentum β de Muon, y el artículo reporta baja sensibilidad para $\beta \in [0{,}8, 0{,}99]$.
- La transformación de signo hace que la estimación de varianza sea lo bastante estable para normalizar, y la alineación RMS cancela cualquier sesgo constante en V_t, por lo que no se necesita corrección de sesgo para ninguno de los dos momentos.
- La alineación RMS iguala la escala de actualización de Adam, lo que permite reutilizar directamente los schedules de tasa de aprendizaje; los autores reportan hasta aproximadamente un 40 % de ganancia en eficiencia de entrenamiento frente a AdamW y mejoras consistentes frente a Muon en GPT-2 (125M–1.5B) y Qwen2.5 (1.5B, 7B).

Dion3 (Ortogonalización Fraccional con Gram-Newton-Schulz). Dion3 [4] es una **revisión de Muon** que reduce el costo del paso de ortogonalización a nivel algorítmico. Reemplaza Newton–Schulz estándar por *Gram Newton–Schulz*, una iteración matemáticamente equivalente que opera sobre la pequeña matriz de Gram simétrica XX^T, y ortogonaliza solo una fracción $f \in (0, 1]$ de las filas del momentum en cada paso, actualizando únicamente esas filas de la matriz de pesos. El *error feedback* decae solo las filas seleccionadas del búfer de momentum, de modo que las filas no seleccionadas siguen acumulándose y terminan siendo elegidas. Con $f = 1$ la actualización se reduce a Muon en aritmética exacta (o a NorMuon con la normalización por filas opcional). Para un peso $W \in \mathbb{R}^{n\times m}$ (operando sobre la dimensión menor, $n \le m$), fracción f, $k = \lceil fn \rceil$, momentum μ, tasa de aprendizaje η y weight decay λ:

$$M_t = M_{t-1} + G_t \quad \text{(acumulación de momentum; amortiguamiento diferido)} \tag{89}$$

$$S_t = \mathrm{topk}_i\big(\|M_{t,i,:}\|_1,\ k\big) \quad \text{(seleccionar las } k \text{ filas mayores)} \tag{90}$$

$$O_t = \mathrm{GramNS}\big(M_t[S_t, :]\big) \quad \text{(ortogonalizar la submatriz } k \times m\text{)} \tag{91}$$

$$W_t = (1 - \eta\lambda)\, W_{t-1}, \qquad W_t[S_t, :] \leftarrow W_t[S_t, :] - \eta\, O_t \tag{92}$$

$$M_t[S_t, :] \leftarrow \mu\, M_t[S_t, :] \quad \text{(error feedback: decaer solo las filas seleccionadas)} \tag{93}$$

Gram Newton–Schulz normaliza $X \leftarrow M_t[S_t, :]/(\|M_t[S_t, :]\|_F + \epsilon)$ con $\epsilon = 10^{-7}$, convierte a `float16`, forma $R_0 = XX^\mathsf{T}$ y $Q_0 = I$, y ejecuta T iteraciones

$$Z_t = b_t R_{t-1} + c_t R_{t-1}^2, \qquad Q_t = Q_{t-1} Z_t + a_t Q_{t-1}, \tag{94}$$

$$(RZ)_t = R_{t-1} Z_t + a_t R_{t-1}, \qquad R_t = Z_t (RZ)_t + a_t (RZ)_t, \tag{95}$$

y devuelve $Q_T X$. Un *reinicio* tras la iteración 2 ($X \leftarrow Q_2 X$, $R_2 \leftarrow XX^\mathsf{T}$, $Q_2 \leftarrow I$) elimina los autovalores negativos espurios que la media precisión introduce en la matriz de Gram; los coeficientes (a_t, b_t, c_t) son las ternas de cinco pasos de Polar Express [3]. El costo baja de $T(3\alpha + 1)n^3$ FLOPs para Newton–Schulz estándar a $(4T + 3\alpha - 3)n^3$ para Gram Newton–Schulz ($\alpha = m/n$), y seleccionar $k = \lceil fn \rceil$ filas reduce la iteración y eleva la relación de aspecto en $1/f$. La variante NorMuon opcional añade un segundo momento por fila $v_i \leftarrow \beta_2 v_i + (1 - \beta_2)\ \mathrm{mean}_j(O_{ij}^2)$ para $i \in S$, reescala $\tilde{O}_{ij} = O_{ij}/(\sqrt{v_i} + \epsilon)$ y renormaliza $\tilde{O} \leftarrow \tilde{O}\, \|O\|_F / \|\tilde{O}\|_F$ en `float32` antes de la actualización.

Algorithm 24 Dion3 (Ortogonalización Fraccional con Retroalimentación de Error)

Require: Matrices W, tasa de aprendizaje η, momentum μ, fracción f, pasos de Gram Newton–Schulz T, ϵ, weight decay λ

Ensure: Matrices actualizadas W_T

for cada matriz de parámetros 2D W con gradiente G **do**
  $M \leftarrow M + G$ ▷ Acumulación de momentum
  $S \leftarrow$ índices de las $k = \lceil fn \rceil$ filas de M con mayor norma ℓ_1 ▷ Selección
  $O \leftarrow \mathrm{GramNS}_T(M[S,:])$ ▷ Ortogonalizar la submatriz seleccionada
  if NorMuon **then**
    $v_i \leftarrow \beta_2 v_i + (1-\beta_2)\,\mathrm{mean}_j(O_{ij}^2)$ para $i \in S$
    $\tilde{O}_{ij} \leftarrow O_{ij}/(\sqrt{v_i} + \epsilon)$; $\quad \tilde{O} \leftarrow \tilde{O}\,\|O\|_F/\|\tilde{O}\|_F$
  end if
  $W \leftarrow (1 - \eta\lambda)W$ ▷ Weight decay en todas las filas
  $W[S,:] \leftarrow W[S,:] - \eta\,\tilde{O}$ ▷ Actualizar solo las filas seleccionadas
  $M[S,:] \leftarrow \mu\, M[S,:]$ ▷ Error feedback
end forreturn W_T

Contras.

- Cada paso actualiza solo una fracción de las filas, por lo que la trayectoria de optimización difiere de Muon; el artículo reporta que el cambio es benigno (Sección 8.1 del reporte Dion3).
- La selección de filas y el error feedback añaden contabilidad top-k y un decaimiento del momentum por filas en cada paso.
- Las mayores ganancias de tiempo de reloj del artículo provienen de kernels GEMM simétricos y comunicación megaagrupada, específicos de hardware y clúster; aquí solo aplican la reducción algorítmica y el único búfer de primer momento.

Ventajas.

- Gram Newton–Schulz es matemáticamente equivalente a Newton–Schulz y reduce los FLOPs de ortogonalización en torno al 55 % en relaciones de aspecto típicas de transformer con cinco iteraciones.
- La selección fraccional de filas reduce la submatriz ortogonalizada a $k \times m$ y eleva su relación de aspecto en $1/f$, potenciando el ahorro de Gram; el artículo reporta pasos de optimizador hasta $6\times$ más rápidos con la misma pérdida.
- Con $f = 1$ la actualización se reduce a Muon (o NorMuon), y basta un único búfer de momentum por parámetro matricial; el error feedback permite que las filas ignoradas participen eventualmente.

Grupo	**Métodos**	**Objetivo Principal**	**Interpretación desde el Estudio**
Línea base clásica	SGD, AdamW, RAdam	estabilidad y referencias base	Definen el piso de comparación para las afirmaciones de optimizadores más nuevos.
Rediseño de momento	Adan, AdEMAMix, MARS, Cautious AdamW	adaptación de primer orden más rápida o segura	Mejor cuando la velocidad de convergencia o la estabilidad ante gradiente ruidoso es la preocupación principal.

Grupo	Métodos	Objetivo Principal	Interpretación desde el Estudio
Simplificación de lotes grandes y programación	LAMB, Schedule-Free AdamW	robustez operacional a escala	Reducen la fragilidad debida al crecimiento del tamaño de lote o la ingeniería de programación.
Eficiencia de memoria	Adafactor, GaLore, APOLLO, APOLLO-Mini, Q-APOLLO, Lion	reducción del estado del optimizador	Más útil cuando la VRAM está dominada por el estado del optimizador en lugar de las activaciones; los métodos de la familia APOLLO reemplazan los momentos densos de AdamW con escalado estructurado proyectado o cuantizado.
Consciente de curvatura	Shampoo, SOAP, Sophia	mejor condicionamiento	Preferir cuando una geometría más rica justifica la sobrecarga de implementación y cómputo.
Orientado a geometría	Muon, Turbo-Muon, AF-Muon, AdaMuon, Dion3	estructura de actualización ortogonalizada	Opciones especializadas para geometría matricial y modelado de representaciones; AdaMuon añade un segundo momento por elemento sobre la dirección ortogonalizada con actualizaciones alineadas por RMS; AF-Muon asigna a la tabla de vocabulario atada una geometría de tabla con tope por fila y a los parámetros unidimensionales una actualización RMS, sin estado auxiliar de AdamW; Dion3 conserva la actualización espectral pero ortogonaliza solo una fracción de las filas del momentum por paso con Gram Newton–Schulz y error feedback.

5. Transformadores Densos, Recurrentes y Aumentados con Memoria

Este anexo sintetiza las familias densas, recurrentes y aumentadas con memoria de transformers. El campo ha evolucionado hacia varios paradigmas principales: atención densa global con variantes, compresión de cabezas clave-valor mediante atención por consultas agrupadas, compresión de estado recurrente con decaimiento, modelos de espacio de estado selectivos, integración numérica de profundidad continua y arquitecturas aumentadas con memoria. Comprender esta taxonomía ilumina las compensaciones fundamentales entre expresividad, costo computacional, huella de memoria y simplicidad de despliegue.

5.1. Atención Densa de Referencia: Estándar y Sigmoide

5.1.1. Atención Softmax Estándar

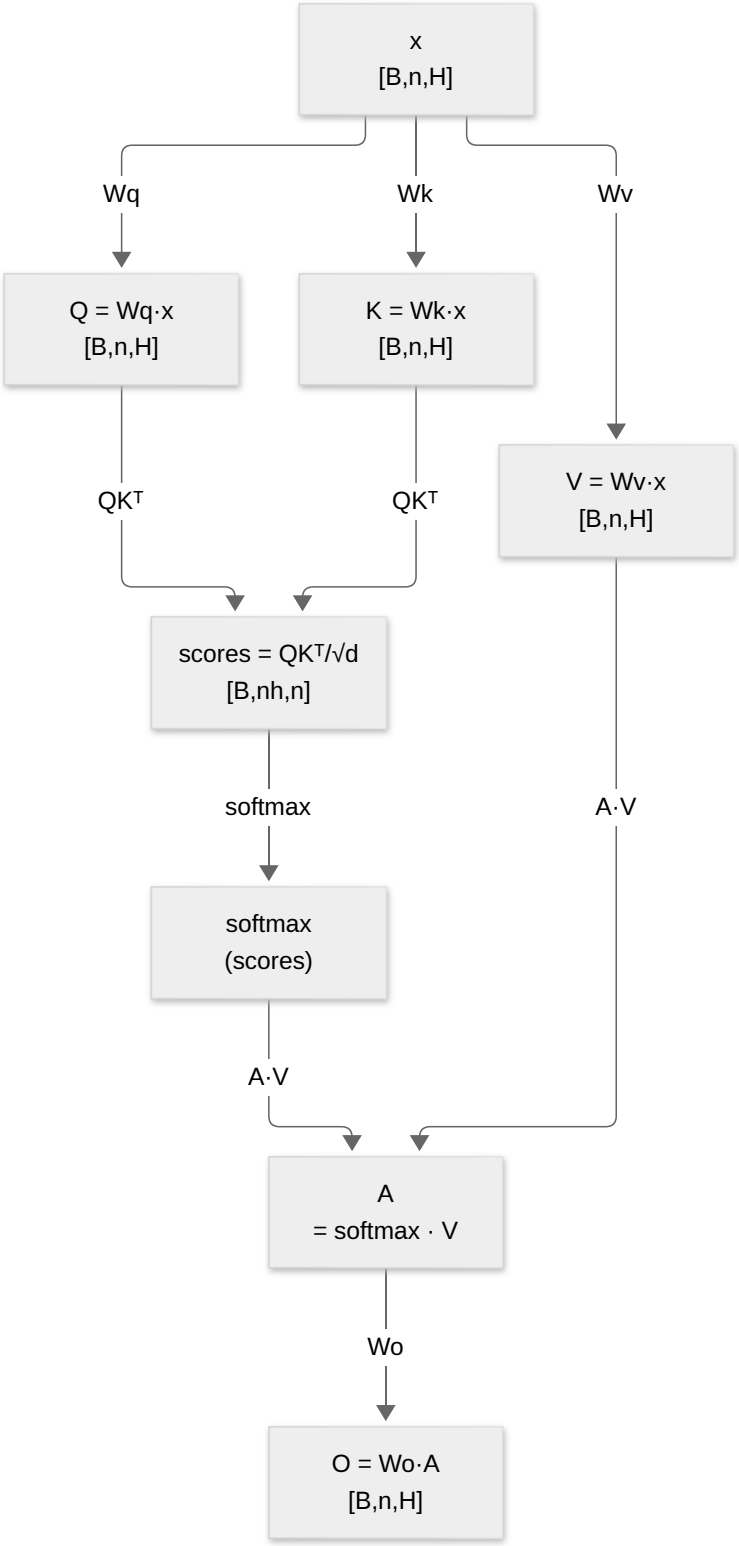


Figura 34: Flujo de datos de la atención softmax estándar: proyecciones Q/K/V paralelas, puntuaciones de producto escalar escaladas, softmax por filas y proyección de salida.

La atención multi-cabeza estándar [113] calcula la similitud de producto punto escalado entre embeddings de tokens:

1: **Entrada:** Matriz de consulta $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Matriz de clave $\mathbf{K} \in \mathbb{R}^{n \times d}$, Matriz de valor $\mathbf{V} \in \mathbb{R}^{n \times d}$
2: puntajes $\leftarrow \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} \in \mathbb{R}^{n \times n}$ ▷ calcular similitudes por pares
3: pesos_atencion $\leftarrow \texttt{softmax}(\text{puntajes}, \text{eje} = 1) \in \mathbb{R}^{n \times n}$ ▷ normalizar sobre las claves
4: **Salida:** $\mathbf{Y} = \text{pesos_atencion} \cdot \mathbf{V} \in \mathbb{R}^{n \times d}$ ▷ combinación ponderada de valores

Para generación autorregresiva, el almacenamiento en caché KV guarda claves y valores pasados para evitar el recálculo $\mathcal{O}(n^2)$. Sin embargo, este caché de crecimiento lineal ocupa $\sim n \cdot d_{\text{oculto}}$ bytes, lo que puede consumir cientos de gigabytes para modelos de miles de millones de parámetros. La atención estándar logra una **expresividad perfecta** dentro de la ventana de contexto—cualquier token puede atender a cualquier otro token con pesos aprendidos—pero paga el precio del cómputo denso.

- **Complejidad de entrenamiento**: $\mathcal{O}(n^2 \cdot d)$ tiempo, $\mathcal{O}(n^2)$ espacio (materialización de la matriz de atención).
- **Complejidad de inferencia**: $\mathcal{O}(n)$ tiempo por token, $\mathcal{O}(n)$ espacio (caché KV).
- **Fortalezas**: Expresividad incomparable; recuperación perfecta del historial; altamente paralelizable.
- **Debilidades**: El cuello de botella cuadrático prohíbe contextos muy largos; el caché KV domina la memoria durante la generación.

5.1.2. Atención Sigmoide

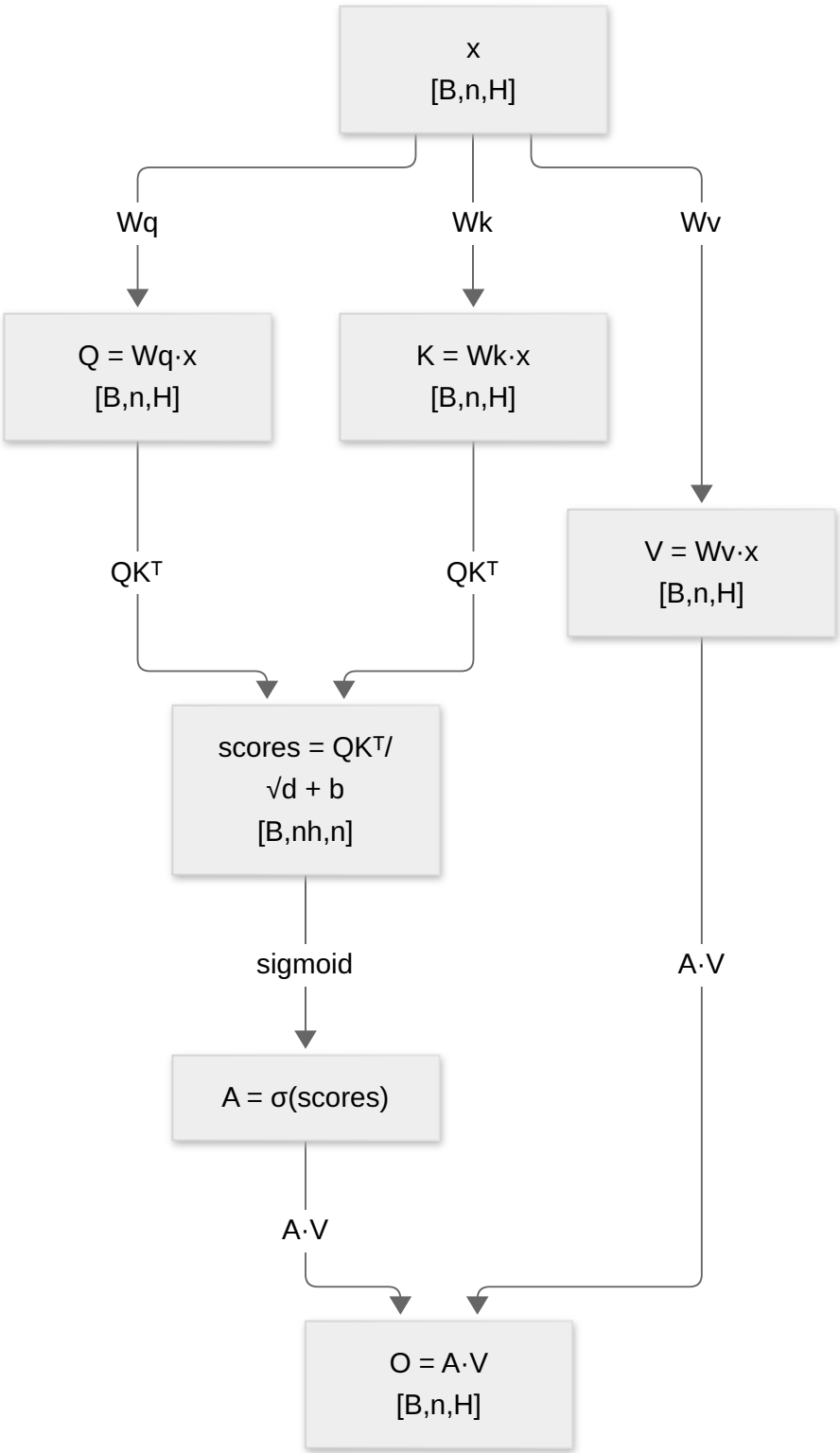


Figura 35: Flujo de datos de la atención sigmoide: una sigmoide elemento a elemento sustituye al softmax por filas, de modo que los pesos de atención no tienen por qué sumar uno.

La atención sigmoide reemplaza el softmax por filas con una activación sigmoide elemento a elemento [97]:

1: **Entrada:** $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ como arriba, sesgo aprendible $\mathbf{b} \in \mathbb{R}^{n \times n}$
2: logits $\leftarrow \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{b}$
3: pesos_atencion $\leftarrow \sigma(\text{logits})$ ▷ sigmoide elemento a elemento, no softmax
4: **Salida:** $\mathbf{Y} = \text{pesos_atencion} \odot \mathbf{V}$

A diferencia de softmax, la sigmoide no impone una distribución de probabilidad (los pesos no necesitan sumar 1), lo que permite una independencia de tokens más fuerte. El análisis teórico mediante mezcla de expertos muestra que la sigmoide alcanza una complejidad muestral superior: convergencia $\mathcal{O}(n^{-0,51})$ para expertos ReLU frente a $\mathcal{O}(n^{-0,24})$ de softmax. Sin embargo, el entrenamiento empírico reveló inestabilidades de gradiente a escala. El remedio es la **norma híbrida**—agregar normalización después de la salida de atención—que estabiliza los gradientes sin sacrificar los beneficios teóricos del enrutamiento elemento a elemento.

- **Complejidad de entrenamiento**: $\mathcal{O}(n^2 \cdot d)$ (idéntico al estándar), pero las operaciones elemento a elemento permiten una aceleración de inferencia del 17 % mediante FlashSigmoid.
- **Complejidad de inferencia**: $\mathcal{O}(n)$ por token con caché KV (asintóticamente igual, pero con factores constantes más bajos).
- **Fortalezas**: Supera la competencia de suma cero; evita la sincronización por filas; implementación eficiente en hardware.
- **Debilidades**: Requiere estabilización cuidadosa (norma híbrida); inestabilidad de entrenamiento a grandes escalas sin pérdida auxiliar.

5.1.3. Atención por Consultas Agrupadas (GQA)

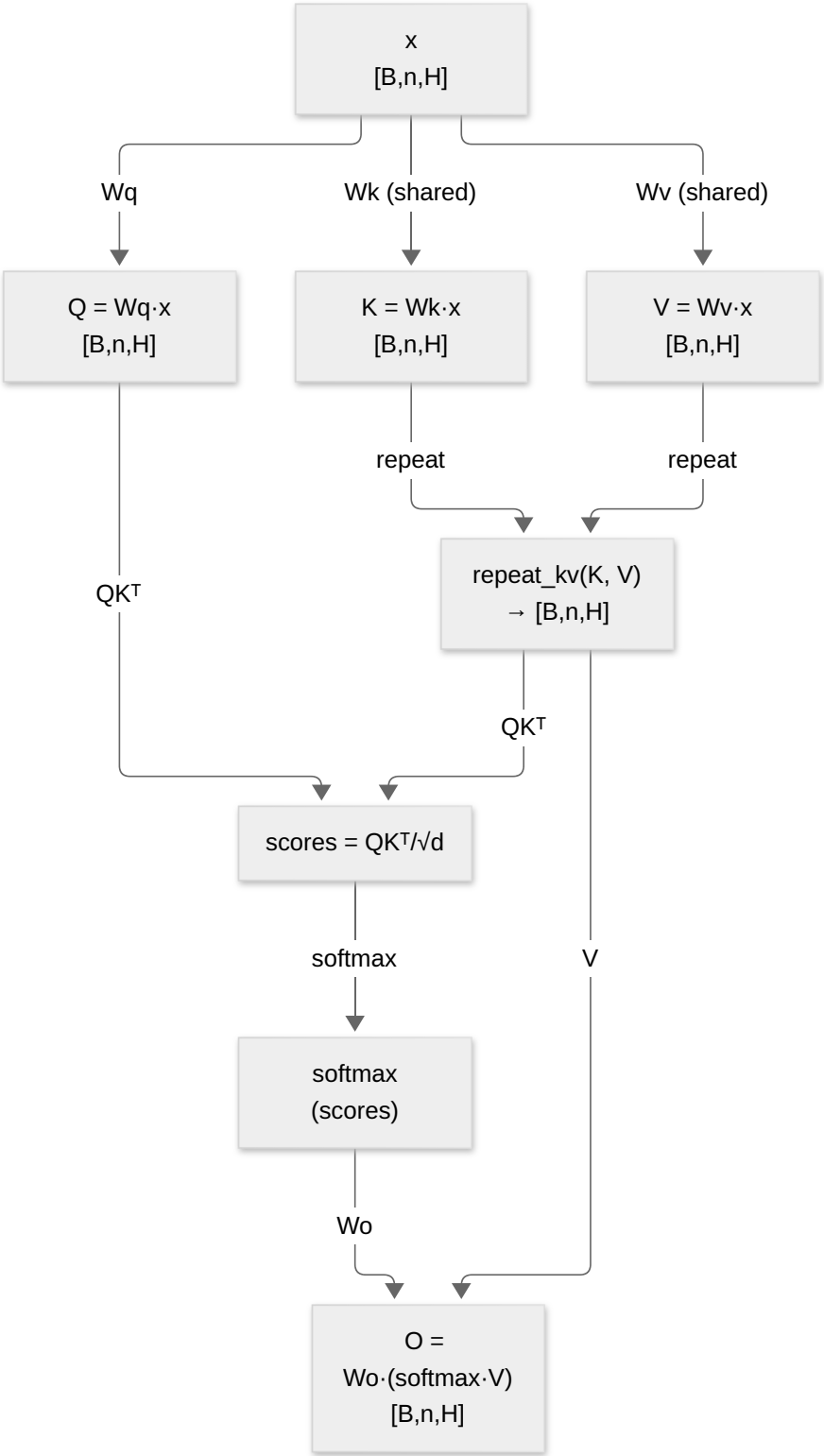


Figura 36: Flujo de datos de la atención por consultas agrupadas: las claves y los valores se proyectan a menos cabezas, se expanden mediante repetición entrelazada y se comparten en cada grupo de cabezas de consulta.

La atención por consultas agrupadas (GQA) [2] tiende un puente entre la atención multi-cabeza (MHA) y la atención multi-consulta (MQA) al particionar las cabezas de consulta en grupos, cada uno compartiendo una sola cabeza clave-valor. Si h es el número total de cabezas de atención, h_k es el número de cabezas clave-valor, y $G = h/h_k$ es el tamaño del grupo, entonces cada grupo de G cabezas de consulta comparte una cabeza KV:

1: **Entrada:** $\mathbf{Q} \in \mathbb{R}^{n \times h \cdot d_h}$, $\mathbf{K} \in \mathbb{R}^{n \times h_k \cdot d_h}$, $\mathbf{V} \in \mathbb{R}^{n \times h_k \cdot d_h}$
2: $\mathbf{K}_{\text{completo}} \leftarrow \texttt{repeat_interleave}(\mathbf{K}, G)$ ▷ expandir de h_k a h cabezas
3: $\mathbf{V}_{\text{completo}} \leftarrow \texttt{repeat_interleave}(\mathbf{V}, G)$
4: puntajes $\leftarrow \frac{\mathbf{Q}\mathbf{K}_{\text{completo}}^{\top}}{\sqrt{d_h}} \in \mathbb{R}^{n \times n}$ ▷ producto punto estándar tras expansión
5: pesos_atencion $\leftarrow \texttt{softmax}(\text{puntajes}, \text{eje} = 1)$
6: **Salida:** $\mathbf{Y} = \text{pesos_atencion} \cdot \mathbf{V}_{\text{completo}} \in \mathbb{R}^{n \times h \cdot d_h}$

GQA interpola suavemente entre MHA ($h_k = h$, $G = 1$) y MQA ($h_k = 1$, $G = h$). La ventaja clave es un balance ajustable entre la huella del caché KV y la capacidad representacional. Cada cabeza KV adicional consume $2 \cdot d_h \cdot n$ bytes extra en el caché, por lo que reducir h_k produce ahorros proporcionales de memoria.

- **Complejidad de entrenamiento**: $\mathcal{O}(n^2 \cdot h \cdot d_h)$ (idéntico a MHA tras la expansión de cabezas).
- **Complejidad de inferencia**: $\mathcal{O}(n)$ por token, espacio de caché KV $\mathcal{O}(h_k \cdot n \cdot d_h)$.
- **Fortalezas**: Balance flexible de cabezas; probado a escala (Llama 2 y 3 usan $h_k = 8$ cabezas KV con $h = 32$ cabezas de consulta); reduce el caché KV en factor $G\times$; soporta inferencia rápida con FlashAttention.
- **Debilidades**: Menos cabezas KV pueden reducir la capacidad representacional en modelos pequeños; requerir que h_k divida a h restringe las opciones arquitectónicas.

5.2. Arquitecturas Recurrentes y Retentivas

5.2.1. Redes Retentivas (RetNet)

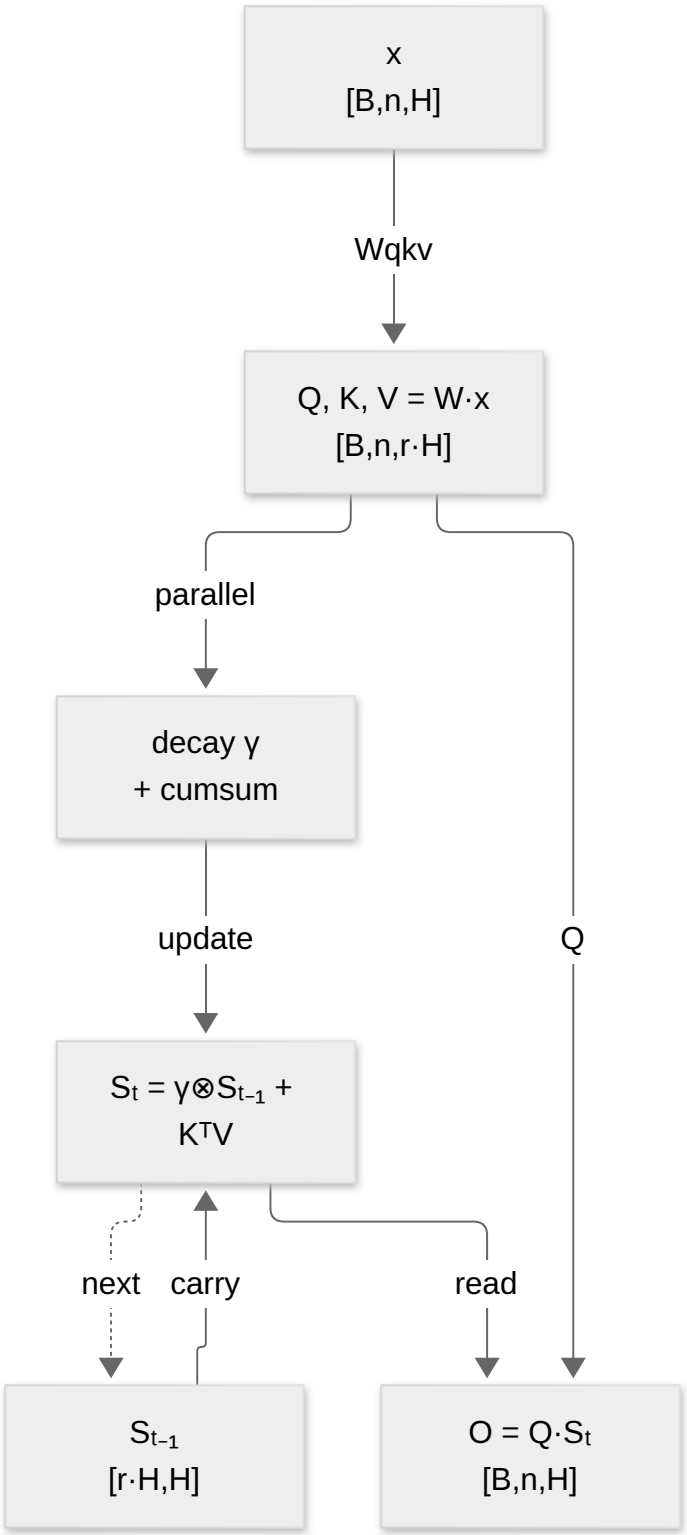


Figura 37: Flujo de datos de RetNet: proyecciones Q/K/V paralelas alimentan una recurrencia de estado con decaimiento; el estado se propaga entre pasos y la salida lo lee con la consulta.

RetNet [109] unifica tres paradigmas de cómputo: entrenamiento paralelo, inferencia recurrente y despliegue por fragmentos. Su innovación central es el **mecanismo de retención**, que utiliza una matriz de decaimiento exponencial fija para modelar la importancia temporal:

Forma paralela (entrenamiento):
matriz_decaimiento$[i,j] \leftarrow \gamma^{i-j}$ para $i \geq j$, si no 0 ▷ decaimiento exponencial causal
matriz_decaimiento$[i,j] \leftarrow 0$ para $i < j$ ▷ enmascaramiento causal
$\mathbf{Y}_{\text{paralelo}} \leftarrow (\mathbf{QK}^\top \odot \text{matriz_decaimiento})\mathbf{V}$ ▷ multiplicación elemento a elemento con decaimiento

Forma recurrente (inferencia):
for $t = 1, \ldots, n$ **do**
    $\mathbf{s}_t \leftarrow \gamma \mathbf{s}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$ ▷ actualización de estado con decaimiento
    $\mathbf{y}_t \leftarrow \mathbf{q}_t \mathbf{s}_t$ ▷ salida mediante interacción consulta-estado
end for

El escalar de decaimiento $\gamma \in (0,1)$ controla la ventana temporal. RetNet utiliza **retención multi-escala** con diferentes valores de γ por cabeza (ej., $\gamma = 1 - 2^{-5}, 1 - 2^{-6}, \ldots$), permitiendo dependencias a corto y largo plazo simultáneamente. El modo recurrente por fragmentos divide las secuencias en fragmentos, procesa cada fragmento en paralelo y enhebra un estado recurrente entre fragmentos.

- **Complejidad de entrenamiento**: $\mathcal{O}(n^2 \cdot d)$ (paralela), o $\mathcal{O}(n \cdot c \cdot d)$ (recurrente por fragmentos con tamaño de fragmento c).
- **Complejidad de inferencia**: $\mathcal{O}(1)$ por token, $\mathcal{O}(d^2)$ espacio de estado (matriz fija).
- **Fortalezas**: Inferencia en tiempo constante; triple paradigma de cómputo; consolidación multi-escala.
- **Debilidades**: El decaimiento fijo impone un sesgo inductivo rígido; puede truncar patrones de largo alcance aprendidos.

5.2.2. Mamba: Modelos de Espacio de Estado Selectivos

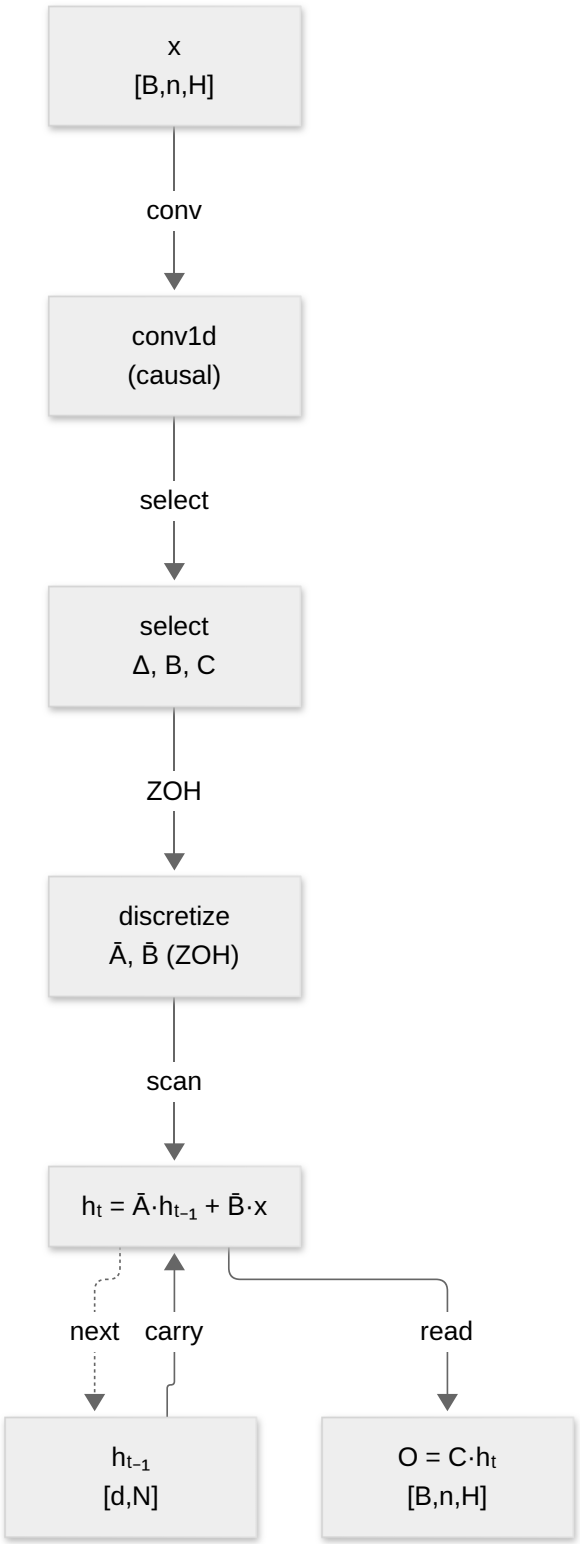


Figura 38: Flujo de datos de Mamba: una convolución causal alimenta selecciones de Δ, B y C dependientes de la entrada, la discretización ZOH y un escaneo selectivo de espacio de estados cuyo estado se propaga entre pasos.

Mamba [43] enmarca la recurrencia como un sistema dinámico de tiempo continuo con parámetros dependientes de la entrada, logrando tanto entrenamiento lineal como inferencia en tiempo constante:

Dinámica continua: $h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t), \quad y(t) = \mathbf{C}h(t)$
Discretización con tamaño de paso Δ_t:
$\bar{\mathbf{A}}_t \leftarrow \exp(\Delta_t \mathbf{A})$
$\bar{\mathbf{B}}_t \leftarrow (\Delta_t \mathbf{A})^{-1}(\exp(\Delta_t \mathbf{A}) - \mathbf{I})\Delta_t \mathbf{B}_t$
Actualización recurrente:
for $t = 1, \ldots, n$ **do**
    $\Delta_t \leftarrow \text{softplus}(\text{Lineal}(x_t))$ ▷ tamaño de paso dependiente de la entrada
    $\mathbf{B}_t \leftarrow \text{Lineal}(x_t), \quad \mathbf{C}_t \leftarrow \text{Lineal}(x_t)$ ▷ matrices de proyección dependientes de la entrada
    $h_t \leftarrow \bar{\mathbf{A}}_t h_{t-1} + \bar{\mathbf{B}}_t x_t$ ▷ transición de estado
    $y_t \leftarrow \mathbf{C}_t h_t$ ▷ proyección de salida
end for

La innovación crítica es que $\mathbf{A}$ (la matriz del sistema) *no* depende de la entrada, pero Δ_t, $\mathbf{B}_t$ y $\mathbf{C}_t$ sí, haciendo que el sistema sea **variante en el tiempo**. Esta selectividad permite al modelo *ignorar* información irrelevante estableciendo Δ_t cerca de cero, creando efectivamente una compuerta. Un algoritmo de escaneo paralelo consciente del hardware implementa la recurrencia eficientemente en GPUs fusionando el cómputo dentro de SRAM, evitando el costoso ancho de banda de HBM.

- **Complejidad de entrenamiento**: $\mathcal{O}(n \cdot d)$ mediante escaneo consciente del hardware (lineal en la longitud de la secuencia).
- **Complejidad de inferencia**: $\mathcal{O}(1)$ por token, $\mathcal{O}(d)$ estado (vector oculto).
- **Fortalezas**: Logra entrenamiento lineal e inferencia constante simultáneamente; práctico en secuencias largas (millones de tokens).
- **Debilidades**: La compresión del vector de estado puede debilitar la copia exacta y la recuperación asociativa densa en comparación con la atención completa.

5.3. Transformers de Profundidad Continua: Integración EDO

5.3.1. Transformer EDO

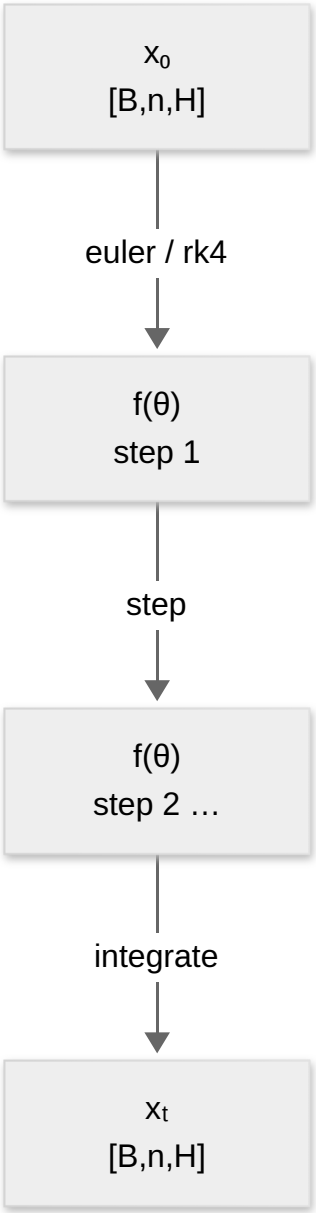


Figura 39: Flujo de datos del bloque transformer EDO: el estado oculto se integra mediante uno o más pasos de Euler/RK4 antes de leer la salida.

El Transformer EDO [130] interpreta la profundidad de la red como integración numérica de un sistema dinámico continuo, utilizando solucionadores Runge-Kutta de orden superior para reducir el error de truncamiento:

1: **Formulación continua:** $\frac{dh(t)}{dt} = f_\theta(h(t), t)$ donde f_θ es la subred del transformer
2: **Aproximación discreta Runge-Kutta-4:**
3: $k_1 \leftarrow f_\theta(h_t, t)$
4: $k_2 \leftarrow f_\theta(h_t + \frac{1}{2}k_1, t + \frac{1}{2}\Delta t)$
5: $k_3 \leftarrow f_\theta(h_t + \frac{1}{2}k_2, t + \frac{1}{2}\Delta t)$
6: $k_4 \leftarrow f_\theta(h_t + k_3, t + \Delta t)$

7: $h_{t+1} \leftarrow h_t + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ ▷ combinación ponderada

En lugar de las conexiones residuales simples de Euler $h_{t+1} = h_t + f_\theta(h_t)$, el bloque RK4 calcula cuatro evaluaciones intermedias y las combina con los pesos clásicos de RK4. Para evitar gradientes que se desvanecen, la arquitectura introduce **compuertas aprendidas** que interpolan entre aproximaciones intermedias:

1: $g \leftarrow \sigma(\text{Lineal}([k_1, k_2, k_3, k_4]))$ ▷ compuerta aprendible
2: $h_{t+1} \leftarrow h_t + g \cdot k_1 + (1 - g) \cdot k_2$ ▷ refinamiento interpolado

Esta formulación reduce el número efectivo de parámetros mediante el uso compartido de pesos—la misma f_θ se evalúa múltiples veces—proporcionando un refinamiento de trayectoria más rico.

- **Complejidad de entrenamiento**: $\mathcal{O}(k \cdot n^2 \cdot d)$ donde k es el orden RK (4 para RK4).
- **Complejidad de inferencia**: $\mathcal{O}(k \cdot n)$ por token (mayor sobrecarga constante).
- **Fortalezas**: Precisión significativamente mayor en tareas de generación (BLEU de última generación); eficiente en parámetros mediante uso compartido de pesos.
- **Debilidades**: Alto costo de cómputo por paso; la latencia de inferencia aumenta por un factor constante k; requiere compuertas complejas para la estabilidad.

5.4. Memoria en Tiempo de Prueba: Titans

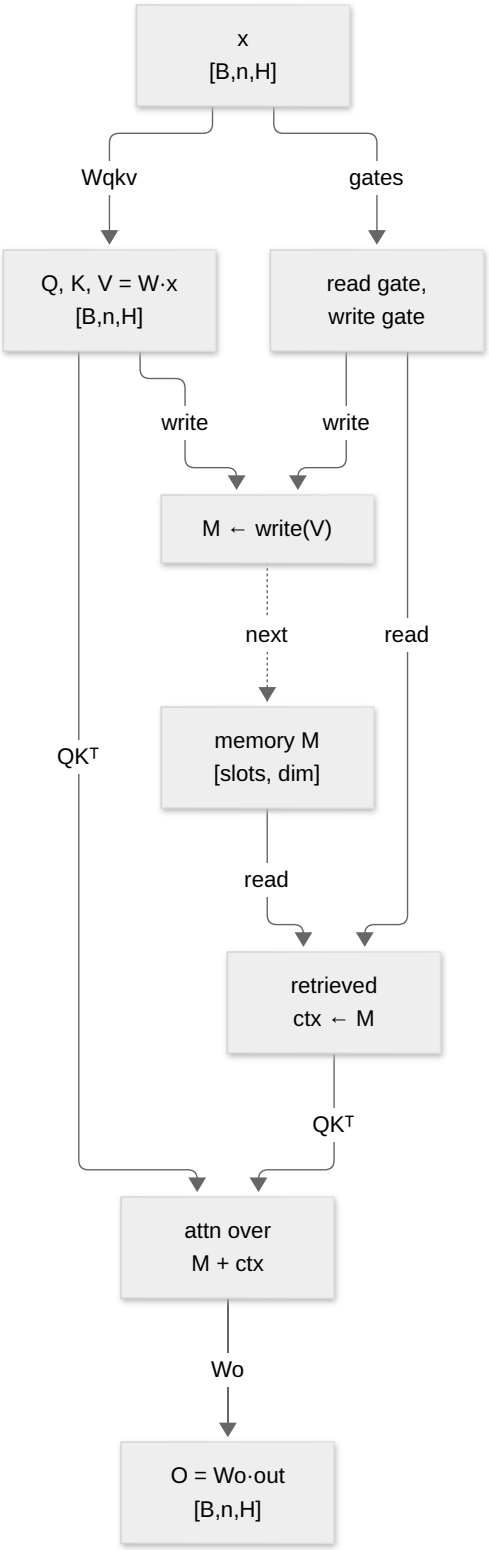


Figura 40: Flujo de datos de la memoria en tiempo de prueba de Titans: la atención densa Q/K/V opera sobre una ventana local mientras compuertas de lectura y escritura actualizan una memoria asociativa que se propaga entre pasos.

La arquitectura Titans [11] introduce una dimensión ortogonal: en lugar de pesos estáticos, el modelo mantiene una memoria aprendible que se *actualiza durante la inferencia* basada en una señal impulsada por sorpresa:

Atención local a corto plazo: Aplicar atención estándar o dispersa dentro de una ventana de contexto fija c
$y_t^{\text{local}} \leftarrow \text{Atencion}(q_t, k_{[t-c:t]}, v_{[t-c:t]})$
Señal de actualización de memoria (sorpresa):
$S_t \leftarrow \eta_t S_{t-1} - \theta_t \nabla_\ell(\mathcal{M}_{t-1}; x_t)$ ▷ sorpresa como gradiente de la pérdida respecto a la memoria
$\mathcal{M}_t \leftarrow (1-\alpha_t)\mathcal{M}_{t-1} + S_t$ ▷ memoria actualizada mediante EMA
Recuperación de memoria a largo plazo:
$y_t^{\text{memoria}} \leftarrow \mathcal{M}_t^*(q_t)$ ▷ consultar módulo de memoria para información recuperada
Combinación de salida:
$y_t \leftarrow y_t^{\text{local}} + \text{compuerta}(y_t^{\text{memoria}})$ ▷ combinar memoria local y recuperada

El módulo de memoria $\mathcal{M}$ se actualiza literalmente durante el paso hacia adelante calculando gradientes de una pérdida asociativa y aplicando pasos de SGD con momento. Las tasas de decaimiento $\eta_t, \theta_t, \alpha_t$ son en sí mismas dependientes de la entrada, permitiendo al modelo cambiar de paradigma de memoria cuando el contexto cambia.

- **Complejidad de entrenamiento**: Aproximadamente $\mathcal{O}(c^2 + n \cdot f)$ donde c es la ventana local y f es la sobrecarga de actualización de memoria.
- **Complejidad de inferencia**: $\mathcal{O}(c^2)$ atención local más $\mathcal{O}(1)$ recuperación de memoria por token.
- **Fortalezas**: Maneja longitudes de contexto extremas; permite recuperación asociativa real; la memoria se adapta a la entrada.
- **Debilidades**: Significativamente más complejo; la inferencia incluye cálculos de gradientes; mayor sobrecarga de coordinación.

5.5. Despacho de Mezclador Guiado por Patrón

El código base selecciona un mezclador por bloque a partir del `layer_pattern` configurado. La lógica de despacho enruta los bloques dispersos en tiempo de inferencia (`fasa_attn`, `sparge_attn`) a atención estándar densa mientras el modelo está en modo entrenamiento, y a su implementación dispersa durante la evaluación; el resto de bloques se enruta a la implementación de familia correspondiente.

El campo exhibe una progresión clara a lo largo de dos ejes: (i) **eficiencia computacional**, pasando de $O(n^2)$ a $O(n)$ u $O(1)$, y (ii) **adaptabilidad de memoria**, pasando de pesos estáticos a modelos dinámicos actualizados en tiempo de prueba. La atención estándar sigue siendo la línea base de expresividad; Mamba y RetNet representan la frontera práctica de eficiencia; Titans introduce un eje de innovación ortogonal (aprendizaje en tiempo de prueba). La elección de arquitectura refleja la restricción fundamental de ingeniería: expresividad versus costo de despliegue. La familia de atención latente, que comprime el caché KV en un latente de rango bajo, se cubre en el Apéndice 6.

6. Familias de Atención Latente y de Rango Bajo

Este anexo cubre la familia de mecanismos de atención de compresión latente / de rango bajo de KV. Estas arquitecturas comprimen el estado clave–valor por token en un vector latente de rango bajo que es la única cantidad materializada en el caché KV durante la decodificación, luego reconstruyen las claves y valores completos de todas las cabezas al vuelo. La familia unifica

Algorithm 25 Paso Adelante del Mezclador Guiado por Patrón (Conceptual)

Require: Estados ocultos H, patrón P, índice de capa ℓ

$m \leftarrow P[\ell \bmod |P|]$
if $m \in \{\texttt{fasa_attn}, \texttt{sparge_attn}\}$ y el modelo está en modo entrenamiento **then**
    $H \leftarrow$ softmax-attention(H) ▷ atención densa sustituta, mismas proyecciones
else if $m = \texttt{standard_attn}$ **then**
    $H \leftarrow$ softmax-attention(H)
else if $m = \texttt{sigmoid_attn}$ **then**
    $H \leftarrow$ sigmoid-attention(H)
else if $m \in \{\texttt{retnet}, \texttt{retnet_attn}\}$ **then**
    $H \leftarrow$ retention(H)
else if $m = \texttt{mamba}$ **then**
    $H \leftarrow$ selective-ssm(H)
else if $m = \texttt{ode}$ **then**
    $H \leftarrow$ rk-step(H)
else if m es una clave de atención dispersa **then**
    $H \leftarrow$ sparse-attention-family(H)
else if m es una clave de atención con compuerta **then**
    $H \leftarrow$ gated-attention-family(H)
else
    $H \leftarrow$ memory-augmented-attn(H)
end if
return H

diseños de consulta agrupada, latente, factorizado y latente temporal bajo un cuello de botella de rango bajo común.

6.1. Atención Latente Multi-Cabeza (MLA)

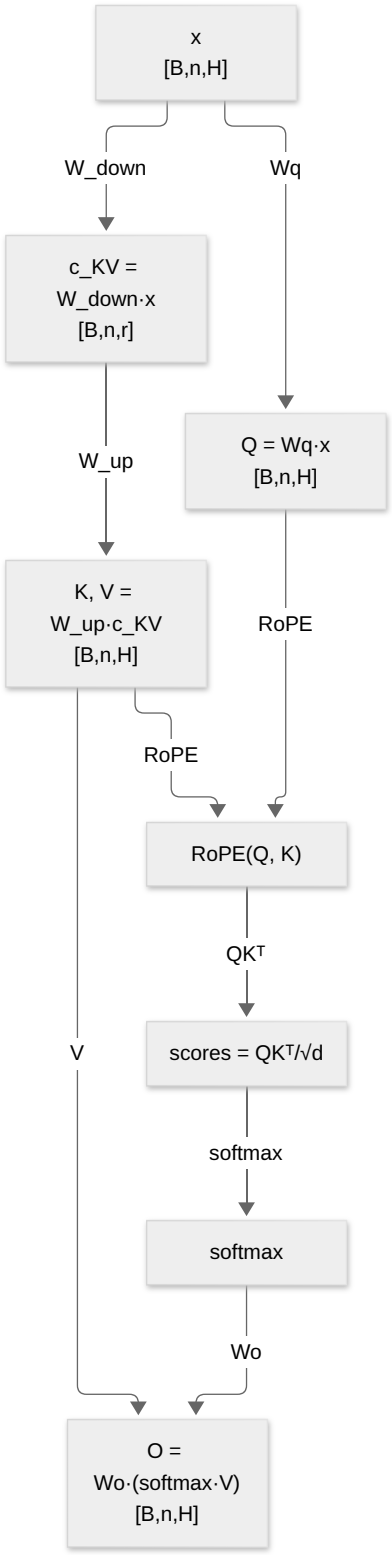


Figura 41: Flujo de datos de la atención latente multi-cabeza: las claves y los valores se proyectan hacia abajo a un latente de rango bajo, se proyectan de vuelta y se atienden con la consulta tras RoPE.

La Atención Latente Multi-Cabeza (MLA, por sus siglas en inglés) [80] comprime el estado clave–valor por token en un único vector latente de rango bajo c_{KV} de rango r_{kv}, el cual es la única cantidad materializada en el caché KV durante la decodificación. Un par de matrices de up-proyección reconstruye las claves y valores completos de todas las cabezas al vuelo, mientras que RoPE se aplica a los tensores de consulta y clave descomprimidos para preservar la estructura posicional. Los autores muestran que el ancho latente óptimo de Pareto se sitúa cerca de $r_{kv} = d_{\text{oculto}}/2$, punto en el cual MLA reduce a la mitad el caché KV pero iguala la calidad de la atención multi-cabeza (MHA) en modelos pequeños.

6.1.1. Formulación Matemática

$$c_{KV} = W_{DKV}\, x \in \mathbb{R}^{r_{kv}}, \qquad k = W_{UK}\, c_{KV}, \qquad v = W_{UV}\, c_{KV}, \qquad q = W_Q\, x.$$

$$\tilde{q} = \text{RoPE}(q), \quad \tilde{k} = \text{RoPE}(k), \qquad \text{atencion} = \texttt{softmax}\left(\frac{\tilde{q}\,\tilde{k}^\top}{\sqrt{d_h}}\right) v.$$

6.1.2. Pseudocódigo Algorítmico

Entrada: estado oculto $x \in \mathbb{R}^{n\times d}$, rango r_{kv}, proyecciones $W_{DKV}, W_{UK}, W_{UV}, W_Q$
$c_{KV} \leftarrow W_{DKV}\, x$ ▷ down-proyección a latente de rango r_{kv} (en caché en inferencia)
$k \leftarrow W_{UK}\, c_{KV}, \quad v \leftarrow W_{UV}\, c_{KV}$ ▷ up-proyección a cabezas completas
$q \leftarrow W_Q\, x$
$\tilde{q} \leftarrow \text{RoPE}(q), \quad \tilde{k} \leftarrow \text{RoPE}(k)$ ▷ aplicar embeddings rotatorios
pesos $\leftarrow \texttt{softmax}(\tilde{q}\,\tilde{k}^\top/\sqrt{d_h})$
Salida: $\mathbf{Y} = \text{pesos} \cdot v$

6.1.3. Características Clave

- **Complejidad de entrenamiento**: $\mathcal{O}(n^2 \cdot d)$ (atención completa sobre cabezas descomprimidas).
- **Complejidad de inferencia**: $\mathcal{O}(n)$ por token, espacio de caché KV $\mathcal{O}(r_{kv} \cdot n)$ (sólo latente).
- **Entrenable**: totalmente entrenable; r_{kv} es un hiperparámetro (óptimo de Pareto $\approx d/2$).
- **Fortalezas**: reduce a la mitad el caché KV frente a MHA con calidad equivalente; compatible con RoPE; cuello de botella latente limpio.
- **Limitaciones**: la up-proyección añade FLOPs de decodificación; el rango latente es un presupuesto global fijo compartido entre cabezas.

6.2. Atención Latente por Consultas Agrupadas (GQLA)

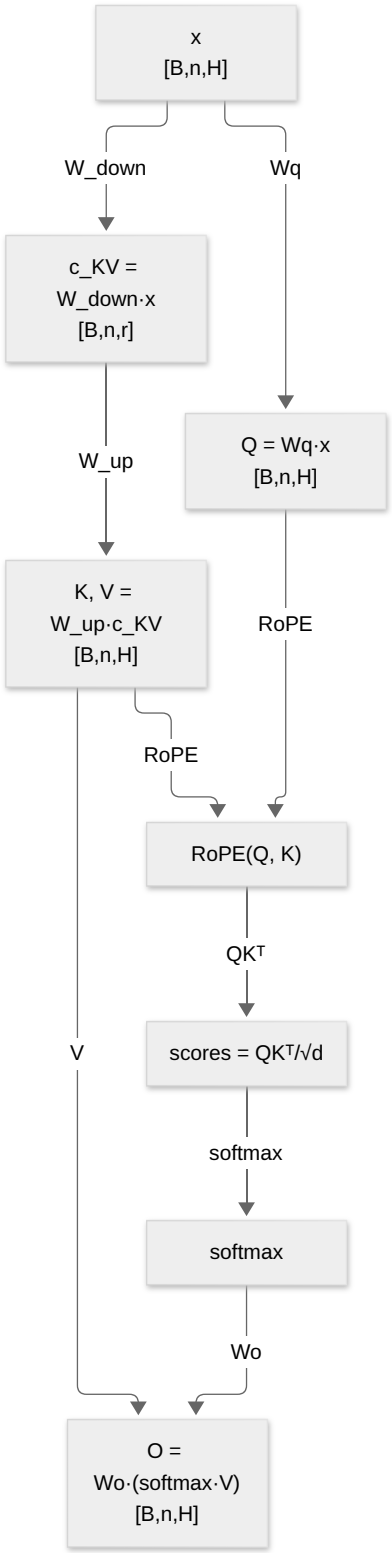


Figura 42: Flujo de datos de la atención latente por consultas agrupadas: la caché latente compartida se proyecta hacia arriba a cabezas de clave/valor y se combina con consultas agrupadas.

La Atención Latente por Consultas Agrupadas (GQLA, por sus siglas en inglés) [81] es una modificación mínima de MLA que expone dos rutas de decodificación algebraicamente equivalentes sobre los mismos pesos entrenados: una ruta MQA-absorb (que fusiona la up-proyección en la consulta, óptima en hardware tipo H100) y una ruta GQA-expanded (que materializa claves/valores completos, óptima en hardware H20 / predicción multi-token). Las dos rutas son matemáticamente idénticas porque la up-proyección es lineal, de modo que $\texttt{softmax}(q_{\text{absorbido}} \cdot c_{KV}^{\top}) = \texttt{softmax}((q \cdot W_{UK}^{\top}) \cdot c_{KV}^{\top})$, permitiendo un despacho adaptativo al hardware sin reentrenamiento.

6.2.1. Formulación Matemática

Mismo latente que MLA:

$$c_{KV} = W_{DKV}\, x, \qquad k = W_{UK}\, c_{KV}, \qquad v = W_{UV}\, c_{KV}, \qquad q = W_Q\, x.$$

$$\text{Ruta A (MQA-absorb): atencion} = \texttt{softmax}\left(\frac{(q\, W_{UK}^{\top})\, c_{KV}^{\top}}{\sqrt{d_h}}\right)(W_{UV}\, c_{KV}).$$

$$\text{Ruta B (GQA-expanded): atencion} = \texttt{softmax}\left(\frac{q\, k^{\top}}{\sqrt{d_h}}\right) v, \quad k = W_{UK} c_{KV},\ v = W_{UV} c_{KV}.$$

6.2.2. Pseudocódigo Algorítmico

Entrada: x, pesos $W_{DKV}, W_{UK}, W_{UV}, W_Q$, bandera de hardware hw $\in \{\text{H100}, \text{H20}\}$
$c_{KV} \leftarrow W_{DKV}\, x$ ▷ latente compartido, en caché
$q \leftarrow W_Q\, x$
if hw == H100 **then** ▷ ruta MQA-absorb
    $q_{\text{abs}} \leftarrow q\, W_{UK}^{\top}$ ▷ absorber up-proyección en la consulta
    pesos $\leftarrow \texttt{softmax}(q_{\text{abs}}\, c_{KV}^{\top} / \sqrt{d_h})$
    $\mathbf{Y} \leftarrow$ pesos $\cdot\, (W_{UV}\, c_{KV})$
else ▷ ruta GQA-expanded
    $k \leftarrow W_{UK}\, c_{KV}, \quad v \leftarrow W_{UV}\, c_{KV}$
    pesos $\leftarrow \texttt{softmax}(q\, k^{\top} / \sqrt{d_h})$
    $\mathbf{Y} \leftarrow$ pesos $\cdot\, v$
end if
Salida: Y

6.2.3. Características Clave

- **Complejidad de entrenamiento**: $\mathcal{O}(n^2 \cdot d)$, idéntica a MLA.
- **Complejidad de inferencia**: $\mathcal{O}(n)$ por token, estado latente $\mathcal{O}(r_{kv} \cdot n)$; ruta elegida por hardware.
- **Entrenable**: se entrena una sola vez; las dos rutas de decodificación reutilizan los mismos pesos.
- **Fortalezas**: decodificación adaptativa al hardware sin pérdida de calidad; la equivalencia algebraica garantiza consistencia.
- **Limitaciones**: mismas compensaciones de rango latente que MLA; la selección de ruta es una heurística de despliegue.

6.3. Atención Multi-Cabeza de Rango Bajo (MLRA)

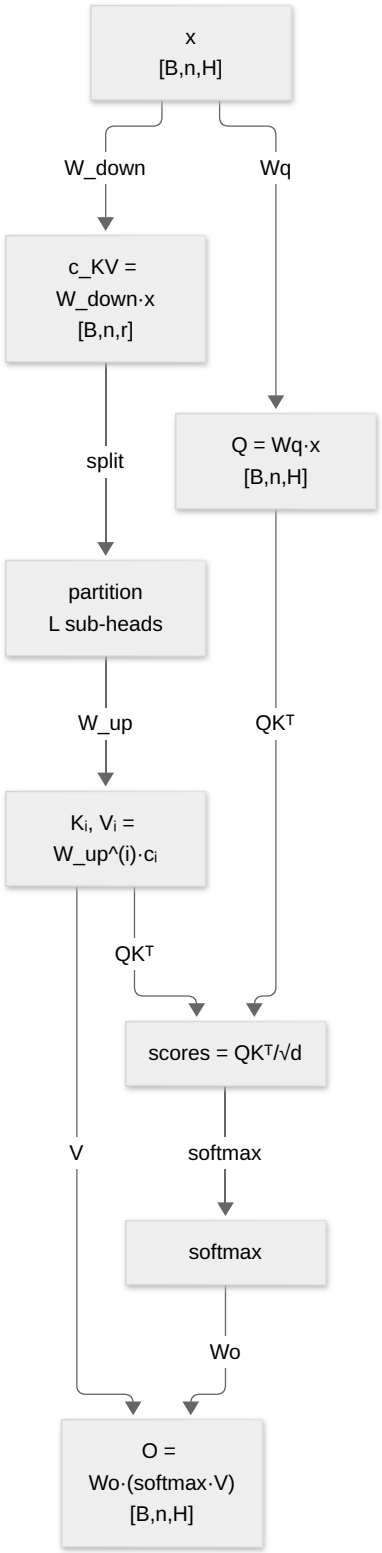


Figura 43: Flujo de datos de la atención multi-cabeza de rango bajo: el latente se particiona entre sub-cabezas, cada una con su propia proyección ascendente, antes de la atención.

La Atención Multi-Cabeza de Rango Bajo (MLRA, por sus siglas en inglés) [74] refactoriza el latente único de MLA en L sub-cabezas disjuntas cada una de rango r/L, de modo que el caché KV puede particionarse entre L dispositivos para una decodificación tensor-paralela de 4 vías (cada dispositivo carga sólo $1/L$ del caché). Las up-proyecciones por sub-cabeza reconstruyen la porción correspondiente de las cabezas completas y se concatenan, logrando una aceleración de decodificación de 2.8× sobre MLA preservando el presupuesto latente global.

6.3.1. Formulación Matemática

$$c_{KV} = [c_1, c_2, \ldots, c_L], \quad c_i \in \mathbb{R}^{r/L}, \qquad c_{KV} = W_{DKV}\, x.$$

$$k_i = W_{UK}^{(i)}\, c_i, \quad v_i = W_{UV}^{(i)}\, c_i, \qquad k = [k_1; \ldots; k_L], \quad v = [v_1; \ldots; v_L].$$

$$\text{atencion} = \texttt{softmax}\left(\frac{q\, k^\top}{\sqrt{d_h}}\right) v.$$

6.3.2. Pseudocódigo Algorítmico

Entrada: x, número de sub-cabezas L, proyecciones por sub-cabeza $W_{UK}^{(i)}, W_{UV}^{(i)}$
$c_{KV} \leftarrow W_{DKV}\, x = [c_1, \ldots, c_L]$ ▷ particionar latente en L bloques
for $i = 1, \ldots, L$ (en paralelo entre L dispositivos) **do**
    $k_i \leftarrow W_{UK}^{(i)}\, c_i, \quad v_i \leftarrow W_{UV}^{(i)}\, c_i$
end for
$k \leftarrow \texttt{concat}(k_1, \ldots, k_L), \quad v \leftarrow \texttt{concat}(v_1, \ldots, v_L)$
pesos $\leftarrow \texttt{softmax}(q\, k^\top / \sqrt{d_h})$
Salida: $\mathbf{Y} =$ pesos $\cdot\, v$

6.3.3. Características Clave

- **Complejidad de entrenamiento**: $\mathcal{O}(n^2 \cdot d)$ (atención completa sobre cabezas concatenadas).
- **Complejidad de inferencia**: $\mathcal{O}(n)$ por token, estado $\mathcal{O}(r_{kv} \cdot n)$ particionado entre L dispositivos.
- **Entrenable**: totalmente entrenable; L controla la granularidad de partición.
- **Fortalezas**: aceleración de decodificación de 2.8×; particionamiento limpio tensor-paralelo de 4 vías del caché; preserva el presupuesto latente total.
- **Limitaciones**: requiere que L divida a r_{kv}; más matrices de proyección que MLA; all-reduce inter-dispositivo en la salida.

6.4. Atención Tucker

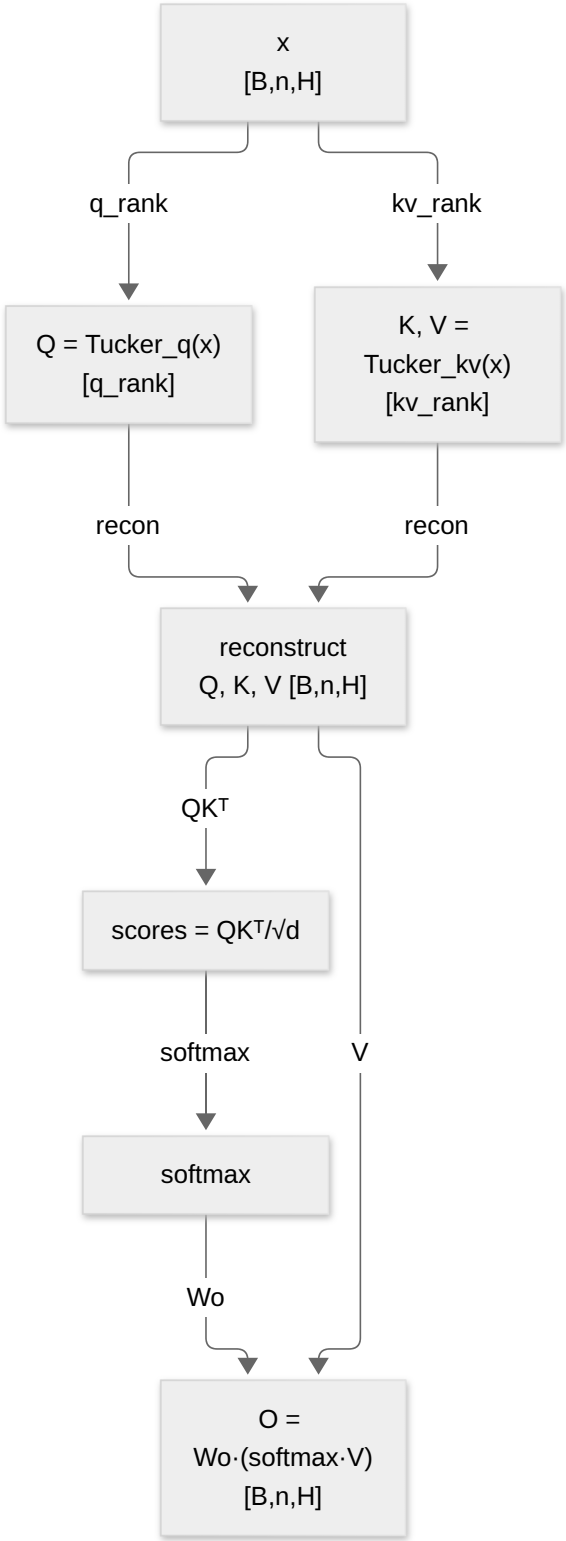


Figura 44: Flujo de datos de la atención Tucker: las consultas y las claves/valores se proyectan mediante factores de Tucker y se reconstruyen antes del producto escalar escalado.

La Atención Tucker [65] unifica MHA, GQA y MLA bajo una única factorización estilo Tucker de los tensores de pesos de consulta/clave/valor con rangos independientes $q_{\text{rank}}, k_{\text{rank}}, v_{\text{rank}}$. Se recupera MHA cuando todos los rangos igualan el tamaño oculto, GQA cuando $k_{\text{rank}} = v_{\text{rank}} < d$, y MLA cuando $k_{\text{rank}} = v_{\text{rank}}$ es igual al rango latente. Como la factorización es una repas lineal de pesos, el método es totalmente compatible con FlashAttention y RoPE, y produce un orden de magnitud menos parámetros para métricas comparables.

6.4.1. Formulación Matemática

$$\mathbf{Q} = W_{Q,\text{core}}\, W_{Q,\text{factor}}\, x, \qquad \mathbf{K} = W_{K,\text{core}}\, W_{K,\text{factor}}\, x, \qquad \mathbf{V} = W_{V,\text{core}}\, W_{V,\text{factor}}\, x.$$

$$\text{atencion} = \texttt{softmax}\left(\frac{\mathbf{Q}\,\mathbf{K}^{\top}}{\sqrt{d_h}}\right)\mathbf{V}.$$

Casos especiales: MHA $\Leftrightarrow$ $q_{\text{rank}} = k_{\text{rank}} = v_{\text{rank}} = d$; GQA $\Leftrightarrow$ $k_{\text{rank}} = v_{\text{rank}} < d$; MLA $\Leftrightarrow$ $k_{\text{rank}} = v_{\text{rank}} = r_{kv}$.

6.4.2. Pseudocódigo Algorítmico

Entrada: x, rangos $q_{\text{rank}}, k_{\text{rank}}, v_{\text{rank}}$, matrices core/factor
$\mathbf{Q} \leftarrow W_{Q,\text{core}}\, W_{Q,\text{factor}}\, x$ ▷ consulta factorizada de rango q_{rank}
$\mathbf{K} \leftarrow W_{K,\text{core}}\, W_{K,\text{factor}}\, x$ ▷ clave factorizada de rango k_{rank}
$\mathbf{V} \leftarrow W_{V,\text{core}}\, W_{V,\text{factor}}\, x$ ▷ valor factorizado de rango v_{rank}
Aplicar RoPE a $\mathbf{Q}, \mathbf{K}$ si se usa codificación posicional
pesos $\leftarrow \texttt{softmax}(\mathbf{Q}\,\mathbf{K}^{\top}/\sqrt{d_h})$ ▷ compatible con FlashAttention
Salida: $\mathbf{Y} =$ pesos $\cdot\, \mathbf{V}$

6.4.3. Características Clave

- **Complejidad de entrenamiento**: $\mathcal{O}(n^2 \cdot d)$ sobre los tensores reconstruidos.
- **Complejidad de inferencia**: $\mathcal{O}(n)$ por token; tamaño de caché gobernado por k_{rank}.
- **Entrenable**: los rangos son hiperparámetros; la factorización es una repas de pesos directa.
- **Fortalezas**: unifica MHA/GQA/MLA; reducción de parámetros de un orden de magnitud con métricas equivalentes; compatible con FlashAttention/RoPE.
- **Limitaciones**: dos matmuls por proyección añaden latencia; la selección de rango es un problema de ajuste por tarea.

6.5. Atención de Cabezas Entrelazadas (IHA)

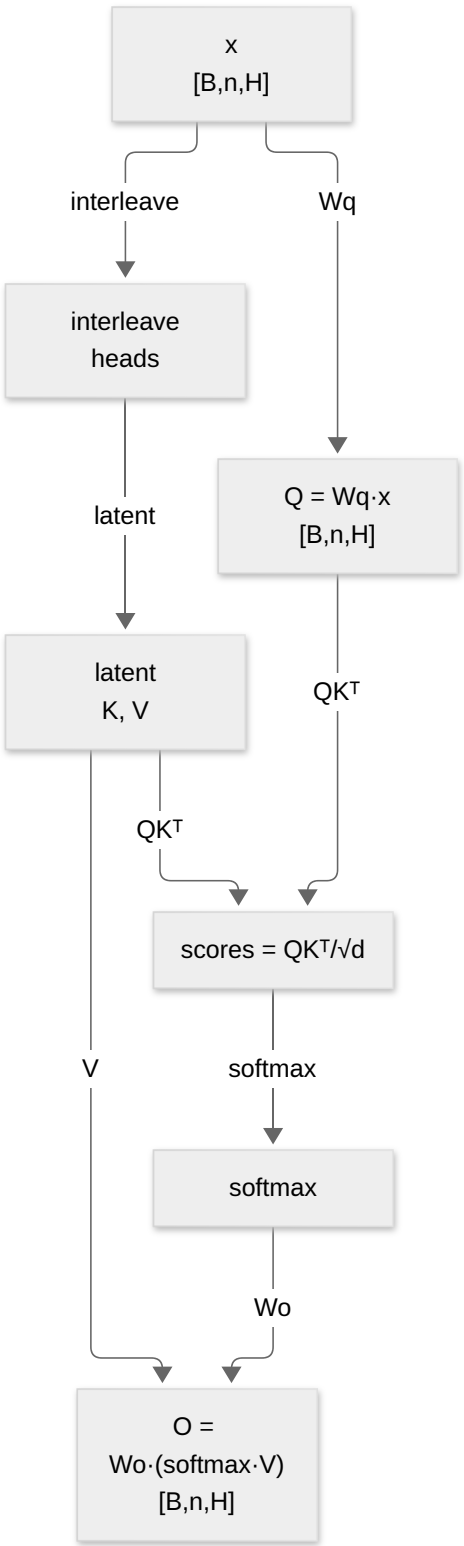


Figura 45: Flujo de datos de la atención de cabezas entrelazadas: las cabezas se entrelazan en una representación latente de clave/valor que la consulta atiende.

La Atención de Cabezas Entrelazadas (IHA, por sus siglas en inglés) [31] construye P pseudo-cabezas por cabeza original (típicamente $P = H$) donde cada consulta/clave/valor pseudo es una combinación lineal aprendida de todas las H cabezas originales. Esto induce hasta P^2 patrones de atención distintos por cabeza con una sobrecarga de $\mathcal{O}(H^2P)$. Teóricamente, IHA necesita sólo $\Theta(\sqrt{k}\,n^2)$ parámetros frente a $\Theta(k\,n^2)$ de MHA en la tarea Polinomial; empíricamente entrega +10–20 % en RULER de recuperación multi-clave, +5,8 % en GSM8K y +2,8 % en MATH-500.

6.5.1. Formulación Matemática

$$q_{\text{pseudo}}[p] = \sum_{h=1}^{H} \text{mix}_q[p,h]\, q[h], \quad k_{\text{pseudo}}[p] = \sum_{h} \text{mix}_k[p,h]\, k[h], \quad v_{\text{pseudo}}[p] = \sum_{h} \text{mix}_v[p,h]\, v[h].$$

$$\text{atencion}_p = \texttt{softmax}\left(\frac{q_{\text{pseudo}}[p]\, k_{\text{pseudo}}[p]^\top}{\sqrt{d_h}}\right) v_{\text{pseudo}}[p], \qquad \mathbf{Y}[h] = \frac{1}{P}\sum_{p} \text{atencion}_p[h].$$

6.5.2. Pseudocódigo Algorítmico

Entrada: por cabeza $q[h], k[h], v[h]$, matrices de mezcla $\text{mix}_q, \text{mix}_k, \text{mix}_v \in \mathbb{R}^{P\times H}$
for $p = 1, \ldots, P$ **do**
  $q_p \leftarrow \sum_h \text{mix}_q[p,h]\, q[h]$ ▷ mezcla aprendida entre cabezas
  $k_p \leftarrow \sum_h \text{mix}_k[p,h]\, k[h]$, $v_p \leftarrow \sum_h \text{mix}_v[p,h]\, v[h]$
  $\text{atencion}_p \leftarrow \texttt{softmax}(q_p\, k_p^\top/\sqrt{d_h})\, v_p$
end for
Salida: $\mathbf{Y}[h] \leftarrow \frac{1}{P}\sum_p \text{atencion}_p[h]$ ▷ promediar salidas pseudo-cabeza por cabeza

6.5.3. Características Clave

- **Complejidad de entrenamiento**: $\mathcal{O}(n^2 \cdot H \cdot P)$ con $P \approx H$, es decir $\mathcal{O}(n^2 \cdot H^2)$.
- **Complejidad de inferencia**: $\mathcal{O}(n)$ por token con caché KV; sobrecarga extra de mezcla $\mathcal{O}(H^2P)$.
- **Entrenable**: las matrices de mezcla se aprenden; P es un hiperparámetro.
- **Fortalezas**: eficiencia de parámetros $\Theta(\sqrt{k})$ en Polinomial; grandes ganancias en recuperación y razonamiento.
- **Limitaciones**: costo de mezcla cuadrático en cabezas; más patrones de atención a computar; latencia añadida con H grande.

6.6. Atención laTenT por Cabezas Agrupadas (GTA)

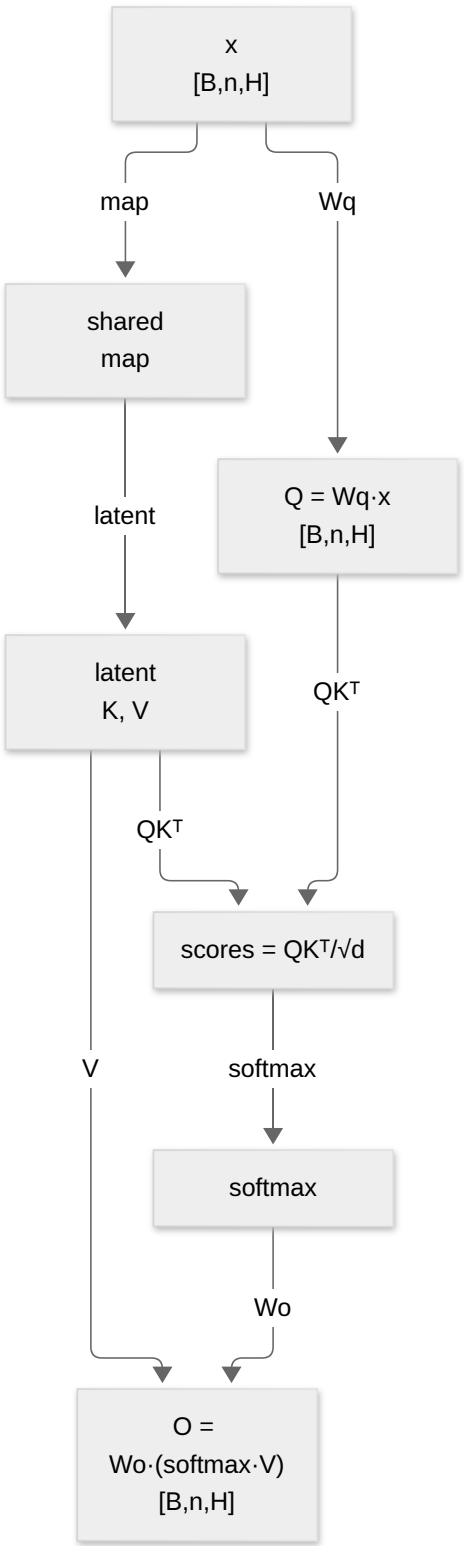


Figura 46: Flujo de datos de la atención latente por cabezas agrupadas: un mapa compartido produce el estado latente de clave/valor que consumen las cabezas agrupadas.

La Atención laTenT por Cabezas Agrupadas (GTA, por sus siglas en inglés) [108] combina dos compresiones: (1) un *mapa de atención compartido* — un tensor de puntajes de atención por grupo de cabezas, reutilizado entre todas las cabezas del grupo, reduciendo el caché de claves; y (2) un *decodificador de valores no lineal* — el caché de valores se comprime a un latente mediante una down-proyección y se reconstruye por una up-proyección con compuerta SiLU antes de la proyección de salida. GTA reduce los FLOPs de atención hasta 62.5 % frente a GQA, encoge el caché KV hasta 70 %, y produce una aceleración de inferencia de 2× de extremo a extremo.

6.6.1. Formulación Matemática

$$q_g = \texttt{mean}(q[\text{grupo } g]), \quad k_g = \texttt{mean}(k[\text{grupo } g]), \quad \text{atencion}_g = \texttt{softmax}\left(\frac{q_g\, k_g^\top}{\sqrt{d_h}}\right) \text{ (reutilizado en el grupo).}$$

$$v_{\text{latente}} = W_{DV}\, x, \qquad v = \text{SiLU}(W_{UV}\, v_{\text{latente}}), \qquad \mathbf{Y} = \text{atencion}_g \cdot v.$$

6.6.2. Pseudocódigo Algorítmico

Entrada: q, k por cabeza, oculto x, partición de grupos $\{g\}$
$v_{\text{latente}} \leftarrow W_{DV}\, x$ ▷ comprimir valores a latente (en caché)
$v \leftarrow \text{SiLU}(W_{UV}\, v_{\text{latente}})$ ▷ decodificación no lineal de valores
for cada grupo g **do**
    $q_g \leftarrow \texttt{mean}(q[g]), \quad k_g \leftarrow \texttt{mean}(k[g])$ ▷ consultas/claves compartidas por grupo
    $\text{atencion}_g \leftarrow \texttt{softmax}(q_g\, k_g^\top / \sqrt{d_h})$
    $\mathbf{Y}[g] \leftarrow \text{atencion}_g \cdot v[g]$ ▷ un mapa reutilizado para todas las cabezas en g
end for
Salida: Y

6.6.3. Características Clave

- **Complejidad de entrenamiento**: $\mathcal{O}(n^2 \cdot G \cdot d_h)$ donde G es el número de grupos (frente a H cabezas en GQA).
- **Complejidad de inferencia**: $\mathcal{O}(n)$ por token; caché de valores $\mathcal{O}(r_v \cdot n)$, caché de claves reducido por compartición de grupo.
- **Entrenable**: la asignación de grupo y el rango latente son entrenables/hiperparámetros.
- **Fortalezas**: hasta 62.5 % de reducción de FLOPs de atención, 70 % de encogimiento de caché KV, 2× de aceleración de extremo a extremo.
- **Limitaciones**: el mapa compartido reduce la diversidad por cabeza; el decodificador no lineal añade latencia; el agrupamiento debe elegirse con cuidado.

6.7. Atención Latente Temporal Multi-Cabeza (MTLA)

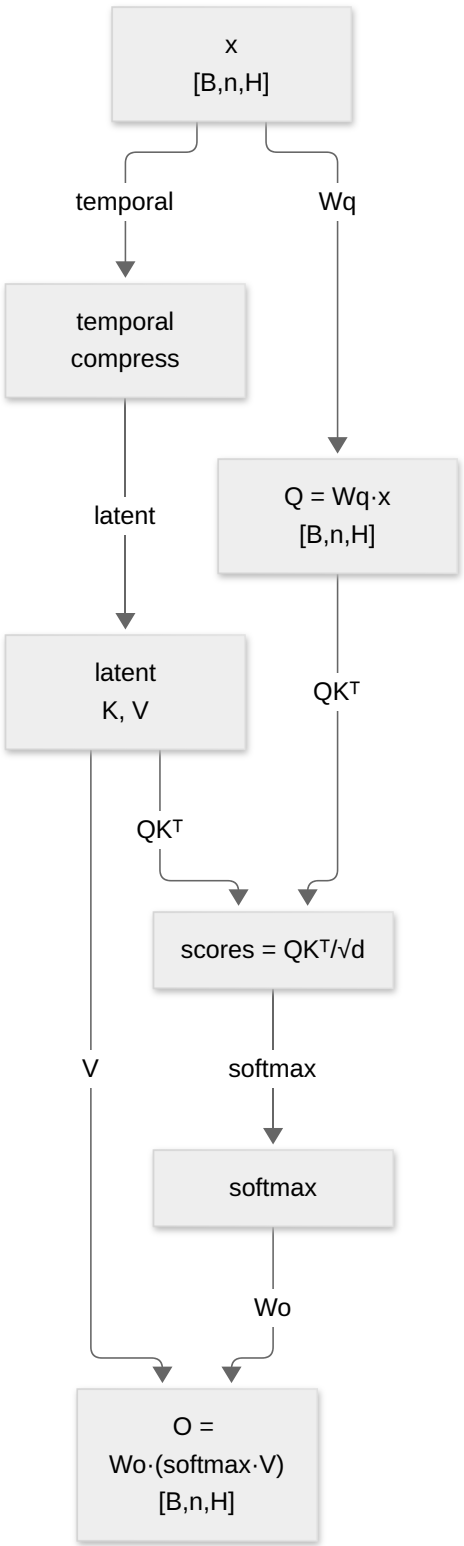


Figura 47: Flujo de datos de la atención latente temporal multi-cabeza: una etapa de compresión temporal produce el estado latente de clave/valor que atiende la consulta.

La Atención Latente Temporal Multi-Cabeza (MTLA, por sus siglas en inglés) [26] extiende MLA a lo largo del eje temporal: una hyper-red fusiona dinámicamente m vectores temporalmente adyacentes del caché KV en un único slot, reduciendo la longitud temporal por un factor de m. Una máscara causal consciente del stride mantiene el entrenamiento paralelo consistente con la inferencia autorregresiva, produciendo una aceleración de decodificación de 5.3× y una reducción de memoria GPU de 8.3× frente a MHA en traducción de voz En–De.

6.7.1. Formulación Matemática

$$c_{KV}^{(t)} = W_{DKV}\, x^{(t)} \in \mathbb{R}^{r_{kv}} \text{ (latente por token).}$$

$$\bar{c}_{KV}^{(j)} = \sum_{i=0}^{m-1} \alpha_i\, c_{KV}^{(sj+i)}, \qquad \alpha_i = \texttt{sigmoid}(\text{HyperNet}(c_{KV}^{(sj+i)})),$$

donde s es el stride y m la ventana de fusión. Las consultas atienden sobre slots fusionados $\bar{c}_{KV}$ con una máscara causal consciente del stride M_{stride}:

$$\text{atencion} = \texttt{softmax}\left(\frac{q\,\bar{k}^{\top}}{\sqrt{d_h}} + M_{\text{stride}}\right)\bar{v}, \quad \bar{k} = W_{UK}\,\bar{c}_{KV},\ \bar{v} = W_{UV}\,\bar{c}_{KV}.$$

6.7.2. Pseudocódigo Algorítmico

Entrada: latentes por token $c_{KV}^{(t)}$, ventana de fusión m, stride s
Fase de fusión (hyper-red):
for cada slot fusionado $j = 0, 1, \ldots$ **do**
    $\alpha_i \leftarrow \texttt{sigmoid}(\text{HyperNet}(c_{KV}^{(sj+i)}))$ para $i = 0, \ldots, m-1$
    $\bar{c}_{KV}^{(j)} \leftarrow \sum_{i=0}^{m-1} \alpha_i\, c_{KV}^{(sj+i)}$ ▷ promedio con compuerta sobre la ventana
end for
$\bar{k} \leftarrow W_{UK}\,\bar{c}_{KV}, \quad \bar{v} \leftarrow W_{UV}\,\bar{c}_{KV}$
pesos $\leftarrow \texttt{softmax}(q\,\bar{k}^{\top}/\sqrt{d_h} + M_{\text{stride}})$ ▷ máscara causal consciente del stride
Salida: $\mathbf{Y} = \text{pesos} \cdot \bar{v}$

6.7.3. Características Clave

- **Complejidad de entrenamiento**: $\mathcal{O}(n^2 \cdot d/m)$ sobre la secuencia fusionada (longitud $\lceil n/m \rceil$) más costo de la hyper-red.
- **Complejidad de inferencia**: $\mathcal{O}(n/m)$ por token sobre slots fusionados; estado $\mathcal{O}(r_{kv} \cdot n/m)$.
- **Entrenable**: la hyper-red y la ventana de fusión m se aprenden/eliges; la máscara consciente del stride garantiza consistencia.
- **Fortalezas**: aceleración de decodificación de 5.3×, reducción de memoria GPU de 8.3× frente a MHA; compresión temporal ortogonal a la compresión de rango.
- **Limitaciones**: la ventana de fusión puede difuminar detalle temporal fino; la hyper-red añade cómputo; el diseño de la máscara no es trivial.

6.8. Comparación y Síntesis Arquitectónica

6.9. Atención Convolucional Comprimida (CCA)

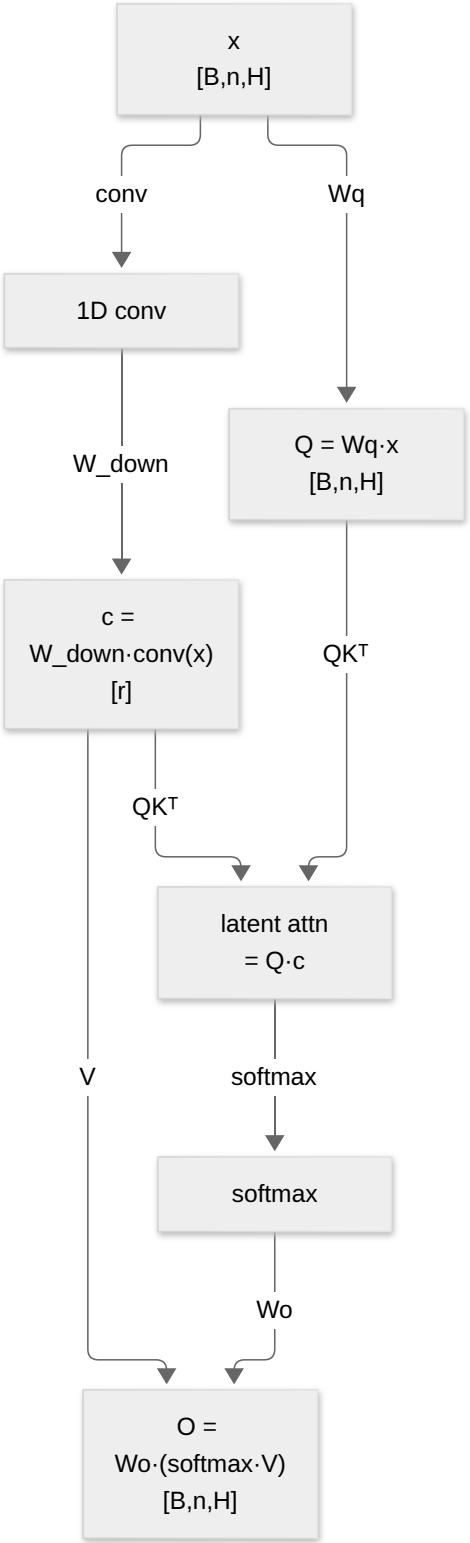


Figura 48: Flujo de datos de la atención convolucional comprimida: una convolución 1D alimenta una compresión latente c que la consulta atiende directamente.

La Atención Convolucional Comprimida (CCA, por sus siglas en inglés) [35] adopta un enfoque fundamentalmente diferente para la atención latente: en lugar de comprimir el caché KV y luego up-proyectar al ancho completo antes de atender (como MLA/GQLA/MLRA), CCA proyecta consultas, claves y valores a un latente compartido y realiza la operación de atención *completa* dentro de él. No hay matrices de up-proyección para Q/K/V — sólo una única up-proyección de salida $\tilde{W}_O$ mapea la salida latente de vuelta al flujo residual. Esto reduce simultáneamente parámetros, caché KV, **y** FLOPs de atención por el factor de compresión $C = E/\tilde{e}$, mientras que MLA sólo reduce el caché.

Para hacer viable la atención en el latente comprimido, CCA introduce tres innovaciones: (1) dos convoluciones causales sobre el tensor empaquetado q/k (una convolución de secuencia depth-wise y una convolución agrupada de canales por cabeza), (2) q-k-mean (añade la media pre-convolución de q y k a los valores post-convolución), y (3) value-shift (cada cabeza recibe la mitad de sus valores del token actual y la mitad del token anterior mediante dos proyecciones independientes). Tras estos pasos, se aplica normalización L2 de QK y una temperatura de clave aprendible β, RoPE se aplica *directamente en el latente* (sin cabeza RoPE separada), y se computa la atención softmax estándar.

6.9.1. Formulación Matemática

$$\tilde{q}_{\text{pre}} = \tilde{W}_Q\, x, \qquad \tilde{k}_{\text{pre}} = \tilde{W}_K\, x, \qquad \tilde{e} = E/C.$$

$$\tilde{q} = \text{Conv}(\tilde{q}_{\text{pre}}) + \tfrac{1}{2}(\tilde{q}_{\text{pre}} + \tilde{k}_{\text{pre}}), \qquad \tilde{k} = \text{Conv}(\tilde{k}_{\text{pre}}) + \tfrac{1}{2}(\tilde{q}_{\text{pre}} + \tilde{k}_{\text{pre}}).$$

$$\tilde{v} = [\tilde{W}_{V_1}\, x_t \parallel \tilde{W}_{V_2}\, x_{t-1}], \qquad \hat{q} = \frac{\tilde{q}\sqrt{d_h}}{\|\tilde{q}\|}, \quad \hat{k} = \frac{\tilde{k}\sqrt{d_h}}{\|\tilde{k}\|} e^{\beta}.$$

$$\text{atencion} = \texttt{softmax}\left(\frac{\text{RoPE}(\hat{q})\,\text{RoPE}(\hat{k})^\top}{\sqrt{d_h}}\right)\tilde{v}, \qquad \mathbf{Y} = \tilde{W}_O\,\text{atencion}.$$

6.9.2. Características Clave

- **Complejidad de entrenamiento**: $\mathcal{O}(n^2 \cdot d/C)$ (atención en latente comprimido, $C\times$ menos FLOPs que MHA).
- **Complejidad de inferencia**: $\mathcal{O}(n)$ por token, espacio de caché KV $\mathcal{O}(\tilde{e} \cdot n)$.
- **Entrenable**: C, tamaños de kernel de conv, y los tres trucos (convs, qk-mean, value-shift) son configurables.
- **Fortalezas**: Reduce params, caché Y FLOPs simultáneamente; RoPE se integra nativamente; $>2\times$ menos params que MLA a igual compresión.
- **Limitaciones**: No elimina la cuadrática S^2 (sólo la divide por C); conv/qk-mean/v-shift añaden sesgo inductivo; se necesita kernel fusionado para la aceleración teórica.

6.10. Atención Convolucional Comprimida por Consultas Agrupadas (CCG-QA)

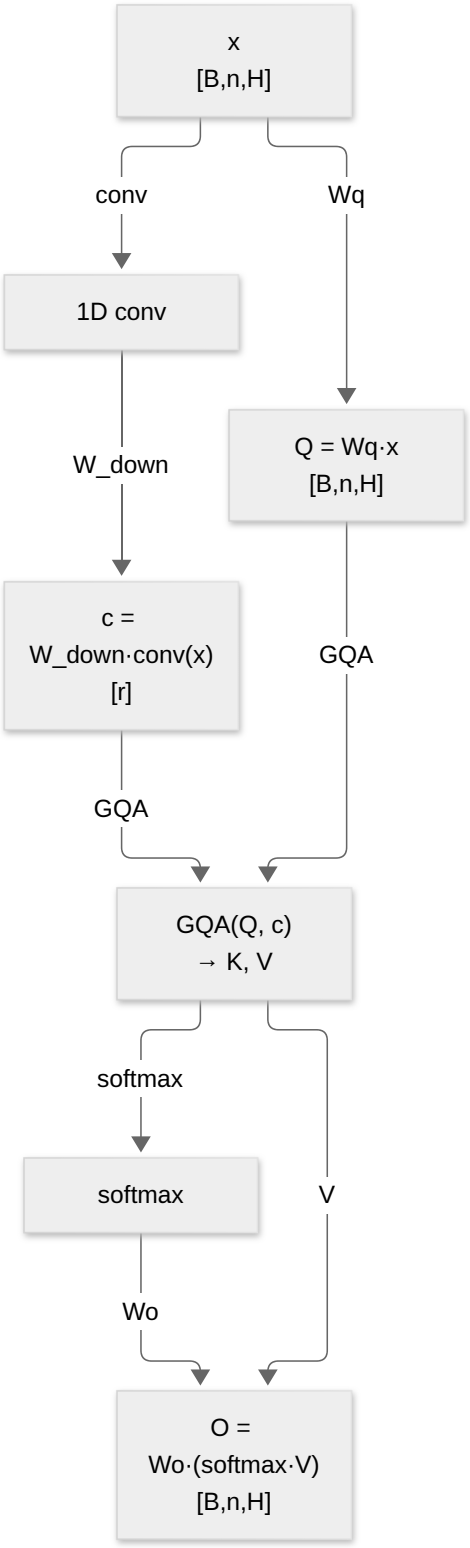


Figura 49: Flujo de datos de la atención convolucional comprimida con consultas agrupadas: el latente de la convolución alimenta una expansión de clave/valor con consultas agrupadas.

La Atención Convolucional Comprimida por Consultas Agrupadas (CCGQA, por sus siglas en inglés) [35] extiende CCA con compartición de cabezas clave/valor estilo GQA aplicada *dentro* del latente comprimido, y **desacopla** las tasas de compresión de query y KV: las consultas se comprimen por C_1 y las claves/valores por $C_2 \geq C_1$. La dimensión latente por cabeza d_h debe coincidir entre cabezas de query y clave, imponiendo la restricción $C_2/C_1 = n_h/n_{kv}$. Las cabezas KV se comparten por grupo (replicadas para coincidir con las cabezas de query), y el sesgo qk-mean usa operadores de broadcast/reducción B_{group} (replicar cabezas KV a cabezas query) y E_{group} (promediar cabezas query dentro de cada grupo). CCGQA logra la mejor pérdida reportada a 8× reducción de caché KV sin pérdida de calidad frente a MHA.

6.10.1. Características Clave

- **Complejidad de entrenamiento**: $\mathcal{O}(n^2 \cdot d/C_1)$ (el término S^2 escala por $1/C_1$; los términos de proyección por $1/C_1 + 1/C_2$).
- **Complejidad de inferencia**: $\mathcal{O}(n)$ por token, espacio de caché KV $\mathcal{O}(\tilde{e}_{kv} \cdot n)$ (comprimido por C_2, compartido por grupo).
- **Entrenable**: C_1, C_2, n_{kv}, y todos los trucos CCA son configurables. Restricción: $C_2/C_1 = n_h/n_{kv}$.
- **Fortalezas**: Compresión desacoplada permite frontera de Pareto suave; misma intensidad aritmética que GQA; mejor pérdida a 8× reducción de caché.
- **Limitaciones**: Misma cuadrática S^2 (sólo reducida por C_1); se necesita kernel fusionado; dos proyecciones de valor añaden complejidad.

La familia latente muestra una progresión clara a lo largo de dos ejes: (i) **compresión de rango**, pasando de cabezas completas (MHA) a un latente compartido (MLA) a sub-cabezas latentes particionadas (MLRA), y (ii) **adaptividad de despliegue**, pasando de una sola ruta de decodificación a despacho adaptativo al hardware (GQLA) y fusión temporal de caché (MTLA). La Atención Tucker proporciona el marco algebraico unificador: MHA, GQA y MLA son todos casos especiales de una factorización Tucker con elecciones de rango apropiadas. CCA y CCGQA toman un enfoque fundamentalmente diferente: en lugar de comprimir el caché KV y luego up-proyectar para atención a ancho completo (MLA/GQLA/MLRA), realizan atención *completamente dentro* del espacio latente comprimido, reduciendo no sólo el caché sino también los FLOPs de atención por el factor de compresión. CCGQA además desacopla las tasas de compresión de query y KV, logrando la mejor pérdida reportada a 8× reducción de caché KV. La Atención de Mezcla Gaussiana (GMA) cambia la filosofía de enrutamiento por completo: en lugar de un cuello de botella lineal de rango bajo, enruta a través de K componentes Gaussianos probabilísticos aprendidos, logrando mezcla de secuencias en tiempo lineal $\mathcal{O}(NK)$ para K fijo manteniendo una interpretación de atención normalizada.

6.11. Atención de Mezcla Gaussiana (GMA)

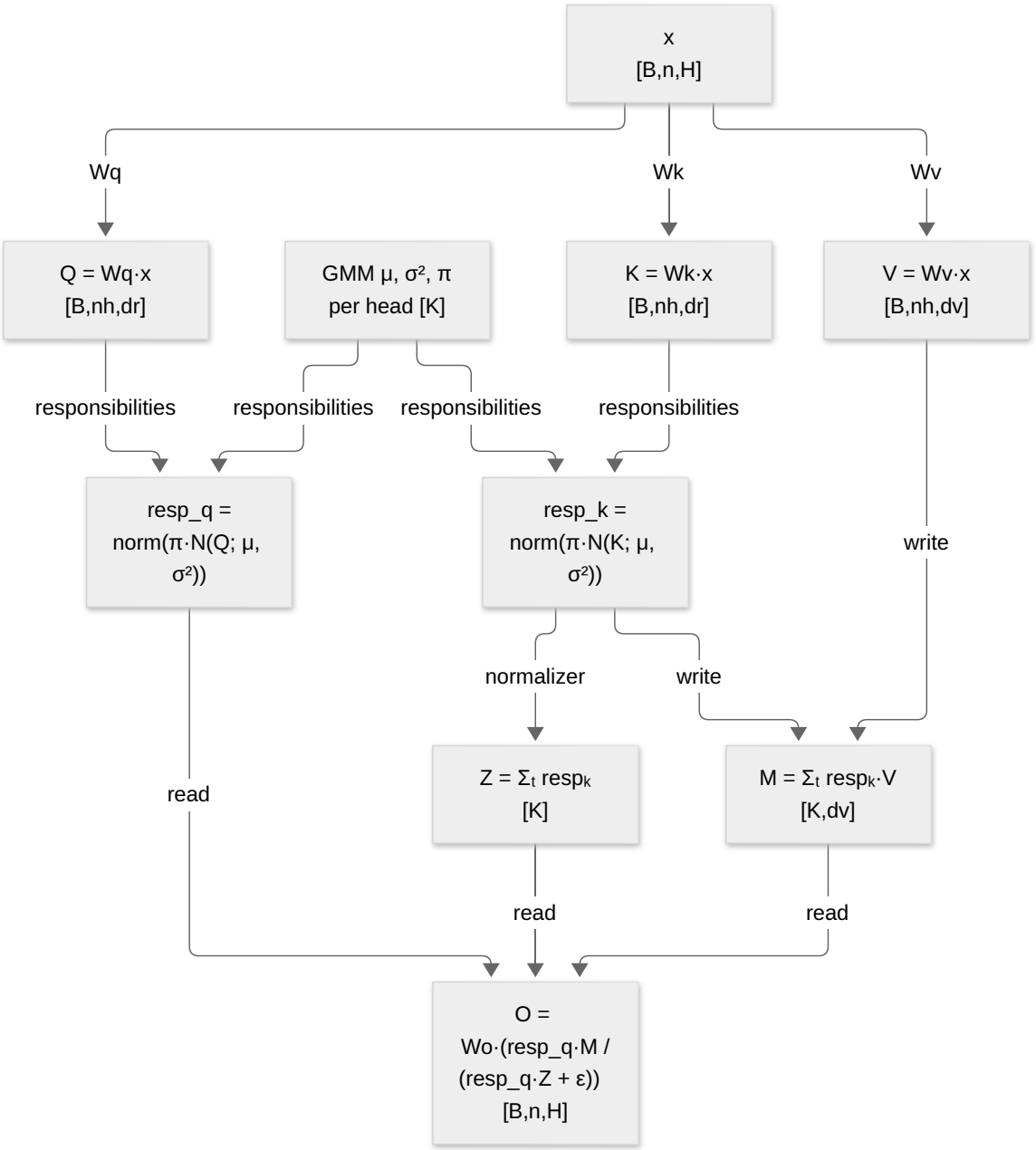


Figura 50: Flujo de datos de la atención de mezcla gaussiana: las responsabilidades gaussianas por cabeza enrutan claves y valores hacia una memoria de K ranuras que la consulta lee de vuelta, normalizada por Z.

La Atención de Mezcla Gaussiana (GMA, por sus siglas en inglés) [57] reemplaza la interacción pareada explícita $QK^\top$ de la atención de producto punto estándar con enrutamiento probabilístico a través de K componentes de mezcla Gaussiana aprendidos por cabeza. Las consultas y claves se mapean a vectores de *responsabilidad* posteriores sobre un espacio de enrutamiento latente compartido; su solapamiento define una afinidad implícita en el espacio de responsabilidades, mientras que los valores se escriben y leen de una memoria latente de K slots. Al explotar la asociatividad de la multiplicación matricial, GMA evita materializar la matriz de afinidad $N \times N$ inducida y en su lugar usa dos matrices de responsabilidad $N \times K$ cuyo almacenamiento de

activaciones dominante escala como $\mathcal{O}(NK)$ en lugar de $\mathcal{O}(N^2)$ para K fijo. Los parámetros de la mezcla Gaussiana—medias de componentes μ, covarianzas diagonales (reparametrizadas vía $\sigma^2 = \text{softplus}(\omega) + \epsilon_\sigma$) y logits de prior de mezcla α (con $\pi = \text{softmax}(\alpha)$)—son parámetros aprendibles totalmente diferenciables optimizados por retropropagación estándar, sin bucle EM ni pérdida auxiliar de clustering.

6.11.1. Formulación Matemática

Por cabeza, con dimensión de enrutamiento d_r y dimensión de valor d_v:

$$Q_X = XW_Q \in \mathbb{R}^{N\times d_r}, \quad K_X = XW_K \in \mathbb{R}^{N\times d_r}, \quad V_X = XW_V \in \mathbb{R}^{N\times d_v}.$$

La responsabilidad posterior del componente k para un vector de enrutamiento x_i es

$$\gamma_{i,k} = p(z{=}k \mid x_i) = \frac{\pi_k \,\mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_{\ell=1}^{K} \pi_\ell \,\mathcal{N}(x_i \mid \mu_\ell, \Sigma_\ell)}, \qquad \Sigma_k = \text{diag}(\sigma_{k,1}^2, \ldots, \sigma_{k,d_r}^2),$$

produciendo las matrices de responsabilidad $\Gamma^Q, \Gamma^K \in \mathbb{R}^{N\times K}$. La salida bidireccional se calcula mediante el par asociativo de escritura–lectura

$$\tilde{V} = (\Gamma^K)^\top V_X \in \mathbb{R}^{K\times d_v}, \qquad Z = (\Gamma^K)^\top \mathbf{1}_N \in \mathbb{R}^K, \qquad O = \frac{\Gamma^Q \tilde{V}}{\Gamma^Q Z + \epsilon},$$

que nunca materializa la afinidad normalizada implícita $A_{ij}^{\text{GMA}} = \frac{\sum_k \gamma_{i,k}^Q \gamma_{j,k}^K}{\sum_\ell \sum_k \gamma_{i,k}^Q \gamma_{\ell,k}^K + \epsilon}$. Para modelado autorregresivo, las estadísticas de escritura globales se reemplazan por sumas acumuladas restringidas al prefijo $\tilde{V}_k^{(i)} = \sum_{j\le i} \gamma_{j,k}^K V_{X,j}$ y $Z_k^{(i)} = \sum_{j\le i} \gamma_{j,k}^K$, imponiendo causalidad preservando el escalado lineal de K fijo.

6.11.2. Pseudocódigo Algorítmico

Entrada: estado oculto $X \in \mathbb{R}^{N\times d}$, K componentes, dim. de enrutamiento d_r, dim. de valor d_v, params $\{\mu_k, \omega_k, \alpha_k\}_{k=1}^K$
$Q_X \leftarrow XW_Q, \quad K_X \leftarrow XW_K, \quad V_X \leftarrow XW_V$ ▷ proyecciones por cabeza
$\sigma_k^2 \leftarrow \text{softplus}(\omega_k) + \epsilon_\sigma, \quad \pi_k \leftarrow \text{softmax}(\alpha)_k$ ▷ reparametrizar
$\Gamma^Q \leftarrow \texttt{softmax}_k\left(\log \pi_k - \frac{d_r}{2}\log 2\pi - \frac{1}{2}\sum_m \log \sigma_{k,m}^2 - \frac{1}{2}\sum_m \frac{(q_{i,m}-\mu_{k,m})^2}{\sigma_{k,m}^2}\right)$
$\Gamma^K \leftarrow$ igual que Γ^Q aplicado a K_X
if causal **then**
    $\tilde{V} \leftarrow \texttt{cumsum}_n(\Gamma^K \otimes V_X), \quad Z \leftarrow \texttt{cumsum}_n(\Gamma^K)$ ▷ restringido al prefijo
else
    $\tilde{V} \leftarrow (\Gamma^K)^\top V_X, \quad Z \leftarrow (\Gamma^K)^\top \mathbf{1}_N$ ▷ escritura global
end if
$O \leftarrow \Gamma^Q \tilde{V} \,/\, (\Gamma^Q Z + \epsilon)$ ▷ lectura normalizada
Salida: $\mathbf{Y} = OW_O$

6.11.3. Características Clave

- **Complejidad de entrenamiento**: $\mathcal{O}(NKd_r + NKd_v)$ para K fijo, lineal en la longitud de secuencia N.
- **Complejidad de inferencia**: $\mathcal{O}(K)$ por token (responsabilidades + lectura latente), almacenamiento de activaciones $\mathcal{O}(NK)$.
- **Entrenable**: parámetros GMM totalmente diferenciables $\{\mu, \omega, \alpha\}$ por cabeza, optimizados end-to-end sin EM.

- **Fortalezas**: Mezcla de secuencias en tiempo lineal con K fijo; enrutamiento probabilístico interpretable; variantes bidireccional y causal; interpretación de afinidad de rango bajo no negativa $A^{\mathrm{GMA}} = \mathrm{diag}(d)^{-1}\Gamma^{Q}(\Gamma^{K})^{\top}$.
- **Limitaciones**: La calidad depende de K y la dimensión de enrutamiento d_r; la GMA causal queda por debajo de SDPA optimizada y modelos de espacio de estados en WikiText-103 en la implementación actual del paper; no es un reemplazo universal para la atención softmax.

7. Mecanismos de Atención Dispersa Exhaustivos

7.1. Resumen Ejecutivo

Los mecanismos de atención dispersa abordan la prohibitiva complejidad computacional y de memoria $\mathcal{O}(n^2)$ de la atención de producto punto escalado estándar al restringir el conjunto de pares clave-valor atendidos. La atención dispersa moderna abarca un rico espacio de diseño distinguido por cuatro dimensiones ortogonales: (i) **patrón de dispersidad** (geométrico fijo, dependiente de datos o basado en frecuencia), (ii) **capacidad de entrenamiento** (entrenado de extremo a extremo o solo inferencia), (iii) **unidad de dispersidad** (tokens individuales, ventanas locales o bloques), y (iv) **mecanismo** (enmascaramiento, selección, filtrado o compresión). Este anexo sintetiza diez familias principales de atención dispersa—Sparse Transformer, Longformer, SWA con sumideros de atención, BigBird, SparseK, NSA, FASA, SpargeAttn, MSA y SparDA—proporcionando formulaciones matemáticas, pseudocódigo algorítmico y comparaciones arquitectónicas exhaustivas.

7.2. Sparse Transformer: Patrones Estrideados y Fijos Factorizados

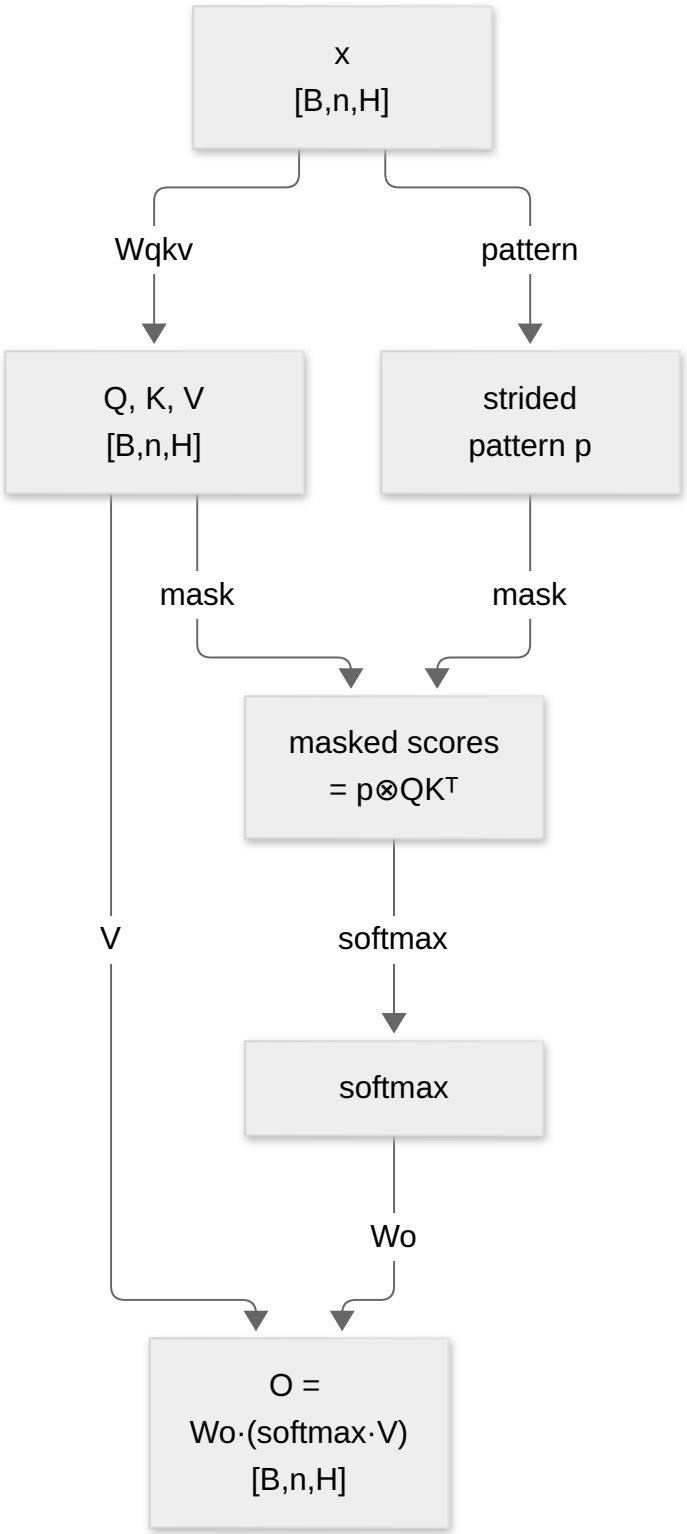


Figura 51: Flujo de datos del Sparse Transformer: un patrón factorial con zancada enmascara la matriz de puntuaciones antes del softmax.

7.2.1. Formulación Matemática

Sparse Transformer [20] reduce la complejidad cuadrática a $\mathcal{O}(n\sqrt{n})$ factorizando la atención dispersa en dos cabezas dispersas complementarias. Sea n la longitud de la secuencia y $l = \lfloor \sqrt{n} \rfloor$ el stride.

Cabeza de atención estrideada : Cada posición i atiende a cada l-ésima posición anterior:

$$\mathcal{A}_i^{(\text{estrideada})} = \{j : j \leq i,\ (i-j) \bmod l = 0\}$$

Cabeza de atención fija : Cada posición i atiende a posiciones dentro de su bloque local más columnas de resumen fijas:

$$\mathcal{A}_i^{(\text{fija})} = \{j : \lfloor j/l \rfloor = \lfloor i/l \rfloor\} \cup \{j : j \bmod l \in \{l-c, \ldots, l-1\}\}$$

donde c es un hiperparámetro que controla el número de columnas de resumen (típicamente $c = 1$ o $c = 2$).

El cálculo de atención para cada cabeza h sigue las reglas estándar de producto punto escalado restringido al conjunto disperso:

$$\text{Attn}_h(Q, K, V)_i = \text{softmax}\left(\frac{q_i^h K_{\mathcal{A}_i}^{h\top}}{\sqrt{d_k}}\right) V_{\mathcal{A}_i}^h$$

La idea clave es que con las dos cabezas factorizadas operando en paralelo a través de una profundidad de L capas transformer, cada posición puede alcanzar cualquier otra posición a través de un camino de longitud máxima $L + 1$, preservando la alcanzabilidad de largo alcance con una fracción del costo de la atención densa.

7.2.2. Pseudocódigo Algorítmico

Algorithm 26 Atención Sparse Transformer: Cabezas Duales Estrideada + Fija

Require: Consulta $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Clave $\mathbf{K} \in \mathbb{R}^{n \times d}$, Valor $\mathbf{V} \in \mathbb{R}^{n \times d}$
Require: Stride $l = \lfloor \sqrt{n} \rfloor$, columnas de resumen $c \in \{1, 2\}$
Ensure: Salida $\mathbf{Y} \in \mathbb{R}^{n \times d}$

Cabeza 1: Atención Estrideada
for $i = 1, \ldots, n$ **do**
  $\mathcal{A}_i \leftarrow \{j : j \leq i \text{ y } (i-j) \bmod l = 0\}$ ▷ Cada l-ésima posición
  $\text{puntajes}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$
  $\text{attn}_i \leftarrow \text{softmax}(\text{puntajes}_i)$
  $\mathbf{y}_i^{(1)} \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$
end for
Cabeza 2: Atención de Bloque Fijo + Resumen
for $i = 1, \ldots, n$ **do**
  $\text{inicio_bloque} \leftarrow \lfloor i/l \rfloor \cdot l$, $\text{fin_bloque} \leftarrow \text{mín}(i+1, \text{inicio_bloque} + l)$
  $\mathcal{A}_i \leftarrow [\text{inicio_bloque}, \text{fin_bloque}) \cup \{\text{columnas de las últimas } c \text{ columnas}\}$
  $\text{puntajes}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$
  $\text{attn}_i \leftarrow \text{softmax}(\text{puntajes}_i)$
  $\mathbf{y}_i^{(2)} \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$
end for
Concatenar: $\mathbf{Y} = [\mathbf{y}^{(1)} \| \mathbf{y}^{(2)}]$ y proyectar mediante capa lineal de salida **return Y**

7.2.3. Características Clave

- **Complejidad**: $\mathcal{O}(n\sqrt{n} \cdot d)$ durante el entrenamiento; $\mathcal{O}(n)$ por token durante la generación.
- **Entrenable**: Sí; retropropagación completa a través de las posiciones seleccionadas.
- **Patrón de dispersidad**: Fijo e independiente de los datos; los patrones no se adaptan al contenido.
- **Fortaleza**: Alcanzabilidad estructurada que garantiza dependencias de largo alcance; demostrado efectivo en secuencias largas (Enwik8, imágenes de texto).
- **Limitación**: Los patrones fijos pueden pasar por alto relaciones importantes pero no contiguas; requiere kernels CUDA personalizados para eficiencia práctica.

7.3. Longformer: Ventana Deslizante, Dilatación y Tokens Globales

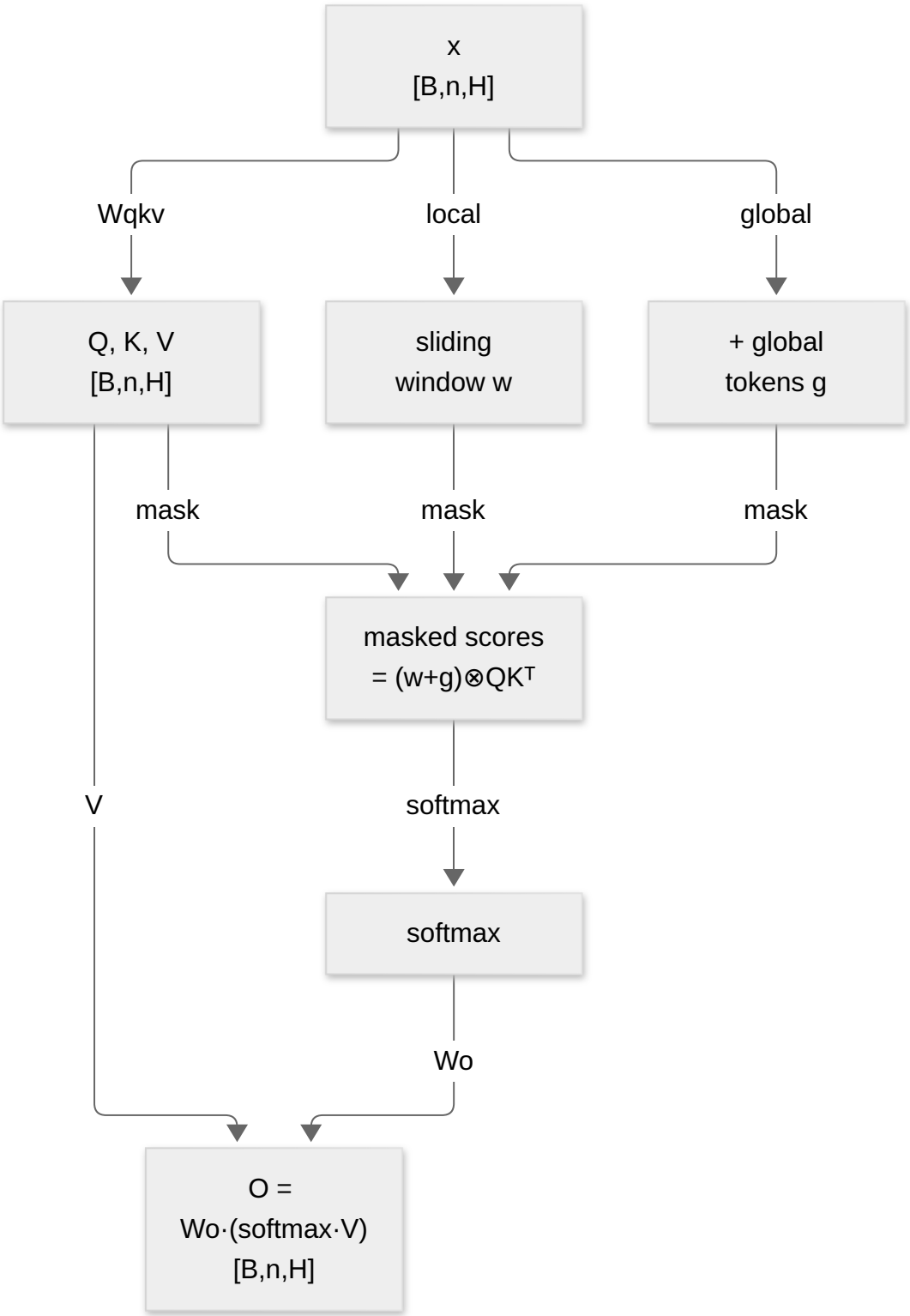


Figura 52: Flujo de datos de Longformer: las máscaras de ventana deslizante y de tokens globales se aplican a la matriz de puntuaciones antes del softmax.

7.3.1. Formulación Matemática

Longformer [12] logra una complejidad lineal $\mathcal{O}(n \cdot w)$ combinando tres patrones de atención complementarios.

Atención de ventana deslizante : Vecindario local de tamaño w:

$$\mathcal{A}_i^{(\text{ventana})} = \{j : |i - j| \leq w/2\}$$

Ventana deslizante dilatada : Atención con ventana y dilatación d, saltando stride $d + 1$:

$$\mathcal{A}_i^{(\text{dilatada})} = \{j : |i - j| \leq (w/2)(d + 1) \text{ y } (i - j) \text{ mód } (d + 1) = 0\}$$

Atención global : Tokens globales designados (ej., [CLS]) atienden a todas las posiciones y todos atienden a ellos:

$$\mathcal{A}_i^{(\text{global})} = \begin{cases} \{1, \ldots, n\} & \text{si } i \in \mathcal{G} \\ \mathcal{A}_i^{(\text{ventana})} \cup \mathcal{G} & \text{en otro caso} \end{cases}$$

Los tres patrones se aplican mediante proyecciones separadas. La atención local y global pueden usar las mismas proyecciones o unas separadas específicas para cada tarea.

7.3.2. Pseudocódigo Algorítmico

Algorithm 27 Longformer: Ventana Deslizante + Dilatada + Global

Require: Consulta $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Clave $\mathbf{K}$, Valor $\mathbf{V}$, tamaño de ventana w, dilatación d, índices de tokens globales $\mathcal{G}$
Ensure: Salida $\mathbf{Y}$

for $i = 1, \ldots, n$ **do**
  if $i \in \mathcal{G}$ **then**
    $\mathcal{A}_i \leftarrow \{1, \ldots, n\}$ ▷ Token global atiende a todos
  else
    Ventana local: $\mathcal{A}^{(\text{local})} \leftarrow [i - w/2, i + w/2] \cap [1, n]$
    Desplazamientos dilatados: $\mathcal{A}^{(\text{dilatada})} \leftarrow \{j : (i-j) \text{ mód } (d+1) = 0\} \cap \mathcal{A}^{(\text{rango dilatado})}$
    Combinar: $\mathcal{A}_i \leftarrow \mathcal{A}^{(\text{local})} \cup \mathcal{A}^{(\text{dilatada})} \cup \mathcal{G}$
  end if
  puntajes$_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$
  attn$_i \leftarrow$ softmax(puntajes$_i$)
  $\mathbf{y}_i \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$
end forreturn $\mathbf{Y}$

7.3.3. Características Clave

- **Complejidad**: $\mathcal{O}(n \cdot w)$ donde w es el tamaño de ventana (lineal cuando $w \ll n$).
- **Entrenable**: Sí; reemplazo directo para la atención estándar.
- **Cobertura**: Con L capas y tamaño de ventana w, el campo receptivo crece a $L \times w$, cubriendo toda la secuencia con redes poco profundas.
- **Fortaleza**: Práctico para tareas a nivel de documentos; la dilatación expande los campos receptivos locales con sobrecarga mínima; los tokens globales proporcionan correspondencia consulta-documento.
- **Limitación**: El tamaño de ventana sigue siendo un hiperparámetro de diseño; subóptimo para tareas que requieren patrones de atención densos.

7.4. SWA con Sumideros de Atención: Ventana Deslizante Más un Prefijo Sumidero

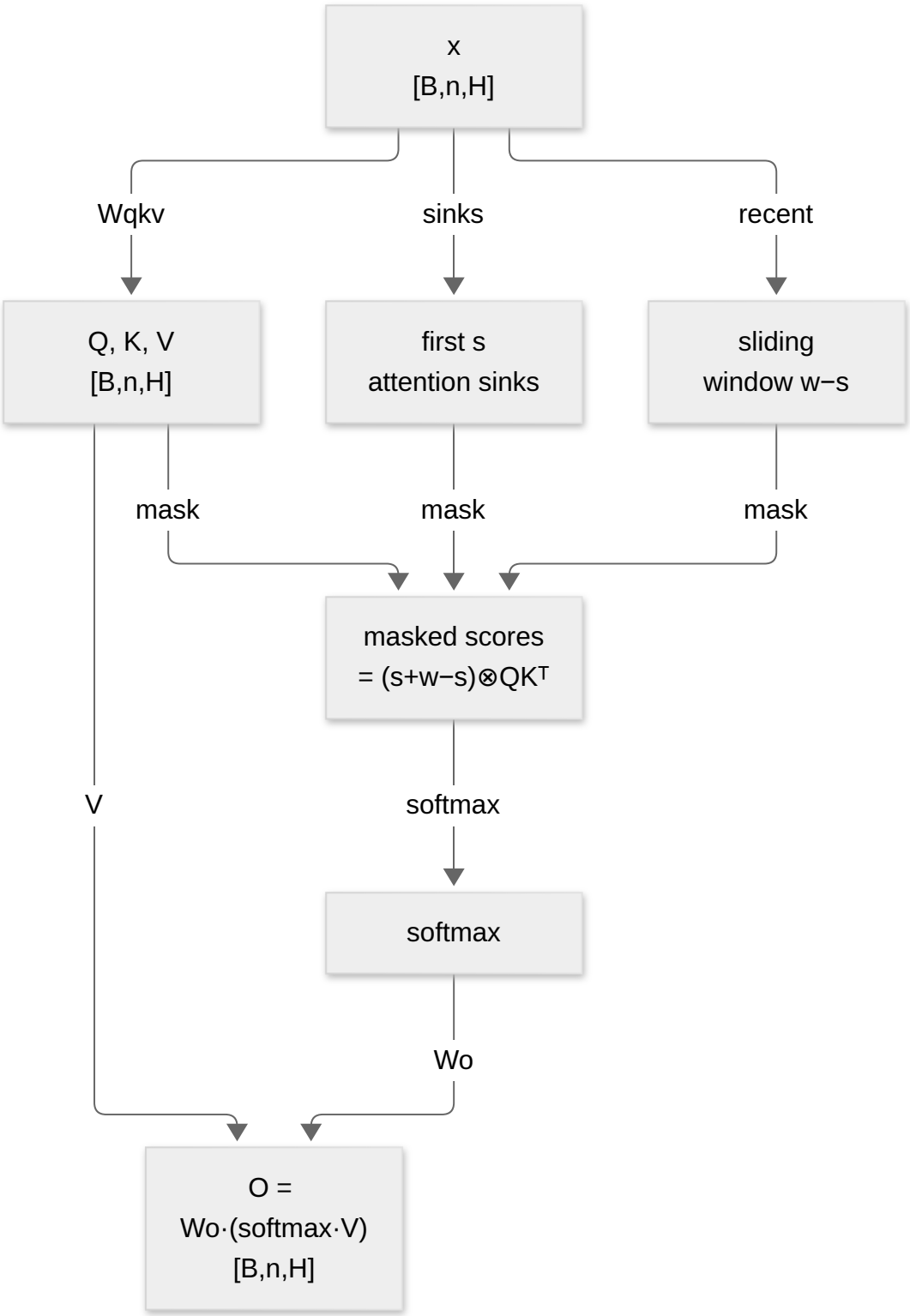


Figura 53: Flujo de datos de SWA con sumideros de atención: los primeros s tokens sumidero y una ventana deslizante causal de $w - s$ se unen en una máscara aplicada a la matriz de puntuaciones antes del softmax.

7.4.1. Formulación Matemática

SWA con sumideros de atención [58] es la política de desalojo de caché KV más simple: en lugar de comprimir la caché o reemplazar la atención por un estado recurrente, restringe cada consulta a un conjunto pequeño y fijo de claves. Denotemos el tamaño de ventana w y el número de sumideros s, es decir SWA(w, s). En modo decoder, la consulta i atiende a

$$\mathcal{A}_i = \underbrace{\{j : j < s,\ j \le i\}}_{\text{sumideros de atención}} \cup \underbrace{\{j : i - (w - s) + 1 \le j \le i\}}_{\text{ventana deslizante de } w-s},$$

de modo que cada consulta ve como máximo w claves. El recorte causal $j \le i$ sobre el término de sumideros es esencial: los tokens anteriores a la posición s no deben ver sumideros futuros.

Por qué los sumideros son imprescindibles Los transformers aprenden a asignar una masa de atención desproporcionadamente grande a los primeros tokens aunque no sean semánticamente relevantes. Estos *sumideros de atención* [118] actúan como repositorios de la atención sobrante. Cuando el prefijo sale de una ventana deslizante sin sumideros, el modelo pierde su destino de descarga y la calidad colapsa catastróficamente; reincluir los primeros s tokens recupera la mayor parte del rendimiento de la atención completa. El paper fija $s = 4$ y evalúa $w \in \{64, 128, 256, 512\}$.

Extensión a encoder El paper define SWA(w, s) para modelos de lenguaje causales. Para encoders bidireccionales la ventana local se hace simétrica, $\{j : |i - j| \le \lfloor (w - s)/2 \rfloor\}$, y se une con el prefijo sumidero—la misma convención que usa Longformer.

Como la intervención es un cambio puro de máscara de atención, no requiere parámetros adicionales ni matemática QKV nueva: se reutilizan las proyecciones estándar W_q, W_k, W_v, W_o. Puede aplicarse a un checkpoint preentrenado en inferencia con cero tokens de entrenamiento, o entrenarse desde cero como un mixer normal.

7.4.2. Pseudocódigo Algorítmico

Algorithm 28 SWA con Sumideros de Atención: Construcción de Máscara y Atención

Require: Consulta $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Clave $\mathbf{K}$, Valor $\mathbf{V}$, ventana w, sumideros s
Ensure: Salida $\mathbf{Y} \in \mathbb{R}^{n \times d}$

Índices de fila i, índices de columna j
causal $\leftarrow [j \le i]$
sumideros $\leftarrow [j < s] \wedge$ causal ▷ Prefijo sumidero recortado causalmente
ventana $\leftarrow [i - (w - s) + 1 \le j \le i]$
$M \leftarrow$ sumideros $\vee$ ventana
$S \leftarrow \mathbf{Q}\mathbf{K}^\top / \sqrt{d_k}$
$S \leftarrow \text{mask}(S, M)$ ▷ $-\infty$ donde $M = 0$
$\mathbf{Y} \leftarrow \text{softmax}(S)\,\mathbf{V}$ **return** $\mathbf{Y}$

7.4.3. Características Clave

- **Complejidad**: $\mathcal{O}(n \cdot w \cdot d)$ en tiempo y $\mathcal{O}(w)$ de almacenamiento KV por capa en inferencia (fijo una vez que la secuencia supera w).
- **Entrenable**: Ambos; sin entrenamiento en inferencia sobre checkpoints preentrenados, o entrenado desde cero.
- **Campo receptivo**: Tras L capas, $L(w - s)$ posiciones más el prefijo sumidero siempre visible.

- **Fortaleza**: Recupera $\approx 99\,\%$ de la puntuación media de atención completa en modelos de 1.3B a 72B con cero tokens de entrenamiento, y supera a la atención lineal retrofiteada por 2–10$\times$ en razonamiento de contexto largo (S-NIAH, BABILong) decodificando más rápido y usando menos memoria.
- **Limitación**: El horizonte es fijo y depende de la tarea—$w = 64$ basta para benchmarks de verosimilitud pero colapsa en matemáticas generativas, donde se necesita $w = 512$; no hay selección adaptativa al contenido.

7.5. BigBird: Grafo Disperso Aleatorio, Local y Global

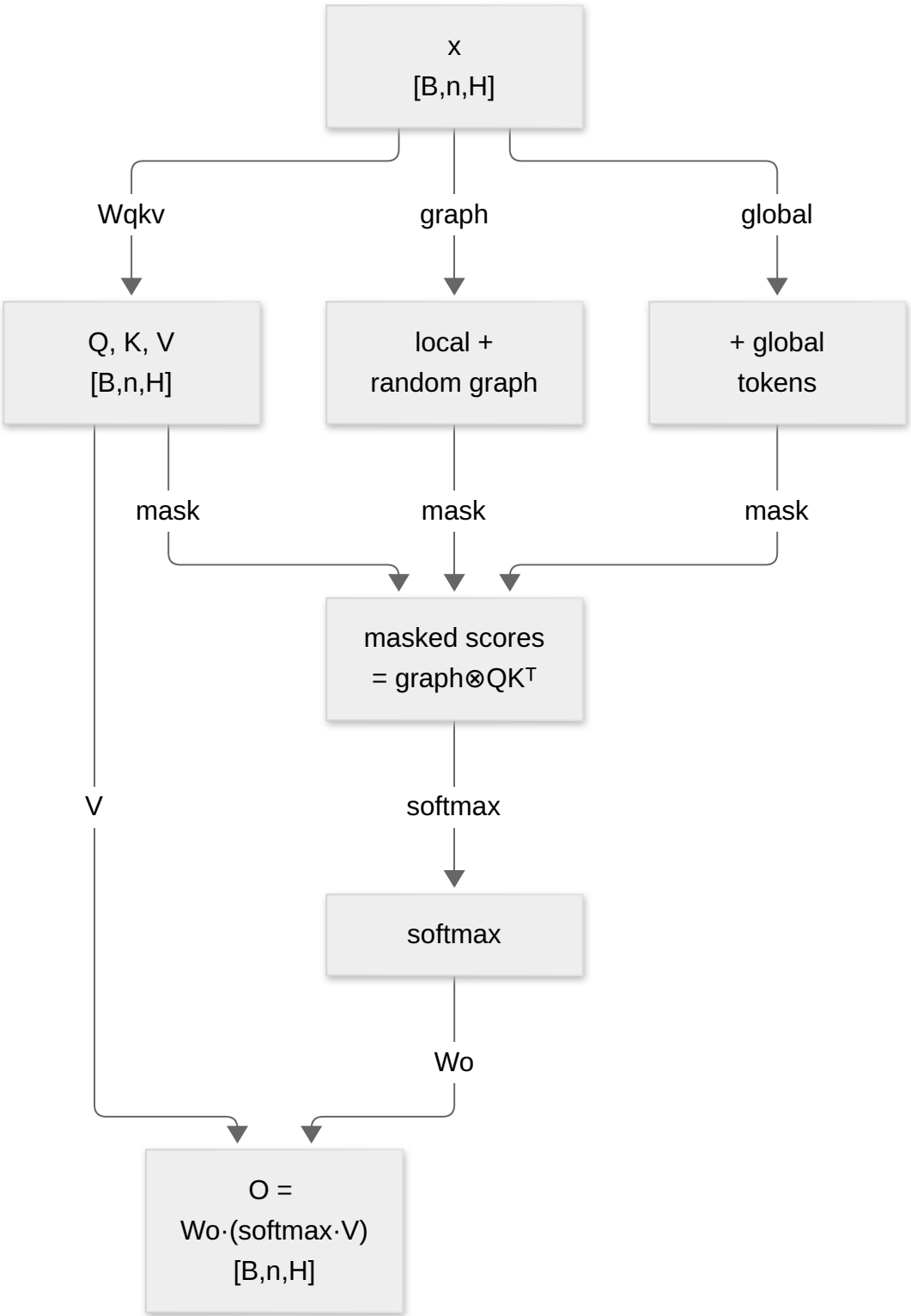


Figura 54: Flujo de datos de BigBird: la adyacencia local, aleatoria y global enmascara la matriz de puntuaciones antes del softmax.

7.5.1. Formulación Matemática

BigBird [127] preserva la aproximación universal y la completitud de Turing utilizando una combinación fundamentada de tres patrones de dispersidad.

Conexiones aleatorias : Cada posición se conecta a r posiciones muestreadas aleatoriamente:

$$\mathcal{A}_i^{(\text{aleatoria})} = \text{MuestraAleatoria}(\{1, \ldots, n\}, r)$$

Ventana local : Vecindario de ventana deslizante:

$$\mathcal{A}_i^{(\text{local})} = \{j : |i - j| \leq w/2\}$$

Tokens globales : Anclas globales específicas de la tarea:

$$\mathcal{A}_i^{(\text{global})} = \begin{cases} \{1, \ldots, n\} & \text{si } i \in \mathcal{G} \\ \mathcal{A}_i^{(\text{aleatoria})} \cup \mathcal{A}_i^{(\text{local})} \cup \mathcal{G} & \text{en otro caso} \end{cases}$$

La máscara dispersa compuesta M es:

$$M_{ij} = \mathbf{1}[j \in \mathcal{A}_i^{(\text{aleatoria})} \cup \mathcal{A}_i^{(\text{local})} \cup \mathcal{A}_i^{(\text{global})}]$$

En la práctica, BigBird agrupa tokens en bloques de tamaño b y aplica la decisión de dispersidad a nivel de bloque, permitiendo implementaciones eficientes de bloques dispersos.

7.5.2. Garantía Teórica

Los autores demuestran que con $O(1)$ conexiones aleatorias y tokens globales, el grafo disperso mantiene las mismas propiedades de aproximación universal y completitud de Turing que la atención densa. Formalmente, cualquier función computable puede representarse y cualquier secuencia de operaciones puede simularse dentro del grafo de conectividad de BigBird.

7.5.3. Pseudocódigo Algorítmico

Algorithm 29 BigBird: Atención Dispersa por Bloques Aleatoria + Local + Global

Require: Consulta $\mathbf{Q}$, Clave $\mathbf{K}$, Valor $\mathbf{V}$, tamaño de bloque b, enlaces aleatorios por bloque r, índices globales $\mathcal{G}$

Ensure: Salida $\mathbf{Y}$

Reformar en bloques: $n_b = \lceil n/b \rceil$ bloques
for bloque $i = 0, \ldots, n_b - 1$ **do**
  for posición j en bloque i **do**
    Ventana local: $\mathcal{A}_j^{(\text{local})}$ = posiciones cercanas en el bloque $\cup$ posiciones frontera
    Muestreo aleatorio de bloques: Muestrear r bloques uniformemente al azar, agregar todas las posiciones de esos bloques
    Global: Agregar todas las posiciones de tokens globales
    $\mathcal{A}_j \leftarrow \mathcal{A}_j^{(\text{local})} \cup \mathcal{A}_j^{(\text{aleatoria})} \cup \mathcal{G}$
  end for
end for
for $i = 1, \ldots, n$ **do**
  puntajes$_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$
  attn$_i \leftarrow$ softmax(puntajes$_i$)
  $\mathbf{y}_i \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$
end forreturn $\mathbf{Y}$

7.5.4. Características Clave

- **Complejidad**: $\mathcal{O}(n)$ o casi lineal en implementación óptima de bloques dispersos.
- **Entrenable**: Sí.
- **Base teórica**: Mantiene demostrablemente la aproximación universal y la completitud de Turing con conectividad dispersa.
- **Fortaleza**: Rendimiento sólido en documentos largos para preguntas-respuestas, resúmenes y tareas de genómica.
- **Limitación**: El muestreo aleatorio introduce varianza y no determinismo; el ajuste de los hiperparámetros r, w, g sigue dependiendo de la tarea.

7.6. Atención SparseK: Selección Diferencial Top-k

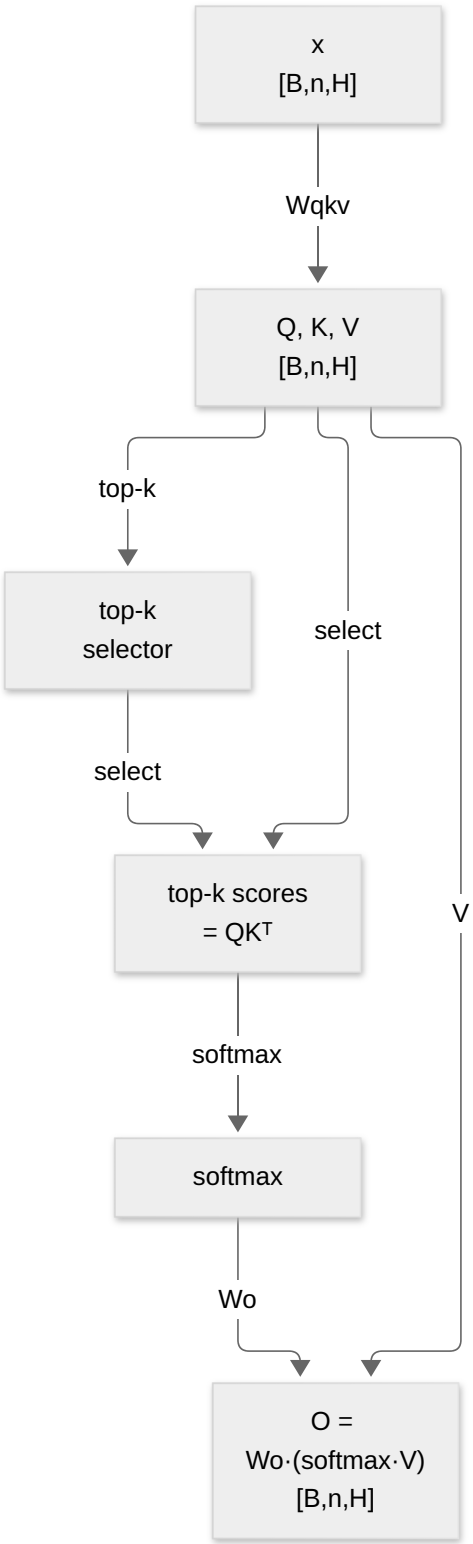


Figura 55: Flujo de datos de SparseK: un selector top-k diferenciable conserva solo las claves con mayor puntuación antes del softmax.

7.6.1. Formulación Matemática

SparseK [77] permite dispersidad aprendible mediante un **operador top-k diferenciable**. Una red de puntuación ϕ_θ evalúa la importancia de cada par clave-valor:

$$u_j = \phi_\theta(\mathbf{k}_j, \mathbf{q}_i) \in \mathbb{R}$$

El **operador SparseK** selecciona los puntajes top-k manteniéndose diferenciable. Calcula un umbral $\tau(u)$ tal que la suma de los puntajes activos sea igual a k:

$$\text{SparseK}(u,k)_j = \text{máx}(u_j - \tau(u), 0), \quad \text{donde} \quad \sum_j \text{máx}(u_j - \tau, 0) = k$$

El umbral τ se encuentra mediante bisección. La atención se convierte en:

$$m_j = \text{SparseK}(u,k)_j, \quad \text{Attn}_i = \text{softmax}\left(\frac{\mathbf{q}_i K_{\text{sel}}^\top}{\sqrt{d_k}}\right) V_{\text{sel}}$$

donde $K_{\text{sel}}, V_{\text{sel}}$ contienen solo las entradas top-k (aquellas con $m_j > 0$).

7.6.2. Pseudocódigo Algorítmico

Algorithm 30 SparseK: Atención Top-k Diferenciable

Require: Consulta $\mathbf{q} \in \mathbb{R}^d$, Matriz de clave $\mathbf{K} \in \mathbb{R}^{n\times d}$, Matriz de valor $\mathbf{V} \in \mathbb{R}^{n\times d}$
Require: Red de puntuación ϕ_θ, dispersidad objetivo k
Ensure: Salida de atención $\mathbf{y}$
Calcular puntajes de importancia:
for $j = 1, \ldots, n$ **do**
    $u_j \leftarrow \phi_\theta(\mathbf{k}_j, \mathbf{q})$
end for
Calcular top-k diferenciable mediante umbral:
$u_{\text{ordenado}} \leftarrow \text{ordenar}(u, \text{descendente} = \text{Verdadero})$
suma_acum $\leftarrow$ suma_acumulada(u_{ordenado})
Encontrar $\rho = \text{máx}\{i : u_{\text{ordenado}}[i] > 0 \text{ y suma_acum}[i] \leq k\}$
$\tau \leftarrow (u_{\text{ordenado}}[\rho] - k)/(\rho + 1)$ ▷ Umbral para k elementos activos
$m \leftarrow \text{máx}(u - \tau, 0)$ ▷ Máscara de selección diferenciable
Aplicar máscara y calcular atención:
$K_{\text{sel}} \leftarrow K[m > 0]$, $V_{\text{sel}} \leftarrow V[m > 0]$
puntajes $\leftarrow \mathbf{q}K_{\text{sel}}^\top/\sqrt{d_k}$
attn $\leftarrow$ softmax(puntajes)
$\mathbf{y} \leftarrow \text{attn} \cdot V_{\text{sel}}$ **return y**

7.6.3. Características Clave

- **Complejidad**: $\mathcal{O}(n \cdot d)$ durante el entrenamiento (lineal en la longitud de la secuencia); $\mathcal{O}(k)$ por token durante la generación.
- **Entrenable**: Sí; flujo de gradiente de extremo a extremo a través del operador SparseK.
- **Generación incremental**: Soporta generación autorregresiva eficiente con memoria constante.
- **Fortaleza**: Se integra perfectamente en arquitecturas LLM existentes; mínimo ajuste fino necesario.
- **Limitación**: El acceso a memoria disperso de la selección top-k puede limitar la eficiencia de caché en algunos hardwares; la red de puntuación añade sobrecarga.

7.7. NSA (Native Sparse Attention): Ramas Jerárquicas Alineadas con el Hardware

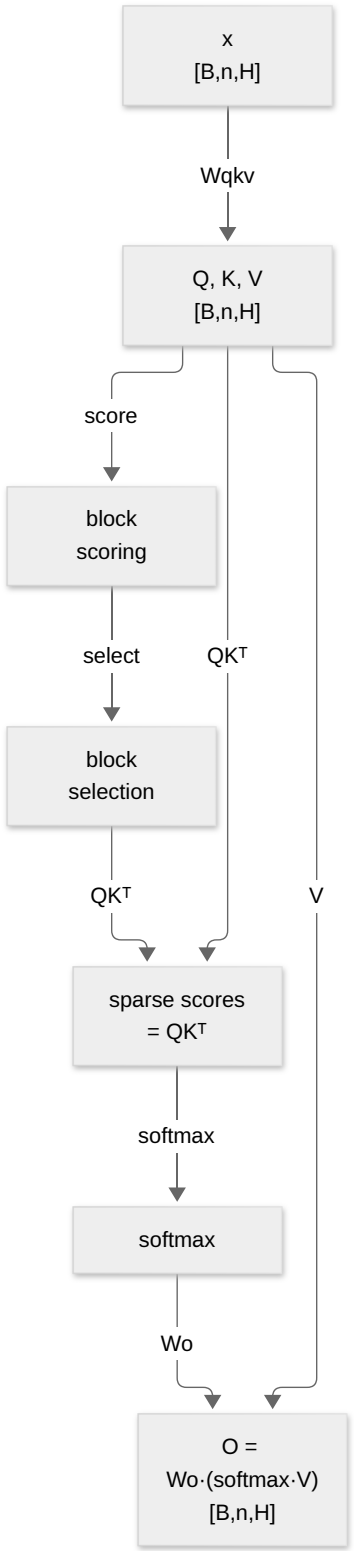


Figura 56: Flujo de datos de NSA: la puntuación y selección jerárquica de bloques restringen la matriz de puntuaciones a los bloques elegidos.

7.7.1. Formulación Matemática

NSA [125] descompone la atención dispersa en tres ramas paralelas que se combinan mediante compuertas aprendidas.

Rama de compresión : Un MLP aprendible φ comprime bloques de clave-valor:

$$\tilde{K}_t^{\text{cmp}} = [\varphi(k_{id+1:id+l})]_{1 \le i \le \lfloor (t-l)/d \rfloor} \quad , \quad \tilde{V}_t^{\text{cmp}} = [\varphi(v_{id+1:id+l})]$$

Rama de selección : Los bloques de alta importancia se seleccionan basándose en puntajes comprimidos:

$$p_t^{\text{cmp}} = \text{softmax}(q_t^\top \tilde{K}_t^{\text{cmp}} / \sqrt{d}), \quad I_t = \text{TopK}(p_t^{\text{cmp}}, n), \quad \tilde{K}_t^{\text{sel}} = \text{Recolectar}(K, I_t)$$

Rama de ventana deslizante : Contexto local fijo:

$$\tilde{K}_t^{\text{win}} = K_{t-w:t}, \quad \tilde{V}_t^{\text{win}} = V_{t-w:t}$$

Combinación con compuertas : Las tres salidas de atención se combinan mediante compuertas aprendidas:

$$\alpha_t^{(\text{cmp})} = \sigma(\text{Lineal}_{\text{cmp}}(q_t)), \quad \alpha_t^{(\text{sel})} = \sigma(\text{Lineal}_{\text{sel}}(q_t)), \quad \alpha_t^{(\text{win})} = \sigma(\text{Lineal}_{\text{win}}(q_t))$$

$$y_t = \alpha_t^{(\text{cmp})} \cdot \text{Attn}(q_t, \tilde{K}^{\text{cmp}}, \tilde{V}^{\text{cmp}}) + \alpha_t^{(\text{sel})} \cdot \text{Attn}(q_t, \tilde{K}^{\text{sel}}, \tilde{V}^{\text{sel}}) + \alpha_t^{(\text{win})} \cdot \text{Attn}(q_t, \tilde{K}^{\text{win}}, \tilde{V}^{\text{win}})$$

7.7.2. Pseudocódigo Algorítmico

Algorithm 31 NSA: Ramas Comprimida + Seleccionada + Ventana con Compuertas Aprendidas

Require: Consulta $\mathbf{q}_t$, secuencias de Clave/Valor almacenadas en caché, tamaño de bloque l, stride d, conteo de selección n, ventana w

Require: MLP de compresión φ, redes de compuerta $\text{Lineal}_{\text{cmp}}, \text{Lineal}_{\text{sel}}, \text{Lineal}_{\text{win}}$

Ensure: Salida $\mathbf{y}_t$

Rama de compresión:

for $i = 1$ to $\lfloor t/d \rfloor$ **do**

  $k_i^{\text{cmp}} \leftarrow \varphi(\mathbf{K}_{id+1:id+l})$ ▷ Comprimir bloque mediante MLP

  $v_i^{\text{cmp}} \leftarrow \varphi(\mathbf{V}_{id+1:id+l})$

end for

$\tilde{\mathbf{K}}^{\text{cmp}} \leftarrow [k_1^{\text{cmp}}, \ldots, k_{\lfloor t/d \rfloor}^{\text{cmp}}]$, $\tilde{\mathbf{V}}^{\text{cmp}} \leftarrow [v_1^{\text{cmp}}, \ldots, v_{\lfloor t/d \rfloor}^{\text{cmp}}]$

$\mathbf{y}_t^{(\text{cmp})} \leftarrow \text{SDPA}(\mathbf{q}_t, \tilde{\mathbf{K}}^{\text{cmp}}, \tilde{\mathbf{V}}^{\text{cmp}})$

Rama de selección:

Calcular puntajes sobre comprimidos: $\mathbf{s} \leftarrow \text{softmax}(\mathbf{q}_t \tilde{\mathbf{K}}^{\text{cmp}\top} / \sqrt{d})$

Seleccionar los n bloques más importantes: $I \leftarrow \text{TopK}(\mathbf{s}, n)$

Recolectar bloques completos: $\tilde{\mathbf{K}}^{\text{sel}} \leftarrow [\mathbf{K}_{I_i d+1:I_i d+l}]_i$, $\tilde{\mathbf{V}}^{\text{sel}} \leftarrow [\mathbf{V}_{I_i d+1:I_i d+l}]_i$

$\mathbf{y}_t^{(\text{sel})} \leftarrow \text{SDPA}(\mathbf{q}_t, \tilde{\mathbf{K}}^{\text{sel}}, \tilde{\mathbf{V}}^{\text{sel}})$

Rama de ventana:

$\mathbf{y}_t^{(\text{win})} \leftarrow \text{SDPA}(\mathbf{q}_t, \mathbf{K}_{t-w:t}, \mathbf{V}_{t-w:t})$

Fusión con compuertas:

$\alpha^{(\text{cmp})} \leftarrow \sigma(\text{Lineal}_{\text{cmp}}(\mathbf{q}_t))$, $\alpha^{(\text{sel})} \leftarrow \sigma(\text{Lineal}_{\text{sel}}(\mathbf{q}_t))$, $\alpha^{(\text{win})} \leftarrow \sigma(\text{Lineal}_{\text{win}}(\mathbf{q}_t))$

$\mathbf{y}_t \leftarrow \alpha^{(\text{cmp})} \mathbf{y}_t^{(\text{cmp})} + \alpha^{(\text{sel})} \mathbf{y}_t^{(\text{sel})} + \alpha^{(\text{win})} \mathbf{y}_t^{(\text{win})}$ **return** $\mathbf{y}_t$

7.7.3. Características Clave

- **Complejidad**: $\mathcal{O}(t/d + n \cdot l + w)$ tokens atendidos por paso (significativamente reducido frente a $\mathcal{O}(t)$ para densa).
- **Entrenable**: Sí; entrenamiento directo con todas las ramas diferenciables.
- **Alineación con hardware**: Diseñado para utilización eficiente de núcleos tensoriales; la compresión y selección reducen la presión sobre el ancho de banda de memoria.
- **Fortaleza**: 11.6× de aceleración en decodificación y 9× de aceleración hacia adelante en secuencias de 64k; supera a la atención completa en muchas tareas.
- **Limitación**: Requiere kernels Triton/CUDA personalizados; la arquitectura multi-rama compleja aumenta la sobrecarga de ingeniería.

7.8. FASA: Atención Dispersa Consciente de la Frecuencia

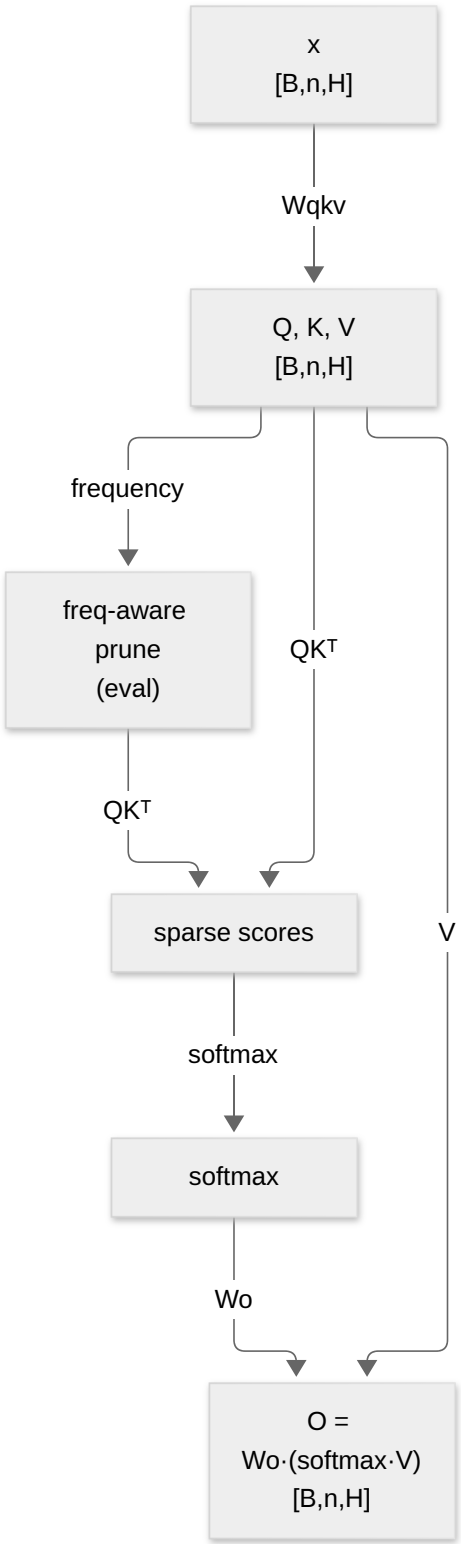


Figura 57: Flujo de datos de FASA: una etapa de poda sensible a la frecuencia (solo evaluación) descarta bloques de claves de bajo valor antes del softmax.

7.8.1. Formulación Matemática

FASA [116] es un **método en tiempo de inferencia sin entrenamiento** que explota la estructura del embedding de posición rotatorio (RoPE). Bajo RoPE, el embedding del token se rota por $\theta_i = B^{-2(i-1)/d}$, descomponiendo el espacio d-dimensional en $d/2$ fragmentos de frecuencia (FC).

Idea clave : Solo un subconjunto pequeño ($< 1\,\%$) de los FC importa para la conciencia contextual; la mayoría codifica patrones posicionales.

Identificación de fragmento de frecuencia dominante : Para cada capa l y cabeza h, identificar FC dominantes mediante concordancia contextual (CA):

$$\mathrm{CA}^{l,h,i} = \frac{|\mathrm{TopK}(\alpha^{l,h}) \cap \mathrm{TopK}(\alpha^{l,h,i})|}{K}$$

donde $\alpha^{l,h}$ es la máscara de atención completa y $\alpha^{l,h,i}$ es la máscara usando solo el FC i. Los FC dominantes tienen CA alta—contribuyen significativamente al patrón de atención final.

Etapa de predicción de importancia de tokens (TIP) : Usando solo FC dominantes, calcular importancia ligera por token:

$$S_t^{l,h} = \sum_{i \in \mathcal{I}_{\mathrm{dom}}^{l,h}} \alpha^{l,h,i}(\mathbf{q}_t, \mathbf{K}_{1:t}), \quad \mathcal{T}_t = \text{TopK-Índices}(S_t, N_{\mathrm{fac}})$$

Etapa de cálculo de atención enfocada (FAC) : Calcular atención de precisión completa en los tokens seleccionados:

$$\hat{\alpha}_{\mathrm{FAC}} = \mathrm{softmax}\left(\frac{\mathbf{q}_t \mathbf{K}_{\mathcal{T}_t}^{\top}}{\sqrt{d}}\right), \quad \mathbf{o}_t = \hat{\alpha}_{\mathrm{FAC}} \mathbf{V}_{\mathcal{T}_t}$$

7.8.2. Pseudocódigo Algorítmico

Algorithm 32 FASA: Atención Dispersa Consciente de la Frecuencia (en Tiempo de Inferencia)

Require: Consulta actual $\mathbf{q}_t$, Clave/Valor almacenados en caché $\mathbf{K}_{1:t}, \mathbf{V}_{1:t}$, base RoPE B, FC dominantes $\mathcal{I}_{\text{dom}}$
Require: Presupuesto TIP N_{tip}, presupuesto FAC N_{fac}
Ensure: Salida $\mathbf{o}_t$

Etapa 1: Predicción de Importancia de Tokens (TIP)
for capa l y cabeza h **do**
  for posición $i = 1$ to t **do**
    Calcular importancia desde FC dominantes: $s_i \leftarrow 0$
    for $fc \in \mathcal{I}_{\text{dom}}^{l,h}$ **do**
      Rotar consulta y clave en FC fc: $\mathbf{q}^{(fc)}, \mathbf{k}_i^{(fc)}$
      $s_i^{(fc)} \leftarrow \text{softmax}(\mathbf{q}^{(fc)} \mathbf{k}_i^{(fc)\top} / \sqrt{d})$
      $s_i \leftarrow s_i + s_i^{(fc)}$
    end for
  end for
  $\mathcal{T}_t \leftarrow \text{TopK-Índices}([s_1, \ldots, s_t], N_{\text{fac}})$ ▷ Seleccionar tokens principales
end for
Etapa 2: Cálculo de Atención Enfocada (FAC)
$\text{puntajes}_{\text{fac}} \leftarrow \mathbf{q}_t \mathbf{K}_{\mathcal{T}_t}^{\top} / \sqrt{d}$ ▷ Producto punto de precisión completa
$\alpha_{\text{fac}} \leftarrow \text{softmax}(\text{puntajes}_{\text{fac}})$ ▷ Softmax completo
$\mathbf{o}_t \leftarrow \alpha_{\text{fac}} \mathbf{V}_{\mathcal{T}_t}$ ▷ Suma ponderada de valores **return** $\mathbf{o}_t$

7.8.3. Características Clave

- **Complejidad**: TIP es $\mathcal{O}(t \cdot N_{\text{tip}})$; FAC es $\mathcal{O}(N_{\text{fac}} \cdot d)$.
- **Entrenable**: No; optimización de inferencia sin entrenamiento.
- **Aplicabilidad**: Requiere modelos basados en RoPE; extendido a ALiBi y MLA con modificaciones.
- **Fortaleza**: Precisión cercana a la óptima con ≤ 256 tokens de entre millones; hasta 2.56× de aceleración en decodificación; ortogonal a la cuantización.
- **Limitación**: Requiere calibración fuera de línea; la identificación de FC dominantes añade sobrecarga de preprocesamiento.

Nota de implementación (entrenamiento vs. inferencia). En este repositorio la ruta dispersa TIP/FAC es solo de inferencia. Mientras el modelo está en modo entrenamiento, la capa híbrida contenedora sustituye atención estándar densa sobre las mismas proyecciones **q**/**k**/**v**/**o**, de modo que el mixer puede entrenarse desde cero y los pesos optimizados durante el entrenamiento son exactamente los que lee la ruta dispersa en evaluación.

7.9. SpargeAttn: Filtrado de Dos Etapas a Nivel de Bloques

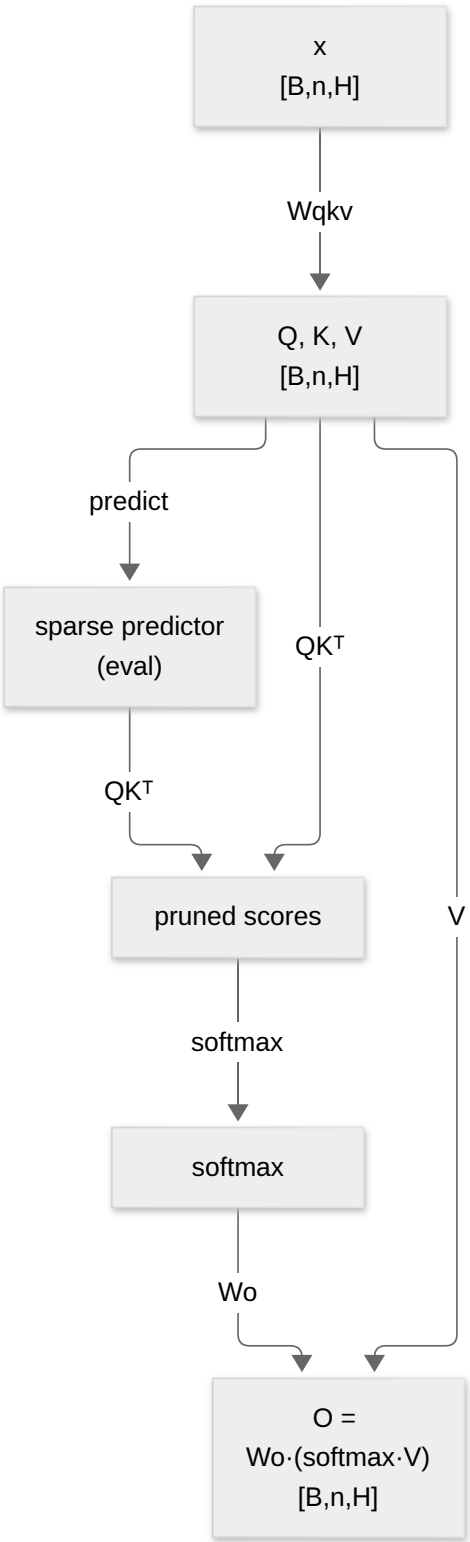


Figura 58: Flujo de datos de SpargeAttn: un predictor disperso (solo evaluación) poda bloques de claves antes del softmax.

7.9.1. Formulación Matemática

SpargeAttn [131] es un **método universal sin entrenamiento** que predice qué bloques de la matriz de atención contendrán valores insignificantes y omite el cálculo para esos bloques.

Etapa 1 — Predicción dispersa : Calcular importancia proxy de bajo costo $\hat{s}_{ij}$ para el bloque (i, j):

$$\hat{s}_{ij} = f_{\text{pred}}(\mathbf{Q}_i, \mathbf{K}_j; \text{hiperparámetros}), \quad \text{omitir si } \hat{s}_{ij} < \epsilon_1$$

Los predictores comunes incluyen similitud de media de bloque o estadísticas de autosimilitud.

Etapa 2 — Filtrado consciente de softmax : Después de calcular $\tilde{P}_{ij} = \text{softmax}(Q_i K_j^\top / \sqrt{d})$, verificar si la probabilidad máxima del bloque es insignificante en relación con el máximo de softmax en ejecución:

$$\text{omitir } P_{ij} V_j \quad \text{si} \quad \text{máx}(\tilde{P}_{ij}) < e^{m_{\text{antiguo}} - m_{\text{nuevo}}} \cdot \epsilon_2$$

donde $m_{\text{antiguo}}, m_{\text{nuevo}}$ son máximos del softmax en línea en ejecución (calculados mediante numéricos estilo FlashAttention).

7.9.2. Pseudocódigo Algorítmico

Algorithm 33 SpargeAttn: Filtrado Disperso de Dos Etapas por Bloques

Require: Bloques de consulta $\mathbf{Q}_i$ para $i = 1, \ldots, n_b$, Bloques de clave $\mathbf{K}_j$ para $j = 1, \ldots, n_b$, Bloques de valor $\mathbf{V}_j$
Require: Umbral de predicción ϵ_1, umbral de softmax ϵ_2, tamaño de bloque b_s
Ensure: Salida $\mathbf{Y}$

Inicializar: máximo de softmax en línea $m_{\text{global}} \leftarrow -\infty$
for fila de bloque $i = 1$ to n_b **do**
  Inicializar: salida de fila de bloque $\mathbf{Y}_i \leftarrow 0$, máximo local $m_i \leftarrow -\infty$
  for columna de bloque $j = 1$ to n_b **do**
    Etapa 1: Predicción Dispersa
    Calcular importancia proxy: $\hat{s}_{ij} \leftarrow f_{\text{pred}}(\mathbf{Q}_i, \mathbf{K}_j)$
    if $\hat{s}_{ij} < \epsilon_1$ **then**
      omitir este bloque $[i, j]$ ▷ Terminación temprana
      continuar ▷ Pasar al siguiente bloque
    end if
    Etapa 2: Calcular y Filtrar
    $\tilde{P}_{ij} \leftarrow \text{softmax}(\mathbf{Q}_i \mathbf{K}_j^\top / \sqrt{d})$
    $m_{\text{bloque}} \leftarrow \text{máx}(\tilde{P}_{ij})$
    if $m_{\text{bloque}} < e^{m_i - m_{\text{global}}} \cdot \epsilon_2$ **then**
      El bloque contribuye insignificantemente; omitir ▷ Poda consciente de softmax
      continuar
    end if
    Actualizar máximo global: $m_i \leftarrow \text{máx}(m_i, m_{\text{bloque}})$
    Acumular: $\mathbf{Y}_i \leftarrow \mathbf{Y}_i + \tilde{P}_{ij} \mathbf{V}_j$
  end for
  $m_{\text{global}} \leftarrow \text{máx}(m_{\text{global}}, m_i)$
end forreturn Y

7.9.3. Características Clave

- **Complejidad**: Empírica $\mathcal{O}(n^2 \cdot s)$ donde s es la fracción de bloques no omitidos (típicamente 0.2–0.5).
- **Entrenable**: No; aceleración plug-and-play para modelos existentes.
- **Universalidad**: Funciona en modelos de lenguaje, modelos de difusión de imágenes y generación de video.
- **Fortaleza**: 2.5–5× de aceleración comparado con métodos densos o dispersos anteriores; compatible con cuantización (integración SageAttention).
- **Limitación**: La aceleración depende de la dispersidad inherente; la granularidad a nivel de bloque puede pasar por alto patrones más finos; se requiere ajuste de umbral por familia de modelos.

Nota de implementación (entrenamiento vs. inferencia). En este repositorio el filtro de bloques en dos etapas es solo de inferencia. Mientras el modelo está en modo entrenamiento, la capa híbrida contenedora sustituye atención estándar densa sobre las mismas proyecciones **q/k/v/o**, de modo que el mixer puede entrenarse desde cero y los pesos optimizados durante el entrenamiento son exactamente los que lee la ruta dispersa en evaluación.

7.10. MSA (MiniMax Sparse Attention): Atención Dispersa por Bloques con Rama de Índice Ligera

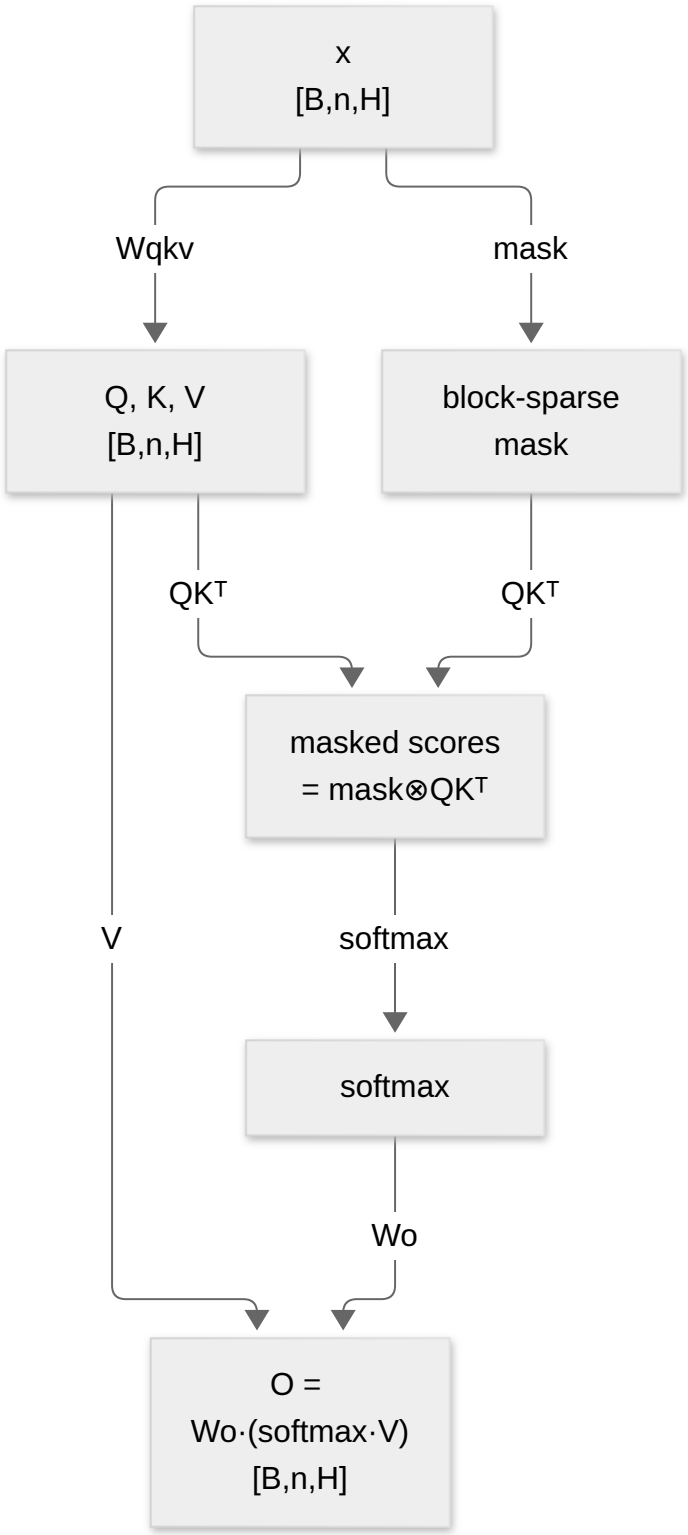


Figura 59: Flujo de datos de MSA: una máscara de bloques dispersos restringe la matriz de puntuaciones antes del softmax.

7.10.1. Formulación Matemática

MSA [67] es un mecanismo de atención dispersa por bloques construido sobre atención agrupada por consultas (GQA) que combina una **Rama de Índice** ligera para la selección de bloques con una **Rama Principal** para atención softmax dispersa por bloques exacta sobre los bloques seleccionados. Sea N la longitud de la secuencia, $B_k = 128$ el tamaño de bloque, y $k = 16$ el número de bloques seleccionados por consulta (lo que yield un presupuesto fijo por consulta de $k \cdot B_k = 2048$ tokens independientemente de N).

Rama de Índice : Una proyección de baja dimensionalidad produce consultas de índice $\mathbf{q}_t^{\text{idx}} \in \mathbb{R}^{d_{\text{idx}}}$ y representantes de bloque $\mathbf{r}_b \in \mathbb{R}^{d_{\text{idx}}}$ para cada bloque KV b. Los puntajes de índice por grupo GQA son:

$$s_{t,b}^{(g)} = \mathbf{q}_t^{\text{idx},(g)} \cdot \mathbf{r}_b, \qquad \mathcal{I}_t^{(g)} = \text{TopK}\left(\{s_{t,b}^{(g)}\}_{b=1}^{N/B_k},\ k\right)$$

La entrada del índice está **desconectada** (stop-gradient), de modo que los gradientes no fluyen hacia atrás a través de la entrada del indexador; la Rama de Índice se entrena en su lugar mediante una pérdida de alineación KL entre su distribución de atención y la distribución de la Rama Principal. Debido a que softmax preserva el orden, los puntajes crudos de índice se alimentan directamente al operador top-k *sin* un softmax explícito (**top-k sin exponencial**), reduciendo el costo numérico.

Bloque local forzado : El bloque que contiene la posición de consulta t **siempre** se incluye en $\mathcal{I}_t^{(g)}$, garantizando cobertura de contexto local.

Rama Principal : La atención estándar de producto punto escalado se calcula exactamente sobre la unión de los bloques seleccionados:

$$\mathbf{o}_t^{(h)} = \text{softmax}\left(\frac{\mathbf{q}_t^{(h)} \mathbf{K}_{\mathcal{I}_t}^{(h)\top}}{\sqrt{d_h}}\right) \mathbf{V}_{\mathcal{I}_t}^{(h)}$$

donde $\mathbf{K}_{\mathcal{I}_t}, \mathbf{V}_{\mathcal{I}_t}$ reúnen los tensores de clave/valor de precisión completa de los bloques seleccionados. La selección se realiza independientemente por grupo GQA, de modo que diferentes grupos de consulta dentro de la misma capa pueden atender a diferentes subconjuntos de bloques.

7.10.2. Pseudocódigo Algorítmico

Algorithm 34 MSA: Atención Dispersa MiniMax con Ramas de Índice + Principal

Require: Consulta $\mathbf{Q} \in \mathbb{R}^{N \times d}$, Clave $\mathbf{K}$, Valor $\mathbf{V}$, tamaño de bloque $B_k = 128$, bloques top-k $k = 16$
Require: Proyección de índice W_{idx} (entrada desconectada), representantes de bloque $\{\mathbf{r}_b\}$
Ensure: Salida $\mathbf{Y} \in \mathbb{R}^{N \times d}$

Calcular representantes de bloque: $\mathbf{r}_b \leftarrow \text{mean}(\mathbf{K}_{bB_k:(b+1)B_k})$ para $b = 1, \ldots, N/B_k$
for posición de consulta $t = 1, \ldots, N$ **do**
  for cada grupo GQA g **do**
    $\mathbf{q}_t^{\text{idx}} \leftarrow \text{sg}(W_{\text{idx}}\mathbf{x}_t)$ ▷ consulta de índice desconectada
    Calcular puntajes de bloque: $s_b \leftarrow \mathbf{q}_t^{\text{idx}} \cdot \mathbf{r}_b$
    Identificar bloque local: $b_{\text{local}} \leftarrow \lfloor t/B_k \rfloor$
    $\mathcal{I}_t^{(g)} \leftarrow \text{TopK}(\{s_b\}, k) \cup \{b_{\text{local}}\}$ ▷ top-k sin exp + local forzado
  end for
  Reunir KV seleccionado: $\tilde{\mathbf{K}}_t \leftarrow \text{Gather}(\mathbf{K}, \mathcal{I}_t)$, $\tilde{\mathbf{V}}_t \leftarrow \text{Gather}(\mathbf{V}, \mathcal{I}_t)$
  Rama principal SDPA: $\mathbf{y}_t \leftarrow \text{softmax}(\mathbf{q}_t\tilde{\mathbf{K}}_t^\top/\sqrt{d_h})\tilde{\mathbf{V}}_t$
end for
Agregar pérdida de alineación KL $\mathcal{L}_{\text{KL}} = \text{KL}(\text{dist índice} \,\|\, \text{dist principal})$ durante el entrenamiento **return Y**

7.10.3. Características Clave

- **Complejidad**: Rama de índice $\mathcal{O}(H_{\text{kv}} \cdot d_{\text{idx}} \cdot N^2)$; Rama principal $\mathcal{O}(H_q \cdot d_h \cdot N \cdot k \cdot B_k)$. Presupuesto por consulta fijo en $k \cdot B_k = 2048$ tokens independientemente de N.
- **Entrenable**: Sí; de extremo a extremo (la Rama de Índice se entrena mediante una pérdida de alineación KL, no es solo inferencia).
- **Estado**: Caché KV $\mathcal{O}(N)$ (tamaño GQA); sin expansión de la dimensión de estado recurrente.
- **Fortaleza**: Desplegado en el modelo 109B MiniMax-M3, MSA logra una reducción de cómputo de atención por token de 28,4× a contexto de 1M, con 14,2× en prefill y 7,6× en decodificación en GPUs H800.
- **Limitación**: La granularidad de selección es fija a nivel de bloque ($B_k = 128$); el bloque local forzado y la selección independiente por grupo aumentan la complejidad de ingeniería y requieren kernels personalizados de bloques dispersos.

7.11. SparDA (Atención Dispersa Desacoplada): Selección de Bloques Anticipada Guiada por Pronóstico

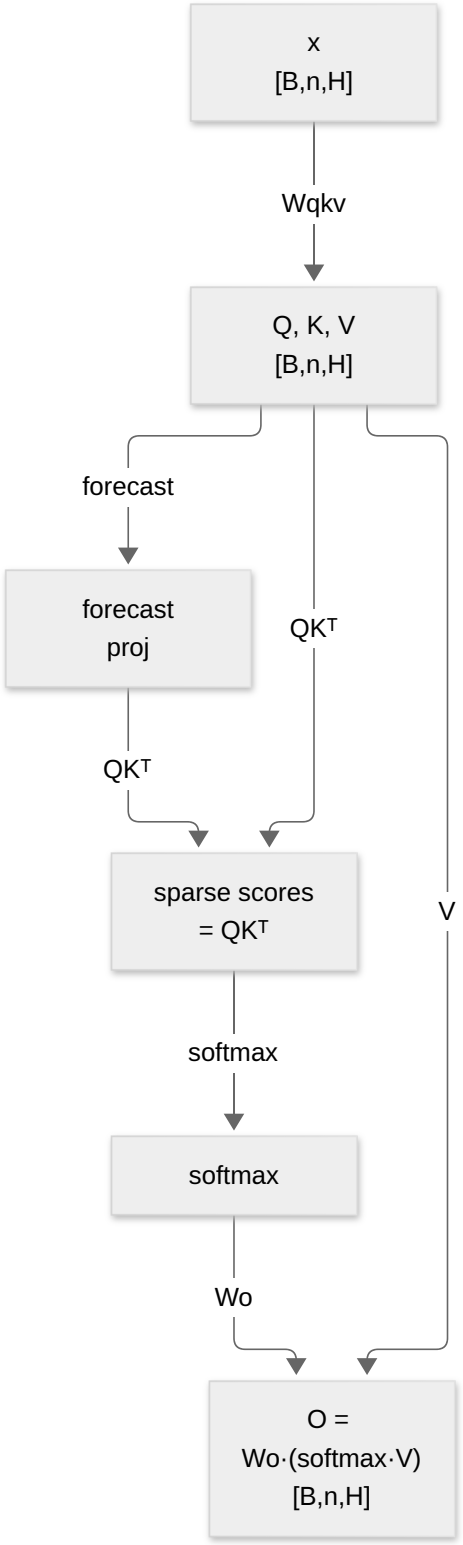


Figura 60: Flujo de datos de SparDA: una proyección de pronóstico guía la selección anticipada de bloques antes del softmax.

7.11.1. Formulación Matemática

SparDA [36] introduce un esquema de atención dispersa **desacoplado** que aumenta las proyecciones estándar por capa Q, K, V con una cuarta proyección, el **Pronóstico**, que predice los bloques KV que serán necesarios en la *siguiente* capa. Esta anticipación permite solapar el prefetch CPU-a-GPU de los bloques seleccionados de la siguiente capa con la ejecución de la capa actual, ocultando la latencia de offload que afecta a las implementaciones de atención dispersa en hardware con memoria limitada.

Proyección de Pronóstico : Para la entrada $\mathbf{x} \in \mathbb{R}^{N \times d}$,

$$\mathbf{F} = \mathbf{x}\mathbf{W}_F \in \mathbb{R}^{N \times H_{\text{kv}} \times d_f}$$

con **una cabeza de Pronóstico por grupo GQA**, lo que reduce la sobrecarga de selección frente a los selectores multi-cabeza usados en trabajos anteriores.

Puntuación y selección de bloques : Para cada consulta i y bloque KV b con representante $\mathbf{r}_b$:

$$S_{i,b} = \mathbf{F}_i \cdot \mathbf{r}_b, \qquad \mathcal{I}_i = \text{TopK}(\{S_{i,b}\}_b, k)$$

Atención dispersa por bloques : La atención estándar de producto punto escalado se calcula entonces sobre los bloques seleccionados:

$$\mathbf{o}_i = \text{softmax}\left(\frac{\mathbf{q}_i \mathbf{K}_{\mathcal{I}_i}^\top}{\sqrt{d_h}}\right)\mathbf{V}_{\mathcal{I}_i}$$

Entrenamiento : SparDA añade menos del 0,5 % de parámetros adicionales. Solo las proyecciones de Pronóstico se entrenan, emparejando la distribución de atención del selector disperso original (congelado)—una destilación ligera que preserva el comportamiento del modelo base mientras habilita el prefetch anticipado.

7.11.2. Pseudocódigo Algorítmico

Algorithm 35 SparDA: Atención Dispersa Desacoplada con Prefetch de Pronóstico

Require: Entrada $\mathbf{x} \in \mathbb{R}^{N \times d}$, bloques de Clave/Valor en caché, bloques top-k k
Require: Proyecciones W_q, W_k, W_v, W_F; selector base congelado π_{orig}
Ensure: Salida $\mathbf{Y}$; bloques prefetched para la siguiente capa
1: Calcular Pronóstico: $\mathbf{F} \leftarrow \mathbf{x}\mathbf{W}_F$ ▷ una cabeza por grupo GQA
2: **for** consulta $i = 1, \ldots, N$ **do**
3: Calcular puntajes de bloque: $S_{i,b} \leftarrow \mathbf{F}_i \cdot \mathbf{r}_b$ para cada bloque b
4: Seleccionar: $\mathcal{I}_i \leftarrow \text{TopK}(\{S_{i,b}\}, k)$
5: **Prefetch** $\mathbf{K}_{\mathcal{I}_i}, \mathbf{V}_{\mathcal{I}_i}$ de CPU a GPU para la *siguiente* capa ▷ solapa con la capa actual
6: **end for**
7: Calcular SDPA dispersa por bloques sobre los bloques seleccionados de la capa actual: $\mathbf{Y} \leftarrow \text{BlockSparseSDPA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}, \{\mathcal{I}_i\})$
8: Pérdida de entrenamiento: $\mathcal{L} \leftarrow \text{KL}(\text{dist Pronóstico} \,\|\, \pi_{\text{orig}})$ ▷ solo se actualiza W_F **return Y**

7.11.3. Características Clave

- **Complejidad**: Pronóstico $\mathcal{O}(H_{\text{kv}} \cdot d_f \cdot N \cdot N_{\text{bloques}})$; Atención principal $\mathcal{O}(H_q \cdot d_h \cdot N \cdot k \cdot B_k)$.

- **Entrenable**: Sí; solo las proyecciones de Pronóstico se entrenan ($< 0{,}5\,\%$ parámetros extra).
- **Fortaleza**: En dos modelos sparse-pretrained de 8B, SparDA logra hasta 1,25× en prefill y 1,7× en decodificación sobre el baseline de offload de atención dispersa, y hasta 5,3× throughput de decodificación frente a un baseline disperso sin offload.
- **Limitación**: Requiere un pipeline de offload CPU–GPU y un selector base congelado; el beneficio anticipado depende de que la demanda de bloques de la siguiente capa sea predecible a partir del Pronóstico actual.

7.12. Espacio de Diseño y Criterios de Selección

Los diez métodos de atención dispersa ocupan posiciones complementarias en un espacio de diseño multidimensional:

- **Patrones geométricos/fijos** (Sparse Transformer, Longformer): Simples de implementar y analizar, pero los patrones no se adaptan al contenido.
- **Desalojo de KV con memoria fija** (SWA con sumideros): La política más barata posible—conservar un prefijo sumidero y una ventana reciente, descartar el resto. Sin parámetros extra, sin entrenamiento, memoria acotada.
- **Selección aprendida** (SparseK, NSA): Permiten adaptación mediante redes de selección entrenables, a costa de mayor complejidad de implementación.
- **Aceleración sin entrenamiento** (FASA, SpargeAttn): Permiten aceleración directa de modelos existentes sin reentrenamiento, adecuados para sistemas desplegados.
- **Garantías teóricas** (BigBird): Proporcionan demostraciones formales de completitud expresiva, importantes para comprender los márgenes de seguridad.

Orientación de selección:

1. **Entrenamiento de nuevos modelos** donde los recursos lo permitan: NSA o SparseK para máxima aceleración de inferencia; BigBird para garantía teórica; SWA con sumideros como el baseline de bajo costo más fuerte.
2. **Comprensión de documentos de contexto largo**: SWA con sumideros (ventana 256–512) para la mejor precisión por byte de caché KV; Longformer o FASA por simplicidad práctica; NSA para escala extrema.
3. **Aceleración de modelos existentes**: SWA con sumideros como baseline sin entrenamiento por defecto (superó a la mayoría de los retrofits de atención lineal en el estudio original); FASA para LLMs basados en RoPE; SpargeAttn para cualquier arquitectura.
4. **Entornos con recursos limitados**: SWA con sumideros para un presupuesto de memoria fijo; Sparse Transformer por simplicidad y eficiencia probada a escala moderada.

8. Familias de Atención con Compuerta—Análisis Exhaustivo de Literatura

8.1. Resumen Ejecutivo: Compuertas para el Control de Memoria

El campo emergente de los *mecanismos de atención con compuerta* aborda un desafío fundamental en el modelado de secuencias: ¿cómo debe retenerse, olvidarse y actualizarse la

información a medida que el modelo procesa nuevos tokens? Los estados recurrentes aditivos tradicionales acumulan toda la información sin borrado, lo que lleva a la saturación de memoria y a la incapacidad de adaptarse a contextos cambiantes. La atención softmax proporciona expresividad completa pero usa un caché KV de $\mathcal{O}(Ld)$ durante la inferencia, limitando el tamaño de la ventana de contexto. Los mecanismos con compuerta proporcionan un punto intermedio: control selectivo sobre la supervivencia de la información.

El principio unificador entre las arquitecturas con compuerta es que las compuertas controlan *qué información sobrevive*. Las compuertas operan en tres niveles distintos:

1. **Decaimiento del estado recurrente** (GLA, HGRN2): Compuertas multiplicativas sobre la matriz de memoria S_t controlan cuánto del estado anterior persiste en el siguiente paso.

2. **Intensidad de escritura en actualizaciones recurrentes** (DeltaNet, Gated DeltaNet, Gated DeltaNet-2): Las compuertas controlan la magnitud y la dirección canalizada de la nueva información escrita en la memoria.

3. **Sesgo de logits softmax** (FoX): Las compuertas inyectan sesgo de actualidad a nivel de token en el cálculo de logits de atención.

4. **Dispersidad post-atención** (Gated Softmax): Las compuertas escalan selectivamente los canales de salida de SDPA.

Este apéndice sintetiza ocho arquitecturas principales de atención con compuerta publicadas entre 2023 y 2026, analizando sus fundamentos matemáticos, características de eficiencia en hardware y compensaciones empíricas.

8.2. 1. Atención Lineal con Compuerta (GLA)

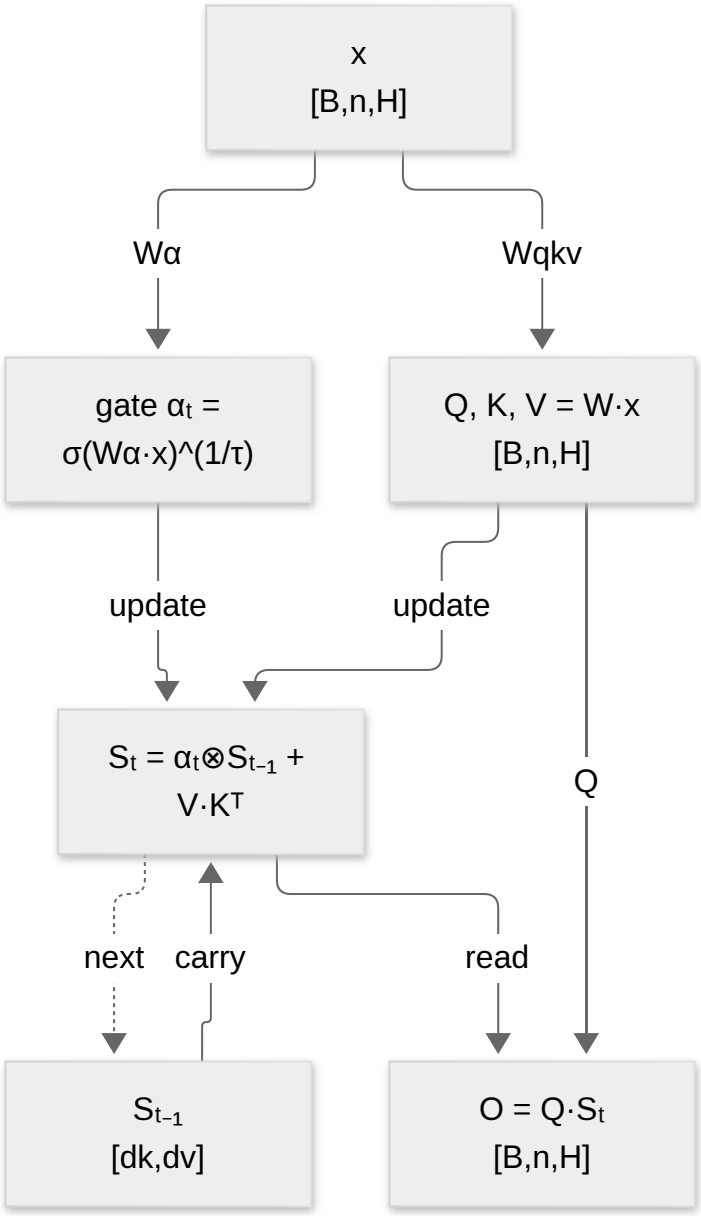


Figura 61: Flujo de datos de GLA: una compuerta sigmoide decae el estado recurrente antes de sumar el producto externo clave/valor.

8.2.1. Fundamento Matemático

La Atención Lineal con Compuerta [121] aumenta la atención lineal estándar (que usa un estado recurrente con valor de matriz) con decaimiento multiplicativo dependiente de los datos. La atención lineal estándar reformula el mecanismo softmax tradicional como una recurrencia sobre estados ocultos:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$$

donde el estado $\mathbf{S}_t \in \mathbb{R}^{d_v \times d_k}$ acumula productos externos. Esta formulación puramente aditiva sufre de "sobrecarga de memoria": la información se acumula monótonamente y no puede borrarse, degradando el rendimiento en tareas que requieren selección de contexto.

GLA introduce una **matriz de compuerta diagonal** $\mathbf{G}_t \in [0,1]^{d_k \times d_k}$ dependiente de los datos:

$$\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$$

La compuerta $\mathbf{G}_t$ se parametriza con una estructura de producto externo:

$$\mathbf{G}_t = \mathbf{1}\boldsymbol{\alpha}_t^\top, \quad \boldsymbol{\alpha}_t = \text{logsigmoid}\left(\mathbf{W}_{gk}\mathbf{x}_t\right)/c$$

donde $\mathbf{W}_{gk}$ es una proyección de rango bajo ($\mathbb{R}^{d \to d_k}$) y $c \approx 16$ es un normalizador. La activación logsigmoid mapea $\alpha_t \in \mathbb{R}$ a aproximadamente $[-1/2, 0]$, y la división por c contrae esto aún más para que esté cerca de la identidad en la inicialización. La compuerta resultante $G_t = 1 + \alpha_t \approx 1$ cerca de la inicialización, luego aprende a decaer ($\alpha_t \to -\infty$) información histórica.

Algorithm 36 Atención Lineal con Compuerta (Forma Recurrente)

Require: Entrada $\mathbf{x}_t$; estado anterior $\mathbf{S}_{t-1}$
Require: Proyecciones: W_q, W_k, W_v (estándar); W_{gk} (proyección de compuerta)
Ensure: Salida $\mathbf{o}_t$ y estado actualizado $\mathbf{S}_t$

$\mathbf{q}_t \leftarrow W_q \mathbf{x}_t$
$\mathbf{k}_t \leftarrow W_k \mathbf{x}_t$
$\mathbf{v}_t \leftarrow W_v \mathbf{x}_t$
Calcular compuerta: $\boldsymbol{\alpha}_t \leftarrow \text{logsigmoid}(W_{gk}\mathbf{x}_t)/16$ ▷ Decaimiento por dimensión de clave
Aplicar compuerta de producto externo: $\mathbf{G}_t \leftarrow \mathbf{1}\boldsymbol{\alpha}_t^\top$ ▷ producto externo diagonal
Decaer estado anterior: $\mathbf{S}_t^{\text{decaimiento}} \leftarrow \mathbf{G}_t \odot \mathbf{S}_{t-1}$ ▷ producto elemento a elemento
Agregar nueva asociación: $\mathbf{S}_t \leftarrow \mathbf{S}_t^{\text{decaimiento}} + \mathbf{v}_t \mathbf{k}_t^\top$
Recuperar mediante consulta: $\mathbf{o}_t^{\text{crudo}} \leftarrow \mathbf{S}_t \mathbf{q}_t$
Aplicar compuerta de salida: $\mathbf{o}_t \leftarrow \text{GroupNorm}(\mathbf{o}_t^{\text{crudo}}) \otimes \text{SiLU}(W_g \mathbf{x}_t)$ **return** $\mathbf{o}_t$, $\mathbf{S}_t$

8.2.2. Entrenamiento Eficiente en Hardware

Para el entrenamiento paralelo, GLA usa un algoritmo por fragmentos que agrupa tokens en fragmentos C y calcula transiciones recurrentes en paralelo:

$$\mathbf{O}_{[t]} = \overleftarrow{\mathbf{Q}}_{[t]} \mathbf{S}_{[t]}^\top + \left(\mathbf{Q}_{[t]} \mathbf{K}_{[t]}^\top \odot \boldsymbol{\Gamma}_{[t]}\right) \mathbf{V}_{[t]}$$

donde $\overleftarrow{\mathbf{Q}}_{[t]}$ atiende al estado en el límite del fragmento (retrospectiva), y $\boldsymbol{\Gamma}_{[t]}$ es la máscara causal con escalado consciente del decaimiento:

$$\boldsymbol{\Gamma}_{[t],ij} = \begin{cases} \prod_{l=j}^{i-1} \alpha_l & \text{si } i > j \text{ (retrospectiva)} \\ \prod_{l=1}^{i-j} \alpha_l & \text{si } i \leq j \text{ (en-fragmento)} \end{cases}$$

Este diseño maximiza las operaciones matmul susceptibles de aceleración mediante núcleos tensoriales, logrando una complejidad total subcuadrática $\mathcal{O}(nLd^2/C)$ donde L es el número de cabezas de atención y C es el tamaño del fragmento.

8.2.3. Fortalezas y Limitaciones

Aspecto	Fortalezas	Limitaciones
Eficiencia computacional	Entrenamiento subcuadrático mediante paralelismo por fragmentos. Inferencia con memoria constante $O(d^2)$.	Requiere kernels CUDA/Triton personalizados; no disponible en todos los frameworks.
Generalización de contexto	Extiende entrenamiento de 2K tokens a más de 20K tokens con degradación de perplejidad insignificante.	Aún rinde por debajo de softmax en benchmarks con mucha recuperación (aguja en el pajar).
Selectividad	Decaimiento dependiente de los datos por dimensión de clave.	La estructura de compuerta limita la selectividad por par clave-valor; no hay control directo sobre qué asociaciones específicas borrar.

8.3. 2. DeltaNet: Atención Lineal con Corrección de Errores

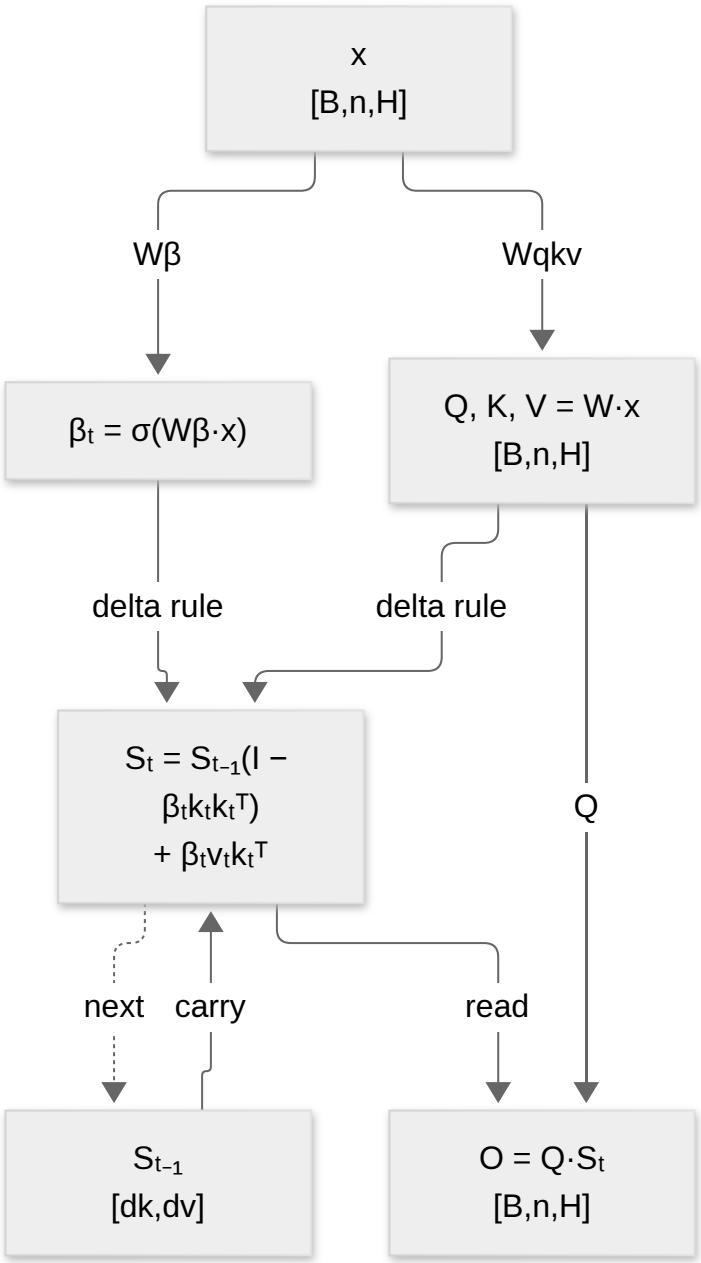


Figura 62: Flujo de datos de DeltaNet: la compuerta beta aplica una regla delta correctora de errores al estado recurrente.

8.3.1. Fundamento Matemático

DeltaNet [122] aplica una **regla de aprendizaje delta** clásica a las actualizaciones de estado de la atención lineal. En lugar de acumulación aditiva, DeltaNet calcula la diferencia entre el valor predicho (recuperado de la memoria) y el valor objetivo, luego realiza una actualización de corrección de errores:

$$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$$

donde $\beta_t \in (0,1)$ es un escalar de **intensidad de escritura** dependiente de los datos. Esta actualización puede descomponerse como una operación de borrado-escritura:

$$\mathbf{v}_t^{\text{anterior}} = \mathbf{S}_{t-1}\mathbf{k}_t \quad \text{(predicción actual)}, \quad \mathbf{v}_t^{\text{actualizado}} = \beta_t \mathbf{v}_t + (1-\beta_t)\mathbf{v}_t^{\text{anterior}} \quad \text{(mezcla anterior/nuevo)}$$

de modo que:

$$\mathbf{S}_t = \mathbf{S}_{t-1} - \mathbf{v}_t^{\text{anterior}}\mathbf{k}_t^\top + \mathbf{v}_t^{\text{actualizado}}\mathbf{k}_t^\top$$

La idea clave es que esta regla de actualización minimiza una pérdida de ECM en línea en cada paso temporal:

$$\mathcal{L}_t(\mathbf{S}) = \tfrac{1}{2}\|\mathbf{S}\mathbf{k}_t - \mathbf{v}_t\|^2 \Rightarrow \nabla_{\mathbf{S}}\mathcal{L}_t = (\mathbf{S}\mathbf{k}_t - \mathbf{v}_t)\mathbf{k}_t^\top$$

Un paso de SGD con tasa de aprendizaje β_t produce la actualización de la regla delta. Esta conexión con el entrenamiento en tiempo de prueba (TTT) proporciona fundamentación teórica: DeltaNet optimiza directamente la predicción de valores en tiempo de inferencia.

8.3.2. Entrenamiento Paralelo Eficiente

La matriz de transición de DeltaNet $\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top$ es una matriz de Householder generalizada (actualización de rango 1 a la identidad). Para el entrenamiento por fragmentos, DeltaNet usa la **representación WY**, que representa el producto de m de estas actualizaciones de rango 1 de forma compacta:

$$\prod_{i=1}^{m}(\mathbf{I} - \beta_i \mathbf{k}_i \mathbf{k}_i^\top) = \mathbf{I} - \mathbf{W}\mathbf{Y}^\top$$

donde $\mathbf{W}, \mathbf{Y} \in \mathbb{R}^{d\times m}$ se construyen recursivamente. Esta factorización permite paralelismo rico en matmul sin materializar el producto completo, manteniendo el entrenamiento por fragmentos eficiente.

Algorithm 37 Actualización de Estado de DeltaNet con Representación WY

Require: Fragmento de entrada $\mathbf{X}_{[t]}$; estado anterior $\mathbf{S}_{t-1}$
Require: Consultas normalizadas L2 $\hat{\mathbf{Q}}_t$, claves $\hat{\mathbf{K}}_t$, valores $\mathbf{V}_t$
Ensure: Salida $\mathbf{O}_{[t]}$ y estado actualizado $\mathbf{S}_t$

1: **Fase por fragmentos:**
2: **for** posición $i = 1$ to tamaño_fragmento **do**
3: Normalizar: $\hat{\mathbf{k}}_i \leftarrow \hat{\mathbf{K}}_t[i]/\|\hat{\mathbf{K}}_t[i]\|$, $\hat{\mathbf{q}}_i \leftarrow \hat{\mathbf{Q}}_t[i]/\|\hat{\mathbf{Q}}_t[i]\|$
4: Calcular intensidad de escritura: $\beta_i \leftarrow \text{sigmoid}(W_\beta \mathbf{x}_i)$
5: Predicción: $\hat{\mathbf{v}}^{\text{pred}} \leftarrow \mathbf{S}_{i-1}^{\text{en-fragmento}}\hat{\mathbf{k}}_i$
6: Mezcla consciente del error: $\mathbf{v}_i^{\text{actualización}} \leftarrow \beta_i(\mathbf{v}_i - \hat{\mathbf{v}}^{\text{pred}})$
7: Borrar-escribir: $\mathbf{S}_i^{\text{en-fragmento}} \leftarrow \mathbf{S}_{i-1}^{\text{en-fragmento}} - \hat{\mathbf{v}}^{\text{pred}}\hat{\mathbf{k}}_i^\top + (\beta_i \mathbf{v}_i + (1-\beta_i)\hat{\mathbf{v}}^{\text{pred}})\hat{\mathbf{k}}_i^\top$
8: Salida: $\mathbf{o}_i \leftarrow \mathbf{S}_i^{\text{en-fragmento}}\hat{\mathbf{q}}_i$
9: **end for**
10: **Fase entre fragmentos:** Usar representación WY de los productos de transición **return** $\mathbf{O}_{[t]}$, $\mathbf{S}_t$

8.3.3. Fortalezas y Limitaciones

Aspecto	Fortalezas	Limitaciones
Recuperación asociativa	Recuperación perfecta en el benchmark MQAR (Recuperación Asociativa Multi-Consulta); la semántica de corrección de errores se ajusta naturalmente a patrones de recuperación.	Sin olvido global, la memoria aún se satura en longitudes de secuencia extremas; requiere mecanismos de reinicio periódico para contexto ilimitado.
Teoría	Fundamentada en optimización de ECM en línea; conexión clara con el paradigma de entrenamiento en tiempo de prueba.	Requiere claves normalizadas L2 para estabilidad numérica; la normalización de doble ancho añade sobrecarga.
Rendimiento	La representación WY permite entrenamiento eficiente por fragmentos.	Ligeramente más lento que Mamba2 por token debido a matrices de transición más ricas (rango 1 en lugar de diagonales).

8.4. 3. Gated DeltaNet: Síntesis de Compuerta y Corrección de Errores

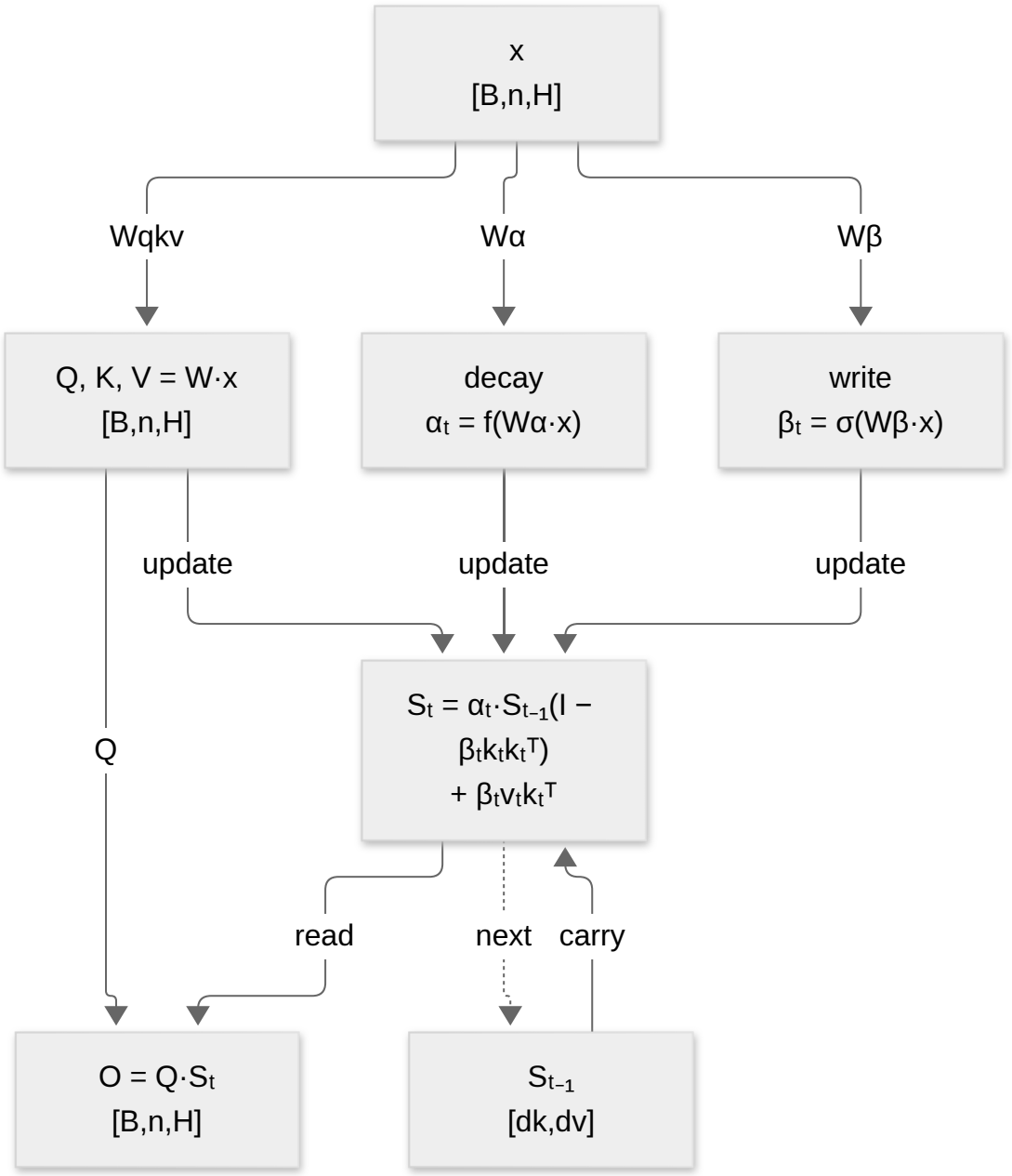


Figura 63: Flujo de datos de Gated DeltaNet: las compuertas de decaimiento y escritura modulan la actualización del estado por regla delta.

8.4.1. Fundamento Matemático

Gated DeltaNet [123] sintetiza las fortalezas de GLA y DeltaNet combinando una **compuerta de decaimiento** α_t (de GLA) con la **compuerta de intensidad de escritura** β_t (de DeltaNet):

$$\mathbf{S}_t = \mathbf{S}_{t-1}\left(\alpha_t(\mathbf{I} - \beta_t \mathbf{k}_t\mathbf{k}_t^\top)\right) + \beta_t \mathbf{v}_t\mathbf{k}_t^\top$$

Reescribiendo para mayor claridad:

$$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t\mathbf{k}_t^\top) + \beta_t \mathbf{v}_t\mathbf{k}_t^\top$$

Las dos compuertas son complementarias:

- $\alpha_t \to 0$ borra rápidamente el estado histórico: $\mathbf{S}_t \approx \beta_t \mathbf{v}_t \mathbf{k}_t^\top$ (reinicio de contexto).
- $\alpha_t \to 1$ recupera el comportamiento puro de regla delta: $\mathbf{S}_t \approx \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$ (actualizaciones dirigidas).
- $\beta_t \to 0$ omite la escritura pero decae: $\mathbf{S}_t \approx \alpha_t \mathbf{S}_{t-1}$ (olvido puro).
- $\beta_t \to 1$ reemplaza la memoria agresivamente: $\mathbf{S}_t \approx \alpha_t \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$ (reemplazo).

Desde una perspectiva de aprendizaje en línea, Gated DeltaNet corresponde a:

$$\min_{\mathbf{S}_t} \|\mathbf{S}_t - \alpha_t \mathbf{S}_{t-1}\|_F^2 - 2\langle \mathbf{S}_t \mathbf{k}_t, \beta_t(\mathbf{v}_t - \alpha_t \mathbf{S}_{t-1}\mathbf{k}_t)\rangle$$

que introduce un decaimiento de pesos adaptativo α_t en una actualización tipo SGD—análogo al decaimiento de pesos desacoplado en la optimización de aprendizaje profundo.

Algorithm 38 Paso Adelante Paralelo por Fragmentos de Gated DeltaNet

Require: Fragmento de entrada $\mathbf{X}_{[i]}$; estado anterior $\mathbf{S}_{i-1}$; tamaño de fragmento C
Require: Proyecciones: $W_q, W_k, W_v, W_\alpha, W_\beta$
Ensure: Salida $\mathbf{O}_{[i]}$ y estado $\mathbf{S}_i$

Cálculo en-fragmento:
for $j = 1$ to C **do**
  $\mathbf{q}_j \leftarrow W_q \mathbf{x}_j$, $\mathbf{k}_j \leftarrow W_k \mathbf{x}_j$, $\mathbf{v}_j \leftarrow W_v \mathbf{x}_j$
  $\alpha_j \leftarrow \text{sigmoid}(W_\alpha \mathbf{x}_j)$, $\beta_j \leftarrow \text{sigmoid}(W_\beta \mathbf{x}_j)$
  Aplicar compuerta combinada: $\mathbf{S}_j \leftarrow \alpha_j \mathbf{S}_{j-1}(\mathbf{I} - \beta_j \mathbf{k}_j \mathbf{k}_j^\top) + \beta_j \mathbf{v}_j \mathbf{k}_j^\top$
  $\mathbf{o}_j \leftarrow \mathbf{S}_j \mathbf{q}_j$
end for
Recurrencia entre fragmentos:
$\mathbf{S}_i \leftarrow \mathbf{S}_C$ del en-fragmento; propagar entre fragmentos mediante máscaras $\prod \alpha$ acumulativas
Compuerta de salida: $\mathbf{O}_{[i]} \leftarrow \text{GroupNorm}(\mathbf{O}^{\text{crudo}}) \otimes \text{SiLU}(W_g \mathbf{X}_{[i]})$ **return** $\mathbf{O}_{[i]}$, $\mathbf{S}_i$

8.4.2. Rendimiento Empírico y Compensaciones

Gated DeltaNet se ha integrado en los modelos de producción Qwen3-Next y Qwen3.5 de Alibaba. Empíricamente:

Tarea	Gated DeltaNet	Mamba2	DeltaNet	GLA
Modelado de Lenguaje	**1,0**×	1,08×	1,05×	1,12×
Recuperación Asociativa	**1,0**×	$--$ ($sinperfecto$)	**1,0**×	0,75
Extrapolación de Longitud	**1,0**×	0,98×	0,96×	0,94×
Rendimiento (tokens/seg)	0,85×	**1,0**×	0,82×	0,88×

Gated DeltaNet logra el mejor equilibrio entre diversas tareas a costa de un rendimiento ligeramente reducido debido a matrices de transición más ricas.

8.5. 4. HGRN2: Compuerta Jerárquica con Expansión de Producto Externo

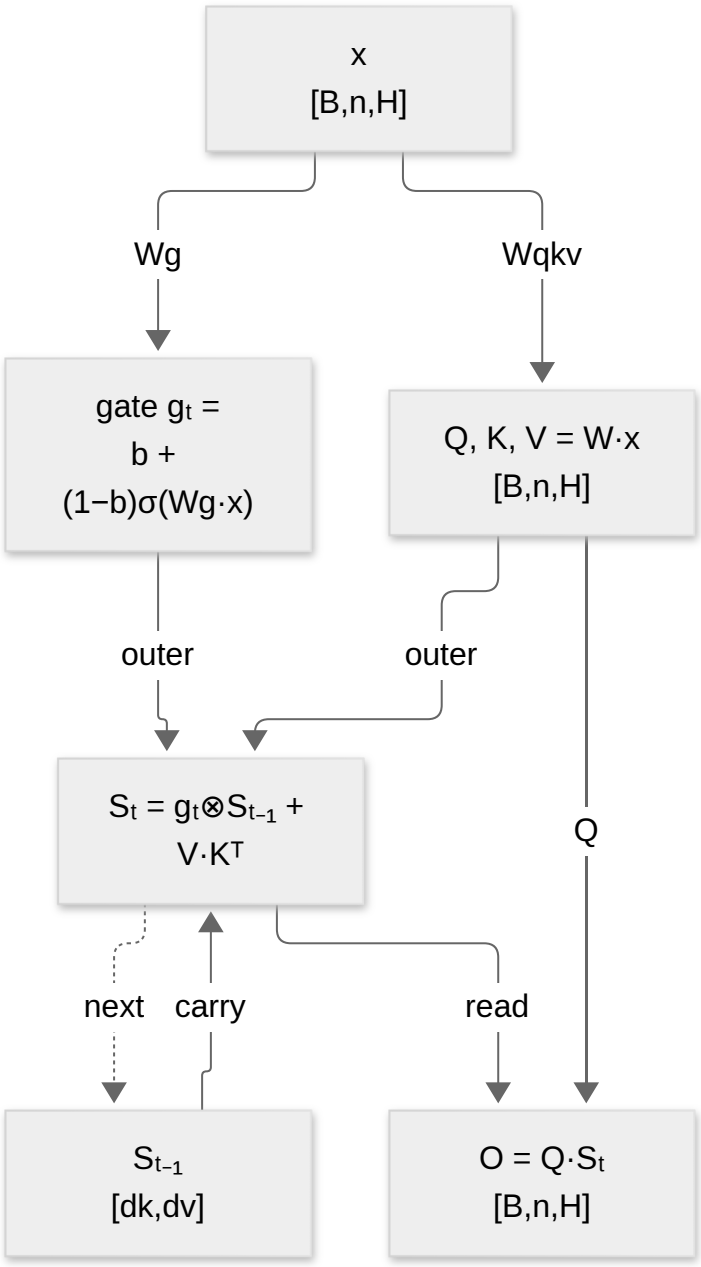


Figura 64: Flujo de datos de HGRN2: una compuerta jerárquica mezcla el estado transportado con el producto externo clave/valor.

8.5.1. Fundamento Matemático

HGRN2 [94] usa una expansión de estado basada en producto externo con **compuertas de olvido acotadas inferiormente de forma jerárquica**. La actualización de estado es:

$$\mathbf{S}_t = \operatorname{diag}(\mathbf{g}_t) \cdot \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$$

donde $\mathbf{g}_t \in [b_\ell, 1]^d$ es una compuerta de olvido con cota inferior para la capa ℓ. Las cotas satisfacen:

$$0 \leq b_{\ell_{\text{poco profunda}}} < b_{\ell_{\text{media}}} < b_{\ell_{\text{profunda}}} \leq 1$$

es decir, las cotas aumentan (se vuelven menos restrictivas) en las capas más profundas. La compuerta en sí se calcula como:

$$\mathbf{g}_t = b_\ell + (1 - b_\ell) \cdot \sigma(W_g \mathbf{x}_t)$$

donde σ es sigmoide. Cuando W_g produce logits débilmente negativos, $\mathbf{g}_t \approx b_\ell$ (retención forzada en capas superficiales). Cuando las salidas son fuertemente positivas, $\mathbf{g}_t \approx 1$ (refresco de memoria en cualquier capa).

La estructura jerárquica anima a diferentes capas a especializarse en diferentes escalas temporales: las capas superficiales modelan dependencias locales (alta retención debido a la cota b_ℓ), mientras que las capas profundas capturan estructura de largo alcance (compuerta flexible con cota superior alta).

Algorithm 39 Paso Adelante de HGRN2 con Cotas Jerárquicas

Require: Entrada $\mathbf{x}_t$; estado anterior $\mathbf{S}_{t-1}$; índice de capa $\ell \in [0, L-1]$
Require: Cotas no decrecientes: $0 \leq b_0 < b_1 < \cdots < b_{L-1} \leq 1$
Ensure: Salida $\mathbf{o}_t$ y estado $\mathbf{S}_t$
1: Cota de retención específica de capa: $b \leftarrow b_\ell$
2: Proyectar: $\mathbf{q}_t \leftarrow W_q \mathbf{x}_t$, $\mathbf{k}_t \leftarrow W_k \mathbf{x}_t$, $\mathbf{v}_t \leftarrow W_v \mathbf{x}_t$
3: Calcular compuerta de olvido con cota: $\mathbf{g}_t \leftarrow b + (1 - b) \cdot \sigma(W_g \mathbf{x}_t)$ ▷ Confinado a $[b, 1]$
4: Multiplicación diagonal (vectorizada): $\mathbf{S}_t \leftarrow \mathbf{S}_{t-1} \circ \text{diag}(\mathbf{g}_t) + \mathbf{v}_t \mathbf{k}_t^\top$
5: Recuperar: $\mathbf{o}_t \leftarrow \mathbf{S}_t \mathbf{q}_t$
6: Normalización opcional: $\mathbf{o}_t \leftarrow \text{RMSNorm}(\mathbf{o}_t)$ **return** $\mathbf{o}_t$, $\mathbf{S}_t$

8.5.2. Escalado y Resultados Empíricos

A escala de 3B en 100B tokens, HGRN2 supera ligeramente a Mamba2 y a los Transformers con arquitectura LLaMA en modelado de lenguaje. La compuerta jerárquica proporciona modelado temporal multi-escala sin variantes arquitectónicas explícitas por capa. Sin embargo, la compuerta diagonal con valor vectorial (en oposición a la selección elemento a elemento) proporciona menos flexibilidad por elemento que las variantes de regla delta.

8.6. 5. Forgetting Transformer (FoX): Compuerta en el Espacio de Logits Softmax

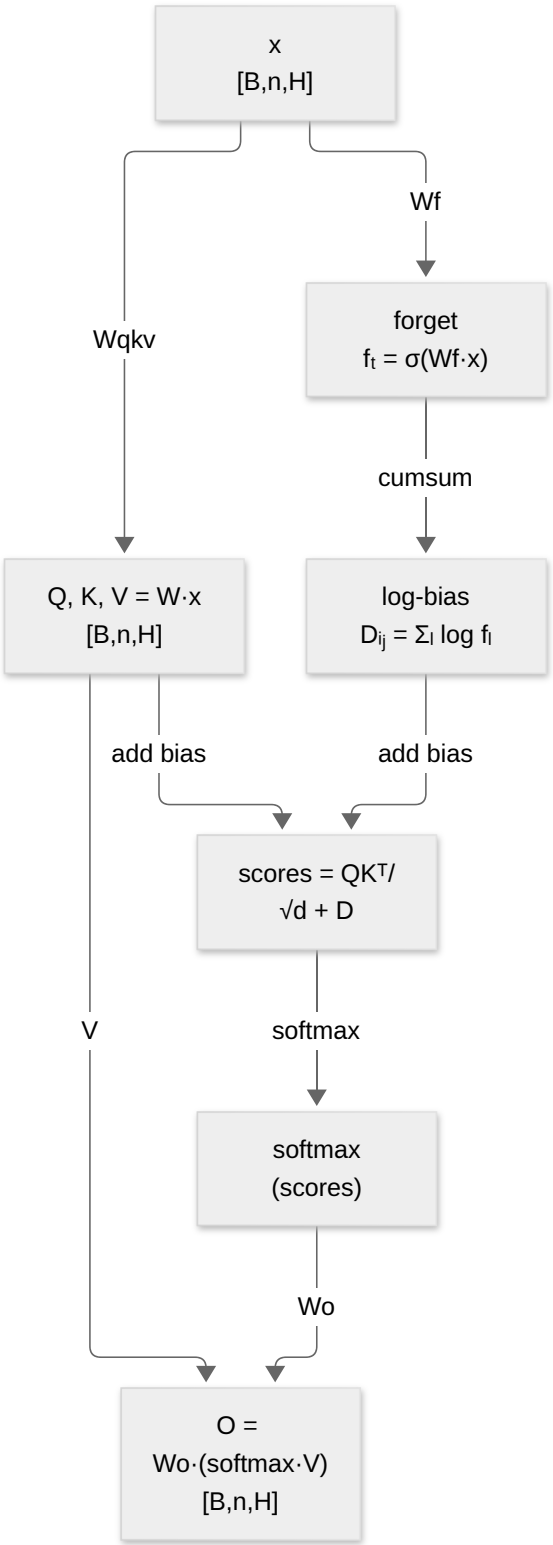


Figura 65: Flujo de datos de FoX: una compuerta de olvido aprendida se convierte en un sesgo aditivo en el espacio de logits.

8.6.1. Fundamento Matemático

Forgetting Transformer (FoX), propuesto por Lin et al. [71], incrusta una compuerta de olvido directamente en los logits de atención softmax. En lugar de reemplazar softmax con recurrencia lineal, FoX preserva la expresividad completa de softmax mientras añade control de actualidad mediante un sesgo de logit dependiente de los datos.

Para cada posición de token t y todas las claves $j \leq t$, calcular un escalar de compuerta de olvido:

$$f_t = \sigma(w_f^\top \mathbf{x}_t + b_f)$$

El sesgo de logit en la posición (i, j) es el producto acumulativo log-olvido:

$$d_{ij} = \sum_{l=j+1}^{i} \log f_l = \log \prod_{l=j+1}^{i} f_l$$

La salida de atención completa se convierte en:

$$\mathbf{O} = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} + \mathbf{D}\right)\mathbf{V}$$

donde $\mathbf{D}_{ij} = d_{ij}$. Equivalentemente:

$$\text{Atencion}(i, j) = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_k} + d_{ij})}{\sum_{j'=1}^{i} \exp(\mathbf{q}_i^\top \mathbf{k}_{j'} / \sqrt{d_k} + d_{ij'})}$$

Esta ponderación reduce el peso de los tokens más antiguos multiplicativamente: un token de hace 10 pasos se escala por $\prod_{l=j+1}^{i} f_l$, que es típicamente mucho menor que 1 si $f_t < 1$ en promedio.

El mecanismo es matemáticamente equivalente a una **variante aprendible dependiente de los datos de ALiBi (Atención con Sesgos Lineales)**, donde trabajos anteriores usaban $d_{ij} = -\infty \cdot |i - j|$ fijo o $d_{ij} = -(i - j)$.

Algorithm 40 FoX: Atención con Olvido y Sesgo de Actualidad

Require: Consultas $\mathbf{Q}$, claves $\mathbf{K}$, valores $\mathbf{V}$; entrada $\mathbf{X}$
Require: Parámetros de compuerta de olvido: $w_f \in \mathbb{R}^d$, $b_f \in \mathbb{R}$
Ensure: Salida de atención $\mathbf{O}$

Calcular compuertas de olvido:
for $t = 1$ to T **do**
    $f_t \leftarrow \sigma(w_f^\top \mathbf{x}_t + b_f)$ ▷ Probabilidad de olvido por token
end for
Calcular sesgos acumulativos de log-olvido:
for $i = 1$ to T **do**
    for $j = 1$ to i **do**
        $d_{ij} \leftarrow \sum_{l=j+1}^{i} \log f_l$ ▷ Producto acumulativo en espacio logarítmico
    end for
end for
SDPA estándar con sesgo:
Calcular puntajes: $\mathbf{S} \leftarrow \mathbf{QK}^\top / \sqrt{d_k}$
Agregar sesgo: $\mathbf{S} \leftarrow \mathbf{S} + \mathbf{D}$
Aplicar softmax: $\mathbf{A} \leftarrow \text{softmax}(\mathbf{S}, \text{dim} = -1)$
Ponderar valores: $\mathbf{O} \leftarrow \mathbf{AV}$ **return** $\mathbf{O}$

8.6.2. Integración con FlashAttention

La ventaja clave de FoX es la compatibilidad con FlashAttention y las implementaciones optimizadas existentes. Los sesgos de olvido se añaden en el cálculo de logits softmax, que ya es una operación central en el algoritmo por bloques de FlashAttention. La sobrecarga es:

$$\text{Sobrecarga} \approx 0{,}5 - -2\,\% \text{ del tiempo de pared}$$

ya que la adición de sesgo se amortiza entre las operaciones matmul.

8.6.3. Fortalezas y Limitaciones

FoX logra resultados de última generación en extrapolación de longitud, recuperación casi perfecta de aguja en el pajar y comprensión superior de contexto largo en comparación con los Transformers. Sin embargo, sigue siendo $\mathcal{O}(L^2 d)$ en entrenamiento y $\mathcal{O}(Ld)$ por paso en inferencia con caché KV, limitando las longitudes de secuencia extremas.

8.7. 6. Atención con Compuerta (Post-SDPA Sigmoide)

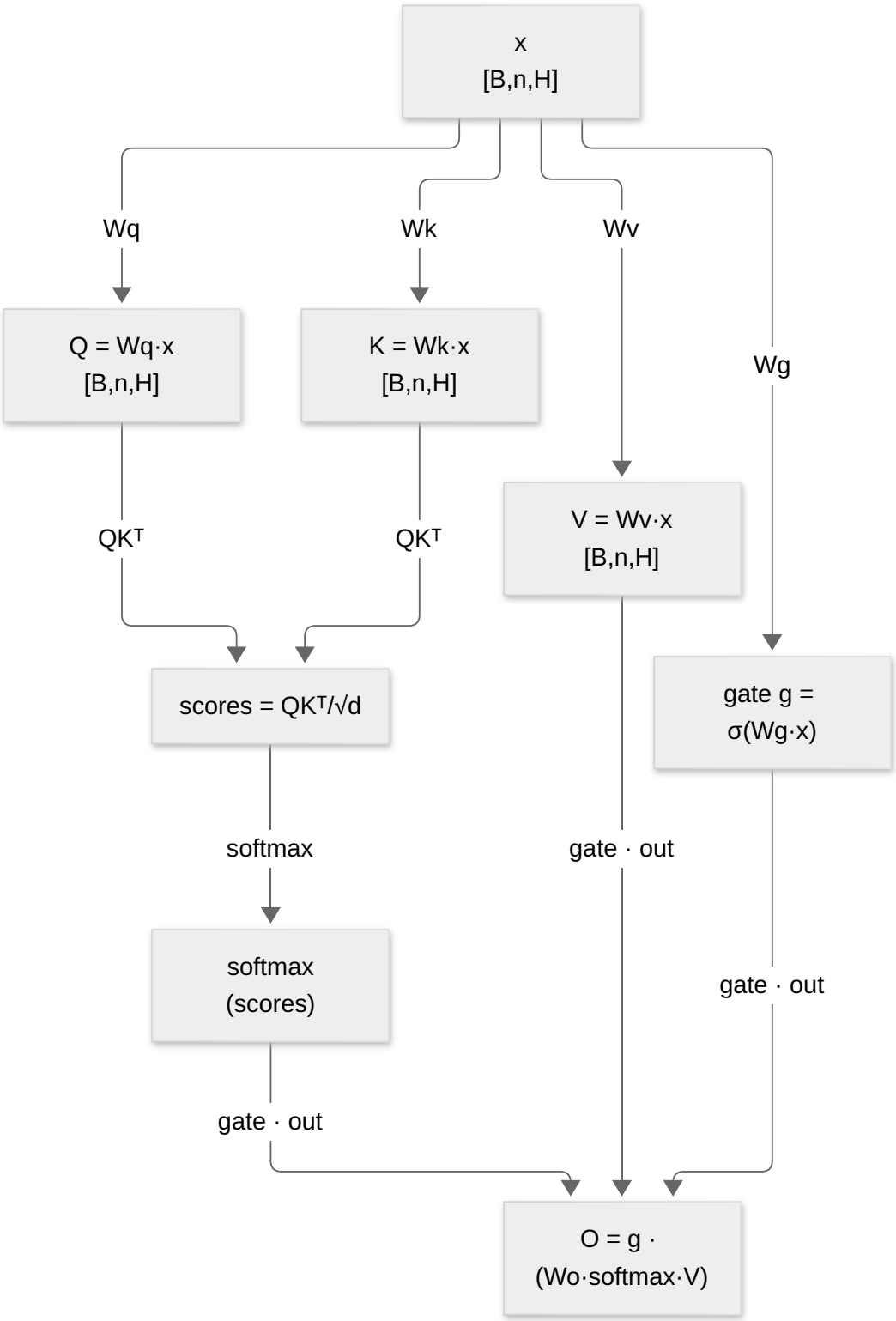


Figura 66: Flujo de datos de la atención con compuerta posterior a SDPA: una compuerta sigmoide escala la salida del softmax antes de la proyección.

8.7.1. Fundamento Matemático

El Mejor Artículo de NeurIPS 2025 del equipo de Qwen propone aplicar una compuerta sigmoide **después** de la atención de producto punto escalado (SDPA):

$$\mathbf{Y} = \text{SDPA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

$$\mathbf{Y}' = \mathbf{Y} \odot \sigma\left(\mathbf{X}\mathbf{W}_\theta\right)$$

donde el puntaje de compuerta $\sigma(\mathbf{X}\mathbf{W}_\theta)$ puede tener forma $\mathbb{R}^{n\times q\times d_k}$ (compuerta elemento a elemento por cabeza de consulta) o $\mathbb{R}^{n\times q}$ (compuerta fija por cabeza). En más de 30 variantes y modelos MoE de 15B + densos de 1.7B entrenados en 3.5T tokens, el equipo encontró:

- La compuerta por cabeza añade solo $\sim 1{,}6M$ parámetros a un modelo de 15B (sobrecarga insignificante).
- Las compuertas efectivas son **dispersas**: activación media $\approx 0{,}116$ (es decir, el 88 % son cero o casi cero).
- Las compuertas actúan como filtros dependientes de la consulta, suprimiendo canales de bajo valor mientras preservan señales de alto valor.
- Las compuertas eliminan demostrablemente el fenómeno de **sumidero de atención**, donde el patrón de atención se vuelve dominado por un único token temprano.

Algorithm 41 Atención Softmax con Compuerta (Post-SDPA)

Require: Consultas $\mathbf{Q}$, Claves $\mathbf{K}$, Valores $\mathbf{V}$; entrada $\mathbf{X}$
Require: Matriz de proyección de compuerta $W_g \in \mathbb{R}^{d\to d_{\text{compuerta}}}$ donde $d_{\text{compuerta}} \in \{1, d_k\}$
Ensure: Salida con compuerta $\mathbf{Y}'$
SDPA estándar:
Calcular atención: $\mathbf{A} \leftarrow \text{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d_k})$
Ponderar valores: $\mathbf{Y} \leftarrow \mathbf{A}\mathbf{V}$
Calcular compuertas:
Proyectar entrada: $\mathbf{G} \leftarrow \mathbf{X}W_g$ ▷ forma: $[n, q, d_{\text{compuerta}}]$
Aplicar sigmoide: $\mathbf{G} \leftarrow \sigma(\mathbf{G})$ ▷ Compuerta elemento a elemento
Aplicar compuerta:
if $d_{\text{compuerta}} = 1$ **then**
    Transmitir y escalar: $\mathbf{Y}' \leftarrow \mathbf{Y} \cdot \mathbf{G}$ ▷ Escalado por cabeza
else ▷ $d_{\text{compuerta}} = d_k$
    Modulación elemento a elemento: $\mathbf{Y}' \leftarrow \mathbf{Y} \odot \mathbf{G}$ ▷ Compuerta por dimensión
end ifreturn $\mathbf{Y}'$

8.7.2. Hallazgos Clave

- **Supresión del sumidero de atención**: La compuerta rompe el ciclo de retroalimentación donde softmax se concentra en el primer token, permitiendo una atención más equilibrada.
- **Estabilidad de entrenamiento**: Los modelos con compuerta toleran tasas de aprendizaje más grandes (+30 % mayor η_{max} estable).
- **Sobrecarga mínima**: El impacto en latencia es solo del 1,6 % en GPUs H100; el rendimiento cae como máximo $2-3$ %.
- **Mejora en la ley de escalado**: Mejora la pérdida en todas las escalas de modelo, desde 800M hasta 110B parámetros de forma consistente.

8.8. 7. Gated DeltaNet-2: Borrado y Escritura Canalizados Decouplados

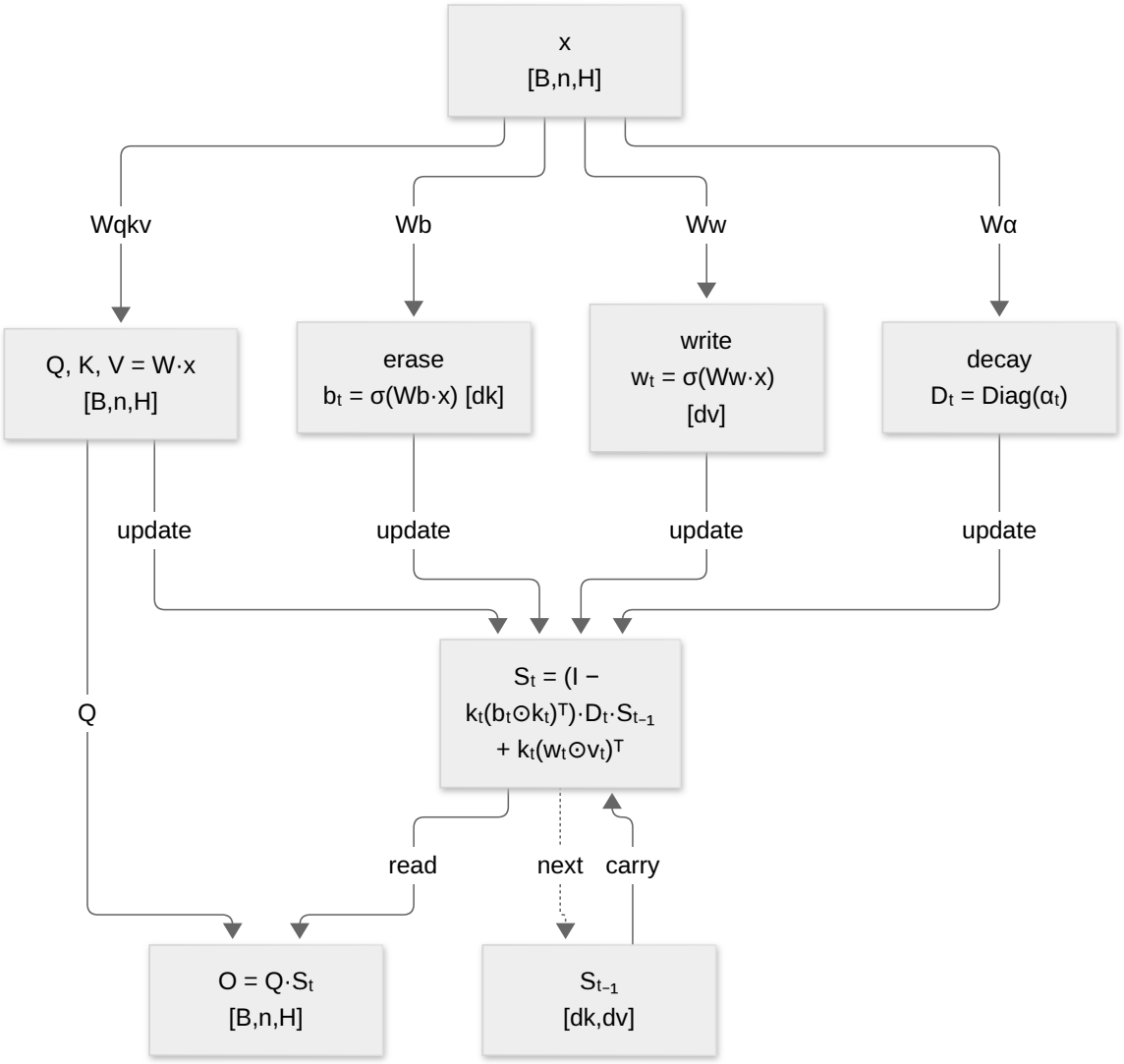


Figura 67: Flujo de datos de Gated DeltaNet-2: compuertas desacopladas de borrado y escritura por canal actúan dentro de la regla delta.

8.8.1. Fundamento Matemático

Gated DeltaNet-2 [48] aborda una limitación compartida de Gated DeltaNet y Kimi Delta Attention (KDA): ambos usan una única *compuerta delta escalar* β_t para controlar simultáneamente (i) cuánto contenido antiguo se borra en la dirección de lectura del eje de claves y (ii) cuánto valor nuevo se escribe en el eje de valores. Gated DeltaNet-2 **desacopla** estos dos roles en una compuerta de borrado canalizada $\mathbf{b}_t$ (eje de claves) y una compuerta de escritura canalizada $\mathbf{w}_t$ (eje de valores), heredando además el decaimiento canalizado $\boldsymbol{\alpha}_t$ de KDA. Con estado $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$, decaimiento $\mathbf{D}_t = \mathrm{Diag}(\boldsymbol{\alpha}_t)$, dirección de borrado $\mathbf{e}_t = \mathbf{b}_t \odot \mathbf{k}_t$ y objetivo de escritura $\mathbf{z}_t = \mathbf{w}_t \odot \mathbf{v}_t$, la actualización de estado es:

$$\mathbf{S}_t = \left(\mathbf{I} - \mathbf{k}_t \mathbf{e}_t^\top\right) \mathbf{D}_t \, \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{z}_t^\top, \qquad \mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t$$

o, expandiendo los productos de las compuertas:

$$\mathbf{S}_t = \left(\mathbf{I} - \mathbf{k}_t (\mathbf{b}_t \odot \mathbf{k}_t)^\top\right) \mathbf{D}_t \, \mathbf{S}_{t-1} + \mathbf{k}_t (\mathbf{w}_t \odot \mathbf{v}_t)^\top$$

Las dos compuertas se producen mediante proyecciones lineales independientes seguidas de sigmoide:

$$\mathbf{b}_t = \sigma(\mathbf{W}_b \mathbf{x}_t), \qquad \mathbf{w}_t = \sigma(\mathbf{W}_w \mathbf{x}_t)$$

y el decaimiento canalizado sigue una parametrización de log-decaimiento (calculada en fp32 para evitar pérdida de precisión en productos acumulativos largos):

$$\mathbf{g}_t = \mathbf{a} - \text{softplus}(\boldsymbol{\delta}), \qquad \boldsymbol{\alpha}_t = \exp(\mathbf{g}_t)$$

Los roles estructurales de las dos compuertas son complementarios:

- $\mathbf{b}_t$ (eje de claves) hace que la **dirección de lectura** sea selectiva por canal: determina qué canales de clave se eliminan antes de escribir.
- $\mathbf{w}_t$ (eje de valores) hace que el **objetivo de escritura** sea selectivo por canal: determina qué canales de valor se depositan en memoria.
- $\boldsymbol{\alpha}_t$ (por canal de clave) proporciona decaimiento global, como en KDA.

Reducciones. Fijar $\mathbf{b}_t = \beta_t \mathbf{1}_{d_k}$ y $\mathbf{w}_t = \beta_t \mathbf{1}_{d_v}$ recupera KDA; colapsar además $\boldsymbol{\alpha}_t$ a un escalar recupera Gated DeltaNet. Desde la perspectiva de aprendizaje en línea/pesos rápidos, la actualización es el minimizador de:

$$\mathcal{L}_t(\mathbf{S}) = \tfrac{1}{2}\|\mathbf{S} - \mathbf{D}_t \mathbf{S}_{t-1}\|_F^2 - 2\left\langle \mathbf{S}^\top \mathbf{k}_t,\ \mathbf{z}_t - (\mathbf{D}_t \mathbf{S}_{t-1})^\top \mathbf{e}_t \right\rangle$$

Algorithm 42 Avance Recurrente de Gated DeltaNet-2 (Forma de Referencia)

Require: Entrada $\mathbf{x}_t$; estado previo $\mathbf{S}_{t-1} \in \mathbb{R}^{d_k \times d_v}$
Require: Proyecciones: W_q, W_k, W_v, W_b, W_w; parámetros de log-decaimiento $\mathbf{a}, \boldsymbol{\delta}$
Ensure: Salida $\mathbf{o}_t$ y estado $\mathbf{S}_t$

$\mathbf{q}_t \leftarrow \text{normalize}(W_q \mathbf{x}_t)$, $\mathbf{k}_t \leftarrow \text{normalize}(W_k \mathbf{x}_t)$, $\mathbf{v}_t \leftarrow \text{silu}(W_v \mathbf{x}_t)$
$\mathbf{b}_t \leftarrow \sigma(W_b \mathbf{x}_t)$ ▷ compuerta de borrado eje-clave, $\mathbb{R}^{d_k}$
$\mathbf{w}_t \leftarrow \sigma(W_w \mathbf{x}_t)$ ▷ compuerta de escritura eje-valor, $\mathbb{R}^{d_v}$
$\boldsymbol{\alpha}_t \leftarrow \exp(\mathbf{a} - \text{softplus}(\boldsymbol{\delta}))$ ▷ decaimiento canalizado, fp32
Dirección de borrado: $\mathbf{e}_t \leftarrow \mathbf{b}_t \odot \mathbf{k}_t$; objetivo de escritura: $\mathbf{z}_t \leftarrow \mathbf{w}_t \odot \mathbf{v}_t$
Decaer estado: $\mathbf{S}_t \leftarrow \text{Diag}(\boldsymbol{\alpha}_t)\, \mathbf{S}_{t-1}$
Lectura: $\mathbf{r}_t \leftarrow \mathbf{S}_t^\top \mathbf{e}_t$ ▷ suma sobre eje de claves
Actualización delta: $\mathbf{S}_t \leftarrow \mathbf{S}_t - \mathbf{k}_t \mathbf{r}_t^\top + \mathbf{k}_t \mathbf{z}_t^\top$
Recuperar: $\mathbf{o}_t \leftarrow \mathbf{S}_t^\top \mathbf{q}_t$
Compuerta de salida: $\mathbf{o}_t \leftarrow \text{GroupNorm}(\mathbf{o}_t) \odot \text{silu}(W_g \mathbf{x}_t)$
return $\mathbf{o}_t$, $\mathbf{S}_t$

El algoritmo de entrenamiento por fragmentos extiende la representación WY de KDA absorbiendo el decaimiento canalizado en una recurrencia delta asimétrica sobre un estado normalizado por decaimiento, permitiendo paralelismo rico en matmuls en tensor cores. Como $\mathbf{b}_t$ y $\mathbf{w}_t$ son operadores diagonales por canal que varían por fila, el atajo de post-escalado escalar de KDA se rompe: el paso hacia atrás requiere una **inversión WY consciente de las compuertas** que incorpora las compuertas en cada producto escalar que acumula el gradiente.

8.8.2. Rendimiento Empírico y Compromisos

A 1.3B parámetros entrenados con 100B tokens FineWeb-Edu, Gated DeltaNet-2 supera a Mamba-2, Mamba-3, Gated DeltaNet y KDA en modelado de lenguaje, razonamiento de sentido común y—más notablemente—recuperación de contexto largo del mundo real y RULER

(especialmente multi-key needle-in-a-haystack). Su rendimiento en H100 se mantiene casi plano (38,0 $\rightarrow$ 36,1 Kt/s) al crecer la longitud de secuencia de 2K a 16K, donde el de un Transformer cae bruscamente. Los estudios de ablación muestran que la compuerta de borrado $\mathbf{b}_t$ explica la mayor parte de la ganancia; la compuerta de escritura $\mathbf{w}_t$ aporta una mejora adicional menor. La variante híbrida (Gated DeltaNet-2 recurrente + atención de ventana deslizante) reduce aún más la brecha con los modelos de atención pura en tareas que requieren agregación de evidencia local.

Aspecto	**Fortaleza**	**Limitación**
Recuperación	La mejor entre modelos delta recurrentes en multi-key NIAH.	La variante recurrente aún supera al Transformer en NQ/DROP.
Rendimiento	Escalado casi plano 2K$\rightarrow$16K; pequeña sobrecarga vs KDA.	Requiere un nuevo kernel Triton WY consciente de las compuertas.
Memoria	Estado de inferencia constante $\mathcal{O}(d^2)$.	Rango de borrado de autovalores negativos $[0, 2]$ no da ganancia consistente a 1.3B.

Cuadro 14: Compromisos de Gated DeltaNet-2 a escala 1.3B / 100B tokens [48].

8.9. 8. Kimi Delta Attention (KDA): Compuerta de Decaimiento Canalizada para la Regla Delta

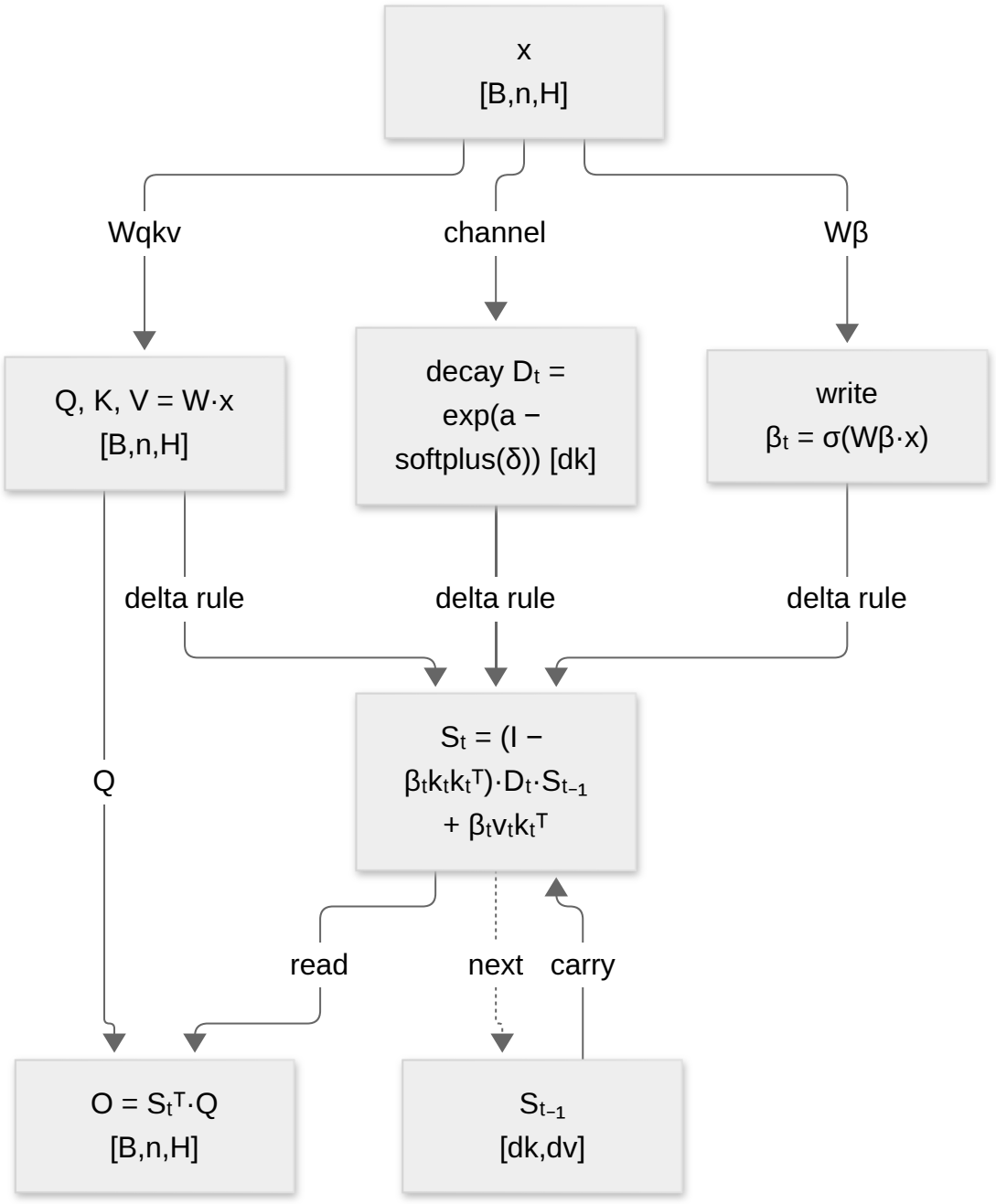


Figura 68: Flujo de datos de KDA: el decaimiento por canal y una compuerta escalar de escritura modulan la actualización del estado por regla delta.

8.9.1. Fundamento Matemático

Kimi Delta Attention [63] es el módulo central de Kimi Linear y extiende Gated DeltaNet con una **compuerta de decaimiento canalizada** de grano más fino. Mientras que Gated DeltaNet usa un único decaimiento escalar por cabeza α_t junto con la compuerta escalar de escritura delta β_t, KDA reemplaza el decaimiento escalar con un decaimiento **por canal de clave** $\boldsymbol{\alpha}_t \in \mathbb{R}^{d_k}$, parametrizado mediante una formulación de log-decaimiento y aplicado como

una matriz diagonal $\mathbf{D}_t = \mathrm{diag}(\boldsymbol{\alpha}_t)$ antes de la actualización de la regla delta. La regla delta en sí retiene una compuerta de escritura escalar por cabeza β_t:

$$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)\, \mathbf{D}_t\, \mathbf{S}_{t-1} + \beta_t \mathbf{v}_t \mathbf{k}_t^\top, \qquad \mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t$$

El decaimiento canalizado sigue una parametrización de log-decaimiento (calculada en fp32 por estabilidad numérica en productos acumulativos largos):

$$\mathbf{g} = \mathbf{a} - \mathrm{softplus}(\boldsymbol{\delta}), \qquad \boldsymbol{\alpha}_t = \exp(\mathbf{g})$$

donde $\mathbf{a}$ y $\boldsymbol{\delta}$ son parámetros aprendidos por canal. Esto da a cada canal de clave una escala temporal de olvido independiente, permitiendo al modelo retener algunos canales más tiempo que otros dentro de la misma cabeza.

Reducción a Gated DeltaNet : Cuando $\boldsymbol{\alpha}_t$ colapsa a un escalar (es decir, todos los canales comparten el mismo decaimiento), KDA se reduce exactamente a Gated DeltaNet. El decaimiento canalizado es por tanto una generalización estricta que añade selectividad por canal sobre el decaimiento/escritura escalares de Gated DeltaNet.

Entrenamiento por fragmentos : Un algoritmo por fragmentos ad hoc usa una variante **Diagonal-Plus-Low-Rank (DPLR)** especializada de la matriz de transición para mantener la eficiencia en hardware. El producto de las transiciones por paso $(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)\mathbf{D}_t$ admite una descomposición DPLR que permite paralelismo rico en matmuls en tensor cores preservando la estructura de decaimiento canalizado.

Algorithm 43 Avance Recurrente de Kimi Delta Attention (Forma de Referencia)

Require: Entrada $\mathbf{x}_t$; estado previo $\mathbf{S}_{t-1} \in \mathbb{R}^{d_k \times d_v}$
Require: Proyecciones: W_q, W_k, W_v, W_β; parámetros de log-decaimiento $\mathbf{a}, \boldsymbol{\delta}$
Ensure: Salida $\mathbf{o}_t$ y estado $\mathbf{S}_t$

$\mathbf{q}_t \leftarrow W_q \mathbf{x}_t$, $\mathbf{k}_t \leftarrow W_k \mathbf{x}_t$, $\mathbf{v}_t \leftarrow W_v \mathbf{x}_t$
$\beta_t \leftarrow \mathrm{sigmoid}(W_\beta \mathbf{x}_t)$ ▷ compuerta de escritura escalar por cabeza
$\boldsymbol{\alpha}_t \leftarrow \exp(\mathbf{a} - \mathrm{softplus}(\boldsymbol{\delta}))$ ▷ decaimiento canalizado, fp32
$\mathbf{D}_t \leftarrow \mathrm{diag}(\boldsymbol{\alpha}_t)$ ▷ matriz diagonal de decaimiento
Decaer estado: $\mathbf{S}_t \leftarrow \mathbf{D}_t \mathbf{S}_{t-1}$
Lectura: $\mathbf{r}_t \leftarrow \mathbf{S}_t^\top \mathbf{k}_t$ ▷ predicción actual a lo largo de la clave
Actualización delta: $\mathbf{S}_t \leftarrow \mathbf{S}_t - \beta_t \mathbf{k}_t \mathbf{r}_t^\top + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$
Recuperar: $\mathbf{o}_t \leftarrow \mathbf{S}_t^\top \mathbf{q}_t$
Compuerta de salida: $\mathbf{o}_t \leftarrow \mathrm{GroupNorm}(\mathbf{o}_t) \odot \mathrm{silu}(W_g \mathbf{x}_t)$
return $\mathbf{o}_t$, $\mathbf{S}_t$

8.9.2. Rendimiento Empírico y Compromisos

KDA está desplegado en Kimi Linear, un modelo híbrido (3B activados / 48B parámetros totales) que combina capas KDA con Multi-head Latent Attention (MLA). Kimi Linear supera a un baseline de MLA completa en todas las tareas evaluadas, reduce el caché KV hasta un 75 %, y logra hasta 6× de throughput de decodificación para contexto de 1M de tokens. El decaimiento canalizado es el principal impulsor de la ganancia: permite control fino sobre qué canales de clave persisten, mejorando el recuerdo de contexto largo frente a Gated DeltaNet con decaimiento escalar.

Aspecto	Fortaleza	Limitación
Recuperación	El decaimiento canalizado mejora el recuerdo de contexto largo frente a baselines de decaimiento escalar; desplegado en Kimi Linear (híbrido 3B/48B).	El estado recurrente matricial $\mathcal{O}(d^2)$ es más pesado que la atención lineal con compuerta diagonal.
Rendimiento	Hasta 6× de throughput de decodificación a 1M de contexto; 75 % de reducción de caché KV frente a MLA completa.	Requiere kernels por fragmentos conscientes de DPLR para eficiencia en hardware.
Memoria	Estado de inferencia constante $\mathcal{O}(d^2)$ (recurrente matricial).	El cálculo log-decaimiento en fp32 añade sobrecarga; la reducción a Gated DeltaNet pierde selectividad por canal.

Cuadro 15: Compromisos de KDA en Kimi Linear (3B activados / 48B total) [63].

8.10. Principio Unificado de Compuerta

Las ocho arquitecturas comparten un principio común: **la compuerta es un mecanismo para controlar el flujo de información y la retención selectiva de memoria**. Las decisiones de diseño específicas divergen a lo largo de varias dimensiones:

1. **Ubicación de la compuerta**: Actualizaciones de estado recurrente (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2) versus espacio de logits/salida softmax (FoX, Gated Softmax, RetNet).

2. **Granularidad de control**: Por dimensión (GLA, HGRN2 diagonal), por clave (DeltaNet), por token (FoX), por cabeza (Gated Softmax) o borrado/escritura canalizados decouplados (Gated DeltaNet-2).

3. **Dependencia de datos**: Completamente dependiente de datos (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, FoX, Gated Softmax) versus esquemas fijos (RetNet con decaimiento exponencial).

4. **Compensación de expresividad**: Eficiencia recurrente lineal (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, RetNet) versus expresividad completa de softmax (FoX, Gated Softmax).

8.11. Implementación y Orientación Práctica

8.11.1. Cuándo Usar Cada Arquitectura

1. **GLA**: Inferencia rápida con contextos moderados a largos (2K–20K tokens), cuando los kernels CUDA personalizados son aceptables y el rendimiento de recuperación no es crítico.

2. **DeltaNet**: Fuerte recuperación asociativa y tareas de aprendizaje en contexto; bueno para tareas sintéticas (MQAR) y pruebas de asociación clave-valor.

3. **Gated DeltaNet**: Modelos de producción que requieren el mejor equilibrio de recuperación, olvido y rendimiento (probado en Qwen3-Next; ICLR 2025).

4. **Gated DeltaNet-2**: Cargas de recuperación de contexto largo (multi-key NIAH) donde el borrado/escritura canalizado decouplado da el recuerdo más fuerte; cuando se dispone de un kernel WY consciente de las compuertas.

5. **RetNet**: Inferencia extremadamente eficiente sin almacenamiento en caché KV; adecuado para despliegues en dispositivos móviles/de borde o modelos de juguete.

6. **HGRN2**: Jerarquías temporales multi-escala; cuando los límites de olvido específicos por capa son deseables.

7. **FoX**: Extrapolación de longitud y comprensión de contexto largo; cuando el entrenamiento cuadrático es aceptable y no se desea reemplazar softmax.

8. **Gated Softmax**: Mejora rápida para modelos softmax existentes con cambios mínimos de código; mejor para profesionales que desean ganancias inmediatas sin una reestructuración importante.

8.11.2. Consideraciones de Hardware

Objetivo de Hardware	**Arquitecturas Recomendadas**	**Notas**
GPU (H100/A100)	Gated Softmax, FoX, Gated DeltaNet, Gated DeltaNet-2	Implementaciones maduras de softmax/lineal. GLA/DeltaNet requieren kernels personalizados.
TPU (alta memoria)	GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2	Hardware soporta matmuls eficientes; los métodos recurrentes lineales brillan.
Móvil/Borde	RetNet, Gated Softmax ligero	Inferencia con memoria constante es crítica.

9. Normalización, Cuantización, Enrutamiento de Profundidad y Algoritmos de Ejecución

Este anexo recopila el detalle a nivel de implementación para las variantes de normalización, cuantización ternaria estilo BitNet, enrutamiento adaptativo de profundidad (Mixture-of-Depths), memoria condicional (Engram), embeddings factorizados y los algoritmos de ejecución (controles de estabilidad del paso de entrenamiento, enrutador de inferencia SBERT) que soportan la tubería impulsada por configuración. (Las conexiones residuales restringidas por variedad de mHC se documentan por separado en el Anexo 9.) Las secciones principales difieren a este anexo para las formulaciones matemáticas y el pseudocódigo; las claves bibliográficas citadas aquí se resuelven contra `docs/bibliography/`.

9.1. Variantes de Normalización: LayerNorm, RMSNorm, pRMSNorm, Tanh Dinámico, Erf Dinámico y FlashNorm

La normalización determina cómo se controla la escala de activación a través de la profundidad. En este repositorio, la normalización no es solo una elección de modelado sino también una cuestión de compatibilidad con el esquema, porque solo ciertos valores son actualmente aceptados por `norm_type`. El contrato del esquema es:

$$\texttt{norm_type} \in \{\texttt{layer_norm}, \texttt{dynamic_tanh}, \texttt{derf}, \texttt{rms_norm}, \texttt{prms_norm}, \texttt{flash_norm}\}$$

Las formulaciones más relevantes para este código base son:

9.1.1. LayerNorm (Línea Base)

LayerNorm [7] es el normalizador por defecto; centra y reescala cada token por su media y varianza por token:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \quad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2, \quad y_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i.$$

9.1.2. RMSNorm

RMSNorm elimina la centralización de la media y solo reescala por la magnitud de la raíz cuadrada media [129]:

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}, \qquad y_i = \gamma_i \frac{x_i}{\mathrm{RMS}(x)}.$$

En comparación con LayerNorm, RMSNorm es computacionalmente más simple (sin resta de la media de características) y se usa a menudo cuando es importante reducir la sobrecarga de normalización (p. ej., Llama, GPT-NeoX). Tiene una escala γ aprendible por dimensión pero sin término de bias.

9.1.3. RMSNorm Parcial (pRMSNorm)

pRMSNorm [129] reduce aún más la sobrecarga estimando el RMS de solo el primer $p\,\%$ de las dimensiones ocultas:

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{k}\sum_{i=1}^{k} x_i^2 + \epsilon}, \qquad k = \lceil d \cdot p \rceil,$$

explotando el supuesto de que las neuronas dentro de una capa son aproximadamente i.i.d. La misma escala γ se aplica a todas las dimensiones. La razón parcial p se controla mediante el campo de configuración `prms_partial_ratio` (por defecto 6,25 %). El paper reporta que se pueden lograr resultados competitivos con tan solo el 6,25 % de las entradas, aunque puede surgir inestabilidad de gradiente con razones muy pequeñas.

9.1.4. Tanh Dinámico (DyT)

El Tanh Dinámico propone reemplazar la normalización explícita con un mapeo elemento a elemento acotado [135]:

$$\mathrm{DyT}(x) = \tanh(\alpha x)$$

donde α se aprende. La idea central es que la contracción no lineal acotada puede proporcionar un escalado de señal estable sin calcular explícitamente estadísticas de normalización por token.

9.1.5. Erf Dinámico (Derf)

Derf extiende la misma dirección libre de normalización usando un mapeo basado en la función error [17]:

$$\mathrm{Derf}(x) = \mathrm{erf}(\alpha x + s)$$

con escala/desplazamiento aprendibles. Los resultados reportados en el trabajo citado indican un rendimiento superior al de DyT y las líneas base de normalización comunes en múltiples dominios.

9.1.6. FlashNorm (RMSNorm sin pesos)

FlashNorm [40] no es una nueva normalización matemática; es una *reescritura algebraica exacta* del ubicuo par "RMSNorm → Lineal" que (i) elimina la escala aprendible por dimensión $\mathbf{g}$ al plegarla en los pesos de la capa lineal subsecuente, y (ii) difiere la reducción escalar RMS a la salida de la multiplicación matricial, de modo que el `matmul` (unidad matricial) y la reducción RMS (unidad vectorial) se ejecuten en paralelo. Tres proposiciones formalizan la reescritura.

Proposición 1 (Plegado de pesos). Dada la activación $\mathbf{a} \in \mathbb{R}^n$, el peso RMSNorm $\mathbf{g} \in \mathbb{R}^n$ y una lineal subsecuente $\mathbf{W} \in \mathbb{R}^{n \times k}$,

$$\mathbf{z} = \mathrm{RMSNorm}(\mathbf{a})\mathbf{W} = \frac{\mathbf{a}}{\mathrm{RMS}(\mathbf{a})}\mathbf{W}^*, \qquad W^*_{i,j} = g_i \cdot W_{i,j} \;\Leftrightarrow\; \mathbf{W}^* = \mathrm{diag}(\mathbf{g})\mathbf{W}.$$

La escala por dimensión $\mathbf{g}$ se vuelve redundante y se pliega en $\mathbf{W}$ una única vez al cargar el modelo. El módulo autónomo `FlashNorm` de este repositorio aplica esto directamente: *no tiene parámetros aprendibles* (la escala está destinada a ser absorbida por la proyección subsecuente).

Proposición 2 (Normalización diferida). Para una lineal sin bias $\mathbf{W}^*$, la conmutatividad escalar–matricial produce

$$\frac{\mathbf{a}}{\mathrm{RMS}(\mathbf{a})}\mathbf{W}^* = (\mathbf{a}\mathbf{W}^*) \cdot \frac{1}{\mathrm{RMS}(\mathbf{a})}.$$

El `matmul` $\mathbf{a}\mathbf{W}^*$ y la reducción RMS son independientes entre sí; en hardware con unidades matriciales y vectoriales diferenciadas se ejecutan en paralelo, y solo una multiplicación vectorial–escalar los sincroniza. El módulo fusionado `FlashNormLinear` implementa esta reescritura: en la ruta sin bias calcula el `matmul` sobre la entrada *no normalizada* y multiplica el resultado por el RMS recíproco por token. Cuando la lineal tiene bias $\mathbf{c}$, la reescritura deja de ser exacta (Observación 1 del paper): $(\mathbf{a}\mathbf{W}^* + \mathbf{c})\,/\mathrm{RMS}(\mathbf{a}) \neq \mathbf{a}\mathbf{W}^*/\mathrm{RMS}(\mathbf{a}) + \mathbf{c}$; la capa entonces aplica el escalar a la entrada primero y deja que la lineal añada su bias como de costumbre.

Proposición 3 (Cancelación de RMS). Por la invariancia de escala de RMS, $\mathrm{RMS}(\alpha\mathbf{a}) = \alpha\,\mathrm{RMS}(\mathbf{a})$. Por tanto, un patrón "RMSNorm → Lineal → RMSNorm" permite que el *primer* RMSNorm se elimine por completo:

$$\mathrm{RMSNorm}\left(\frac{\mathbf{a}}{\mathrm{RMS}(\mathbf{a})}\mathbf{W}^*\right) = \mathrm{RMSNorm}(\mathbf{a}\mathbf{W}^*)\,.$$

Esta cancelación es relevante para la normalización QKV (Gemma 4) y la normalización latente MLA (DeepSeek-V2, Mistral Small 4); no se aplica automáticamente en este repositorio pero se documenta por completitud.

Composición con RMS parcial. FlashNorm se compone naturalmente con el truco pRMSNorm (§9, subsección pRMSNorm): cuando `flashnorm_partial_ratio` $= p > 0$, el RMS se estima solo de los primeros $k = \lceil d \cdot p \rceil$ elementos de la entrada, reduciendo aún más el costo de la reducción RMS en la unidad vectorial.

Interacción con BitNet. Plegar un $\mathbf{g}$ de coma flotante en el tensor de pesos ternarios de BitNet $\{-1, 0, +1\}$ [115] destruiría la cuantización, y BitLinear aplica su propio LayerNorm interno antes de la cuantización de activaciones, el cual necesita la entrada normalizada. El módulo fusionado `FlashNormBitLinear` por tanto recurre a la composición secuencial FlashNorm → BitLinear: el `matmul` y la reducción RMS aún se ejecutan en paralelo, pero la multiplicación escalar post-`matmul` de la Prop. 2 se sacrifica. El plegado de $\mathbf{g}$ en coma flotante en un tensor de pesos de 1.58 bits se deja a trabajo futuro a nivel kernel.

Pseudocódigo PyTorch.

Entrada: activación $\mathbf{a} \in \mathbb{R}^{T \times n}$, peso RMSNorm $\mathbf{g} \in \mathbb{R}^n$, lineal $\mathbf{W} \in \mathbb{R}^{n \times k}$, bias $\mathbf{c} \in \mathbb{R}^k$ o `None`
`// Prop. 1: plegado de pesos al cargar`
$\mathbf{W}^* \leftarrow \mathbf{g} \odot \mathbf{W}$ ▷ por columnas; `g.unsqueeze(0) * W` en PyTorch
$\mathbf{g} \leftarrow \mathbf{1}$ ▷ la norma autónoma ahora es sin pesos
`// Prop. 2: paso forward con RMS diferida (ruta sin bias)`
$\mathbf{b} \leftarrow \mathbf{a}\mathbf{W}^{*\top}$ ▷ unidad matricial / tensor cores
rms_inv $\leftarrow$ rsqrt(mean($\mathbf{a}^2$) + ε) ▷ unidad vectorial, independiente de $\mathbf{b}$
if $\mathbf{c} =$ `None` **then**
    retornar $\mathbf{b} \cdot$ rms_inv ▷ una sola multiplicación vectorial–escalar
else
    `// Obs. 1: ruta con bias secuencial; matmul/RMS paralelizan`
    retornar $(\mathbf{a} \cdot \text{rms_inv})\mathbf{W}^{*\top} + \mathbf{c}$
end if

Latencia reportada (GPU NVIDIA T4). El paper reporta las siguientes aceleraciones sobre la operación *norm-then-project*, con un despacho adaptativo que cae al camino secuencial en el régión limitada por memoria (decodificación):

Escala	Tokens	Secuencial (ms)	FlashNorm (ms)	Aceleración	Notas
SmolLM2-135M	4096	0.704	0.468	+33,6 %	limitada por cómputo (prefill)
SmolLM2-135M	8192	0.929	0.599	+35,5 %	limitada por cómputo (prefill)
Llama-7B	1024	1.882	1.654	+12,1 %	limitada por cómputo (prefill)
Llama-7B	4096	7.628	6.570	+13,9 %	limitada por cómputo (prefill)

Validación de plegado sin pérdida. El plegado de pesos (Prop. 1) es matemáticamente exacto; el paper verificó salidas bit-idénticas en tres checkpoints abiertos:

Modelo	Precisión	Resultado
SmolLM2-135M	fp16	diff máxima = 0,0, similitud coseno = 1,0, texto idéntico
Llama-3.2-1B	fp32	generación voraz bit-idéntica
Llama-3.1-8B	fp32	generación voraz bit-idéntica

La preservación de calidad en `lm-evaluation-harness` (wikitext word_ppl, MMLU 5-shot, HellaSwag 0-shot) se mantiene dentro del ruido del benchmark (peor $\Delta = +0{,}0017$ word_ppl).

9.1.7. Comparación

Método	Fórmula	Estadísticas Necesarias	Notas
LayerNorm	$\gamma_i(x_i - \mu)/\sqrt{\sigma^2 + \epsilon} + \beta_i$	media + varianza	Por defecto; centrado y reescalado completo [7].
RMSNorm	$\gamma_i x_i / \sqrt{\frac{1}{d}\sum_j x_j^2 + \epsilon}$	Solo RMS	Menor sobrecarga; línea base ampliamente usada [129].
pRMSNorm	$\gamma_i x_i / \sqrt{\frac{1}{k}\sum_{j \le k} x_j^2 + \epsilon}$	RMS parcial	RMS del primer p % de dims; `prms_partial_ratio` por defecto 6,25 % [129].
Tanh Dinámico	$\tanh(\alpha x)$	ninguna	Transformación acotada libre de normalización; reemplazo directo [135].
Erf Dinámico	$\text{erf}(\alpha x + s)$	ninguna	Alternativa libre de normalización; mejora sobre DyT [17].

Método	Fórmula	Estadísticas Necesarias	Notas
FlashNorm	$x_i/\sqrt{\frac{1}{d}\sum_j x_j^2+\epsilon}$ (sin pesos)	Solo RMS	Reescritura exacta de RMSNorm→Lineal: pliega $\mathbf{g}$ en $\mathbf{W}$ (Prop. 1), difiere el escalar RMS a la salida del `matmul` (Prop. 2); 12–35 % de reducción de latencia en GPU T4 [40].

9.2. Normalización Query–Key y Control de Logits de Atención

Los logits de atención $\ell_{i,t,h} = \mathbf{q}_{i,h}^\top \mathbf{k}_{t,g(h)}/\sqrt{d_h}$ son la entrada del softmax. Su crecimiento descontrolado es una fuente clásica de inestabilidad de entrenamiento: saturación de la atención, colapso de entropía, picos de pérdida y, a escala, divergencia [53, 62]. Este repositorio expone cinco mecanismos bajo `norm.qk_norm.method`; tres actúan en el forward y dos sobre el estado del optimizador. Son mutuamente excluyentes por configuración.

Normalización QK en el forward. La normalización QK [53] normaliza queries y keys a lo largo de la dimensión de características por cabeza antes del producto punto. En forma RMSNorm,

$$\hat{\mathbf{q}}_{i,h} = \frac{\mathbf{q}'_{i,h} \odot \boldsymbol{\gamma}_q}{S_q(\mathbf{q}'_{i,h})}, \qquad S_q(\mathbf{q}) = \sqrt{\frac{1}{d_h}\sum_{j=1}^{d_h} q_j^2 + \epsilon},$$

e idénticamente para $\hat{\mathbf{k}}$ con $\boldsymbol{\gamma}_k$. Los pesos afines se comparten entre cabezas y se difunden por el eje de cabezas, por lo que el coste paramétrico es $2d_h$ (`rms_norm`) o $2 \cdot 2d_h$ con sesgo (`layer_norm`); `l2_norm` no tiene pesos. Dado que las normas normalizadas cumplen $\|\hat{\mathbf{q}}\|_2 \approx \|\boldsymbol{\gamma}_q\|_2$ salvo dirección y ϵ, el crecimiento residual de los logits queda mediado por el producto afín aprendido $\boldsymbol{\gamma}_q \odot \boldsymbol{\gamma}_k$ en lugar de por normas crudas de query/key sin control.

Normalización QK con Multi-Head Latent Attention. MLA cachea un latente de bajo rango $\mathbf{c}_t$ y reconstruye las keys por up-proyección, por lo que la QK RMSNorm post-proyección parecería requerir las keys completas que MLA evita cachear. Han et al. [47] muestran que la incompatibilidad es un artefacto de implementación: RMSNorm se descompone en un peso afín estático y una estadística escalar dinámica,

$$\hat{\mathbf{k}}^c_{t,g} = \frac{(\mathbf{c}_t \mathbf{W}^K_g) \odot \boldsymbol{\gamma}^c_k}{S^c_k(\mathbf{c}_t \mathbf{W}^K_g)}, \qquad \ell^c_{i,t,h} = \big((\mathbf{q}'^c_{i,h} \odot \boldsymbol{\gamma}^c_q \odot \boldsymbol{\gamma}^c_k)(\mathbf{W}^K_{g(h)})^\top\big) \cdot \mathbf{c}_t \; \alpha^q_{i,h}\alpha^k_{t,g(h)},$$

donde $\alpha^q = 1/S_q$ y $\alpha^k = 1/S_k$ son escalares. El peso estático se absorbe en la proyección del lado de la query y solo se cachea un escalar de RMS inverso por token y grupo KV ($T \times G$ escalares frente a $T \times G \times d_c$ keys de contenido completas), preservando la ruta latente de decodificación; el paper reporta menor pérdida que QK-Clip y $< 2\,\%$ de sobrecarga de decodificación hasta 256k tokens. En este repositorio la variante MLA de Mehta et al. [80] ya re-expande las keys desde el latente cacheado en cada paso, de modo que la normalización explícita por cabeza de $\mathbf{q}$ y $\mathbf{k}$ descomprimidos es *exactamente* equivalente a la formulación absorbida tanto en entrenamiento como en decodificación con caché; no se necesita un caché escalar. Los bloques de contenido y RoPE se normalizan por bloques, siguiendo la frontera natural entre la ruta latente de contenido y la ruta rotatoria materializada.

QK-Clip. MuonClip de Kimi K2 [62] ataca el mismo modo de fallo después del producto punto. Para cada cabeza h define el logit pre-softmax máximo por cabeza sobre el lote,

$$S^h_{\text{máx}} = \frac{1}{\sqrt{d}} \max_{\mathbf{X}\in\mathcal{B}} \max_{i,j} \mathbf{Q}^h_i \mathbf{K}^{h\top}_j,$$

y, tras cada paso del optimizador, reescala los pesos de proyección con

$$\gamma_h = \min\left(1, \frac{\tau}{S^h_{\max}}\right), \qquad \mathbf{W}^h_q \leftarrow \gamma_h^{\alpha}\mathbf{W}^h_q, \qquad \mathbf{W}^h_k \leftarrow \gamma_h^{1-\alpha}\mathbf{W}^h_k.$$

La intervención nunca altera el forward/backward del paso actual; las cabezas ya por debajo de τ reciben $\gamma_h = 1$, por lo que el mecanismo se autodesactiva cuando el entrenamiento se estabiliza (K2 usó $\tau = 100$ y observó todas las cabezas por debajo del umbral tras aproximadamente el 30 % de los pasos). Se aplica por cabeza a los bloques de filas de `q_proj` y `k_proj`; para atención latente el lado de la key es `uk_proj`, y para grouped-query attention cada cabeza KV recibe el menor γ_h de las cabezas de query que la leen, lo que acota todos los logits del grupo. `clip_alpha` vale 0,5 por defecto (es decir, $\sqrt{\gamma_h}$ en cada lado, la elección balanceada de K2).

QuacK. En lugar de una intervención en el forward o en los pesos, QuacK [5] asigna learning rates dependientes del parámetro a los pesos de query y key. Guardando las normas de Frobenius en la inicialización, antes de cada paso fija

$$\eta_{W_Q} = \tau\,\eta\,\frac{\|\mathbf{W}_K\|_{\text{init}}}{\|\mathbf{W}_K\|}, \qquad \eta_{W_K} = \tau\,\eta\,\frac{\|\mathbf{W}_Q\|_{\text{init}}}{\|\mathbf{W}_Q\|},$$

lo que se sigue de controlar el cambio de primer orden $\Delta\ell \approx (\mathbf{q}\,\Delta\mathbf{k}^\top + \Delta\mathbf{q}\,\mathbf{k}^\top)/\sqrt{d}$: cuando un lado crece, el learning rate del otro decrece. Las normas de Frobenius y espectral se comportaron de forma indistinguible en el paper, por lo que se usa Frobenius. La tasa relativa τ vale 1,0 por defecto; el paper reporta una estabilidad comparable a QK-norm con learning rates base altos y alrededor de un 10 % de aceleración porque desaparecen dos reducciones RMS por bloque. Aquí el mecanismo es independiente del optimizador: el trainer separa los pesos de query/key en grupos de parámetros dedicados, de modo que cualquiera de los optimizadores registrados puede soportarlo.

Contrato de esquema y validación. `norm.qk_norm` es un sub-objeto con `enabled`, `method` $\in$ {`rms_norm`, `layer_norm`, `l2_norm`, `qk_clip`, `quack`}, `clip_tau`, `clip_alpha`, `quack_tau` y `eps`. El mecanismo requiere proyecciones explícitas de query/key por cabeza: activarlo para un patrón de capas que contenga un mixer recurrente o de espacio de estados se rechaza al cargar (`QK_NORM_MIXERS`); `qk_clip` requiere además logits de producto punto por cabeza y por tanto excluye el router de mezcla Gaussiana (`QK_CLIP_MIXERS`).

Pseudocódigo PyTorch (QK-Clip).

Entrada: modelo, umbral τ, exponente α, $S^h_{\max}$ por cabeza cacheado
for cada cabeza de atención h con $S^h_{\max} > \tau$ **do**
    $\gamma_h \leftarrow \tau / S^h_{\max}$
    $\mathbf{W}^h_q \leftarrow \gamma_h^{\alpha}\mathbf{W}^h_q$ ▷ bloque de filas $[hd_h : (h+1)d_h]$
    $\mathbf{W}^h_k \leftarrow \gamma_h^{1-\alpha}\mathbf{W}^h_k$ ▷ `k_proj` o `uk_proj`; mínimo sobre el grupo GQA
end for
limpiar $S^h_{\max}$ ▷ evitar reaplicar una estadística obsoleta

Comparación.

Método	Intervención	Estado extra	Notas
`rms_norm`	RMSNorm por cabeza en el forward	$2d_h$	QK-norm estándar; exacta con el MLA re-expansivo de este repo [53, 47].
`layer_norm`	LayerNorm por cabeza en el forward	$4d_h$	Variante con centrado de media (SAN); algo más costosa.

Método	Intervención	Estado extra	Notas
`l2_norm`	L2 por cabeza en el forward	ninguno	Ablación sin pesos.
`qk_clip`	reescalado de pesos post-paso	$S_{\text{máx}}$ por cabeza	Se autodesactiva; necesita logits por cabeza [62].
`quack`	learning rates de query/key por paso	normas iniciales	Independiente del optimizador; ∼10 % más barato que QK-norm [5].

9.3. Ruta de Cuantización Ternaria BitNet

Cuando `use_bitnet=true`, las capas BitLinear reemplazan a `nn.Linear` estándar con cuantización ternaria de pesos $(-1, 0, +1)$ siguiendo BitNet [115]. Dado el tensor de pesos W, un escalado práctico es:

$$s = \text{mean}(|W|), \qquad \tilde{W} = \text{clip}\left(\text{round}\left(\frac{W}{s}\right), -1, 1\right).$$

El mapeo empaquetado utiliza dos bits por símbolo de peso para eficiencia de almacenamiento. Esto permite una compresión significativa ($\sim 32\times$ frente a FP32) pero es experimental y puede afectar la calidad del modelo; es más adecuado para el despliegue de inferencia.

La cuantización de activaciones para la ruta INT8 es:

$$q = \text{round}\left(x \cdot \frac{127}{\text{máx}(|x|) + \epsilon}\right), \qquad q \in [-128, 127]$$

con des-cuantización $x \approx q/\alpha$. Para N parámetros, las estimaciones de tamaño antes de metadatos y sobrecarga de empaquetado son:

$$\text{Tamaño FP32} \approx 4N, \quad \text{Tamaño FP16} \approx 2N, \quad \text{Tamaño 1.58-bit} \approx \frac{1{,}58}{8}N$$

lo que se alinea con objetivos de despliegue ligero [100, 16].

9.4. Enrutamiento de Tokens Mixture-of-Depths (MoD)

Cuando `use_mixture_of_depths=true`, cada bloque del transformer puntúa los tokens con un enrutador ligero. Solo el subconjunto superior de `mixture_of_depths_capacity_ratio` se actualiza; los tokens omitidos pasan sin cambios [136]. Esto asigna más profundidad a los tokens destacados, reduciendo el cómputo por capa.

1: **Entrada:** estados ocultos $H \in \mathbb{R}^{n \times d}$, enrutador R, ratio de capacidad ρ
2: puntajes $\leftarrow R(H)$ ▷ puntaje del enrutador por token
3: $\mathcal{S} \leftarrow$ `top-k`$(\text{puntajes}, k = \lceil \rho \cdot n \rceil)$ ▷ seleccionar subconjunto destacado
4: $H_{\text{actualizar}} \leftarrow$ `bloque`$(H[\mathcal{S}])$ ▷ aplicar bloque transformer a tokens seleccionados
5: $H[\mathcal{S}] \leftarrow H_{\text{actualizar}}$ ▷ escribir de vuelta; los tokens no seleccionados bypassan el bloque
6: **Salida:** H ▷ pérdida auxiliar de balance de carga añadida desde `mixture_of_depths_router_aux_loss_weight`

9.4.1. Configuración

- `use_mixture_of_depths`: bool — habilitar enrutamiento MoD de tokens.
- `mixture_of_depths_capacity_ratio`: float en $(0, 1]$ — fracción de tokens actualizados por capa.
- `mixture_of_depths_router_aux_loss_weight`: float ≥ 0 — peso de pérdida auxiliar para regularización del enrutador.

9.5. Memoria Condicional Engram

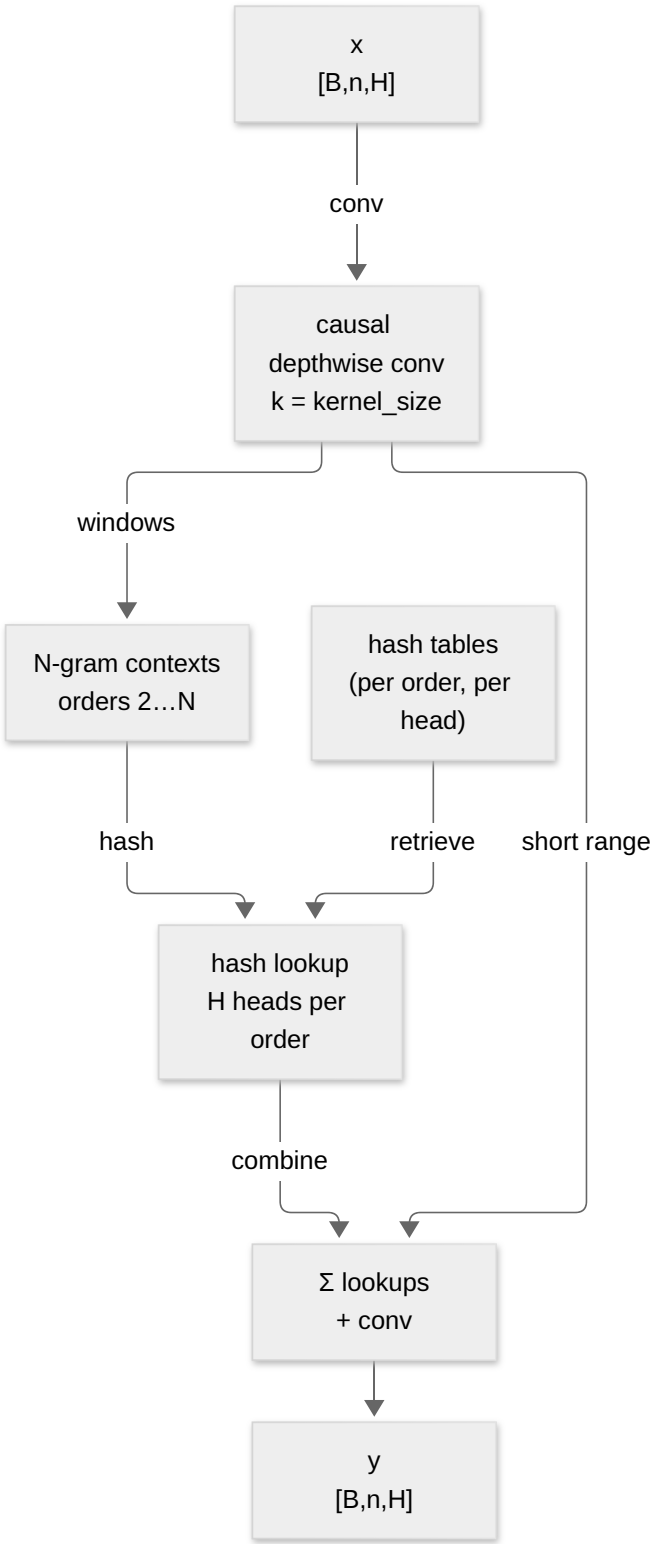


Figura 69: Flujo de datos de la memoria condicional de Engram: una convolución causal depthwise aporta contexto de corto alcance, las búsquedas hash de N-gramas recuperan embeddings por orden, y las búsquedas se suman a la salida de la convolución.

Engram (`engram_attn`) implementa memoria condicional mediante búsqueda N-gram escalable [19]. Construye tablas de búsqueda basadas en hash para contextos N-gram desde tamaño 2 hasta `engram_max_ngram_size`, con cabezas hash independientes por orden N-gram. Una convolución causal depthwise (`engram_kernel_size`) proporciona contexto de corto alcance antes de la búsqueda N-gram.

Entrada: flujo de tokens $x_{1:n}$, órdenes N-gram $\{2, \ldots, N\}$, cabezas hash por orden H
$h_{\text{conv}} \leftarrow \text{DepthwiseConv1d}(x_{1:n}, k = \texttt{engram_kernel_size})$ ▷ contexto de corto alcance
for cada orden N-gram $k \in \{2, \ldots, N\}$ **do**
    $\text{ctx}_t^{(k)} \leftarrow x_{t-k+1:t}$ ▷ ventana de contexto N-gram
    $\text{lookup}_t^{(k)} \leftarrow \text{HashTable}^{(k)}[\text{hash}(\text{ctx}_t^{(k)})]$ ▷ recuperación por cabeza, H cabezas por orden
end for
$y_t \leftarrow \sum_k \text{lookup}_t^{(k)} + h_{\text{conv},t}$ ▷ combinar memorias condicionales
Salida: $y_{1:n}$

9.5.1. Configuración

- `engram_max_ngram_size`: int ≥ 2 — orden N-gram más alto (por defecto 3).
- `engram_n_heads_per_ngram`: int ≥ 1 — cabezas hash por orden N-gram (por defecto 4).
- `engram_embed_dim_per_head`: int ≥ 1 — dimensión de embedding por cabeza hash (por defecto 32).
- `engram_kernel_size`: int ≥ 1 — ancho del kernel de conv causal depthwise (por defecto 4).
- `engram_seed`: int — semilla hash para reproducibilidad (por defecto 42).

Tamaño oculto total de Engram = (`max_ngram_size` − 1) × `n_heads_per_ngram` × `embed_dim_per_head`.

9.6. Embeddings Factorizados y Convolución de Embedding

9.6.1. Embeddings Factorizados

Cuando `use_factorized_embedding=true`, la matriz de embedding se factoriza en dos matrices más pequeñas, reduciendo parámetros de $O(V \times H)$ a $O(V \times R + R \times H)$ donde $R =$ `factorized_embedding_dim`.

- `use_factorized_embedding`: bool — habilitar factorización.
- `factorized_embedding_dim`: int ≥ 1 — dimensión intermedia (típico: 64–256).

9.6.2. Convolución de Embedding

Cuando `use_embedding_conv=true`, se aplica una Conv1d depthwise sobre el flujo de embeddings antes del primer bloque transformer, con tamaño de kernel `embedding_conv_kernel`. Esto proporciona un filtro ligero de contexto local sobre embeddings de tokens.

9.7. Controles de Estabilidad del Paso de Entrenamiento

El paso de entrenamiento guiado por esquema aplica acumulación de gradientes, recorte de norma global, guardas de explosión post-recorte y reintentos acotados de NaN/Inf. El gradiente acumulado es:

$$g_{\text{acc}} = \frac{1}{K} \sum_{i=1}^{K} g_i, \quad K = \texttt{gradient_accumulation_steps}$$

Algorithm 44 Paso de Entrenamiento Guiado por Esquema con Controles de Estabilidad

Require: Flujo de lotes, configuración C

Inicializar contador de reintentos $r \leftarrow 0$
for cada paso del optimizador **do**
  Acumular gradientes para $K = C$.`gradient_accumulation_steps` micro-lotes
  Aplicar recorte de norma global con $\tau = C$.`grad_clip_max_norm`
  if el gradiente post-recorte excede C.`inf_post_clip_threshold` o se detecta NaN/Inf **then**
    if $r < C$.`max_nan_retries` **then**
      restaurar estado seguro / saltar paso; $r \leftarrow r + 1$
      continue
    else
      detener entrenamiento con estado de fallo
    end if
  end if
  ejecutar paso del optimizador seleccionado por `optimizer_class`
  actualizar scheduler (`cosine`, `constant`, o `linear_warmup_then_constant`)
  if paso mod `checkpoint_every_n_steps`$= 0$ **then**
    guardar checkpoint rodante y podar a `max_rolling_checkpoints`
  end if
  actualizar mejores checkpoints hasta `num_best_checkpoints`
  emitir CSV + telemetría según `gradient_log_interval` y `telemetry_log_interval`
end for

$$g_{\text{clip}} = g_{\text{acc}} \cdot \min\left(1, \frac{\tau}{\|g_{\text{acc}}\|_2 + \epsilon}\right), \quad \tau = \texttt{grad_clip_max_norm}$$

luego los guardas de desbordamiento usan `inf_post_clip_threshold` y lógica de reintentos limitada por `max_nan_retries`.

9.8. Partición de Test Reservada y Evaluación al Final del Entrenamiento

`training.test_percentage` reserva una fracción determinista del dataset de la tarea antes del primer paso del optimizador. Con n ejemplos, porcentaje reservado $\rho = \texttt{test_percentage}/100$ y una permutación π sorteada una sola vez con la semilla de la ejecución `training.seed`, la partición es

$$n_{\text{test}} = \text{clip}(\text{round}(\rho n),\, 1,\, n-1)\,, \qquad \mathcal{D}_{\text{train}} = \{x_{\pi(i)}\}_{i=n_{\text{test}}+1}^{n}, \qquad \mathcal{D}_{\text{test}} = \{x_{\pi(i)}\}_{i=1}^{n_{\text{test}}},$$

de modo que la misma configuración siempre reserva los mismos ejemplos, sea cual sea el mezclado del DataLoader. Tras la última época se evalúan los pesos finales una vez con el objetivo de la tarea bajo `no_grad`, y la pérdida y precisión ponderadas por muestra

$$\hat{L} = \frac{1}{n_{\text{test}}} \sum_{x \in \mathcal{D}_{\text{test}}} \ell(f_\theta; x), \qquad \widehat{\text{acc}} = \frac{1}{n_{\text{test}}} \sum_{x \in \mathcal{D}_{\text{test}}} a(f_\theta; x)$$

se escriben en `test_metrics.json` junto al log CSV. La partición cubre modelado de lenguaje enmascarado y causal, clasificación de texto, las tres tareas de visión, ASR (donde el campo de precisión lleva la tasa de error de caracteres, etiquetada `metric: cer`) y clasificación de audio. SBERT conserva su propia partición de validación mediante `training.sbert.max_eval_samples` y no se ve afectado. Fijar el campo en cero desactiva la partición y restaura el comportamiento histórico sobre todos los datos.

Algorithm 45 Enrutador de Modo de Inferencia SBERT

Require: modo m, modelo E, entradas X

if $m =$ `similarity` **then**
    retornar $\cos(E(x_1), E(x_2))$
else if $m =$ `search` **then**
    retornar top-k por producto-punto/coseno contra embeddings del corpus
else if $m =$ `cluster` **then**
    retornar etiquetas de clustering sobre $E(X)$
else
    retornar embeddings serializados $E(X)$
end if

9.9. Enrutador de Modo de Inferencia SBERT

El despachador de inferencia SBERT selecciona entre puntuación de similitud, búsqueda sobre corpus, agrupamiento y codificación persistente sobre un único codificador compartido [98]. La similitud coseno y la pérdida de regresión son:

$$\cos(e_1, e_2) = \frac{e_1^\top e_2}{\|e_1\|\|e_2\|}, \qquad \mathcal{L}_{\cos} = (\cos(e_1, e_2) - y)^2$$

con $y \in [-1, 1]$ en este pipeline.

10. Funciones de Activación

Este anexo documenta la librería completa de funciones de activación disponibles en el modelo. La librería proporciona **43 funciones de activación feed-forward**: 40 activaciones elementales y 3 variantes de FFN con compuerta. Las activaciones se agrupan en cinco familias, siguiendo la taxonomía de Dubey et al. [30] y la visión analítica de Lederer [68], y se añade la *Rational Activation Function* (RAF) aprendible de Fang et al. [34].

La selección se controla mediante el campo de configuración `ffn_activation` (un enumerado de cadenas) y se despacha en tiempo de ejecución mediante una factoría que replica el patrón de las capas de normalización (Anexo 7).

$$\texttt{ffn_activation} \in \{\texttt{silu}, \texttt{gelu}, \ldots, \texttt{raf}, \texttt{swiglu}, \texttt{geglu}, \texttt{reglu}\}$$

10.1. Familia Clásica / Sigmoid–Tanh (12 activaciones)

La familia clásica comprende activaciones suaves, sin estado, basadas en funciones logísticas y algebraicas [68, 30]. Todas comparten monotonicía o casi-monotonicía y son continuamente derivables.

$$\text{Sigmoid}(x) = \frac{1}{1+e^{-x}} \qquad \text{Rango: } (0,1),\ \text{monótona, suave} \quad [68] \tag{96}$$

$$\text{Tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\,\sigma(2x) - 1 \qquad \text{Rango: } (-1,1),\ \text{monótona, suave} \quad [68] \tag{97}$$

$$\text{Arctan}(x) = \arctan(x) \qquad \text{Rango: } (-\tfrac{\pi}{2}, \tfrac{\pi}{2}),\ \text{monótona, suave} \quad [68] \tag{98}$$

$$\text{Softsign}(x) = \frac{x}{1+|x|} \qquad \text{Rango: } (-1,1),\ \text{suave, deriv. una vez en 0} \quad [68] \tag{99}$$

$$\text{Elliott}(x) = \frac{x}{1+|x|} \qquad \text{Alias de Softsign (notación “elliottsig” del survey)} \quad [68] \tag{100}$$

$$\text{Identity}(x) = x \qquad \text{Rango: } (-\infty,\infty),\ \text{lineal, } C^\infty \quad \text{(interna)} \tag{101}$$

$$\text{Softplus}(x) = \log(1 + e^x) \qquad \text{Rango: } [0, \infty),\ \text{suave, } C^\infty \quad [37] \tag{102}$$

$$\text{Mish}(x) = x\,\tanh\big(\text{softplus}(x)\big) = x\,\tanh\big(\log(1 + e^x)\big) \qquad \text{Rango: } [\approx -0{,}31, \infty),\ \text{no monótona, suave} \quad [83] \tag{103}$$

$$\text{GELU}(x) = x\,\Phi(x) = x \cdot \tfrac{1}{2}\Big(1 + \text{erf}\Big(\frac{x}{\sqrt{2}}\Big)\Big) \qquad \text{Rango: } (\approx -0{,}17, \infty),\ \text{suave, probabilística} \quad [52] \tag{104}$$

$$\text{GELU-tanh}(x) = \tfrac{x}{2}\Big(1 + \tanh\Big(\sqrt{\tfrac{2}{\pi}}(x + 0{,}044715\,x^3)\Big)\Big) \qquad \text{Aproximación con tanh de GELU (más rápida)} \quad [52] \tag{105}$$

$$\text{ReLU}(x) = \text{máx}(0, x) \qquad \text{Rango: } [0, \infty),\ \text{tramos lineales, no suave en 0} \quad [85] \tag{106}$$

$$\text{SiLU}(x) = x\,\sigma(x) = \frac{x}{1 + e^{-x}} \qquad \text{Rango: } (\approx -0{,}278, \infty),\ \text{no monótona, suave} \quad [32] \tag{107}$$

SiLU (Sigmoid Linear Unit) es la activación por defecto (`ffn_activation=silu`). Cuando $\beta = 1$, el Swish paramétrico se reduce a SiLU. GELU y su aproximación con tanh son el estándar en BERT y GPT-2/3 [52]. Mish [83] es una composición auto-regularizada, no monótona, de Softplus y Tanh, que supera a Swish en varios benchmarks.

10.2. Familia Rectificada (12 activaciones)

La familia de rectificadores extiende ReLU con pendientes aprendibles, variantes acotadas y formas hiperbólicas rectificadas [85, 30].

$$\text{LeakyReLU}(x) = \begin{cases} x & x \geq 0 \\ a\,x & x < 0 \end{cases}, \quad a = 0{,}01 \qquad \text{Rango: } (-\infty, \infty) \quad [79] \tag{108}$$

$$\text{ReLU6}(x) = \text{mín}\big(\text{máx}(0, x),\ 6\big) \qquad \text{Rango: } [0, 6],\ \text{acotada, amigable para coma fija} \quad [56] \tag{109}$$

$$\text{HardSwish}(x) = x \cdot \frac{\text{ReLU6}(x + 3)}{6} \qquad \text{Rango: } (\approx -1{,}67, \infty),\ \text{aprox. barata de Swish} \quad [55] \tag{110}$$

$$\text{PReLU}(x) = \text{máx}(0, x) + \mathbf{p} \odot \text{mín}(0, x) \qquad \text{Pendiente por canal aprendible } \mathbf{p},\ \text{init } 0.25 \quad [49] \tag{111}$$

$$\text{AbsReLU}(x) = \text{máx}(0, x) - \text{máx}(0, -x) = x\ \text{(rampa de signo)} \qquad \text{Rango: } (-\infty, \infty) \quad [30] \tag{112}$$

$$\text{NLReLU}(x) = \beta\,\log\big(1 + \text{máx}(0, x)\big) \qquad \text{Rango: } [0, \infty),\ \beta = 1{,}0,\ \text{compresión log.} \quad [30] \tag{113}$$

$$\text{BReLU}(x) = \text{mín}\big(\text{máx}(0, x),\ t\big), \quad t = 1{,}0 \qquad \text{Rango: } [0, t],\ \text{rectificador acotado} \quad [30] \tag{114}$$

$$\text{VReLU}(x) = |x| \qquad \text{Rango: } [0, \infty),\ \text{rectificador simétrico} \quad [30] \tag{115}$$

$$\text{Hexpo}(x) = a\ \text{máx}(0, x) - c\ \text{máx}(0, -x), \quad a{=}c{=}1{,}0 \qquad \text{Rectificador asimétrico bilateral} \quad [30] \tag{116}$$

$$\text{PenalizedTanh}(x) = \text{máx}\big(0,\ \tanh(x)\big) \qquad \text{Rango: } [0, 1),\ \text{tanh rectificado, suave cerca de 0} \quad [30] \tag{117}$$

$$\text{DisReLU}(x) = \text{máx}(0,\ x - \delta), \quad \delta = 0{,}0 \qquad \text{ReLU desplazado a la derecha} \quad [30] \tag{118}$$

$$\text{LiSHT}(x) = x\,\tanh(x) \qquad \text{Rango: } [0, \infty)\ \text{(magnitud), no monótona, suave} \quad [99] \tag{119}$$

PReLU [49] convierte la pendiente negativa en un parámetro aprendible por canal, permitiendo a la red aprender la fuga óptima por característica. **HardSwish** [55] es la aproximación barata de Swish usada en MobileNetV3. **ReLU6** [56] se introdujo en MobileNetV1 por su compatibilidad con cuantización de coma fija.

10.3. Familia Exponencial / ELU (12 activaciones)

La familia exponencial-lineal proporciona saturación negativa suave y variantes auto-normalizadoras [21, 64, 30]. Varios miembros introducen parámetros aprendibles por canal.

$$\mathrm{ELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}, \quad \alpha = 1{,}0 \qquad \text{Rango: } (-\alpha, \infty) \quad [21] \tag{120}$$

$$\mathrm{SELU}(x) = \lambda\, \mathrm{ELU}_{\alpha'}(x) \qquad \alpha' \approx 1{,}6733,\ \lambda \approx 1{,}0507, \quad [64] \tag{121}$$

auto-normalizadora (media 0, var 1)

$$\mathrm{CELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^{x/\alpha} - 1) & x < 0 \end{cases}, \quad \alpha = 1{,}0 \qquad \text{Derivable una vez en 0 para cualquier } \alpha \quad [8] \tag{122}$$

$$\mathrm{PELU}(x) = \begin{cases} \frac{\alpha}{\beta}\, x & x \geq 0 \\ \alpha(e^{x/\beta} - 1) & x < 0 \end{cases} \qquad \alpha, \beta \text{ aprendibles por canal} \quad [112] \tag{123}$$

$$\mathrm{MPELU}(x) = \mathrm{ReLU}(x) + \beta \cdot \mathrm{ELU}_{\alpha}(x) \cdot \mathbb{1}_{x<0} \qquad \alpha, \beta \text{ aprendibles por canal} \quad [30] \tag{124}$$

$$\mathrm{FELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases} \qquad \alpha \text{ aprendible, acotado a } [0, 1] \quad [30] \tag{125}$$

$$\mathrm{EELU}(x) = \beta\, \mathrm{ReLU}(x) + \alpha(e^x - 1) \cdot \mathbb{1}_{x<0} \qquad \alpha, \beta \text{ aprendibles por canal} \quad [30] \tag{126}$$

$$\mathrm{PDELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^{x/\alpha} - 1) + (1 - \alpha)\, x & x < 0 \end{cases} \qquad \alpha \text{ aprendible, interpola CELU–identidad} \quad [30] \tag{127}$$

$$\mathrm{PREU}(x) = \beta\, \mathrm{ReLU}(x) + \alpha(e^x - 1) \cdot \mathbb{1}_{x<0}^{\leq 0} \qquad \alpha, \beta \text{ aprendibles por canal} \quad [30] \tag{128}$$

$$\mathrm{SoftExp}(x) = \begin{cases} \frac{1}{\alpha}\left(e^{\alpha x} - 1\right) + \alpha & \alpha > 0 \\ x & \alpha = 0 \\ -\frac{1}{\alpha}\ln\left(1 - \alpha(x + \alpha)\right) & \alpha < 0 \end{cases} \qquad \alpha \text{ aprendible; interp. exp/lineal/log} \quad [38] \tag{129}$$

$$\mathrm{ELiSH}(x) = \begin{cases} x\, \sigma(x) & x \geq 0 \\ (e^x - 1)\, \sigma(x) & x < 0 \end{cases} \qquad \text{Rama pos. Swish, rama neg. con compuerta ELU} \quad [10] \tag{130}$$

$$\mathrm{HardELiSH}(x) = \begin{cases} x \cdot \mathrm{hard_sigmoid}(x) & x \geq 0 \\ (e^x - 1) \cdot \mathrm{hard_sigmoid}(x) & x < 0 \end{cases} \qquad \text{Variante con sigmoide rígida de ELiSH} \quad [10] \tag{131}$$

donde $\mathrm{hard_sigmoid}(x) = \mathrm{ReLU6}(x + 3)/6$.
ELU [21] aporta saturación negativa suave. **SELU** [64] induce auto-normalización con $\alpha' \approx 1{,}6733$, $\lambda \approx 1{,}0507$ fijos. **CELU** [8] es derivable una vez en 0 para cualquier α (a diferencia de ELU).

PELU [112] hace ambos α, β aprendibles por canal. Las variantes paramétricas (MPELU, FELU, EELU, PDELU, PREU) generalizan ELU y PReLU introduciendo parámetros aprendibles por canal para las ramas negativa y positiva. **SoftExp** [38] interpola continuamente entre los regímenes logarítmico ($\alpha < 0$), lineal ($\alpha = 0$) y exponencial ($\alpha > 0$). **ELiSH** y **HardELiSH** [10] combinan comportamientos de Swish y ELU mediante una definición a tramos.

10.4. Swish y Activaciones Paramétricas

Swish [96] es una familia parametrizada por una pendiente β:

$$\text{Swish}_\beta(x) = x\,\sigma(\beta x) = \frac{x}{1 + e^{-\beta x}}.$$

Cuando $\beta = 1$ se obtiene SiLU (el valor por defecto). Con β grande se recupera ReLU; con $\beta \to 0$ se aproxima a la función lineal $x/2$.

- **Swish** (`swish`): β fijo (por defecto 1.0, configurado mediante `ffn_activation_config.swish_beta`). Sin parámetros aprendibles.
- **SwishTrainable** (`swish_trainable`): β es un parámetro aprendible por canal, inicializado a 1.0. Equivalente a SiLU al inicio, se adapta durante el entrenamiento.
- **Maxout** (`maxout`): calcula k proyecciones lineales y toma el máximo elemento a elemento: $\max_{i=0}^{k-1}(W_i x + b_i)$. Aproximador universal de cualquier función continua con suficientes piezas. El número de parámetros aumenta por un factor k (por defecto $k = 2$, configurado mediante `ffn_activation_config.maxout_pieces`).

Referencias: Swish [96], Maxout [39].

10.5. Funciones de Activación Racionales (RAF)

Siguiendo a Fang et al. [34], una *Rational Activation Function* es una razón aprendible de dos polinomios de bajo grado (un aproximante de Padé):

$$F(x) = \frac{P(x)}{Q(x)} = \frac{\displaystyle\sum_{j=0}^{m} a_j\, x^j}{1 + R(x)}$$

donde $\mathbf{a} \in \mathbb{R}^{m+1}$ y $\mathbf{b} \in \mathbb{R}^n$ son *aprendibles*, y el término del denominador $R(x)$ depende de la versión. La configuración por defecto coincide con el artículo: grado $(m, n) = (5, 4)$, versión "A" (segura), inicializada mediante un ajuste por mínimos cuadrados a GELU en $[-3, 3]$.

La implementación admite cinco variantes de denominador:

- **Versión A** (por defecto, "segura"): $Q(x) = 1 + \sum_k |b_k\, x^{k+1}|$ (valor absoluto por término). Garantiza $Q(x) \geq 1$, sin división por cero.
- **Versión B**: $Q(x) = 1 + \left|\sum_k b_k\, x^{k+1}\right|$ (valor absoluto de la suma completa). También garantiza $Q(x) \geq 1$.
- **Versión C**: $Q(x) = 0{,}1 + \left|\sum_k b_k\, x^{k}\right|$ con un suelo de 0,1. Denominador mínimo 0,1.
- **Versión D**: como B, pero inyecta ruido multiplicativo uniforme $U(1-\varepsilon, 1+\varepsilon)$ sobre los pesos del denominador *sólo durante el entrenamiento* ($\varepsilon = 0{,}1$). Efecto de regularización.
- **Versión N** ("no segura"): $Q(x) = 1 + \sum_k b_k\, x^{k+1}$ sin valor absoluto. Puede tener polos (ceros del denominador); usar con cuidado.

Escalado de entrada RAFT. Dado que un aproximante de Padé es un aproximador universal fiable sólo en un intervalo acotado, el modelo RAFT preprocesa las pre-activaciones de cada token con un escalado min–max por token hacia $[-3, 3]$:

$$\tilde{x} = \frac{x - \text{mín}(x)}{\text{máx}(x) - \text{mín}(x)} \cdot 6 - 3.$$

Se activa con `ffn_activation_config.raf_input_scaling` (por defecto `false`).

Objetivos de inicialización. Los coeficientes racionales pueden inicializarse ajustándose a cualquiera de las siguientes funciones objetivo: `gelu` (por defecto), `relu`, `leaky_relu`, `leaky_relu_0.1`, `sigmoid`, `tanh`, `swish`/`silu`, o `identity`. El ajuste usa una resolución por mínimos cuadrados sobre 2001 muestras uniformemente espaciadas en $[-3, 3]$.

Congelado. Fijar `raf_trainable=false` congela los parámetros racionales, útil para ajuste eficiente en parámetros. Las RAF congeladas se emparejan típicamente con una tasa de aprendizaje separada (mayor) para el resto de parámetros de la red.

Nota sobre el nombre. El acrónimo RAF significa *Rational* Activation Function, no "Rectified"; corresponde a la razón polinómica aprendible descrita arriba.

10.6. Variantes de FFN con Compuerta: SwiGLU, GEGLU, ReGLU

SwiGLU, GEGLU y ReGLU [101] son unidades feed-forward con compuerta que reemplazan el bloque estándar Linear → activación → Linear. Poseen sus propias proyecciones lineales y, por tanto, se implementan como módulos FFN, *no* como activaciones elementales:

$$\text{GatedFFN}(x) = \text{dropout}\Big(W_{\text{down}} \Big(\text{act}(x\, W_{\text{gate}}) \odot (x\, W_{\text{up}}) \Big) \Big),$$

donde $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{d \times d_{\text{ff}}}$ y $W_{\text{down}} \in \mathbb{R}^{d_{\text{ff}} \times d}$. La función de compuerta es:

- **SwiGLU** (`swiglu`): act = SiLU (por defecto, usado en Llama, PaLM).
- **GEGLU** (`geglu`): act = GELU (usado en algunas variantes de T5).
- **ReGLU** (`reglu`): act = ReLU (la opción más barata).

Cuando `ffn_activation` es uno de `swiglu`/`geglu`/`reglu`, la capa feed-forward sustituye los bloques FFN densos y MoE por un FFN con compuerta construido con la misma clase de proyección (BitLinear bajo BitNet). El sesgo está desactivado por defecto.

10.7. Esquema de Configuración

El objeto `ffn_activation_config` agrupa todos los parámetros de las activaciones aprendibles. Es opcional y se ignora para activaciones sin estado.

- `raf_degrees`: $[m, n]$ grados numerador/denominador del Padé (enteros ≥ 1). Por defecto: $[5, 4]$.
- `raf_version`: forma del denominador, uno de `A` (por defecto), `B`, `C`, `D`, `N`.
- `raf_approx_func`: objetivo de inicialización. Uno de `gelu` (por defecto), `relu`, `leaky_relu`, `leaky_relu_0.1`, `sigmoid`, `tanh`, `swish`, `silu`, `identity`.
- `raf_trainable`: congela los parámetros racionales cuando es `false`. Por defecto: `true`.
- `raf_input_scaling`: escalado min–max por token a $[-3, 3]$ antes del racional (preprocesamiento RAFT). Por defecto: `false`.

- `prelu_init`: pendiente negativa inicial para PReLU. Por defecto: 0,25.
- `elu_alpha`: constante de saturación de ELU. Por defecto: 1,0.
- `celu_alpha`: constante de saturación de CELU. Por defecto: 1,0.
- `swish_beta`: β fijo para Swish / β inicial para SwishTrainable. Por defecto: 1,0.
- `leaky_relu_slope`: pendiente negativa para LeakyReLU. Por defecto: 0,01.
- `maxout_pieces`: número de piezas lineales k para Maxout (entero ≥ 1). Por defecto: 2.
- Claves adicionales de la familia ELU paramétrica: `pelu_alpha`, `mpelu_alpha`, `mpelu_beta`, `felu_alpha`, `eelu_alpha`, `eelu_beta`, `pdelu_alpha`, `preu_alpha`, `preu_beta`, `softexp_alpha`. Todas por defecto 1,0 (excepto `softexp_alpha` que por defecto es 0,0).

10.8. Guía de Selección

El siguiente árbol de decisión ayuda a elegir la activación:

- **Por defecto seguro** → `silu` (SiLU/Swish$_1$, usado en Llama, PaLM).
- **Estabilidad clásica** → `gelu` o `relu`.
- **No monótona suave** → `mish` o `swish`.
- **Amigable para móvil** → `hardswish` o `relu6`.
- **Forma por tarea aprendible** → `raf`, `prelu`, `swish_trainable`, o `maxout`.
- **Redes auto-normalizadoras** → `selu` (con AlphaDropout).
- **FFN con compuerta (proyecciones propias)** → `swiglu`, `geglu`, o `reglu`.

Con cuantización BitNet, `silu` puede ser más robusta que las activaciones suaves que dependen de un cálculo de gradiente preciso. Las activaciones aprendibles (`raf`, `prelu`, `swish_trainable`) añaden parámetros pero pueden adaptarse a la distribución de la tarea.

11. Hyper-Connexiones Restringidas por Variedad (mHC)

Este anexo documenta la integración de las *Hyper-Connexiones Restringidas por Variedad* (*Manifold-Constrained Hyper-Connections*, mHC) [120], un marco de conexiones residuales que reemplaza el residual de identidad estándar por un flujo residual de n streams cuya matriz de mezcla de flujo se restringe al politopo de Birkhoff. El cuerpo principal difiere a este anexo la formulación matemática, los detalles de implementación y los resultados reportados; las claves bibliográficas citadas aquí se resuelven contra `docs/bibliography/`.

11.1. Motivación: la Propiedad de Mapeo de Identidad

La conexión residual estándar [50] propaga señales a través de la profundidad sin modificación: con $x_l \in \mathbb{R}^C$ y una función de capa F,

$$x_{l+1} = x_l + F(x_l, W_l),$$

de modo que, recursivamente, $x_L = x_l + \sum_{i=l}^{L-1} F(x_i, W_i)$. El término directo x_l (el *mapeo de identidad*) permite que la información fluya de capas superficiales a profundas sin cambios, lo que He et al. [50] identificaron como clave para el entrenamiento estable de redes muy profundas.

Las *Hyper-Connections* (HC) [120] ensanchan el flujo residual por un factor de expansión n: el flujo pasa a ser $x_l \in \mathbb{R}^{n \times C}$ mientras que la función interna F de cada capa sigue operando en dimensión C. Tres mapeos lineales aprendibles gobiernan el flujo en cada capa:

- $H_l^{\text{pre}} \in \mathbb{R}^{1\times n}$ agrega el flujo de dimensión nC a la entrada de capa de dimensión C;
- $H_l^{\text{post}} \in \mathbb{R}^{1\times n}$ proyecta la salida de la capa de vuelta al flujo;
- $H_l^{\text{res}} \in \mathbb{R}^{n\times n}$ mezcla características *dentro* del flujo residual.

La propagación de una sola capa es

$$x_{l+1} = H_l^{\text{res}} x_l + \big(H_l^{\text{post}}\big)^\top F\big(H_l^{\text{pre}} x_l, W_l\big),$$

y, recursivamente, la señal profunda-a-superficial está gobernada por el mapeo compuesto $\prod_{i=L-1}^{l} H_{i+1}^{\text{res}}$. Debido a que H_l^{res} no está restringida en HC, este compuesto se desvía de la identidad, la norma del flujo no se conserva y la señal hacia adelante/hacia atrás puede amplificarse o atenuarse sin límite. En un modelo MoE de 27B parámetros los autores reportan magnitudes de ganancia compuesta con picos cercanos a 3000 (frente a 1 para la identidad), correlacionadas con picos de norma de gradiente y una subida inesperada de pérdida alrededor del paso 12k. HC además multiplica el costo de acceso a memoria por aproximadamente n (el "muro de memoria").

11.2. Formulación

Sea $x_l \in \mathbb{R}^{n\times C}$ el flujo en la capa l. Los coeficientes se calculan a partir del flujo aplanado $\tilde{x}_l = \text{vec}(x_l) \in \mathbb{R}^{1\times nC}$:

$$\tilde{H}_l = \frac{1}{r_l}\Big(\alpha \odot \big(\tilde{x}_l\, \varphi_l\big)\Big) + b_l, \qquad r_l = \frac{\|\tilde{x}_l\|_2}{\sqrt{nC}},$$

donde $\varphi_l \in \mathbb{R}^{nC\times(n^2+2n)}$ es una proyección lineal aprendida, $b_l \in \mathbb{R}^{1\times(n^2+2n)}$ un sesgo aprendido y $\alpha^{\text{pre}}, \alpha^{\text{post}}, \alpha^{\text{res}} \in \mathbb{R}$ son escalares de compuerta aprendibles inicializados a un valor pequeño (0,01 en el paper y en este código base). El escalado por $1/r_l$ es matemáticamente equivalente a RMSNorm sobre el flujo aplanado, con la escala por dimensión absorbida en φ_l. Los tres bloques de coeficientes son entonces:

$$H_l^{\text{pre}} = \sigma\big(\tilde{H}_l^{\text{pre}}\big), \qquad H_l^{\text{post}} = 2\,\sigma\big(\tilde{H}_l^{\text{post}}\big), \qquad H_l^{\text{res}} = \text{SK}\big(\exp(\tilde{H}_l^{\text{res}})\big),$$

donde σ es la sigmoide logística (que impone no negatividad y evita la cancelación de señal por coeficientes de signo mixto) y $\text{SK}(\cdot)$ es la proyección Sinkhorn–Knopp sobre el conjunto de matrices doblemente estocásticas

$$\mathcal{M}^{\text{res}} = \big\{\, H \in \mathbb{R}^{n\times n} \;:\; H \ge 0,\;\; H\,\mathbf{1}_n = \mathbf{1}_n,\;\; \mathbf{1}_n^\top H = \mathbf{1}_n^\top \,\big\},$$

es decir, el politopo de Birkhoff (la envolvente convexa de las matrices de permutación). Partiendo de $M^{(0)} = \exp(\tilde{H}_l^{\text{res}})$, la proyección alterna normalizaciones de filas y columnas,

$$M^{(t)} = T_c\big(T_r(M^{(t-1)})\big),$$

para $t = 1, \ldots, t_{\text{máx}}$ rondas ($t_{\text{máx}} = 20$ por defecto tanto en el paper como en este código base). Finalmente, la entrada de capa y el flujo actualizado son

$$F_l^{\text{pre}} = H_l^{\text{pre}} x_l \in \mathbb{R}^{1\times C}, \qquad x_{l+1} = H_l^{\text{res}} x_l + \big(H_l^{\text{post}}\big)^\top \otimes F\big(F_l^{\text{pre}}, W_l\big).$$

11.3. Propiedades de la Restricción de Variedad

Restringir H_l^{res} al politopo de Birkhoff restaura el comportamiento de conservación del residual de identidad y garantiza tres propiedades [120]:

1. **Preservación de norma**: la norma espectral satisface $\|H_l^{\text{res}}\|_2 \leq 1$ (mapeo no expansivo), mitigando la explosión de gradientes.

2. **Cerradura composicional**: las matrices doblemente estocásticas son cerradas bajo multiplicación, por lo que el compuesto $\prod_i H_i^{\text{res}}$ sigue siendo doblemente estocástico a profundidad arbitraria, preservando la propiedad de mapeo de identidad entre *cualesquiera* dos profundidades.

3. **Interpretación geométrica**: como el politopo de Birkhoff es la envolvente convexa de las matrices de permutación, $H_l^{\text{res}}x$ es una combinación convexa de permutaciones de características, es decir, un mecanismo monótono de mezcla de información entre streams.

Cuando $n = 1$ la condición doblemente estocástica degenera al escalar 1, recuperando el mapeo de identidad exacto. Con $t_{\text{máx}}$ finito la proyección es aproximada, y la ganancia compuesta hacia atrás se desvía de 1 pero permanece acotada (alrededor de 1,6 en las ejecuciones de 27B del paper, una reducción de 3 órdenes de magnitud frente a los ≈ 3000 de HC).

11.4. Implementación en Frankenstein Transformer

mHC es un reemplazo opcional de la conexión residual estándar, controlado por el sub-objeto `model.mhc` (esquema jerárquico) o por las claves planas `use_mhc`, con `additionalProperties: false` a nivel de sub-objeto:

- `enabled` (`use_mhc`, bool, por defecto `false`): reemplazar el residual estándar por el flujo residual de n streams.
- `expansion_rate` (`mhc_expansion_rate`, int ≥ 1, por defecto 4): factor de expansión del flujo n; el flujo pasa a ser $n \times$ `hidden_size` mientras los internos de la capa permanecen en `hidden_size`. 1 recupera el mapeo de identidad.
- `sinkhorn_iters` (`mhc_sinkhorn_iters`, int ≥ 1, por defecto 20): rondas de normalización Sinkhorn–Knopp $t_{\text{máx}}$.
- `gating_init` (`mhc_gating_init`, float > 0, por defecto 0,01): valor inicial de los escalares de compuerta α.
- `checkpoint` (`mhc_checkpoint`, bool, por defecto `false`): gradient checkpointing en capas mHC, intercambiando cómputo por memoria para mitigar el aumento de $\sim n\times$ en la memoria de activaciones del flujo residual de n streams.
- `full_prec_under_bitnet` (`mhc_full_prec_under_bitnet`, bool, por defecto `true`): mantener la proyección de coeficientes φ_l como `nn.Linear` de precisión completa incluso cuando `use_bitnet` es `true`, evitando el ruido de cuantización ternaria en los pequeños coeficientes mHC; cuando es `false` (y BitNet está activado), φ_l usa `BitLinear`.

El módulo vive en `src/model/mhc.py`:

- `SinkhornKnoppFunction` (`torch.autograd.Function`): el pase hacia adelante aplica las normalizaciones alternadas de filas y columnas de $\exp(\cdot)$; el pase hacia atrás recomputa la iteración completa bajo `torch.enable_grad` y diferencia a través de ella, obteniendo el producto Jacobiano-vector exacto (la estrategia de recomputación en chip descrita en el paper).
- `ManifoldHyperConnections` (`nn.Module`): contiene φ_l (`proj`), b_l (`bias`) y los tres escalares de compuerta `alpha_pre`, `alpha_post`, `alpha_res`; expone `fpre` (proyectar el flujo a la entrada de capa), `recombine` (actualizar el flujo con la salida de la capa) y `mappings` (calcular $H^{\text{pre}}, H^{\text{post}}, H^{\text{res}}$).

Cableado en `src/model/frankenstein_model.py`:

- `HybridLayer` gana módulos opcionales `mhc_attn` y `mhc_ffn` (un módulo mHC por función de capa); la ruta `_forward_dense_mhc` ejecuta atención y FFN como funciones de capa sobre el flujo compartido (B, S, n, C).
- `FrankensteinTransformer` expande el embedding de dimensión C a (B, S, n, C) al entrar mediante `mhc_in_proj` y lo colapsa de vuelta a C mediante `mhc_out_proj` antes de la cabeza.
- Los parámetros mHC se enrutan al grupo de parámetros `other` del optimizador.
- mHC es **incompatible con** `use_mixture_of_depths`: el enrutamiento MoD de tokens opera sobre un flujo C-dimensional único, en conflicto con el flujo residual de n streams; se lanza un `ValueError` si ambos están habilitados.

Ejemplo de configuración: `configs/examples/es_arch_mhc_adamw.yaml`.

11.5. Resultados Reportados

El paper evalúa mHC (y HC) con $n = 4$, $t_{\text{máx}} = 20$ e inicialización de α en 0,01 sobre modelos MoE estilo DeepSeek-V3 (atención MLA, RoPE, RMSNorm, balance de carga sin pérdida auxiliar) de 3B a 27B parámetros [120]:

- **Estabilidad**: mHC elimina las subidas de pérdida de HC (p. ej., cerca del paso 12k en el modelo de 27B) y mantiene acotada la ganancia residual compuesta ($\approx 1{,}6$ frente a picos de HC de ≈ 3000).
- **Calidad**: en el modelo de 27B, mHC mejora la pérdida final de entrenamiento en 0,021 sobre la línea base, supera a la línea base en los 8 benchmarks de evaluación (BBH, DROP, GSM8K, HellaSwag, MATH, MMLU, PIQA, TriviaQA) y a HC en 7 de 8, con las mayores ganancias en tareas de razonamiento (+2,1 BBH, +2,3 DROP sobre HC).
- **Escalado**: la mejora relativa de pérdida sobre la línea base se mantiene a través de escalas de cómputo (3B → 9B → 27B) y a través de 1T tokens de escalado de datos.
- **Sobrecarga**: con kernels fusionados (TileLang), cálculo de coeficientes en precisión mixta, recomputación selectiva y solapamiento de comunicación DualPipe, mHC añade solo 6,7 % de tiempo adicional de entrenamiento con $n = 4$ en el modelo de 27B.

11.6. Hyperloop: Hiperconexiones a Nivel de Bucle en Transformers en Bucle

El *Hyperloop Transformer* [128] combina Transformers en bucle — que reutilizan los pesos de capa a través de la profundidad — con hiperconexiones *a nivel de bucle*, orientándose a la eficiencia de parámetros en lugar de a la profundidad bruta. La pila de capas se particiona en *ciclo medio*: un bloque inicial (se ejecuta una vez), un bloque intermedio en bucle y un bloque final (se ejecuta una vez); el bloque intermedio en bucle se envuelve en hiperconexiones aplicadas solo tras cada iteración de bucle. Con $Y_t^{(l)} \in \mathbb{R}^{n \times C}$ el flujo matricial en el bucle l, $z_t^{(l)} = \text{RMSNorm}(\text{flatten}(Y_t^{(l)}))$, y parámetros por bucle $\{W_l^\tau, b_l^\tau, \alpha_l^\tau\}$ para $\tau \in \{\text{pre}, \text{post}, \text{res}\}$ más un embedding de posición de bucle $e_l \in \mathbb{R}^C$, la recurrencia es

$$y_t^{(l+1)} = H_l^{\text{res}}\, y_t^{(l)} + H_l^{\text{post}} \otimes \big(F(H_l^{\text{pre}}\, y_t^{(l)}) + e_l\big), \tag{132}$$

con los coeficientes dependientes de datos

$$H_l^{\text{pre}} = \sigma\big(\alpha_l^{\text{pre}}(W_l^{\text{pre}} z_t^{(l)}) + b_l^{\text{pre}}\big), \qquad H_l^{\text{post}} = 2\sigma\big(\alpha_l^{\text{post}}(W_l^{\text{post}} z_t^{(l)}) + b_l^{\text{post}}\big),$$
$$H_l^{\text{res}} = \text{diag}\big(\sigma(\alpha_l^{\text{res}}(W_l^{\text{res}} z_t^{(l)}) + b_l^{\text{res}})\big) \qquad W_l^{\text{res}} \in \mathbb{R}^{n \times nC}$$

(el H^{res} diagonal sustituye el sinkhorn$(\cdot)$ de mHC sobre una proyección densa $n^2 \times nC$ por una sigmoide sobre una compuerta diagonal — la ablación de la Tabla 6 del paper muestra que esto es a la vez más barato y más preciso: 14.40 de perplejidad frente a 14.59 Sinkhorn y 14.61 identity).

Cuatro propiedades distinguen el cableado Hyperloop del mHC por subcapa (§11): (1) las hiperconexiones se disparan una vez por *bucle*, no tras cada subcapa — también la ubicación más eficaz en la ablación de la Tabla 5 del paper; (2) el flujo se crea *copiando* la salida del bloque inicial n veces y se colapsa *promediando* los streams antes del bloque final, reemplazando las proyecciones aprendidas de mHC; (3) los parámetros son *por bucle*, relajando la compartición estricta de pesos de los Transformers en bucle ordinarios — el análisis de similitud de representaciones del paper (Tabla 8) muestra que las representaciones de las capas en bucle se desvían más entre bucles, lo que identifica como la fuente de las ganancias de calidad; y (4) H^{res} usa por defecto la parametrización sigmoide diagonal.

La integración en Frankenstein se controla con el booleano `mhc.hyperloop` (plano `mhc_hyperloop`; requiere `mhc.enabled` y `dims.num_loops` ≥ 2), con tres claves adicionales:

- `hyperloop_begin_layers` / `hyperloop_end_layers` (int ≥ 0, por defecto 0): tamaños de los bloques inicial/final, que se ejecutan exactamente una vez. El paper asigna $\sim$25 % de los parámetros a cada uno y 50 % al bloque intermedio (profundidad desenrollada = inicial + intermedio $\times$ bucles + final; p. ej. $2L \to 4L(\times 3) \to 2L$ en 16 capas). 0/0 recorre la pila completa.
- `hyperloop_res_parameterization` (enum, por defecto `diagonal`): `diagonal`, `sinkhorn` (mezcla doblemente estocástica completa estilo mHC, reutilizando `SinkhornKnoppFunction`) o `identity`.

Implementación: `src/model/hyperloop.py` aloja `HyperloopConnections` con `projs` / `biases` / `alphas_` / `loop_pos_embs` (e_l, inicializados a cero para que al inicio la actualización se reduzca a la recurrencia del Transformer en bucle simple). `FrankensteinEncoder` (y `FrankensteinViT`) lo construyen *en lugar de* la pareja `mhc_in_proj`/`mhc_out_proj`; el pase hacia adelante ejecuta inicial $\to$ copia $\to$ lectura/capa/actualización por bucle $\to$ promedio $\to$ final. `HybridLayer` desactiva sus módulos por capa `mhc_attn`/`mhc_ffn` bajo Hyperloop, de modo que las capas del bloque intermedio ejecutan la ruta C-dimensional estándar — lo cual **restaura la compatibilidad con Mixture-of-Depths** (el mHC por capa prohíbe MoD). Hyperloop es incompatible con los residuales AttnRes (`residuals.type full_attn` / `block_attn`), que asumen un flujo de n streams por capa lógica. El presupuesto de parámetros por bucle es $3n^2C + 3n + 3 + C$ (diagonal): para el modelo de 136M del paper ($C = 1024$, $n = 4$, 3 bucles) solo $\approx$150K parámetros más allá del Transformer en bucle simple.

Ejemplo de configuración: `configs/examples/hyperloop_mhc_adamw.yaml`.

Resultados reportados (arXiv:2604.21254)

Entrenado en FineWeb-Edu con $n = 4$ streams y 3 bucles (reparto de parámetros 25 %/50 %/25 %), contra líneas base Transformer y mHC con profundidad equiparada:

- **Eficiencia de parámetros**: a escala equiparada de 240M/1B/2B, Hyperloop usa $\approx$50 % menos parámetros (136M/580M/991M) e iguala o supera las líneas base en perplejidad (14.40 vs 14.65 a escala 240M; 9.65 vs 10.19 a 1B; 8.49 vs 8.60 a 2B).
- **Cuantización**: la perplejidad INT4 (GPTQ) se degrada menos que en el modelo en bucle simple (14.68 vs 15.18 a escala 240M), por lo que el ahorro de parámetros sobrevive a la compresión post-entrenamiento.
- **Sobrecarga**: la colocación a nivel de bucle más el H^{res} diagonal mantienen el throughput de entrenamiento dentro de $\approx$5 % del Transformer en bucle simple en PyTorch sin optimizar (el mHC por capa pierde $\approx$33 % sin kernels fusionados).

- **Ablaciones**: la colocación por bucle supera a la por capa (Tabla 5); el H^{res} diagonal supera a Sinkhorn e identity (Tabla 6); el número de streams n muestra rendimientos decrecientes más allá de 4 (Tabla 4); LoRA por bucle (Transformers recursivos relajados) ayuda mucho menos por parámetro que la relajación por hiperconexiones (Tabla 7).

12. Residuales de Atención (AttnRes)

Este anexo documenta la integración de los *Residuales de Atención* (AttnRes) [110], una generalización de la conexión residual que reemplaza la suma residual de coeficiente unitario fijo por *atención softmax aprendida sobre la profundidad*. El cuerpo principal delega a este anexo la formulación matemática, las cuatro estrategias disponibles (estándar, sin residual, Full AttnRes, Block AttnRes), el cableado de implementación, la integración con mHC y Mixture-of-Depths, y los resultados reportados. Las claves de bibliografía citadas aquí se resuelven contra `docs/bibliography/`.

12.1. Motivación: De una Suma Fija a Atención sobre la Profundidad

La conexión residual estándar [50] propaga señales a través de la profundidad sin modificar: con $x_l \in \mathbb{R}^C$ y una función de capa F,

$$x_{l+1} = x_l + F(x_l, W_l),$$

de modo que, recursivamente, $x_L = x_l + \sum_{i=l}^{L-1} F(x_i, W_i)$. El término identidad x_l (el *mapeo de identidad*) permite que la información fluya de capas superficiales a profundas sin cambios, lo que He et al. [50] identificaron como clave para el entrenamiento estable de redes muy profundas. Las redes Highway [106] generalizan esto con compuertas elemento-a-elemento aprendidas $\mathbf{g}_l \in [0,1]^C$,

$$x_{l+1} = (1 - \mathbf{g}_l) \odot x_l + \mathbf{g}_l \odot F(x_l, W_l),$$

que interpolan entre la ruta de identidad y la ruta de transformación con pesos dependientes de la entrada.

Los *Residuales de Atención* [110] dan el siguiente paso: en lugar de ponderar un único estado comprimido x_l, la capa l *atiende* sobre todas las salidas de capas anteriores. Sea $v_0 = h_1$ (el embedding de tokens) y $v_i = F(x_i, W_i)$ para $i \geq 1$. AttnRes computa, para cada capa l,

$$q_l = w_l, \qquad k_i = \text{RMSNorm}(v_i), \qquad \alpha_{i \to l} = \text{softmax}_i(q_l^\top k_i), \qquad x_{l+1} = \sum_{i=0}^{l} \alpha_{i \to l}\, v_i,$$

donde $w_l \in \mathbb{R}^C$ es un pseudo-query aprendido por capa, *inicializado a cero* según el paper. Con la inicialización a cero, el softmax es uniforme, por lo que AttnRes degenera a un promedio equiponderado de todas las salidas previas — igualando el residual estándar en el paso cero y evitando volatilidad de entrenamiento. RMSNorm sobre las keys evita que las capas con magnitudes naturalmente grandes dominen el softmax (ablación en el paper: § 5.3).

12.2. Cuatro Estrategias

El campo de esquema `model.residuals.type` selecciona una de las cuatro estrategias, cada una implementada como un módulo separado bajo `src/model/residuals/`.

12.2.1. Residual Estándar (`"standard"`)

La estrategia por defecto, retrocompatible:

$$x_{l+1} = x_l + F(x_l, W_l).$$

Sin estado, sin parámetros extra, sin cómputo extra. Implementada como un pass-through no-op en `StandardResidual` que delega el merge a `HybridLayer`.

12.2.2. Sin Residual ("none", experimental)

Elimina la conexión skip por completo:

$$x_{l+1} = F(x_l, W_l).$$

Sin estado, sin parámetros extra. Útil solo como sonda de ablación; el término identidad es crítico para arquitecturas profundas (paper § 2.1) y el entrenamiento suele ser inestable sin él. Implementado como `NoResidual`.

12.2.3. Residuales de Atención Completa ("full_attn", paper § 3.1)

Cada capa atiende sobre *todas* las salidas de capas anteriores:

$$x_{l+1} = \sum_{i=0}^{l} \alpha_{i\to l}\, v_i, \qquad \alpha_{i\to l} = \mathrm{softmax}_i\big(w_l^\top \mathrm{RMSNorm}(v_i)\big).$$

Añade $L{\cdot}C$ parámetros (L = profundidad lógica = `num_layers` × `num_loops`, C = `hidden_size`), un vector de query por capa. La memoria y el cómputo crecen como $O(L\,d)$ por token; a profundidad típica ($L < 1000$) esto es insignificante. Implementado como `FullAttentionResidual`; soporta `attnres_gradient_checkpoint` para envolver la atención en `torch.utils.checkpoint` en ejecuciones con restricciones de memoria.

12.2.4. Residuales de Atención por Bloques ("block_attn", paper § 3.2)

Particiona las L capas lógicas en N bloques de $S = \lceil L/N \rceil$ capas cada uno. Dentro de un bloque las salidas de capa se acumulan como suma parcial estándar b_n^i; a través de los bloques la atención se aplica sobre las N representaciones de bloque completas y la suma parcial en curso. Para la capa l en el bloque n,

$$x_{l+1} = \sum_{m=0}^{n} \alpha_{m\to l}\, v_m, \qquad \alpha_{m\to l} = \mathrm{softmax}_m\big(w_l^\top \mathrm{RMSNorm}(v_m)\big),$$

con $v_m = b_m$ para $m < n$ y $v_n = b_n^{il}$ (la suma parcial intra-bloque actual). N interpola entre Full AttnRes ($N = L$, un bloque por capa) y residuales estándar ($N = 1$, el embedding aislado como b_0). El sweet spot empírico del paper es $N \approx 8$; este código base usa $N = 8$ por defecto y valida $N \le L$. Implementado como `BlockAttentionResidual`, que sigue el Algoritmo 1 del paper (cómputo en dos fases con fusión online-softmax).

12.3. Implementación en Frankenstein Transformer

El paquete `src/model/residuals/` aloja una clase por estrategia (`StandardResidual`, `NoResidual`, `FullAttentionResidual`, `BlockAttentionResidual`) más un `ResidualBase` abstracto que define los hooks del ciclo de vida `set_streams`, `register_state`, `reset_state`, `forward` y `finalize`. La factoría `build_residual(config)` en `src/model/residuals/factory.py` selecciona la clase correcta desde `FrankensteinModelConfig`.

Cableado en `src/model/frankenstein_encoder.py`:

- `FrankensteinEncoder` posee el módulo `ResidualBase` como `self.residual`; la factoría se invoca una vez en `__init__`.
- Antes del bucle de profundidad en loop, el encoder llama a `reset_state` y, para variantes AttnRes, a `set_embedding(x)` para que la atención sobre la profundidad conozca la fuente $v_0 = h_1$.

- Tras cada llamada a `HybridLayer`, si `self.residual.is_attn_res` es true, el encoder aplica `x = self.residual(layer_idx, x)` para sobrescribir el flujo con la agregación atendida. Para estrategias sin estado (`standard`, `none`) este paso se omite y `HybridLayer` continúa aplicando su propio merge residual internamente.

  Cableado en `src/model/hybrid_layer.py`:

- `HybridLayer` lee `residual_type` de la configuración y despacha el merge residual según corresponda. Para `standard` el merge es el original `x = residual + mixer(norm(x))`; para `none` el skip se elimina (`x = mixer(norm(x))`); para las variantes AttnRes la capa realiza el merge estándar internamente porque la atención sobre la profundidad se aplica externamente por el encoder.

- `_forward_dense_mhc` (la ruta mHC) no se ve afectada porque mHC tiene su propia residual de expansión de flujos; combinar AttnRes con mHC lo maneja el campo `mhc_stream_mode` del módulo residual.

  Esquema (jerárquico, con los renombres a claves planas entre paréntesis) y valores por defecto:

- `model.residuals.type` (`residual_type`, enum `[standard, none, full_attn, block_attn]`, default `standard`): selecciona la estrategia.

- `model.residuals.full_attn.init_query_zero` (`full_attn_init_query_zero`, bool, default `true`): inicializa a cero los vectores de query por capa $\mathbf{w}_l$.

- `model.residuals.full_attn.use_rmsnorm_keys` (`full_attn_use_rmsnorm_keys`, bool, default `true`): aplica RMSNorm sobre las keys antes del producto punto.

- `model.residuals.block_attn.num_blocks` (`block_attn_num_blocks`, int ≥ 1, default 8): número de representaciones de bloque N; validado $N \leq L$.

- `model.residuals.block_attn.init_query_zero` (`block_attn_init_query_zero`, bool, default `true`): inicializa a cero los vectores de query por capa.

- `model.residuals.block_attn.use_rmsnorm_keys` (`block_attn_use_rmsnorm_keys`, bool, default `true`): RMSNorm sobre las keys.

- `model.residuals.mhc_stream_mode` (`attnres_mhc_stream_mode`, enum `[independent, joint]`, default `independent`): cómo AttnRes interactúa con el residual de n streams de mHC.

- `model.residuals.gradient_checkpoint` (`attnres_gradient_checkpoint`, bool, default `false`): envuelve la atención AttnRes en gradient checkpointing.

12.4. Integración con mHC y Mixture-of-Depths

La estrategia residual es *ortogonal* a mHC y Mixture-of-Depths (MoD): todas las combinaciones están soportadas. Los modos de interacción son:

mHC × AttnRes (`mhc_stream_mode`). Cuando `use_mhc=true` y el residual es `full_attn` o `block_attn`, la atención sobre la profundidad se aplica *por stream* (`ïndependent"`) o sobre la proyección *aplanada* nC-dimensional (`"joint"`). El modo independent ejecuta n atenciones en paralelo, una por stream — coincidiendo con el encuadre del paper donde cada stream es una "vista"paralela del residual. El modo joint trata los n streams como un único residual grueso, más expresivo pero más hambriento de parámetros. Las proyecciones de coeficientes de mHC (φ_l, b_l, compuertas α) no se ven afectadas; AttnRes añade sus propios parámetros encima.

Mixture-of-Depths × AttnRes. MoD opera *dentro* de una capa (selecciona qué tokens reciben la actualización completa de capa) y es ortogonal a la elección residual. Cuando `use_mixture_of_depths=true` junto con un residual AttnRes, solo los tokens seleccionados por MoD contribuyen al buffer de salidas de capa (Full AttnRes) o a la suma parcial intra-bloque (Block AttnRes); el resto pasa sin cambios y se re-ensambla mediante la lógica existente de MoD. MoD es mutuamente excluyente con mHC (se lanza un `ValueError` en `HybridLayer.__init__` si ambos están habilitados); las combinaciones correspondientes son (no_mHC, MoD, AttnRes), (mHC, no_MoD, AttnRes), o (mHC, no_MoD, no_AttnRes).

12.5. Resultados Reportados

El paper [110] valida AttnRes mediante leyes de escalado, ablaciones y una extensión Kimi Linear de 48B parámetros:

- **Leyes de escalado**: tanto Full como Block AttnRes superan consistentemente al baseline PreNorm a través de cinco tamaños de modelo. A 5.6 PFLOP/s-días, Block AttnRes alcanza pérdida 1,692 versus 1,714 del baseline — equivalente a una ventaja de cómputo de 1,25×. La brecha entre Full y Block AttnRes se reduce con la escala (Full 1,737 vs. Block 1,746 en el modelo de 16 capas).
- **Ablaciones**: la atención softmax dependiente de la entrada (1,737) supera al mezclado escalar independiente de la entrada (1,749) y a la atención sigmoid (1,741). RMSNorm sobre las keys es esencial (Full 1,737 vs. 1,743 sin ella; Block 1,746 vs. 1,750). La agregación multi-cabeza sobre la profundidad ($H = 16$) perjudica a Block AttnRes (1,752 vs. 1,746), indicando que la mezcla óptima es en gran medida uniforme a través de los canales. La atención de ventana deslizante sobre las últimas $W = 8$ capas se queda corta frente a Block AttnRes (1,764 vs. 1,746).
- **Extensión Kimi Linear**: un modelo Kimi Linear de 48B parámetros (3B activados) con Block AttnRes ($N \approx 9$, $S = 6$) pre-entrenado sobre 1.4T tokens mejora al baseline en los 14 benchmarks downstream (MMLU-Pro, GPQA-Diamond, BBH, HellaSwag, TriviaQA, GSM8K, MGSM, Math, CMath, HumanEval, MBPP, CMMLU, C-Eval, ARC-C). Las mayores ganancias son en razonamiento multi-paso (GPQA-Diamond +7,5, Minerva Math +3,6) y generación de código (HumanEval +3,1). El análisis de la dinámica de entrenamiento muestra que AttnRes mitiga la dilución PreNorm: las magnitudes del estado oculto permanecen acotadas a lo largo de la profundidad y las normas del gradiente se distribuyen más uniformemente entre capas.
- **Sobrecosto**: el sobrecosto de entrenamiento es marginal cuando no se habilita paralelismo de pipeline; con paralelismo de pipeline Block AttnRes añade $< 4\,\%$ de sobrecosto end-to-end (caching cross-stage elimina transferencias redundantes) y $< 2\,\%$ de sobrecosto de latencia de inferencia (cómputo en dos fases con fusión online-softmax).

Configuraciones de ejemplo: `configs/examples/full_attn_res_adamw.yaml`, `configs/examples/block_attn_res_adamw.yaml`, `configs/examples/mhc_full_attn_res_adamw.yaml`.

13. Vision Transformer: Predicción de Parches, Clasificación y Segmentación

Este anexo documenta la clase de modelo `frankenstein_vit`, una implementación del Vision Transformer (ViT) [28] que reutiliza la pila de `HybridLayer` agnóstica de modalidad del encoder. Soporta tres tareas: predicción autosupervisada de parches enmascarados, clasificación de imágenes y segmentación de imágenes (por-pixel o estilo EoMT [60]).

13.1. Embedding de Parches y Codificación Posicional

Embedding de Parches. La imagen de entrada $x \in \mathbb{R}^{H\times W\times C}$ se divide en $N = HW/P^2$ parches no solapados de $P \times P$, proyectados linealmente a D dimensiones mediante una Conv2d (Ec. 1 del paper ViT). La secuencia de entrada se construye como

$$z_0 = [x_{\text{class}};\ x_p^1 E;\ \ldots;\ x_p^N E] + E_{\text{pos}},$$

donde $E \in \mathbb{R}^{P^2C\times D}$ es la matriz de proyección de parches y E_{pos} es la codificación posicional.

Codificación Posicional. Por defecto, se usa una `nn.Parameter` aprendible 1D de forma $(1, N + 1, D)$, fiel al paper ViT. La alternativa `none` confía en RoPE/HoPE dentro de los mezcladores de atención, delegando la codificación posicional a los componentes de atención reutilizables.

Token [CLS]. Un token aprendible opcional se antepone a la secuencia de parches para clasificación. La salida en la posición 0 alimenta la cabeza de clasificación.

13.2. Predicción de Parches Enmascarados (Autosupervisado)

El preentrenamiento autosupervisado sigue el esquema de enmascaramiento estilo BERT del Apéndice B.1.2 del paper ViT: se corrompe el 50 % de los parches, de los cuales el 80 % se reemplaza por un token de máscara, el 10 % por un parche aleatorio y el 10 % se mantiene sin cambios.

Se soportan tres objetivos de reconstrucción:

1. **Color medio de 3 bits**: cross-entropy sobre 512 clases (mejor desempeño reportado).
2. **Parche 4×4 submuestreado de 3 bits**: 16×512 cabeceras de cross-entropy.
3. **L2 de parche completo**: MSE sobre los píxeles crudos (estilo MAE [51]).

El modelo predice solo en las posiciones enmascaradas; la pérdida se calcula exclusivamente sobre los parches corrompidos, evitando que la reconstrucción trivial de parches no enmascarados domine la señal de aprendizaje.

13.3. Clasificación de Imágenes

Pooling. La representación de salida se obtiene del token [CLS] (posición 0) o mediante Global Average Pooling (GAP) sobre todos los parches.

Cabeza. Una capa lineal Linear(D, K) inicializada en cero, siguiendo ViT Sec. 3.2, para fine-tuning estable sobre las clases K.

Pérdida. Cross-entropy estándar sobre las K clases.

13.4. Segmentación de Imágenes

Dos tipos de cabeza, seleccionados por el campo de esquema `seg_head_type`:

13.4.1. Cabeza Por-Pixel (`pixel`)

Cada token de parche se proyecta a `num_seg_classes` canales mediante una Linear(D, num_seg_classes), se redimensiona a $(H/P, W/P)$ y luego un upsampler de convoluciones transpuestas estilo ViTDet lo escala a la resolución completa (H, W). La pérdida combina cross-entropy + Dice. Se soporta segmentación 1D en escala de grises (2 clases) y multicolor no solapada ($N + 1$ clases).

13.4.2. Encoder-only Mask Transformer (`eomt`)

Basado en EoMT [60]:

- El backbone se divide en L_1 (solo parches) + L_2 (parches + K consultas aprendibles).
- La MHSA dentro de los bloques L_2 procesa los cuatro cuadrantes de atención (parche↔parche, consulta↔consulta, consulta↔parche, parche↔consulta) en una sola operación.
- **Módulo de máscara**: logits de clase lineales + MLP de 3 capas para el embedding de máscara, producto punto con $\tilde{F}_4$ escalado.
- **Pérdida**: BCE (×5) + Dice (×5) + CE (×2) con emparejamiento húngaro.
- **Recocido de máscara**: P_{mask} decae polinomialmente (factor 0,9) a 0 en inferencia, lo que revierte el modelo a un ViT plano, 4× más rápido.

13.5. Reutilización de la Infraestructura Existente

Los 34 mezcladores de atención (excepto `engram_attn`, que requiere IDs de tokens), 6 normas, 4 residuales, 42 activaciones y 23 optimizadores se reutilizan sin cambios mediante `HybridLayer`, que es agnóstica de modalidad y opera sobre tensores de forma (B, S, D).

Las tres tareas de visión leen `batch_size`, `num_epochs`, `optimizer`, `scheduler_type`, `grad_clip_max_norm` y el checkpointing del mismo bloque `training:` top-level que `mlm` y `causal_lm`. El sub-bloque opcional `training.<task>:` solo contiene parámetros específicos de tarea: `label_smoothing` para clasificación, y `seg_loss_bce_weight` / `seg_loss_dice_weight` / `seg_loss_ce_weight` / `mask_annealing_factor` para segmentación EoMT.

La clase `FrankensteinViT` refleja el patrón envoltorio de `FrankensteinDecoder`: posee un `PatchEmbed` + una pila de `HybridLayer` + una cabeza específica de tarea, y fuerza `mode='encoder'` en tiempo de ejecución.

13.6. Pseudocódigo en PyTorch

El siguiente pseudocódigo ilustra el paso forward para la tarea de clasificación:

Entrada: `pixel_values` $\in \mathbb{R}^{B\times C\times H\times W}$
$x \leftarrow$ `patch_embed`(`pixel_values`) ▷ (B, N, D)
$x \leftarrow$ [`cls_token`]; x ▷ anteponer token [CLS] → $(B, N{+}1, D)$
$x \leftarrow x +$ `pos_embed` ▷ codificación posicional aprendible 1D
$x \leftarrow$ `dropout`(x)
for cada `layer` en `layers` **do**
    $x \leftarrow$ `layer`(x) ▷ `HybridLayer` agnóstica de modalidad
end for
$x \leftarrow$ `final_norm`(x)
`logits` $\leftarrow$ `head`$(x[:, 0])$ ▷ token [CLS]
Salida: `logits` $\in \mathbb{R}^{B\times K}$

14. Codificaciones Posicionales de Todo el Modelo

Este anexo documenta el subsistema unificado de codificación posicional (PE) que aplica una única elección de `model.positional_encoding` a todos los mezcladores de atención de la red. El contrato del esquema es:

`positional_encoding` ∈ {`rope`, `hope`, `nope`, `alibi`, `bam`, `pape`, `pape_efficient`, `pape_ri`, `sinusoidal_absolute`, `sinusoidal_rotary`, `learned_absolute`, `none`}

La implementación reside en `src/model/embeddings/` y los ayudantes de inyección compartidos en `src/model/attention/common.py`. Las claves bibliográficas citadas aquí se resuelven contra `docs/bibliography/`.

14.1. Arquitectura: Un Módulo Compartido, Exclusión Por Mezclador

Un único módulo de PE se construye una vez en el codificador (`FrankensteinEncoder` o `FrankensteinViT`) mediante `build_pos_encoder(config)` y se inyecta en cada `HybridLayer`, que lo reenvía a cada mezclador en `__init__(config, pos_encoder=None)` y `forward(..., pos_encoder=None)`.

Cada mezclador lleva una bandera `use_pe` por mezclador (`<mezclador>_attn_use_pe`, por defecto `True` para mezcladores de atención, `False` para mezcladores recurrentes/de decaimiento como `retnet`, `mamba`, `ode`, `gla_attn`, `deltanet_attn`, `mtla_attn` y los seis mezcladores Falcon `falcon{1,2,3,1a,2a,3a}_attn`). Cuando `use_pe` es `False`, el mezclador ignora completamente el módulo inyectado.

La PE se aplica en dos fases mediante los ayudantes compartidos `apply_pe_to_qk()` (llamado antes del cálculo de puntuaciones de atención) y `apply_pe_to_scores()` (llamado después). El ayudante despacha según el tipo de PE y se convierte en una operación nula cuando la bandera `use_pe` del mezclador es `False`.

14.2. Estilos de Inyección

Los 12 valores del enum se agrupan en cuatro estilos de inyección:

1. **Rotación** (`rope`, `hope`, `sinusoidal_rotary`): rota los tensores de consulta y clave antes del cálculo de puntuaciones. El producto interno $\mathbf{q}_i^\top \mathbf{k}_j$ se convierte en función del desplazamiento relativo $i - j$.

2. **Sesgo de puntuación** (`alibi`): añade un sesgo proporcional a la distancia a la matriz de puntuaciones de atención *después* de que se calcule.

3. **Aumentación de Q/K** (`pape`, `pape_efficient`, `pape_ri`): concatena dimensiones posicionales adicionales a los vectores de consulta y clave antes del producto punto, aumentando la dimensión efectiva de la cabeza.

4. **Aditivo a nivel de embedding** (`sinusoidal_absolute`, `learned_absolute`): se añade directamente a los embeddings de tokens antes de la primera capa; ninguna acción por mezclador se realiza en el bloque de atención.

`nope` y `none` son ambos operaciones nulas: `nope` desactiva explícitamente la codificación posicional (el modelo debe aprender la posición implícitamente), mientras que `none` es el módulo nulo devuelto por la fábrica cuando no se desea ninguna PE.

14.3. Formulaciones Matemáticas

14.3.1. RoPE — Codificación Posicional Rotatoria

RoPE [107] rota la consulta y clave de cada cabeza un ángulo proporcional al índice del token. Para dimensión de cabeza d_h y posición de token m, la matriz de rotación $\mathbf{R}_m \in \mathbb{R}^{d_h \times d_h}$ actúa sobre sub-espacios 2-D (x_{2i}, x_{2i+1}):

$$\mathbf{R}_m = \operatorname{diag}\left(\begin{bmatrix} \cos m\theta_0 & -\sin m\theta_0 \\ \sin m\theta_0 & \cos m\theta_0 \end{bmatrix}, \ldots, \begin{bmatrix} \cos m\theta_{d_h/2-1} & -\sin m\theta_{d_h/2-1} \\ \sin m\theta_{d_h/2-1} & \cos m\theta_{d_h/2-1} \end{bmatrix} \right),$$

con frecuencias $\theta_i = \texttt{rope_base}^{-2i/d_h}$. La puntuación de atención se convierte en $\mathbf{q}_m^\top \mathbf{R}_{m-n} \mathbf{k}_n$, función del desplazamiento relativo $m-n$. Un factor `rope_scaling` s reescala la posición efectiva a m/s para modos de extrapolación de longitud.

14.3.2. HoPE — Codificación Posicional Rotatoria Hiperbólica

HoPE [23] generaliza RoPE a la geometría hiperbólica del modelo de Lorentz. En lugar de rotaciones euclidianas, HoPE aplica rotaciones de Lorentz construidas a partir de funciones hiperbólicas, imponiendo un *decaimiento monótono* de los pesos de atención con el aumento de la distancia entre tokens — una propiedad que RoPE carece porque su patrón de atención oscilatorio puede re-surgir en rangos largos. RoPE se recupera como caso especial. Las perillas `hope_base` y `hope_damping` controlan la frecuencia hiperbólica y la tasa de decaimiento.

14.3.3. NoPE — Sin Codificación Posicional

NoPE [59] es la ausencia explícita de cualquier inyección posicional. El modelo debe descubrir la posición implícitamente a partir de la máscara causal (decodificador) o de los datos mismos. El trabajo citado muestra que NoPE supera a los métodos explícitos de PE (incluyendo RoPE y ALiBi) en benchmarks de generalización de longitud sin requerir cómputo adicional, porque un decodificador con atención causal puede codificar implícitamente la posición relativa a través de sus patrones de atención.

14.3.4. ALiBi — Atención con Sesgos Lineales

ALiBi [93] no modifica los embeddings ni los vectores de consulta/clave. En su lugar añade un sesgo lineal estático específico por cabeza a la matriz de puntuaciones de atención:

$$\text{puntuación}_{i,j}^{(h)} = \frac{\mathbf{q}_i^{(h)\top}\mathbf{k}_j^{(h)}}{\sqrt{d_h}} - m_h \cdot (i-j), \qquad m_h = \frac{1}{2^{\lfloor h/\lceil H/8 \rceil \rfloor}},$$

donde m_h es una pendiente geométrica por cabeza h y el sesgo penaliza las claves distantes. La perilla `alibi_num_heads` controla el número de cabezas del calendario de pendientes. ALiBi habilita la extrapolación de longitud: un modelo entrenado en 1024 tokens generaliza a 2048 sin reentrenamiento.

14.3.5. BAM — Mecanismo de Atención Bayesiana

BAM [13] reformula la codificación posicional como un prior probabilístico sobre posiciones y unifica métodos existentes: NoPE corresponde a un prior Uniforme, ALiBi a un prior Uniforme×Laplace. La instanciación práctica, GGD-BAM, añade una matriz de sesgo de posición relativa con Gaussiana Generalizada $\mathbf{B}$ a los logits de atención, exactamente como ALiBi pero con una forma no lineal y aprendible:

$$B_{i,j}^{(h)} = -\big(|j - \alpha_h\, i - \mu_h| + \varepsilon\big)^{\beta_h}, \qquad \alpha_h = e^{-\theta_\beta^{(h)}}\,\theta_\alpha^{(h)} \geq 0,$$

con parámetros aprendibles por cabeza $\theta_\alpha, \theta_\beta$ (init `bam_theta_init`=0, el prior Uniforme) y localización opcional θ_μ (fijado a 0 por defecto vía `bam_learn_mu=false`). El piso ε (`bam_eps`, por defecto 10^{-5}) estabiliza la exponenciación cuando $\beta < 0$, un régimen relajado que suprime el contexto local y actúa como "cabeza de recuperación" de largo alcance. Casos especiales: $\beta = 1, \mu = 0$ recupera ALiBi; $\beta = 2$ da un prior Normal; $0 < \beta < 1$ produce colas más pesadas que Laplace. El paper aparea BAM con Scalable Softmax (SSMax), un reescalado transversal de logits $s \cdot \ln(n)$ (con s aprendible por cabeza) que contrarresta el "attention fading" y es componible con cualquier PE vía la bandera `use_ssmax` de nivel superior. BAM SSMax logra recuperación precisa de información a 500× la longitud de contexto de entrenamiento.

14.3.6. PaPE — Codificación Posicional Parabólica

PaPE [86] es una codificación centrada en visión que aumenta la consulta y clave con dimensiones posicionales extra construidas a partir de funciones base parabólicas. Dados P parábolas (`pape_num_parabolas`) y S posiciones (`pape_num_positions`), la característica posicional para la coordenada $\mathbf{p} \in \mathbb{R}^D$ es

$$\phi_k(\mathbf{p}) = \sum_{j=1}^{S} \alpha_{k,j}\, f_j(\mathbf{p}), \qquad f_j(\mathbf{p}) = -\|\mathbf{p} - \mathbf{c}_j\|^2 + b_j,$$

una suma de S paraboloides centrados en $\mathbf{c}_j$ con desplazamiento b_j. Las $P \cdot S$ dimensiones extra se concatenan a la consulta y clave antes del producto punto, por lo que la dimensión efectiva de la cabeza crece de d_h a $d_h + P \cdot S$. Se proveen tres variantes:

- `pape` — la implementación de referencia.
- `pape_efficient` — una reformulación en PyTorch puro que evita kernels Triton personalizados preservando la misma matemática.
- `pape_ri` — la variante invariante a rotación (PaPE-RI en el paper), que satisface invariancia rotacional por construcción y se controla con `pape_rotation_invariant`.

PaPE está diseñado para tokens de visión n-D (imágenes, video, cámaras de eventos, nubes de puntos) y es la PE recomendada para la ruta ViT.

14.3.7. Sinusoidal Absoluta

La codificación absoluta original del Transformer [113] añade un vector sinusoidal fijo al embedding del token:

$$\mathrm{PE}_{(m,2i)} = \sin\!\left(\frac{m}{\texttt{sinusoidal_base}^{2i/d}}\right), \quad \mathrm{PE}_{(m,2i+1)} = \cos\!\left(\frac{m}{\texttt{sinusoidal_base}^{2i/d}}\right),$$

con `sinusoidal_max_len` limitando la tabla precomputada y `sinusoidal_scale` multiplicando el embedding. Esta es una PE aditiva: se aplica una vez a nivel de embedding y los mezcladores no aplican ninguna PE por capa.

14.3.8. Sinusoidal Rotatoria

Una variante rotatoria de la codificación sinusoidal: las mismas frecuencias sinusoidales se usan para construir una matriz de rotación aplicada a la consulta y clave (como en RoPE), pero con las perillas `sinusoidal_base` y `sinusoidal_scale`. Esta es una PE de estilo rotacional aplicada por capa mediante `apply_pe_to_qk`.

14.3.9. Absoluta Aprendida

Una codificación absoluta aprendible: una matriz de parámetros de `learned_max_len` $\times\, d$ se añade a los embeddings de tokens a nivel de embedding. La desviación estándar de inicialización se controla con `learned_init_std`. Al igual que la sinusoidal absoluta, los mezcladores no realizan ninguna acción por capa.

14.4. Banderas use_pe Por Mezclador

Cada mezclador expone un campo de configuración <mezclador>_attn_use_pe (p. ej. standard_attn_use_pe, titan_attn_use_pe). Los valores por defecto son:

- **True** (mezcladores de atención): standard_attn, titan_attn, sigmoid_attn, gated_softmax_attn, gqa_attn, todos los mezcladores latentes (mla_attn, gqla_attn, mlra_attn, tucker_attn, iha_attn, gta_attn, cca_attn, ccgqa_attn, gma_attn), todos los mezcladores dispersos (longformer_attn, bigbird_attn, sparse_transformer_attn, sparsek_attn, nsa_attn, msa_attn, sparda_attn, fasa_attn, sparge_attn), y engram_attn.
- **False** (mezcladores recurrentes/de decaimiento): retnet, retnet_attn, mamba, ode, gla_attn, deltanet_attn, gated_deltanet_attn, gated_deltanet2_attn, hgrn2_attn, fox_attn, kda_attn, mtla_attn.

El valor por defecto False para mezcladores recurrentes/de decaimiento refleja que estas arquitecturas incorporan su propio sesgo inductivo temporal/posicional en su recurrencia (p. ej. la rotación compleja de RetNet, el barrido selectivo de Mamba). Sobrescribir use_pe=True está permitido pero es típicamente redundante.

14.5. Unificación del Transformer de Visión

La ruta ViT (FrankensteinViT) usaba previamente su propio campo pos_embedding_type con valores learned_1d, none. Esto se ha unificado con el enum de todo el modelo: learned_1d se migra a learned_absolute, y none se mapea a none. El ViT construye el mismo módulo build_pos_encoder(config) y lo inyecta en sus HybridLayers, por lo que un ViT ahora puede usar RoPE, HoPE, ALiBi, PaPE, o cualquier otra PE del enum.

14.6. Compatibilidad con Legado

La bandera booleana legada use_hope está deprecada y mapeada al nuevo enum: use_hope=True → positional_encoding=hope, use_hope=False → positional_encoding=rope. La sobrescritura positional_encoding por mezclador que previamente existía solo en titan_attn es ahora una operación nula: el enum de todo el modelo es la única fuente de verdad, y la lista de permitidos de TitanAttention que restringía PE a {hope, rope, none, nope} se ha eliminado — los 12 valores ahora funcionan con cada mezclador.

14.7. Perfil Arquitectónico: Codificaciones Posicionales

La Tabla 18 resume las 12 codificaciones posicionales soportadas.

Nombre	Estilo	Idea Clave	Parámetros	Referencia
rope	Rotación	Rotación basada en frecuencia de Q/K; posición relativa mediante geometría de producto interno	rope_base, rope_scaling	[107]
hope	Rotación	Rotación de Lorentz hiperbólica; decaimiento monótono de atención con distancia	hope_base, hope_damping	[23]

continúa en la siguiente página

Nombre	Estilo	Idea Clave	Parámetros	Referencia
nope	Ninguno	Sin PE explícita; el modelo aprende posición implícitamente de la máscara causal	—	[59]
alibi	Sesgo de puntuación	Penalización lineal estática de distancia añadida a puntuaciones de atención; extrapolación de longitud	`alibi_num_heads`	[93]
bam	Sesgo de puntuación	Sesgo de posición relativa con Gaussiana Generalizada; forma (β) y escala (α) aprendibles por cabeza; recupera ALiBi en $\beta = 1$; reescalado SSMax vía `use_ssmax`	`bam_theta_init`, `bam_learn_mu`, `bam_eps`, `use_ssmax`, `ssmax_s_init`	[13]
pape	Aum. Q/K	Funciones base parabólicas concatenadas a Q/K; centrada en visión, n-D	`pape_num_parabolas`, `pape_num_posiciones`	[86]
pape_efficient	Aum. Q/K	PaPE en PyTorch puro; sin kernels Triton personalizados	igual que pape	[86]
pape_ri	Aum. Q/K	PaPE invariante a rotación (PaPE-RI)	`pape_rotation_invariant`	[86]
sinusoidal_absolute	Aditivo	Sinusoidal fijo añadido a embeddings (Transformer original)	`sinusoidal_base`, `sinusoidal_max_len`, `sinusoidal_scale`	[113]
sinusoidal_rotary	Rotación	Frecuencias sinusoidales como matriz de rotación en Q/K	igual que sinusoidal	[113]
learned_absolute	Aditivo	Matriz de parámetros aprendible añadida a embeddings	`learned_max_len`, `learned_init_std`	[113]
none	Ninguno	Módulo nulo; ninguna información posicional inyectada	—	—

Cuadro 18: Catálogo de codificaciones posicionales. "Estilo" es el mecanismo de inyección (rotación, sesgo de puntuación, aumentación de Q/K, aditivo). "Parámetros" lista las perillas principales de configuración.

15. SSOG: Atención Separable Sum of Gaussians

Este anexo cubre SSOG (Separable Sum of Gaussians), un mixer de atención de campo geométrico. SSOG [91] reemplaza la interacción puntuada por contenido de la atención de producto escalado por una mezcla aprendida de Gaussianas sobre *posición relativa*: cada cabeza posee un puñado de átomos Gaussianos—cinco números por átomo, un desplazamiento de centro (μ_y, μ_x), un ancho por eje (σ_y, σ_x) y un peso de mezcla λ—que forman un campo fijo sobre el desplazamiento entre tokens. No hay productos punto consulta–clave en ninguna parte: el contenido nunca puntúa al contenido, solo *dirige* el campo mediante residuos pequeños y acotados.

Dado que una Gaussiana 2D se factoriza en un producto de dos Gaussianas 1D, aplicar el campo son dos pasadas de filtro 1D por átomo (filas y luego columnas de un raster de tokens de $\sqrt{N} \times \sqrt{N}$) seguidas de una contracción de mezcla λ. La matriz de atención $N \times N$ inducida nunca existe, por lo que el costo específico de atención es $\mathcal{O}(R \cdot N\sqrt{N} \cdot d)$ para R átomos en lugar de $\mathcal{O}(N^2 \cdot d)$. SSOG también omite por completo las proyecciones de consulta y clave,

pagando $2d^2$ de proyecciones por capa en lugar de los $4d^2$ de SDPA. En la receta de referencia, el mecanismo completo supera a su gemelo SDPA igualado en ImageNet-1k (65,28 % vs 64,34 % con ∼3M de parámetros; 72,02 % vs 71,84 % con ∼12M) siendo ∼20 % más pequeño y ∼30 % más barato por pasada, y por +17 puntos en CIFAR-100 con receta igualada.

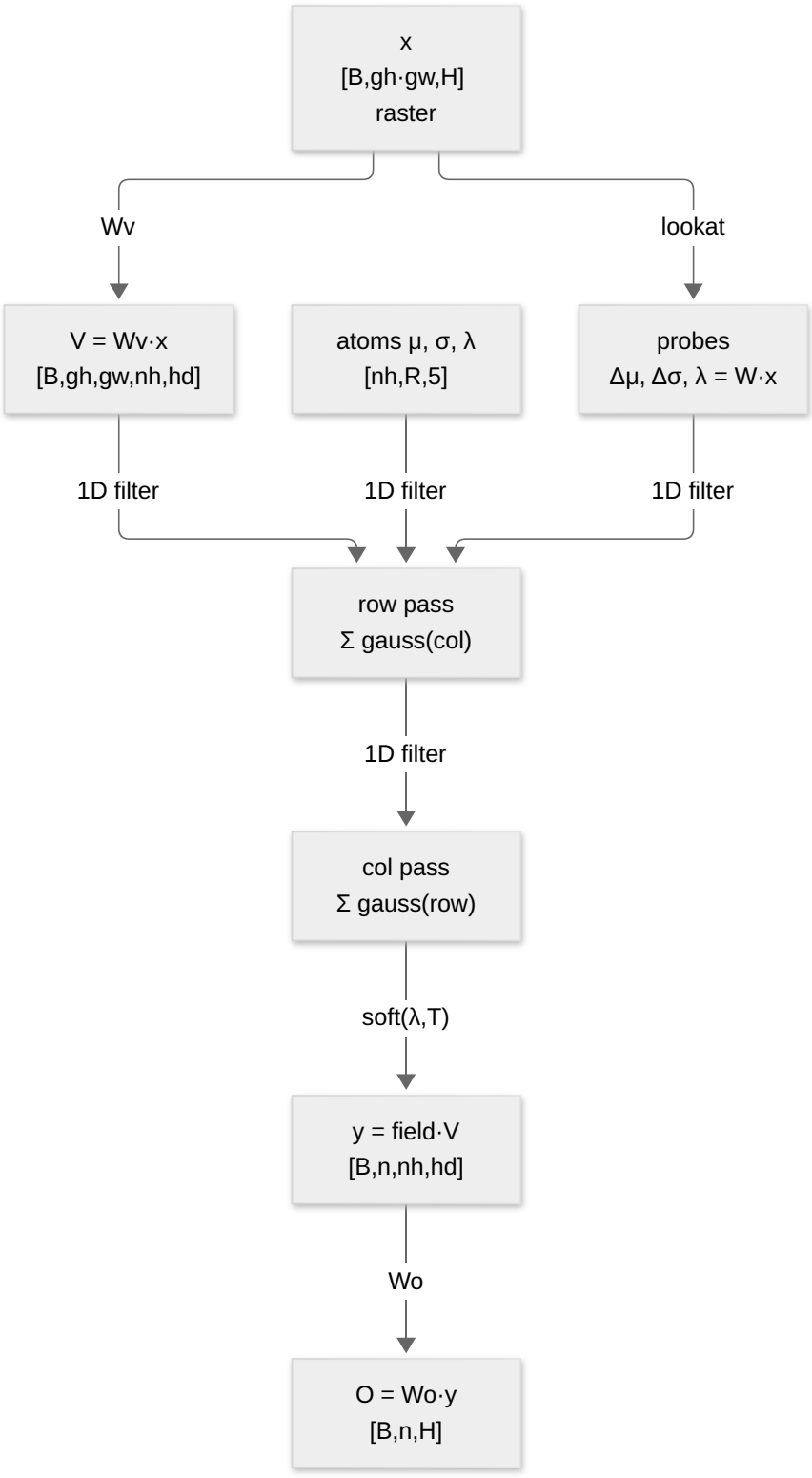


Figura 70: Flujo de datos de SSOG: los valores pasan por pasadas separables de filtrado gaussiano por filas y columnas para cada átomo, con direccionamiento opcional de los parámetros del átomo por contenido.

15.1. Formulación Matemática

Sea el raster row-major de una rejilla $G_h \times G_w$ los tokens. El peso de atención del token p al token q es el log-sum-exp con softmax y temperatura de los log-pesos de los átomos evaluados en su desplazamiento:

$$A(p,q) = \mathrm{softmax}_q\big(\tfrac{1}{\tau}\, s(p,q)\big)\,, \qquad s(p,q) = \mathrm{logsumexp}_r\,\big(\log\lambda_r + \log\mathcal{N}(p-q;\ \mu_r, \sigma_r)\big),$$

donde τ es una temperatura aprendible y $\mathcal{N}(\cdot\,;\mu_r,\sigma_r)$ es una Gaussiana 2D con covarianza diagonal. Como la Gaussiana 2D se factoriza, el campo con softmax se aplica de forma separable: por eje, $a_y[i,j] = \mathrm{softmax}_j(\log\mathcal{N}(\Delta y_{ij};\mu_y,\sigma_y)/\tau)$ y análogamente a_x, seguido de

$$Y = \sum_{r=1}^{R} \lambda_r\,\big(a_x^{(r)}\big(a_y^{(r)}\,V\big)\big), \qquad V = xW_V,$$

es decir, tres contracciones einsum por capa. Con direccionamiento por contenido (`lookat`) habilitado, sondas lineales inicializadas en cero predicen residuos por token acotados sobre los parámetros del campo detrás de compuertas softplus de arranque en frío:

$$\mu \leftarrow \mu_0 + s_\mu \cdot m \cdot \tanh(W_\mu x), \qquad \sigma \leftarrow \sigma_0\, e^{\,s_\sigma \tanh(W_\sigma x)}, \qquad \lambda \leftarrow \mathrm{softmax}\,\big(\log\lambda_0 + s_\lambda \tanh(W_\lambda x)\big),$$

con m el desplazamiento máximo en celdas de rejilla y escalas de compuerta $s_\bullet = \mathrm{softplus}(\cdot)$ inicializadas en -8 ($\approx 3\times10^{-4}$), de modo que el modelo comienza como un campo geométrico congelado y aprende cuánto abrir cada toma de contenido. Los anchos se reparametrizan como $\sigma = \mathrm{softplus}(\omega) + \epsilon + \sigma_{\text{piso}}$ para mantenerse estrictamente positivos.

15.2. Pseudocódigo Algorítmico

Entrada: estado oculto $x \in \mathbb{R}^{B\times N\times d}$ como raster $G_h \times G_w$, átomos $\{\mu_r, \omega_r, \log\lambda_r\}_{r=1}^{R}$ por cabeza, temperatura τ
$V \leftarrow xW_V$ reorganizado a (B, G_h, G_w, H, d_h) ▷ sin proyecciones Q/K
$\sigma_r \leftarrow \mathrm{softplus}(\omega_r) + \epsilon + \sigma_{\text{piso}}$ ▷ anchos positivos
if direccionamiento (lookat) **then**
    $\mu, \sigma, \lambda \leftarrow$ residuos por token acotados vía sondas init-cero y compuertas ▷ el contenido dirige
end if
$a_y \leftarrow \mathrm{softmax}\big(\log\mathcal{N}(\Delta y;\mu_y,\sigma_y)/\tau\big)$, $a_x \leftarrow$ análogamente ▷ kernels 1D, uno por átomo
$Y \leftarrow \sum_r \lambda_r\, a_x^{(r)}\big(a_y^{(r)} V\big)$ ▷ pasada filas, pasada columnas, mezcla λ: sin $N \times N$
Salida: $\mathbf{Y}W_O$ reorganizado a (B, N, d)

15.3. Características Clave

- **Complejidad de entrenamiento**: $\mathcal{O}(R \cdot N\sqrt{N} \cdot d)$ para R átomos fijos—casi lineal solo por geometría, no por aproximación de kernels.
- **Complejidad de inferencia**: mismo costo por pasada; solo encoder (el campo separable no tiene formulación causal), por lo que no hay semánticas de caché KV.
- **Entrenable**: los átomos $\{\mu, \omega, \log\lambda\}$, la temperatura y las sondas de direccionamiento son completamente diferenciables; las compuertas de arranque en frío se abren durante la optimización.
- **Fortalezas**: prior geométrico fuerte (+17 pts sobre SDPA en datos pequeños con receta igualada; supera a SDPA en ImageNet-1k con 3M y 12M parámetros con ∼20 % menos

parámetros y ∼30 % menos FLOPs); interpretable por diseño (cada cabeza son unos pocos blobs graficables—las capas tempranas se vuelven convoluciones encubiertas, las intermedias detectores de franjas, las tardes se vuelven globales); el campo vive en coordenadas, lo que otorga transferencia de resolución zero-shot.

- **Limitaciones**: solo encoder; requiere un raster row-major fijo $G_h \times G_w$ (ViT necesita `cls_token=false`; las secuencias necesitan una rejilla $1 \times L$ explícita); los kernels de direccionamiento por query añaden un término de memoria; los resultados en lenguaje no están probados—allí el puntaje por contenido probablemente es estructural.

16. Atención de Peso Rápido para Aprendizaje Continuo

Este anexo cubre la familia Falcon [132], un marco de atención de peso rápido que trata la transición recurrente del estado de peso rápido como una **regla de aprendizaje continuo en línea** bajo semántica autoregresiva de *lectura tras escritura* (read-after-write, RAW). La observación central del marco es de alineación temporal: para el objetivo de predicción de prefijo, el ejemplo local de memoria rápida revelado en el paso t es el **par prefix-alineado** $(x_t, y_t) = (\phi(k_{t-1}),\, v_t)$ — desplazado un paso respecto a la asociación de mismo paso $(\phi(k_t), v_t)$ usada por DeltaNet y la atención lineal estándar. La asociación de mismo paso sigue siendo causal, pero optimiza un objetivo interno diferente. El marco deriva **actualizaciones de primer orden normalizadas** para dos objetivos (regresión de error cuadrático y producto interno negativo) y tres estructuras de tamaño de paso (escalar, por columna, ventana deslizante), dando seis variantes: Falcon-1/2/3 (familia de regresión) y Falcon-1A/2A/3A (familia de producto interno). Las seis variantes están implementadas en el código bajo `model.attention.falcon` con un bloque de configuración compartido, cada una proporcionando tanto un escaneo recurrente de referencia como un kernel chunk-parallel estilo SSD.

16.1. Preliminares: Alineación, Normalización, Renormalización

Semántica de lectura tras escritura. Con mapas de características $\phi(k_t)$ (QK-RMSNorm por defecto), el flujo prefix-alineado fija $x_t = \phi(k_{t-1})$ para $t \geq 2$ con centinela de frontera $x_1 = 0$; la lectura es $o_t = S_t^\top \phi(q_t)$. La escritura en el paso t por tanto actualiza la memoria *antes* de que la consulta del paso $t+1$ la lea, de modo que ninguna clave se asocia jamás con su propio token.

Tamaños de paso normalizados. Un paso de gradiente sobre el objetivo local $\ell_t(S)$ con el denominador acoplado al objetivo da

$$\eta_t = \frac{\beta_t}{\text{estadístico}_t + \lambda_t + \varepsilon}, \qquad \beta_t \in (0, 2),$$

donde el estadístico es la energía cuadrática de la clave $\|x_t\|^2$ (Falcon-1/1A/2/2A), el estadístico espectral de ventana $\mu_t^{(B)} = \lambda_{\text{máx}}(X^\top X)/B_t$ (Falcon-3), o la energía media de ventana $\bar{E}_t^{(B)}$ (Falcon-3A). La normalización acoplada al objetivo garantiza descenso por paso y elimina la sensibilidad a la escala de las tasas de aprendizaje fijas. La ganancia de **plasticidad** β_t, el ridge de **olvido** λ_t, la **alineación** temporal y la ventana de repetición B quedan así separados como cuatro perillas previamente entrelazadas.

Renormalización de decaimiento positivo. Cuando $\lambda_t > 0$, la contracción por paso $\alpha_t = \text{mín}(\eta_t \lambda_t,\, 1 - \varepsilon_\gamma)$ debe aplicarse de forma estable. La implementación fija α_t estrictamente por debajo de 1, calcula prefijos logarítmicos chunk-locales en fp32 vía $\log(1 - \alpha_t)$ y sumas acumuladas, reescala los generadores al dominio sin decaimiento ($\tilde{\eta}_t = \eta_t/\gamma_t$, $\tilde{v}_t = v_t/c_{t-1}$ con $c_{t-1} = \prod_{r \leq t-1} \gamma_r$) y reescala salidas y el estado emitido por el δ acumulado. Los centinelas de frontera son $(\alpha_1, \gamma_1) = (0, 1)$ y $\eta_1 = 0$, de modo que el primer paso es una lectura pura.

16.2. La Familia de Regresión (Falcon-1/2/3)

El objetivo de error cuadrático sobre el par desplazado es

$$\ell_t(S) = \tfrac{1}{2}\|v_t - S^\top x_t\|^2 + \tfrac{\lambda_t}{2}\|S\|_F^2, \qquad \nabla_S \ell_t = x_t(S^\top x_t - v_t)^\top + \lambda_t S,$$

que es L_t-suave con $L_t = \|x_t\|^2 + \lambda_t$.

16.2.1. Falcon-1: Regresión NLMS Escalar

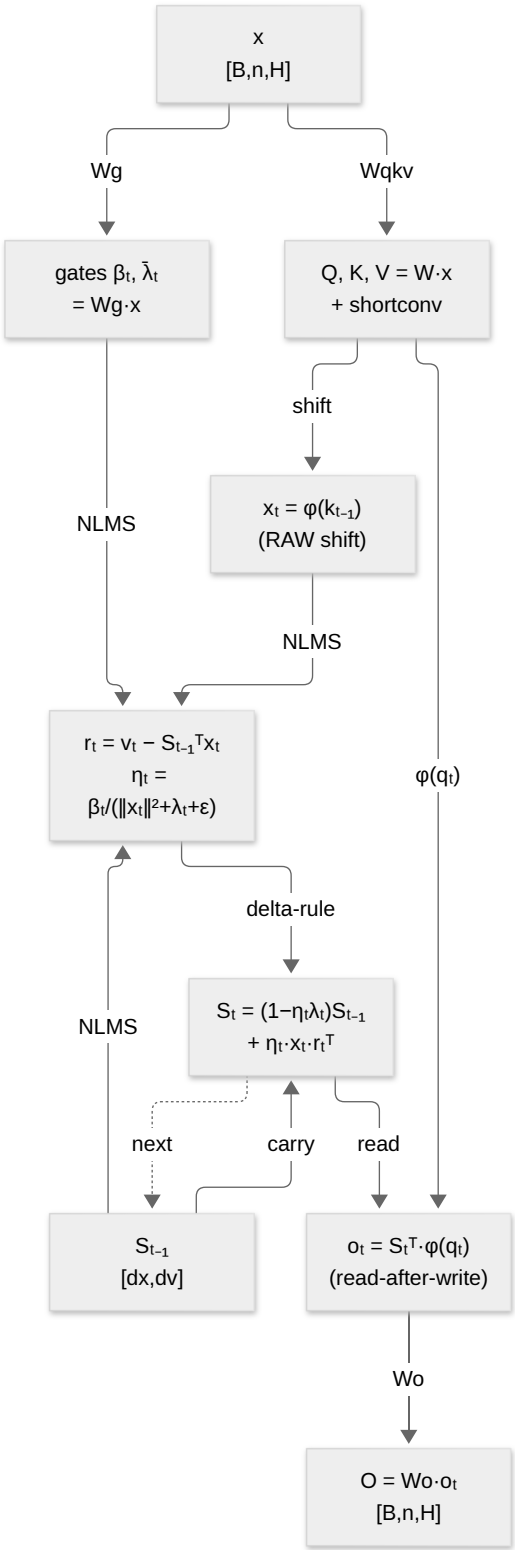


Figura 71: Flujo de datos de Falcon-1: el par clave/valor alineado al prefijo impulsa una actualización escalar NLMS por regla delta del estado de pesos rápidos.

La variante escalar de la regla delta — la recursión NLMS clásica del filtrado adaptativo, mejorada con el flujo desplazado y un ridge explícito:

$$S_t = (1 - \eta_t \lambda_t)\, S_{t-1} + \eta_t\, x_t \big(v_t - S_{t-1}^\top x_t\big)^\top, \qquad o_t = S_t^\top \phi(q_t).$$

Con $\varepsilon = 0$, $\lambda_t = 0$ y la asignación sin desplazar $x_t = \phi(k_t)$ esto recupera DeltaNet exactamente. La eigenestructura de la transición $A_t = \gamma_t I - \eta_t x_t x_t^\top$ (con $\gamma_t = 1 - \eta_t \lambda_t$) muestra que la dirección de escritura se contrae por $1 - \beta_t$ mientras el subespacio ortogonal lleva γ_t: $\beta_t > 1$ produce un cambio de signo estable a lo largo de la dirección de escritura, útil para seguimiento de estado. La forma chunk-parallel usa la representación WY $\prod_s (I - \eta_s x_s x_s^\top) = I - UTU^\top$ con una única solución triangular unitaria inferior de residuo simple $L = \mathrm{tril}(G, -1) + I$ donde $G = X^\top X$ (Algoritmo 8 del artículo), tras la renormalización de decaimiento positivo.

16.2.2. Falcon-2: Regresión NLMS por Columnas

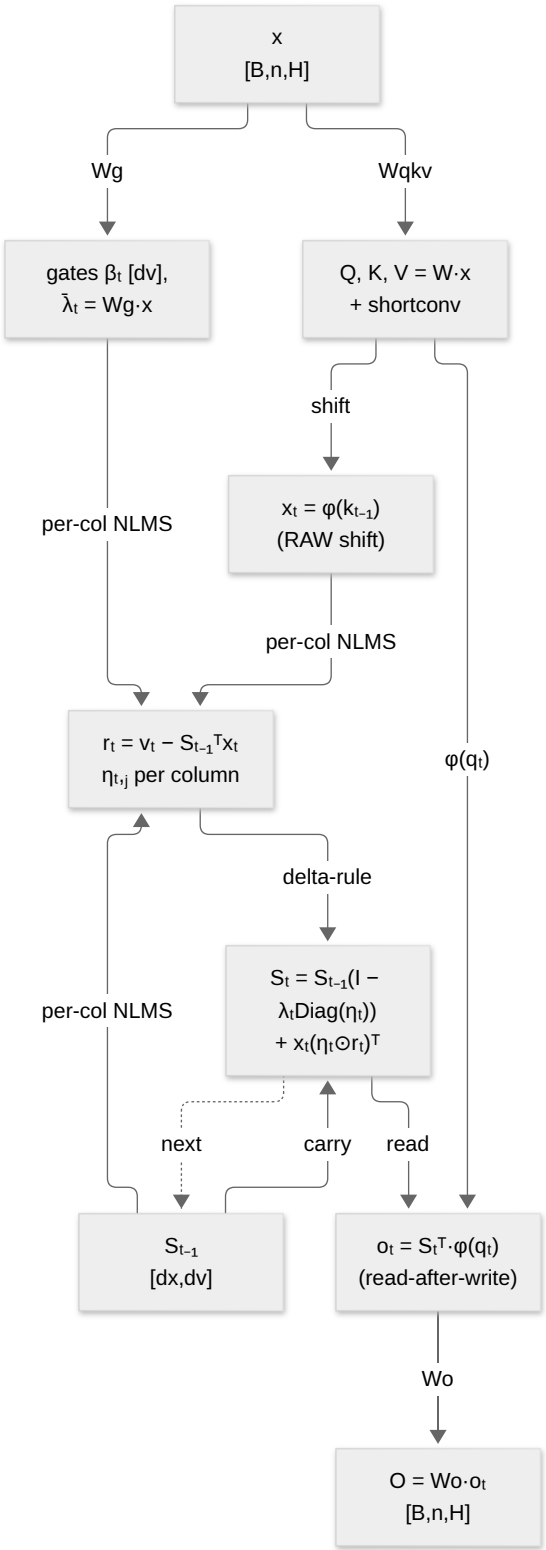


Figura 72: Flujo de datos de Falcon-2: tamaños de paso NLMS por columna actualizan el estado de pesos rápidos canal a canal.

La pérdida ridge se descompone a lo largo de las coordenadas de valor, de modo que cada canal de valor j recibe su propio tamaño de paso compartiendo el normalizador NLMS:

$$\eta_{j,t} = \frac{\beta_{j,t}}{\|x_t\|^2 + \lambda_t + \varepsilon}, \qquad S_t = S_{t-1}\big(I - \lambda_t \operatorname{Diag}(\boldsymbol{\eta}_t)\big) + x_t\big(\boldsymbol{\eta}_t \odot \mathbf{r}_t\big)^\top.$$

El kernel chunk-parallel forma el Gram de claves compartido $G = X^\top X$ y resuelve el lote de sistemas triangulares unitarios inferiores por canal $L_j = I + \operatorname{tril}\big(G \odot (\Sigma_j \Sigma_j^\top), -1\big)$ con $\Sigma = \sqrt{\tilde{\boldsymbol{\eta}}}$ (Algoritmo 1). Definida pero no evaluada por separado en el artículo.

16.2.3. Falcon-3: Regresión de Minilote con Ventana Deslizante

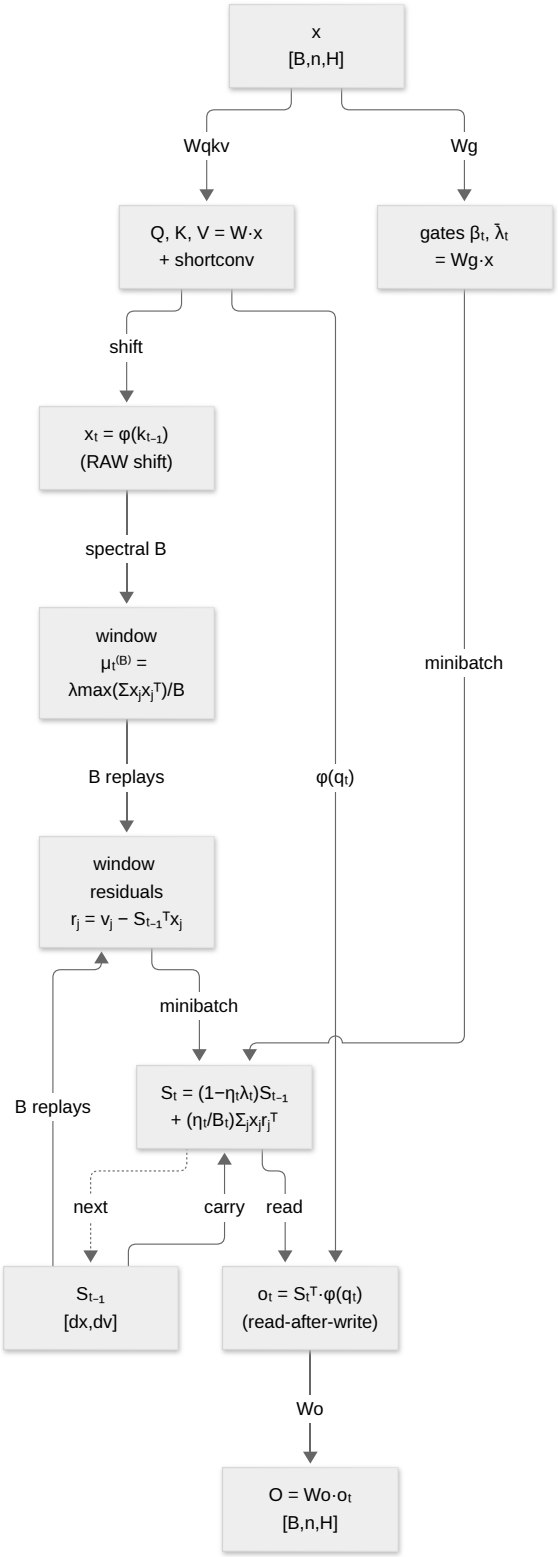


Figura 73: Flujo de datos de Falcon-3: una ventana deslizante de pares alineados al prefijo aporta el estadístico espectral y la escritura por minilote.

Repetición acotada (bounded rehearsal): cada paso regresa la memoria rápida sobre los últimos $B_t \le B$ pares prefix-alineados, con *todos los residuos de ventana evaluados en el estado pre-actualización* S_{t-1}:

$$\mu_t = \frac{\lambda_{\max}\big(X_{\mathcal{I}_t}^\top X_{\mathcal{I}_t}\big)}{B_t}, \qquad \eta_t = \frac{\beta_t}{\mu_t + \lambda_t + \varepsilon}, \qquad S_t = (1 - \eta_t\lambda_t)\, S_{t-1} + \frac{\eta_t}{B_t} \sum_{j \in \mathcal{I}_t} x_j \big(v_j - S_{t-1}^\top x_j\big)^\top,$$

con $\mathcal{I}_t = \{\max(2, t - B + 1), \ldots, t\}$. El promediado por ventana mantiene la magnitud de la actualización invariante a B: cada par se repite en exactamente B actualizaciones consecutivas con peso $1/B$, por lo que su inyección acumulada es independiente de B. El estado por sí solo *no* es Markov: la continuación exacta requiere una cola FIFO de los últimos $B - 1$ pares, y los kernels chunk recogen una porción extendida con $|\mathcal{J}| \le C + B - 1$. La forma chunk-parallel es una recurrencia afín de rango B resuelta por sustitución hacia adelante por bloques sobre el espacio de índices extendido (tiempo $\times$ rango), con componentes de rango del mismo tiempo desacoplados (Algoritmo 3).

16.3. La Familia de Producto Interno (Falcon-1A/2A/3A)

El objetivo de producto interno negativo

$$\ell_t^{\mathrm{ip}}(S) = -\langle S^\top x_t,\, v_t\rangle + \tfrac{\lambda_t}{2}\|S\|_F^2 \qquad \Rightarrow \qquad \nabla_S \ell_t^{\mathrm{ip}} = -x_t v_t^\top + \lambda_t S$$

produce escrituras puramente aditivas (hebbianas) con ganancias normalizadas por energía.

16.3.1. Falcon-1A y Falcon-2A

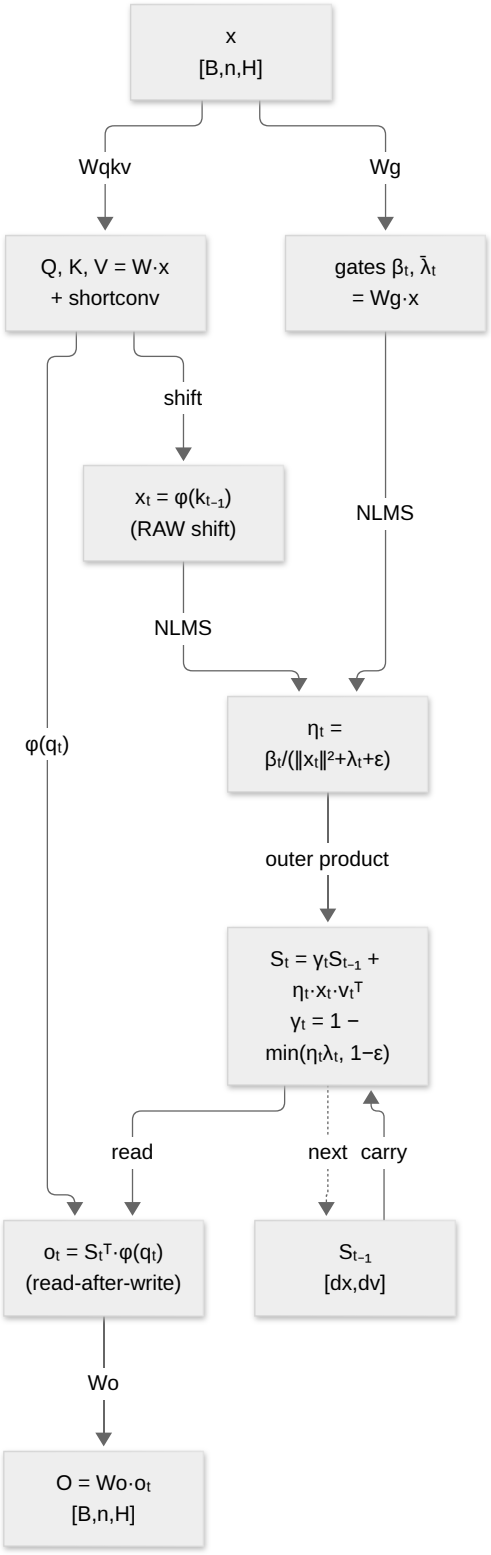


Figura 74: Flujo de datos de Falcon-1A: una escritura aditiva (hebbiana) normalizada por energía con decaimiento derivado.

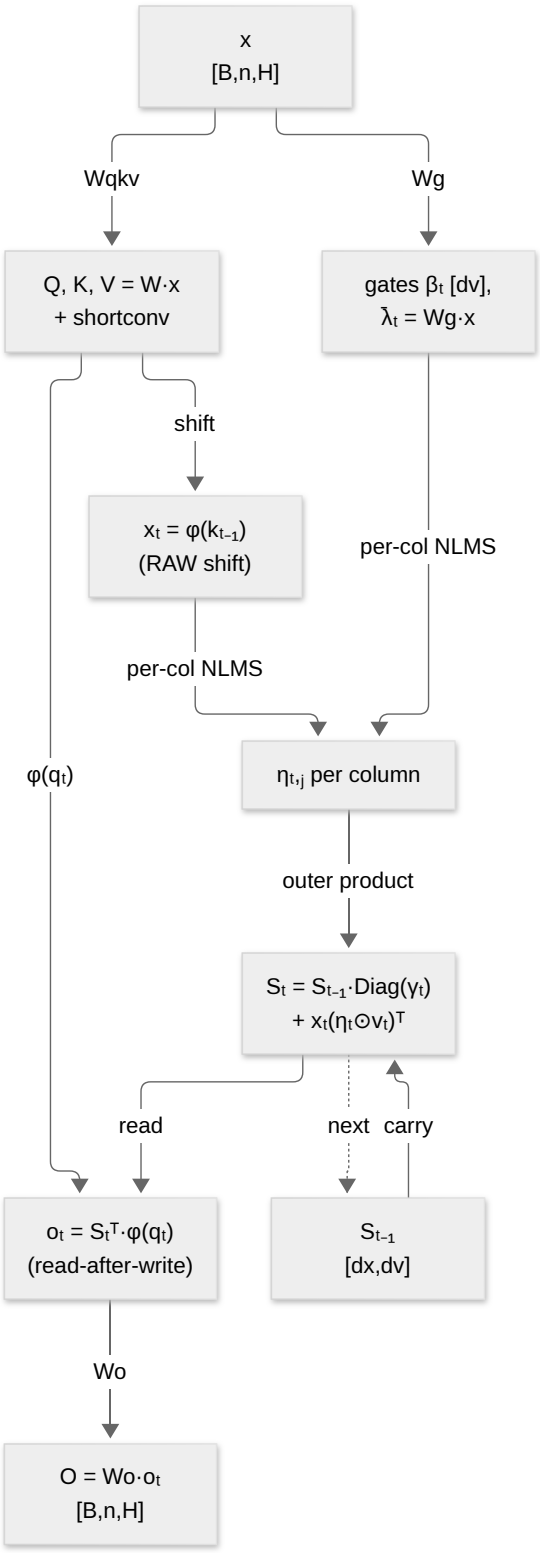


Figura 75: Flujo de datos de Falcon-2A: escrituras aditivas por columna con decaimiento por canal.

$$\text{Falcon-1A:}\quad S_t = \gamma_t S_{t-1} + \eta_t\, x_t v_t^\top, \quad \gamma_t = 1 - \min(\eta_t \lambda_t,\, 1 - \varepsilon_\gamma);$$
$$\text{Falcon-2A:}\quad S_t = S_{t-1}\,\mathrm{Diag}(\boldsymbol{\gamma}_t) + x_t(\boldsymbol{\eta}_t \odot v_t)^\top.$$

Con $\lambda_t = 0$ y la asignación sin desplazar esto es exactamente la atención lineal estándar / acumulación Mamba-2; el flujo desplazado lo convierte en la variante next-latent desplazada un paso. El denominador de la familia A no lo exige la curvatura (el objetivo es lineal) — conserva la escritura normalizada por energía como estabilizador de magnitud. La forma chunk-parallel es atención lineal con máscara de decaimiento $M_{t,i} = \eta_i \prod_{r=i+1}^{t} \gamma_r$ (máscaras por canal para Falcon-2A).

16.3.2. Falcon-3A: Producto Interno con Ventana Deslizante

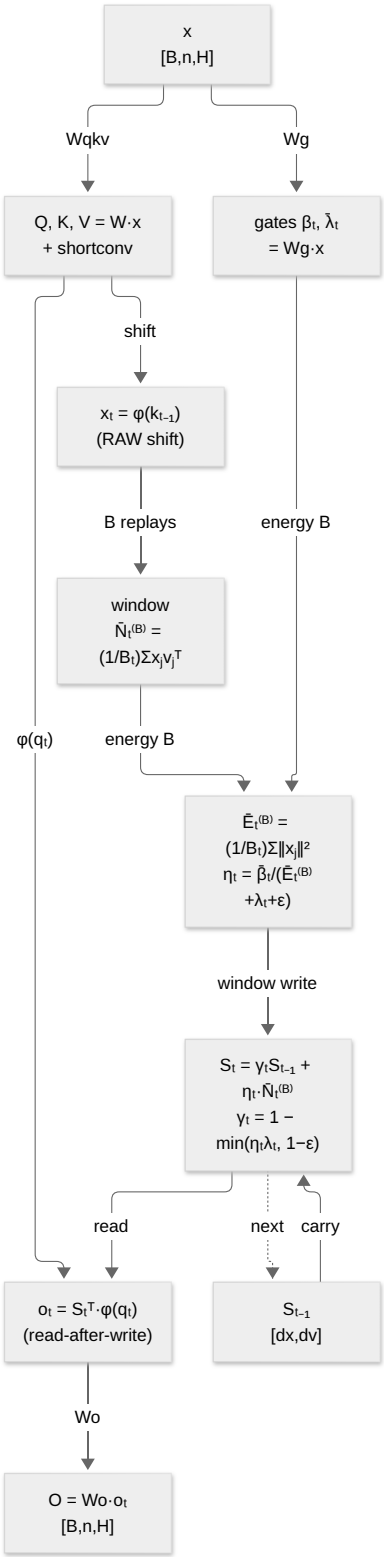


Figura 76: Flujo de datos de Falcon-3A: un estadístico de energía media en ventana deslizante normaliza la escritura aditiva de la ventana.

La variante aditiva con ventana — la mejor configuración de extrapolación de longitud de la familia:

$$\bar{N}_t^{(B)} = \frac{1}{B_t}\sum_{j\in\mathcal{I}_t} x_j v_j^\top, \quad \bar{E}_t^{(B)} = \frac{1}{B_t}\sum_{j\in\mathcal{I}_t} \|x_j\|^2, \quad \eta_t = \frac{\beta_t}{\bar{E}_t^{(B)} + \lambda_t + \varepsilon}, \qquad S_t = \gamma_t S_{t-1} + \eta_t \bar{N}_t^{(B)}.$$

La máscara de decaimiento con banda de ventana factoriza como $M = D \cdot \mathrm{Diag}(\boldsymbol{\eta}) \cdot A$, donde A es el operador de ventana de banda B y D el kernel de decaimiento causal, evaluada sobre la porción extendida (Algoritmo 4).

16.4. Interfaz de Configuración

Las seis variantes comparten un bloque anidado, `model.attention.falcon`, seleccionado mediante las entradas de `layer_pattern` `falcon1_attn`, `falcon2_attn`, `falcon3_attn`, `falcon1a_attn`, `falcon2a_attn`, `falcon3a_attn`:

Clave	Por defecto	Descripción
`chunk_size`	64	Tamaño de chunk de los kernels chunk-parallel; `null` selecciona el escaneo recurrente de referencia
`qk_norm`	`rms_norm`	Normalización QK (`rms_norm` o `l2_norm`); RMSNorm calculado en fp32
`beta_mode`	`ctx_eta`	Parametrización de la compuerta de plasticidad: `static`, `ctx_beta` (ganancia condicionada al contenido) o `ctx_eta` (paso normalizado condicionado al contenido)
`lambda_mode`	`ctx`	Compuerta de olvido: `static` o `ctx` (acoplada en escala con el estadístico de curvatura desacoplado)
`beta` / `lambda`	1,0 / 0,0	Ganancia y ridge estáticas (usadas cuando el modo correspondiente es `static`)
`window`	4	Ventana deslizante B (solo Falcon-3 / Falcon-3A)
`short_conv` / `conv_kernel`	`true` / 4	Convoluciones cortas causales en profundidad sobre las proyecciones q/k/v
`eps` / `eps_gamma`	10^{-6} / 10^{-4}	Estabilizador del tamaño de paso y piso de fijación del decaimiento positivo

Cuadro 19: Perillas de configuración compartidas de Falcon (`model.attention.falcon`). Cada variante expone además un opt-out de codificación posicional por mezclador `falcon{1,2,3,1a,2a,3a}_attn_use_pe`, por defecto `False` como los demás mezcladores recurrentes/de decaimiento.

Notas de implementación: los kernels recurrente y chunk-parallel están verificados numéricamente equivalentes (chunk $\equiv$ recurrente a $\leq 3{,}6 \times 10^{-7}$ en las seis variantes, incluidas las rutas con decaimiento, multi-chunk y ventana); los gradientes fluyen a través de las islas de solución triangular en fp32 (los generadores de características toman solo $\sqrt{\tilde{\eta}}$ mientras los objetivos de valor toman $\sqrt{\tilde{\eta}}\, e^{-u_{t-1}}$, con la raíz cuadrada fijada en 10^{-12} para retropropagación sin NaN en la frontera $\eta = 0$); los exponentes positivos se fijan en 80.

16.5. Resultados Publicados

Modelo (124M, 49.2B tokens FineWeb-Edu)	WikiText ppl	LMB. ppl	FineEdu ppl	Prom. 0-shot
Transformer (RoPE)	33.25	47.43	17.38	48.16
Mamba-2	34.53	48.74	17.70	48.80
DeltaNet	34.19	52.84	17.84	48.88
Gated DeltaNet	30.99	46.70	17.32	48.78
Falcon-1A.3	34.02	49.84	17.40	48.95
Falcon-1.3	33.00	48.70	**17.10**	**49.18**

Cuadro 20: Resultados de modelado de lenguaje reportados por Zhang et al. [132] a 124M de parámetros. En extrapolación de longitud para sumas de dígitos variables (33–48 dígitos), Falcon-3A.3 alcanza **87.2 %** de precisión media frente a 65.8 % de un Transformer con RoPE y 85.9 % de Falcon-1A.3: la familia de producto interno lleva la aritmética, mientras que la familia de regresión lleva la perplejidad.

16.6. Relación con Trabajos Previos

- **DeltaNet**: la regla delta es el paso de descenso de gradiente del objetivo de error cuadrático con el emparejamiento de mismo paso; Falcon lo recupera con el cambio crítico de índice más normalización acoplada al objetivo y contracción λ_t explícita.
- **Gated DeltaNet**: añade una compuerta de arrastre aprendida libre; Falcon *deriva* el arrastre $\gamma_t = 1 - \eta_t \lambda_t$ del objetivo local y la parametrización de paso normalizada en lugar de añadir una compuerta independiente, y usa por defecto QK-RMSNorm en lugar de normalización ℓ_2.
- **Atención lineal / Mamba-2**: el numerador aditivo es un paso de gradiente del objetivo de producto interno con la asignación sin desplazar; los kernels chunkwise reutilizan el esquema SSD.
- **TTT / Titans / ATLAS**: actualizaciones de peso rápido como entrenamiento en tiempo de prueba; Falcon-3 es una especialización de ventana deslizante de la vista de objetivo interno con alineación estricta next-latent.
- **Filtrado adaptativo (LMS/NLMS/RLS)**: Falcon importa los principios de estabilidad y normalización de NLMS a la atención de peso rápido bajo causalidad estricta.

16.6.1. Cuándo Usar Cada Variante

1. **Falcon-1**: la mejor perplejidad de modelado de lenguaje de la familia (FineEdu 17.10, superando a Gated DeltaNet 17.32); olvido principista ligado al objetivo local.
2. **Falcon-2**: plasticidad más fina que Falcon-1 al costo de kernels chunk por canal más pesados; sin evaluación en el artículo.
3. **Falcon-3**: horizonte de repetición controlado con estadísticos de ventana exactos; el kernel más complejo.
4. **Falcon-1A**: variante aditiva simple y estable; segunda mejor extrapolación de longitud (85.9 %).
5. **Falcon-2A**: granularidad de olvido por canal; sin evaluación.
6. **Falcon-3A**: la mejor extrapolación de longitud del artículo (87.2 % en sumas de 33–48 dígitos); carry de tareas aritméticas.

17. Caché KV y Decodificación Incremental

La decodificación autorregresiva está dominada por una asimetría estructural: la etapa de *prefill* procesa el prompt completo en una sola pasada cuadrática, mientras que cada paso de decodificación añade un único token y sólo necesita una fila de consulta nueva contra una historia ya calculada. Recomputar la secuencia completa en cada paso convierte la generación en un procedimiento de $\mathcal{O}(S^2)$ por token; mantener la historia por capa la convierte en $\mathcal{O}(S)$ por token con trabajo de tamaño constante por posición nueva. Este anexo documenta el subsistema de caché KV del toolkit (`src/model/cache/`), los contratos de corrección que hacen numéricamente intercambiables la ejecución con y sin caché, las recetas incrementales por familia, y los cuantizadores de escritura y las políticas de expulsión de lectura que se apilan sobre él.

Tres compromisos de diseño enmarcan todo lo anterior. Primero, la caché es *opcional*: sin `inference.kv_cache.enabled` (por defecto), la generación ejecuta el bucle original de recomputación completa de forma bit-idéntica. Segundo, la caché es *central y declarativa*: un único gestor se propaga por el modelo, y un registro de capacidades por mezclador decide qué capas pueden cachear — nunca al revés. Tercero, la caché *nunca es fatal* (decisión D5): una capa cuya formulación de origen no admite una forma incremental exacta se ejecuta sin caché en silencio, de modo que habilitarla sólo puede cambiar la velocidad, no la corrección ni la disponibilidad.

17.1. Motivación y Alcance

Sea S la longitud del prompt y T el número de tokens generados. Sin caché, el paso t introduce $S + t - 1$ tokens por todas las capas y descarta todas las claves y valores intermedios, con un trabajo de atención total

$$\mathcal{O}\left(\sum_{t=0}^{T-1}(S+t)^2\right) \;=\; \mathcal{O}\big(TS^2 + T^2S + T^3\big)\,.$$

Con caché, el prefill cuesta $\mathcal{O}(S^2)$ una vez y cada paso de decodificación cuesta $\mathcal{O}(S+t)$:

$$\mathcal{O}\left(S^2 + \sum_{t=0}^{T-1}(S+t)\right) \;=\; \mathcal{O}\big(S^2 + TS + T^2\big)\,.$$

La diferencia es la conocida entre re-codificar una conversación y continuarla.

El subsistema cubre los cuarenta y cuatro tipos de capa despachables. Veintiséis tienen ramas incrementales verificadas por paridad; el resto queda registrado con una justificación explícita, basada en el artículo, para ejecutarse sin caché (§17.4.5). La cuantización y la expulsión son ortogonales a la matemática del mezclador: los códecs transforman tensores en *escritura* y las políticas actúan en *lectura*, de modo que el mismo códec `int8` o `int4_kivi` sirve a un slot denso, latente o de anillo sin código por mezclador [75, 54, 29].

17.2. Arquitectura de la Caché

17.2.1. Gestor Central e Identidad de Capa

La unidad de cacheo es una capa lógica, no un módulo. El encoder puede re-ejecutar la misma pila física `num_loops` veces (profundidad en bucle) y, bajo Hyperloop, dividirla en un bloque inicial, un bloque medio en bucle y un bloque final. El mismo objeto `HybridLayer` desempeña por tanto varios roles distintos durante una pasada. La caché indexa cada slot por el **índice de capa lógica** global

$$\ell \;=\; 0, \ldots, \texttt{num_layers} \times \texttt{num_loops} - 1,$$

que el encoder ya propaga a cada llamada de mezclador. Dos roles lógicos nunca comparten almacenamiento aunque compartan pesos.

Algorithm 46 Bucle de generación con caché (prefill + decodificación)

Require: ids del prompt $\mathbf{x}_{1:S}$; configuración de generación θ; modelo f
Ensure: ids generados

$\mathcal{C} \leftarrow$ CACHE.NEW(f, θ) ▷ slots perezosos, cola de tokens, cursor de escritura
$\mathbf{L}, \mathcal{C} \leftarrow f(\mathbf{x}_{1:S}; \text{past} = \mathcal{C}, \text{use_cache} = \top, \text{return_cache} = \top)$ ▷ prefill
$\mathbf{z} \leftarrow \mathbf{L}_{:,S,:}$
for $t \leftarrow 0$ **hasta** $T - 1$ **do**
    $y \leftarrow$ SAMPLE$(\mathbf{z}; \theta)$ ▷ greedy, top-k, top-p
    añadir y a la salida
    if $t < T - 1$ **then**
        $p \leftarrow \mathcal{C}$.next_position()
        $\mathbf{L}, \mathcal{C} \leftarrow f(y; \text{past} = \mathcal{C}, \text{use_cache} = \top, \text{cache_position} = p, \text{return_cache} = \top)$
        $\mathbf{z} \leftarrow \mathbf{L}_{:,-1,:}$
    end if
end for

El gestor posee el diccionario de slots por capa, una cola FIFO de ids de tokens recientes (consumida por mezcladores de nivel de token como Engram), un cursor de escritura que avanza en cada pasada, y el códec de cuantización y la política de expulsión resueltos. La creación de slots es perezosa y la dispara la capa en su primer uso, de modo que un `layer_pattern` heterogéneo — por ejemplo una capa densa, una latente y una recurrente en una misma pila — mantiene tres tipos de slot distintos sin casos especiales. La reversión es una operación de primera clase: `truncate(n)` conserva las primeras n posiciones en todos los slots (truncado de listas barato con contabilidad de longitud; los slots de estado recurrente son no-ops porque no están indexados por posición), que es la primitiva que requieren la decodificación especulativa y la verificación de salida temprana [33].

17.2.2. Slots Tipados

Seis tipos de slot cubren la taxonomía de §17.4. Los slots indexados por posición almacenan una lista ordenada de *partes*; una parte es de precisión completa o una carga codificada, y la lectura concatena [sinks | historia | ventana protegida].

Tipo de slot	Almacena	Usado por
`kv`	claves y valores post-RoPE $(\mathrm{B}, H_{kv}, T, d)$; posiciones absolutas; extras por token opcionales	mezcladores softmax cuadráticos
`latent_kv`	flujos de bajo rango con nombre: latente $\mathbf{c}_{KV}$, clave RoPE desacoplada, claves agrupadas, valores latentes	familia latente
`state`	una matriz recurrente (+ acumuladores de compuerta), actualizada in situ; memoria constante	familia recurrente / lineal
`ring`	K/V acotados por capacidad con una región de sinks fijada	familia de ventana deslizante
`conv_tail`	últimas $k-1$ filas pre-convolución (FIFO)	convoluciones causales cortas
`token_tail`	últimos n ids de token	búsqueda de n-gramas de Engram

Cuadro 21: Slots de caché tipados. Los tipos indexados por posición (`kv`, `ring`, `latent_kv`) llevan metadatos de posición absoluta, de modo que los huecos por expulsión mantienen distancias relativas correctas; `state` y `conv_tail` no tienen posición y nunca se expulsan.

17.2.3. Registro de Capacidades

Un registro congelado por cada valor del enum `layer_pattern` declara si una capa cachea y cómo:

$$\textsc{CacheSpec} \;=\; \big(\texttt{supported},\ \texttt{slot_type},\ \texttt{slot_types},\ \texttt{extras},\ \texttt{notes}\big).$$

El registro es la fuente única de verdad consultada tanto en ejecución (resolución de slot) como estáticamente (avisos de configuración). Una capa recibe un slot sólo cuando la formulación de su mezclador admite caché y existe una rama incremental implementada y verificada por paridad; en caso contrario recibe `None` y ejecuta la ruta legada. El registro es, por tanto, también el inventario honesto del proyecto sobre qué significa cachear para cada artículo: `supported` registra la formulación de origen, el conjunto implementado registra el estado de ingeniería, y `notes` registra el motivo.

17.2.4. Propagación en el Forward y Semántica de Respaldo

Todos los cambios de firma son aditivos y por defecto reproducen el comportamiento legado:

$$f_{\text{mixer}}(\mathbf{x}; \ell, \text{pe}, \texttt{cache} = \varnothing, \texttt{cache_position} = \varnothing),$$
$$f_{\text{model}}(\mathbf{x}; \texttt{past_key_values} = \varnothing, \texttt{use_cache} = \bot, \texttt{return_cache} = \bot).$$

`HybridLayer` resuelve el slot una vez por llamada y respeta las cuatro formas históricas de despacho: el lineal sin estado (`mamba`) se invoca sin argumentos porque no tiene nada que cachear; los dos mezcladores que reemplazan el flujo completo (`ode`, `retnet`) reciben el slot sólo cuando su especificación lo admite; Engram recibe además `input_ids`; el resto de mezcladores recibe `cache` y `cache_position`. Las capas no admitidas o deshabilitadas se invocan exactamente como antes, lo que mantiene bit-idéntica la ruta sin caché.

17.3. Contratos de Corrección

La ejecución con y sin caché debe coincidir en la fila de consulta que comparten: la última fila de una pasada completa y la fila producida por un paso de decodificación. Los contratos siguientes establecen esa coincidencia.

17.3.1. Almacenamiento Post-RoPE y el Calendario Absoluto

Las claves se almacenan *después* de la transformación rotatoria, la convención de la industria: la caché guarda los vectores ya rotados y no hace falta re-rotar cuando un anillo da la vuelta. El requisito es que la rotación use posiciones absolutas a ambos lados de la frontera de la caché. RoPE, HoPE y sinusoidal-rotatoria aceptan un desplazamiento de posición π_0 y sitúan los tokens entrantes en

$$\pi_i = \big(\pi_0 + i\big) \cdot \texttt{scaling}, \qquad i = 0, \ldots, S_{\text{nuevo}} - 1,$$

de modo que un paso de decodificación en el cursor p rota exactamente como lo hizo la pasada completa en la posición p. Los metadatos del slot versionan el factor de escalado; un cambio en tiempo de ejecución del calendario RoPE invalida las claves ya rotadas y se reporta como error de configuración al inicio de la generación, nunca a mitad de decodificación [107, 23].

17.3.2. Sesgos Aditivos en las Puntuaciones

ALiBi y BAM inyectan la posición a través de la matriz de puntuaciones, así que no llevan tensor posicional en la caché — precisamente su atractivo para la extrapolación de longitud [93, 13]. El constructor de matriz completa se generaliza a uno de ventana sobre posiciones absolutas de consulta y clave:

$$b_{h,i,j} \;=\; -\,\text{pendiente}_h \cdot \big|q_i - \kappa_j\big| \quad \text{(ALiBi)}, \qquad b_{h,i,j} \;=\; -\Big(\big|\kappa_j - \alpha_h q_i - \mu_h\big| + \varepsilon\Big)^{\beta_h} \quad \text{(BAM)},$$

donde $q_i = \pi_0 + i$ para las posiciones de consulta entrantes y κ_j son las posiciones absolutas de las claves cacheadas. Con $\pi_0 = 0$ y $\kappa_j = j$ esto reproduce exactamente la matriz legada; tras una expulsión, κ_j puede ser no contigua y las distancias siguen siendo correctas por construcción.

17.3.3. Reescalado Dependiente de la Longitud

Scalable Softmax multiplica los logits pre-softmax por $s \cdot \ln n$, donde n es la longitud de contexto. En decodificación con caché el contexto de la consulta más reciente es la caché acumulada, así que el reescalado usa

$$\ln\big(n_{\text{cacheado}} + S_{\text{nuevo}}\big)$$

en lugar del tamaño del fragmento entrante. Para la fila final de una pasada completa sobre $S_{\text{cacheado}} + S_{\text{nuevo}}$ tokens ambas expresiones coinciden, que es exactamente la fila que comparan los tests de paridad.

17.3.4. Enmascaramiento Causal desde Posiciones

Las máscaras se derivan de metadatos y no de índices de arreglo:

$$M_{ij} \;=\; \big[\, q_i < \kappa_j \,\big] \;=\; \big[\, \pi_0 + i < \kappa_j \,\big],$$

que se reduce a la máscara triangular superior clásica cuando la caché empieza vacía y $\kappa_j = j$. Para una única consulta de decodificación toda clave cacheada cumple $\kappa_j \le q_i$, así que la máscara es vacía y el paso no necesita máscara alguna. Las familias dispersas sustituyen el predicado causal por un patrón basado en posiciones (§17.4).

17.3.5. Codificaciones Posicionales Absolutas

Las codificaciones absolutas aditivas (sinusoidal, aprendida) se aplican una vez a los embeddings. En decodificación con caché la búsqueda del embedding del token nuevo recibe el mismo desplazamiento, de modo que el token p se aumenta con la fila p de la tabla:

$$\mathbf{x}_i \;\leftarrow\; \mathbf{x}_i + \mathrm{PE}\big[\pi_0 + i\big].$$

17.3.6. Interbloqueos con Mezcla de Profundidades, mHC, Hyperloop y AttnRes

La mezcla de profundidades (MoD) enruta una fracción de tokens por cada capa. Enrutar un prompt produciría cachés irregulares y dependientes del token, algo que v1 excluye deliberadamente; cuando la caché está activa, el prefill se fuerza denso mientras que la decodificación degenera a densa automáticamente (un token siempre cabe en la capacidad). La divergencia es unidireccional: forzar denso sólo puede añadir claves, nunca atender a claves que no existen. Los flujos residuales de mHC y Hyperloop son estado por token y atraviesan una pasada con caché sin arrastre entre pasos; las hiper-conexiones a nivel de bucle se disparan por iteración y no necesitan slot propio. Las variantes de residual de atención mantienen búferes entre capas que deben acumularse a través de los pasos, un modo aún no implementado; la combinación deshabilita la caché para esa pasada con un aviso y recurre a la recomputación completa. Las augmentaciones PaPE dependen del contenido, así que cachear claves post-augmentación no es demostrablemente exacto; esas capas retroceden por capa [86].

17.4. Recetas Incrementales por Familia

17.4.1. Familia Softmax Cuadrática

Para una capa de atención densa con H cabezas de consulta, H_{kv} cabezas de clave/valor y dimensión de cabeza d, la rama cacheada añade las claves y valores nuevos y atiende sobre todo el prefijo retenido:

$$\mathbf{K} \leftarrow \left[\mathbf{K}_{\text{cacheado}};\ \mathbf{k}_{\text{nuevo}}\right], \quad \mathbf{V} \leftarrow \left[\mathbf{V}_{\text{cacheado}};\ \mathbf{v}_{\text{nuevo}}\right], \quad \mathbf{S} = \frac{\mathbf{q}\mathbf{K}^{\top}}{\sqrt{d}} + \mathbf{B}, \quad \mathbf{o} = \operatorname{softmax}(\mathbf{S})\,\mathbf{V},$$

con $\mathbf{B}$ el sesgo posicional de §17.3. La atención de consultas agrupadas cachea sólo las H_{kv} cabezas compactas y expande los grupos de cabezas después de la lectura, que es de donde proviene su ventaja de caché [2]. Las variaciones de la familia se capturan sustituyendo el normalizador o el sesgo:

- **Atención sigmoide.** El normalizador se sustituye por una normalización ℓ_1 por fila, $\hat{\mathbf{S}}_{ij} = \sigma(\mathbf{S}_{ij}) / \left(\sum_{j'} \sigma(\mathbf{S}_{ij'}) + \varepsilon\right)$. Como la sigmoide no es invariante a desplazamientos, el denominador debe re-sumarse sobre todo el prefijo cacheado en cada paso; no existe truco de máximo estilo softmax en línea [97].
- **Softmax con compuerta.** La compuerta $\sigma(\mathbf{x}\mathbf{W}_g)$ multiplica la salida de atención y depende sólo del token actual, así que no se cachea nada extra [95].
- **Atención con olvido.** El sesgo de logits es $D_{ij} = c_i - c_j$ con $c_i = \sum_{l \le i} \log f_l$ la suma acumulada de log-olvido. La caché guarda un escalar por token y cabeza como extra; la decodificación extiende la suma corrida desde el último token cacheado y evalúa una sola fila de D contra la columna cacheada [71].
- **Titans.** La capa implementada es la ventana de atención; el módulo de memoria queda fuera de la ruta cacheada. Se aplica el append/read estándar [11].
- **Atención de cabezas intercaladas.** Los tensores pseudo-clave/valor son mezclas estáticas aprendidas de cabezas calculadas al vuelo; sólo se cachea el K/V real por token, y la mezcla se aplica después de la lectura (la mezcla es un mapa por token en el espacio de cabezas, así que aplicarla post-lectura es exacto) [31].

17.4.2. Familia Latente

Los mezcladores latentes comprimen claves y valores en un latente de bajo rango y cachean sólo ese latente. Escribiendo $\mathbf{c}_{KV} = \mathbf{W}_{DKV}\mathbf{x}$, la decodificación re-expande y rota por paso:

$$\mathbf{k} = \mathbf{W}_{UK}\mathbf{c}_{KV}, \qquad \mathbf{v} = \mathbf{W}_{UV}\mathbf{c}_{KV}, \qquad [\mathbf{q}, \mathbf{k}] \leftarrow \mathrm{RoPE}\left([\mathbf{q}, \mathbf{k}], \pi_0\right),$$

con la atención como en el caso denso [80]. El MLA estilo DeepSeek cachea una clave RoPE desacoplada y puede absorber las proyecciones ascendentes en la consulta, evitando la expansión por paso; la variante de este toolkit aplica RoPE a la consulta y la clave descomprimidas, así que la absorción no está disponible y la expansión es el coste documentado de reducir la caché a la mitad. GQLA reutiliza un conjunto de pesos con dos rutas de decodificación algebraicamente equivalentes, y el diseño del slot sigue la ruta configurada [81]. MLRA particiona el latente en sub-cabezas disjuntas, lo que es una ganancia de particionado en paralelismo tensorial más que de compresión; la caché total es del tamaño de MLA [74]. La atención de Tucker cachea los latentes factorizados de clave y valor y expande por las proyecciones del núcleo [65]. GTA cachea una clave por *grupo* de cabezas más valores latentes; como la rotación en una posición fija es independiente de la cabeza, las medias de grupo conmutan con la rotación y cachear la media es exacto [108]. MTLA es el único método que comprime a lo largo del tiempo: una hiper-red fusiona vectores adyacentes de la caché en fronteras de zancada, el calendario es dinámico y su paridad con un calendario de entrenamiento a nivel de fragmento es un riesgo declarado, por lo que actualmente retrocede [26].

17.4.3. Familias Dispersas y de Ventana

Los patrones dispersos se evalúan como predicados sobre posiciones absolutas, nunca sobre índices de arreglo [20]:

$$\mathrm{atender}(q_i, \kappa_j) \iff \mathrm{patrón}\big(\pi_0 + i,\ \kappa_j\big) \ \wedge\ \kappa_j \le \pi_0 + i.$$

- **Ventana deslizante con sinks.** Cada consulta atiende a las primeras s posiciones más las últimas $w - s$; la política de retención del anillo *es* el patrón de atención, así que un paso de decodificación atiende directamente al conjunto cacheado y un fragmento multi-token se enmascara por patrón [58, 118]. Eliminar los sinks colapsa la calidad, por lo que se fijan tanto por el patrón como por el anillo.
- **Longformer.** Ventanas simétricas más tokens globales especificados por la tarea; como un token global puede estar en cualquier parte, el slot es ilimitado y el patrón se evalúa sobre posiciones [12].
- **BigBird.** Ventana, globales y enlaces aleatorios. Los enlaces aleatorios deben ser reproducibles entre pasos incrementales, así que se extraen de un generador sembrado por (ℓ, q_i) sobre $[0, q_i]$: los enlaces más allá de la consulta quedan causalmente enmascarados de todos modos, de modo que la máscara efectiva no cambia y a la vez se vuelve determinista [127].
- **Sparse Transformers.** Patrones factorizados con zancada y fijos, donde el rol de columna de resumen de un token cambia al completarse los bloques; la evaluación por posiciones lo maneja, y una zancada automática se resuelve desde la longitud acumulada para que las rutas completa y cacheada coincidan [20].
- **FASA.** La importancia por cabeza se calcula a partir de los fragmentos de frecuencia RoPE dominantes y se reordena por consulta; es local a la fila, así que un paso de decodificación reordena exactamente las claves cacheadas [116]. SpargeAttn, NSA, MiniMax y SparDA seleccionan bloques a partir de estadísticos a nivel de fragmento y por ello retroceden en v1 [131, 125, 67, 36].

17.4.4. Familia Recurrente y de Estado Lineal

Los mezcladores recurrentes llevan un estado de tamaño fijo; cachear significa persistir ese estado, así que la memoria es constante y ninguna expulsión tiene sentido. Con la convención del artículo $\mathbf{S}_t \in \mathbb{R}^{d_v \times d_k}$ y lectura $\mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$:

$$\text{RetNet:} \quad \mathbf{S}_t = \gamma_h \, \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \qquad \gamma_h \text{ multi-escala por cabeza}, \tag{133}$$

$$\text{GLA:} \quad \mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \qquad \mathbf{G}_t = \operatorname{diag}\big(\exp(\text{logsigmoid}(\mathbf{W}_{gk}\mathbf{x}_t)/c)\big), \tag{134}$$

$$\text{DeltaNet:} \quad \mathbf{S}_t = \mathbf{S}_{t-1}\big(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top\big) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top, \tag{135}$$

$$\text{Gated DeltaNet:} \quad \mathbf{S}_t = \alpha_t \, \mathbf{S}_{t-1}\big(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top\big) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top, \tag{136}$$

$$\text{GDN-2:} \quad \mathbf{S}_t = \big(\mathbf{I} - \mathbf{k}_t(\mathbf{b}_t \odot \mathbf{k}_t)^\top\big)\mathbf{D}_t \mathbf{S}_{t-1} + \mathbf{k}_t(\mathbf{w}_t \odot \mathbf{v}_t)^\top, \tag{137}$$

$$\text{HGRN2:} \quad \mathbf{S}_t = \operatorname{diag}(\mathbf{g}_t)\mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \qquad g_{t,j} = b_\ell + (1 - b_\ell)\sigma(\cdot), \tag{138}$$

$$\text{KDA:} \quad \mathbf{S}_t = \big(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top\big)\mathbf{D}_t \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top, \tag{139}$$

$$\text{GMA:} \quad \tilde{\mathbf{V}}_t = \tilde{\mathbf{V}}_{t-1} + \boldsymbol{\gamma}_t^K \otimes \mathbf{v}_t, \quad \mathbf{Z}_t = \mathbf{Z}_{t-1} + \boldsymbol{\gamma}_t^K, \quad \mathbf{o}_t = \frac{\sum_k \gamma_{t,k}^Q \tilde{\mathbf{V}}_{t,k}}{\sum_k \gamma_{t,k}^Q Z_{t,k} + \varepsilon}, \tag{140}$$

refiriendo a Sun et al. [109], Yang et al. [121], Yang et al. [122], Yang et al. [123], Hatamizadeh et al. [48], Qin et al. [94], Kimi Team et al. [63] y Huang and Raza [57].

Dos sutilezas merecen énfasis. Primero, la suma de retención paralela de RetNet y la recurrencia anterior producen la misma fila para la misma consulta — algebraicamente la forma paralela factoriza a través del estado corrido — así que la ruta cacheada puede usar la forma recurrente mientras la ruta sin caché usa la paralela, y el test de paridad compara la fila compartida. Segundo, GDN-2 y KDA decaen a lo largo del eje de *clave* ($\mathbf{D}_t$ multiplica el estado por la izquierda), mientras que GLA y HGRN2 compuertan el eje de clave elemento a elemento; ambas son diagonales en la dimensión de clave, que es lo que hace exacta una recurrencia simple in situ. El bloque de atención ODE queda excluido: su solver re-deriva el estado intermedio de cada token en cada paso de integración, así que una caché de estado convergido no puede reproducir la ruta completa.

17.4.5. Estado Auxiliar y Respaldos Documentados

Algunos mezcladores necesitan una pequeña FIFO junto a su caché de activaciones. Las variantes chunk-parallel de Falcon acotan una ventana de repetición, así que la continuación exacta requiere los últimos $B - 1$ pares alineados al prefijo; CCA/CCGQA apilan dos convoluciones causales y necesitan los últimos $k - 1$ latentes pre-convolución más un valor desplazado; Engram sólo necesita los últimos $n_{\text{máx}} - 1$ ids de token. Estos son los tipos `conv_tail` y `token_tail`.

Mezclador	Por qué se ejecuta sin caché en v1
`mamba`	marcador de posición lineal; sin estado por token, inherentemente incremental
`ssog_attn`	rejilla de visión bidireccional; no existe formulación causal
`ode`	el solver re-deriva los estados intermedios de todos los tokens en cada paso
`falcon1..3a`	FIFO de repetición chunk-parallel (diseño `conv_tail` pendiente)
`cca_attn`, `ccgqa_attn`	dos convoluciones causales + valor desplazado
`engram_attn`	cola de ids de token (diseño `token_tail` pendiente)
`nsa`, `msa`, `sparda`, `sparge`	la selección de bloques usa estadísticos de fragmento, no locales a la fila
`mtla_attn`	calendario dinámico de fusión por hiper-red (riesgo R1)
`sparsek_attn`	diseño de política de expulsión (SparseK es una política, no un slot)
capas PaPE	augmentación q/k dependiente del contenido; exactitud no demostrada

Cuadro 22: Respaldos sin caché documentados. Cada entrada lleva una justificación basada en el artículo en el registro de capacidades y un aviso del cargador cuando la caché está habilitada; ninguna puede hacer fallar la inferencia.

17.5. Cuantización de la Caché KV

La cuantización es una transformación de escritura: el slot codifica al añadir y descuantiza al leer, de modo que la huella almacenada se reduce mientras el mezclador consume tensores de punto flotante ordinarios. Esta ubicación sigue a la literatura y mantiene los códecs ortogonales a los mezcladores: el mismo códec sirve a slots densos, latentes y de anillo, y los slots latentes pueden declarar una granularidad distinta por flujo. Los estados recurrentes *no* se cuantizan en v1: la base de evidencia es allí la más delgada y su tamaño constante hace que la ganancia sea pequeña.

17.5.1. int8 Simétrico con Ejes Asimétricos

Las claves exhiben valores atípicos persistentes por canal (una estructura de frecuencia RoPE), mientras que los valores exhiben atípicos por token. El códec `int8` usa por tanto ejes de reducción distintos:

$$\text{claves (eje de canal):}\quad s_{h,c} = \frac{\max_{t\in g} |K_{t,h,c}|}{127}, \qquad \hat{K} = \operatorname{round}\big(K/s\big) \in [-127, 127];$$

$$\text{valores (eje de token):}\quad s_{t,h} = \frac{\max_{c} |V_{t,h,c}|}{127}, \qquad \hat{V} = \operatorname{round}\big(V/s\big) \in [-127, 127].$$

Cada bloque almacenado lleva sus propias escalas, así que un bloque de decodificación token a token obtiene escalas exactas por token y el error relativo de redondeo queda acotado por $1/127 \approx 0{,}8\,\%$ independientemente del tamaño del bloque. La descuantización es $\hat{K} \odot s$; fusionar dos cargas es descuantizar, concatenar a lo largo del tiempo y recuantizar.

17.5.2. int4 por Grupos con Ventana Protegida

El códec `int4_kivi` combina la cuantización por grupos de bajo bit de KIVI con el patrón de protección de dos regiones de los sinks de streaming y SKVQ. Los niveles con signo son $[-7, 7]$; g tokens forman un grupo a lo largo del tiempo; la granularidad sigue al códec int8. Dos regiones se mantienen en precisión completa:

- los w_p tokens más recientes (la *ventana protegida*), nunca cuantizados;
- los primeros s tokens (*sinks*), nunca cuantizados y nunca expulsados.

La semántica de append refleja la ventana residual de KIVI: un paso de decodificación empuja filas a la ventana de precisión completa, y en cuanto el fragmento más antiguo completa un grupo de g tokens ese grupo se cuantiza y pasa a la historia; el prefill cuantiza grupos completos de inmediato. La lectura concatena [historia cuantizada | ventana protegida]. Con niveles de 4 bits con signo, el peor error absoluto de un cuantizador por grupos es medio paso,

$$|K - \hat{K}| \;\le\; \frac{s}{2} \;=\; \frac{\max_g |K|}{2 \cdot 7} \;\approx\; 7{,}1\,\% \text{ del máximo del grupo,}$$

que es la cota que afirman los tests del códec. Como los sinks y la ventana protegida quedan exactos, los tokens que dominan la masa de atención nunca se perturban [75, 54, 29, 118].

17.5.3. Interfaz y Códecs Futuros

Ambos códecs implementan el mismo contrato de tres métodos: `encode_write`, `decode_read` y `update`. Los códecs nunca deben lanzar excepciones ante tensores desconocidos: ante cualquier error interno recurren a la identidad y registran un aviso una sola vez, preservando D5 también en la capa de almacenamiento. El registro v1 contiene `none`, `int8` e `int4_kivi`; otros métodos están listos a nivel de interfaz pero deliberadamente sin implementar, y solicitarlos es un error de configuración al inicio de la generación: `int2_kivi` (2 bits con normalización de varianza de doble eje), `palu_lowrank` (proyección de rango r con reconstrucción), `commvq` (libros de códigos conmutativos con RoPE, atención en el espacio de códigos), `qjl_sketch` (signos de Johnson–Lindenstrauss de 1 bit con estimador insesgado del producto interno) y `block_gtq` (asignación de bits por bloque de frecuencia RoPE). Los metadatos del slot ya registran el tipo de codificación posicional y la versión de escalado, que es lo que un códec de libros de códigos o pre-RoPE necesita para declarar su compatibilidad. La asignación de bits por capa sensible a la sensibilidad (`k_bits` $\neq$ `v_bits`) se expone por configuración, ya que la masa de atención es piramidal en profundidad.

17.6. Compresión y Expulsión

La expulsión es una transformación de lectura; mantenerla ortogonal a la cuantización es lo que permite que una caché cuantizada se expulse y una caché expulsada se cuantice. La política v1 es la ventana con sinks de StreamingLLM: fijar las primeras s posiciones, retener las últimas w posiciones y descartar todo lo intermedio. El presupuesto retenido es

$$\mathcal{B} \;=\; s + w,$$

opcionalmente sobrescrito por capa mediante un vector de presupuestos $\mathcal{B}_\ell$ (la masa de atención es piramidal en profundidad, así que las capas profundas toleran presupuestos menores). Las políticas actúan sólo sobre slots indexados por posición; los estados recurrentes son de tamaño constante y MTLA gestiona su propio calendario de fusión, por lo que las políticas externas quedan excluidas allí por construcción. Las políticas futuras listas para el enum son H2O (acumular masa de atención por token y expulsar la más ligera), SnapKV (compresión única tras el prefill a partir de una ventana de anticipación), PyramidKV (presupuestos estáticos dependientes de la profundidad) y SparseK (puntuador aprendido con expulsión permanente dentro de la generación, razón por la cual se modela como política y no como slot).

17.7. Interfaz de Configuración y Despliegue

El bloque de ejecución lo valida un módulo de esquema dedicado y se aplana en una clase de datos de ejecución que deliberadamente *no* forma parte de la configuración arquitectónica, reflejando la separación de `training.*`.

Clave	Defecto	Descripción
inference.generation.max_new_tokens	64	tokens generados tras el prompt
inference.generation.temperature	1,0	temperatura softmax
inference.generation.top_k	50	filtro top-k; 0 lo desactiva
inference.generation.top_p	1,0	masa nucleus; 1,0 lo desactiva
inference.generation.greedy	false	decodificación argmax
kv_cache.enabled	false	opcional; sin fijarlo el bucle legado es bit-idéntico
quantization.method	none	none, int8, int4_kivi
quantization.k_bits / v_bits	4 / 4	anchos de bits int4 (int8 fija 8)
quantization.group_size	64	tokens por grupo
quantization.protected_window	32	tokens más recientes en precisión completa
compression.method	none	none, window_sinks
compression.num_sinks	4	posiciones iniciales fijadas
compression.window	512	tamaño de la ventana deslizante
compression.budget	null	presupuesto escalar por capa; null = ilimitado
compression.per_layer_budgets	[]	vector de presupuestos por capa, sobrescribe el escalar
per_mixer[].enabled	true	exclusión por mezclador

Cuadro 23: Configuración de ejecución de inferencia. Reglas entre claves: la compresión requiere enabled, y window $\geq$ protected_window para que la región protegida nunca pueda expulsarse.

El orden de resolución es **banderas CLI > YAML** `inference:` **> defectos del esquema**. Cada bandera CLI tiene por defecto "sin fijar", de modo que un preset YAML se parchea hoja a hoja en lugar de reemplazarse:

```
infer -task causal_lm -config <preset>-prompt "...-kv-cache -kv-quant int4_kivi -kv-compress
                                window_sinks -kv-window 512
```

El desplegador serializa el bloque resuelto en el config.json del artefacto bajo ïnference", y el motor de inferencia lo vuelve a leer, de modo que la generación con caché es reproducible sin el YAML original. El envoltorio causal-LM de la exportación a HuggingFace acepta past_key_values, use_cache y cache_position y los reenvía al decodificador en lugar de descartar extras, e implementa prepare_inputs_for_generation para que un bucle estándar de transformers alimente sólo el último token una vez que existe caché.

17.8. Validación

El test central es un protocolo de paridad: para un mezclador y una codificación posicional, comparar una pasada completa sobre S tokens contra un prefill sobre $S - k$ tokens seguido de k pasos de decodificación, exigiendo coincidencia en la fila compartida. La generación greedy se compara token a token primero; si aparece una discrepancia numérica, se comparan los logits de la última posición con rtol $= 10^{-4}$, atol $= 10^{-5}$ en fp32 con cuantizador none y BitNet deshabilitado.

Suite	Afirmaciones
slots	append/read/truncate/movimientos de dispositivo; vuelta de anillo con metadatos de posición; estado in situ; FIFO de cola de convolución; cola de tokens
códecs	error int8 $\leq 10^{-2}$; error int4 $\leq 7{,}5 \times 10^{-2}$; ventana protegida exactamente fp; fronteras de grupo; `update` = descuantizar→concatenar→recuantizar
mezcladores	cada mezclador implementado × {rope, none, ALiBi}: pasada completa vs prefill+decodificación
respaldo	los 44 valores del enum corren con la caché activa; mezcladores no implementados bit-idénticos on/off; respaldo PaPE por capa
mecanismos	profundidad en bucle (slots distintos por capa lógica), Hyperloop, mHC, prefill denso de MoD, respaldo AttnRes
cuantización	cotas de deriva int8/int4 con BitNet on y off; int4 se activa en prompts cortos; ventana protegida exacta
compresión	sinks fijados tras la expulsión; región retenida correcta; composición con int8
generación	paridad greedy exacta; determinismo de muestreo con semilla; reversión con `truncate`
configuración	paridad de defectos del esquema, aplanado, precedencia, reglas entre claves, avisos de respaldo, serialización

Cuadro 24: Matriz de validación. El registro también se verifica cruzadamente contra el enum de patrones de capa del esquema, de modo que un mezclador nuevo no puede añadirse sin una decisión de caché.

17.9. Relación con Trabajo Previo

- **Cuantización de caché KV** (KIVI, KVQuant, SKVQ): adoptada como códecs de escritura con granularidad asimétrica y regiones protegidas de atípicos; la semántica de append por ventana de grupo sigue la ventana residual de KIVI.
- **Expulsión y dispersión** (StreamingLLM, H2O, SnapKV, PyramidKV, Quest): adoptadas como políticas de lectura; los metadatos de página estilo Quest y la lectura consciente de la consulta son trabajo futuro dentro de los extras del slot.
- **Paginación y compartición de prefijos** (PagedAttention/vLLM, RadixAttention): estructuralmente compatibles — los slots son agnósticos al contenido y serializables — pero las tablas de bloques y la compartición de prefijos quedan fuera de v1.
- **Planificadores híbridos** (Jamba, Griffin, Zamba): el mismo diccionario de slots alberga K/V de atención y estados recurrentes, que es el patrón que establecieron estos sistemas.
- **Decodificación especulativa y salida temprana** (LayerSkip): servidas por la misma primitiva `truncate` que ejercitan los tests.

Cuándo habilitar la caché. Cualquier carga de generación decoder-only se beneficia: el régimen donde domina la decodificación (salidas largas, prompts cortos) gana más, mientras que una única pasada completa (clasificación, embeddings, predicción enmascarada) no gana nada y debe dejar `enabled` en falso. La cuantización merece habilitarse cuando la restricción vinculante es la memoria y no la latencia, y `int4_kivi` debe conservar una ventana protegida de al menos el tramo de los sinks de atención; la expulsión de ventana con sinks es el defecto correcto para contextos de streaming o ilimitados. Para las capas recurrentes el estado ya es de tamaño constante, así que la expulsión carece de sentido y la cuantización está deshabilitada por diseño.

18. Softmax y Cross-Entropy Numéricamente Corregidos

Las implementaciones estándar de softmax, log-softmax y cross-entropy restan el logit máximo antes de exponenciar, lo que evita el desbordamiento pero deja dos defectos de punto flotante [9]. El primero, el *Colapso de Softmax*, redondea a cero el gradiente del objetivo (o del argmax) cuando los logits se vuelven suficientemente confiados, rompiendo la propiedad de suma cero del gradiente y produciendo los picos de pérdida "Slingshot" observados con optimizadores adaptativos. El segundo, la *Cancelación de Softmax*, degrada los gradientes de objetivo blando (destilación) cuando los logits del estudiante y del profesor casi coinciden. Este anexo documenta la implementación opcional de la primera corrección en el toolkit — softmax, log-softmax y cross-entropy de objetivo duro sin cancelación —, su paso backward personalizado, su superficie de integración y su validación. La cross-entropy de objetivo blando queda fuera de alcance porque el toolkit no expone un objetivo de destilación; la ruta de embeddings de frases delega sus pérdidas en `sentence-transformers`.

18.1. Los Dos Defectos de Punto Flotante

Para logits $z \in \mathbb{R}^K$ y clase objetivo t, los cálculos estabilizados estándar son

$$\tilde{z}_k = z_k - z_{\max}, \qquad \operatorname{logsoftmax}(z)_k = \tilde{z}_k - \log \sum_j e^{\tilde{z}_j}, \qquad q_k = \frac{e^{\tilde{z}_k}}{\sum_j e^{\tilde{z}_j}}, \tag{141}$$

y la cross-entropy es $L(z,t) = -\operatorname{logsoftmax}(z)_t$, cuyo gradiente exacto es $g = q - p$ para una distribución objetivo p.

Colapso de Softmax. Defínase el margen de logits

$$m = z_t - \log \sum_{j \neq t} e^{z_j}. \tag{142}$$

Cuando el objetivo es también el logit máximo, $\tilde{z}_t = 0$ y el denominador pasa a ser $1 + e^{-m}$. Los números en punto flotante son mucho más densos cerca de cero que cerca de uno: en fp32, los valores cerca de 0 se resuelven hasta $\sim 10^{-38}$, mientras que los términos sumados a 1,0 se pierden por debajo del redondeo unitario $u = 2^{-p}$. El denominador se redondea por tanto a exactamente 1,0 cuando m supera el *acantilado de precisión*

$$c_d = -\ln u = p \ln 2, \qquad c_{16} \approx 5{,}5, \quad c_{32} \approx 16{,}6, \quad c_{64} \approx 36{,}7, \tag{143}$$

para bf16 ($p = 8$), fp32 ($p = 24$) y fp64 ($p = 53$). Pasado el acantilado, la pérdida forward se vuelve $\log(1) = 0$ y el gradiente objetivo $q_t - 1$ pasa a ser exactamente 0, mientras que los gradientes no objetivo $q_j = e^{z_j - z_t}/(1 + e^{-m})$ siguen siendo positivos hasta el subdesbordamiento del exponente ($m \approx 88$ en fp32). El gradiente deja de sumar cero,

$$\sum_k g_k \approx \sum_{j \neq t} q_j > 0, \tag{144}$$

lo que sesga cada actualización de parámetros e interactúa mal con la normalización por coordenada de Adam: cuando el gradiente objetivo perdido reaparece más tarde, la estimación del segundo momento ya ha decaído y la actualización produce un pico.

Cancelación de Softmax. Para objetivos blandos $p = \operatorname{softmax}(z^T)$, el gradiente $q - p$ es la diferencia de dos vectores de probabilidad casi iguales. Calculados por separado, se pierde la pequeña diferencia que porta la señal y queda un residuo distinto de cero incluso cuando $z^S = z^T$. La corrección del artículo calcula la diferencia directamente en espacio logarítmico con `expm1`/`log1p`; este toolkit no tiene pérdida de destilación y por tanto no la implementa.

18.2. Formulaciones Sin Cancelación

Ambas correcciones excluyen la entrada argmax del log-sum-exp y la reintroducen mediante un softplus estable. Sean $a = \arg\max_k z_k$, $z_{\max} = z_a$ y

$$s = \operatorname{softplus}(-m) = \log\bigl(1 + e^{-m}\bigr) = \log \sum_{j \neq a} e^{z_j - z_{\max}}, \qquad m = z_{\max} - \log \sum_{j \neq a} e^{z_j}. \tag{145}$$

Entonces

$$\operatorname{logsoftmax}(z)_k = z_k - z_{\max} - s, \qquad q_k = \exp(z_k - z_{\max} - s)\,, \tag{146}$$

con la entrada argmax evaluada exactamente como $-s$ y $\exp(-s)$ respectivamente. Dado que s se calcula como $\log(1 + e^{-m})$ (equivalentemente $\operatorname{log1p}(e^{-m})$), el diminuto término e^{-m} sobrevive mucho más allá del acantilado estándar, hasta el subdesbordamiento del exponente.

El complemento de la probabilidad máxima es la cantidad crítica para el paso backward. En lugar de formar $1 - q_{\max}$ (que se cancela en el dtype de salida), la implementación usa la identidad

$$1 - q_{\max} = -\operatorname{expm1}(-s), \tag{147}$$

que es exacta para s arbitrariamente pequeño.

Cross-entropy de objetivo duro. Usando el logit objetivo como referencia, la pérdida es

$$L(z, t) = \operatorname{softplus}\bigl(\operatorname{logsumexp}_{j \neq t}(z_j - z_t)\bigr) = \log\left(1 + \sum_{j \neq t} e^{z_j - z_t}\right), \tag{148}$$

algebraicamente idéntica a $\log(1 + e^{-m})$ pero sin sumar nunca el término pequeño a uno. Escribir z_t mediante las diferencias $z_j - z_t$ hace que la diferenciación automática devuelva

$$g_t = -\frac{e^{-m}}{1 + e^{-m}}, \qquad g_j = \frac{e^{z_j - z_t}}{1 + e^{-m}} \quad (j \neq t), \tag{149}$$

de modo que el gradiente objetivo se obtiene como menos la suma de los gradientes no objetivo y $\sum_k g_k = 0$ se cumple por construcción, con cualquier margen. La magnitud $e^{-m}/(1 + e^{-m})$ se preserva hasta el subdesbordamiento del exponente — precisamente la componente que la implementación estándar redondea a cero.

Jacobianos de softmax y log-softmax. Para un softmax aislado, los valores forward de la Ecuación 146 aún pueden redondear a uno en el dtype de salida (es inevitable para la probabilidad almacenada), pero el jacobiano no debe colapsar. Dado un gradiente aguas arriba g, el producto vector–jacobiano del softmax es

$$\frac{\partial L}{\partial z} = q \odot (g - \langle g, q \rangle)\,, \tag{150}$$

cuya entrada argmax se reescribe como

$$\left.\frac{\partial L}{\partial z_a}\right|_{\text{fixed}} = q_{\max}\left(g_{\max}\,(1 - q_{\max}) - \sum_{j \neq a} g_j q_j\right), \tag{151}$$

con $1 - q_{\max}$ tomado de la Ecuación 147. La entrada verdadera del jacobiano $q_{\max}(1 - q_{\max}) \approx e^{-m}$ sobrevive así al acantilado. Para log-softmax ($\partial \operatorname{logsoftmax}_k / \partial z_j = \delta_{kj} - q_j$) la entrada argmax se reescribe igualmente como

$$\left.\frac{\partial L}{\partial z_a}\right|_{\text{fixed}} = -\sum_{j \neq a} g_j + (1 - q_{\max}) \sum_k g_k. \tag{152}$$

La Tabla 25 contrasta ambos regímenes. Los valores son las cantidades exactas de las Ecuaciones 149–151; la columna estándar es lo que produce la Ecuación 141 una vez que $m > c_d$.

Cantidad	**Estándar**	**Corregido**
$q_{\text{máx}}$	1,0 (redondeado)	e^{-s} (valores; puede redondear)
$1 - q_{\text{máx}}$	0,0 (redondeado)	$-\operatorname{expm1}(-s)$
Pérdida CE	$\log(1) = 0$	$\operatorname{softplus}(s) > 0$
g_t	0,0	$-e^{-m}/(1 + e^{-m})$
$\sum_k g_k$	$\sum_{j \neq t} q_j > 0$	0 (por construcción)
$\partial q_{\text{máx}}/\partial z_{\text{máx}}$	0,0	$q_{\text{máx}}(1 - q_{\text{máx}})$

Cuadro 25: Comportamiento de los cálculos estándar y corregido más allá del acantilado de precisión $m > c_d$. La columna corregida preserva cada cantidad exponencialmente pequeña de las Ecuaciones 149–151; la columna estándar redondea a cero los términos del objetivo/argmax.

18.3. Implementación en el Toolkit

Las correcciones viven en `src/model/numerics.py` y se exponen como dos casillas booleanas bajo el grupo de esquema `model.numerics`, ambas con valor por defecto `false`:

```
model:
  numerics:
    fixed_softmax: true          # todo softmax / log-softmax explícito
    fixed_cross_entropy: true    # pérdidas de tarea con objetivo duro
```

Ninguna bandera añade parámetros, buffers ni entradas al *state-dict*, por lo que los checkpoints, el conteo de parámetros y la exportación (PyTorch, Transformers, GGUF) no se ven afectados; las banderas viajan con la configuración del modelo. Cuando están desactivadas, las rutas de código se reducen a las llamadas originales de `torch` con una única consulta de atributo por softmax, de modo que el comportamiento y el rendimiento por defecto no cambian. Las pérdidas corregidas aceptan la misma semántica de `ignore_index` y de reducción que `torch.nn.functional.cross_entropy` y calculan en el dtype de entrada (sin promoción implícita), lo que mantiene la corrección efectiva tanto en bf16 como en fp32/fp64.

18.3.1. Paso Backward

El grafo de autograd de PyTorch para la Ecuación 146 sigue conteniendo la resta que produce $1 - q_{\text{máx}}$, y evaluarla allí reintroduce la cancelación. Por ello la implementación define las dos funciones softmax como objetos `torch.autograd.Function` cuyo `forward` calcula la Ecuación 146 y cuyo `backward` usa las Ecuaciones 150–152. La cross-entropy no necesita backward personalizado: el grafo de la Ecuación 148 ya produce la Ecuación 149 exactamente, porque z_t entra en cada logit desplazado con coeficiente -1 y la fila objetivo queda excluida del log-sum-exp.

Algorithm 47 Softmax corregido (forward + producto vector–jacobiano)

Require: logits $z \in \mathbb{R}^K$ a lo largo de un eje de normalización, gradiente aguas arriba g

$a \leftarrow \arg\max_k z_k, \quad z_{\max} \leftarrow z_a$
$\ell \leftarrow \operatorname{logsumexp}\{z_k - z_{\max} : k \neq a\}$ ▷ argmax excluido
$s \leftarrow \operatorname{softplus}(\ell)$ ▷ $= \log(1 + e^{-m})$
$c \leftarrow -\operatorname{expm1}(-s)$ ▷ $= 1 - q_{\max}$, sin cancelación
$q \leftarrow \exp(z - z_{\max} - s)$ ▷ salida forward
backward:
$\tau \leftarrow \sum_k g_k q_k, \quad \sigma \leftarrow \sum_{k \neq a} g_k q_k$
$\nabla z \leftarrow q \odot (g - \tau)$
$\nabla z_a \leftarrow q_a \, (g_a \, c - \sigma)$ ▷ entrada argmax exacta
return q; **return** ∇z

18.3.2. Superficie de Integración

Un único interruptor debe llegar a cada lugar donde el toolkit materializa probabilidades o logits. `HybridLayer` adjunta la bandera resuelta a cada mixer como `use_fixed_softmax`; la fábrica de residuales hace lo mismo para los residuales de atención en profundidad; los hooks de la caché KV reciben un argumento explícito `use_fixed_softmax`. Un helper de despacho selecciona entonces la implementación en cada punto de llamada. La Tabla 26 enumera las superficies cubiertas.

Superficie	**Puntos cubiertos**
Mixers de atención	`standard`, `titan`, `gqa`, `ode`; familia latente (`mla`, `gqla`, `mlra`, `tucker`, `iha`, `gta`, `mtla`, `cca`, `ccgqa`, `gma`); con compuerta (`gated_softmax`, `fox`); dispersos (`fasa`, `sparge`, `msa`, `nsa`, `bigbird`, `longformer`, `sparse_transformer`, `sparsek`, `swa`, `sparda`); sustituto denso de los mixers sin entrenamiento
Routers	softmax top-k de MoE y Mixture-of-Depths
Residuales	softmax en profundidad de Full y Block AttnRes
Caché KV	normalizador de recomputación/fallback en `src/model/cache/hooks.py`
Decodificación	filtrado de núcleo top-p y distribución de muestreo final
Despliegue	lectura de probabilidades de `inference.predict`
Pérdidas de entrenamiento	`mlm`, `causal_lm`, `classification`, `segmentation` (CE y softmax de Dice), `text_classification`, `patch_prediction`

Cuadro 26: Cobertura de las banderas `fixed_softmax` y `fixed_cross_entropy`. Los núcleos LSE internos de SSOG (Gaussianas separables) conservan su propia formulación: no son softmax categóricos sobre el mismo eje.

El lado de las pérdidas se despacha mediante un único helper del trainer, de modo que todas las tareas comparten la misma semántica:

```
def _task_cross_entropy(self, logits, targets, **kwargs):
    if self._use_fixed_cross_entropy():
        return fixed_cross_entropy(
            logits, targets, ignore_index=kwargs.get("ignore_index", -100))
    return F.cross_entropy(logits, targets, **kwargs)
```

18.4. Costo, Alcance y Validación

Costo. Las rutas corregidas tienen la misma complejidad asintótica que las estándar y tocan cada fila con un número constante de operaciones elementales adicionales (un log-sum-exp enmascarado y un softplus; un `expm1` en el backward). No hay multiplicaciones de matrices, ni memoria adicional de orden K más allá de temporales, ni parámetros. Con logits ordinarios, los valores corregidos y estándar coinciden dentro de la tolerancia de punto flotante; solo cambia el régimen sub-redondeo que la ruta estándar colapsa.

Alcance. Las banderas apuntan a pérdidas de clasificación con objetivo duro. La cross-entropy de objetivo blando (destilación de conocimiento, regularización KL) necesitaría la construcción de diferencias con `expm1`/`log1p` del artículo, que este toolkit no implementa porque no tiene ruta de destilación. Los núcleos LSE internos del mixer SSOG y el normalizador de atención sigmoide (que no es un softmax) se dejan intactos deliberadamente.

Validación. El repositorio incluye `tests/test_fixed_numerics.py`, que verifica (i) la paridad forward con `torch` para softmax, log-softmax y cross-entropy en fp32/fp64, incluidas entradas enmascaradas ($-\infty$) y de una sola clave; (ii) las regresiones del acantilado de la Tabla 25 con márgenes $25 > c_{32}$ y $40 > c_{64}$ — los gradientes estándar del objetivo/argmax son exactamente cero y los corregidos no, con gradiente de suma cero en el caso corregido; (iii) la semántica de `ignore_index`, reducción y despacho; (iv) la paridad forward del modelo completo (corregido frente a estándar es matemáticamente idéntico) y pasadas backward finitas; (v) la paridad prefill+decode de la caché KV con la bandera activada; y (vi) el despacho del trainer. El arnés combinatorio de extremo a extremo entrena además modelos MLM y causal-LM reales con `fixed_softmax`, `fixed_cross_entropy`, ambos juntos y la interacción con SSMax y BitNet, todo en CPU. Un preset listo para ejecutar es `configs/examples/numerics_fixed.yaml`.

Cuándo activarlo. La corrección importa cuando los márgenes de logits pueden crecer mucho durante el entrenamiento: conjuntos algorítmicos pequeños que exhiben *grokking*, entrenamientos largos con decaimiento de pesos y optimizadores adaptativos, y cualquier régimen en el que la pérdida se vuelva extremadamente confiada. En el entrenamiento ordinario a gran escala la ruta estándar suele ser indistinguible; la corregida es una candidata segura por defecto porque es matemáticamente idéntica en la pasada forward y está estrictamente mejor condicionada en la backward.

19. Conformer: Atención Aumentada con Convolución

19.1. Resumen Ejecutivo

El bloque Conformer [45] es el mezclador de secuencia aumentado con convolución del conjunto de herramientas, expuesto como `conformer_attn`. Conserva el campo receptivo global de la atención de producto punto escalada y añade una rama de convolución depthwise que modela estructura local de banda estrecha—el sesgo inductivo que necesita el reconocimiento de voz y que la atención pura no aporta. El bloque intercala atención multi-cabeza (MHSA) con codificación posicional sinusoidal relativa entre dos capas feed-forward de medio paso estilo Macaron [78], e inserta el módulo de convolución entre la atención y la segunda FFN. En la receta de referencia alcanza 1,9 %/3,9 % WER en LibriSpeech *test-clean*/*test-other* sin modelo de lenguaje externo (Conformer-L, 118.8M parámetros), y es el mezclador recomendado para la clase de modelo de audio del Anexo 20. El conjunto de herramientas ejecuta ambas capas feed-forward a paso completo (la variante "full-step" de la Tabla 5 del paper) y deja que la capa exterior posea la segunda; la Tabla 27 lista las desviaciones.

El mezclador está disponible en todas las clases de modelo: encoder de texto, decoder causal (donde el padding depthwise se desplaza para seguir siendo seguro en streaming), Vision Transformer y `frankenstein_audio`. Como el bloque vive dentro de `HybridLayer`, compone con los 45 mezcladores, las 6 normas, las 4 estrategias de residual, MoE, Mixture-of-Depths, mHC, AttnRes, BitNet y las 23 familias de optimizadores.

19.2. Conformer: Atención Aumentada con Convolución

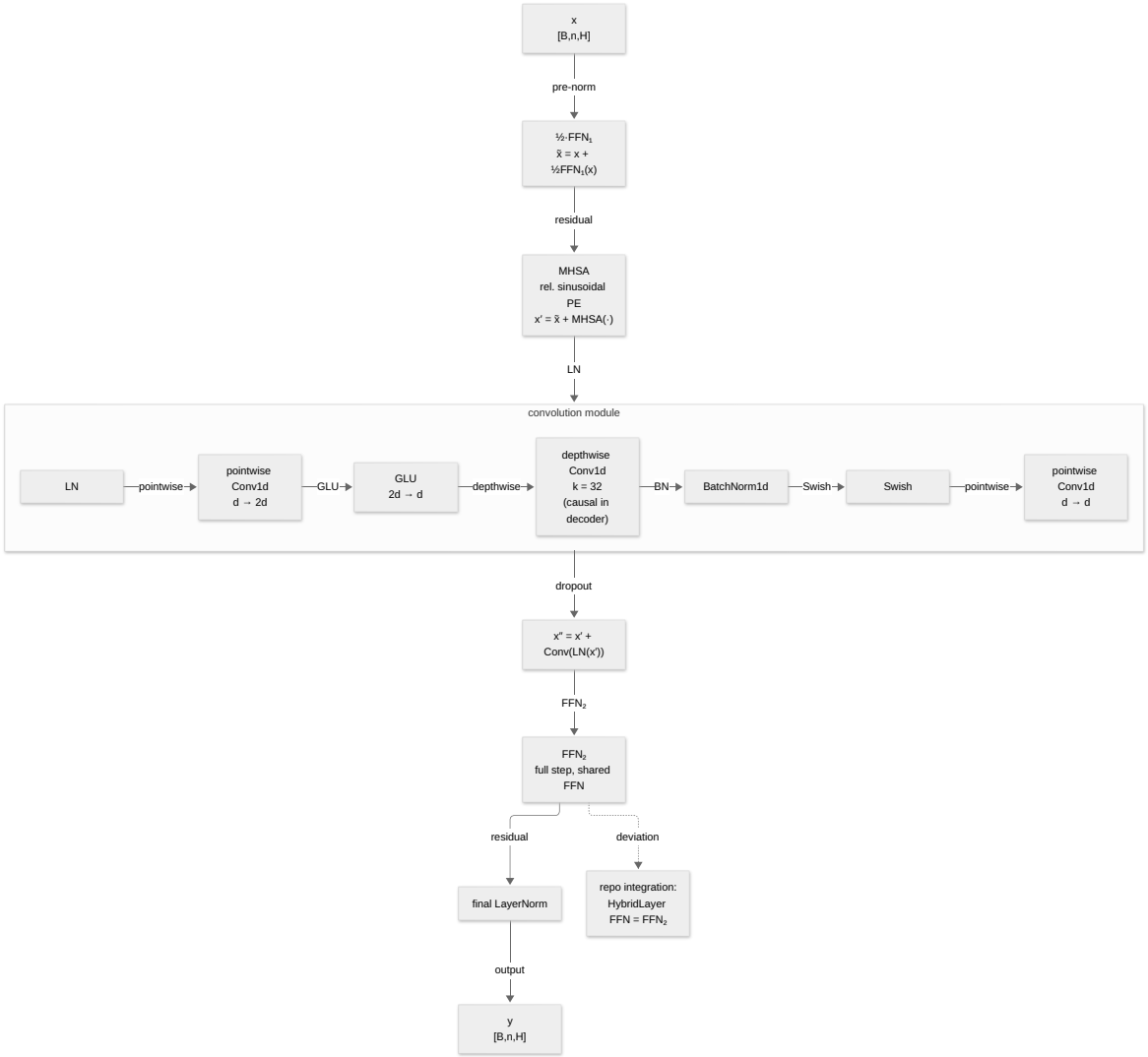


Figura 77: Flujo de datos del bloque Conformer: una capa feed-forward de medio paso Macaron, atención multi-cabeza con codificación posicional sinusoidal relativa, el módulo de convolución (pointwise → GLU → depthwise → BatchNorm → Swish → pointwise) y la segunda capa feed-forward seguida de la LayerNorm final. En este conjunto de herramientas la FFN compartida de `HybridLayer` ocupa la ranura FFN_2 con un paso residual completo (anotación punteada).

19.2.1. Formulación Matemática

Para una entrada de bloque $x \in \mathbb{R}^{B\times n\times d}$, el bloque de referencia de la Fig. 1 de Gulati et al. [45] es

$$\tilde{x} = x + \tfrac{1}{2}\,\mathrm{FFN}_1\big(\mathrm{LN}(x)\big), \tag{153}$$
$$x' = \tilde{x} + \mathrm{MHSA}\big(\mathrm{LN}(\tilde{x})\big), \tag{154}$$
$$x'' = x' + \mathrm{Conv}\big(\mathrm{LN}(x')\big), \tag{155}$$
$$y = \mathrm{LN}\Big(x'' + \tfrac{1}{2}\,\mathrm{FFN}_2\big(x''\big)\Big). \tag{156}$$

Los dos términos feed-forward llevan el coeficiente de *medio paso* de Macaron: cada uno aporta la mitad de una actualización residual completa, uno antes y otro después del par atención–convolución.

Módulo feed-forward. Con ancho oculto d_{ff},

$$\mathrm{FFN}(x) = W_2\,\mathrm{Dropout}\big(\mathrm{Swish}(W_1\,\mathrm{LN}(x) + b_1)\big) + b_2, \qquad \mathrm{Swish}(a) = a\,\sigma(a), \tag{157}$$

donde $W_1 \in \mathbb{R}^{d_{ff}\times d}$, $W_2 \in \mathbb{R}^{d\times d_{ff}}$ y σ es la sigmoide logística. El paper usa $d_{ff} = 4d$.

Atención multi-cabeza con codificación posicional relativa. Las consultas, claves y valores son proyecciones lineales de la entrada normalizada del bloque, $Q = \mathrm{LN}(\tilde{x})W_Q$, $K = \mathrm{LN}(\tilde{x})W_K$, $V = \mathrm{LN}(\tilde{x})W_V$, divididas en H cabezas de ancho $d_h = d/H$. Siguiendo el esquema relativo de Transformer-XL [24], la puntuación de la consulta i contra la clave j suma una función sinusoidal del *desplazamiento* $i - j$ en lugar de una posición absoluta:

$$e_{ij} = q_i^\top k_j + (q_i + v)^\top R_{i-j}, \qquad A_{ij} = \mathrm{softmax}_j\Big(\frac{e_{ij}}{\sqrt{d_h}}\Big), \qquad O = AVW_O, \tag{158}$$

con v un vector de sesgo aprendido y, para $m = 0, \ldots, d_h/2 - 1$,

$$R_{\delta,\,2m} = \sin\big(\delta \,/\, 10000^{2m/d_h}\big), \qquad R_{\delta,\,2m+1} = \cos\big(\delta \,/\, 10000^{2m/d_h}\big). \tag{159}$$

Como la codificación depende solo de $\delta = i - j$, el bloque generaliza entre longitudes de utterance. El conjunto de herramientas canaliza la información posicional por la maquinaria de todo el modelo que usan los demás mezcladores: `model.positional_encoding` (RoPE por defecto) se aplica a Q/K, y `conformer_attn_use_pe` (`true` por defecto) permite que una capa lo omita. El esquema sinusoidal relativo del paper se aproxima mejor con `sinusoidal_rotary` (frecuencias sinusoidales aplicadas como rotación sobre Q/K, relativo por construcción); `alibi`/`bam` aportan un sesgo de distancia en lugar de una rotación. No se implementa un módulo sinusoidal relativo dedicado estilo Transformer-XL. En modo decoder la matriz de puntuaciones recibe la máscara causal habitual antes del softmax, y se usa la ruta de softmax fijo del Anexo 18.3 cuando `numerics.fixed_softmax` está habilitado.

Módulo de convolución. El módulo aplica una convolución depthwise con compuerta sobre el eje temporal:

$$h = \mathrm{LN}(x'), \tag{160}$$
$$[u,\, g] = \mathrm{Conv1d}_{d\to 2d}(h), \qquad \tilde{h} = u \odot \sigma(g), \tag{161}$$
$$z_t = \sum_{k=0}^{K-1} w_k\, \tilde{h}_{t+k-\lfloor K/2\rfloor}, \tag{162}$$
$$c = \mathrm{Dropout}\Big(\mathrm{Conv1d}_{d\to d}\big(\mathrm{Swish}(\mathrm{BN}(z))\big)\Big), \tag{163}$$

donde $\odot$ es el producto elemento a elemento (una GLU que reduce los $2d$ canales a d), $w \in \mathbb{R}^K$ es el kernel depthwise y BN normaliza cada canal sobre los ejes de batch y tiempo,

$$\mathrm{BN}(z)_{b,t,c} = \gamma_c \frac{z_{b,t,c} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}} + \beta_c. \tag{164}$$

En modo decoder la suma de la Ecuación 162 se desplaza a $z_t = \sum_{k=0}^{K-1} w_k \tilde{h}_{t-K+1+k}$, de modo que ninguna trama futura entra en la convolución; los modos encoder y ViT conservan el padding simétrico "same" del paper.

19.2.2. Pseudocódigo Algorítmico

Algorithm 48 Bloque Conformer (`conformer_attn`)

Require: entrada de bloque $x \in \mathbb{R}^{B\times n\times d}$, kernel $K = 32$, cabezas H, bandera Macaron, modo
Ensure: delta de bloque Δ tal que $x + \Delta$ entra en la $\mathrm{norm}_2 + \mathrm{FFN}_2$ exterior

$\tilde{x} \leftarrow x + \mathrm{FFN}_1(\mathrm{LN}(x))$ ▷ FFN_1 Macaron ($\frac{1}{2}$ en el paper)
$Q, K_{proj}, V \leftarrow \mathrm{LN}(\tilde{x})W_Q,\ \mathrm{LN}(\tilde{x})W_K,\ \mathrm{LN}(\tilde{x})W_V$
$e_{ij} \leftarrow q_i^\top k_j + (q_i + v)^\top R_{i-j}$ ▷ PE sinusoidal relativa, Ec. 158
if modo = decoder **then**
    $e_{ij} \leftarrow -\infty$ para $j > i$ ▷ máscara causal
end if
$x' \leftarrow \tilde{x} + \mathrm{softmax}(e/\sqrt{d_h})\, V\, W_O$
$h \leftarrow \mathrm{LN}(x')$
$[u, g] \leftarrow \mathrm{Conv1d}_{d\to 2d}(h);\quad \tilde{h} \leftarrow u \odot \sigma(g)$ ▷ GLU
$z \leftarrow \mathrm{DepthwiseConv1d}_K(\tilde{h})$ ▷ desplazamiento causal en modo decoder
$c \leftarrow \mathrm{Dropout}(\mathrm{Conv1d}_{d\to d}(\mathrm{Swish}(\mathrm{BN}(z))))$
$x'' \leftarrow x' + c$
return x'' ▷ `HybridLayer` añade FFN_2 y la LN final

19.2.3. Mapeo de Implementación y Desviaciones

La Tabla 27 mapea el bloque del paper sobre el conjunto de herramientas. Las desviaciones sustantivas son los dos escalados residuales y la segunda capa feed-forward: el paper conserva ambos coeficientes $\frac{1}{2}$ de Macaron y envuelve la FFN_2 en el bloque, mientras que `conformer_attn` usa pasos completos y devuelve el estado tras el módulo de convolución, dejando que el `HybridLayer` exterior aplique su $\mathrm{norm}_2 + \mathrm{FFN}$ compartida (también a paso completo). La descomposición es deliberada: mantiene disponibles para la segunda FFN el enrutamiento MoE, Mixture-of-Depths, los flujos mHC/Hyperloop, AttnRes, la fusión FlashNorm, las proyecciones BitNet y el enrutamiento de optimizadores por grupo de parámetros, nada de lo cual sobreviviría a una FFN privada dentro del mezclador.

Componente del paper	Implementación en el toolkit	Notas
$\frac{1}{2}$FFN$_1$ (pre-norm)	FFN privada dentro de `ConformerAttention`; Swish; residual a paso completo	`conformer.macaron: true` (por defecto); el coeficiente $\frac{1}{2}$ del paper se omite; deshabilitarla deja solo atención + convolución
MHSA, PE sinusoidal relativa	proyecciones propias $q/k/v/out$; PE de todo el modelo vía `conformer_attn_use_pe` (`true` por defecto)	la sinusoidal relativa del paper $\approx$ `sinusoidal_rotary`; sin módulo Transformer-XL dedicado (RoPE es el valor por defecto del modelo)
Módulo de convolución	LN $\rightarrow$ pointwise $\rightarrow$ GLU $\rightarrow$ depthwise $\rightarrow$ BN $\rightarrow$ Swish $\rightarrow$ pointwise $\rightarrow$ dropout	`conformer.kernel_size` (32), `conformer.conv_norm` (`batch`/`group`/`layer`)
$\frac{1}{2}$FFN$_2$ a medio paso	norm$_2$ + FFN de `HybridLayer`, paso completo	justificado por la Tabla 5 de Gulati et al. [45]: 1,9/4,4 vs. 1,9/4,5 WER dev (ambos coeficientes omitidos)
LayerNorm final	norma final de `HybridLayer`	idéntica a la de cualquier otro mezclador
Submuestreo convolucional 4$\times$	frontend de `frankenstein_audio`	Anexo 20; texto y ViT usan sus propios embeddings
Dropout ($p = 0{,}1$)	`model.dims.dropout`	global, por bloque

Cuadro 27: Mapeo de implementación del bloque Conformer. Las desviaciones son los escalados Macaron $\frac{1}{2}$ omitidos, la segunda FFN (aportada por la capa exterior a paso completo) y el cableado de la codificación posicional.

Por qué la FFN a paso completo es segura. Omitir el $\frac{1}{2}$ de Macaron cambia la escala residual de cada rama, no la forma expresiva del bloque: la variante de paso completo es exactamente la que Gulati et al. [45] ablacionan en su Tabla 5, donde mueve el WER dev-clean/dev-other de 1,9/4,4 (medio paso) a 1,9/4,5 (paso completo), es decir, una décima. El conjunto de herramientas usa pasos completos en ambas capas feed-forward; la primera permanece privada en el mezclador y la segunda es la compartida, lo que preserva la composabilidad descrita arriba.

19.3. Ablaciones Publicadas

La Tabla 28 resume las ablaciones que motivan la configuración por defecto. Los valores son los reportados por Gulati et al. [45]; el orden cualitativo es lo que codifican los valores por defecto.

Ablación	Configuraciones comparadas	Resultado reportado
Módulo de convolución (Tabla 3)	presente vs. eliminado	la mayor degradación individual del bloque al eliminarlo; la rama local es portante, no decorativa
Par FFN Macaron (Tabla 3)	$\frac{1}{2}\text{FFN}_1 + \frac{1}{2}\text{FFN}_2$ vs. una FFN	eliminar el par cuesta aproximadamente 0,2 dev-clean / 0,5 dev-other WER
Medio vs. paso completo (Tabla 5)	residuales $\frac{1}{2}$ vs. completos	1,9/4,4 vs. 1,9/4,5 WER dev: como máximo 0,1 punto
Tamaño de kernel (Tabla 7)	$K \in \{3, 7, 17, 32, 65\}$	$K = 32$ es el mejor; kernels pequeños subajustan el contexto local, $K = 65$ añade costo sin ganancia
Ubicación de la convolución	tras MHSA, antes de FFN_2	se conserva el orden de referencia; mover la rama no lo mejora

Cuadro 28: Ablaciones del Conformer y los valores por defecto que justifican (`kernel_size: 32`, `macaron: true`, FFN_2 compartida a paso completo).

19.4. Reutilización de la Infraestructura Existente

Como `ConformerAttention` devuelve un delta y hereda el constructor `mixer(config, pos_encoder=...)`, toda la pila se reutiliza sin casos especiales:

- **Clases de modelo:** encoder de texto (`mlm`/`sbert`/`text_classification`), decoder causal (`causal_lm`), Vision Transformer (`patch_prediction`/`classification`/`segmentation`) y `frankenstein_audio` (`asr`, `audio_classification`).
- **Normalización:** las 6 variantes, incluida pRMSNorm y la fusión FlashNorm en $\text{norm}_1/\text{norm}_2$; el módulo de convolución conserva su BatchNorm salvo que `conformer.conv_norm` seleccione group o layer normalization.
- **Enrutamiento y residuales:** MoE, Mixture-of-Depths, mHC, Hyperloop, AttnRes y las proyecciones ternarias BitNet (la FFN privada y las proyecciones de la convolución son compatibles con BitLinear).
- **Optimizadores:** las 23 familias con prefijos por grupo; los parámetros del mezclador, incluido el kernel depthwise y las normas de la convolución, se enrutan al grupo `attention`.
- **Caché KV:** `conformer_attn` se registra como *no* cacheable (`supported=False`), porque el estado de la convolución depthwise y las estadísticas de BatchNorm deberían acarrearse entre pasos. `generate()` cae a la ruta de recómputo completo, que es exacta.

19.5. Cuándo Usarlo

- **Voz:** mezclador por defecto para `asr` y `audio_classification`; con tramas de 40 ms tras el submuestreo 4×, el kernel de 32 abarca ∼1.28 s de contexto, aproximadamente el ancho de una secuencia de fonos.
- **Texto y visión:** sustituto sólido de `standard_attn` cuando los datos tienen estructura local (modelos a nivel de carácter, código, OCR, secuencias de parches); cuesta una convolución depthwise y dos normas extra por capa frente a un bloque de atención plano.
- **Decodificación en streaming:** usable en `mode: decoder` con padding convolucional causal, pero sin la caché incremental del Anexo 17; la recurrencia depthwise es la pieza que falta.

20. Audio: Reconocimiento de Voz y Clasificación de Audio

20.1. Resumen Ejecutivo

`frankenstein_audio` es la cuarta clase de modelo del conjunto de herramientas. Posee un frontend de audio—bancos de filtros log-mel o un espectrograma crudo, SpecAugment opcional [88] y dos convoluciones con stride 2 que reducen la tasa de tramas 4×—y luego ejecuta la misma pila de `HybridLayer` que el encoder de texto sobre la secuencia de tramas resultante. Como la pila es agnóstica de modalidad, todos los mezcladores de secuencia (incluido el Conformer del Anexo 19), normas, estrategias de residual, codificaciones posicionales, optimizadores y bloques de enrutamiento están disponibles sin cambios. La clase soporta dos tareas: **reconocimiento automático de voz** (`asr`, con pérdida CTC [42] o RNN-Transducer [41]) y **clasificación de audio** (`audio_classification`, entropía cruzada a nivel de clip). El mismo frontend, con `audio.feature: raw`, acepta señales de sensores multicanales unidimensionales como EEG, EMG o acelerómetro para clasificación.

La receta de referencia es el frontend Conformer de Gulati et al. [45]: bancos de filtros log-mel de 80 canales con ventana de 25 ms y salto de 10 ms, dos convoluciones 3×3 con stride 2, una proyección lineal al ancho del modelo y SpecAugment con ancho de frecuencia $F = 27$, diez máscaras temporales y una razón máxima de máscara temporal $p_S = 0{,}05$. Los tres tamaños de referencia (S/M/L) se reproducen en la Tabla 29.

20.2. La Clase de Modelo `frankenstein_audio`

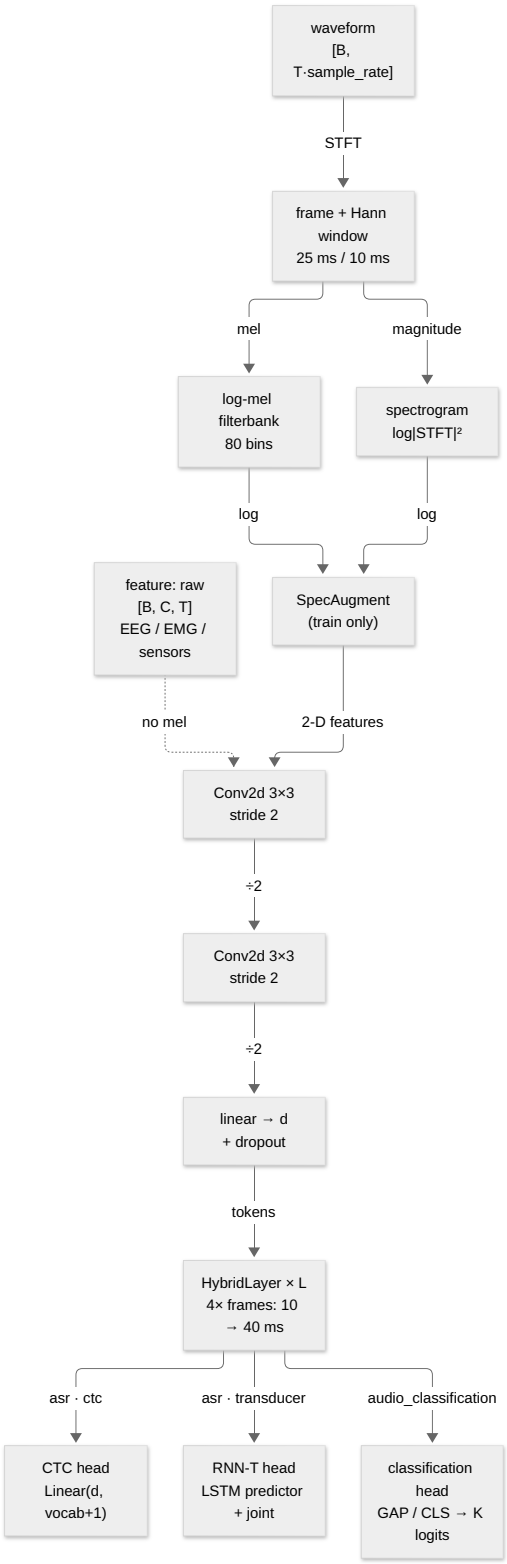


Figura 78: Flujo de datos del frontend de audio: waveform → enmarcado 25 ms/10 ms → log-mel de 80 bins (o espectrograma, o tramas multicanal crudas) → SpecAugment (solo entrenamiento) → dos etapas Conv2d con stride 2 (4×) → proyección lineal → pila `HybridLayer` → cabeza CTC, RNN-T o de clasificación.

20.2.1. Frontend de Audio

El módulo frontend convierte un lote de formas de onda $w \in \mathbb{R}^{B\times T}$ (o un lote de clips multicanal $w \in \mathbb{R}^{B\times C\times T}$ en modo `raw`) en un mapa de características:

1. **Enmarcado:** la forma de onda se corta en ventanas solapadas de `window_ms` = 25 ms cada `stride_ms` = 10 ms, con ventana de Hann, a `sample_rate` = 16000 Hz. A la tasa de referencia esto produce tramas de 400 muestras con salto de 160, es decir, 100 tramas por segundo.

2. **Representación espectral** (`audio.feature`):

   - `logmel` (por defecto): banco de filtros mel de 80 bins (Ecuación 166) seguido de compresión logarítmica;
   - `spectrogram`: $\log|\mathrm{STFT}(w)|^2$ sin proyección mel;
   - `raw`: sin transformada espectral; el clip se entrega como mapa $C \times T$ (Sección 20.8).

3. **SpecAugment** [88]: se enmascaran F canales de frecuencia por clip y hasta `time_masks` bandas temporales de ancho máximo $p_S \cdot T$, ambas con relleno de ceros (solo entrenamiento).

El banco de filtros mel es

$$M_k(f) = \max\Big(0,\ \min\Big(\tfrac{f-f_{k-1}}{f_k-f_{k-1}},\ \tfrac{f_{k+1}-f}{f_{k+1}-f_k}\Big)\Big), \tag{165}$$

para $K = 80$ filtros triangulares espaciados en la escala mel, $\mathrm{mel}(f) = 2595\log_{10}(1 + f/700)$, y la característica log-mel es

$$x_k = \log\Big(\sum_f M_k(f)\,|X(f)|^2 + \epsilon\Big). \tag{166}$$

Cuando `torchaudio` es importable, el frontend usa `torchaudio.transforms.MelSpectrogram` para la Ecuación 166; en caso contrario recurre a una implementación en PyTorch puro construida sobre `torch.stft` y una matriz de filtros en caché. Ambas rutas son numéricamente equivalentes salvo la convención del banco de filtros (espaciado mel HTK vs. Slaney), y la ruta en PyTorch puro es la que mantiene la CI solo-CPU libre del extra opcional `audio`.

20.2.2. Submuestreo Convolucional

Dos etapas convolucionales reducen la tasa de tramas 4× (10 ms → 40 ms), que es lo que convierte un utterance de 30 segundos en una secuencia de ∼750 tokens en lugar de 3000:

$$F = \mathrm{Dropout}\Big(W_p\,\mathrm{Conv2d}_{3\times3,\,s=2}\big(\mathrm{Conv2d}_{3\times3,\,s=2}(X)\big)\Big), \qquad W_p \in \mathbb{R}^{d\times d_c}, \tag{167}$$

donde X es el mapa de características `n_mels` × T' con `conv_channels` canales tras la primera etapa, y W_p proyecta los canales aplanados al ancho del modelo d. Ambas etapas son fijas, con kernels 3×3 y stride 2; `audio.conv_channels` (32) fija el ancho de canales de la primera etapa y `audio.n_fft` (400) el tamaño de la STFT.

20.2.3. Modo Encoder Forzado y la Pila HybridLayer

`frankenstein_audio` fuerza `mode: encoder`, exactamente como `frankenstein_vit`: las tareas de audio son bidireccionales (un encoder CTC o transducer debe ver tramas pasadas y futuras), y el padding convolucional del Conformer es simétrico en modo encoder. La clase refleja el patrón envoltorio del ViT—frontend, pila de `HybridLayer`, cabeza de tarea—y su bucle de capas conserva los mismos hooks, de modo que AttnRes, mHC, Hyperloop, profundidad en bucle, MoE, Mixture-of-Depths y BitNet funcionan sin código específico de audio. Las cabezas de tarea se construyen tras la normalización final:

- **CTC:** Linear$(d, V+1)$ sobre la secuencia de tramas, donde la clase extra es el blank.
- **RNN-T:** una red de predicción LSTM de una capa sobre las etiquetas no-blank previas (`audio.asr_decoder_dim`, `asr_decoder_layers`, `asr_decoder_dropout`), y una red conjunta $z_{t,u} = \tanh(W_e h_t + W_p h_u + b)$, $\text{logits}_{t,u} = W_o z_{t,u}$.
- **Clasificación:** un vector agrupado (global average pooling, o un token `[CLS]` aprendido antepuesto a la secuencia de tramas, según `pooling_mode`) proyectado a `num_classes` logits.

20.3. Reconocimiento Automático de Voz

20.3.1. CTC

La clasificación temporal conexionista [42] marginaliza sobre todas las alineaciones a nivel de trama que colapsan a la secuencia objetivo y bajo el mapa muchos-a-uno $\mathcal{B}$ (fusionar repeticiones, borrar blanks):

$$\mathcal{L}_{\text{CTC}} = -\log \sum_{\pi \in \mathcal{B}^{-1}(y)} \prod_{t=1}^{T} p(\pi_t \mid x), \tag{168}$$

calculada con la implementación `ctc_loss` de `torch.nn.functional` en espacio logarítmico con `zero_infinity=False`, para que las alineaciones imposibles no envenenen el gradiente.

20.3.2. RNN-Transducer

La pérdida RNN-T [41] suma sobre alineaciones de *etiqueta-y-blank* en la retícula $T \times (U+1)$. Con $\alpha_{t,u}$ la log-probabilidad del prefijo $y_{1:u}$ tras t tramas de encoder, la recursión forward es

$$\alpha_{t,u} = \text{logsumexp}\Big(\underbrace{\alpha_{t-1,u} + \log p(\varnothing \mid t-1, u)}_{\text{avanzar tiempo}}, \underbrace{\alpha_{t,u-1} + \log p(y_u \mid t, u-1)}_{\text{emitir etiqueta}} \Big), \tag{169}$$

con $\alpha_{0,0} = 0$ y $-\infty$ en el resto, $p(\cdot \mid t, u) = \text{softmax}(\text{logits}_{t,u})$, y

$$\mathcal{L}_{\text{RNN-T}} = -\alpha_{T,U}. \tag{170}$$

La implementación es compatible con autograd en PyTorch puro (programación dinámica en dominio logarítmico sobre la retícula, con el gradiente obtenido por la recursión backward estándar), y toma la ruta rápida `torchaudio.functional.rnnt_loss` cuando la dependencia opcional está instalada. La retícula es $O(T \cdot U)$ en memoria, por lo que el predictor se mantiene superficial (`audio.asr_decoder_layers`, por defecto 1) y estrecho (`audio.asr_decoder_dim`, 320 por defecto). La pérdida se selecciona con `audio.asr_loss` (`ctc` por defecto).

20.3.3. Decodificación y Tasa de Error

La decodificación CTC voraz toma el argmax por trama, colapsa repeticiones y elimina blanks. La decodificación RNN-T voraz alterna una emisión de etiqueta con un avance de trama hasta que el blank puntúa más alto que la siguiente etiqueta. Ambas se evalúan con la tasa de error de caracteres

$$\text{CER} = \frac{S + D + I}{N}, \tag{171}$$

donde S, D, I son las sustituciones, deleciones e inserciones del alineamiento de Levenshtein contra la transcripción de referencia y N es la longitud de referencia.

20.4. Clasificación de Audio

La clasificación reutiliza las convenciones de pooling y cabeza del Vision Transformer (Anexo 13): `pooling_mode: cls` antepone un token aprendido y lee la posición 0, `pooling_mode: gap` (el valor por defecto) promedia las salidas de las tramas. La cabeza es un Linear(d, K) inicializado en cero, entrenado con entropía cruzada y el mismo control de suavizado de etiquetas que las tareas de visión. SpecAugment se aplica al mapa de características antes del submuestreo, de modo que aumenta la entrada, nunca la secuencia de tokens.

20.5. SpecAugment

Siguiendo a Park et al. [88] en su configuración "LibriSpeech basic", los dos tipos de máscara se aplican de forma independiente por clip:

$$\Delta f \sim \mathcal{U}[0, F], \quad f_0 \sim \mathcal{U}[0, \nu - \Delta f], \qquad \Delta t \sim \mathcal{U}[0, p_S \cdot T], \quad t_0 \sim \mathcal{U}[0, T - \Delta t], \tag{172}$$

con $F = 27$, `time_masks` $= 10$ y $p_S = 0{,}05$, cada máscara poniendo a cero un rectángulo $\Delta f \times \Delta t$. La *deformación temporal* (time warping), la tercera transformación de SpecAugment, no está implementada: requiere una deformación del eje temporal por muestra que interactúa mal con el bucketing de longitud fija del dataset. El enmascaramiento de frecuencia y de tiempo concentran la mayor parte del beneficio de regularización en las propias ablaciones del paper, y ambos se desactivan en evaluación.

20.6. Tamaños de Referencia

Variante	Capas	d	Cabezas	Kernel	Paráms.	RNN-T d_{dec}
Conformer-S	16	144	4	32	10.3M	320
Conformer-M	17	256	4	32	30.7M	640
Conformer-L	17	512	8	32	118.8M	640

Cuadro 29: Tamaños de referencia del Conformer de Gulati et al. [45] (Tabla 1), que siguen los presets de ejemplo. El ancho del decoder RNN-T solo se usa cuando `audio.asr_loss: transducer`.

20.7. Configuraciones de Ejemplo

Los ejemplos de ASR usan `astroza/chilean-jokes-festival-de-vina` (audio en español a 16 kHz con transcripciones, 50 filas), un dataset lo bastante pequeño para ejecutarse de extremo a extremo en CPU como prueba de humo. El ejemplo de clasificación recurre al `DummyAudioDataset` sintético cuando no hay dataset accesible.

```
model_class: frankenstein_audio
model:
  dims: {hidden_size: 144, num_layers: 16, num_heads: 4}
  dims.layer_pattern: [conformer_attn, ...]   # 16 entradas
  attention.conformer: {kernel_size: 32, macaron: true, conv_norm: batch}
audio:
  feature: logmel
  sample_rate: 16000
  n_mels: 80
  window_ms: 25
  stride_ms: 10
  n_fft: 400
```

```
  conv_channels: 32
  asr_loss: ctc          # o transducer
  asr_decoder_dim: 320
  asr_decoder_layers: 1
  asr_decoder_dropout: 0.1
  spec_augment: {enabled: true, freq_mask_width: 27,
                 time_masks: 10, time_mask_ratio: 0.05}
audio_dataset:
  dataset_name: astroza/chilean-jokes-festival-de-vina
  split: test
  audio_column: audio
  text_column: text
training:
  task: asr
  asr: {loss: ctc}       # sobrescritura opcional de audio.asr_loss
```

SpecAugment forma parte del frontend acústico ("`audio.spec_augment`"), por lo que se aplica al mapa de características antes del submuestreo y solo está activo en entrenamiento. El bloque opcional `training.asr.loss` sobrescribe `audio.asr_loss` para presets que quieran alternar entre CTC y RNN-T sin editar el bloque de arquitectura; ambos deben coincidir porque la cabeza transducer se construye a partir del parámetro del modelo.

20.8. Señales 1-D Genéricas: EEG y Más Allá

El frontend no está atado a la voz. Con `audio.feature: raw`, el banco de filtros mel y SpecAugment se omiten y el clip se trata como un mapa multicanal $C \times T$, con `audio.in_channels: C`. Las dos convoluciones con stride 2 actúan entonces como un banco de filtros aprendido sobre el tiempo (y, como efecto secundario, sobre el eje de canales), la proyección lineal produce tokens de ancho d, y la pila de `HybridLayer` ve una secuencia de tramas exactamente como en voz. Esto hace usable la clase, al menos para **clasificación**, en:

- **EEG:** C electrodos $\times$ T muestras por época, etiquetadas con un estado cognitivo, fase de sueño o patología; los electrodos juegan el papel de los canales mel.
- **EMG / ECG / EDA:** uno o pocos canales, etiquetados con gesto, actividad o clase de anomalía.
- **Acelerómetro / IMU:** $C \in \{3, 6, 9\}$ ejes, etiquetados con actividad o estado del dispositivo.
- **Sensores industriales y ambientales:** trazas de vibración, presión o emisión acústica, etiquetadas por ventana.

Qué se conserva. Todo lo que está aguas abajo del frontend: los 45 mezcladores (Conformer es un buen valor por defecto porque su rama de convolución coincide con la morfología local de las bioseñales), las 6 normas, AttnRes/mHC/MoE/MoD, los 23 optimizadores, la receta de encoder congelado de `text_classification` y las rutas de checkpoint y despliegue. El bucketing por longitud, las máscaras de padding y la evaluación tipo CER son del lado del dataset y no dependen de la modalidad.

Limitaciones. El modo raw es una comodidad, no un reemplazo de un frontend de dominio:

1. **Submuestreo del eje de canales.** Las dos etapas `Conv2d` con stride 2 son fijas y reducen a la mitad ambos ejes espaciales, así que un montaje con menos de cuatro canales pierde su

eje de canales por completo; mantenga $C \geq 4$ (o proyecte el montaje a un mapa más ancho primero).

2. **Sin geometría del montaje.** Las posiciones de los electrodos, los montajes referenciales vs. bipolares y las correlaciones inter-canal no se codifican; la convolución ve los canales como una pila sin orden, a diferencia de los modelos específicos de EEG con priors topográficos o de grafo.

3. **Sin prior espectral.** `logmel` y `spectrogram` asumen límites de banda y espaciado mel que no aplican por debajo de 20 Hz ni por encima de 8 kHz; el modo raw evita el desajuste pero también pierde el sesgo inductivo alineado a la banda.

4. **Solo clasificación.** CTC y RNN-T requieren una secuencia objetivo a nivel de token. El etiquetado secuencia-a-secuencia de ventanas de sensores (p. ej. secuencias de fases de sueño) queda fuera de alcance; use clasificación por ventana y un decodificador posterior.

5. **Estadísticas y normalización.** La BatchNorm del módulo de convolución usa estadísticas por lote; en EEG la escala de amplitud varía por sujeto y sesión, por lo que se recomienda normalizar por canal en el dataset en lugar de confiar en el frontend.

6. **Supuestos de tasa de muestreo.** `sample_rate` se usa para convertir `window_ms`/`stride_ms` en muestras; para señales crudas ajústelo a la tasa real de adquisición y elija la ventana en milisegundos en consecuencia.

20.9. Pseudocódigo en PyTorch

Entrada: formas de onda w, longitudes ℓ, transcripciones y (ASR) o etiquetas c (clasificación)
$F \leftarrow \texttt{frontend}(w)$ ▷ log-mel / espectrograma / raw, $(B, C, 80, T')$
$F \leftarrow \texttt{spec_augment}(F)$ ▷ solo entrenamiento
$F \leftarrow \texttt{conv2d}_{3\times3,s=2}(F)$; $F \leftarrow \texttt{conv2d}_{3\times3,s=2}(F)$
$x \leftarrow \texttt{dropout}(\texttt{linear}(F))$ ▷ tokens $(B, T'/4, d)$
$x \leftarrow x + \texttt{pos_encoder}(x)$ ▷ posiciones de trama, no bins mel
for cada `layer` en `layers` **do**
    $x \leftarrow \texttt{layer}(x)$ ▷ `HybridLayer`, modo encoder forzado
end for
$x \leftarrow \texttt{final_norm}(x)$
if tarea = asr y pérdida = ctc **then**
    logits $\leftarrow \texttt{ctc_head}(x)$; $\mathcal{L} \leftarrow \texttt{ctc_loss}(\log\text{softmax}(\text{logits}), y, \ell/4)$
else if tarea = asr y pérdida = transducer **then**
    $h^{pred} \leftarrow \texttt{predictor}(y)$; $\text{logits}_{t,u} \leftarrow \texttt{joint}(x_t, h_u^{pred})$
    $\mathcal{L} \leftarrow \texttt{rnnt_loss}(\text{logits}, y, \ell/4)$
else
    logits $\leftarrow \texttt{class_head}(\texttt{pool}(x))$; $\mathcal{L} \leftarrow \text{CE}(\text{logits}, c)$
end if

20.10. Desviaciones y Notas de Implementación

- **torchaudio opcional.** El frontend prefiere `torchaudio.transforms` pero nunca lo requiere; la alternativa en PyTorch puro mantiene la CI solo-CPU y el extra `cpu` libres de torchaudio (decisión D7 de la guía de arquitectura).

- **Deformación temporal.** No implementada (la Ecuación 172 cubre solo enmascaramiento de frecuencia y tiempo); es la transformación que el paper usa menos.

- **Predictor superficial.** La red de predicción RNN-T es una sola capa LSTM por defecto (`audio.asr_decoder_layers`), cambiando el predictor más profundo del paper por una retícula $O(T \cdot U)$ que cabe en memoria con los tamaños de lote de referencia.
- **Exportación.** Las exportaciones a HuggingFace y GGUF rechazan explícitamente los artefactos de audio, como hacen con SBERT: no existe un mapeo upstream `FrankensteinForAudioClassification`. `deploy -format standard|quantized` funciona sin cambios.
- **Caché.** No hay caché de encoder incremental para audio; ASR consume un utterance completo por forward.

21. Embeddings de Tokens Dinámicos (EELBERT)

21.1. Resumen Ejecutivo

`embedding.dynamic` reemplaza la tabla de embedding de entrada entrenable $V \times H$ por un *hash de agrupación de n-gramas* calculado al vuelo sobre la cadena del token, siguiendo EELBERT [22]. La tabla de búsqueda desaparece del conteo de parámetros, de los checkpoints y de los artefactos nativos de despliegue. El efecto es mayor en los modelos pequeños, donde la capa de búsqueda domina el presupuesto (en BERT-tiny el embedding de entrada concentra cerca del 90 % de los parámetros), y es complementario a cuantización, poda y destilación.

Los embeddings dinámicos son *congelados*: nunca se entrenan ni reciben gradientes. Solo cambian cómo un id de token se convierte en vector; todo lo demás (mezcladores, normalización, residuales, enrutamiento, optimizadores) permanece igual. Como en el paper, la cabeza de salida sigue siendo una matriz independiente y no atada: durante el preentrenamiento MLM todavía almacena una matriz $V \times H$, de modo que la ventaja de tamaño se materializa en el artefacto solo-backbone o posterior al ajuste fino (el checkpoint de entrenamiento no es automáticamente 15× más pequeño). La implementación vive en `src/model/embeddings/dynamic_embedding.py`, el contrato de configuración es el bloque `model.embedding.dynamic.*`, y el vocabulario del tokenizador lo inyectan los pipelines de entrenamiento, despliegue e inferencia mediante `src/utils/tokenizer_vocab.py`.

21.2. Algoritmo

Sea w una cadena de token de l bytes con valores $b_1, \ldots, b_l$, y $d = H$ el ancho del embedding. El algoritmo tiene cinco pasos, con N el tamaño máximo de n-grama y B el bucket de hash.

1. **Semillas.** Se extraen d semillas de un único estado aleatorio almacenado s,

$$\mathbf{h} = (h_1, \ldots, h_d) \in \{0, \ldots, B-1\}^d, \qquad \mathbf{h} \sim \text{Uniform}(\text{semilla} = s).$$

   Solo se persiste el entero s; el vector de semillas es una función determinista de él.

2. **Firmas de i-gramas.** Para $i = 1, \ldots, N$ se calcula el hash polinómico rodante de cada i-grama,

$$\sigma_i(j) = \Big(\sum_{t=0}^{i-1} b_{j+t}\, a^{\,i-1-t} \Big) \bmod B, \qquad j = 1, \ldots, k_i, \qquad k_i = \max(l - i + 1, 0),$$

   con $a = 257$ (primo, mayor que cualquier byte UTF-8). Los hashes se calculan de forma incremental: el hash del $(i{+}1)$-grama reutiliza el del i-grama y añade un byte, por lo que el token completo cuesta $\mathcal{O}(lN)$ operaciones enteras.

3. **Proyección particionada.** Se divide $\mathbf{h}$ en N partes contiguas $\mathbf{h}_i$ de longitud d_i con

$$\sum_{i=1}^{N} d_i = d, \qquad d_i \propto i,$$

de modo que los n-gramas más largos reciben más dimensiones. La matriz de proyección de cada tamaño de grama es el producto externo

$$\mathbf{P}_i = \sigma_i \otimes \mathbf{h}_i \in \mathbb{Z}^{k_i \times d_i}.$$

4. **Plegado.** Cada entrada se lleva a $[-1, 1]$ mediante

$$\phi(x) = \frac{(x \bmod B) - \mathbf{1}[x \bmod B > B/2]\, B}{B/2} \in (-1, 1],$$

lo que mantiene el embedding acotado sin almacenar ni normalizar una tabla.

5. **Agrupación y concatenación.** Se promedia cada proyección sobre sus k_i filas y se concatenan las N piezas:

$$\mathbf{e}_i = \frac{1}{k_i} \sum_{j=1}^{k_i} \phi\big(\mathbf{P}_i[j, :]\big), \qquad \mathbf{e} = [\mathbf{e}_1; \ldots; \mathbf{e}_N] \in \mathbb{R}^d.$$

Si un token es más corto que i, entonces $k_i = 0$ y esa partición aporta un vector nulo.

Como el hash es polinómico sobre caracteres, los tokens morfológicamente relacionados (p. ej. `runner` y `running`) comparten muchos 1/2/3-gramas y quedan cerca en el espacio de embedding. El paper muestra que esto importa sobre todo en modelos pequeños: sustituir el hash de n-gramas por un hash puramente aleatorio (que preserva posición) cuesta 5,0 puntos de GLUE en BERT-mini pero solo 0,3 en BERT-base.

21.3. Contrato de Configuración

Clave (`model.embedding.dynamic.*`)	**Defecto**	**Efecto**
`enabled`	`false`	Reemplaza la tabla de búsqueda por el embedding hash congelado. Mutuamente excluyente con `embedding.factorized.enabled`.
`ngram_max`	`3`	Tamaño máximo de n-grama N; debe cumplir $1 \le N \le H$.
`bucket`	`1000000007`	Bucket de hash B ($10^9 + 7$ en el paper). Cambiarlo cambia todos los embeddings.
`seed`	`42`	Estado aleatorio s de las semillas; semillas idénticas reproducen embeddings idénticos.
`precompute`	`true`	Memoiza la tabla $V \times H$ en un buffer *no persistente* (caché pura; nunca se guarda ni se exporta).

Cuadro 30: Configuración de embeddings dinámicos. El bloque compone con `embedding.conv.*`, que aplica un Conv1d sobre el flujo dinámico de ancho H.

El esquema impone la exclusión mutua con embeddings factorizados dos veces: una regla condicional `if/then` en `src/schema/_conditional_rules.yaml` y una comprobación en `__post_init__` de `FrankensteinModelConfig`. El bloque solo es válido para las clases de texto (`frankenstein_text_encoder` y `frankenstein_text_decoder`); la ViT usa un embedding convolucional de parches y el frontend de audio no es una búsqueda de vocabulario, por lo que `config_loader` rechaza la combinación en esas clases.

21.4. Inyección del Vocabulario y Ciclo de Vida

Los embeddings hash consumen cadenas de tokens, mientras que los modelos reciben ids enteros. Por eso la configuración lleva un campo de ejecución sin esquema, `token_vocab` (cadenas de tokens ordenadas por id). `inject_token_vocab(config, tokenizer)` las extrae de un tokenizador HuggingFace (`get_vocab()`) o del envoltorio legacy de SentencePiece (`vocab`, token → id), las adjunta antes de `build_model` y alinea `vocab_size` con los ids del tokenizador. El motor de entrenamiento (`src/engine.py` y `src/training/main.py`), el cargador de checkpoints de despliegue y el motor de inferencia llaman al ayudante, de modo que el vocabulario no necesita aparecer en el YAML. Construir `DynamicEmbedding` sin vocabulario lanza un error accionable.

El checkpoint no almacena pesos de embedding: `state_dict()` no contiene nada para la capa de entrada, y el vocabulario viaja con el tokenizador guardado junto al checkpoint. Con `precompute: true` la tabla calculada es un buffer no persistente: sigue a `.to(device)` pero queda excluida de checkpoints y exportaciones, por lo que es exactamente equivalente al cálculo al vuelo. Con `precompute: false` el hash se recalcula por lote (menos memoria residente, pasos más lentos); ambas rutas coinciden salvo por el orden de reducción en float32.

Los artefactos nativos de despliegue y el exportador GGUF conservan por tanto la ventaja de tamaño. La exportación a HuggingFace Transformers no puede ejecutar el hash personalizado, así que materializa la tabla calculada como un `nn.Embedding` congelado, registra la conversión en el informe de compatibilidad y emite una advertencia de que la ventaja de tamaño de EELBERT no se preserva en ese artefacto.

21.5. Contabilidad de Tamaño y Resultados del Paper

El bloque eliminado son pesos $V \times H$; el bloque añadido es solo el Conv1d opcional ($kH^2 + H$) cuando `embedding.conv.enabled` está activo. En nuestro preset `configs/eelbert_tiny.yaml` (geometría BERT-tiny: 2 capas, $H = 128$, 2 cabezas, FFN reducida a 128, $V = 30{,}522$), la tabla de entrada elimina 3,91M pesos mientras la cabeza MLM no atada conserva 3,94M, dejando 4,15M parámetros totales: la misma contabilidad del paper, donde la cabeza se descarta solo para el despliegue.

La Tabla 31 resume los resultados de referencia reportados por Cohn et al. [22].

Modelo	Paráms.	Tamaño	GLUE	ΔGLUE	Latencia
BERT-base	109.5M	438 MB	0.775	—	162.0 ms
EELBERT-base	86.1M	344 MB	0.760	−1,5	165.0 ms
BERT-mini	11.17M	44.8 MB	0.753	—	7.0 ms
EELBERT-mini	3.36M	13.4 MB	0.746	−0,7	9.9 ms
BERT-tiny	4.39M	17.7 MB	0.671	—	1.7 ms
EELBERT-tiny	0.48M	2.04 MB	0.666	−0,5	3.9 ms
UNO-EELBERT	0.31M	1.24 MB	0.632	−3,9	—

Cuadro 31: Resultados de referencia de EELBERT (Tabla 2 de 22); latencia en un MacBook Pro M1 con 32 GB de RAM. UNO-EELBERT es EELBERT-tiny con el tamaño intermedio reducido a 128.

La complejidad de un embedding de token es $\mathcal{O}(l\,d)$ frente a la búsqueda $\mathcal{O}(1)$ de BERT: para los anchos de 128–768 usados por BERT, la latencia adicional es despreciable a escala base pero visible en modelos diminutos (cerca de 2,3× para EELBERT-tiny en la tabla). El paper propone tres mitigaciones que esta implementación deja como opciones de ingeniería: una máscara dispersa de ancho fijo sobre el producto externo ($\mathcal{O}(l\,s)$ con $s \ll d$), el hash paralelo por token, y una caché acotada de los n-gramas más frecuentes.

21.6. Mapa de Implementación

- `src/model/embeddings/dynamic_embedding.py` — `DynamicEmbedding`: semillas, hash rodante por bloques, producto externo particionado, plegado, agrupación, Conv1d opcional y `embedding_table()`.
- `src/model/frankenstein_encoder.py` — despacho de `emb` (`dynamic` → `factorized` → `nn.Embedding`); el decoder lo reutiliza a través de su backbone encoder. La exclusión mutua y los rangos viven en `src/model/config.py`.
- `src/utils/tokenizer_vocab.py` — `token_strings()` e `inject_token_vocab()`, compartidos por el motor, la ruta CLI de entrenamiento, despliegue e inferencia.
- `src/deploy/transformers\_export.py` — materialización de la tabla congelada y advertencia de exportación.
- `tests/test\_dynamic\_embedding.py` — determinismo por semilla, cotas $[-1, 1]$, similitud morfológica, paridad precompute vs. al vuelo, cero parámetros entrenables, errores de vocabulario y configuración, forwards de encoder/decoder, regla del esquema y materialización de exportación.
- `configs/eelbert_tiny.yaml` y `configs/examples/embedding_dynamic.yaml` — presets; ambos pasan smoke tests, y el arnés E2E cubre la combinación como `trans_dynamic_emb`.

21.7. Alcance y Limitaciones

Los embeddings dinámicos son un mecanismo de compresión, no un nuevo objetivo de aprendizaje de representaciones: no aproximan un embedding BERT entrenado y no pueden entrenarse ni ajustarse. Como el mapeo es una función determinista de (cadena, semilla, B, N), cambiar cualquiera de ellos invalida los checkpoints entrenados previamente. El diseño también acopla el modelo al tokenizador que produjo los ids de entrenamiento: el vocabulario (y por tanto las cadenas de tokens) forma parte del contrato del artefacto, y cambiar el tokenizador altera silenciosamente todos los embeddings de entrada. Por último, la exportación a HuggingFace pierde la ventaja de tamaño por construcción, así que los despliegues en dispositivo deberían preferir el checkpoint nativo, el paquete de despliegue o GGUF.

22. El arnés de experimentación

Este apéndice documenta la maquinaria detrás de la Sección 9. Su propósito es la reproducibilidad entre rondas: un experimento nuevo es una carpeta nueva bajo `experimentos/`, y los mismos scripts de consolidación, reporte y publicación operan sobre ella sin cambios. El arnés está deliberadamente separado del código del modelo—solo lee lo que entrenamiento y despliegue escriben—así que un experimento nunca depende de la versión de la biblioteca que lo produjo, y la biblioteca nunca se entera de la campaña que la usó.

22.1. Etapas y registros

Cada configuración es un archivo YAML y un slug. El slug codifica la familia y los mezcladores: `attn_nsa_attn__engram_attn` es un par híbrido, `attn_single_mamba` una línea base de un solo mezclador, y las ablaciones llevan sus propios prefijos (`opt_`, `norm_`, `pe_`, `trans_`, `vit_`, `task_`, `tokenizer_`, `batch_`, `realdata_`, `audio_`, `bitnet_`). Todas las corridas ejecutan las mismas cinco etapas: entrenar, exportar el checkpoint, desplegar en formato estándar y cuantizado, evaluar cuatro consignas enmascaradas reservadas por formato y exportar el paquete de Hugging Face.

Un fallo en una etapa se registra, no aborta la tanda y aparece después como una fila ausente en lugar de como un cero silencioso.

Tres registradores escriben de forma independiente y se unen solo al momento del reporte:

1. **El log de entrenamiento paso a paso.** Un CSV por corrida con una fila por paso de optimización: pérdida, precisión, learning rate, normas de gradiente por grupo de parámetros, razón de recorte, tiempo por paso y rendimiento. La ronda 2 registró 2,656,372 filas, que se procesan en streaming y se reducen a medias por época antes de dibujar cualquier gráfico.

2. **El censo de parámetros.** Para cada configuración, el número de parámetros de cada grupo de módulos (atención, embeddings, FFN, cabeza, normalización, codificación posicional, mHC, router, residual). Esto es lo que separa "el modelo es más grande" de "el bloque de atención es más grande".

3. **El libro del pipeline.** Una fila por ejecución de etapa con categoría, estado, duración y código de retorno. Es el único registro de los fallos y, por tanto, la única fuente para el análisis de fiabilidad.

22.2. El libro de etapas de la ronda 2

La Tabla 32 reproduce el libro. El entrenamiento domina tanto el conteo de registros como el coste: 1,039 intentos, 868 éxitos y 171 fallos, una mediana de 68 minutos y un P90 de 84 minutos por corrida exitosa, para 906 horas de GPU en total. El despliegue es la otra etapa que falla (104 veces de 1,734), y los fallos son simétricos: una configuración cuyo export estándar falla también falla en el cuantizado, lo que apunta a la serialización del checkpoint o de la configuración y no a la cuantización. La evaluación y el export a Hugging Face nunca fallaron en la población analizada.

Cuadro 32: Libro de etapas del pipeline para la ronda 2. Las duraciones están en segundos; los totales son horas de reloj de toda la campaña.

Etapa	Registros	OK	Fallos	Mediana (s)	P90 (s)	Total (h)
Entrenamiento	1,039	868	171	4,101,45	5,037,3	906,0
Export a HF	867	867	0	17,41	44,7	5,4
Evaluación de inferencia	1,630	1,630	0	10,37	13,1	4,9
Despliegue	1,734	1,630	104	8,64	38,0	8,8
BitNet GGUF	1	1	0	3,32	3,3	0,0

Los 275 fallos son informativos y no solo inconvenientes. Los 171 fallos de entrenamiento quedan fuera de las 868 configuraciones analizadas, que se eligieron precisamente porque entrenaron; los 104 fallos de despliegue ocurren en pares. La distribución de fallos por familia y el análisis de huellas idénticas de los cuatro mezcladores que colapsan con 32 tokens se grafican en el Apéndice 23.

22.3. Datasets consolidados

La cadena de reporte parte de las carpetas crudas y produce un conjunto pequeño de tablas unidas. Se publican con el informe para que cualquier figura se pueda reconstruir sin los 702 MB de logs paso a paso, y los CSV crudos por corrida se publican aparte, comprimidos con gzip, para quien necesite datos a nivel de paso.

Cuadro 33: Archivos publicados con una ronda. `master.csv` es la clave de unión: una fila por configuración con parámetros, entrenamiento, pipeline e inferencia.

Archivo	Contenido
`master.csv`	Una fila por configuración; parámetros por grupo, métricas de entrenamiento finales y promediadas, duraciones y estados de etapa, puntuaciones de inferencia por formato.
`training_per_epoch.csv`	Medias por corrida y época de pérdida, precisión, learning rate, normas de gradiente y rendimiento (2,66 M de pasos reducidos a 86,274 filas).
`inference_samples.csv`	Cada registro de inferencia: cuatro muestras, dos formatos, una fila por muestra.
`pipeline_stages.csv`	El libro de etapas completo (5,271 filas).
`param_metrics.csv`	El censo crudo de parámetros por grupo de módulos.
`training_metrics_avg.csv`	Promedios por corrida de las 34 métricas registradas.
`training_metrics_final.csv`	Instantánea del último paso de cada corrida.
`training_metrics_std.csv`	Desviaciones estándar muestrales por corrida.
`results.md.csv`	Resumen legible de los resultados del pipeline.
`data/raw/*.csv.gz`	Los 868 logs de paso por corrida, un archivo por configuración.

22.4. Reconstruir una ronda

El pipeline son tres scripts y un publicador, todos de biblioteca estándar o solo pandas/numpy/matplotlib:

1. `python experimentos/to_csv.py N` lee `experimentos/N/` sin modificarlo y escribe CSVs consolidados en `experimentos/N/csvs/`: la tabla de inferencia con la columna `notes` expandida en una columna por cada par `key=value`, el censo de parámetros como una fila por configuración, el libro de resultados como tabla plana y los CSV de entrenamiento por corrida concatenados más su media, desviación estándar y última fila por corrida.

2. `python experimentos/N/report/build_data.py` procesa los logs crudos en streaming, une los tres registradores por slug y escribe `master.csv` y las tablas derivadas.

3. `python experimentos/N/report/make_charts.py` renderiza cada figura dos veces (oscura y clara) y registra los pies en `registry.json`; `make_report.py` consume ambos para emitir la página estática bilingüe.

4. `bash experimentos/N/report/publish.sh` copia la página, las figuras y los datasets al espejo de GitHub Pages. Nunca hace commit ni push por sí mismo.

Las figuras del artículo son un subconjunto curado de los mismos constructores de gráficos, exportado como PDF vectorial por `make_paper_figures.py`. Toda la cadena es solo CPU y reproduce cada número de este apéndice a partir de los datasets publicados.

22.5. Ronda 1: el piloto

La ronda 1 ejercitó el mismo arnés de extremo a extremo a escala de humo: 5,129 registros de etapas, corridas de entrenamiento de aproximadamente medio minuto y la misma secuencia de cinco etapas. Su propósito era validar la generación de slugs, la contabilidad del libro, la simetría del despliegue y las uniones del reporte antes de comprometer tiempo de GPU. No se analiza más aquí porque las corridas eran demasiado cortas para separar arquitecturas; su valor es que la ronda 2 heredó un arnés en el que cada modo de fallo ya se había visto una vez.

23. Ronda 2: el atlas de los mixers

Este apéndice reporta la ronda 2 en su totalidad. Sigue el orden del informe publicado: el paisaje de parámetros, los resultados de entrenamiento, la dinámica de aprendizaje, el atlas A×B, las barridas de ablación, las mediciones de despliegue e inferencia, la fiabilidad del pipeline y las correlaciones entre las variables medidas. Todos los números provienen de `master.csv` del dataset publicado; todas las figuras se regeneran con `make_paper_figures.py` a partir de los mismos constructores de gráficos que el informe web.

23.1. Configuración

La población es de 868 configuraciones: 703 registros de pares híbridos (667 pares únicos tras colapsar el duplicado `retnet`/`retnet_attn`), 38 líneas base de un solo mezclador, 48 transformers de visión y 79 ablaciones. Cada corrida entrena 100 épocas (unos 3,000 pasos) sobre un vocabulario español de 512 tokens con secuencias de 32 tokens, una semilla, el mismo calendario de warm-up más coseno y BitNet activado por defecto. Los pares híbridos son las combinaciones de dos elementos de la lista de 42 mezcladores, con la capa 0 ejecutando el mezclador A y la capa 1 el mezclador B; no existe ningún par invertido, así que la asimetría primero/segundo es descriptiva. Un par está cubierto si ambos mezcladores entrenan en todas las configuraciones que los contienen: la rejilla tiene $42 \times 41/2 = 861$ celdas y se midieron 667.

23.2. El paisaje de parámetros

Las 868 configuraciones abarcan de 127 328 a 415 904 parámetros, con una mediana de 220 160. Los pares híbridos ocupan el centro del rango porque combinan dos módulos de atención; las corridas más pequeñas son líneas base de un solo mezclador como `attn_single_mamba` (127 328 parámetros), y la mayor es `engram__conformer_attn` con 415 904, impulsada por el banco de memoria `engram`, que añade una gran tabla clave-valor al bloque de atención.

El gráfico de composición es el diagnóstico más útil de esta sección. Los embeddings y la cabeza de salida son costes fijos de unos 49k parámetros cada uno, así que cualquier diferencia entre configuraciones se paga casi por completo en el bloque de atención y la FFN. Los transformers de visión son la familia más costosa porque añaden embeddings de parches y una cabeza de clasificación mayor, mientras que las ablaciones de optimizador, normalización o codificación posicional mantienen por diseño el mismo esqueleto de 210k. El coste tiene una lectura clara: `engram` añade entre 100k y 160k parámetros cada vez que aparece, y `mamba` y `retnet` están entre los segundos mezcladores más baratos aunque sean los peores—barato y malo, la peor combinación.

23.3. Resultados de entrenamiento

Entre los pares híbridos, la pérdida final va de 8,116 a 10,450 nats (mediana 9,595). El mejor par es `nsa__engram_attn` con 8,116, seguido de `gated_deltanet2__sparda_attn` (8,567) y `falcon3__conformer_attn` (8,620). El peor, `gqa__tucker_attn`, necesita 10,450 nats. La brecha entre extremos supera la brecha entre casi todas las ablaciones de optimizadores, y ese es el argumento más fuerte para elegir bien el par de atención.

Pérdida y precisión de validación no coinciden. Su correlación de rangos entre pares es débil, el par con mejor pérdida marca 0,0 % de top-1, y el de mejor precisión (`fox__falcon1a_attn`) alcanza 12,55 % con una pérdida de entrenamiento peor (9,292). Con cuatro frases de evaluación, un token afortunado reescribe el ranking, así que hay que reportar ambas señales. La Tabla 34 lista los diez mejores pares por pérdida final y la Tabla 35 los diez mejores por precisión de validación.

Cuadro 34: Los diez mejores pares híbridos por pérdida final de entrenamiento.

Par (A + B)	Paráms.	Pérdida final	Top-1	Top-5	Entreno (min)
`nsa + engram`	383 999	8,116	0,00 %	25,0 %	154,5
`gated_deltanet2 + sparda`	254 336	8,567	0,00 %	25,0 %	78,2
`falcon3 + conformer`	254 640	8,620	4,67 %	25,0 %	32,6
`falcon1a + conformer`	254 640	8,633	2,38 %	25,0 %	23,0
`falcon1 + conformer`	254 640	8,639	4,67 %	25,0 %	21,3
`fox + falcon2a`	222 768	8,666	2,40 %	25,0 %	84,2
`gqa + conformer`	242 720	8,670	5,87 %	25,0 %	88,3
`falcon3a + conformer`	254 640	8,681	3,97 %	25,0 %	23,2
`fox + falcon2`	222 768	8,705	2,40 %	25,0 %	85,8
`gla + engram`	387 296	8,736	5,17 %	25,0 %	149,2

Cuadro 35: Los diez mejores pares híbridos por precisión top-1 de validación.

Par (A + B)	Paráms.	Pérdida final	Top-1	Top-5	Entreno (min)
`fox + falcon1a`	214 232	9,292	12,55 %	0,0 %	79,3
`fox + falcon3`	214 232	9,618	12,55 %	0,0 %	106,1
`fox + falcon3a`	214 232	9,235	12,55 %	0,0 %	77,3
`gated_deltanet2 + falcon2`	250 216	9,105	12,41 %	0,0 %	80,7
`gated_deltanet2 + falcon2a`	250 216	8,968	12,41 %	0,0 %	82,1
`hgrn2 + gated_softmax`	238 688	9,386	12,30 %	25,0 %	71,7
`hgrn2 + tucker`	238 688	9,221	12,30 %	25,0 %	81,6
`gqla + mlra`	194 624	9,461	11,85 %	0,0 %	56,0
`mla + mlra`	194 624	9,464	11,85 %	0,0 %	86,6
`deltanet + kda`	231 304	9,024	11,37 %	0,0 %	78,6

23.4. Dinámica de entrenamiento

Todas las corridas usan el mismo calendario de warm-up más coseno, así que las curvas de aprendizaje son directamente comparables. La población de pares sigue descendiendo en la época 100: la pérdida mediana cae de unos 10,2 nats en la época 0 a 9,49 al final. La tanda mide, por tanto, velocidad de aprendizaje y no calidad convergida, y todo ranking debe leerse como "qué par aprende más rápido en 3,000 pasos".

Las normas de gradiente son estables entre familias (1,87 para el bloque de atención, 0,92 para la FFN, 3,47 para la cabeza), y la razón de recorte mediana se sitúa en 0,90, por debajo del umbral de 1,0, así que la mayoría de los pasos pasan sin recorte y este solo actúa en los picos. El rendimiento mediano es 357 tokens/s para la población de pares; la cola de 3k–31k pertenece a ablaciones de tokenizer y audio cuyo cronómetro cubre solo parte del trabajo, por lo que el análisis de Pareto la excluye.

23.5. El atlas A×B

El atlas es el centro del experimento: cada celda es un par híbrido, las filas son el primer mezclador y las columnas el segundo. Leído columna a columna, el segundo mezclador domina: un B bueno (`sparsek`, `engram`, `mlra`) mantiene casi todo A en la banda de 8,9–9,6 nats, mientras que un B malo (`mamba`, `retnet`, `bigbird`) arrastra incluso al mejor A por encima de 10 nats. Leído fila a fila, ningún primer mezclador rescata a un segundo malo.

La cobertura es casi completa: se entrenaron 667 de los 861 pares posibles. Los 158 pares que nunca entrenaron son exactamente los que contienen `cca`, `ccgqa`, `msa` o `ssog`, cuatro mezcladores que fallan en toda configuración ($4 \times 41 - \binom{4}{2} = 158$).

Qué segundo mezclador elegir. La Tabla 36 ordena los segundos mezcladores por pérdida final media en todos los pares que los contienen. La columna Pares indica el tamaño de muestra detrás de cada media; una media de un solo par (por ejemplo `mamba`) es anecdótica. La mitad inferior es tan informativa como la superior: `mamba`, `bigbird`, `longformer` y `retnet` son baratos pero arrastran sistemáticamente al par, justo el patrón que muestran los mapas de calor.

Cuadro 36: Segundos mezcladores ordenados por pérdida final media en los pares que los contienen (mejores ocho y peores ocho).

Segundo mezclador	Pares	Pérdida media	Top-1 medio	Paráms. medios
`mlra`	22	9,368	5,74 %	209 581
`conformer`	36	9,369	3,39 %	259 861
`engram`	18	9,373	2,05 %	379 284
`sparsek`	9	9,382	4,82 %	208 926
`falcon2a`	34	9,387	4,33 %	229 973
`fox`	16	9,409	2,23 %	216 038
`mla`	20	9,457	4,66 %	217 989
`mtla`	26	9,459	5,29 %	217 223
`falcon1`	30	9,779	3,27 %	221 856
`gta`	25	9,788	3,27 %	219 828
`titan`	3	9,814	6,28 %	204 768
`gla`	11	9,814	3,90 %	222 555
`standard`	4	9,960	6,65 %	206 264
`sigmoid`	5	10,045	4,37 %	207 162
`sparse_transformer`	6	10,108	5,59 %	207 760
`mamba`	1	10,279	2,04 %	192 608

La interacción supera a los marginales. El gráfico de sinergia mide cuánto mejor es un par frente a la media de sus dos marginales,

$$S_{A,B} = \frac{\mu_A + \mu_B}{2} - L_{A,B},$$

donde μ_A y μ_B son las pérdidas medias de todos los pares que contienen A y B. `nsa+engram` gana 1,304 nats y los diez primeros comparten un patrón: un primer mezclador barato y limpio con un segundo de memoria pesada. La cola negativa está dominada por `gta`, `gqa` y `tucker` como B—mezcladores que parecen promedio en aislamiento pero chocan al apilarse tras un A fuerte.

Cuadro 37: Las mejores y peores interacciones entre mezcladores por sinergia. Los valores positivos indican que el par es mejor de lo que sugieren sus partes.

Par	Pérdida del par	Esperada	Sinergia	Top-1
`nsa + engram`	8,116	9,420	+1,304	0,00 %
`gated_deltanet2 + sparda`	8,567	9,448	+0,881	0,00 %
`fox + falcon2`	8,705	9,446	+0,741	2,40 %
`gqa + falcon1`	8,784	9,518	+0,734	2,86 %
`fox + falcon2a`	8,666	9,390	+0,724	2,40 %
`bigbird + gta`	10,445	9,725	−0,720	1,79 %
`retnet + gated_softmax`	10,239	9,533	−0,706	7,70 %
`mamba + engram`	10,277	9,543	−0,734	1,10 %
`mamba + sparda`	10,391	9,643	−0,749	5,09 %
`gqa + tucker`	10,450	9,432	−1,018	7,76 %

23.6. Ablaciones

Optimizadores. La barrida de optimizadores es el gráfico más aleccionador: 23 optimizadores, el mismo esqueleto de 210k y un rango de 10,10–10,77 nats. `q_apollo` gana con 10,098 y `galore_adamw` cierra la tabla con 10,772, pero una docena de optimizadores aterrizan exactamente en 10,404, así que la corrida es demasiado corta para separar la elección del ruido de semilla. Las diferencias de velocidad sí son reales: el tiempo por paso varía un 30 % y `shampoo` y `muon` pagan sobrecoste visible.

Cuadro 38: Ablación de optimizadores: pérdida final, top-1 de validación y rendimiento sobre el mismo esqueleto.

Optimizador	Pérdida final	Top-1	Tok/s
`q_apollo`	10,098	3,63 %	436
`sophia`	10,104	1,79 %	378
`lion`	10,245	4,64 %	279
`shampoo`	10,401	7,76 %	382
`adopt`	10,404	7,76 %	391
`mars_adamw`	10,404	7,76 %	343
`adan`	10,404	7,76 %	359
`adamw`	10,404	7,76 %	384
`schedulefree_adamw`	10,404	7,76 %	487
`soap`	10,404	7,76 %	412
`ademamix`	10,404	7,76 %	366
`cautious_adamw`	10,404	7,76 %	365
`prodigy`	10,439	0,00 %	349
`sgd_momentum`	10,450	0,00 %	347
`turbo_muon`	10,454	0,00 %	536
`anon`	10,458	0,00 %	387
`muon`	10,461	0,00 %	507
`apollo`	10,519	0,00 %	401
`adafactor`	10,530	0,00 %	395
`ada_muon`	10,620	1,79 %	497
`lamb`	10,653	7,76 %	464
`radam`	10,738	7,76 %	502
`galore_adamw`	10,772	9,54 %	451

Normalización y codificación posicional. Entre las normalizaciones, `flash_norm` gana con claridad (9,749 nats, medio nat por delante del segundo), mientras que `prms_norm` y `rms_norm` quedan atrás. En codificaciones posicionales la familia PAPE (`pape_ri`, `pape`, `pape_efficient`) barre el podio, y RoPE clásico queda a media tabla a esta escala—recordatorio de que "moderno" no es automáticamente mejor en una corrida de 100 épocas.

Cuadro 39: Ablación de normalización sobre el esqueleto compartido.

Variante	Pérdida final	Top-1	Paráms.
`flash_norm`	9,749	1,43 %	209 600
`dynamic_tanh`	10,246	0,00 %	210 752
`derf`	10,295	1,79 %	210 762
`layer_norm`	10,401	7,76 %	210 752
`rms_norm`	10,436	7,76 %	210 272
`prms_norm`	10,573	0,00 %	210 272

Cuadro 40: Ablación de codificación posicional.

Codificación	Pérdida final	Top-1	Paráms.
pari	9,235	0,00 %	211 520
pape	9,455	4,17 %	216 928
pape_efficient	9,455	4,17 %	216 928
learned_absolute	9,525	7,02 %	216 896
hope	10,348	—	210 752
sinusoidal_absolute	10,364	6,68 %	210 752
bam	10,397	9,54 %	210 768
nope	10,397	9,54 %	210 752
none	10,397	9,54 %	210 752
rope	10,401	7,76 %	210 752
sinusoidal_rotary	10,401	7,76 %	210 752
alibi	10,417	7,76 %	210 752

Conmutadores estructurales. Los conmutadores estructurales se separan más que los optimizadores. Los embeddings factorizados (9,623), la FFN SwiGLU (9,901) y dos bucles de Hyperloop (9,908) son las tres victorias; los conmutadores de BitNet quedan en 10,016, mHC en 10,186 y las variantes de residuales en 10,43–10,49. Mixture-of-Depths es el peor conmutador con 10,854, consistente con su incompatibilidad con mHC a esta profundidad.

Cuadro 41: Conmutadores estructurales, ordenados por pérdida final.

Conmutador	Pérdida final	Top-1	Paráms.
factorized_emb	9,623	1,43 %	181 056
ffn_swiglu	9,901	2,49 %	228 800
num_loops_2	9,908	6,94 %	154 880
fixed_softmax_ssmax_bitnet	9,936	2,60 %	209 864
bitnet_conv	10,016	2,60 %	209 856
bitnet_on	10,016	2,60 %	209 856
mhc	10,186	1,79 %	254 092
bitnet_routers	10,327	—	247 104
ssmax	10,359	9,54 %	210 760
ffn_relu	10,393	3,83 %	210 752
fixed_both	10,401	7,76 %	210 752
embedding_conv	10,401	7,76 %	210 752
fixed_ce	10,401	7,76 %	210 752
pos_rope	10,401	7,76 %	210 752
fixed_softmax	10,401	7,76 %	210 752
residuals_full_attn	10,433	9,54 %	210 944
residuals_none	10,476	5,71 %	210 752
moe	10,477	—	248 384
residuals_block_attn	10,487	1,79 %	210 944
mod	10,854	—	210 944

Visión y tareas. El bloque de visión funciona. La segmentación alcanza 90,7 % de precisión media de validación, la clasificación 75,8 % y la predicción de parches 69,9 %. Estas cifras están un orden de magnitud por encima de los pares MLM (mediana 4,0 %) porque la tarea es más fácil, no porque el mezclador importe más, y por eso el informe mantiene las familias separadas. Las ablaciones de tokenizer y tarea se comportan como se espera: el tokenizer cambia la pérdida en varios nats porque cambia el vocabulario y la longitud media de secuencia, y la tarea la cambia porque cambia el objetivo.

23.7. Despliegue e inferencia

La cuantización post-entrenamiento es la buena noticia de la ronda. En las 810 configuraciones con ambos formatos, el top-1 medio pasa de 5,62 % a 5,43 % y el top-5 medio de 13,36 % a 13,24 %, deltas de −0,19 y −0,12 puntos porcentuales. Ochocientas configuraciones quedan exactamente iguales; las pocas que se mueven lo hacen por una muestra de cuatro. Para un modelo a escala de humo es el resultado esperado, pero vale la pena medirlo en vez de asumirlo.

La historia de velocidad es menos halagüeña. El despliegue cuantizado es más rápido que el estándar solo en el 32,3 % de las corridas (ratio mediano 0,64×) porque la etapa de despliegue está dominada por la serialización y la conversión, no por la inferencia; la evaluación es más rápida el 51,2 % de las veces con una aceleración mediana de 1,03×. La inferencia en sí tarda unos 10,5 s para cuatro frases, que es la métrica que importa en producción.

La mejor configuración de inferencia no es la de mejor pérdida. `mamba__engram_attn` acierta tres de cuatro tokens en el top-5 (75,0 %) con una pérdida de entrenamiento de 10,277, por encima de la mediana de los pares. Con cuatro muestras por configuración, el top-5 es un instrumento romo: sirve para detectar fallos catastróficos, no para coronar un ganador.

Cuadro 42: Configuraciones con mejor tasa top-5 en formato estándar.

Configuración	Top-5 est.	Top-1 est.	Top-5 cuant.	Pérdida final	Paráms.
`mamba__engram_attn`	75 %	50 %	75 %	10,277	347 168
`mamba__nsa_attn`	75 %	0 %	75 %	9,262	192 479
`retnet__fox_attn`	75 %	0 %	75 %	9,873	220 936
`bigbird__kda_attn`	50 %	0 %	50 %	10,196	221 128
`fox__engram_attn`	50 %	25 %	50 %	9,535	375 496
`fox__gated_softmax_attn`	50 %	25 %	50 %	9,892	220 744
`fox__tucker_attn`	50 %	25 %	50 %	9,890	220 744
`gated_deltanet__tucker_attn`	50 %	25 %	50 %	9,796	230 928
`gla__falcon1_attn`	50 %	0 %	50 %	9,511	226 032
`gla__mlra_attn`	50 %	25 %	50 %	9,538	211 808

23.8. Fiabilidad del pipeline

El libro registra 5,271 ejecuciones de etapas. 868 entrenamientos tuvieron éxito y 171 fallaron con código de salida distinto de cero, todos fuera de las 868 configuraciones analizadas, que se eligieron justamente porque entrenaron. El despliegue falló 104 veces de 1,734, siempre en pares simétricos: una configuración que falla el export estándar también falla el cuantizado, lo que apunta a una causa raíz compartida (serialización del checkpoint o de la configuración) y no a la cuantización en sí.

Dos hallazgos de calidad de datos condicionan la interpretación. Primero, cuatro mixers nunca entrenan: `cca`, `ccgqa`, `msa` y `ssog` fallan en sus cuatro singles y en los 158 pares que los contienen, exactamente las 158 celdas ausentes del atlas. Segundo, cuatro mixers son indistinguibles con contexto de 32 tokens: `standard`, `bigbird`, `longformer` y `titan` comparten pérdidas y parámetros bit a bit idénticos porque sus ventanas (al menos 64 tokens) cubren toda la secuencia y la variante aumentada con posición se reduce a atención multi-cabeza pura; coinciden en los 33 segundos mixers compartidos. Por separado, 37 pares aparecen como `attn_retnet__X` y `attn_retnet_attn__X` con métricas idénticas—dos nombres para una implementación—que el atlas colapsa a una fila.

23.9. Qué predice la calidad

Sobre los pares híbridos, el número de parámetros correlaciona con la pérdida final a $r = -0{,}20$ y con la precisión de validación a $r = -0{,}07$: el tamaño apenas compra calidad aquí. Lo que

sí predice de forma fiable es el coste ($r = 0{,}68$ con el tiempo por paso, $r = 0{,}65$ con el tiempo total). La pérdida de entrenamiento apenas informa sobre la calidad de inferencia ($r = 0{,}14$ con el top-5). La norma de gradiente y la razón de recorte son exactamente redundantes entre sí ($r = 1{,}00$), lo que es esperable porque la razón de recorte se calcula a partir de la norma de gradiente, y ninguna correlaciona con la calidad.

La matriz de correlación de la Figura 107 se calcula sobre las 868 filas. Leída así, el número de parámetros correlaciona con la pérdida a $-0{,}43$ y con la precisión a $+0{,}42$, lo que contradice los números de solo pares. Ambas lecturas son correctas: la matriz de todas las filas está dominada por el contraste entre transformers de visión (más grandes, mucha menor pérdida, mucha mayor precisión) y pares MLM, así que mide la brecha de familias y no el efecto del tamaño. Dentro de la población que importa para el atlas—los pares híbridos—el tamaño es casi irrelevante.

23.10. Lectura práctica y próximos pasos

Si hay que elegir a ciegas, usa un segundo mezclador con memoria pesada (`engram`, `sparsek`, `mlra`) y un primero barato y estable (`fox`, `gqa`, `gated_deltanet2`), y evita `mamba`, `retnet` y `bigbird` como B. Espera unos $\pm 0{,}3$ nats de ruido de interacción, mayor que la diferencia entre la mayoría de los optimizadores.

La ronda también define a su propia sucesora. El atlas debería repetirse con 256–512 tokens, donde los cuatro mezcladores que colapsan se vuelven distinguibles y los patrones dispersos se dispersan de verdad; la evaluación debería usar un conjunto reservado de 200 frases con log-verosimilitud por token, porque cuatro frases no pueden ordenar 800 configuraciones; los mejores pares deberían entrenarse 300 épocas, porque todas las curvas siguen bajando; el alias `retnet` y los cuatro mezcladores que fallan deberían corregirse o retirarse de la lista; y deberían correrse al menos tres semillas en las familias de ablación, donde la dispersión actual está dentro del ruido de semilla. La cuantización debería ser el formato de despliegue por defecto: costó 0,12 puntos de top-5 y aceleró la evaluación en la mitad de las corridas.

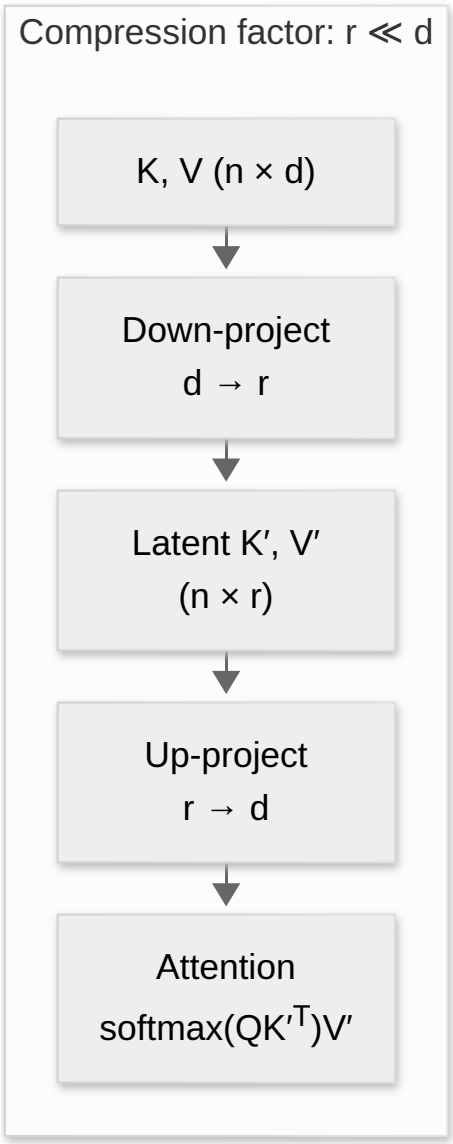


Figura 5: Atención latente y de bajo rango. Los pares clave–valor se proyectan a un espacio latente de bajo rango, donde la atención se calcula a menor costo, y se proyectan de vuelta. Las variantes difieren en factorización, intercalado de cabezas, compresión temporal, formulación de covarianza cruzada y enrutamiento probabilístico.

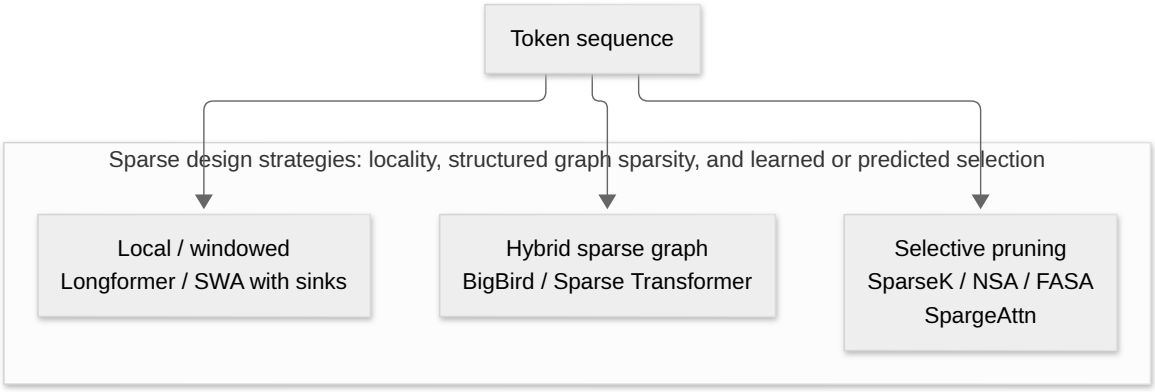


Figura 6: Decisiones de diseño en atención dispersa. Los métodos reducen el costo restringiendo vecindarios, construyendo grafos dispersos o seleccionando solo tokens o bloques de alto valor.

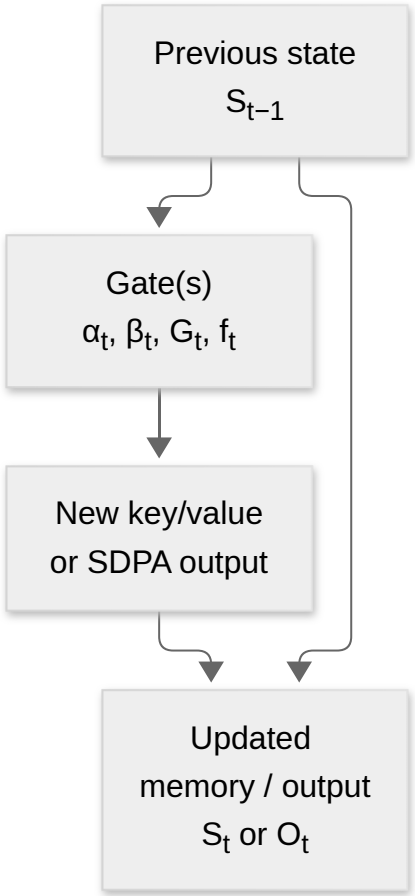


Figura 7: Plantilla genérica de compuerta. Una compuerta puede decaer la memoria existente, regular la fuerza de escritura o modular las salidas de atención densa, según la familia de bloques.

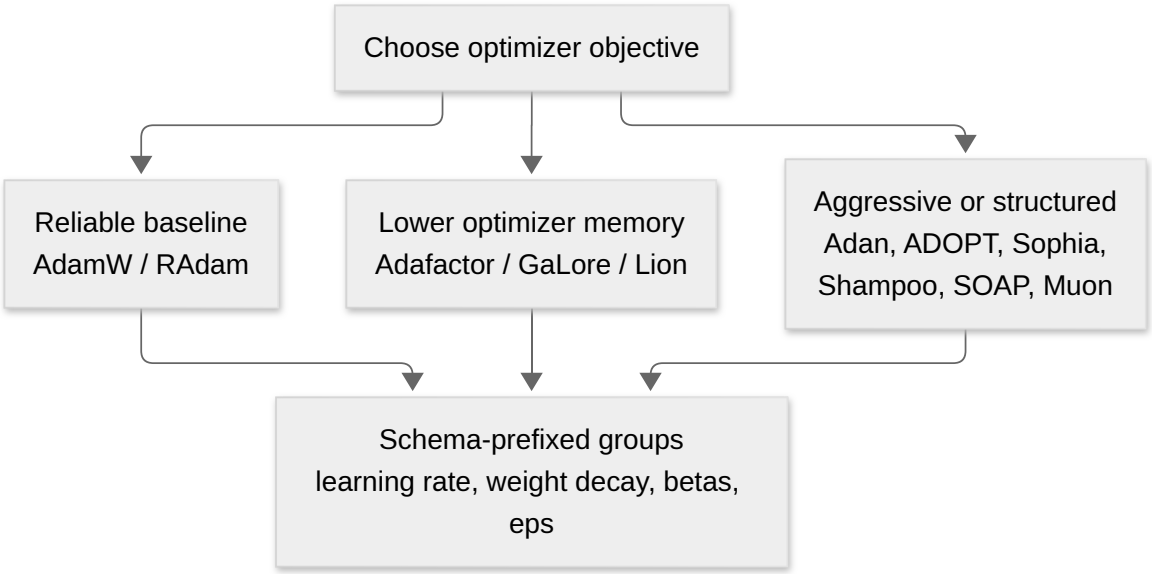


Figura 8: Selección de optimizador en la práctica: la clase determina la regla de actualización, mientras que el esquema controla cómo se aplican los hiperparámetros en los diez bloques canónicos de parámetros (embeddings, encodings posicionales, atención, feed-forward, normalización, heads, mHC, routers, residuales y el resto de parámetros).

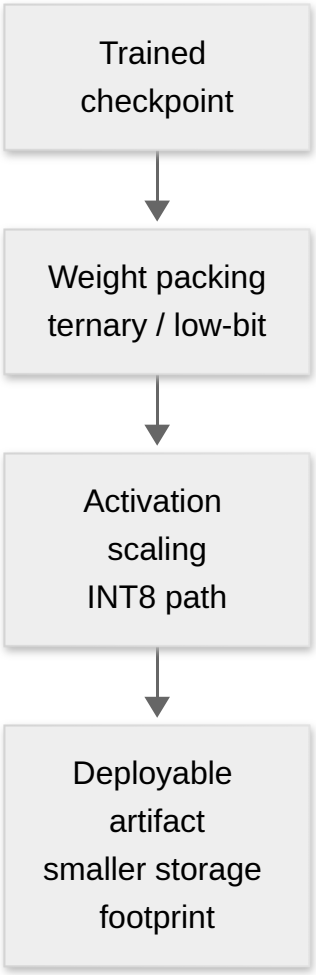


Figura 9: Ruta de despliegue desde un checkpoint entrenado hasta un artefacto compacto. La cuantización es una transformación de la etapa de despliegue, no una familia de modelos separada.

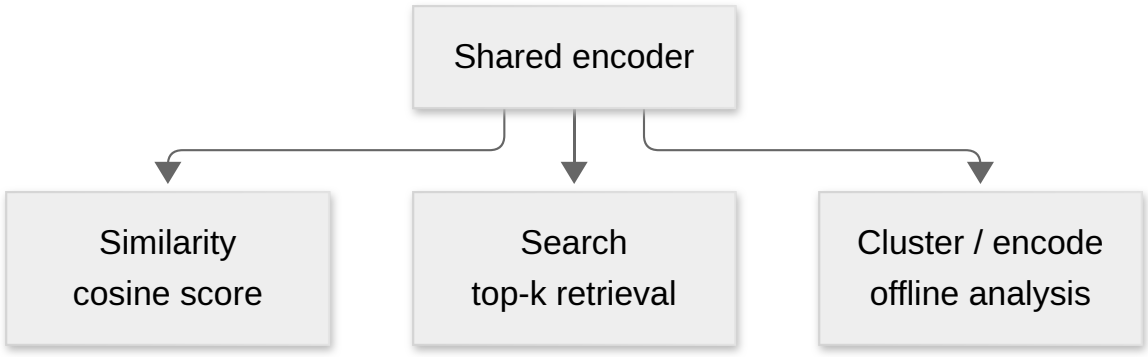


Figura 10: Reutilización del flujo de trabajo SBERT. Un único codificador soporta puntuación de pares en línea, recuperación sobre corpus, clustering y exportación persistente de embeddings.

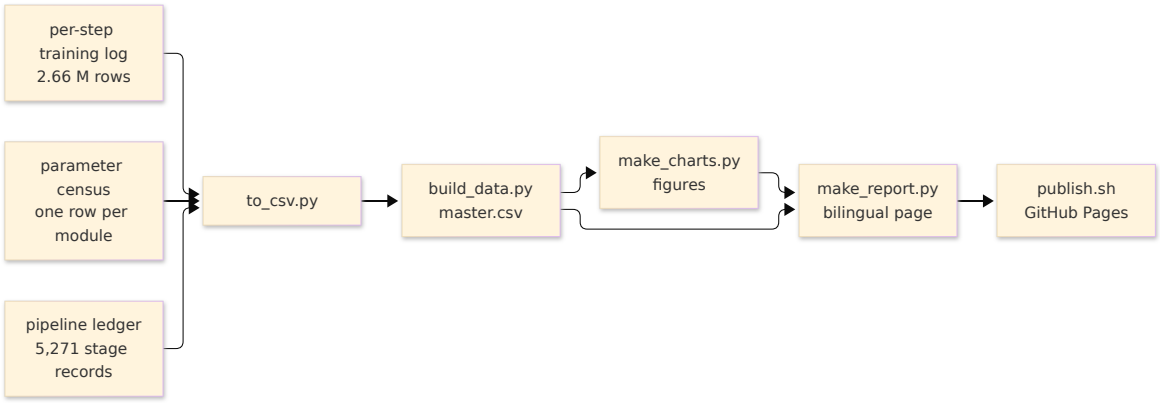


Figura 79: La cadena de reporte offline. Los tres registradores se consolidan en archivos CSV, se unen por configuración en `master.csv`, se grafican en ambos temas y se renderizan en una página estática bilingüe con los datasets unidos publicados al lado. Nada de esta cadena necesita GPU.

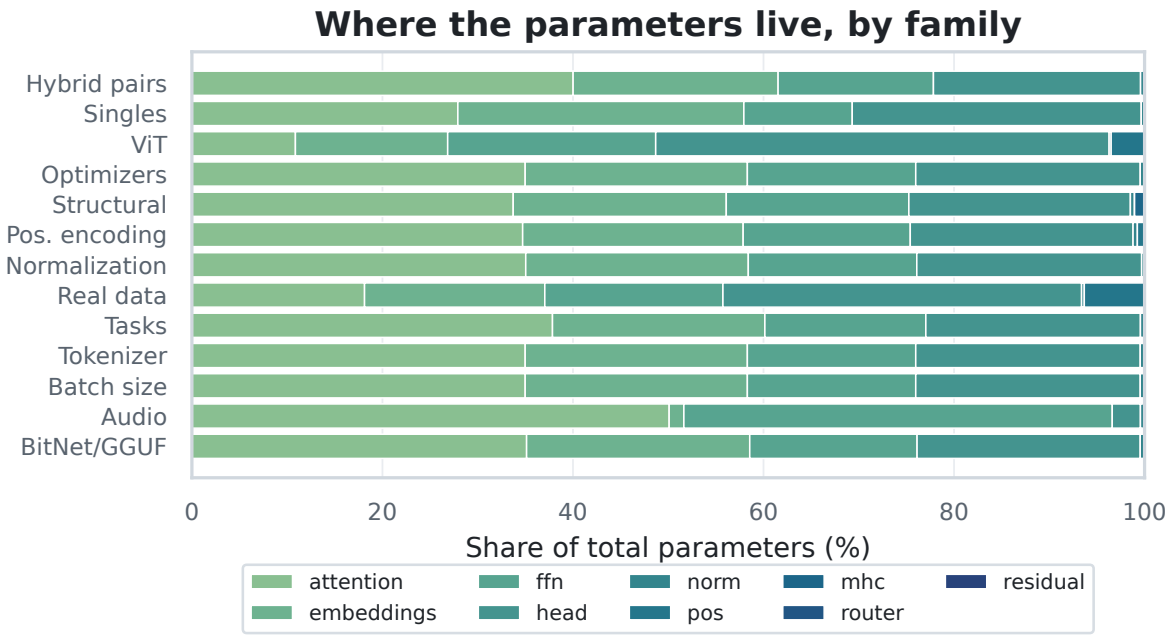


Figura 80: Dónde viven los parámetros, por familia. Los embeddings y la cabeza son costes fijos; el bloque de atención y la FFN cargan las diferencias arquitectónicas.

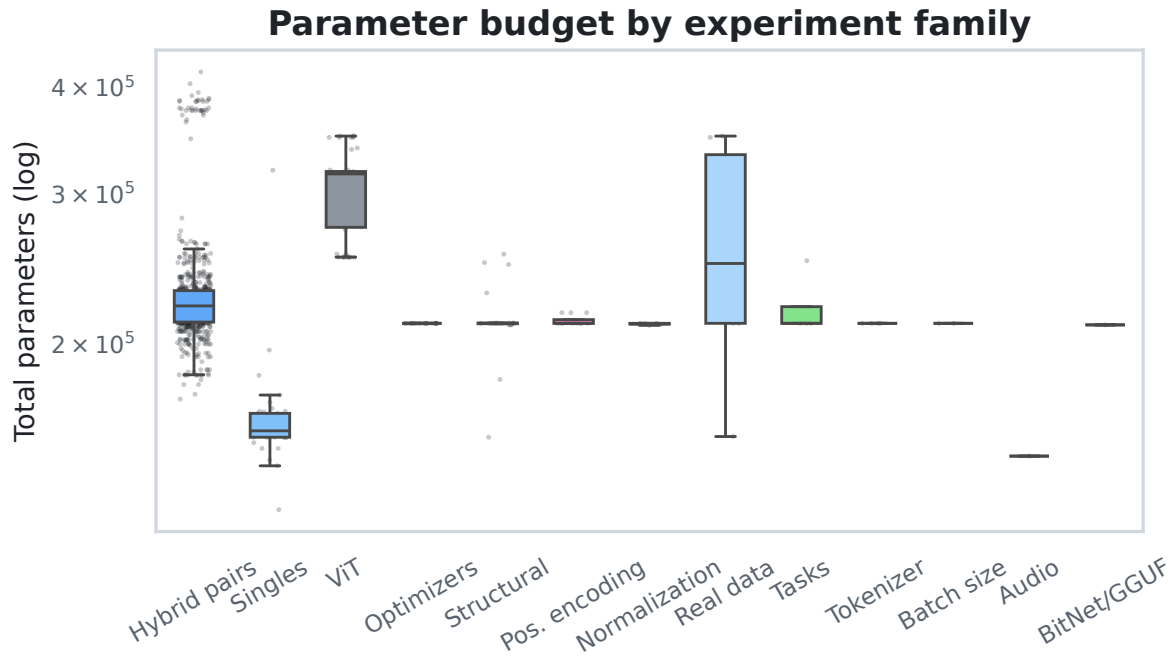


Figura 81: Presupuesto de parámetros por familia de experimentos (escala log). La familia ViT es la más cara; las familias de ablación comparten un esqueleto.

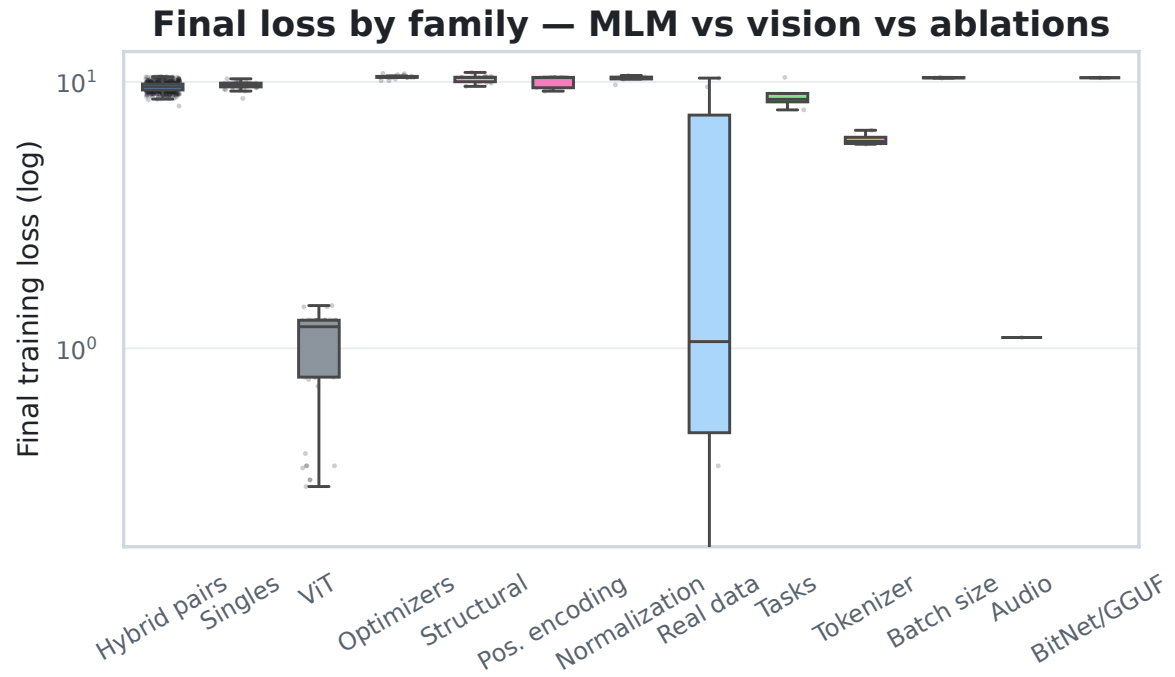


Figura 82: Pérdida final de entrenamiento por familia (escala log). Las familias de visión y datos reales no son comparables con las pérdidas MLM; las familias se mantienen separadas.

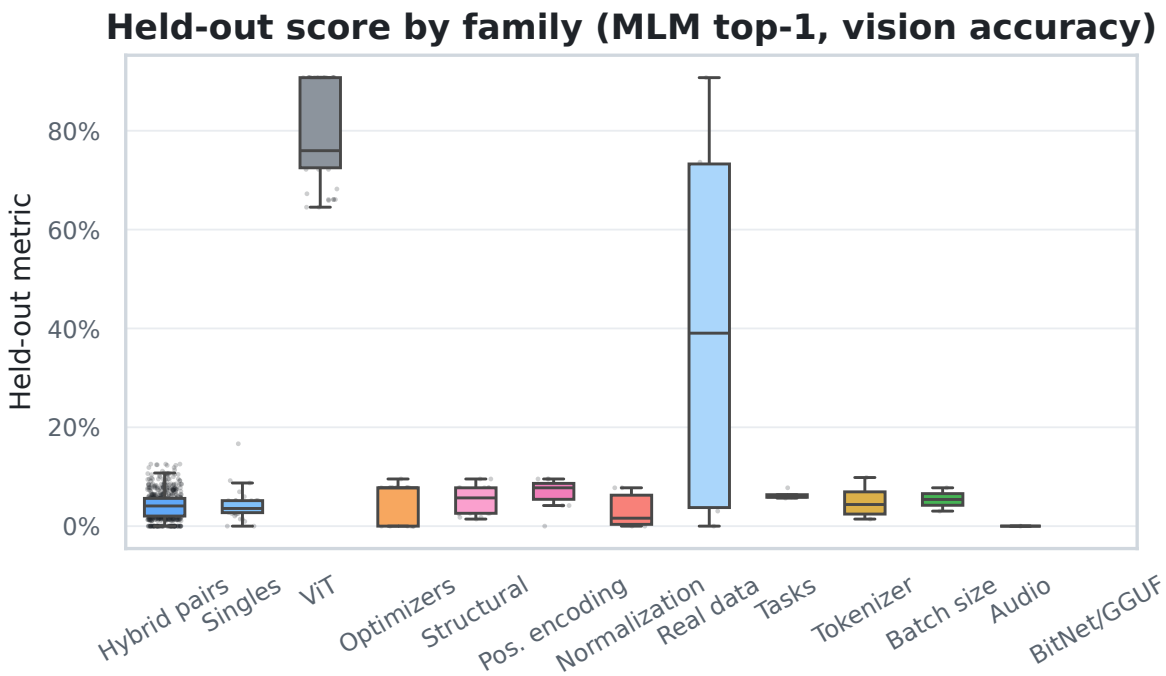


Figura 83: Métrica de validación por familia. Las tareas de visión puntúan un orden de magnitud por encima del MLM porque la tarea es más fácil, no porque el mezclador importe más.

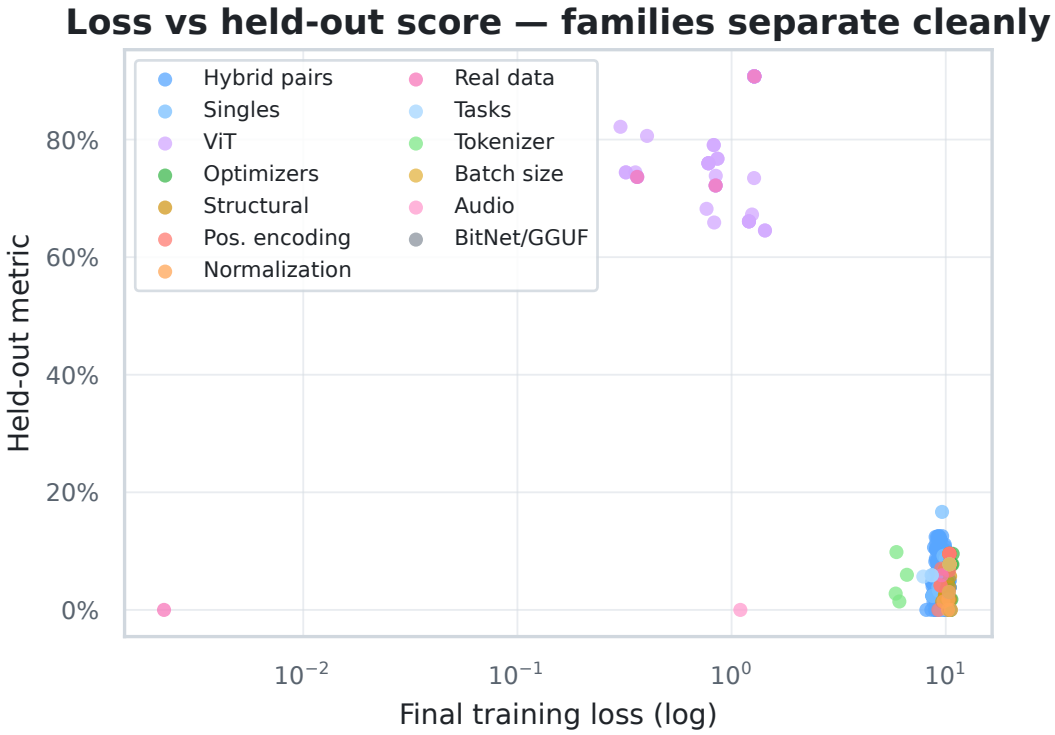


Figura 84: Pérdida final frente a métrica de validación. La relación es débil dentro de los pares: el mejor par por pérdida marca 0,0 % de top-1 y el mejor por precisión tiene peor pérdida.

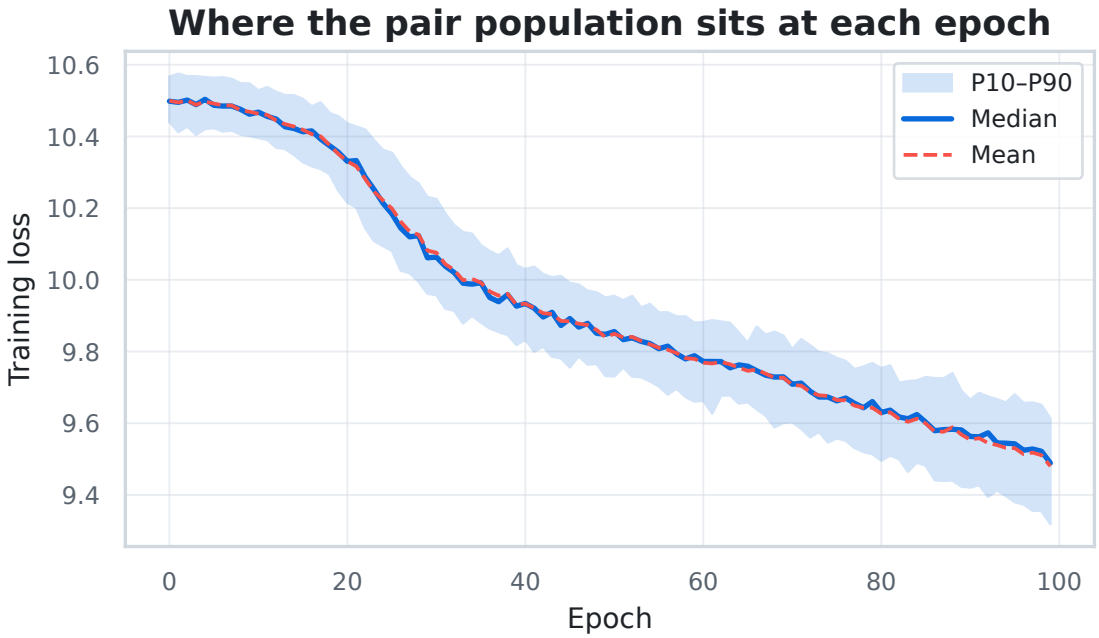


Figura 85: Curvas de la población con banda P10–P90. Las curvas siguen bajando en la época 100, así que la tanda mide velocidad de aprendizaje y no calidad convergida.

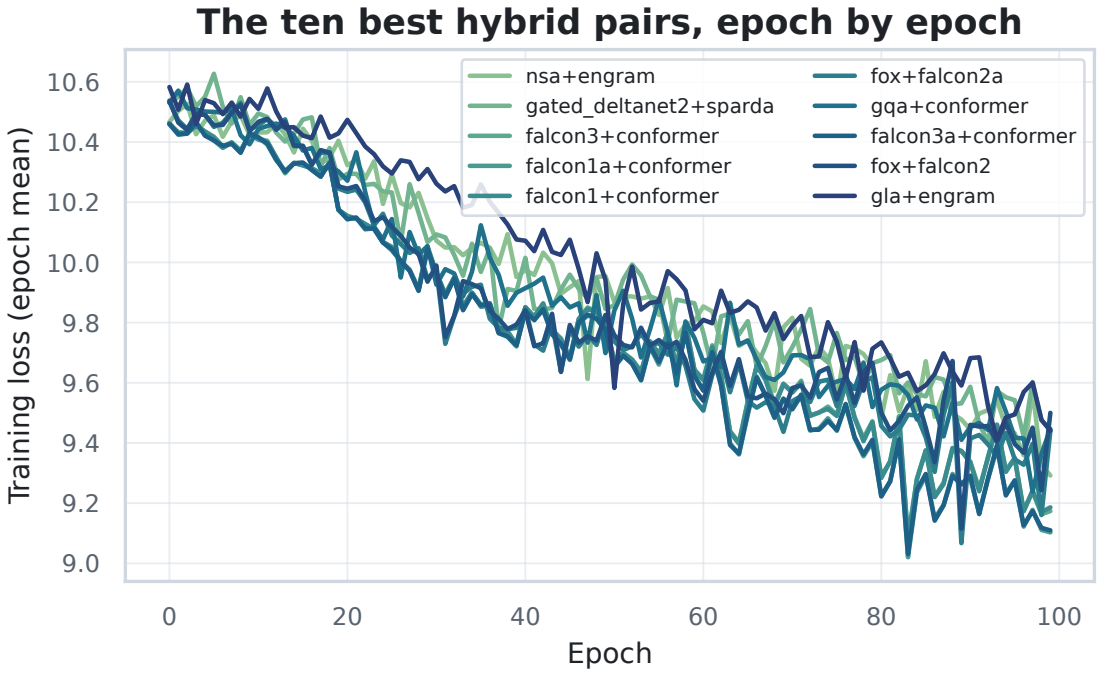


Figura 86: Los diez mejores pares a lo largo del tiempo. El orden se establece pronto y rara vez cambia después de la época 50.

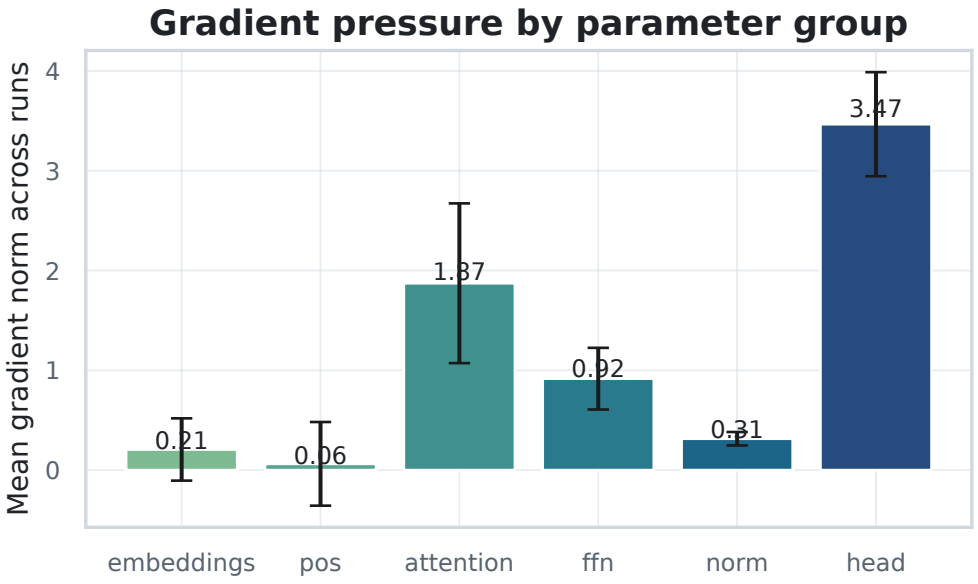


Figura 87: Presión de gradiente por grupo de parámetros. La cabeza soporta el gradiente mayor y la FFN el menor; el recorte actúa rara vez porque la razón mediana es 0,90.

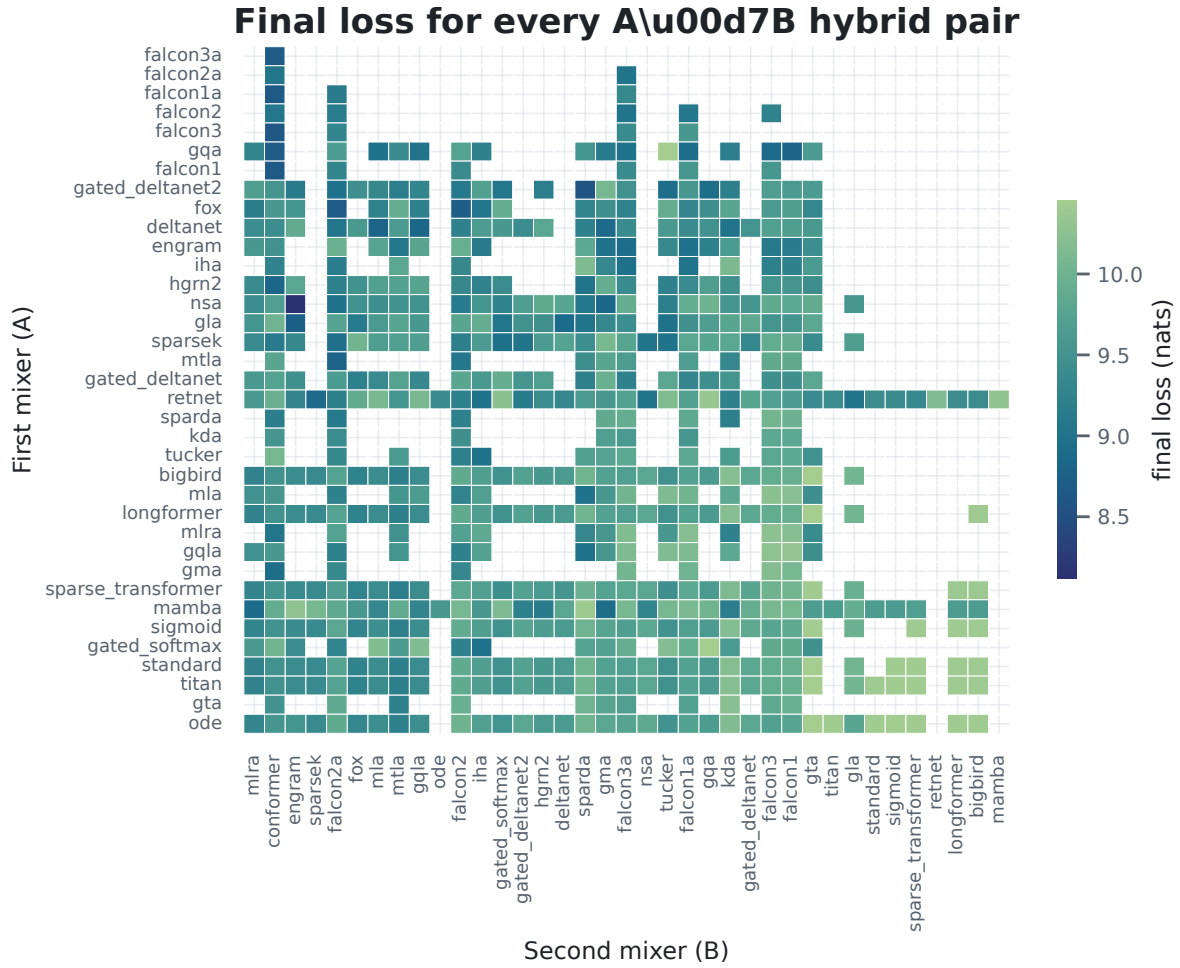


Figura 88: El atlas A×B de pérdida final. Las filas son el primer mezclador y las columnas el segundo; las celdas vacías son pares que nunca entrenaron.

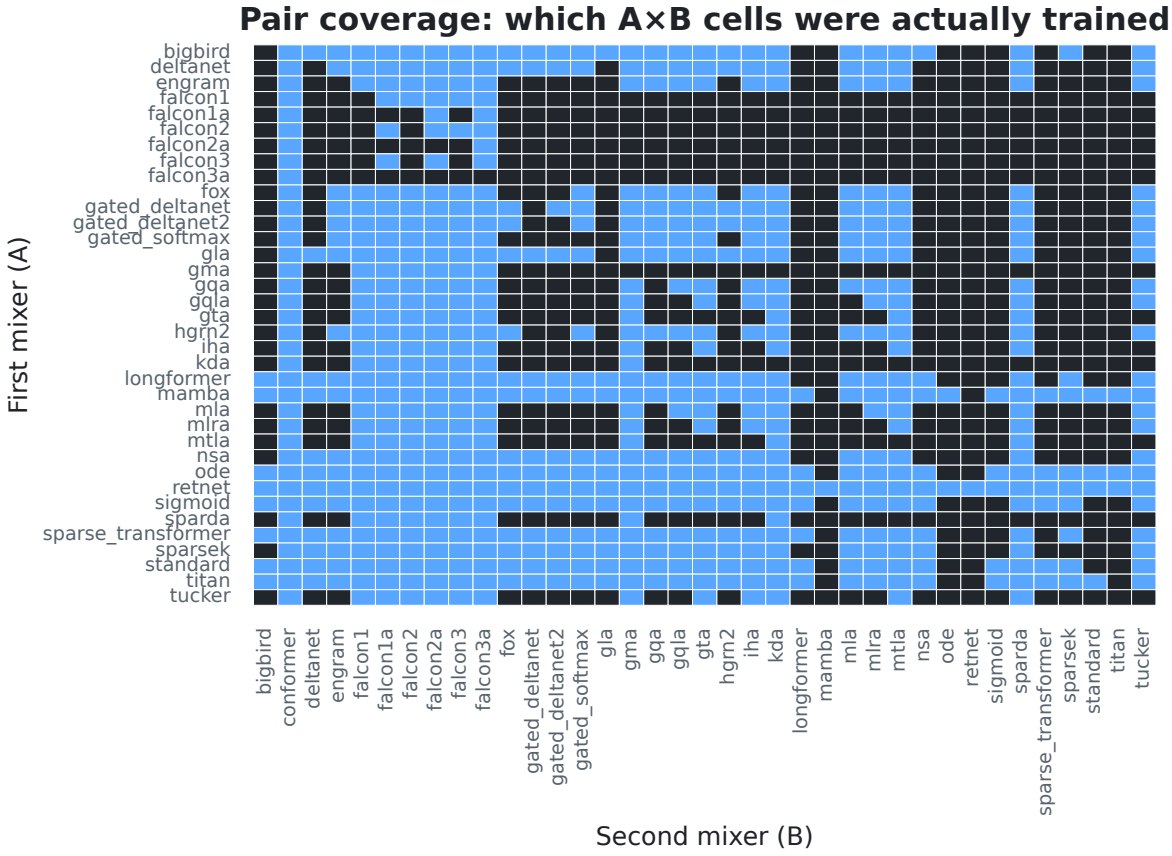


Figura 89: Cobertura de la rejilla A×B. Las celdas ausentes son los 158 pares que contienen uno de los cuatro mezcladores que nunca entrenan.

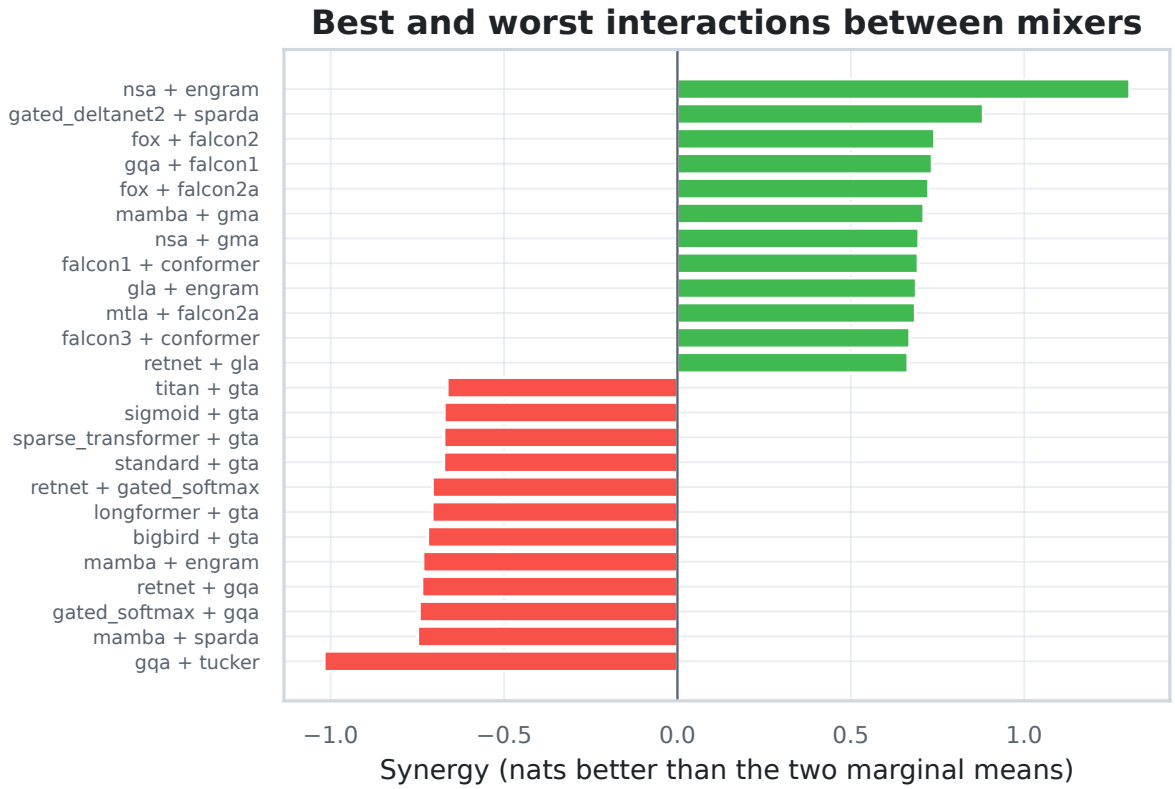


Figura 90: Mejores y peores interacciones entre mixers. La cola positiva empareja un primer mezclador barato y estable con un segundo de memoria pesada; la cola negativa está dominada por segundos mezcladores que chocan tras un A fuerte.

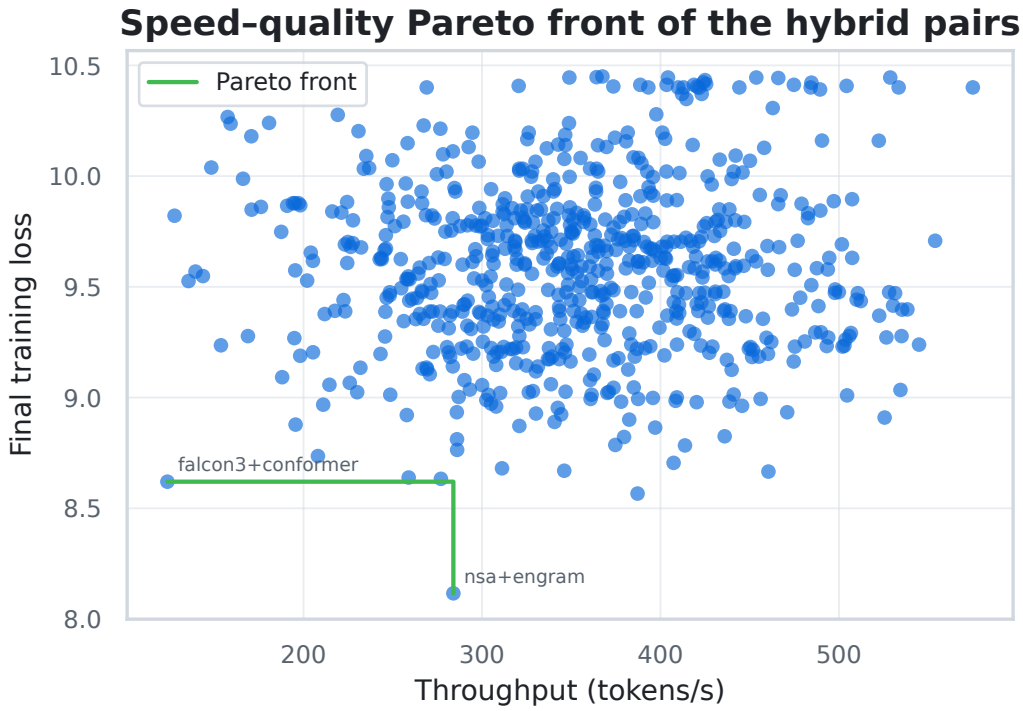


Figura 91: Frontera de Pareto velocidad–calidad de los pares híbridos. La frontera la pueblan pares cuyo segundo mezclador tiene memoria pesada pero cuyo primero es barato.

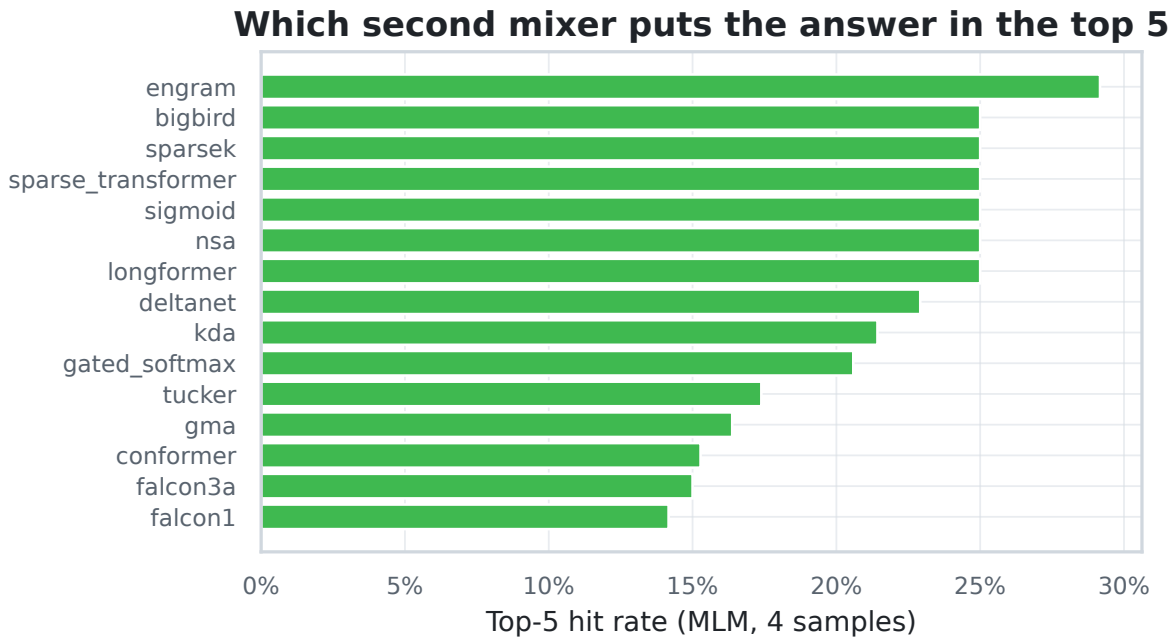


Figura 92: Top-5 por segundo mezclador. Con cuatro muestras por configuración, el intervalo entre barras adyacentes es una muestra, así que el orden es indicativo.



Figura 93: Calidad de inferencia frente a pérdida de entrenamiento. La correlación es débil ($r = 0{,}14$): la pérdida de entrenamiento no sustituye a la evaluación en validación.

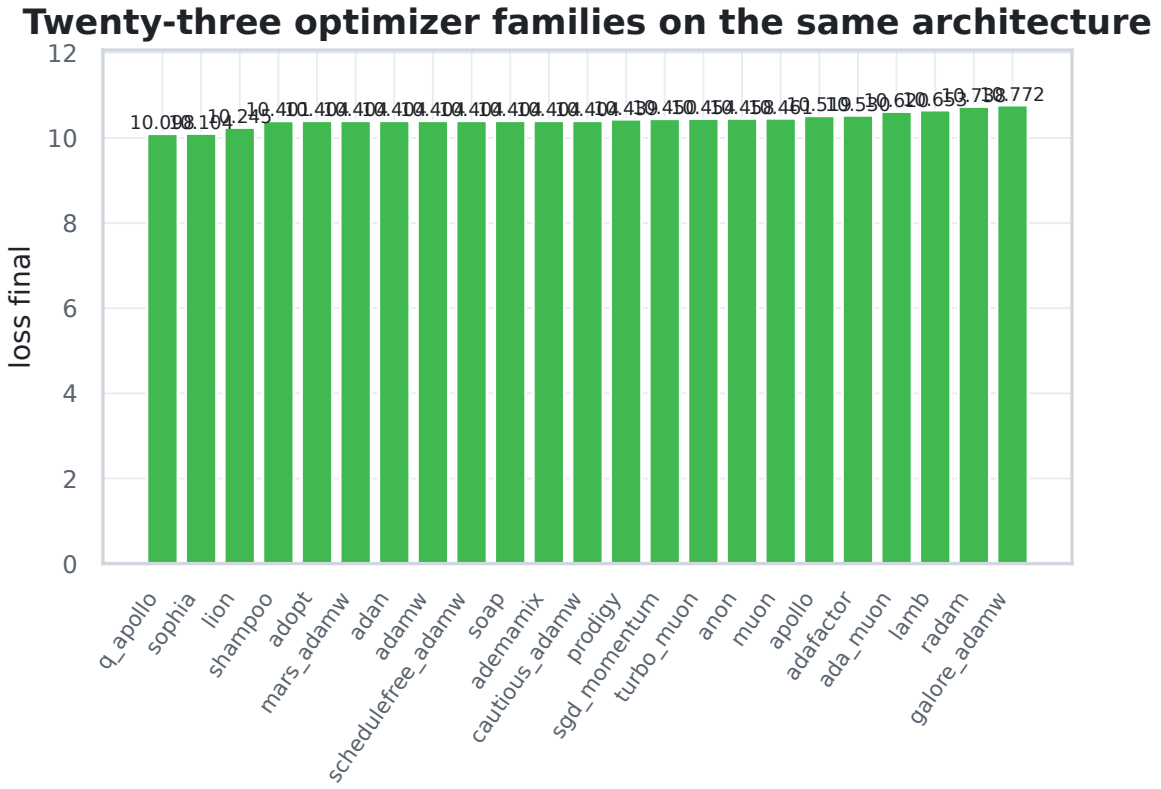


Figura 94: Ablación de optimizadores: pérdida final. Una docena converge al mismo valor, lo que mide más la brevedad de la corrida que la calidad del optimizador.

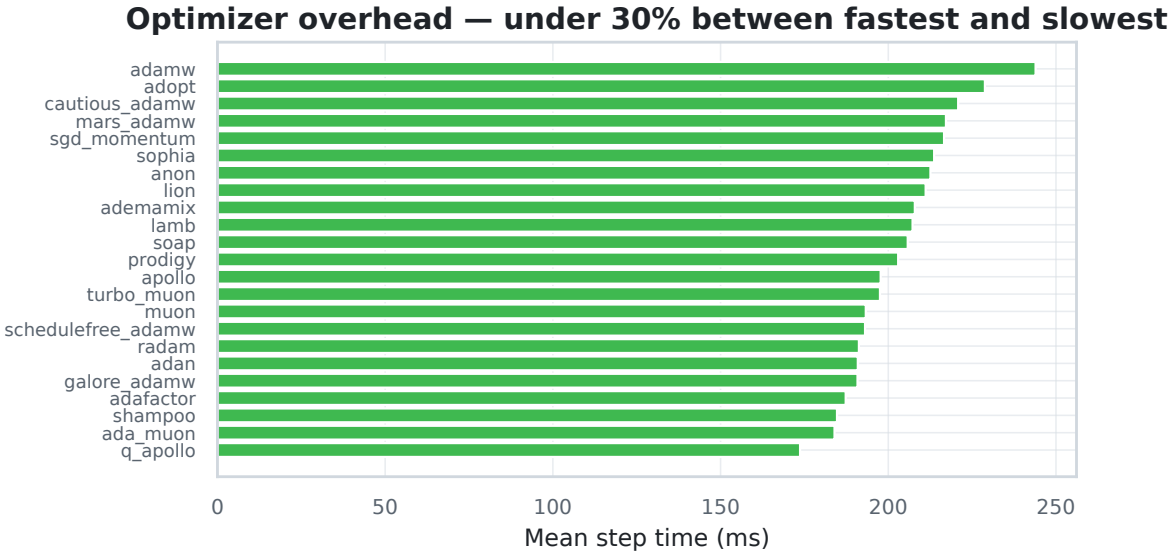


Figura 95: Ablación de optimizadores: tiempo por paso medio. La dispersión es de un 30 %; los métodos de segundo orden y ortogonalizados pagan el sobrecoste visible.

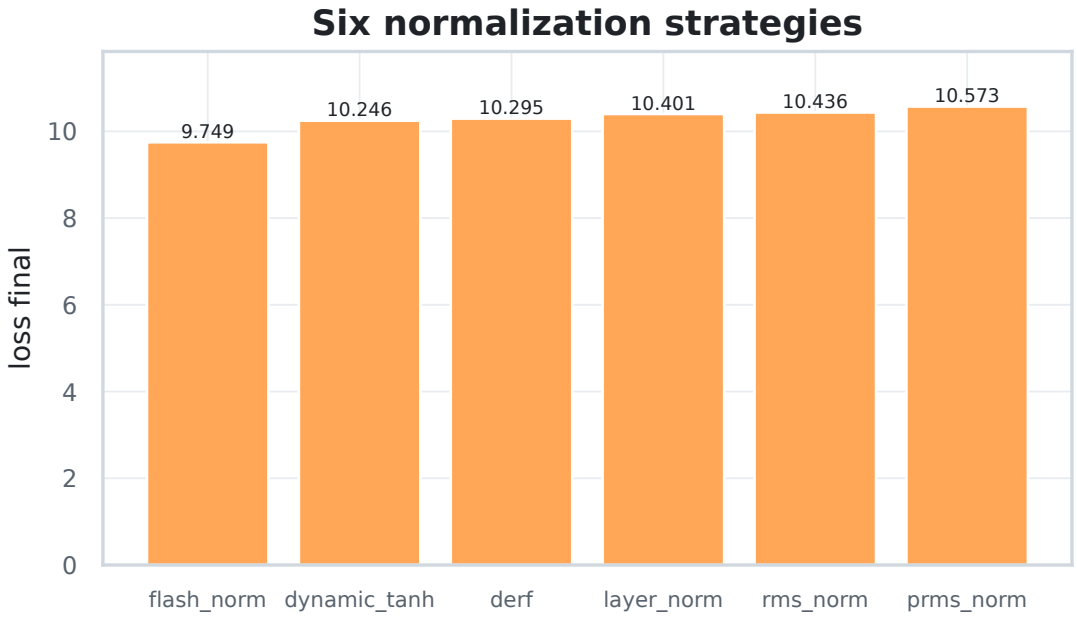


Figura 96: Ablación de normalización: `flash_norm` se separa del resto por medio nat.

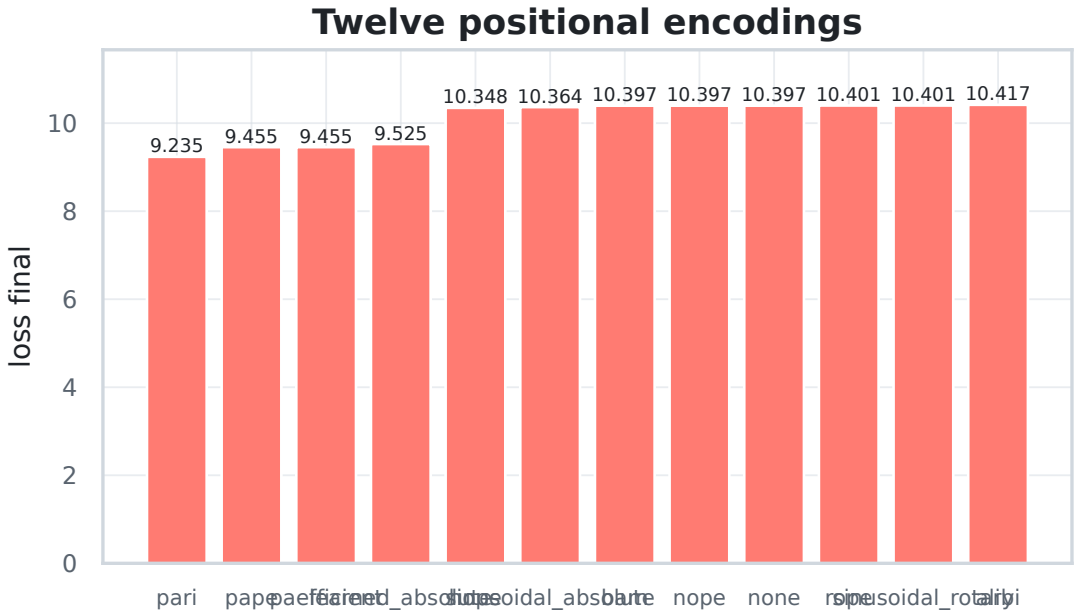


Figura 97: Ablación de codificación posicional: la familia PAPE lidera y RoPE clásico queda a media tabla a esta escala.

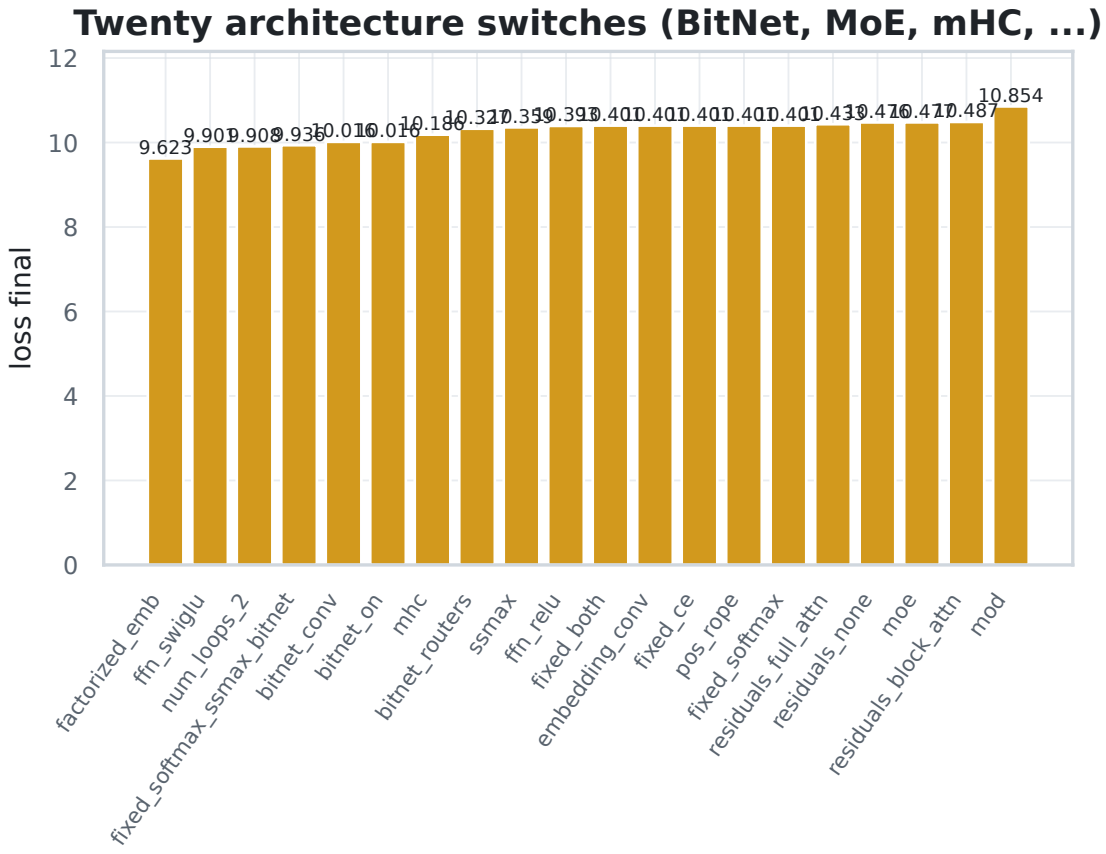


Figura 98: Conmutadores estructurales. Los embeddings factorizados, SwiGLU y Hyperloop son las tres victorias; Mixture-of-Depths es el peor conmutador.

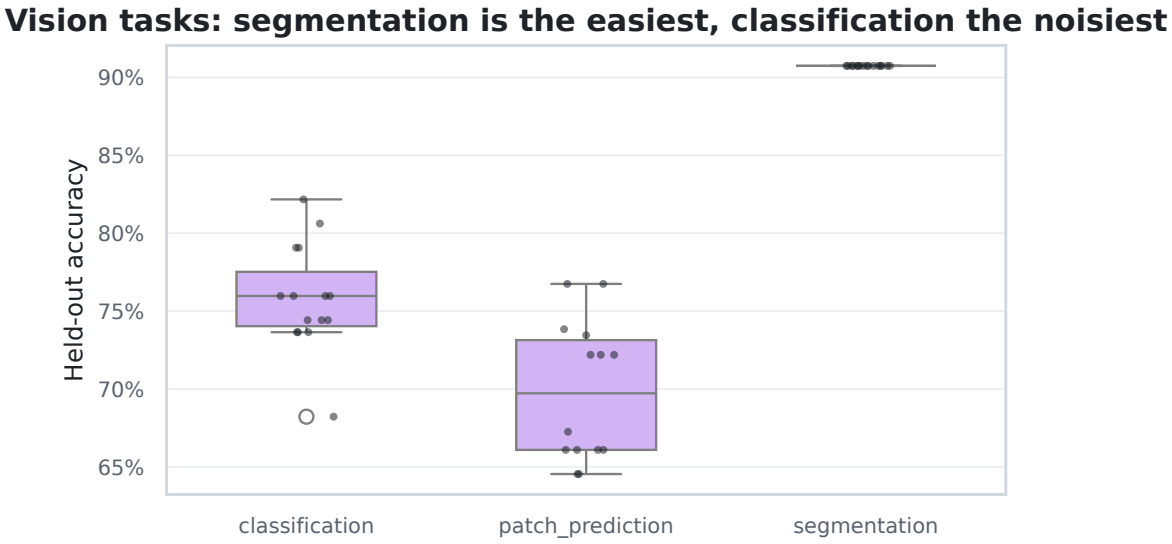


Figura 99: Tareas de visión por objetivo. La segmentación es la más fácil y la predicción de parches la más difícil.

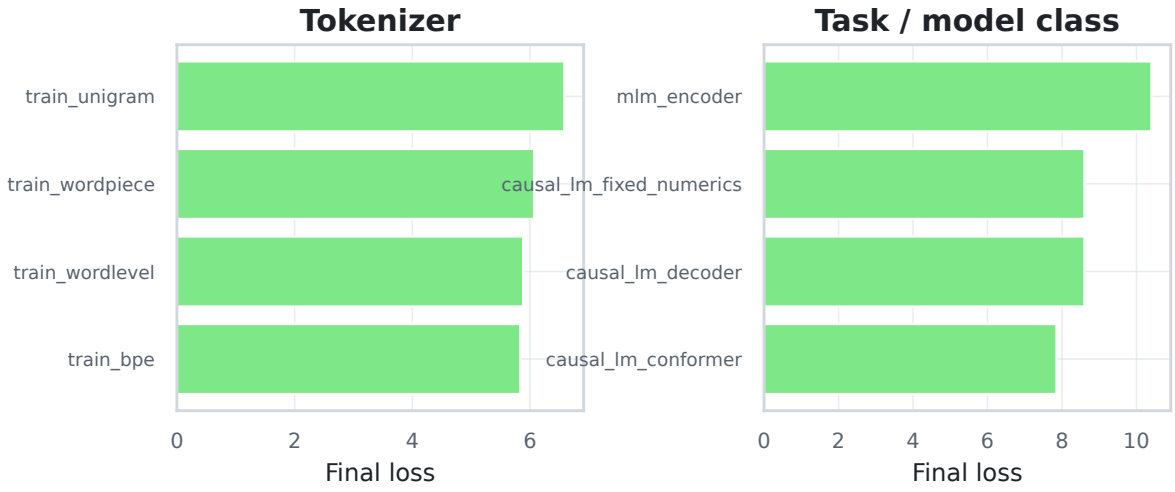


Figura 100: Ablaciones de tokenizer y tarea. Ambas cambian la escala de la pérdida, así que no son comparables con la barrida de mezcladores.

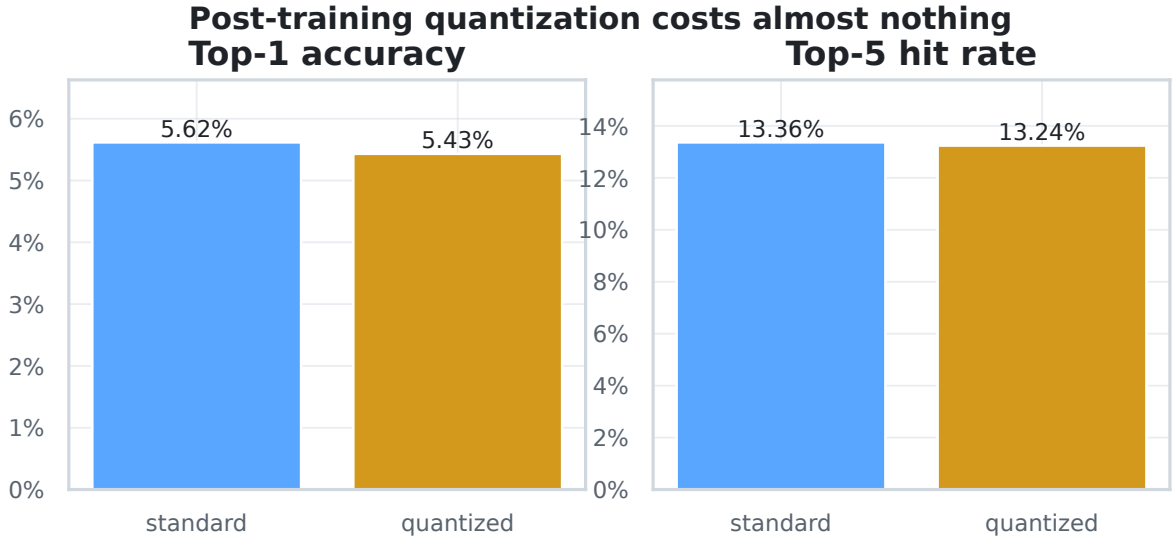


Figura 101: Inferencia estándar frente a cuantizada. Las distribuciones se solapan casi por completo.

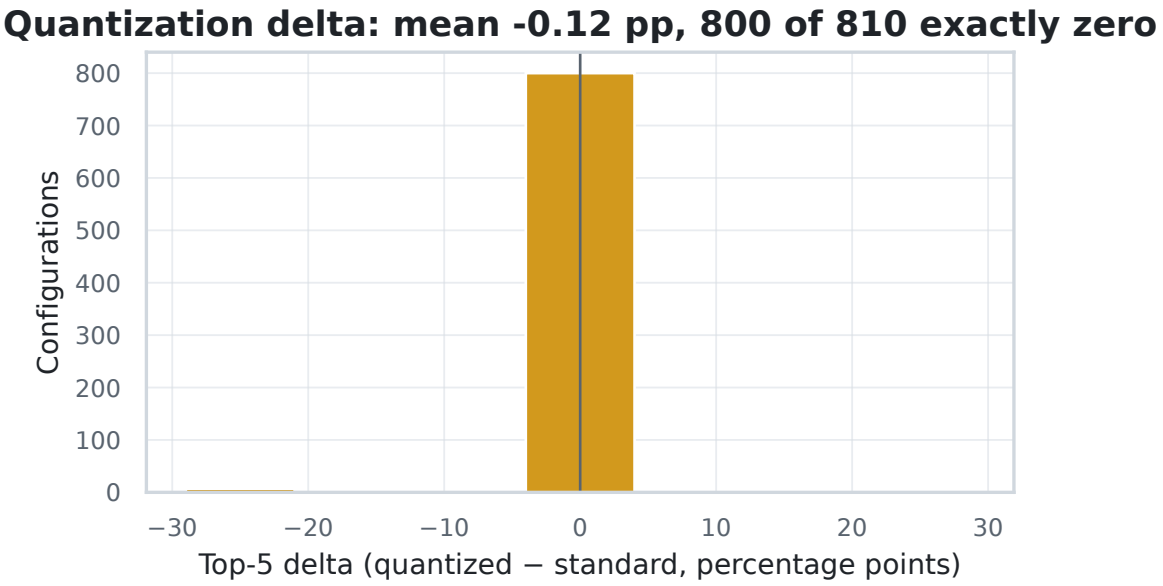


Figura 102: Distribución del delta de cuantización. Ochocientas de 810 configuraciones tienen un delta exactamente cero.

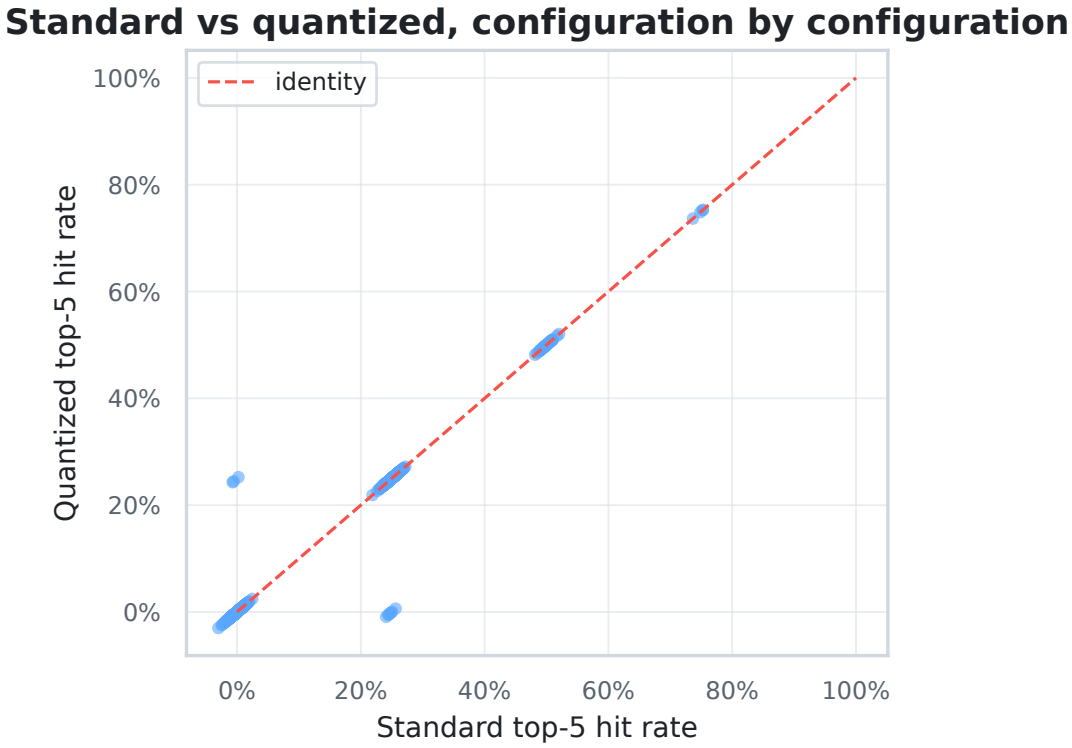


Figura 103: Top-5 estándar frente a cuantizado. Cada punto en la diagonal es una configuración que la cuantización no cambió.

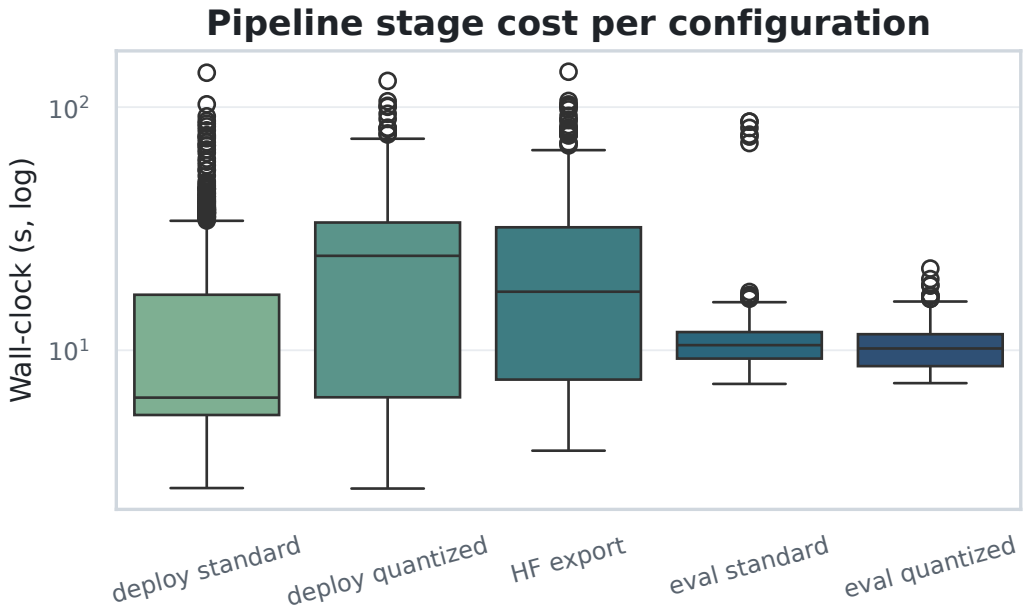


Figura 104: Coste por etapa del pipeline. El despliegue y la evaluación cuestan segundos; el entrenamiento cuesta horas.

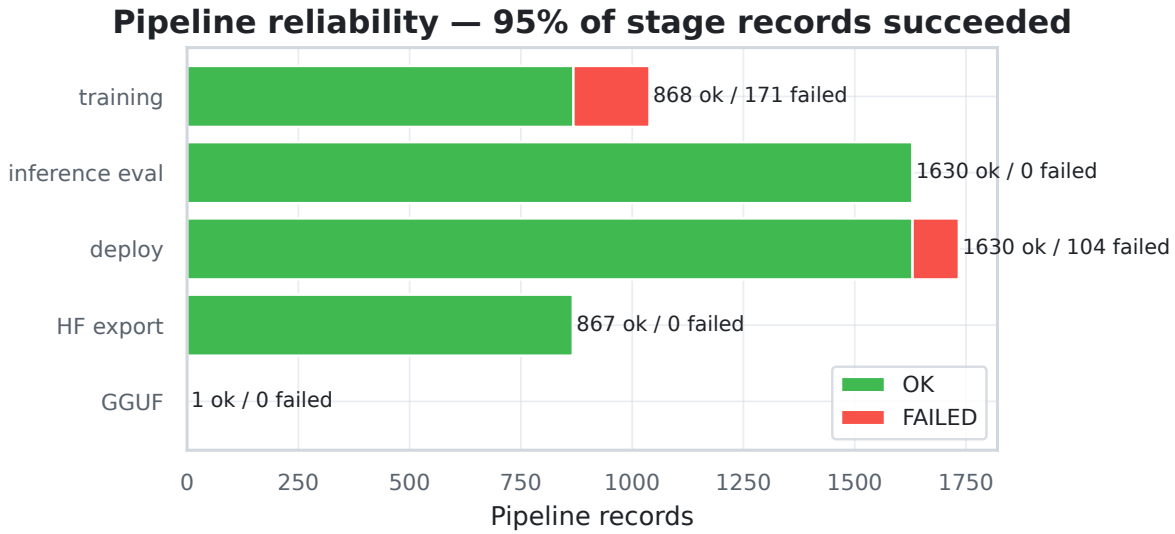


Figura 105: Fiabilidad del pipeline. El 95 % de los registros de etapa tuvo éxito; entrenamiento y despliegue explican todos los fallos.

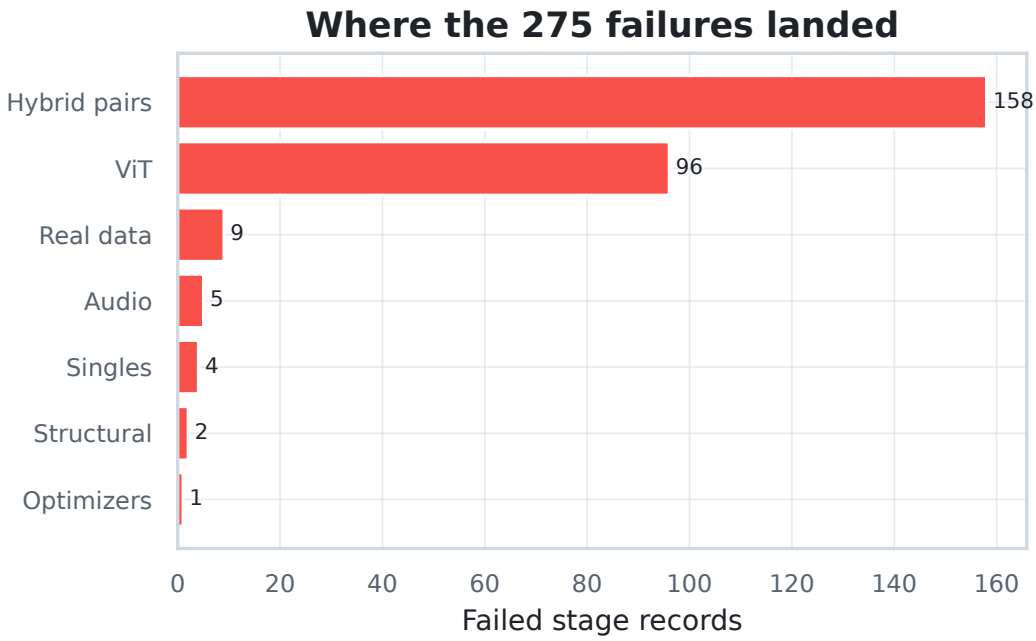


Figura 106: Dónde cayeron los 275 fallos. Los fallos de entrenamiento se concentran en las familias cuyos mezcladores nunca entrenan; los de despliegue son simétricos entre formatos.

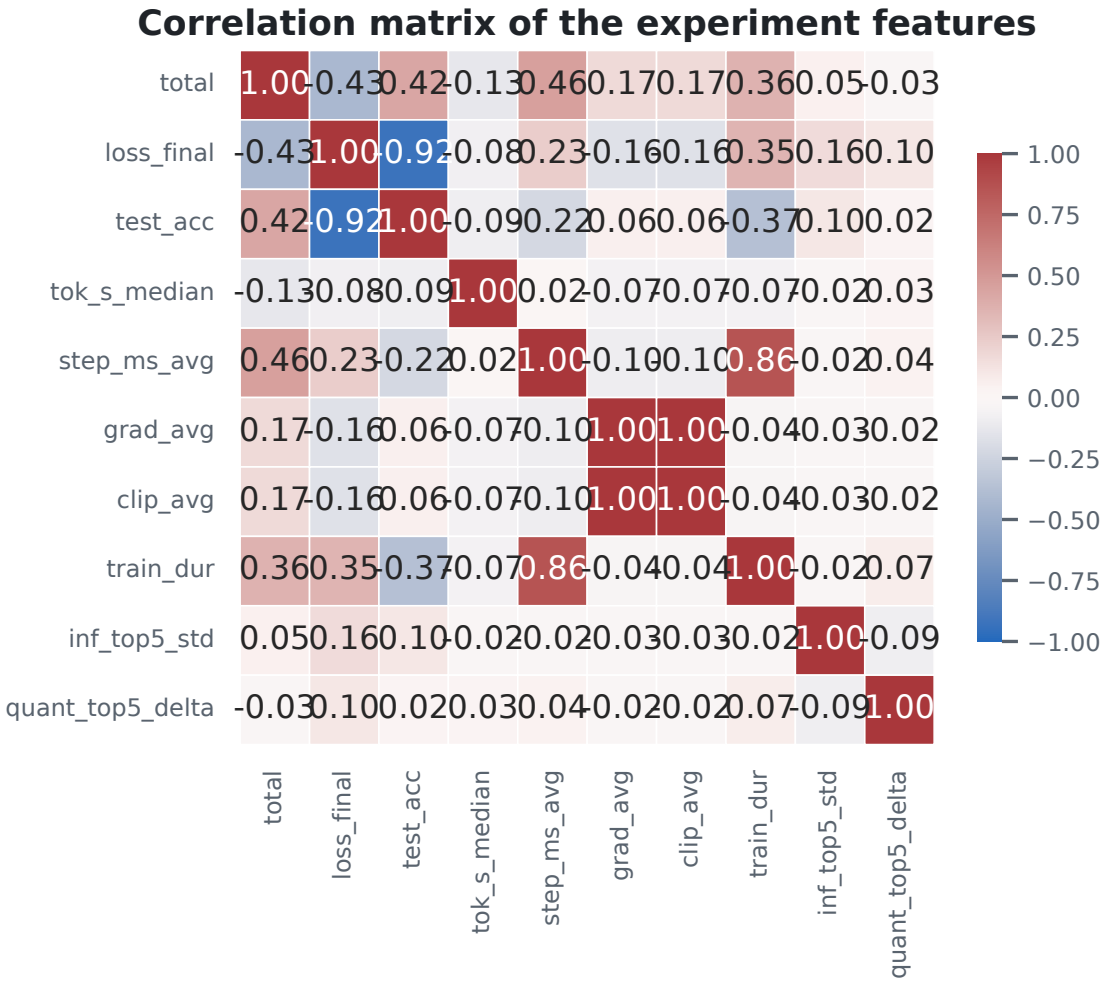


Figura 107: Matriz de correlación sobre las 868 configuraciones. Las fuertes correlaciones parámetros–calidad son una confusión de familias; dentro de los pares híbridos caen a −0,20 y −0,07.